CAPÍTULO 1

Introducción

1.1. Organización y arquitectura

1.2. Estructura y funcionamiento

Funcionamiento
Estructura

1.3. ¿Por qué estudiar la organización y arquitectura de los computadores?

Este libro trata sobre la estructura y funcionamiento de los computadores. Su objetivo es presentar, tan clara y completamente como sea posible, la naturaleza y las características de los computadores de hoy día. Este objetivo es todo un reto por dos razones.

Primero, hay una gran variedad de sistemas que pueden recibir correctamente el nombre de *computador*, desde micropocesadores de un solo chip, que cuestan unos pocos dólares, a supercomputadores que cuestan decenas de millones de dólares. Esta variedad es patente no solo en costes sino en tamaño, prestaciones y aplicaciones. Segundo, el rápido ritmo de cambio que ha caracterizado siempre a la tecnología de computadores continúa sin pausa. Estos cambios cubren todos los aspectos de la tecnología de computadores. Desde la tecnología subyacente de circuitos integrados, usados para construir componentes de computadores, hasta el creciente uso de conceptos de organización paralela para combinar esos componentes.

A pesar de la variedad y el ritmo de cambio en el campo de los computadores, se aplican sistemáticamente ciertos conceptos fundamentales. La aplicación de estos conceptos depende del desarrollo actual de la tecnología y de los objetivos en precio-aplicación del diseñador. La intención de este libro es ofrecer un concienzudo análisis de los fundamentos de la arquitectura y organización de los computadores y relacionar estos con materias de diseño actuales. Este capítulo introduce la aproximación descriptiva que se va a considerar.

1.1. ORGANIZACIÓN Y ARQUITECTURA

Cuando se describe un computador, frecuentemente se distingue entre *arquitectura* y *organización*. Aunque es difícil dar una definición precisa para estos términos, existe un consenso sobre las áreas generales cubiertas por cada uno de ellos (por ejemplo, *veáse* [VRAN80], [SIEW82], y [BELL78a]).

La arquitectura de computadores se refiere a los atributos de un sistema que son visibles a un programador, o para decirlo de otra manera, aquellos atributos que tienen un impacto directo en la ejecución lógica de un programa. La organización de computadores se refiere a las unidades funcionales y sus interconexiones, que dan lugar a especificaciones arquitectónicas. Entre los ejemplos de atributos arquitectónicos se encuentran el conjunto de instrucciones, el número de bits usados para representar varios tipos de datos (por ejemplo, números, caracteres), mecanismos de E/S y técnicas para direccionamiento de memoria. Entre los atributos de organización se incluyen aquellos detalles de hardware transparentes al programador, tales como señales de control, interfaces entre el computador y los periféricos y la tecnología de memoria usada.

Para poner un ejemplo, una cuestión de diseño arquitectónico es si el computador tendrá la instrucción de multiplicar. Una cuestión de organización es si esa instrucción será implementada por una unidad especializada en multiplicar o por un mecanismo que haga un uso iterativo de la unidad de suma del sistema. La decisión de organización puede estar basada en la frecuencia prevista del uso de la instrucción de multiplicar la velocidad relativa de las dos aproximaciones, y el coste y el tamaño físico de una unidad especializada en multiplicar.

Históricamente, y aún hoy día, la distinción entre arquitectura y organización ha sido importante. Muchos fabricantes de computadores ofrecen una familia de modelos, todos con la misma arquitectura pero con diferencias en la organización. Consecuentemente los diferentes modelos de la familia tienen precios y prestaciones distintas. Más aún, una arquitectura puede sobrevivir muchos años, pero su organización cambia con la evolución de tecnología. Un ejemplo destacado de ambos fenómenos

es la arquitectura IBM Sistema/370. Esta arquitectura apareció por primera vez en 1970 e incluía varios modelos. Un cliente con necesidades modestas podía comprar un modelo más barato y lento, y, si la demanda se incrementaba, cambiarse más tarde a un modelo más caro y rápido sin tener que abandonar el software que ya había sido desarrollado. A través de los años IBM ha introducido muchos modelos nuevos con tecnología mejorada para reemplazar a modelos más viejos, ofreciendo al consumidor mayor velocidad, precios más bajos o ambos a la vez. Estos modelos más nuevos conservaban la misma arquitectura para proteger así la inversión en software del consumidor. Podemos destacar que la arquitectura del Sistema/370 con unas pocas mejoras ha sobrevivido hasta hoy día como la arquitectura de la línea de grandes productos de computación IBM.

En una clase de sistemas, llamados microcomputadores, la relación entre arquitectura y organización es muy estrecha. Los cambios en la tecnología no solo influyen en la organización, sino que también dan lugar a la introducción de arquitecturas más ricas y potentes. Generalmente hay menos requisitos de compatibilidad generación a generación para estas pequeñas máquinas. Así, hay más interacción entre las decisiones de diseño arquitectónicas y de organización. Un ejemplo interesante de esto son los computadores de repertorio reducido de instrucciones (RISC, *Reduced Instruction Set Computer*), que veremos en el Capítulo 13.

En este libro se examina tanto la organización como la arquitectura de un computador. Se da, quizá, más énfasis a la parte de organización. Sin embargo, como la organización de un computador debe ser diseñada para implementar la especificación de una arquitectura particular, un estudio exhaustivo de la organización requiere también un análisis detallado de la arquitectura.

1.2. ESTRUCTURA Y FUNCIONAMIENTO

Un computador es un sistema complejo; los computadores de hoy en día contienen millones de componentes electrónicos básicos. ¿Cómo podríamos describirlos claramente? La clave está en reconocer la naturaleza jerárquica de la mayoría de los sistemas complejos, incluyendo el computador [SIMO69]. Un sistema jerárquico es un conjunto de subsistemas interrelacionados cada uno de los cuales, a su vez, se organiza en una estructura jerárquica hasta que se alcanza el nivel más bajo del subsistema elemental.

La naturaleza jerárquica de los sistemas complejos es esencial tanto para su diseño como para su descripción. El diseñador necesita tratar solamente con un nivel particular del sistema a la vez. En cada nivel el sistema consta de un conjunto de componentes y sus interrelaciones. El comportamiento en cada nivel depende solo de una caracterización abstracta y simplificada del sistema que hay en el siguiente nivel más bajo. De cada nivel al diseñador le importan la estructura y el funcionamiento:

- **Estructura***:* el modo en que los componentes están interrelacionados.
- **Funcionamiento***:* la operación de cada componente individual como parte de la estructura.

En términos de descripción tenemos dos opciones: empezar por lo más bajo y construir una descripción completa, o comenzar con una visión desde arriba y descomponer el sistema en sus subpartes. La experiencia a partir de muchos campos nos ha enseñado que la descripción de arriba abajo (*top-down*) es la más clara y efectiva [WEIN75].

El enfoque seguido en este libro considera este punto de vista. El computador será descrito de arriba abajo. Comenzamos con los componentes principales del sistema describiendo su estructura y

funcionamiento, y seguimos sucesivamente hacia capas más bajas de la jerarquía. Lo que queda de esta sección ofrece una breve visión global de este plan de ataque.

FUNCIONAMIENTO

Tanto la estructura como el funcionamiento de un computador son en esencia sencillos. La Figura 1.1 señala las funciones básicas que un computador puede llevar a cabo. En términos generales hay solo cuatro:

- Procesamiento de datos
- Almacenamiento de datos
- Transferencia de datos
- Control

El computador, por supuesto, tiene que ser capaz de **procesar datos**. Los datos pueden adoptar una gran variedad de formas, y el rango de los requisitos de procesado es amplio. Sin embargo, veremos que hay solo unos pocos métodos o tipos fundamentales de procesado de datos.

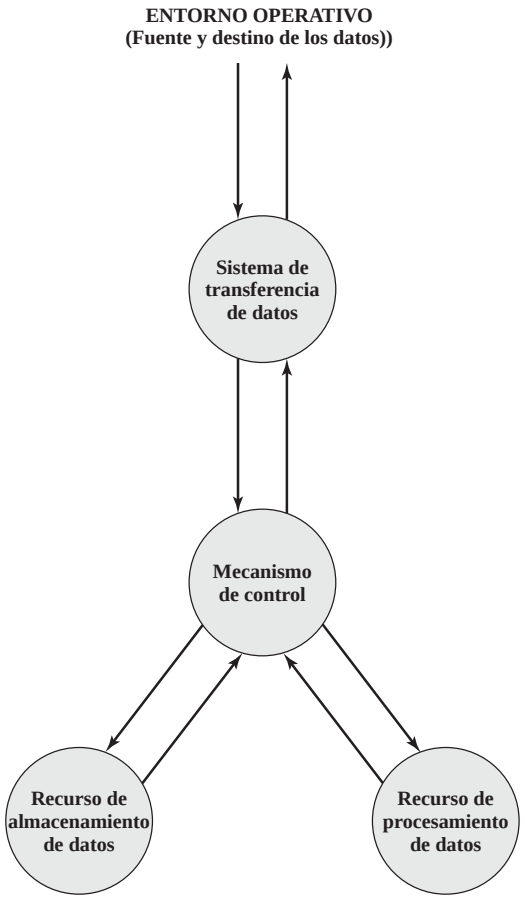


Figura 1.1. Una visión funcional de un computador.

También es esencial que un computador **almacene datos**. Incluso si el computador está procesando datos al vuelo (es decir, los datos se introducen, se procesan, y los resultados se obtienen inmediatamente), el computador tiene que guardar temporalmente al menos aquellos datos con los que está trabajando en un momento dado. Así hay al menos una función de almacenamiento de datos a corto plazo. Con igual importancia el computador lleva a cabo una fución de almacenamiento de datos a largo plazo. El computador almacena ficheros de datos para que se recuperen y actualicen en un futuro.

El computador tiene que ser capaz de **transferir datos** entre él mismo y el mundo exterior. El entorno de operación del computador se compone de dispositivos que sirven bien como fuente o bien como destino de datos. Cuando se reciben o se llevan datos a un dispositivo que está directamente conectado con el computador, el proceso se conoce como *entrada-salida* (E/S), y este dispositivo recibe el nombre de *periférico*. El proceso de transferir datos a largas distancias, desde o hacia un dispositivo remoto, recibe el nombre de *comunicación de datos*.

Finalmente, debe haber un **control** de estas tres funciones. Este control es ejercido por el(los) ente(s) que proporciona(n) al computador instrucciones. Dentro del computador, una unidad de control gestiona los recursos del computador y dirige las prestaciones de sus partes funcionales en respuesta a estas instrucciones.

A este nivel general de discusión, el número de operaciones posibles que pueden ser realizadas es pequeño. La Figura 1.2 muestra los cuatro posibles tipos de operaciones. El computador puede funcionar como un dispositivo de transferencia de datos (Figura 1.2a), simplemente transfiriendo datos de un periférico o línea de comunicaciones a otro. También puede funcionar como un dispositivo de almacenamiento de datos (Figura 1.2b), con datos transferidos desde un entorno externo al almacén de datos del computador (leer) y viceversa (escribir). Los dos diagramas siguientes muestran operaciones que implican procesamiento de datos, en datos, o bien almacenados (Figura 1.2c) o en tránsito entre el almacén y el entorno externo (Figura 1.2d).

La exposición precedente puede parecer absurdamente generalizada. Es posible, incluso en el nivel más alto de la estructura de un computador, diferenciar varias funciones, pero citando [SIEW82]:

> «Hay, sorprendentemente, muy pocas formas de estructuras de computadores que se ajusten a la función que va a ser llevada a cabo. En la raíz de esto subyace el problema de la naturaleza de uso general de los computadores, en la cual toda la especialización funcional se tiene cuando se programa y no cuando se diseña».

ESTRUCTURA

La Figura 1.3 es la representación más sencilla posible de un computador. El computador es una entidad que interactúa de alguna manera con su entorno externo. En general, todas sus conexiones con el entorno externo pueden ser clasificadas como dispositivos periféricos o líneas de comunicación. Diremos algo más adelante sobre ambos tipos de conexiones.

Pero tiene más interés en este libro la estructura interna del computador mismo, que mostramos, en su nivel más alto, en la Figura 1.4. Hay cuatro componentes estructurales principales:

- **Unidad Central de Procesamiento (CPU, *Central Processing Unit*)**: controla el funcionamiento del computador y lleva a cabo sus funciones de procesamiento de datos. Frecuentemente se le llama simplemente **procesador**.

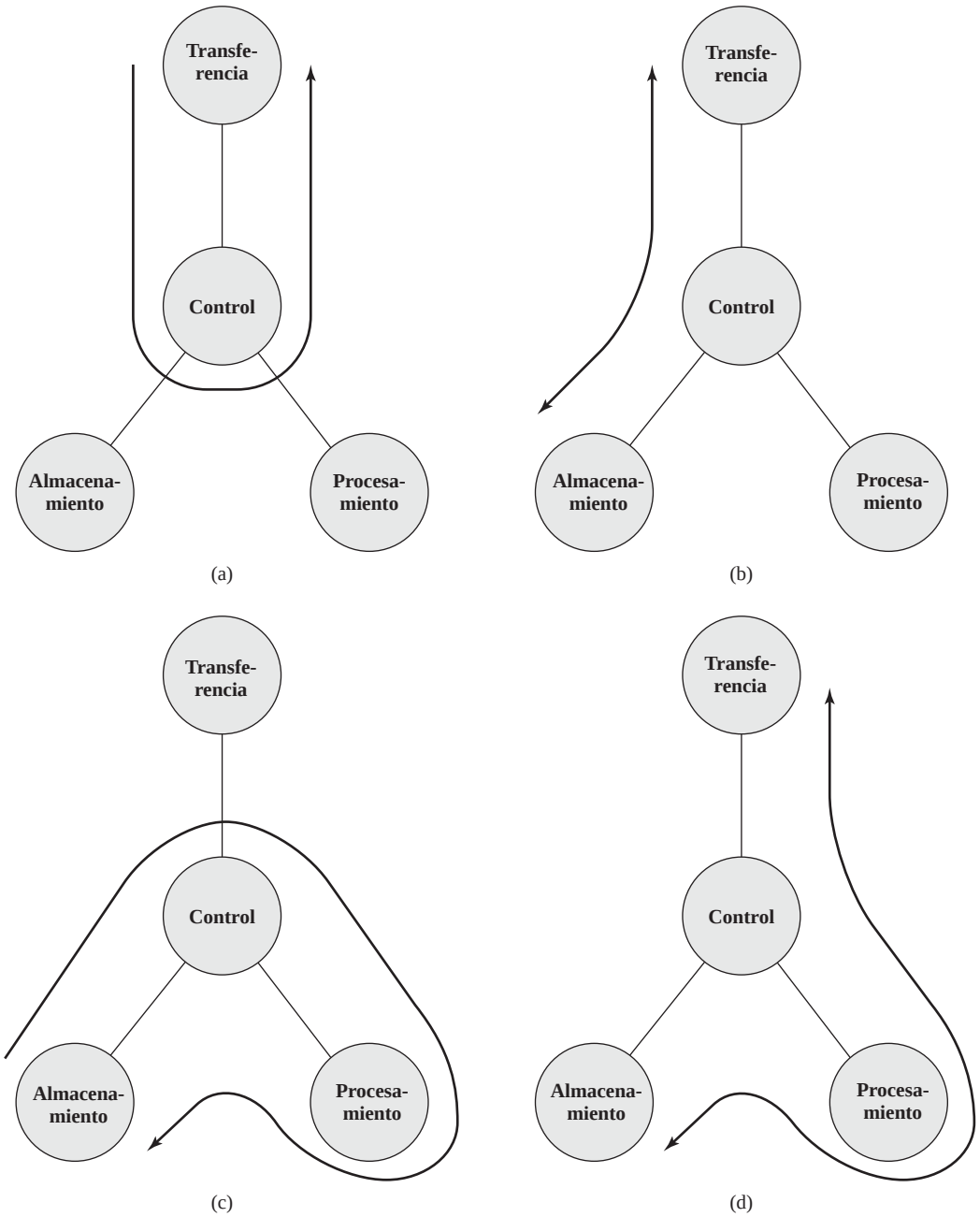


Figura 1.2. Posibles operaciones de un computador.

- **Memoria principal:** almacena datos.
- **E/S:** transfiere datos entre el computador y el entorno externo.
- **Sistema de interconexión:** es un mecanismo que proporciona la comunicación entre la CPU, la memoria principal y la E/S.

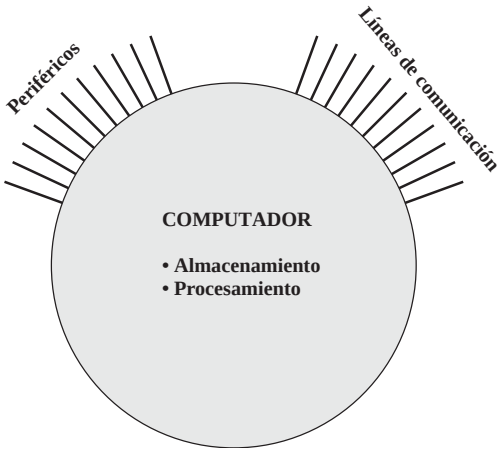


Figura 1.3. El computador.

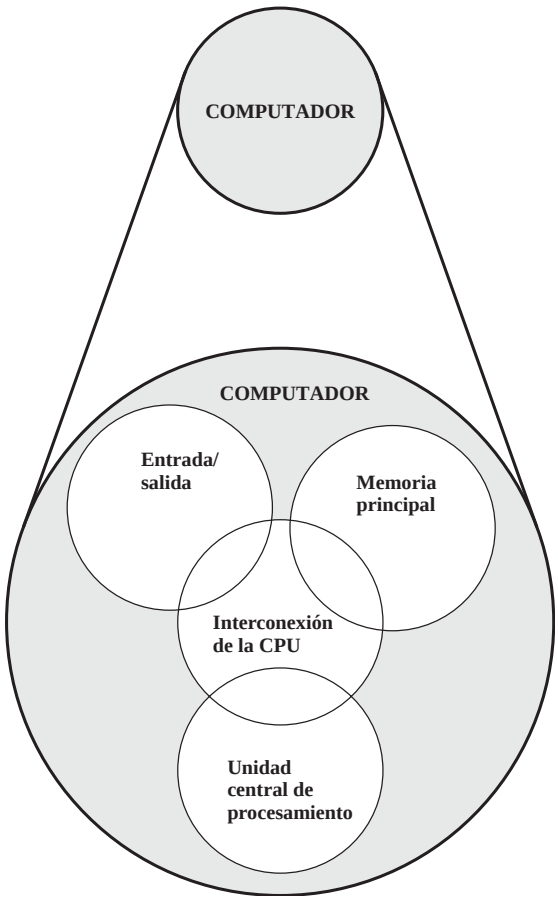


Figura 1.4. El computador: estructura del nivel superior.

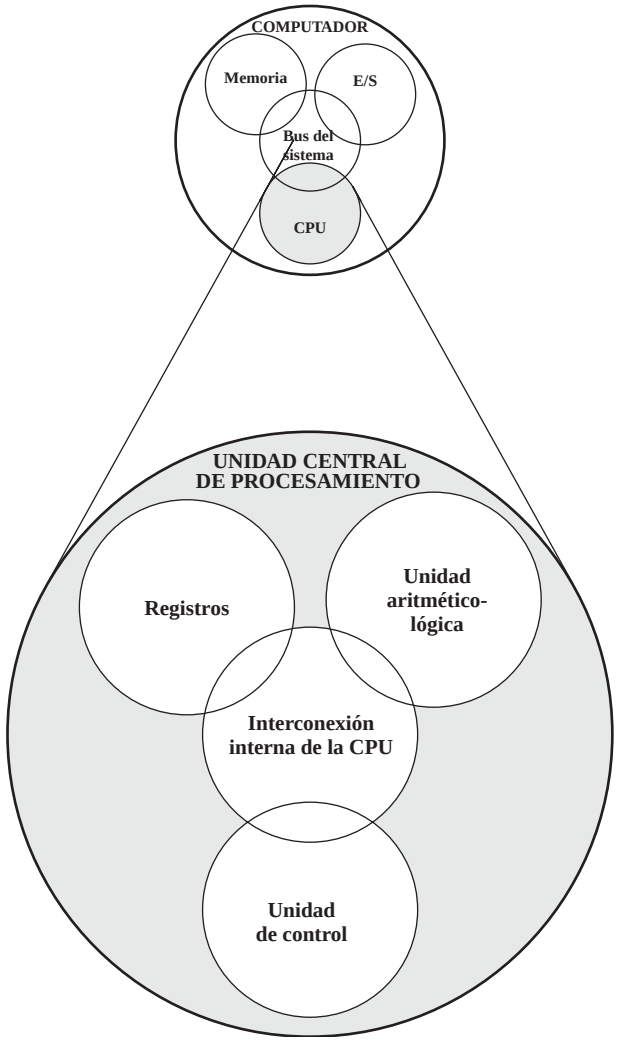


Figura 1.5. La unidad central de procesamiento (CPU).

Puede que haya uno o más de cada uno de estos componentes. Tradicionalmente ha habido solo una CPU. En los últimos años ha habido un uso creciente de varios procesadores en un solo sistema. Surgen algunas cuestiones relativas a multiprocesadores y se discuten conforme el texto avanza; la Parte Cinco se centra en tales sistemas.

Cada uno de estos componentes será examinado con cierto detalle en la Parte Dos. Sin embargo, para nuestros objetivos, el componente más interesante y de algún modo el más complejo es la CPU; su estructura se muestra en la Figura 1.5. Sus principales componentes estructurales son:

- **Unidad de control:** controla el funcionamiento de la CPU y por tanto del computador.
- **Unidad aritmético-lógica (ALU, *Arithmetic Logic Unit*):** lleva a cabo las funciones de procesamiento de datos del computador.

- **Registros:** proporcionan almacenamiento interno a la CPU.
- **Interconexiones CPU:** son mecanismos que proporcionan comunicación entre la unidad de control, la ALU y los registros.

Cada uno de estos componentes será analizado con detalle en la Parte Tres, donde veremos que la complejidad aumenta con el uso de técnicas de organización paralelas y de segmentación de cauce. Finalmente, hay varias aproximaciones para la implementación de la unidad de control; una de las aproximaciones más comunes es la implementación *microprogramada*. Básicamente, una unidad de control microprogramada actúa ejecutando microinstrucciones que definen la funcionalidad de la unidad de control. Con esta aproximación, la estructura de la unidad de control puede ser como la mostrada en la Figura 1.6. Esta estructura será examinada en la Parte Cuatro.

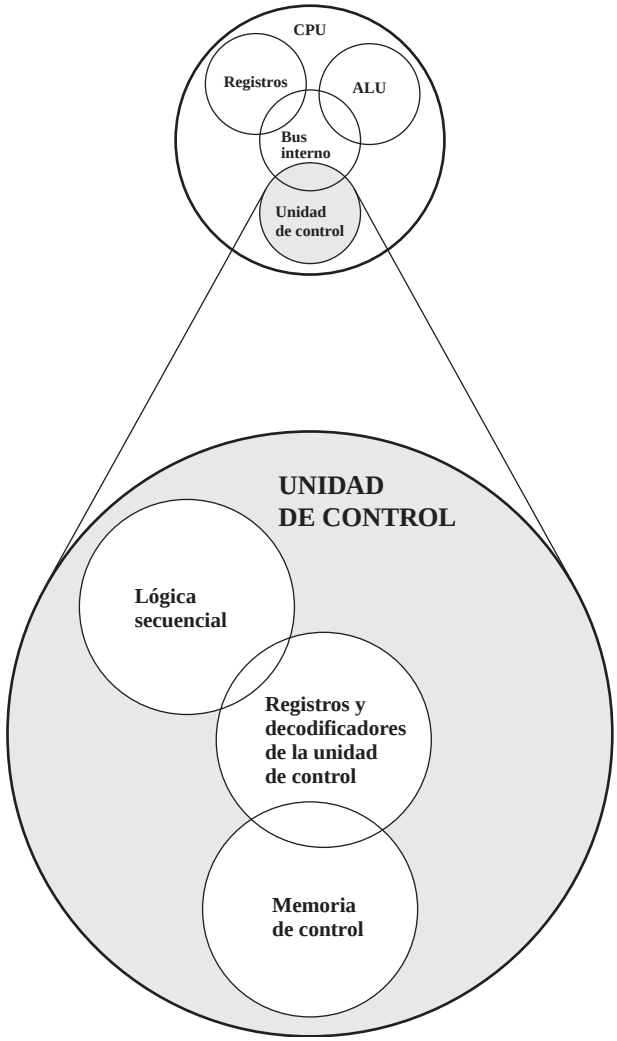


Figura 1.6. La unidad de control.

1.3. ¿POR QUÉ ESTUDIAR LA ORGANIZACIÓN Y ARQUITECTURA DE LOS COMPUTADORES?

El «IEEE/ACM Computer Curricula 2001» [JTF01], preparado por la *Joint Task Force* de currículo de computadores de la Sociedad de Computadores IEEE (*Institute of Electrical and Electronics Engineers*) y la ACM (*Association for Computing Machinery*), citan la arquitectura de computadores como uno de los temas troncales que debe estar en todos los currículos de todos los estudiantes de licenciatura e ingeniería informática. El informe dice lo siguiente:

> «El computador está en el corazón de la informática. Sin él la mayoría de las asignaturas de informática serían hoy una rama de la matemática teórica. Para ser hoy un profesional en cualquier campo de la informática uno no debe ver al computador como una caja negra que ejecuta programas mágicamente. Todos los estudiantes de informática deben, en cierta medida, comprender y valorar los componentes funcionales de un computador, sus características, su funcionamiento y sus interacciones. También sus implicaciones prácticas. Los estudiantes necesitan comprender la arquitectura del computador para estructurar un programa de forma que este sea más eficiente en una máquina real. Seleccionando el sistema que se va a usar, debe ser capaz de comprender el compromiso entre varios componentes, como la velocidad del reloj de la CPU frente al tamaño de la memoria».

En [CLEM00] se dan los siguientes ejemplos como razones para estudiar arquitectura de computadores:

1. Supóngase que un licenciado trabaja en la industria y se le pide seleccionar el computador con la mejor relación calidad precio para utilizarlo en una gran empresa. Comprender las implicaciones de gastar más en distintas alternativas, como una caché grande o una velocidad de reloj mayor, es esencial para tomar esta decisión.
2. Hay muchos procesadores que no forman pare de equipos PC o servidores, pero sí en sistemas embebidos. Un diseñador debe ser capaz de programar un procesador en C que esté embebido en algún sistema en tiempo real o sistema complejo, como un controlador electrónico de un coche inteligente. Depurar el sistema puede requerir utilizar un analizador lógico que muestre la relación entre las peticiones de interrupción de los sensores del sistema y el código máquina.
3. Los conceptos utilizados en arquitectura de computadores tienen aplicación en otros cursos. En particular, la forma en la que el computador ofrece un soporte arquitectural a los lenguajes de programación y funciones en principio propias del sistema operativo, refuerza los conceptos de estas áreas.

Como se puede deducir del índice de este libro, la organización y arquitectura de computadores abarca un amplio rango de temas y conceptos. Una buena comprensión de estos conceptos será útil tanto en otras áreas de estudio como en un futuro trabajo después de licenciarse.

CAPÍTULO 2

Evolución y prestaciones de los computadores

2.1. Una breve historia de los computadores

La primera generación: los tubos de vacío
La segunda generación: los transistores
La tercera generación: los circuitos integrados
Últimas generaciones

2.2. Diseño buscando mejores prestaciones

Velocidad del microprocesador
Equilibrio de prestaciones
Mejoras en la organización y arquitectura de chips

2.3. Evolución del Pentium y del PowerPC

Pentium
PowerPC

2.4. Lecturas y sitios web recomendados

Palabras clave

2.5. Palabras clave, peguntas de repaso y problemas

Palabras clave
Preguntas de repaso
Problemas

PUNTOS CLAVE

- La evolución de los computadores se ha caracterizado por un incremento de la velocidad del procesador, una disminución del tamaño de los componentes, un aumento del tamaño de memoria, y un aumento de la capacidad de E/S y de la velocidad.
- Otro factor responsable del gran aumento de la velocidad del procesador es la disminución del tamaño de los componentes del microprocesador; esto reduce la distancia entre componentes, y, por tanto, aumenta la velocidad. Sin embargo, la verdadera ganancia en velocidad en los últimos años se debe a la organización del procesador, incluyendo un uso amplio de la segmentación de cauce, de las técnicas de ejecución paralela y del uso de técnicas de ejecución especulativa, que conducen a la ejecución tentativa de instrucciones futuras que se puedan necesitar. Todas estas técnicas se diseñan para mantener al procesador ocupado el mayor tiempo posible.
- Otro asunto crítico en el diseño de computadores es hacer un balance de las prestaciones de los distintos elementos, de forma que esta ganancia en prestaciones en un área no perjudique a otras áreas. En particular, la velocidad del procesador ha aumentado más rápidamente que el tiempo de acceso a memoria. Se han usado distintas técnicas para compensar este desacoplo, incluyendo memorias caché, caminos de datos más anchos de la memoria al procesador y más circuitos de memoria inteligentes.

Empezamos nuestro estudio de los computadores con una breve historia. Esta historia es interesante por sí misma y además sirve para proporcionar una visión general de la estructura y funcionamiento de los computadores. Luego se trata el tema de las prestaciones. La consideración de la necesidad de equilibrar los recursos de un computador nos da un contexto útil en todo el libro. Finalmente, veremos brevemente la evolución de dos sistemas que sirven como ejemplos clave en todo el libro: Pentium y PowerPC.

2.1. UNA BREVE HISTORIA DE LOS COMPUTADORES

LA PRIMER GENERACIÓN: LOS TUBOS DE VACÍO

ENIAC El ENIAC (*Electronic Numerical Integrator And Computer*), diseñado y construido bajo la supervisión de John Mauchly y John Presper Eckert en la Universidad de Pennsylvania, fue el primer computador electrónico de propósito general del mundo.

El proyecto fue una respuesta a necesidades militares durante la Segunda Guerra Mundial. El BRL (*Ballistcs Research Laboratory*, Laboratorio de Investigación de Balística) del Ejército, una agencia responsable del desarrollo de tablas de tiro y de trayectoria para nuevas armas, tenía dificultades para elaborarlas con exactitud y dentro de un plazo de tiempo razonable. Sin estas tablas de tiro, las nuevas armas y piezas de artillería eran inútiles para los artilleros. El BRL empleó a más de doscientas personas, la mayoría mujeres, que utilizando calculadoras de mesa resolvían las ecuaciones

balísticas necesarias. La preparación de las tablas para una sola arma le habría llevado a una persona muchas horas, incluso días.

Mauchly, un catedrático de Ingeniería Eléctrica de la Universidad de Pennsylvania, y Eckert, uno de sus alumnos de licenciatura, propusieron construir un computador de propósito general usando tubos de vacío para utilizarlo en las aplicaciones de la BRL. En 1943 esta proposición fue aceptada por el ejército y se comenzó a trabajar en el ENIAC. La máquina que construyeron era enorme, pesaba treinta toneladas, ocupaba 15 000 pies cuadrados y contenía más de 18 000 tubos de vacío. Cuando funcionaba consumía 140 Kilowatios de potencia. También era bastante más rápida que cualquier computador electromecánico, ya que era capaz de efectuar 5 000 sumas por segundo.

El ENIAC era una máquina decimal y no binaria. Es decir, los números estaban representados en forma decimal y la aritmética se realizaba también en el sistema decimal. Su memoria consistía en veinte «acumuladores», cada uno capaz de contener un número decimal de diez dígitos. Cada dígito estaba representado por un anillo de diez tubos de vacío. En un momento dado, solo uno de los tubos de vacío estaba en estado ON, representando uno de los diez dígitos. Uno de los mayores inconvenientes del ENIAC era que tenía que ser programado manualmente mediante conmutadores y conectando y desconectando cables.

El ENIAC se terminó en 1946, demasiado tarde para ser utilizado durante la guerra. En su lugar, su primera misión fue realizar una serie de cálculos complejos que se usaron para ayudar a determinar la viabilidad de la bomba de hidrógeno. El uso del ENIAC para una función distinta de aquella para la que fue construido demostró su naturaleza de propósito general. Así, 1946 marcó el comienzo de la nueva era de los computadores electrónicos, culminando años de esfuerzo. El ENIAC siguió funcionando bajo la dirección del BRL hasta 1955, cuando fue desmontado.

La máquina de von Neumann La tarea de cargar y modificar programas para el ENIAC era extremadamente tediosa. El proceso de programación podría ser más fácil si el programa se representara en una forma adecuada para ser guardado en la memoria junto con los datos. Entonces, un computador podría conseguir sus instrucciones leyéndolas de la memoria, y se podría hacer o modificar un programa colocando los valores en una zona de memoria.

Esta idea conocida como *concepto del programa-almacenado*, se atribuye a los diseñadores del ENIAC, sobre todo al matemático John von Neumann, que era asesor del proyecto ENIAC. La idea fue también desarrollada aproximadamente al mismo tiempo por Turing. La primera publicación de la idea fue en una propuesta de von Neumann para un nuevo computador en 1945, el EDVAC (*Electronic Discrete Variable Computer*).

En 1946 von Neumann y sus colegas empezaron, en el Instituto para Estudios Avanzados de Princeton, el diseño de un nuevo computador de programa-almacenado, que llamaron IAS. El computador IAS, no completado hasta 1952, es el prototipo de toda una serie de computadores de propósito general.

La Figura 2.1 muestra la estructura general del computador IAS. Esta consta de:

- Una memoria principal que almacena tanto datos como instrucciones [1].

[1] En este libro, a menos que se diga lo contrario, el término instrucción hace referencia a una instrucción máquina que es directamente interpretada y ejecutada por el procesador, a diferencia de una instrucción de un lenguaje de alto nivel tal como Ada o Pascal, que previamente a ser ejecutada tiene que ser compilada en una serie de instrucciones máquina.

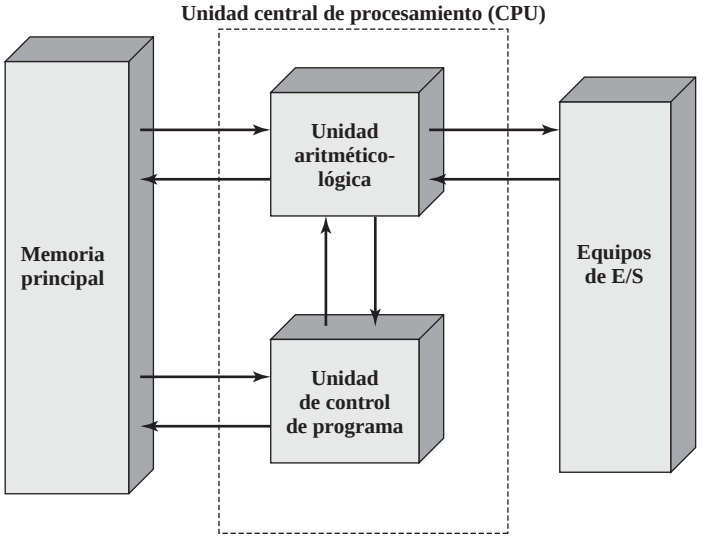


Figura 2.1. Estructura del computador IAS.

- Una unidad aritmético-lógica (ALU) capaz de hacer operaciones con datos binarios.
- Una unidad de control que interpreta las instrucciones en memoria y provoca su ejecución.
- Un equipo de entrada salida (E/S) dirigido por la unidad de control.

Esta estructura fue esbozada en la primera proposición de von Neumann, que merece la pena mencionar en este momento [VONM45]:

> 2.2. **Primero:** como el dispositivo es principalmente un computador, tendrá que realizar las operaciones aritméticas elementales muy frecuentemente. Estas son la suma, la resta, la multiplicación y la división: $+, -, \times, \div$. Es por tanto razonable que contenga elementos especializados solo en estas operaciones.
>
> Debe observarse, sin embargo, que aunque este principio parece consistente, la manera específica de cómo se aplica requiere un examen cuidadoso... En cualquier caso, tendrá que existir la parte de *aritmética central* que constituirá la *primera parte específica:* CA (*Central Arithmetical*).
>
> 2.3. **Segundo**: el control lógico del dispositivo, es decir, la secuenciación adecuada de las operaciones, debe ser realizado eficientemente por un órgano de control central. Si el dispositivo tiene que ser *versátil*, es decir, lo más cercano posible a servir para *todo uso*, entonces hay que hacer una distinción entre las instrucciones específicas que se dan y definir un problema particular, y los órganos de control general que se ocupan de que se lleven a cabo estas instrucciones —sean cuales sean—. Las primeras deben almacenarse en algún lugar; las otras deben representarse definiendo partes operativas del dispositivo. Con el *control central* nos referimos solo a esta última función, y los órganos que la realizan forman la *segunda parte específica:* CC (*Central Control*).
>
> 2.4. **Tercero**: cualquier dispositivo que realice secuencias largas y complicadas de operaciones (concretamente de cálculo) debe tener una memoria considerable [...].
>
> (b) Las instrucciones que gobiernan un problema complicado pueden constituir un material considerable, sobre todo si el código es circunstancial (lo cual ocurre en la mayoría de las situaciones).

Este material debe tenerse en cuenta [...].

En cualquier caso, la *memoria* total es *la tercera parte específica del dispositivo: M* (Memoria).

2.6. Las tres partes específicas CA, CC (juntas C) y M corresponden a las neuronas *asociativas* del sistema nervioso humano. Queda por discutir los equivalentes a las neuronas *sensiorales* o *aferentes* y las *motoras* o *eferentes*. Estos son los órganos del dispositivo de *entrada y salida* [...].

El dispositivo tiene que estar dotado con la habilidad de mantener contacto de entrada y salida (sensorial y motor) con medios específicos de este tipo (cf.1.2): el medio será llamado el *medio de grabación exterior del dispositivo: R* (*Recording*) [...].

2.7. **Cuarto**: el dispositivo tiene que tener órganos para transferir [...] información a partir de R a sus partes específicas C y M. Estos órganos forman su *entrada*, la *cuarta parte específica: I* (*Input*). Veremos que lo mejor es hacer todas las transferencias a partir de R (mediante I) hasta M y nunca directamente a partir de C [...].

2.8. **Quinto**: El dispositivo tiene que tener órganos para transferir [...] información a partir de sus partes específicas C y M hacia R. Estos órganos forman su *salida, la quinta parte específica: O* (*Output*). Veremos que es mejor, de nuevo, hacer todas las transferencias a partir de M (mediante O) a R, y nunca directamente a partir de C [...].

Salvo raras excepciones, todos los computadores de hoy en día tienen la misma estructura general y funcionamiento que la indicada en las máquinas de von Neumann. Por tanto, merece la pena en este momento describir brevemente la manera de operar del computador IAS [BURK46]. Siguiendo [HAYE88], la terminología y la notación de von Neumann han cambiado para ajustarse más a las necesidades actuales; los ejemplos e ilustraciones que acompañan a esta exposición están basados en el último texto.

La memoria del IAS consiste en 1 000 posiciones de almacenamiento, llamadas *palabras*, de cuarenta dígitos binarios (bits) cada una [2]. Tanto los datos como las instrucciones se almacenan ahí. Por tanto, los números se pueden representar en forma binaria y cada instrucción tiene también un código binario. La Figura 2.2 muestra estos formatos. Cada número se representa con un bit de signo y 39 bits de valor. Una palabra puede contener también dos instrucciones de veinte bits, donde cada instrucción consiste en un código de operación de ocho bits (codop) que especifica la operación que se va a realizar y una dirección de doce bits que indica una de las palabras de la memoria (numeradas de 0 a 999).

La unidad de control dirige el IAS captando instrucciones de la memoria y ejecutando una a una. Para explicar esto, se necesita un diagrama de estructura más detallado, como se indica en la Figura 2.3. Esta figura muestra que tanto la unidad de control como la ALU contienen posiciones de almacenamiento, llamadas *registros*, definidos de la siguiente manera:

- **Registro Temporal de Memoria (MBR, *Memory Buffer Register*):** contiene una palabra que debe ser almacenada en la memoria, o es usado para recibir una palabra procedente de la memoria.

[2] No hay una definición universal del término *palabra*. En general, una palabra es un conjunto ordenado de bytes o bits que representa la unidad básica de almacenamiento de información que se puede almacenar, transmitir o con la que se puede operar en un determinado computador. Normalmente, si un procesador tiene un conjunto de instrucciones de longitud fija, entonces la longitud de las instrucciones es igual a la longitud de palabra.

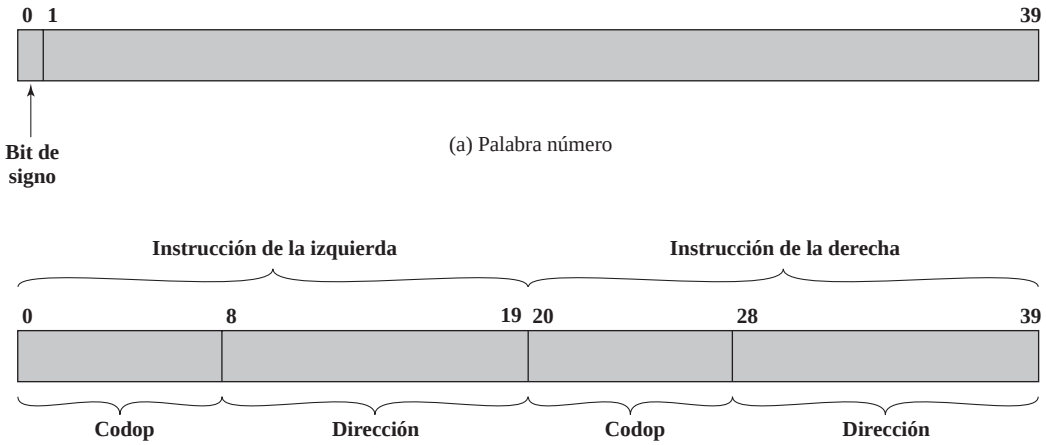


Figura 2.2. Formatos de memoria IAS.

- **Registro de Dirección de Memoria (MAR, *Memory Address Register*):** especifica la dirección en memoria de la palabra que va a ser escrita o leída en MBR.
- **Registro de Instrucción (IR, *Instruction Register*):** contiene los ocho bits del código de operación de la instrucción que se va a ejecutar.
- **Registro Temporal de Instrucción (IBR, *Instruction Buffer Register*):** empleado para almacenar temporalmente la instrucción contenida en la parte derecha de una palabra en memoria.
- **Contador de Programa (PC, *Program Counter*):** contiene la dirección de la próxima pareja de instrucciones que van a ser captadas de la memoria.
- **Acumulador (AC) y Multiplicador cociente (MQ, *Multiplier Quotient*):** Se emplean para almacenar operandos y resultados de operaciones de la ALU temporalmente. Por ejemplo, el resultado de multiplicar dos números de cuarenta bits es un número de ochenta bits; los cuarenta bits más significativos se almacenan en el AC y los menos significativos en el MQ.

El IAS opera ejecutando repetidamente un *ciclo instrucción*, como se puede ver en la Figura 2.4. Cada ciclo instrucción consta de dos subciclos. Durante el *ciclo de captación*, el codop de la siguiente instrucción es cargado en el IR y la parte que contiene la dirección es almacenada en el MAR. Esta instrucción puede ser captada desde el IBR, o puede ser obtenida de la memoria cargando una palabra en el MBR, y luego en IBR, IR y MAR.

¿Por qué la indirección? Todas estas operaciones están controladas por circuitos electrónicos, y dan lugar al uso de caminos de datos. Para simplificar la electrónica, se usa un solo registro para especificar la dirección en memoria para lectura o escritura, y un solo registro para la fuente o el destino.

Una vez que el codop está en el IR, se lleva a cabo el *ciclo de ejecución*. Los circuitos de control interpretan el codop y ejecutan la instrucción enviando las señales de control adecuadas para provocar que los datos se transfieran o que la ALU realice una operación.

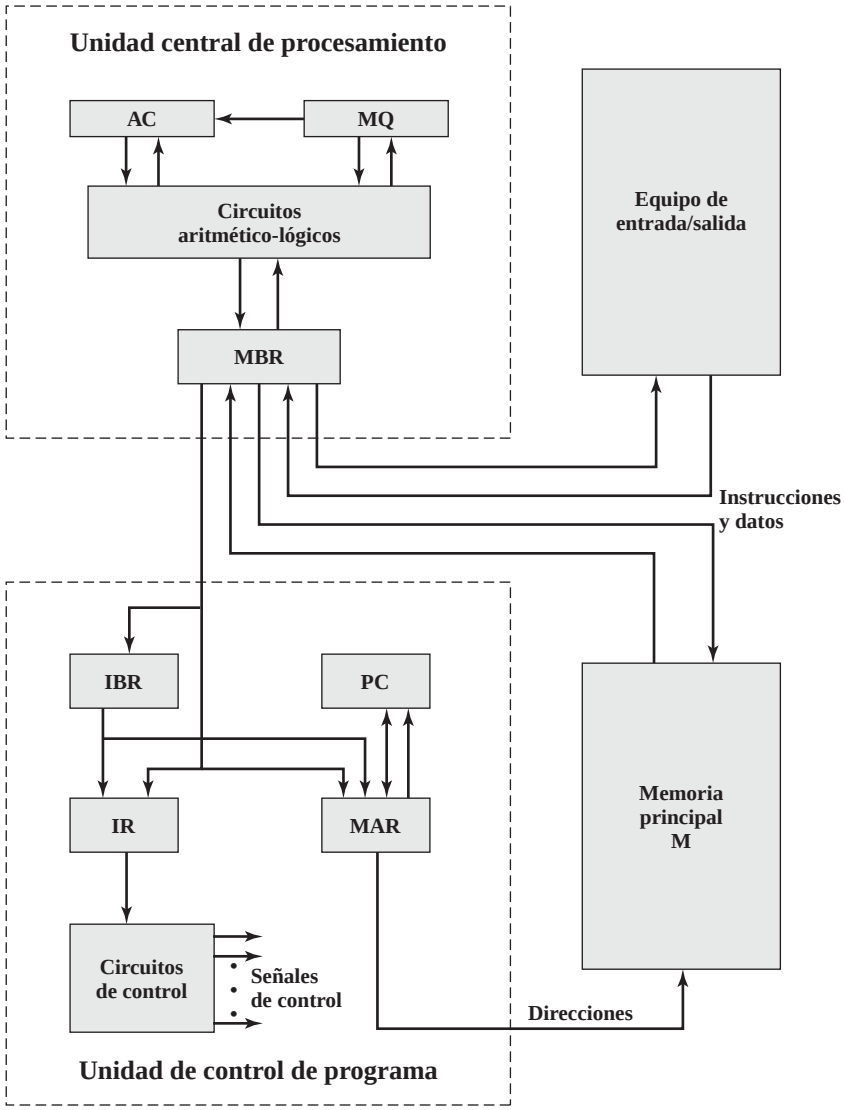


Figura 2.3. Estructura ampliada del computador IAS.

El computador IAS tiene un total de 21 instrucciones, que se indican en la Tabla 2.1. Estas se pueden agrupar de la siguiente manera:

- **Transferencia de datos:** transferir datos entre la memoria y los registros de la ALU o entre dos registros de la ALU.
- **Salto incondicional:** normalmente la unidad de control ejecuta instrucciones secuencialmente en la memoria. Las instrucciones de salto pueden cambiar esta secuencialidad. Esto facilita las operaciones repetitivas.
- **Salto condicional:** el salto depende de una condición, lo que permite puntos de decisión.

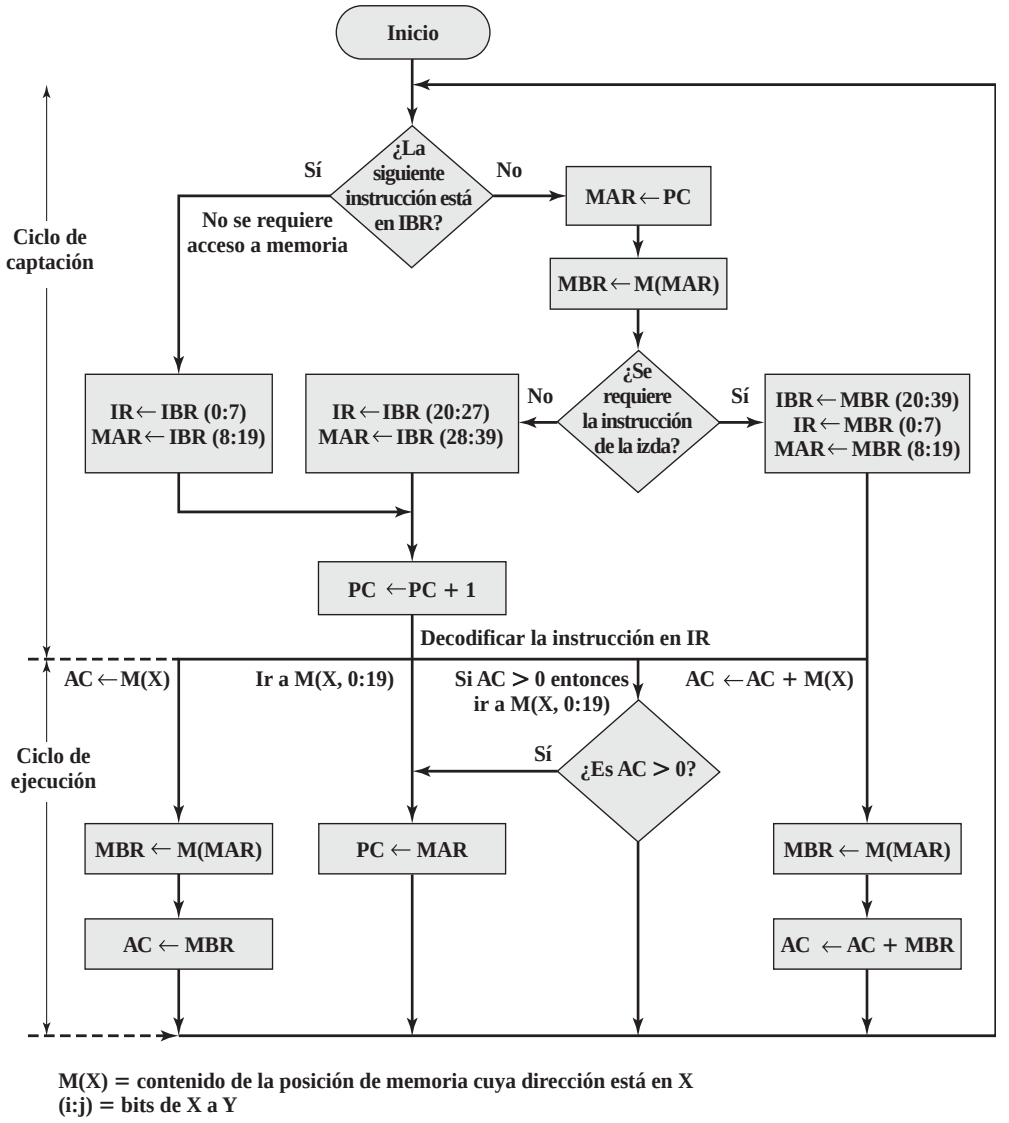


Figura 2.4. Diagrama de flujo parcial de las operaciones del IAS.

- **Aritmética:** operaciones realizadas por la ALU.
- **Modificación de direcciones:** permite que la ALU haga operaciones con las direcciones y las inserte en instrucciones almacenadas en memoria. Esto permite una considerable flexibilidad de direccionamiento en un programa.

La Tabla 2.1 presenta las instrucciones en una forma simbólica y fácil de leer. En realidad, cada instrucción debe tener el formato de la Figura 2.2b. La parte de codop (los ocho primeros bits) especifican cual de las 21 instrucciones va a ser ejecutada. La parte de la dirección (los doce bits restantes) especifican cual de las 1 000 posiciones de memoria está implicada en la ejecución de la instrucción.

Tabla 2.1. El conjunto de instrucciones del IAS.

Tipo de instrucción	Codop	Representación simbólica	Descripción
Transferencia de datos	00001010	LOAD MQ	Transferir el contenido del registro MQ al acumulador AC
	00001001	LOAD MQ, M(X)	Transferir el contenido de la posición de memoria X a MQ
	00100001	STOR M(X)	Transferir el contenido del acumulador a la posición de memoria X
	00000001	LOAD M(X)	Trasferir M(X) al acumulador
	00000010	LOAD – M(X)	Transferir – M(X) al acumulador
	00000011	LOAD |M(X)|	Transferir el valor absoluto de M(X) al acumulador
	00000100	LOAD – |M(X)|	Transferir – |M(X)| al acumulador
Salto incondicional	00001101	JUMP M(X,0:19)	Captar la siguiente instrucción de la mitad izquierda de M(X)
	00001110	JUMP M(X,20:39)	Captar la siguiente instrucción de la mitad derecha de M(X)
Salto condicional	00001111	JUMP + M(X,0:19)	Si el número en el acumulador es no negativo, captar la siguiente instrucción de la mitad izquierda de M(X)
	00010000	JUMP + M(X,20:39)	Si el número en el acumulador es no negativo, captar la siguiente instrucción de la mitad derecha de M(X)
Aritmética	00000101	ADD M(X)	Sumar M(X) a AC; colocar el resultado en AC
	00000111	ADD |M(X)|	Sumar |M(X)| a AC; colocar el resultado en AC
	00000110	SUB M(X)	Restar M(X) a AC; colocar el resultado en AC
	00001000	SUB |M(X)|	Restar |M(X)| a AC; colocar el resultado en AC
	00001011	MUL M(X)	Multiplicar M(X) por MQ: colocar los bits más significativos del resultado de AC, y los menos significativos en MQ
	00001100	DIV M(X)	Dividir AC por M(X); colocar el cociente en MQ y el resto en AC
	00010100	LSH	Multiplicar el acumulador por 2; esto es, desplazar su contenido una posición a la derecha
	00010101	RSH	Dividir el acumulador por 2; esto es, desplazar su contenido una posición a la derecha
Modificación de direcciones	00010010	STOR M(X,8:19)	Reemplazar el campo de dirección de la izquierda de M(X) por los 12 bits de la derecha de AC
	00010011	STOR M(X,28:39)	Reemplazar el campo de dirección de la derecha de M(X) por los doce bits de la derecha de AC

La Figura 2.4 muestra varios ejemplos de la ejecución de una instrucción por la unidad de control. Hay que destacar que cada operación requiere varios pasos. Algunas son bastante complejas. ¡La operación de multiplicación requiere 39 suboperaciones, una para cada bit excepto para el bit de signo!

Computadores comerciales Los años cincuenta contemplaron el nacimiento de la industria de computadores con dos compañías, Sperry e IBM, dominando el mercado.

En 1947 Eckert y Mauchly formaron la Eckert-Mauchly Computer Corporation para fabricar computadores con fines comerciales. Su primera máquina de éxito fue el UNIVAC I (*Universal Automatic Computer*), que fue empleada por la oficina del censo para sus cálculos en 1950. La Eckert-Mauchly Computer Corporation formó luego parte de la división UNIVAC de la Sperry-Rand Corporation, que siguió construyendo una serie de máquinas sucesoras de la primera.

El UNIVAC I fue el primer computador comercial de éxito. Estaba diseñado, como su nombre indica, tanto para aplicaciones científicas como comerciales. El primer documento que describía el sistema mencionaba como ejemplos de tareas que podía realizar operaciones algebraicas con matrices, problemas de estadística, reparto de primas para las compañías de seguros de vida y problemas logísticos.

El UNIVAC II, que tenía una capacidad de memoria mayor y más aplicaciones que el UNIVAC I, salió al mercado al final de los cincuenta e ilustra varias tendencias que han permanecido como características de la industria de computadores. Primera, los avances en la tecnología permiten a las compañías seguir construyendo computadores más grandes y más potentes. Segunda, cada compañía intenta hacer sus nuevas máquinas superiores y compatibles con las anteriores. Esto significa que los programas escritos para las viejas máquinas pueden ejecutarse en las nuevas máquinas. Esta estrategia se adopta para retener la base de clientes; es decir, que cuando un cliente decide comprar una máquina nueva, probablemente la comprará a la misma compañía para evitar perder su inversión en programas.

La división UNIVAC comenzó también el desarrollo de la serie de computadores 1100, que fue su producto principal. Esta serie ilustra una distinción que existió en aquella época. El primer modelo, el UNIVAC 1103, y sus sucesores durante muchos años, estaban diseñados principalmente para aplicaciones científicas que implicaban cálculos largos y complejos. Otras compañías se centraron en el campo de la gestión, lo que conllevaba el procesamiento de grandes cantidades de textos. Esta separación desapareció hace tiempo, pero fue patente durante algunos años.

IBM, que había ayudado a construir el Mark I y era entonces el principal fabricante de equipos de procesamiento con tarjetas perforadas, sacó su primer computador con programas almacenados electrónicamente, el 701, en 1953. El 701 fue diseñado principalmente para aplicaciones científicas [BASH81]. En 1955 IBM presentó los productos 702, que tenían varias características hardware que lo hacían adecuado para aplicaciones de gestión. Estos fueron los primeros de una larga serie de computadores 700/7000 que situaron a IBM como el fabricante de computadores dominante, con gran diferencia.

LA SEGUNDA GENERACIÓN: LOS TRANSISTORES

El primer cambio importante en los computadores electrónicos vino con la sustitución de los tubos de vacío por transistores. El transistor es más pequeño, más barato, disipa menos calor y puede ser usado

Tabla 2.2. Generación de computadores.

Generación	Fechas aproximadas	Tecnología	Velocidad típica (operaciones por segundo)
1	1946-1957	Válvulas	40 000
2	1958-1964	Transistores	200 000
3	1965-1971	Pequeña y media integración	1 000 000
4	1972-1977	Gran integración (LSI)	10 000 000
5	1978-1991	Alta integración (VLSI)	100 000 000
6	1991-	Ultra alta integración (ULSI)	1 000 000 000

de la misma forma que un tubo de vacío en la construcción de computadores. Mientras que un tubo de vacío requiere cables, placas de metal, una cápsula de cristal y vacío, el transistor es un *dispositivo de estado sólido*, hecho con silicio.

El transistor fue inventado en los Laboratorios Bell en 1947 y en los años cincuenta y provocó una revolución electrónica. Sin embargo, los computadores completamente transistorizados no estuvieron disponibles comercialmente hasta el final de los cincuenta. IBM no fue la primera compañía que lanzó esta nueva tecnología. NCR y, con más éxito, RCA fueron los primeros en sacar pequeñas máquinas de transistores. IBM los siguió pronto con la serie 7000.

El uso del transistor define la *segunda generación* de computadores. La clasificación de los computadores en generaciones basándose en la tecnología hardware empleada fue ampliamente aceptada (Tabla 2.2). Cada nueva generación se caracteriza por la mayor velocidad, mayor capacidad de memoria y menor tamaño que la generación anterior.

También hay otros cambios. En la segunda generación se introdujeron unidades lógicas y aritméticas y unidades de control más complejas, el uso de lenguajes de programación de alto nivel, y se proporcionó un *software del sistema* con el computador.

La segunda generación es destacable también por la aparición de la empresa Digital Equipment Corporation (DEC). DEC fue fundada en 1957 y en este año sacó su primer computador, el PDP-1. Este computador y esta compañía iniciaron el desarrollo de los minicomputadores que fue de gran importancia en la tercera generación.

El IBM 7094 Desde la introducción en 1952 de la serie 700 y la introducción del último miembro en 1964 de la serie 7000, esta línea de productos sufrió la evolución típica de los computadores. Los productos sucesivos de la línea presentaron un aumento de prestaciones y capacidad y/o la disminución de precios.

Esta tendencia se puede ver en la Tabla 2.3. El tamaño de la memoria principal, en múltiplos de 2^{10} palabras de 36 bits, creció de 2K ($1K = 2^{10}$) a 32K palabras [3], mientras que el tiempo de acceso a una palabra de memoria, el *tiempo de ciclo de memoria*, cayó de 30 μs a 1,4 μs. El número de códigos de operación creció de un modesto 24 a 185.

[3] El uso de prefijos numéricos, como kilo y giga, se explica y trata en un documento de la web de Recursos del Estudiante de Informática en WilliamStallings.com/StudentSupport.html.

Tabla 2.3. Ejemplos de miembros de la serie IBM 700/7000.

Modelos	Primera entrega	Tecnología de la CPU	Tecnología de la memoria	Tiempo de ciclo (μs)	Tamaño de memoria (K)	Número de codops	Número de registros índice	Punto flotante por hardware	Solapamiento de E/S (Canales)	Solapamiento de captación instrucciones	Velocidad de captación (relativa a 701)
701	1952	Tubos de vacío	Tubos electro-está.	30	2-4	24	0	no	no	no	1
704	1955	Tubos de vacío	Núcleos	12	4-32	80	3	sí	no	no	2,5
709	1958	Tubos de vacío	Núcleos	12	32	140	3	sí	sí	no	4
7090	1960	Transistor	Núcleos	2,18	32	169	3	sí	sí	no	25
7094.I	1962	Transistor	Núcleos	2	32	185	7	sí (doble precisión	sí	sí	30
7094.II	1964	Transistor	Núcleos	1,4	32	185	7	sí (doble precisión	sí	sí	50

La columna final indica la velocidad de ejecución relativa de la CPU. El incremento de velocidad se logró mejorando la electrónica (por ejemplo, una implementación con transistores es más rápida que con tubos de vacío) y con una circuitería más compleja. Por ejemplo, el IBM 7094 incluye un registro de respaldo de instrucciones, usado como buffer de la siguiente instrucción. La unidad de control capta las dos palabras adyacentes de la memoria para captar una instrucción. Excepto en una instrucción de salto, que suele ser poco frecuente, esto significa que la unidad de control tiene que acceder a la memoria en busca de una instrucción en solo la mitad de los ciclos de instrucción. Esta precaptación reduce considerablemente el tiempo medio de ciclo de instrucción.

El significado del resto de las columnas de la Tabla 2.3 es claro tras la explicación anterior.

La Figura 2.5 muestra una configuración (con muchos periféricos) del IBM 7094, que es representativo de los computadores de la segunda generación [BELL71a]. Merece la pena señalar varias diferencias con el computador IAS. La más importante es el uso de *canales de datos*. Un canal de datos es un módulo de E/S independiente con su propio procesador y su propio conjunto de instrucciones. En un computador con tales dispositivos, la CPU no ejecuta instrucciones detalladas de E/S. Tales instrucciones son almacenadas en una memoria principal para ser ejecutadas con un procesador de uso específico para el canal de datos mismo. La CPU inicia una transferencia de E/S enviando señales de control al canal de datos, instruyéndolo para ejecutar una secuencia de instrucciones en

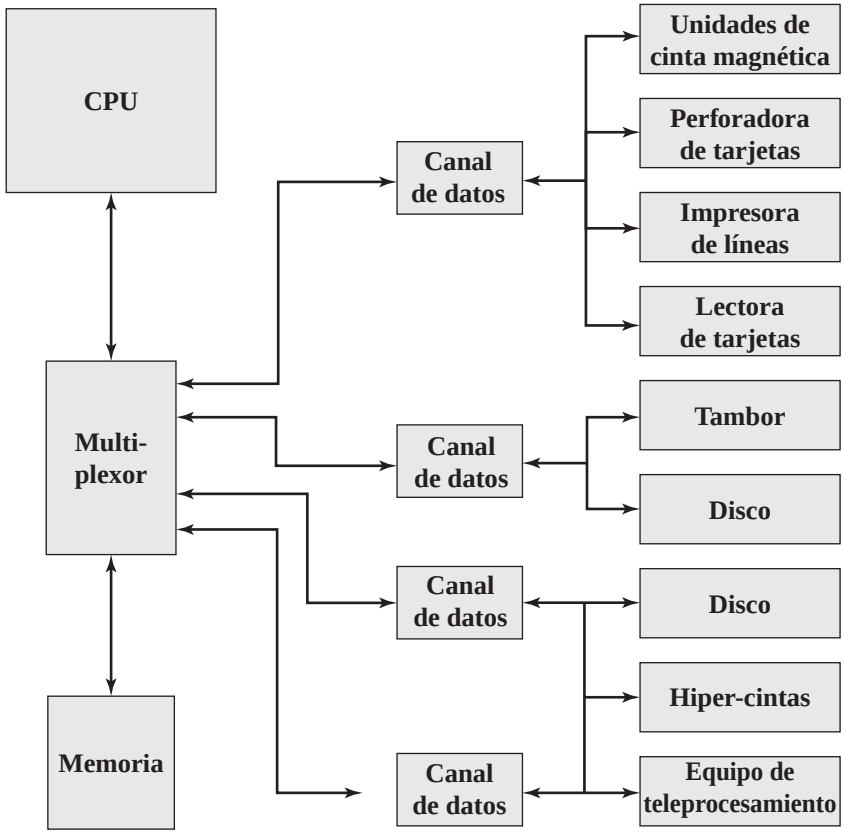


Figura 2.5. Configuración de un IBM 7094.

memoria. El canal de datos realiza esta tarea independientemente de la CPU y de las señales de la CPU hasta que la operación se completa. Esta disposición libera a la CPU de una carga de procesamiento considerable.

Otra característica es el *multiplexor*, que es el punto de conexión central de los canales de datos, la CPU y la memoria. El multiplexor organiza los accesos a la memoria desde la CPU y los canales de datos, permitiendo a estos dispositivos actuar de forma independiente.

LA TERCERA GENERACIÓN: LOS CIRCUITOS INTEGRADOS

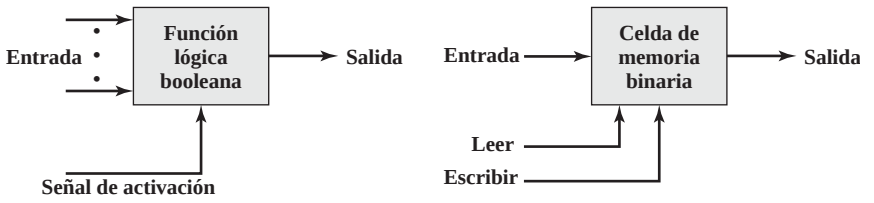
A un transistor simple y autocontenido se le llama *componente discreto*. A través de los años cincuenta y principios de los sesenta, los equipos electrónicos estaban compuestos en su mayoría por componentes discretos —transistores, resistencias, capacidades, etc.—. Los componentes discretos se fabricaban separadamente, encapsulados en sus propios contenedores, y soldados o cableados juntos en tarjetas de circuitos en forma de panel, que eran instalados en computadores, osciloscopios y otros equipos electrónicos. Cuando un dispositivo necesitaba un transistor, había que soldar este, que tenía una forma de un pequeño tubo de metal y contenía una pieza de silicio del tamaño de la cabeza de un alfiler, en una tarjeta de circuitos. Todo el proceso de fabricación desde el transistor hasta el panel de circuitos era caro y engorroso.

Estos hechos fueron el comienzo del surgimiento de problemas en la industria de computadores. Los primeros computadores de la segunda generación contenían alrededor de 10 000 transistores. Esta cantidad creció a cientos de miles, haciendo cada vez más difícil la fabricación de máquinas nuevas y más potentes.

En 1958 ocurrió algo que revolucionó la electrónica y comenzó la era de la microelectrónica: la invención del circuito integrado. El circuito integrado define la tercera generación de computadores. En esta sección haremos una breve introducción a la tecnología de circuitos integrados. Después veremos los que quizá sean los dos miembros más importantes de la tercera generación, que surgieron al principio de la era: el IBM Sistema/360 y el DEC PDP-8.

Microelectrónica Microelectrónica significa literalmente «pequeña electrónica». Desde los comienzos de la electrónica digital y la industria de computadores, ha habido una tendencia persistente y consistente hacia la reducción del tamaño de los circuitos electrónicos digitales. Antes de examinar las implicaciones y beneficios de esta tendencia, necesitamos decir algo sobre la naturaleza de la electrónica digital. En el Apéndice A se encuentra una discusión más detallada.

Los elementos básicos de un computador digital, como ya sabemos, deben ofrecer almacenamiento, procesamiento y control de funciones. Solo se requieren dos tipos fundamentales de componentes (Figura 2.6): puertas y celdas de memoria. Una puerta es un dispositivo que implementa una función lógica o booleana simple, como SI *A* AND *B* ES CIERTO ENTONCES *C* ES CIERTO (puerta AND). A tales dispositivos se les llama puertas porque controlan el flujo en cierta manera, como lo hacen las puertas de un canal. La celda de memoria es un dispositivo que puede almacenar un dato de un bit; es decir, el dispositivo puede estar, en un instante dado, en uno de dos estados estables. Interconectando muchos de estos dispositivos fundamentales, podemos construir un computador. Podemos relacionar esto con nuestras cuatro funciones básicas de la siguiente forma:



(a) Puerta (b) Celda de memoria

Figura 2.6. Elementos de un computador básico.

- **Almacén de datos:** proporcionado por las celdas de memoria.
- **Procesamiento de datos:** proporcionado por las puertas.
- **Transferencias de datos:** los caminos entre componentes se usan para llevar datos de memoria a memoria y de memoria, a través de las puertas, a memoria.
- **Control:** los caminos entre componentes pueden llevar las señales de control. Por ejemplo, una puerta tendrá dos entradas de datos más una entrada de control que activará la puerta. Cuando la señal de control está en ON, la puerta realiza su función con los datos de entrada y produce un dato de salida. De manera similar, las celdas de memoria almacenarán el bit en su entrada si la señal de control WRITE está ON y situarán el bit en la salida cuando la señal de control READ esté ON.

Por tanto, un computador consta de puertas, celdas de memoria e interconexiones entre estos elementos. Las puertas y las celdas de memoria están constituidas por componentes electrónicos simples.

Los circuitos integrados utilizaron el hecho de que componentes como transistores, resistencias y conductores podían ser fabricados a partir de un semiconductor como el silicio. Es simplemente un avance del arte del estado sólido consistente en fabricar un circuito entero en un pequeño trozo de silicio, en vez de ensamblar componentes discretos hechos a partir de trozos de silicio separados en el mismo circuito. Se pueden construir cientos e incluso miles de transistores al mismo tiempo en una sola oblea de silicio. Igualmente importante es que estos transistores pueden ser conectados con un proceso de metalización para formar circuitos.

La Figura 2.7 muestra los conceptos clave de un circuito integrado. Se divide una fina *oblea* de silicio en una matriz de pequeñas áreas, cada una de unos pocos milímetros cuadrados. Se fabrica el mismo patrón de circuito en cada área, y la oblea se divide en *chips*. Cada chip consiste en muchas puertas más una serie de puntos para conexiones de entrada y salida. El chip es encapsulado en una carcasa que lo proteje y proporciona patas para conectar dispositivos fuera del chip. Varios de estos elementos pueden ser interconectados en una tarjeta de circuito impreso para producir circuitos más complejos y mayores.

Inicialmente solo podían fabricarse y encapsularse juntas, con fiabilidad, unas pocas puertas o celdas de memoria. A estos primeros circuitos integrados se les llama de *pequeña escala de integración* (SSI, *Small-Scale Integration*). A medida que el tiempo pasó, fue posible encapsular más y más componentes en un mismo chip. Este crecimiento en densidad se puede ver en la Figura 2.8; esta es

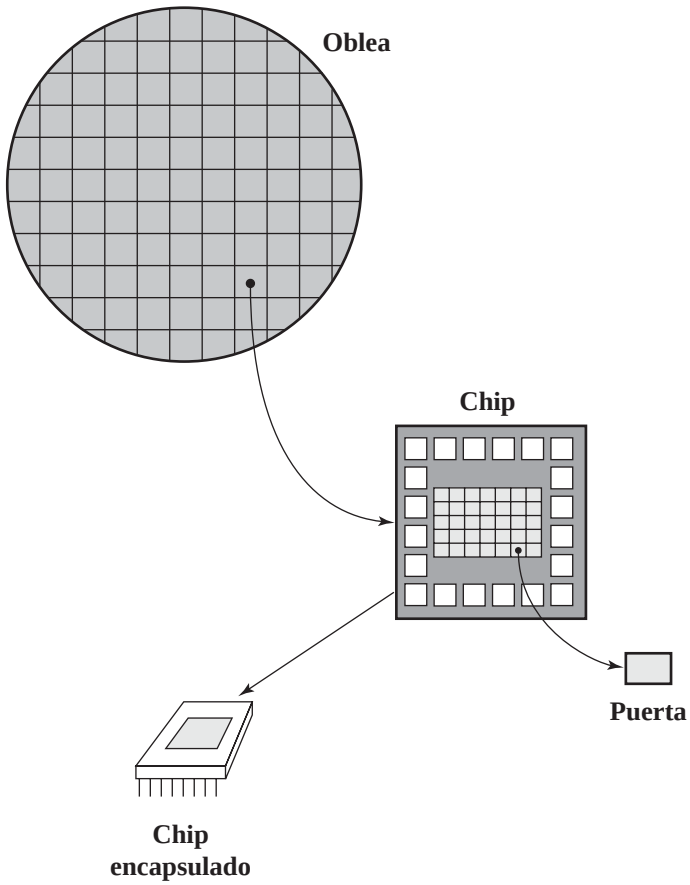


Figura 2.7. Relación entre oblea, chip y puerta.

una de las tendencias tecnológicas más importantes que nunca se han visto [4]. Esta figura refleja la famosa ley de Moore, que fue propuesta por Gordon Moore, cofundador de Intel, en 1965 [MOOR65]. Moore observó que el número de transistores que se podrían integrar en un solo chip se duplicaba cada año y se predecía correctamente que esto continuaría en un futuro cercano. Para sorpresa de muchos, incluido Moore, este ritmo continuaría año tras año y década tras década. El ritmo disminuyó duplicándose cada dieciocho meses en los setenta, pero ha mantenido esta velocidad desde entonces.

Las consecuencias de la ley de Moore son profundas:

1. El precio de un chip ha permanecido prácticamente invariable a través de este periodo de rápido crecimiento en densidad. Esto significa que el coste de la lógica del computador y de la circuitería de la memoria ha caído a una velocidad drástica.
2. Ya que los elementos de la lógica y la memoria están más próximos en chips más densamente encapsulados, la longitud de las interconexiones eléctricas ha disminuido, incrementándose así la velocidad operativa.

[4] Nótese que el eje vertical utiliza una escala logarítmica. En el documento de repaso de matemáticas en la web de Recursos del Estudiante de Informática en WilliamStallings.com/StudentSupport.html se hace un repaso básico de las escalas logarítmicas.

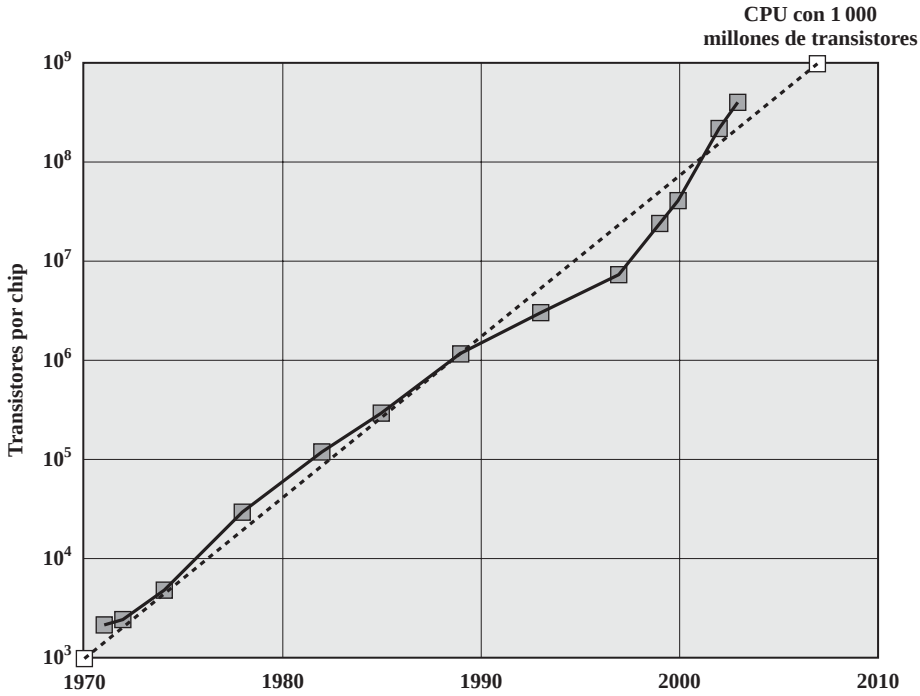


Figura 2.8. Crecimiento en el número de transistores en la CPU [BOHR03].

3. El computador es ahora más pequeño, lo que lo hace más adecuado para más entornos.
4. Hay una reducción de las necesidades de potencia y refrigeración.
5. Las interconexiones de los circuitos integrados son mucho más fiables que las conexiones soldadas. Con más circuitos en cada chip hay menos conexiones entre chips.

En 1964, IBM tenía un firme dominio del mercado con sus máquinas de la serie 7000. Aquel año, IBM anunció el Sistema/360, una nueva familia de productos de computadores. Aunque el anuncio mismo no fue ninguna sorpresa, contenía algunas noticias desagradables para los clientes habituales de IBM: la línea de productos 360 era incompatible con las máquinas IBM anteriores. Por ello la transición al 360 sería difícil para los clientes de IBM. Este fue un paso audaz de IBM, pero sentían que era necesario romper con algunas de las limitaciones de la arquitectura 7000 y producir un sistema capaz de evolucionar junto con la nueva tecnología de circuitos integrados [PADE81, GIFF87]. La estrategia resultó provechosa tanto técnica como financieramente. El 360 fue el éxito de la década y consolidó a IBM como el dominante absoluto en las ventas de computadores, con una cuota de mercado por encima del setenta por ciento. Y, con algunas modificaciones y ampliaciones, la arquitectura del 360 permanece hasta hoy en día en la arquitectura de los grandes computadores (*mainframe*) de IBM [5]. A lo largo del texto se pueden encontrar ejemplos que utilizan esta arquitectura.

[5] El término *mainframe* (gran computador), se usa para designar a los computadores más grandes y potentes, después de los supercomputadores. Las características típicas de un gran computador son que soporta grandes bases de datos, tienen un hardware de E/S elaborado, y se usan para procesamiento de datos centralizados.

El Sistema/360 fue la primera *familia* de computadores de la historia que se planeó. La familia abarcaba un amplio rango de prestaciones y precios. La Tabla 2.4 indica alguna de las características clave de los distintos modelos en 1965 (cada miembro de la familia se distingue por un número de modelo). Los distintos modelos eran compatibles en el sentido de que un programa escrito para un modelo, tenía que ser capaz de ser ejecutado por otro modelo de la serie, con la única diferencia del tiempo de ejecución.

El concepto de familia de computadores compatibles era a la vez novedoso y extremadamente exitoso. Un cliente con necesidades modestas y un presupuesto limitado podía empezar con el modelo 30, relativamente barato. Más tarde, si las necesidades del cliente crecían, era posible pasarse a una máquina más rápida, con más memoria, sin sacrificar la inversión ya realizada en software. Las características de una familia son:

- **Conjunto de instrucciones similar o idéntico:** en muchos casos, se encuentran exactamente el mismo conjunto de instrucciones máquina en todos los miembros de la familia. Así, un programa que se ejecuta en una máquina, se podrá ejecutar en cualquier otra. En algunos casos el computador más bajo de la familia tiene un conjunto de instrucciones que es un subconjunto del computador más alto de la familia. Esto quiere decir que los programas se pueden mover hacia arriba pero no hacia abajo.
- **Sistemas operativos similares o idénticos:** el mismo sistema operativo básico está disponible para todos los miembros de la familia. En algunos casos, se añaden características complementarias a los miembros más altos.
- **Velocidad creciente:** la velocidad de ejecución de las instrucciones se incrementa conforme se sube desde los miembros más bajos a los más altos de la familia.
- **Número creciente de puertos de E/S:** conforme se va desde lo más bajo a los más alto de la familia.
- **Tamaño de memoria creciente:** conforme se va de lo más bajo a lo más alto de la familia.
- **Coste creciente:** conforme se va de lo más bajo a lo más alto de la familia.

¿Cómo podría implementarse tal concepto de familia? Las diferencias entre los modelos se basaron en tres factores: la velocidad básica, el tamaño y el grado de simultaneidad [STEV64]. Por ejemplo, podría lograrse mayor velocidad en la ejecución de una instrucción dada usando una circuitería

Tabla 2.4. Características clave de la familia Sistema/360.

Características	Modelo 30	Modelo 40	Modelo 50	Modelo 65	Modelo 75
Tamaño máximo de memoria (bytes)	64K	256K	256K	512K	512K
Velocidad de transferencia de datos procedentes de la memoria (MB/segundo)	0,5	0,8	2,0	8,0	16,0
Tiempo de ciclo del procesador (μ/segundo)	1,0	0,625	0,5	0,25	0,2
Velocidad relativa	1	3,5	10	21	50
Número máximo de canales de datos en un canal (KB/segundo)	250	400	800	1.250	1.250

más compleja en la ALU, permitiendo que las suboperaciones se llevaran a cabo en paralelo. Otro modo de incrementar la velocidad era incrementar la amplitud del camino de los datos entre la memoria principal y la CPU. En el Modelo 30, solo se podía captar un byte (8 bits) a la vez de la memoria principal, mientras que en el Modelo 75 se podían captar ocho bytes a la vez.

El Sistema /360 no solamente dictó la carrera hacia el futuro de IBM, sino también tuvo un profundo impacto en toda la industria. Muchas de sus características se han convertido en un estándar para otros grandes computadores.

DEC PDP-8 En el mismo año que IBM lanzó su primer Sistema/360 tuvo lugar otro lanzamiento trascendental: el PDP-8 de DEC. En aquella época, cuando la mayoría de los computadores requerían una habitación con aire acondicionado, el PDP-8 (llamado por la industria minicomputador en honor a la minifalda de aquellos tiempos) era lo bastante pequeño para ser colocado en lo alto de una mesa de laboratorio o embutido en otro equipo. No podía hacer todo lo que hacían los grandes computadores, pero a 16 000 dólares era suficientemente barato para que cada técnico de laboratorio tuviera uno. Por contra, los computadores de la serie Sistema/360, presentados solo unos meses antes costaban cientos de miles de dólares.

El bajo costo y pequeño tamaño del PDP-8 permitía a otros fabricantes comprarse un PDP-8 e integrarlo en un sistema global para revenderlo. Estos otros fabricantes se conocían como fabricantes de equipos originales (OEM), y el mercado de OEM llegó a tener y aún tiene la mayor cuota del mercado de computadores.

El PDP-8 fue un éxito inmediato y logró el enriquecimiento de DEC. Esta máquina y los otros miembros de la familia PDP-8 que la siguieron (*véase* Tabla 2.5) lograron un status de producción antes reservado a los computadores IBM, con alrededor de 50 000 máquinas vendidas en los siguientes doce años. Como se dice en la historia oficial de DEC, el PDP-8 «estableció el concepto de minicomputador, abriendo el camino a una industria de miles de millones de dólares». También estableció a DEC como el vendedor de minicomputadores número uno y cuando el PDP-8 alcanzó el fin de su vida útil, DEC era el segundo fabricante de computadores detrás de IBM.

En contraste con la arquitectura de conmutador central (Figura 2.5) usada por IBM en sus sistemas 700/7000 y 360, los últimos modelos del PDP-8 usaban una estructura que ahora es prácticamente universal para minicomputadores y microcomputadores: la estructura de bus. Esto se muestra en la Figura 2.9. El bus PDP-8, llamado Omnibus, consiste en 96 hilos conductores separados, usados para control, direccionamiento y datos. Como todos los componentes del sistema comparten un conjunto de caminos, su uso debe estar controlado por la CPU. Esta arquitectura es altamente flexible, permitiendo conectar módulos al bus para crear varias configuraciones.

ÚLTIMAS GENERACIONES

Más allá de la tercera generación hay menos acuerdo general en la definición de las generaciones de computadores. En la Tabla 2.2 se sugieren las que serían la cuarta y la quinta generación, basadas en los avances de la tecnología de los circuitos integrados. Con la introducción de la integración a gran escala (LSI, *Large-Scale Integration*), podía haber más de 1 000 componentes en un simple chip de circuito integrado. Con la integración a muy gran escala (VLSI, *Very-Large Scale Integration*), se lograron más de 10 000 componentes por chip, y los chips VLSI actuales pueden contener más de 100 000 componentes.

Tabla 2.5. Evolución del PDP-8 [VOEL88].

Modelo	Primera venta	Coste del procesador +4K palabras de 12 bit de memoria (miles de dólares)	Velo. transfer. de memoria (Pal./µseg.)	Vol. (pies cúbicos)	Innovaciones y mejoras
PDP-8	4/65	16,2	1,26	8,0	Producción automática con conductores enrollados (*wire-wrapping*)
PDP-8/5	9/66	8,79	0,08	3,2	Implementación de instrucciones serie
PDP-8/1	4/68	11,6	1,34	8,0	Circuitos integrados de media escala
PDP-8/L	11/68	7,0	1,26	2,0	Chasis menor
PDP-8/E	3/71	4,99	1,52	2,2	Omnibus
PDP-8/M	6/72	3,69	1,52	1,8	Chasis de la mitad de tamaño y menos ranuras que el 8/E
PDP-8/1	1/75	2,6	1,34	1,2	Memoria de semiconductor; procesador de punto flotante

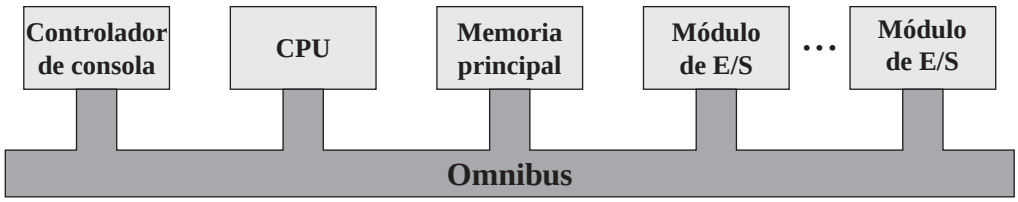


Figura 2.9. Estructura del bus PDP-8.

Con el gran avance de la tecnología, la rápida introducción de nuevos productos, y la importancia del software y las comunicaciones así como del hardware, la clasificación en generaciones se vuelve cada vez menos clara y menos significativa. Se podría decir que la aplicación comercial de nuevos desarrollos resultó uno de los principales cambios de principios de los años setenta y los resultados de estos cambios duran todavía. En esta sección mencionaremos dos de los más importantes.

Memoria semiconductora La primera aplicación de la tecnología de circuitos integrados en computadores dejó a un lado la construcción del procesador (la unidad de control y la unidad aritmético lógica) con chips de circuitos integrados. Sin embargo, se encontró que esta misma tecnología podía usarse para construir memorias.

En los años cincuenta y sesenta, la mayoría de las memorias de los computadores se hacían con pequeños anillos de material ferromagnético, cada uno de un dieciseisavo de pulgada de diámetro. Estos anillos de ferrita se insertaban en mallas de finos cables engarzados en pequeños marcos dentro del computador. Se magnetizaba en un sentido el anillo (llamado *núcleo*) y representaba un uno; magnetizado en el otro sentido, representaba un cero. La memoria de núcleo magnético era más bien rápida; tardaba tan poco como una milésima de segundo en leer un bit almacenado en memoria. Pero era cara, voluminosa y usaba lectura destructiva: el simple hecho de leer un núcleo borraba los datos almacenados en él. Era, por consiguiente, necesario hacer circuitos que recuperaran el dato tan pronto como se extraía.

Entonces, en 1970, Fairchild produjo la primera memoria semiconductora con relativa capacidad. Este chip, del tamaño de un sencillo núcleo de ferrita, podía tener 256 bits de memoria. Era no destructiva y mucho más barata que un núcleo. Tardaba solamente setenta mil millonésimas de segundo en leer un bit. Sin embargo, el coste por bit era mayor que el de un núcleo.

En 1974, ocurrió un hecho sorprendente: el precio por bit de memoria semiconductora cayó por debajo del precio por bit de memoria de núcleo. Siguiendo esto, ha habido una continua disminución del precio de la memoria acompañado de un correspondiente aumento de la densidad de memoria. Esto ha llevado, en pocos años, a hacer máquinas más pequeñas y más rápidas con el mismo tamaño de memoria que máquinas más grandes y más caras. El desarrollo de la tecnología de memorias, junto con el desarrollo de la tecnología de procesadores, del que hablaremos después, cambiaron la naturaleza de los computadores en menos de una década. Aunque los computadores caros y voluminosos permanecieron dentro del panorama, el computador se ha llevado también al «consumidor final», en forma de máquinas de oficina y de computadores personales.

A partir de 1970, la memoria semiconductora ha tenido ocho generaciones: 1K, 4K, 16K, 64K, 256K, 1M, 4M, 16M, 64M, 256M y ahora un giga bits en un solo chip ($1K = 2^{10}$, $1M = 2^{20}$, $1G = 2^{30}$). Cada generación ha proporcionado cuatro veces más densidad de almacenamiento que la generación previa, junto con un menor coste por bit y una mayor velocidad de acceso.

Microprocesadores Igual que la densidad de elementos en los chips de memoria ha continuado creciendo, también lo ha hecho la densidad de elementos de procesamiento. Conforme el tiempo pasaba, en cada chip había más y más elementos, así que cada vez se necesitaban menos y menos chips para construir un procesador de un computador.

En 1971 se hizo una innovación sensacional, cuando Intel desarrolló su 4004. el 4004 fue el primer chip que contenía *todos* los componentes de la CPU en un solo chip: el microprocesador había nacido.

El 4004 podía sumar dos números de cuatro bits y multiplicar solo con sumas sucesivas. Según los estándares de hoy en día, el 4004 es muy primitivo, pero marcó el comienzo de la evolución continua en capacidad y potencia de los microprocesadores.

Esta evolución se puede ver más fácilmente considerando el número de bits que el procesador trata a la vez. No hay una medida clara de esto, pero quizá la mejor medida es la anchura del bus de datos: el número de bits de un dato que pueden venir o ir al procesador a la vez. Otra medida es el número de bits del acumulador o del conjunto de registros de uso general. A menudo, estas medidas coinciden, pero no siempre. Por ejemplo, hay una serie de microprocesadores que operan con números de 16 bits en los registros, pero que solo pueden leer y escribir ocho bits a la vez.

El siguiente paso importante en la evolución de los microprocesadores fue la introducción en 1972 del Intel 8008. Este fue el primer microprocesador de ocho bits y era casi dos veces más complejo que el 4004.

Ninguno de estos pasos tuvo el impacto del siguiente acontecimiento importante: la introducción del Intel 8080 en 1974. Este fue el primer microprocesador de uso general. Mientras que el 4004 y el 8008 habían sido diseñados para aplicaciones específicas, el 8080 fue diseñado para ser la CPU de un microcomputador de propósito general. Al igual que el 8008, el 8080 es un microprocesador de ocho bits. El 8080, sin embargo, es más rápido, tiene un conjunto de instrucciones más rico y tiene una capacidad de direccionamiento mayor.

Sobre la misma época empezaron a desarrollarse los microprocesadores de 16 bits. Sin embargo, hasta el final de los setenta no aparecieron estos potentes microprocesadres de 16 bits de propósito general. Uno de estos fue el 8086. El siguiente paso en esta tendencia ocurrió en 1981, cuando los Laboratorios Bell y Hewlett-Packard desarrollaron microprocesadores de un solo chip de 32 bits. Intel introdujo su microprocesador de 32 bits, el 80386, en 1985 (Tabla 2.6).

Tabla 2.6. Evolución de los microprocesadores Intel.

a) Procesadores de los años setenta

	4004	8008	8080	8086	8088
Fecha de introducción	1971	1972	1974	1978	1979
Velocidad de reloj	108 kHz	108 kHz	2 MHz	5 MHz, 8 MHz, 10 MHz	5 MHz, 8 MHz
Ancho del bus	4 bits	8 bits	8 bits	16 bits	8 bits
N.º de transistores	2.300	3.500	6.500	29.000	29.000
Tamaño (μm)	10	—	6	3	6
Memoria direccionable	640 Bytes	16 KB	64 KB	1 MB	1 MB
Memoria virtual	—	—	—	—	—

Tabla 2.6. Evolución de los microprocesadores Intel (continuación).

b) Procesadores de los años ochenta

	80286	386TM DX	386TM SX	486TM DX CPU
Fecha de introducción	1982	1985	1988	1989
Velocidad de reloj	6-12,5 MHz	16-33 MHz	16-33 MHz	25-50 MHz
Ancho del bus	16 bits	32 bits	16 bits	32 bits
N.º de transistores	134.000	275.000	275.000	1,2 millones
Tamaño (μm)	1,5	1	1	0,8-1
Memoria direccionable	16 megabytes	4 gigabytes	16 megabytes	4 gigabytes
Memoria virtual	1 gigabyte	64 terabytes	64 terabytes	64 terabytes

c) Procesadores de los años noventa

	486TM SX	Pentium	Pentium Pro	Pentium II
Fecha de introducción	1991	1993	1995	1997
Velocidad de reloj	16-33 MHz	60-166 MHz	150-200 MHz	200-300 MHz
Ancho del bus	32 bits	32 bits	64 bits	64 bits
N.º de transistores	1,185 millones	3,1 millones	5,5 millones	7,5 millones
Tamaño (μm)	1	0,8	0,6	0,35
Memoria direccionable	4 gigabytes	4 gigabytes	64 gigabytes	64 gigabytes
Memoria virtual	64 terabytes	64 terabytes	64 terabytes	64 terabytes

d) Procesadores recientes

	Pentium III	Pentium 4	Itanium	Itanium II
Fecha de introducción	1999	2000	2001	2002
Velocidad de reloj	450-660 MHz	1,3-1,8 GHz	733-800 MHz	900 MHz-1 GHz
Ancho del bus	64 bits	64 bits	64 bits	64 bits
N.º de transistores	9,5 millones	42 millones	25 millones	220 millones
Tamaño (μm)	0,25	0,18	0,18	0,18
Memoria direccionable	64 gigabytes	64 gigabytes	64 gigabytes	64 gigabytes
Memoria virtual	64 terabytes	64 terabytes	64 terabytes	64 terabytes

Fuente: Intel Corp. **http://www.intel.com/intel/museum/**

2.2. DISEÑO BUSCANDO MEJORES PRESTACIONES

Año tras año, el precio de los computadores continúa cayendo dramáticamente, mientras que las prestaciones y la capacidad de estos sistemas sigue creciendo. En una tienda se puede conseguir un computador, por menos de 1000 dólares, con muchas mejores prestaciones que un IBM de hace diez años. Dentro de un computador personal, incluyendo el microprocesador y la memoria, y otros chips, se pueden conseguir unos cien millones de transistores. No se pueden comprar cien millones de nada por tan poco dinero. Esa cantidad en papel higiénico costaría más de 100 000 dólares.

Por tanto, tenemos prácticamente la potencia del computador «gratis». Esta continua revolución tecnológica, ha habilitado el desarrollo de una sorprendente complejidad y potencia. Por ejemplo, las aplicaciones de oficina que requieren la mayor potencia de los sistemas de hoy en día basados en microprocesadores incluyen:

- Procesamiento de imágenes.
- Reconocimiento del habla.
- Vídeo conferencias.
- Aplicaciones multimedia.
- Almacenamiento de ficheros de voz y vídeo.
- Modelado de simulaciones.

Las estaciones de trabajo ahora soportan aplicaciones de ingeniería y ciencia altamente sofisticadas, así como simulaciones, y pueden aplicar los principios de trabajo en grupo a aplicaciones de imagen y vídeo. Además, en los gestión se está confiando en la creciente potencia de los servidores para manejar transacciones y procesamiento de bases de datos y para soportar redes cliente-servidor masivas que han reemplazado los gigantescos centros de computadores de antaño.

Lo que es fascinante sobre todo desde la perspectiva de la organización y arquitectura de computadores es que, por una parte, los bloques básicos de los portentosos computadores de hoy en día son prácticamente los mismos que los del computador IAS de hace casi cincuenta años, mientras que por otra parte, las técnicas para sacar hasta la última gota del rendimiento de los elementos disponibles se han vuelto cada vez más sofisticadas.

Esta observación sirve de guía principal para la presentación de este libro. A medida que avanzamos en los distintos elementos y componentes de un computador, se persiguen dos objetivos. Primero, el libro explica la funcionalidad fundamental en cada área que se considera, y segundo, el libro explora las técnicas requeridas para conseguir el máximo de prestaciones. En el resto de esta sección, destacamos algunos de los factores que hay tras la necesidad de diseñar para obtener mejores prestaciones.

VELOCIDAD DEL MICROPROCESADOR

Lo que le da al Pentium o al PowerPC esa increíble potencia es la persecución sin descanso del incremento de velocidad por parte de los fabricantes del procesador. La evolución de estas máquinas

continúa confirmando la ley de Moore, mencionada previamente. Siempre y cuando esta ley se cumpla, los fabricantes de chips pueden crear una nueva generación de chips cada tres años (con hasta cuatro veces más de transistores). En los chips de memoria, se ha cuadriplicado cada tres años la capacidad de las memorias dinámicas de acceso aleatorio (DRAM) y esta sigue siendo la tecnología básica de la memoria principal de los computadores actuales. En microprocesadores, la adición de nuevos circuitos, y la potenciación de la velocidad que proviene de la reducción de las distancias entre ellos, ha conseguido cuadruplicar o quintuplicar las prestaciones cada tres años desde que Intel lanzó su familia X86 en 1978.

Pero la velocidad bruta del procesador no alcanzará su potencial al menos que se le alimente con un flujo constante de trabajo en forma de instrucciones. Cualquier cosa que se interponga en el camino de este flujo limita la potencia del procesador. Conforme a esto, mientras que los fabricantes de chips han estado ocupados aprendiendo cómo se fabrican chips de densidad cada vez mayor, los diseñadores del procesador tienen que producir técnicas cada vez más elaboradas para alimentar al «monstruo». Entre las técnicas incorporadas a los procesadores de hoy en día están:

- **Predicción de ramificación:** el procesador se anticipa al software y predice qué ramas o grupos de instrucciones se van a procesar después con mayor probabilidad. Si el procesador acierta la mayoría de las veces, puede precaptar las instrucciones correctas y almacenarlas para mantener al procesador ocupado. Los ejemplos más sofisticados de esta estrategia predicen no solo la siguiente rama sino varias de ellas. Por tanto, la predicción de ramificación incrementa la cantidad de trabajo disponible que el procesador debe de ejecutar.
- **Análisis del flujo de datos:** el procesador analiza qué instrucciones dependen de los resultados de otras instrucciones, o datos, para crear una organización optimizada de instrucciones. De hecho, las instrucciones se planifican para ser ejecutadas cuando estén listas, independientemente del orden original del programa. Esto evita retrasos innecesarios.
- **Ejecución especulativa:** utilizando la predicción de ramificación y el análisis de flujo de datos, algunos procesadores ejecutan especulativamente instrucciones antes de que aparezcan en la ejecución del programa, manteniendo los resultados en posiciones temporales. Esto permite al procesador mantener sus elementos de ejecución tan ocupados como sea posible ejecutando anticipadamente instrucciones que es probable que se necesiten.

Estas y otras sofisticadas técnicas se hacen necesarias simplemente porque el procesador es muy potente, haciendo así posible explotar la potencia bruta del procesador.

EQUILIBRIO DE PRESTACIONES

Mientras que la velocidad del procesador ha crecido con increíble rapidez, otros componentes esenciales del computador no lo han hecho tan rápido. El resultado de esto es que ahora hace falta prestar atención al equilibrio de las prestaciones: ajustar la organización y la arquitectura para compensar las desigualdades de capacidad entre los distintos componentes.

El problema creado por tales desigualdades no es de ninguna manera más grave que en la interfaz entre el procesador y la memoria principal. Observemos la historia representada en la Figura 2.10. Mientras la velocidad del procesador y la capacidad de la memoria han crecido rápidamente, la velocidad con la que los datos pueden ser transferidos entre la memoria principal y el procesador se ha

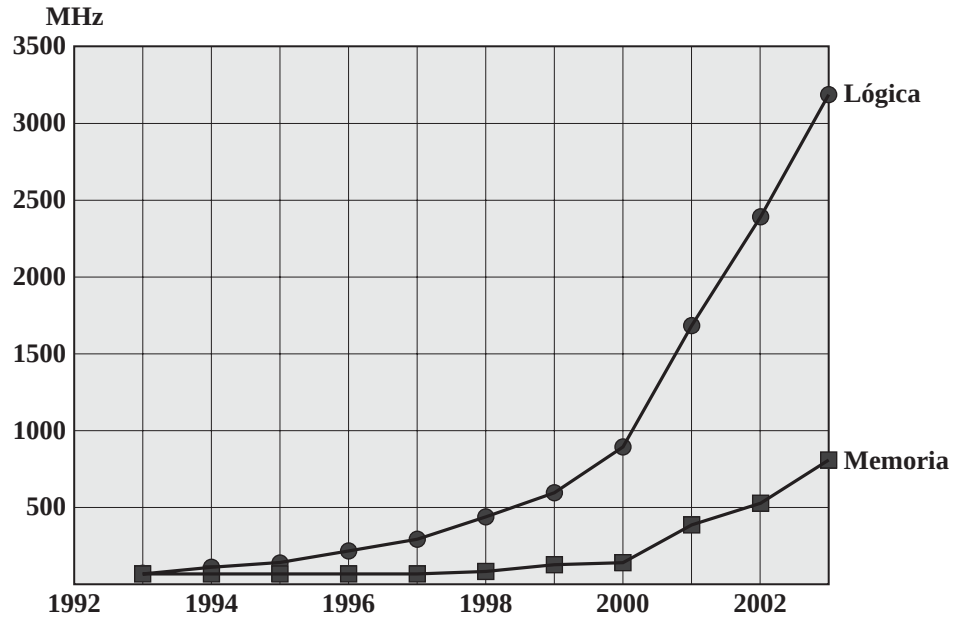


Figura 2.10. Prestaciones de la memoria y la lógica [BORK03].

quedado dramáticamente retrasada. La interfaz entre el procesador y la memoria principal es el camino más importante de todo el computador, ya que es el responsable de llevar el constante flujo de instrucciones y datos entre los chips de la memoria y el procesador. Si la memoria o la interfaz no logran mantener el ritmo de las insistentes demandas del procesador, este se estanca en una posición de espera y se pierde así tiempo de procesamiento valioso.

Hay varias maneras de que una arquitectura pueda atacar este problema, y todas se reflejan en los diseños de computadores contemporáneos. He aquí algunos ejemplos:

- Incrementar el número de bits que se recuperan de una sola vez haciendo las DRAM más «anchas» en lugar de más «profundas» utilizando buses de datos más anchos.
- Cambiar la interfaz DRAM para hacerla más eficiente, incluyendo una caché [6] u otro esquema de almacenamiento temporal en el chip DRAM.
- Reducir la frecuencia del acceso a memoria incorporando, entre el procesador y la memoria principal, unas caché cada vez más complejas y eficientes. Esto incluye la incorporación de una o más caches en el chip del procesador así como una caché fuera del chip cerca del procesador.
- Incrementar el ancho de banda entre el procesador y la memoria usando buses de más alta velocidad y una jerarquía de buses para almacenar y estructurar el flujo de datos.

[6] Una caché es un memoria rápida relativamente pequeña colocada entre una memoria mayor y más lenta y la lógica que accede a la memoria grande. La caché mantiene los datos recientemente accedidos y se diseña para acelerar los sucesivos accesos a los mismos datos. Las cachés se estudian en el Capítulo 4.

Otra área de diseño se centra en el manejo de dispositivos de E/S. Conforme los computadores se hacen más rápidos y potentes, se desarrollan aplicaciones más sofisticadas que se apoyan en el uso de periféricos con demandas intensivas de E/S. La Figura 2.11 muestra algunos ejemplos de los dispositivos periféricos típicos que se usan en ordenadores personales y estaciones de trabajo. Estos dispositivos crean una tremenda demanda de procesamiento de datos. La generación actual de procesadores puede manejar los datos producidos por esos dispositivos, pero aún queda el problema de mover esos datos entre el procesador y los periféricos. Las estrategias en esto incluyen esquemas de cachés y almacenamiento además del uso de buses de interconexión de más alta velocidad y con estructuras más elaboradas. También el uso de configuraciones multiprocesador puede ayudar a satisfacer las demandas de E/S.

La clave en todo esto es el equilibrio. Los diseñadores luchan constantemente por alcanzar el equilibrio en la demanda de rendimiento y procesamiento por parte de los componentes del procesador, la memoria principal, los dispositivos de E/S y de las estructuras de interconexión. Y este diseño tiene que ser constantemente replanteado para hacer frente a dos factores en continua evolución:

- La velocidad a la que el rendimiento está cambiando en las distintas áreas tecnológicas (procesador, buses, memoria, periféricos) difiere enormemente de un tipo de elemento a otro.
- Las nuevas aplicaciones y nuevos dispositivos periféricos cambian constantemente la naturaleza de la demanda en el sistema en cuanto al perfil de instrucción típico y el modelo de acceso de datos.

Así, el diseño de computadores es una forma de arte en constante evolución. Este libro intenta plantear los fundamentos en los que se basa esta forma de arte y dar una visión general de su estado actual.

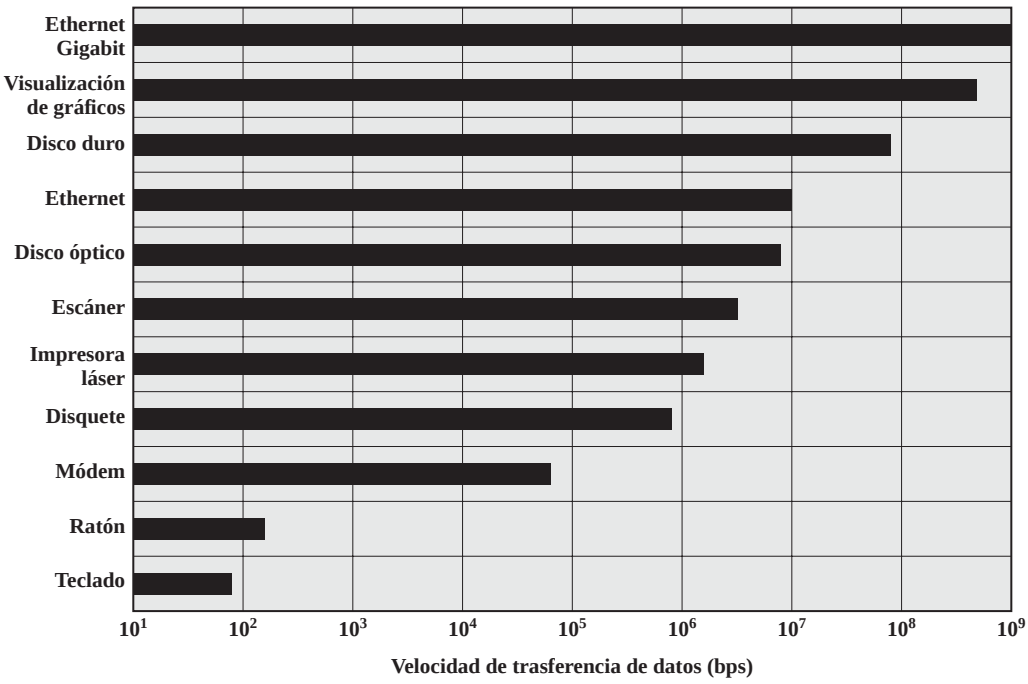


Figura 2.11. Velocidades de transmisión de datos en E/S típicas.

MEJORAS EN LA ORGANIZACIÓN Y ARQUITECTURA DE CHIPS

Además del reto de los diseñadores de equilibrar las prestaciones del procesador con la memoria principal y con otros componentes del computador, persiste la necesidad de aumentar la velocidad del procesador. Hay tres formas de conseguir incrementar la velocidad del procesador:

- Incrementando la velocidad del hardware del procesador: esto se consigue fundamentalmente disminuyendo el tamaño de las puertas lógicas del chip del procesador, de forma que se puedan encapsular más puertas, más cercanas y así incrementar la frecuencia del reloj. Con las puertas más juntas, el tiempo de propagación de las señales se reduce significativamente, haciendo posible un aumento de la velocidad del procesador. Un incremento en la velocidad del reloj implica que las operaciones individuales se ejecuten más rápidamente.
- Incrementando el tamaño y la velocidad de las cachés interpuestas entre el procesador y la memoria principal. En particular, dedicando una parte del chip del procesador a la caché, el tiempo de acceso de la caché disminuye considerablemente.
- Haciendo cambios en la organización y arquitectura del procesador de forma que se incremente la velocidad efectiva de la ejecución de una instrucción. Usualmente, esto implica utilizar paralelismo de una forma u otra.

Tradicionalmente, el factor dominante en la ganancia de prestaciones se debe al aumento de la velocidad del reloj y a la densidad de la lógica. En la Figura 2.12 se muestra esta tendencia en los procesadores de Intel. Sin embargo, al aumentar la velocidad de reloj y la densidad de la lógica, una serie de obstáculos se hacen más significativos [INTE04b]:

- **Potencia**: a medida que la densidad de la lógica y la velocidad del reloj de un chip aumentan, también lo hace la densidad de potencia (watios/cm^2). La dificultad en disipar el calor generado por la gran densidad y la alta velocidad en el chip se convierte en un problema de diseño serio ([GIBB04], [BORK03]).
- **Retardo RC**: la velocidad a la que los electrones pueden fluir en un chip entre transistores está limitada por la resistencia y capacidad de los hilos metálicos que los conectan; concretamente, el retardo aumenta al hacerlo el producto RC. Como los componentes del chip disminuyen de tamaño, los hilos que los interconectan son más finos, aumentando su resistencia. Además, los hilos están más juntos, aumentando la capacidad.
- **Latencia de memoria**: la velocidad de la memoria ralentiza la velocidad del procesador, como se ha visto anteriormente.

Por tanto, es necesario hacer más énfasis en la organización y arquitectura para mejorar las prestaciones. La Figura 2.12 destaca los principales cambios que se han hecho a lo largo de los años para aumentar el paralelismo y, por tanto, la eficiencia computacional de los procesadores. Estas técnicas se verán en capítulos posteriores de este libro.

Empezando por finales de los ochenta, y siguiendo quince años más, se utilizaban principalmente dos estrategias para aumentar las prestaciones, más allá de lo que se puede conseguir sencillamente incrementando la velocidad del reloj. Primero, hubo un incremento de la capacidad de la caché. Ahora hay usualmente dos o tres niveles de caché entre el procesador y la memoria principal. Al aumentar la densidad del chip, se ha incorporado más memoria caché en el chip, haciendo posible un acceso más

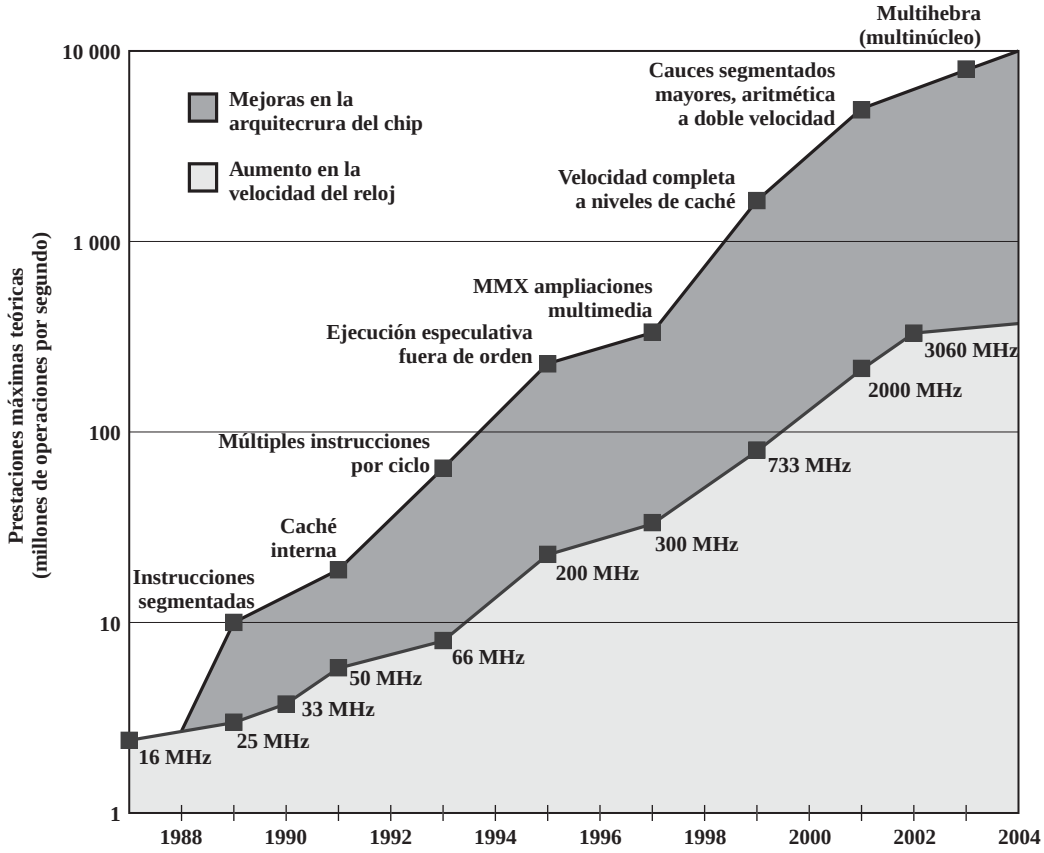


Figura 2.12. Prestaciones de los microprocesadores Intel [GIBB04].

rápido a la caché. Por ejemplo, el chip original Pentium cedía alrededor de un diez por ciento del área del chip para la caché. El más reciente Pentium 4 deja la mitad del área del chip para las cachés.

Segundo, la lógica de ejecución de una instrucción dentro de un procesador está siendo cada vez más compleja para posibilitar la ejecución paralela de instrucciones dentro del procesador. Dos enfoques de diseño notables han sido la segmentación de cauce y el superescalado. Una segmentación de cauce funciona como una línea de ensamblado de una planta industrial, haciendo posible que se ejecuten a la vez varias etapas de diferentes instrucciones a lo largo del cauce. La aproximación superescalar permite, en esencia, varios cauces dentro de un único procesador, de forma que instrucciones que no dependan unas de otras se puedan ejecutar en paralelo.

Ambas aproximaciones están alcanzando un punto que no da más de sí. La organización interna de los procesadores actuales es excesivamente compleja y se puede conseguir mucho paralelismo fuera del conjunto de instrucciones. Parece probable que otras mejoras significativas en esta dirección serán relativamente modestas [GIBB04]. Con tres niveles de caché en el chip del procesador, proporcionando cada nivel capacidad sustancial, también parece que los beneficios de la caché están alcanzando un límite.

Sin embargo, fiarse simplemente del aumento de la velocidad de reloj para aumentar las prestaciones lleva al problema de la disipación de potencia que ya hemos mencionado. Cuanto mayor sea la velocidad del reloj, mayor será la potencia que hay que disipar, y se pueden alcanzar algunos límites físicos básicos.

Con todas estas dificultades en mente, los diseñadores han recurrido a una aproximación fundamentalmente nueva: situar varios procesadores en el mismo chip, con una gran caché compartida. El uso de varios procesadores en el mismo chip, también conocido como varios núcleos, da potencia para incrementar las prestaciones sin aumentar la velocidad del reloj. Hay estudios que indican que, dentro de un procesador, el aumento de las prestaciones es aproximadamente proporcional a la raíz cuadrada del incremento de la complejidad [BORK03]. Pero si el software puede soportar el uso efectivo de varios procesadores, entonces duplicando el número de procesadores casi se duplican las prestaciones. Por tanto, la estrategia es usar dos simples procesadores en el chip en lugar de un procesador más complejo.

Además, con dos procesadores se pueden justificar grandes cachés. Esto es importante porque el consumo de potencia de la lógica de la memoria en un chip es mucho menor que la de la lógica de un procesador. Podemos esperar, en próximos años, que la mayoría de los nuevos chips de procesadores tendrán varios procesadores [ANTH04].

El primer chip multiprocesador comercialmente disponible se introdujo en el 2001, y era el chip POWER4 de IBM, que se basaba en la misma organización y arquitectura usada en los chips del PowerPC. La Figura 2.13, basada en una figura de [TEND02], da una vista en forma de niveles de la arquitectura del chip.

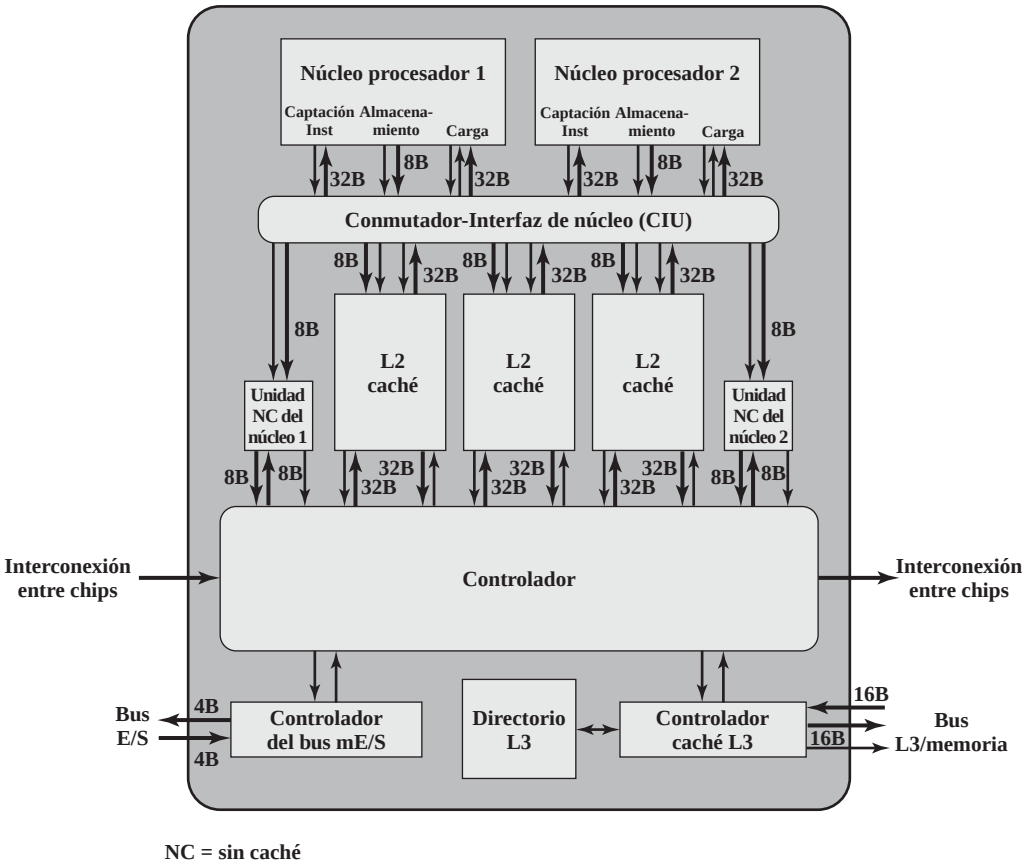


Figura 2.13. Organización del chip POWER4.

2.3. EVOLUCIÓN DEL PENTIUM Y DEL POWERPC

A través de este libro nos basamos en muchos ejemplos concretos de diseño e implementación de computadores para ilustrar conceptos y aclarar los compromisos. El libro se basa la mayor parte de las veces en ejemplos de dos familias de computadores: el Pentium de Intel y el PowerPC. El Pentium representa el resultado de décadas de esfuerzo de diseño en computadores de repertorio complejo de instrucciones (CISC). Incorpora los sofisticados principios de diseño que antes se encontraban solo en ordenadores grandes y supercomputadores, y es un excelente ejemplo de diseño CISC. El PowerPC es descendiente directo del primer sistema RISC, el IBM 801, y es uno de los sistemas basados en RISC más potentes y mejor diseñados del mercado.

En esta sección damos una breve visión general de ambos sistemas.

PENTIUM

Intel ha sido el número uno de los fabricantes de microprocesadores durante décadas, una posición que no parece probable que abandone. La evolución de su microprocesador más representativo es un buen indicador de la evolución de la tecnología de computadores en general.

La Tabla 2.6 muestra esta evolución. Es interesante que, como los microprocesadores han crecido rápidamente y de forma mucho más compleja, Intel ha sabido aprovechar la oportunidad. Intel solía desarrollar un microprocesador cada cuatro años. Pero Intel espera dejar sus rivales atrás reduciendo este tiempo de desarrollo a un año o dos, y así lo ha hecho con sus generaciones recientes de Pentium.

Merece la pena enumerar algunos de los rasgos más destacados de la evolución de los productos Intel:

- **8080:** es el primer microprocesador de propósito general del mundo. Era una máquina de ocho bits, con datos de memoria de ocho bits. El 8080 se uso en el primer computador personal, el Altair.
- **8086:** un circuito de 16 bits, mucho más potente. Además de un camino de datos más ancho y registros más grandes, el 8086 tenía una caché de instrucción, o cola, que precaptaba algunas instrucciones antes de ser ejecutadas. Una variante de este procesador, el 8088, se utilizó en el primer computador personal de IBM, asegurando el éxito de Intel.
- **80286:** esta ampliación del 8086 permitía direccionar una memoria de 16 MB en lugar de solo 1 MB.
- **80386:** fue la primera máquina de Intel con 32 bits, y constituyó una gran revisión del modelo anterior. Con una arquitectura de 32 bits, el 80386 rivalizaba en complejidad y potencia con los minicomputadores y grandes computadores introducidos en el mercado pocos años antes. Este fue el primer procesador de Intel que admitió multitarea, significando esto que podría ejecutar varios programas a la vez.
- **80486:** el 80486 introduce el uso de tecnología de caché mucho más sofisticada y potente e instrucciones de segmentación de cauce sofisticadas. El 80486 también tenía un coprocesador matemático, descargando a la CPU principal de las operaciones matemáticas complejas.
- **Pentium:** con el Pentium, Intel introduce el uso de técnicas superescalares que permiten que varias instrucciones se ejecuten en paralelo.

- **Pentium Pro:** el Pentium Pro continuó la tendencia iniciada con el Pentium hacia la organización superescalar, con el uso agresivo del renombrado de registros, predicción de ramificaciones, análisis del flujo de datos, y ejecución especulativa.
- **Pentium II:** en el Pentium II se incorporó la tecnología Intel MMX, que se diseñó específicamente para procesar de forma eficiente datos de vídeo, audio y gráficos.
- **Pentium III:** el Pentium III incorpora instrucciones adicionales en coma flotante para procesar software de gráficos 3D.
- **Pentium 4:** el Pentium 4 incluye coma flotante adicional y otras mejoras multimedia [7].
- **Itanium:** esta nueva generación de procesadores Intel usan una organización de 64 bits con arquitectura IA-64, que se verá con detalle en el Capítulo 15.
- **Itanium 2:** esta actualización del Itanium original incluye una serie de mejoras en el hardware para aumentar la velocidad.

POWERPC

En 1975, el proyecto minicomputador 801 de IBM fue el primero en muchos de los conceptos de arquitectura usados en sistemas RISC. El 801, junto con el procesador RISC I de Berkeley, inició el movimiento RISC. El 801, sin embargo, era simplemente un prototipo que intentaba demostrar nuevos conceptos de diseño. El éxito del proyecto 801 permitió a IBM desarrollar una estación de trabajo RISC comercial, el RT PC. El RT PC, introducido en 1986, adaptaba los conceptos arquitectónicos del 801 a un producto real. El RT PC no tuvo éxito comercial y tuvo muchos rivales con prestaciones comparables o mejores. En 1990, IBM produjo un tercer sistema que incorporaba las lecciones aprendidas con el 801 y el RT PC. El IBM RISC System/6000 era una máquina superescalar RISC comercializada como una estación de trabajo de altas prestaciones; poco después de su introducción, IBM comenzó a llamarla arquitectura POWER.

Como siguiente paso, IBM se alió con Motorola, que había desarrollado las series 68000 de microprocesadores, y Apple, que usaba el chip de Motorola en sus computadores Macintosh. El resultado es una serie de máquinas que implementan la arquitectura PowerPC. Esta arquitectura deriva de la arquitectura Power (Figura 2.13). Se hicieron cambios para añadir características clave que no estaban y para permitir una implementación más eficiente eliminando algunas instrucciones y relajando la especificación para eliminar casos especialmente problemáticos. La arquitectura PowerPC resultante es un sistema RISC superescalar.

Hasta ahora se han presentado cuatro miembros de la familia PowerPC (Tabla 2.7):

- **601:** el propósito del 601 era llevar la arquitectura PowerPC al mercado lo más rápidamente posible. El 601 es una máquina de 32 bits.
- **603:** pensado para computadores de aplicación final de sobremesa y portátiles. Es también una máquina de 32 bits, comparable en prestaciones con el 601, pero con coste más bajo e implementación más eficiente.

[7] Con el Pentium 4, Intel cambió de la numeración romana a la arábiga para designar los números de la serie.

Tabla 2.7. Resumen de los procesadores PowerPC.

	Año de comercialización	Velocidad de reloj	Cache L1	Cache L1	Número de transistores (10^6)
601	1993	50-120 MHz	—	—	2,8
603/603e	1994	100-300 MHz	16 KB instr 16 KB data	—	1,6-2,6
604/604e	1994	166-350 MHz	32 KB instr 32 KB data	—	3,6-5,1
740/750 (G3)	1997	200-366 MHz	32 KB instr 32 KB data	256 KB-1 MB	6,35
G4	1999	500 MHz	32 KB instr 32 KB data	256 KB-1 MB	—
G5	2003	2,5 GHz	64 KB instr 32 KB data	512 KB	58

- **604:** pensado para computadores de aplicación final de sobremesa y portátiles. De nuevo es una máquina de 32 bits pero utiliza técnicas de diseño superescalares, mucho más avanzadas para lograr mayores prestaciones.
- **620:** pensado para servidores de aplicación final. El primer miembro de la familia PowerPC que implementa una arquitectura completa de 64 bits, incluyendo registros y buses de datos de 64 bits.
- **740/750:** también conocido como procesador G3. Este procesador integra dos niveles de caché en el chip del procesador principal, ofreciendo mejoras significativas en las prestaciones sobre otras máquinas comparables con organización de cache fuera del chip.
- **G4:** este procesador continúa incrementando el paralelismo y la velocidad interna del chip del procesador.
- **G5:** introduce mejoras adicionales en el paralelismo, en la velocidad interna y constituye una organización de 64 bits.

2.4. LECTURAS Y SITIOS WEB RECOMENDADOS

Se puede encontrar una descripción de la serie IBM 7000 en [BELL71a]. Hay un buen tratamiento del IBM 360 en [SIEW82] y del PDP-8 y otras máquinas DEC en [BELL78a]. Estos tres libros también contienen numerosos y detallados ejemplos de otros computadores que abarcan la historia de los computadores de principios de los años ochenta. Un libro más reciente que incluye un excelente conjunto de estudios sobre máquinas históricas se encuentra en [BLAA97]. En [BETK97] se incluye una buena historia de los microprocesadores.

[OLUK96], [HAMM97] y [SAKA02] tratan sobre la utilización de varios procesadores en un solo chip.

Uno de los mejores tratamientos del Pentium se encuentra en [SHAN05]. La propia documentación de Intel también es buena [INTE04]. [BREY03] proporciona una buena visión de la línea de microprocesadores Intel. Sobre la arquitectura Itanium, *véase* [INTE02] y [EVAN03].

Véase [HUTC96], [SCHA97] y [BOHR98] para discusiones interesantes sobre la ley de Moore y sus consecuencias.

BELL71a BELL, C. y NEWELL, A.: *Computer Structures: Readings and Examples.* New York. McgGraw-Hill, 1971.

BELL78a BELL, C.; MUDGE, J. y McNamara, J.: *Computer Engineering: A DEC View of Hardware Systems Design.* Bedford, MA: Digital Press, 1978.

BETK97 BETKER, M.; FERNANDO, J. y Whalen, S.: «The History of the Microprocessor». *Bell Labs Technical Journal,* otoño, 1997.

BLAA97 BLAAUW, G. y BROOKS, F.: *Computer Architecture: Concepts and Evolution.* Reading, MA. Addison-Wesley, 1997.

BOHR98 BOHR, M.: «Silicon Trends and Limits for Advanced Microprocessors». *Communications of the ACM,* marzo, 1998.

BREY03 BREY, B.: *The Intel Microprocessors: 8086/8066, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro Processor, Pentium II, Pentium III, Pentium 4.* Upper Saddle River, NJ. Prentice Hall, 2000.

EVAN03 EVANS, J. y TRIMPER, G.: *Itanium Architecture for Programmers.* Upper Saddle River, NJ. Prentice Hall, 2003.

HAMM97 HAMMOND, L.; NAYFAY, B. y OLUKOTUN, K.: «A Single-Chip Multiprocessor». *Computer,* septiembre, 1997.

HUTC96 HUTCHESON, G. y HUTCHESON, J.: «Technology and Economics in the Semiconductor Industry». *Scientific American,* enero, 1996.

INTE02 Intel Corp.: *Intel Itanium Architecture Software Developer's Manual (3 volúmenes).* Aurora, CO, 2002.

INTE04a Intel Corp.: *IA-32 Intel Architecture Software Developer's Manual (3 volúmenes).* Aurora, CO, 2004.

OLUK96 OLUKOTUN, K., *et al.*: «The Case for a Single-Chip Multiprocessor». *Proceedings, Seventh International Conference on Architectural Support for Programming Languages and Operating Systems,* 1996.

SAKA02 SAKAI, S.: «CMP on SoC. architect's view». Proccedings. 15th international symposium on System Synthesis, 2002.

SCHA97 SCHALLER, R.: «Moore's Law: Past, Present, and Future». *IEEE Spectrum,* junio, 1997.

SHAN05 SHANLEY, T.: *Unabridged Pentium 4, The: IA32 Processor Genealogy.* Reading, MA. Addison-Wesley, 2005.

SIEW82 SIEWIOREK, D.; BELL, C. y NEWELL, A.: *Computer Structures: Principles and Examples.* New York. McGraw-Hill, 1982.



SITIOS WEB RECOMENDADOS

- **Intel developer's page:** página web de desarrollos de Intel; es un punto de inicio para acceder a la información sobre Pentium. También incluye la revista de tecnología de Intel (*Intel Technology Journal*).
- **PowerPC:** dos sitios web sobre el PowerPC, una de IBM y otra de la empresa Freescale Semiconductor, formalmente sector de semiconductores de Motorola.

- **Standard Performance Evaluation Corporation (empresa de estándares de evaluación de prestaciones):** SPEC es una organización ampliamente reconocida en la industria de los computadores por sus desarrollos en estándares de programas de pruebas (*benchmarks*) utilizados para medir y comparar las prestaciones de distintos computadores.
- **El Top500 de los supercomputadores:** contiene una breve descripción de la arquitectura y organización de los supercomputadores actuales, y además comparaciones.
- **Instituto Charles Babbage:** proporciona enlaces a numerosos sitios web sobre la historia de los computadores.

2.5. PALABRAS CLAVE, PREGUNTAS DE REPASO Y PROBLEMAS

PALABRAS CLAVE

acumulador (AC)	registro de instrucción (IR)	código de operación
unidad aritmético lógica (ALU)	conjunto de instrucciones	fabricante de equipos originales (OEM)
chip	circuito integrado (IC)	unidad de control de programa
canal de datos	memoria principal	contador de programa (PC)
fase de ejecución	registro de dirección de memoria (MAR)	programa cargado
fase de captación	registro buffer de memoria (MBR)	compatibilidad hacia arriba.
entrada/Salida	microprcesador	máquina von Neumann
registro buffer de instrucción (IR)	multiplexor	oblea
ciclo de instrucción		palabra

PREGUNTAS DE REPASO

2.1. ¿Qué es un computador de programa almacenado?
2.2. ¿Cuáles son los cuatro componentes principales de un computador de uso general?
2.3. A nivel de circuito integrado, ¿cuáles son los tres componentes principales de un computador?
2.4. Explicar la ley de Moore.
2.5. Enumerar las características clave de una familia de computadores.
2.6. ¿Cuál es la clave para distinguir las características de un microprocesador?

PROBLEMAS

2.1. Sean A = A(1), A(2)..., A(1000) y B = B(1), B(2)..., B(1000) dos vectores (unidimensionales) que comprenden 1000 números cada uno, que van a ser sumados para formar un vector C, tal que C(I) = A(I) + B(I), donde I = 1, 2..., 1000. Usando el conjunto de instrucciones IAS, escribir un programa para resolver este problema.

2.2. *a)* En el IAS, ¿cómo sería el código de la instrucción máquina para cargar el contenido de memoria de la dirección 2?

b) ¿Cuántos accesos a memoria tendría que hacer la CPU para completar esta instrucción durante el ciclo instrucción?

2.3. En el IAS, describir con palabras el proceso que tiene que seguir la CPU para leer un valor de memoria y escribir un valor en memoria, indicando cómo cambian MAR, MBR, el bus de direcciones, el bus de datos y el bus de control.

2.4. Dados los contenidos de memoria siguientes de un computador IAS:

Direcciones	Contenidos
08A	010FA210FB
08B	010FA0F08D
08C	020FA210FB

Mostrar el código del programa en lenguaje ensamblador, empezando en la dirección 08A. Explicar lo que hace el programa.

2.5. Indicar, en la Figura 2.3, el ancho en bits de cada camino de datos (por ejemplo, entre el AC y la ALU).

2.6. En el IBM 360 modelos 65 y 75, las direcciones están situadas en dos unidades de memoria principal separadas (por ejemplo, todas las palabras pares en una unidad y todas las impares en otra). ¿Cuál puede ser el objetivo de esta técnica?

2.7. En la Tabla 2.4 se puede ver que las prestaciones relativas del Modelo 75 del IBM 360 son cincuenta veces las del Modelo 30, mientras que el tiempo de ciclo de instrucción es solo cinco veces más rápido. ¿Cómo se explica esta discrepancia?

2.8. En la tienda de computadores de Miguel Ángel Valenzuela, escuchas a un cliente preguntando por el computador más rápido de la tienda que pueda comprar. Miguel Ángel Valenzuela le contesta, «estás mirando nuestros Macintosh. El Mac más rápido que tenemos tiene una velocidad de reloj de 1,2 Gigahercios. Si realmente quieres una máquina rápida, debes comprarte un Intel Pentium IV a 2,4 Gigahercios». ¿Es correcto lo que dice Miguel Ángel Valenzuela? ¿Qué debería decirle para ayudar a su cliente?

2.9. El ENIAC era una máquina decimal, donde un registro se representaba con un anillo de diez tubos de vacío. En un instante dado, solo un tubo de vacío estaba en estado ON, representando uno de 10 dígitos. Suponiendo que el ENIAC tuviera la capacidad de tener varios tubos de vacío en los estados ON y OFF simultáneamente, ¿por qué esta representación es un «despilfarro» y qué rango de valores enteros se podrían representar con diez tubos de vacío?

2.10. Un procesador está sincronizado por un reloj con una frecuencia constante f o, lo que es lo mismo, un tiempo de ciclo constante τ, donde $\tau = 1/f$. El tamaño de un programa se puede medir con el número de instrucciones máquina, o número de instrucciones I_c, que contiene el programa. Distintas instrucciones máquina consumen distintos ciclos de reloj. Un parámetro importante es el número medio de ciclos por instrucción CPI de un programa. El tiempo T que el procesador necesita para ejecutar un programa dado se puede expresar:

$$T = I_c \times CPI \times \tau$$

Esta fórmula se puede reescribir teniendo en cuenta que durante la ejecución de una instrucción, parte del trabajo lo realiza el procesador, y parte del tiempo se está transfiriendo a, o desde memoria, una palabra. En este último caso, el tiempo de transferencia depende del tiempo de ciclo de memoria, que puede ser mayor que el tiempo de ciclo del procesador. Se puede reescribir la ecuación anterior:

$$T = I_c \times [\, p \; + \; (m \times k)] \times \tau$$

donde p es el número de ciclos de procesador necesarios para decodificar y ejecutar la instrucción, m es el número de accesos a memoria, y k es el cociente entre el tiempo de ciclo de memoria y el tiempo de ciclo del procesador. Los cinco factores de prestaciones de la ecuación anterior (I_c, p, m, k, τ) dependen de los cuatro atributos del sistema: el diseño del conjunto de instrucciones (conocido como arquitectura del conjunto de instrucciones), de la tecnología del compilador (cómo de efectivo es el compilador produciendo un programa en lenguaje máquina a partir de un programa en un lenguaje de alto nivel), de la implementación del procesador, y de la jerarquía de memoria y de la caché. Hacer un matriz en la que una dimensión muestre los cinco factores de las prestaciones y la otra los cuatro atributos del sistema. Poner una X en cada celda en la que el atributo del sistema afecte al factor de prestaciones.

2.11. Una medida usual de las prestaciones de un procesador es la velocidad a la que ejecuta instrucciones, expresada en millones de instrucciones por segundo (MIPS). Expresar la velocidad en MIPS en función de la velocidad de reloj y *CPI*.

2.12. Los primeros ejemplos de diseños CISC y RISC son el VAX 11/780 y el IBM RS/6000, respectivamente. Utilizando un programa de pruebas típico, se obtienen los siguientes resultados:

Procesador	Frecuencia de reloj	Tasa de ejecución de instrucciones	Tiempo de la CPU
VAX 11/780	5 MHz	1 MIPS	12 *x* segundos
IBM RS/6000	25 MHz	18 MIPS	*x* segundos

La última columna muestra que el VAX necesita doce veces más tiempo de CPU que el IBM.

a) ¿Cuál es el valor relativo del número de instrucciones del código máquina para este programa de prueba en las dos máquinas?

b) ¿Cuáles son los valores CPI de las dos máquinas?

2.13. Un programa de prueba se está ejecutando en un procesador de 40 MHz. El código objeto consta de 100 000 instrucciones, con el siguiente conjunto de instrucciones y ciclo de reloj:

Tipo de Instrucción	Contador de instrucciones	Ciclo de reloj
Aritméticos enteros	45 000	1
Datos transferidos	32 000	2
Punto flotante	15 000	2
Control transferidos	8 000	2

Determinar el CPI efectivo, la velocidad en MIPS, y el tiempo de ejecución para este programa.

2.14. Para obtener una comparación fiable de las prestaciones de varios computadores, es preferible ejecutar diferentes programas de pruebas en cada máquina, y entonces promediar los resultados. Por ejemplo, con m programas diferentes, se puede calcular una simple media aritmética como sigue:

$$R_a = \frac{1}{m}\sum_{i=1}^{m} R_i$$

Donde R_i es la velocidad en MIPS del i-ésimo programa de prueba. Una alternativa es calcular la media armónica:

$$R_h = \frac{m}{\sum_{i=1}^{m} \frac{1}{R_i}}$$

a) Comentar las ventajas y desventajas de ambos métodos. Consejo: considerar el tiempo de ejecución medio (en microsegundos) por instrucción, para el programa i, $T_i = 1/R_i$.

b) Se ejecutan cuatro programas de prueba en tres computadores con los siguientes resultados:

	Computador A	**Computador B**	**Computador C**
Programa 1	1	10	20
Programa 2	1 000	100	20
Programa 3	500	1 000	50
Programa 4	100	800	100

La tabla muestra el tiempo de ejecución en segundos, al ejecutar 100 000 000 de instrucciones en cada uno de los cuatro programas. Calcular las medias aritmética y armónica, y establecer un *ranking* basándose en la media armónica.

PARTE 2

EL COMPUTADOR

CUESTIONES A TRATAR EN LA SEGUNDA PARTE

Un computador consta de procesador, memoria, E/S, y las interconexiones entre estos componentes principales. Con la excepción del procesador, que es suficientemente complejo para dedicar a su estudio toda la Parte Tres, la Parte Dos examina cada uno de estos componentes con detalle.

ESQUEMA DE LA SEGUNDA PARTE

CAPÍTULO 3. PERSPECTIVA DE ALTO NIVEL DEL FUNCIONAMIENTO Y DE LAS INTERCONEXIONES DEL COMPUTADOR

A alto nivel, un computador consta de procesador, memoria, y unidades de E/S. El sistema funciona intercambiando señales de datos y control entre sus componentes. Para permitir dicho intercambio, los componentes deben estar interconectados. El Capítulo 3 comienza con un breve examen de los componentes del computador y sus requisitos de entrada/salida. Después, el capítulo analiza los aspectos clave que afectan el diseño de la estructura de interconexión, especialmente los relativos a las interrupciones. La mayor parte del capítulo se centra en el estudio de la aproximación más usual para interconectar los componentes del computador: las estructuras de buses.

CAPÍTULO 4. MEMORIA CACHÉ

La memoria del computador presenta una amplia variedad de tipos, tecnologías, organizaciones, prestaciones y costes. Un computador típico posee una jerarquía de subsistemas de memoria, incluyendo tanto memoria interna (el procesador puede acceder a ella directamente) como externa (el procesador accede a ella a través de una unidad de E/S). El Capítulo 4 comienza con una revisión de esta jerarquía. A continuación, el capítulo aborda el diseño de la memoria caché, incluyendo las cachés separadas para datos e instrucciones y las cachés de dos niveles.

CAPÍTULO 5. MEMORIA INTERNA

El diseño del sistema de memoria principal constituye una guerra sin cuartel entre tres requisitos de diseño: capacidad de almacenamiento elevada, tiempo de acceso reducido, y bajo coste. A medida que evoluciona la tecnología de las memorias, cada uno de estos aspectos cambia de forma que las decisiones en el diseño de la organización de la memoria principal debe reconsiderarse a medida que surgen nuevas implementaciones. El Capítulo 5 se centra en el diseño de la memoria interna. En primer lugar se examinan la naturaleza y la organización de la memoria principal de semiconductores. Después, se exploran los avances más recientes en el ámbito de la organización de la memoria DRAM.

CAPÍTULO 6. MEMORIA EXTERNA

Para disponer de una capacidad de almacenamiento verdaderamente elevada y conseguir un almacenamiento más duradero que el que proporciona la memoria principal, se necesita la memoria externa. El tipo de memoria externa más ampliamente utilizado es el disco magnético, de forma que la mayor parte del Capítulo 5 se centra en dicho tópico. En primer lugar, se analizan la tecnología de los discos magnéticos y las consideraciones de diseño. A continuación se estudia el uso de la organización RAID para mejorar las prestaciones de la memoria de disco. El Capítulo 6 también examina el almacenamiento óptico y las cintas magnéticas.

CAPÍTULO 7. ENTRADA/SALIDA

Las unidades de E/S están conectadas con el procesador y la memoria principal, y cada una de ellas controla uno o más dispositivos periféricos. El Capítulo 7 se dedica a distintos aspectos de la organización de E/S. Se trata de un área compleja, y mucho menos comprendida que otras áreas del diseño de un computador en lo que se refiere a cómo satisfacer los niveles de prestaciones exigidos. El Capítulo 7 examina los mecanismos a través de los cuales un módulo de E/S interactúa con el resto del computador utilizando las técnicas de E/S programada, E/S por interrupciones, y acceso directo a memoria (DMA). También se describe la interfaz entre los módulos de E/S y los dispositivos externos.

CAPÍTULO 8. SISTEMAS OPERATIVOS

Un examen detallado de los sistemas operativos está fuera del ámbito de este libro. No obstante, es importante entender sus funciones básicas y cómo aprovecha el hardware para proporcionar el nivel de prestaciones deseado. El Capítulo 8 describe los principios básicos de los sistemas operativos, y discute las características de diseño específicas del hardware del computador orientadas a dar soporte al sistema operativo. El capítulo comienza con una breve introducción histórica, que sirve para identificar los principales tipos de sistemas operativos y presentar las razones por las que se utilizan. A continuación se explica la multiprogramación analizando las funciones de planificación (*scheduling*) a corto y largo plazo. Para terminar, se estudia la gestión de memoria incluyendo una discusión de los conceptos de segmentación, paginación, y memoria virtual.

CAPÍTULO 3

Perspectiva de alto nivel del funcionamiento y de las interconexiones del computador

3.1. Componentes del computador

3.2. Funcionamiento del computador

Los ciclos de captación y ejecución
Interrupciones
Funcionamiento de las E/S

3.3. Estructuras de interconexión

3.4. Interconexión con buses

Estructura del bus
Jerarquías de buses múltiples
Elementos de diseño de un bus

3.5. PCI

Estructura del bus
Órdenes del PCI
Transferencias de datos
Arbitraje

3.6. Lecturas y sitios web recomendados

3.7. Palabras clave, cuestiones y problemas

Palabras clave
Cuestiones
Problemas

Apéndice 3A Diagramas de tiempo

PUNTOS CLAVE

- Un ciclo de instrucción consiste en la captación de la instrucción, seguida de ninguno o varios accesos a operandos, ninguno o varios almacenamientos de operandos, y la comprobación de las interrupciones (si estas están habilitadas).
- Los principales componentes del computador (procesador, memoria principal, módulos de E/S) necesitan estar interconectados para intercambiar datos y señales de control. El medio de interconexión más popular es un bus compartido constituido por un conjunto de líneas. En los computadores actuales, es usual utilizar una jerarquía de buses para mejorar el nivel de prestaciones.
- Los aspectos clave del diseño de los buses son el arbitraje (si el permiso para enviar las señales a través de las líneas del bus se controla de forma centralizada o distribuida); la temporización (sin las señales del bus se sincronizan mediante un reloj central o se envían asíncronamente); y la anchura (número de líneas de dirección y datos).

A alto nivel, un computador está constituido por CPU (unidad central de procesamiento), memoria, y unidades de E/S, con uno o varios módulos de cada tipo. Estos componentes se interconectan de modo que se pueda llevar a cabo la función básica del computador, que es ejecutar programas. Así, a este nivel, se puede describir un computador (1) mediante el comportamiento de cada uno de sus componentes, es decir, mediante los datos y las señales de control que un componente intercambia con los otros, y (2) mediante la estructura de interconexión y los controles necesarios para gestionar el uso de dicha estructura.

Esta visión de alto nivel en términos de estructura y funcionamiento es importante debido a su capacidad explicativa de cara a la comprensión de la naturaleza del computador. Igualmente importante es su utilidad para entender los cada vez más complejos problemas de evaluación de prestaciones. Entender la estructura y el funcionamiento a alto nivel permite hacerse una idea de los cuellos de botella del sistema, los caminos alternativos, la importancia de los fallos del sistema si hay un componente defectuoso, y la facilidad con que se pueden mejorar las prestaciones. En muchos casos, los requisitos de mayor potencia y capacidad de funcionamiento tolerante a fallos se satisfacen mediante cambios en el diseño más que con un incremento en la velocidad y en la fiabilidad de los componentes individuales.

Este capítulo se centra en las estructuras básicas utilizadas para la interconexión de los componentes del computador. A modo de revisión, el capítulo comienza con un somero examen de los componentes básicos y sus necesidades de interconexión. Después, se revisan los aspectos funcionales.

Más adelante estaremos preparados para analizar el uso de los buses que interconectan los componentes del sistema.

3.1. COMPONENTES DEL COMPUTADOR

Como se discutió en el Capítulo 2, virtualmente todos los computadores actuales se han diseñado basándose en los conceptos desarrollados por John von Neumann en el Instituto de Estudios

Avanzados (*Institute for Advances Studies*) de Priceton. Tal diseño se conoce con el nombre de *Arquitectura de von Neumann* y se basa en tres conceptos clave:

- Los datos y las instrucciones se almacenan en una sola memoria de lectura-escritura.
- Los contenidos de esta memoria se direccionan indicando su posición, sin considerar el tipo de dato contenido en la misma.
- La ejecución se produce siguiendo una secuencia de instrucción tras instrucción (a no ser que dicha secuencia se modifique explícitamente).

Las razones que hay detrás de estos conceptos se discutieron en el Capítulo 2 pero merecen ser resumidas aquí. Hay un conjunto pequeño de componentes lógicos básicos que pueden combinarse de formas diferentes para almacenar datos binarios y realizar las operaciones aritméticas y lógicas con esos datos. Si se desea realizar un cálculo concreto, es posible utilizar una configuración de componentes lógicos diseñada específicamente para dicho cálculo. Se puede pensar en el proceso de conexión de los diversos componentes para obtener la configuración deseada como si se tratase de una forma de programación. El «programa» resultante es hardware y se denomina *programa cableado (hardwired program).*

Considérese ahora la siguiente alternativa. Se construye una configuración de uso general de funciones lógicas y aritméticas. Este hardware realizará funciones diferentes según las señales de control aplicadas. En el caso del hardware específico, el sistema acepta datos y produce resultados (Figura 3.1a). Con el hardware de uso general, el sistema acepta datos y señales de control y produce

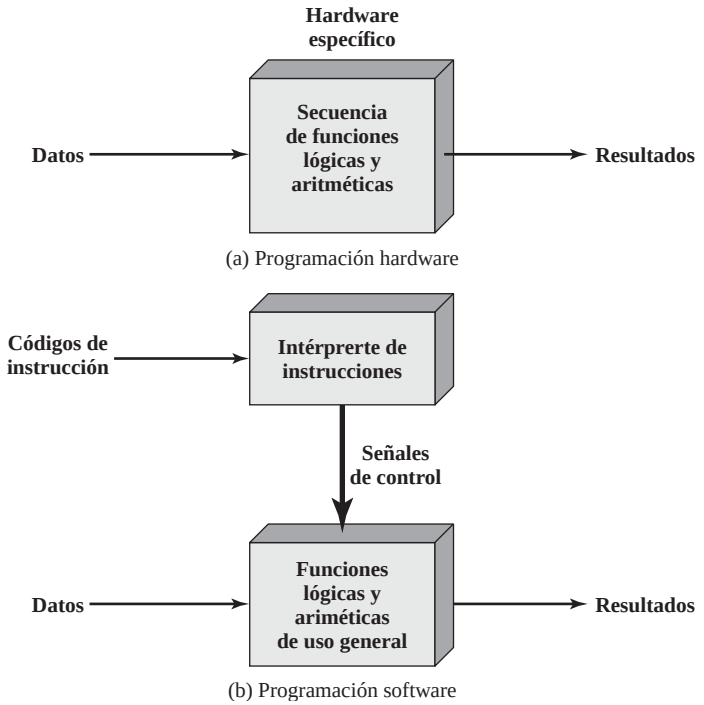


Figura 3.1. Alternativas hardware y software.

resultados. Así, en lugar de reconfigurar el hardware para cada nuevo programa, el programador simplemente necesita proporcionar un nuevo conjunto de señales de control.

¿Cómo se suministran las señales de control? La respuesta es simple pero ingeniosa. El programa es realmente una secuencia de pasos. En cada paso, se realiza una operación aritmética o lógica con ciertos datos. Para cada paso, se necesita un nuevo conjunto de señales de control. La solución consiste en asociar un código específico a cada posible conjunto de señales de control, y añadir al hardware de uso general una parte encargada de generar las señales de control a partir del código (Figura 3.1b).

Programar es ahora mucho más fácil. En lugar de tener que reconfigurar el hardware para cada programa, todo lo que se necesita es proporcionar una nueva secuencia de códigos. Cada código es, de hecho, una instrucción, y una parte del hardware interpreta cada instrucción y genera las señales de control. Para distinguir este nuevo método de programación, una secuencia de códigos o instrucciones se denomina *software*.

La Figura 3.1b muestra dos componentes esenciales del sistema: un intérprete de instrucciones y un módulo de uso general para las funciones aritméticas y lógicas. Estos dos elementos constituyen la CPU. Se requieren varios componentes adicionales para que el computador pueda funcionar. Los datos y las instrucciones deben introducirse en el sistema. Para eso se necesita algún tipo de módulo de entrada. Este módulo contiene los componentes básicos para captar datos e instrucciones en cierto formato y traducirlos al formato de señales que utiliza el sistema. Se necesita un medio para proporcionar los resultados, el módulo de salida. Globalmente, estos módulos se conocen con el nombre de *componentes de E/S (Entrada/Salida)*.

Se necesita un componente más. Un dispositivo de entrada proporcionará los datos y las instrucciones secuencialmente, uno tras otro. Pero un programa no siempre ejecuta las instrucciones según la misma secuencia; puede saltarse ciertas instrucciones (por ejemplo, al ejecutar la instrucción de salto IAS). De la misma forma, las operaciones con datos pueden necesitar acceder a más de un operando y según una secuencia determinada. Por ello, debe existir un sitio para almacenar temporalmente tanto las instrucciones como los datos. Ese módulo se llama *memoria,* o *memoria principal* para distinguirlo de los periféricos y la memoria externa. Von Neumann indicó que la misma memoria podría ser usada tanto para las instrucciones como para los datos.

La Figura 3.2 muestra estos componentes de alto nivel y sugiere las interacciones entre ellos. Típicamente, la CPU se encarga del control. Intercambia datos con la memoria. Para ello, usualmente utiliza dos registros internos (en la CPU): un registro de direcciones de memoria (MAR, *Memory Address Register*), que especifica la dirección en memoria de la próxima lectura o escritura, y un registro para datos de memoria (MBR, *Memory Buffer Register*), que contiene el dato que se va a escribir en memoria o donde se escribe el dato que se va a leer de memoria. Igualmente, un registro de direcciones de E/S (E/SAR, *E/S Address Register*) especifica un dispositivo de E/S. Un registro para datos de E/S (E/S BR, *E/S Buffer Register*) se utiliza para intercambiar datos entre un módulo de E/S y la CPU.

Un módulo de memoria consta de un conjunto de posiciones, designadas por direcciones numeradas secuencialmente. Cada posición contiene un número binario que puede ser interpretado como una instrucción o como un dato. Un módulo de E/S transfiere datos desde los dispositivos externos a la CPU y a la memoria, y viceversa. Contiene los registros (buffers) internos para almacenar los datos temporalmente, hasta que puedan enviarse.

Tras esta breve descripción de los principales componentes, revisaremos cómo funcionan estos cuando ejecutan programas.

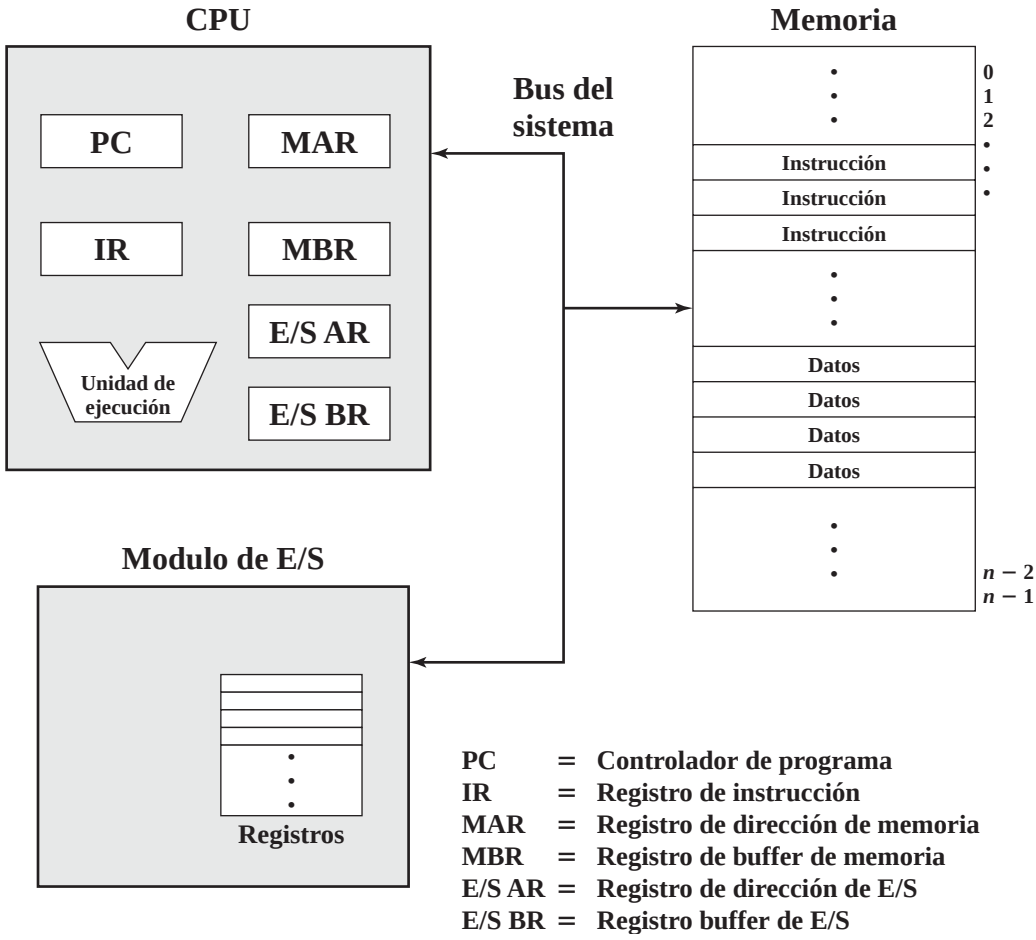


Figura 3.2. Componentes del computador: esquema de dos niveles.

3.2. FUNCIONAMIENTO DEL COMPUTADOR

La función básica que realiza un computador es la ejecución de un programa, constituido por un conjunto de instrucciones almacenadas en memoria. El procesador es precisamente el que se encarga de ejecutar las instrucciones especificadas en el programa. Esta sección proporciona una revisión de los aspectos clave en la ejecución de un programa, que en su forma más simple consta de dos etapas: El procesador lee (*capta*) la instrucción de memoria, y la ejecuta. La ejecución del programa consiste en la repetición del proceso de captación de instrucción y ejecución de instrucción. Por supuesto, la ejecución de la instrucción puede a su vez estar compuesta de cierto número de pasos (obsérvese, por ejemplo, la parte inferior de la Figura 2.4).

El procesamiento que requiere una instrucción se denomina *ciclo de instrucción*. Se representa en la Figura 3.3 utilizando la descripción simplificada de dos etapas explicada más arriba. Los dos pasos se denotan como *ciclo de captación* y *ciclo de ejecución*. La ejecución del programa se para solo si

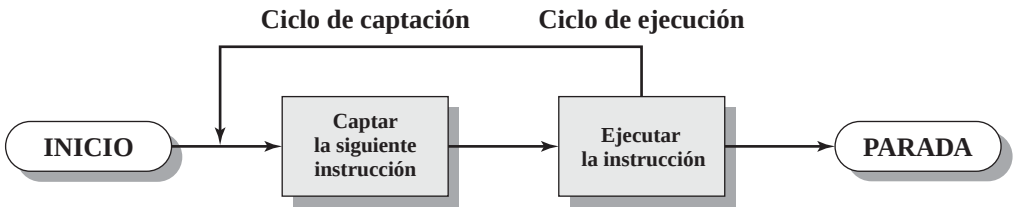


Figura 3.3. Ciclo de instrucción básico.

la máquina se desconecta, se produce algún tipo de error *irrecuperable* o ejecuta una instrucción del programa que detiene al computador.

LOS CICLOS DE CAPTACIÓN Y EJECUCIÓN

Al comienzo de cada ciclo de instrucción, la CPU capta una instrucción de memoria. En una CPU típica, se utiliza un registro llamado contador de programa (PC, *Program Counter*) para seguir la pista de la instrucción que debe captase a continuación. A no ser que se indique otra cosa, la CPU siempre incrementa el PC después de captar cada instrucción, de forma que captará la siguiente instrucción de la secuencia (es decir, la instrucción situada en la siguiente dirección de memoria). Considérese, por ejemplo, un computador en el que cada instrucción ocupa una palabra de memoria de 16 bits. Se supone que el contador de programa almacena el valor 300. La CPU captará la próxima instrucción almacenada en la posición 300. En los siguientes ciclos de instrucción, captará las instrucciones almacenadas en las posiciones 301, 302, 303, y así sucesivamente. Esta secuencia se puede alterar, como se explicará en breve.

La instrucción captada se almacena en un registro de la CPU conocido como registro de instrucción (IR, *Instruction Register*). La instrucción se escribe utilizando un código binario que especifica la acción que debe realizar la CPU. La CPU interpreta la instrucción y lleva a cabo la acción requerida. En general, esta puede ser de cuatro tipos:

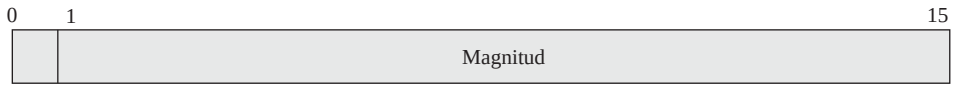
- **Procesador-Memoria:** deben transferirse datos desde la CPU a la memoria, o desde la memoria a la CPU.
- **Procesador-E/S:** deben transferirse datos a o desde el exterior mediante transferencias entre la CPU y un módulo de E/S.
- **Procesamiento de Datos:** la CPU ha de realizar alguna operación aritmética o lógica con los datos.
- **Control:** una instrucción puede especificar que la secuencia de ejecución se altere (como la instrucción de salto IAS, Tabla 2.1). Por ejemplo, la CPU capta una instrucción de la posición 149 que especifica que la siguiente instrucción debe captarse de la posición 182. La CPU registrará este hecho poniendo en el contador de programa 182. Así, en el próximo ciclo de captación, la instrucción se cargará desde la posición 182 en lugar de desde la posición 150.

La ejecución de una instrucción puede implicar una combinación de estas acciones.

Considérese un ejemplo sencillo utilizando una máquina hipotética que incluye las caraterísticas enumeradas en la Figura 3.4. El procesador posee un único registro de datos llamado



(a) Formato de instrucción



(b) Formato de enteros

Contador de programa (PC) = Dirección de instrucción
Registro de instrucción (IR) = Instrucción en ejecución
Acumulador (AC) = Almacenamiento temporal

(c) Registros internos de la CPU

0001 = Cargar AC desde memoria
0010 = Almacenar AC en memoria
0101 = Sumar a AC un dato de memoria

(d) Lista parcial de Codops («códigos de operación»)

Figura 3.4. Características de una máquina hipotética.

acumulador (AC). Tanto las instrucciones como los datos son de 16 bits. Así, es conveniente organizar la memoria utilizando posiciones de 16 bits, o palabras. El formato de instrucción indica que puede haber $2^4 = 16$ códigos de operación (codops) diferentes, y se pueden direccionar directamente hasta $2^{12} = 4096$ (4K) palabras de memoria.

La Figura 3.5 ilustra la ejecución de una parte de un programa, mostrando las partes relevantes de la memoria y los registros de la CPU [1]. El fragmento de programa suma el contenido de la palabra de memoria en la dirección 940 con el contenido de la palabra de memoria en la dirección 941 y almacena el resultado en esta última posición. Se requieren tres instrucciones, que consumen tres ciclos de captación y tres de ejecución:

1. El contador de programa (PC) contiene el valor 300, la dirección de la primera instrucción. Esta instrucción (el valor hexadecimal 1940) se carga en el registro de instrucción (IR). Obsérvese que este proceso implicaría el uso del registro de dirección de memoria (MAR) y el registro de datos de memoria (MBR). Por simplicidad, se han ignorado estos registros intermedios.
2. Los primeros cuatro bits de IR (primer dígito hexadecimal) indican que el acumulador (AC) se va a cargar. Los restantes 12 bits (tres dígitos hexadecimales) especifican la dirección (940) que se va a cargar.
3. El registro PC se incrementa, y se capta la siguiente instrucción (5941) desde la dirección 301.
4. El contenido anterior de AC y el de la posición de memoria 941 se suman, y el resultado se almacena en AC.

[1] Se utiliza notación hexadecimal, en la que cada dígito representa cuatro bits. Esta es la notación más conveniente para representar los contenidos de la memoria y los registros cuando la longitud de palabra es múltiplo de 4 (por ejemplo, 8, 16, o 32). Para los lectores no familiarizados con este notación, se resume en el apéndice del Capítulo 8.

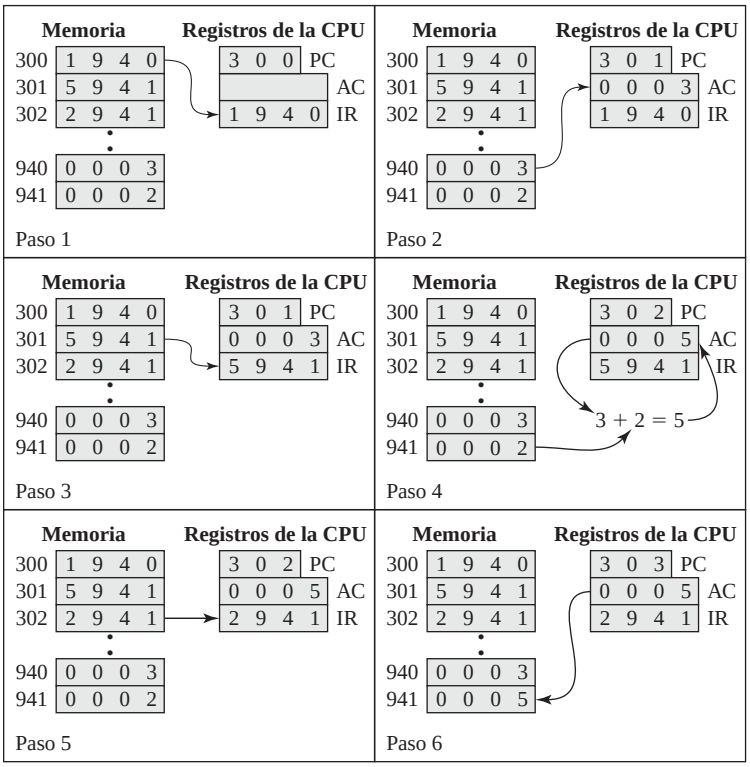


Figura 3.5. Ejemplo de ejecución de un programa (contenidos de la memoria y de los registros en hexadecimal).

5. El registro PC se incrementa, y se capta la siguiente instrucción (294) desde la posición 302.
6. El contenido de AC se almacena en la posición 941.

En este ejemplo, se necesitan tres ciclos de instrucción, cada uno con un ciclo de captación y un ciclo de ejecución, para sumar el contenido de la posición 940 y el contenido de la 941. Con un conjunto de instrucciones más complejo, se hubieran necesitado menos ciclos. Así, en algunos procesadores más antiguos se incluían instrucciones con más de una dirección. De esta forma, el ciclo de ejecución de una instrucción generaría más de una referencia a memoria. Además, en lugar de referencias a memoria, una instrucción puede especificar una operación de E/S.

Por ejemplo, la instrucción del PDP-11 expresada simbólicamente como ADD B,A almacena la suma de los contenidos de las posiciones B y A en la posición de memoria A. Se produce un solo ciclo de instrucción con los siguientes pasos:

- Se capta la instrucción ADD.
- El contenido de la posición de memoria A se lee y pasa al procesador
- El contenido de la posición de memoria B se lee y pasa al procesador. Para que el contenido de A no se pierda, el procesador debe tener al menos dos registros para almacenar valores de memoria, en lugar de un solo acumulador.

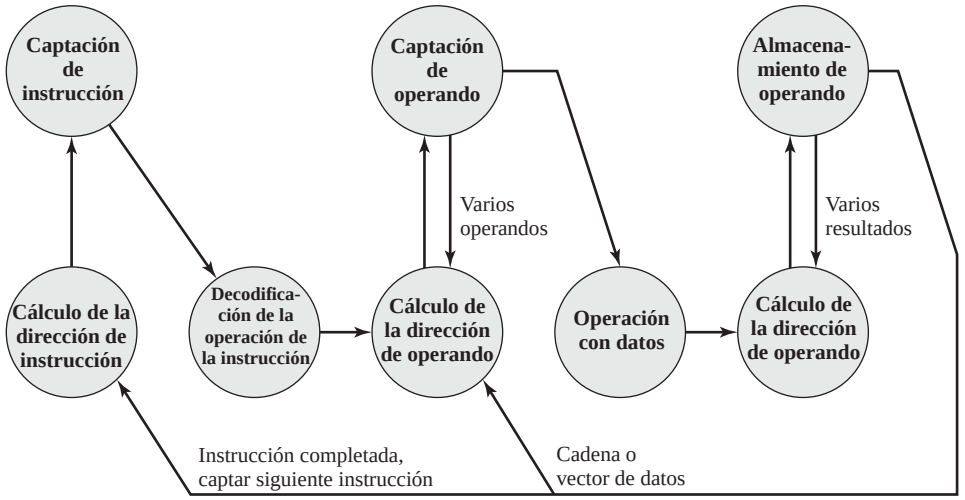


Figura 3.6. Diagrama de estados del ciclo de intrucción.

- Se suman los dos valores.
- El procesador escribe el resultado en la posición de memoria A.

Así, el ciclo de ejecución de una instrucción particular puede ocasionar más de una referencia a memoria. Además, en lugar de referencias a memoria, una instrucción puede especificar una operación de E/S. Con estas consideraciones adicionales en mente, la Figura 3.6 proporciona una visión más detallada del ciclo de instrucción básico de la Figura 3.3. La figura tiene la forma de un diagrama de estados. Para un ciclo de instrucción dado, algunos estados pueden no darse y otros pueden visitarse más de una vez. Los estados se describen a continuación:

- **Cálculo de la dirección de la instrucción (IAC, *Instruction Address Calculation*):** determina la dirección de la siguiente instrucción a ejecutar. Normalmente, esto implica añadir un número fijo a la dirección de la instrucción previa. Por ejemplo, si las instrucciones tienen un tamaño de 16 bits y la memoria se organiza en palabras de 16 bits, se suma 1 a la dirección previa. En cambio, si la memoria se organiza en bytes (8 bits) direccionables individualmente, entonces hay que sumar 2 a la dirección previa.
- **Captación de instrucción (if, *Instruction Fetch*):** la CPU lee la instrucción desde su posición en memoria.
- **Decodificación de la operación indicada en la instrucción (IOD, *Instruction Operation Decoding*):** analiza la instrucción para determinar el tipo de operación a realizar y el (los) operando(s) a utilizar.
- **Cálculo de la dirección del operando (OAC, *Operand Address Calculation*):** si la instrucción implica una referencia a un operando en memoria o disponible mediante E/S, determina la dirección del operando.
- **Captación de operando (OF, *Operand Fetch*):** capta el operando desde memoria o se lee desde el dispositivo de E/S.

- **Operación con los datos (DO, *Data Operation*):** realiza la operación indicada en la instrucción.
- **Almacenamiento de operando (OS, *Operand Store*):** escribe el resultado en memoria o lo saca a través de un dispositivo de E/S.

Los estados en la parte superior de la Figura 3.6 ocasionan intercambios entre la CPU y la memoria o un módulo de E/S. Los estados en la parte inferior del diagrama solo ocasionan operaciones internas a la CPU. El estado oac aparece dos veces, puesto que una instrucción puede ocasionar una lectura, una escritura, o ambas cosas. No obstante, la acción realizada en ese estado es la misma en ambos casos, y por eso sólo se necesita un único identificador de estado.

Obsérvese además que en el diagrama se considera la posibilidad de múltiples operandos y múltiples resultados puesto que se necesitan en algunas instrucciones de ciertas máquinas. Por ejemplo, la instrucción ADD A,B del PDP-11 da lugar a la siguiente secuencia de estados: iac,if,iod,oac,of,oac, of,do,oac,os.

Por último, en algunas máquinas, con una única instrucción se puede especificar una operación a realizar con un vector (matriz unidimensional) de números o con una cadena (matriz unidimensional) de caracteres. Como indica la Figura 3.6, esto implicaría una repetición de estados de captación y/o almacenamiento de operando.

INTERRUPCIONES

Prácticamente todos los computadores disponen de un mecanismo mediante el que otros módulos (E/S, memoria) pueden interrumpir el procesamiento normal de la CPU. La Tabla 3.1 enumera las clases de interrupciones más comunes. La naturaleza específica de estas interrupciones se examina en este libro más tarde, especialmente en los Capítulos 7 y 12. Sin embargo, necesitamos introducir el concepto ahora para comprender más claramente la esencia del ciclo de instrucción y los efectos de las interrupciones en la estructura de interconexión. En este momento, el lector no necesita conocer

Tabla 3.1. Clases de interrupciones.

Programa	Generadas por alguna condición que se produce como resultado de la ejecución de una instrucción, tal como desbordamiento aritmético (*overflow*), división por cero, intento de ejecutar una instrucción máquina inexistente e intento de acceder fuera del espacio de memoria permitido para el usuario.
Temporización	Generadas por un temporizador interno al procesador. Esto permite al sistema operativo realizar ciertas funciones de manera regular.
E/S	Generadas por un controlador de E/S, para indicar la finalización sin problemas de una operación o para avisar de ciertas condiciones de error.
Fallo de hardware	Generadas por un fallo tal como la falta de potencia de alimentación o un error de paridad en la memoria.

los detalles de la generación y el procesamiento de las interrupciones, sino solamente concentrarse en la comunicación entre módulos que resultan de las interrupciones.

En primer lugar las interrupciones proporcionan una forma de mejorar la eficiencia del procesador. Por ejemplo, la mayoría de los dispositivos externos son mucho más lentos que el procesador. Supóngase que el procesador está transfiriendo datos a una impresora utilizando el esquema del ciclo de instrucción de la Figura 3.3. Después de cada operación de escritura, el procesador tendrá que parar y permanecer ocioso hasta que la impresora complete la escritura. La longitud de esta pausa puede ser del orden de muchos cientos o incluso miles de ciclos de instrucción que no implican acceso a memoria. Claramente, esto supone un derroche en el uso del procesador.

La Figura 3.7a ilustra la situación del ejemplo referido en el párrafo precedente. El programa de usuario realiza una serie de llamadas de escritura (WRITE) entremezcladas con el procesamiento. Los segmentos de código 1, 2, y 3 corresponden a secuencias de instrucciones que no ocasionan operaciones de E/S. Las llamadas de escritura (WRITE) correponden a llamadas a un programa de E/S que es una de las utilidades del sistema operativo y que se encarga de la operación de E/S considerada. El programa de E/S está constituido por tres secciones:

- Una secuencia de instrucciones, rotulada con 4 en la figura, de preparación para la operación de E/S a realizar. Esto puede implicar la copia del dato que se va a proporcionar en un registro intermedio (buffer) especial, y preparar los parámetros de control del dispositivo de E/S.
- La orden de E/S propiamente dicha. Si no se utilizan interrupciones, una vez que se ejecuta esta orden, el programa debe esperar a que el dispositivo de E/S complete la operación solicitada. El programa esperaría comprobando repetidamente una condición que indique si se ha realizado la operación de E/S.
- Una secuencia de instrucciones, rotulada con 5 en la figura, que terminan la operación de E/S. Estas pueden incluir la activación de un indicador (*flag*) que señale si la operación se ha completado correctamente o con errores.

Debido a que la operación de E/S puede necesitar un tiempo relativamente largo, el programa de E/S debe detenerse a esperar que concluya dicha operación; por consiguiente, el programa de usuario estará parado en las llamadas de escritura (WRITE) durante un periodo de tiempo considerable.

Las Interrupciones y el ciclo de instrucción. Con el uso de interrupciones, el procesador puede dedicarse a ejecutar otras instrucciones mientras una operación de E/S está en curso. Considérese el flujo de control de la Figura 3.7b. Como antes, el programa de usuario llega a un punto en el que realiza una llamada al sistema para realizar una escritura (WRITE). El programa de E/S al que se llama en este caso está constituido solo por el código de preparación y la orden de E/S propiamente dicha. Después de que estas pocas instrucciones se hayan ejecutado, el control se devuelve al programa de usuario. Mientras tanto, el dispositivo externo está ocupado aceptando el dato de la memoria del computador e imprimiéndolo. Esta operación de E/S se realiza concurrentemente con la ejecución de instrucciones del programa de usuario.

Cuando el dispositivo externo pasa a estar preparado para actuar, es decir, cuando está listo para aceptar más datos del procesador, el módulo de E/S de este dispositivo externo envía una señal de *petición de interrupción* al procesador. El procesador responde suspendiendo la operación del programa que estaba ejecutando y salta a un programa, conocido como gestor de interrupción, que da

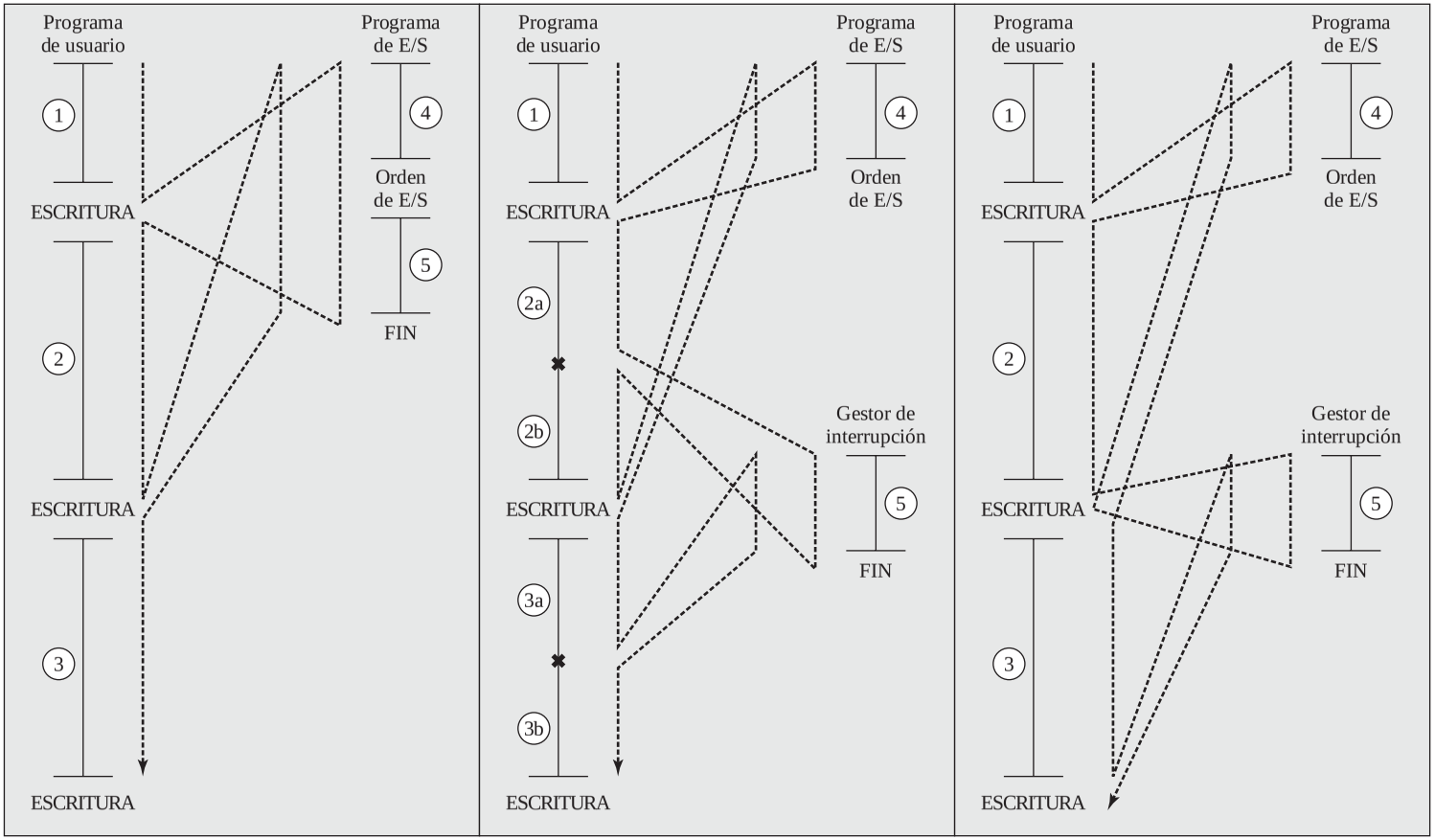


Figura 3.7. Flujo de control de un programa sin y con interrupción.

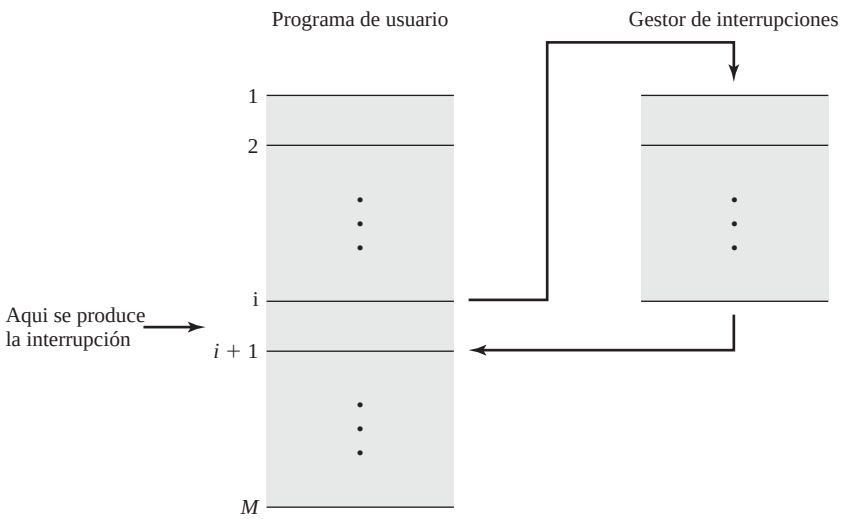


Figura 3.8. Transferencia de control debida a una interrupción.

servicio a ese dispositivo concreto, y prosigue con la ejecución del programa original después de haber dado dicho servicio al dispositivo. En la Figura 3.7b, los puntos en los que se producen las interrupciones se indican con una equis (X).

Desde el punto de vista del programa de usuario, una interrupción es precisamente eso: una interrupción en la secuencia normal de funcionamiento. Cuando el procesamiento de la interrupción se completa, la ejecución prosigue (Figura 3.8). Así, el programa de usuario no tiene que incluir ningún código especial para posibilitar las interrupciones; el procesador y el sistema operativo son los responsables de detener el programa de usuario y después permitir que prosiga en el mismo punto.

Para permitir el uso de interrupciones, se añade un *ciclo de interrupción* al ciclo de instrucción, como muestra la Figura 3.9. En el ciclo de interrupción, el procesador comprueba si se ha generado

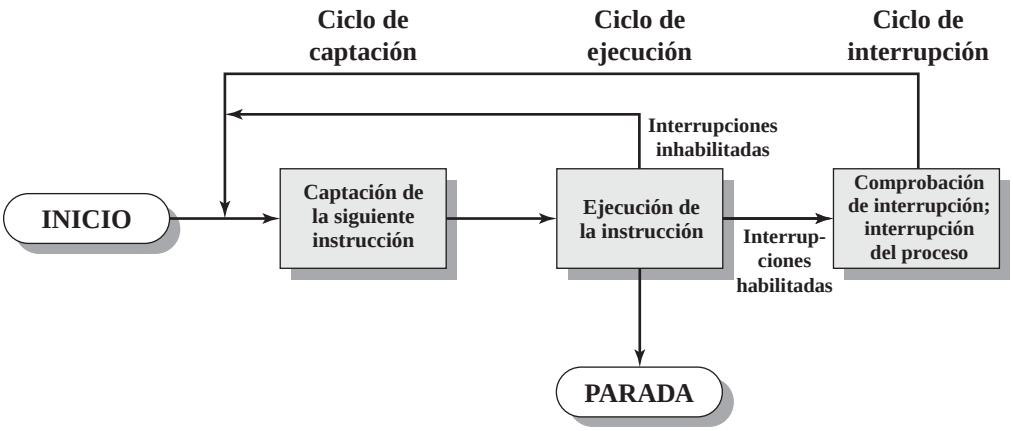


Figura 3.9. Ciclo de instrucción con interrupciones.

alguna interrupción, indicada por la presencia una señal de interrupción. Si no hay señales de interrupción pendientes, el procesador continúa con el ciclo de captación y accede a la siguiente instrucción del programa en curso. Si hay alguna interrupción pendiente, el procesador hace lo siguiente:

1. Suspende la ejecución del programa en curso y guarda su contexto. Esto significa almacenar la dirección de la siguiente instrucción a ejecutar (contenido actual del contador de programa) y cualquier otro dato relacionado con la actividad en curso del procesador.
2. Carga el contador de programa con la dirección de comienzo de una rutina de *gestión de interrupción.*

A continuación el procesador prosigue con el ciclo de captación y accede a la primera instrucción del programa de gestión de interrupción, que dará servicio a la interrupción. Generalmente el programa de gestión de interrupción forma parte del sistema operativo. Normalmente, este programa determina el origen de la interrupción y realiza todas las acciones que sean necesarias. Por ejemplo, en el caso que hemos estado analizando, el gestor determina qué módulo de E/S generó la interrupción, y puede saltar a un programa que escribe más datos en ese módulo de E/S. Cuando la rutina de gestión de interrupción se completa, el procesador puede proseguir la ejecución del programa de usuario en el punto en el que se interrumpió.

Es claro que este proceso supone una cierta penalización (*overhead*). Deben ejecutarse instrucciones extra (en el gestor de interrupción) para determinar el origen de la interrupción y para decidir la acción apropiada. No obstante, debido a la relativamente gran cantidad de tiempo que se perdería simplemente por la espera asociada a la operación de E/S, el procesador puede emplearse de manera mucho más eficiente utilizando interrupciones.

Para apreciar el aumento de eficiencia, considérese la Figura 3.10, que es un diagrama de tiempos basado en el flujo de control de las Figuras 3.7a y 3.7b. Las Figuras 3.7b y 3.10 asumen que el tiempo necesario para la operación de E/S es relativamente corto: menor que el tiempo para completar la ejecución de las instrucciones del programa de usuario que hay entre operaciones de escritura. La situación más frecuente, especialmente para un dispositivo lento como una impresora, es que la operación de E/S requiera mucho más tiempo para ejecutar una secuencia de instrucciones de usuario. La Figura 3.7c ilustra esta situación. En este caso, el programa de usuario llega a la segunda llamada de escritura (WRITE) antes de que la operación de E/S generada por la primera llamada se complete. El resultado es que el programa de usuario se detiene en este punto. Cuando la operación de E/S precedente se completa, esta nueva llamada de escritura se puede procesar, y se puede iniciar una nueva operación de E/S. La Figura 3.11 muestra la temporización para esta situación con y sin interrupciones. Podemos ver que existe una mejora de eficiencia porque parte del tiempo durante el cual la operación de E/S está en marcha se solapa con la ejecución de instrucciones de usuario.

La Figura 3.12 muestra el diagrama de estados del ciclo de instruccion modificado para incluir al procesamiento del ciclo de interrupción.

Interrupciones múltiples. Hasta ahora únicamente se ha discutido la existencia de una sola interrupción. Supóngase, no obstante, que se puedan producir varias interrupciones. Por ejemplo, un programa puede estar recibiendo datos a través de una línea de comunicación e imprimiendo resultados. La impresora generará interrupciones cada vez que complete una operación de escritura. El controlador de la línea de comunicación generará una interrupción cada vez que llegue una unidad de datos. La unidad de datos puede ser un carácter o un bloque, según el protocolo de comunicación. En

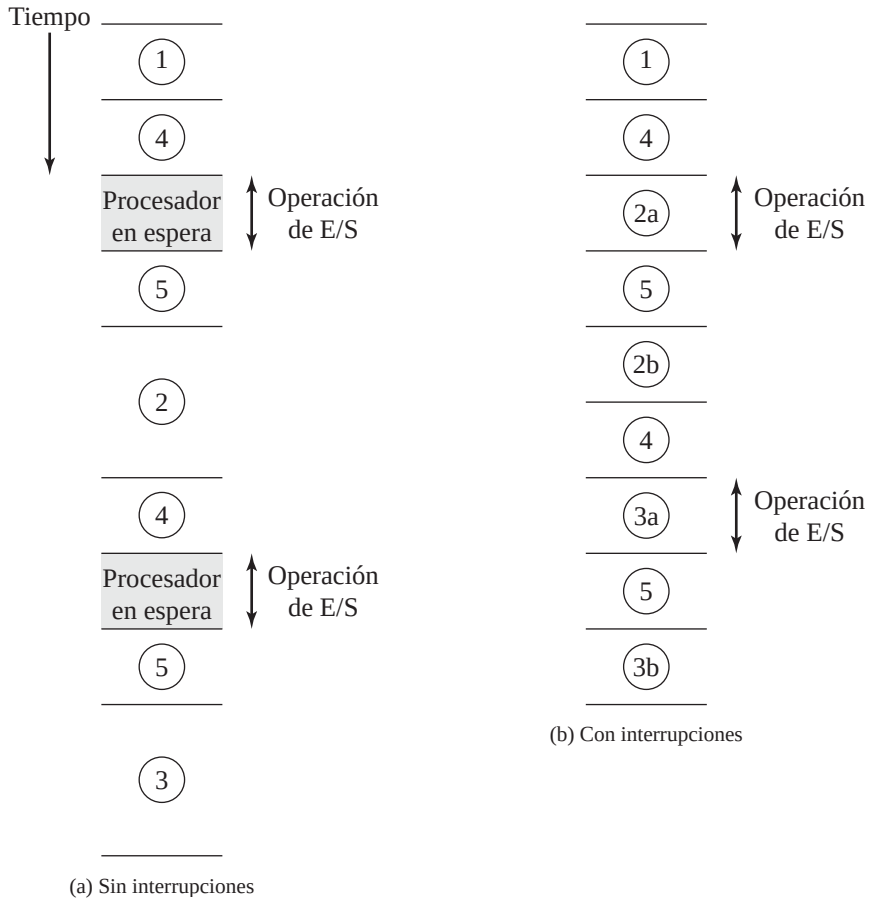


Figura 3.10. Temporización de un programa: espera corta de E/S.

cualquier caso, es posible que se produzca una interrupción de comunicaciones mientras se está procesando la interrupción de la impresora.

Se pueden seguir dos alternativas para tratar las interrupciones múltiples. La primera es desactivar las interrupciones mientras se está procesando una interrupción. Una *interrupción inhabilitada (disabled interrupt)* simplemente significa que el procesador puede y debe ignorar la señal de petición de interrupción. Si se produce una interrupción en ese momento, generalmente se mantiene pendiente y será examinada por el procesador una vez este haya activado las interrupciones. Así, cuando un programa de usuario se está ejecutando y se produce una interrupción, las interrupciones se inhabilitan inmediatamente. Después de que la rutina de gestión de interrupción termine, las interrupciones se habilitan antes de que el programa de usuario prosiga, y el procesador comprueba si se han producido interrupciones adicionales. Esta aproximación es correcta y simple, puesto que las interrupciones se manejan en un orden secuencial estricto (Figura 3.13a).

El inconveniente del enfoque anterior es que no tiene en cuenta la prioridad relativa ni las solicitudes con un tiempo crítico. Por ejemplo, cuando llega una entrada desde la línea de comunicaciones, esta debe tramitarse rápidamente para dejar espacio a los datos siguientes. Si los primeros datos no se han procesado antes de que lleguen los siguientes, se pueden perder.

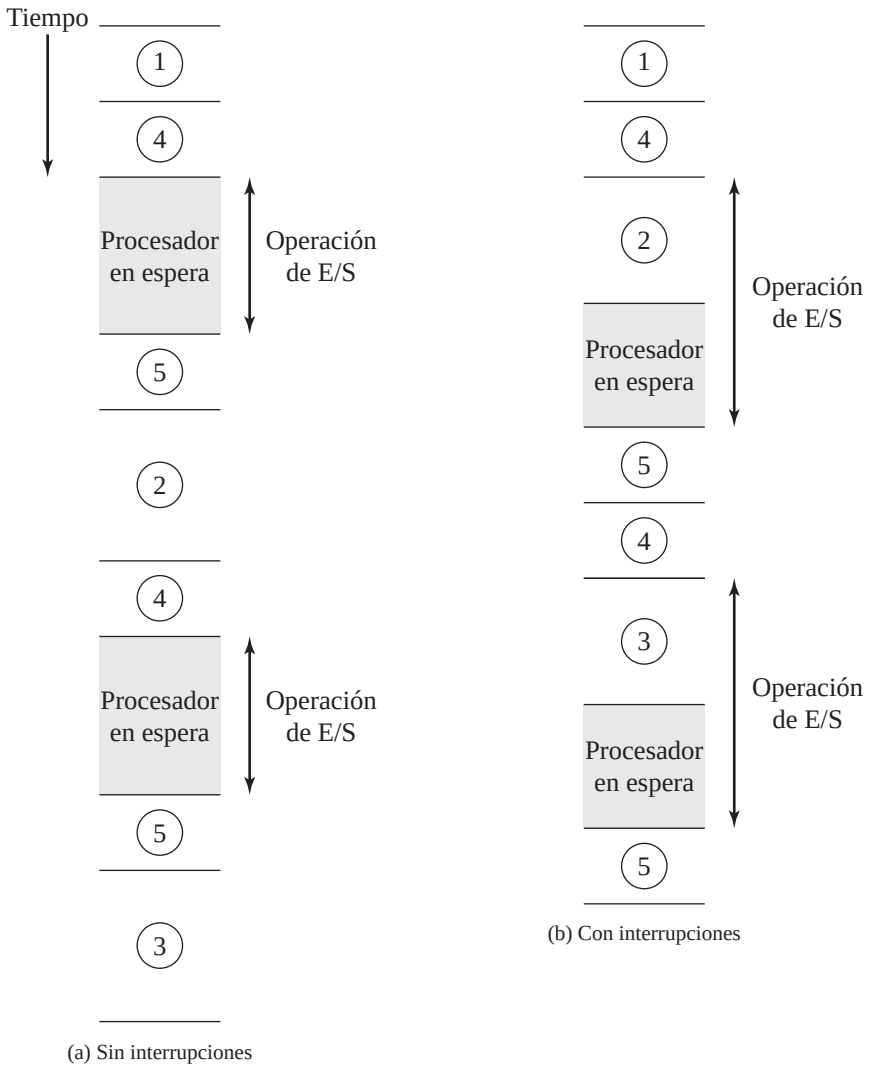


Figura 3.11. Temporización de un programa: espera larga de E/S.

Una segunda alternativa consiste en definir prioridades para las interrupciones y permitir que una interrupción de prioridad más alta pueda interrumpir a un gestor de interrupción de prioridad menor (Figura 3.13b). Como ejemplo de esta segunda alternativa, considérese un sistema con tres dispositivos de E/S: una impresora, un disco, y una línea de comunicaciones, con prioridades crecientes de 2, 4, y 5 respectivamente. La Figura 3.14, basada en un ejemplo de [TANE97] muestra una posible secuencia. Un programa de usuario comienza en $t = 0$. En $t = 10$, la impresora produce una interrupción; la información del programa de usuario se sitúa en la pila del sistema, y la ejecución continúa con la rutina de servicio de interrupción (ISR) de la impresora. Mientras se está ejecutando esta rutina, en $t = 15$, se produce una interrupción de comunicaciones. Como la línea de comunicaciones tiene una prioridad mayor que la impresora, se acepta la interrupción. La ISR de la impresora se interrumpe, su estado se introduce en la pila, y la ejecución continúa con la rutina de comunicaciones. Mientras se está ejecutando esta rutina, el disco ocasiona una interrupción ($t = 20$). Puesto que esta

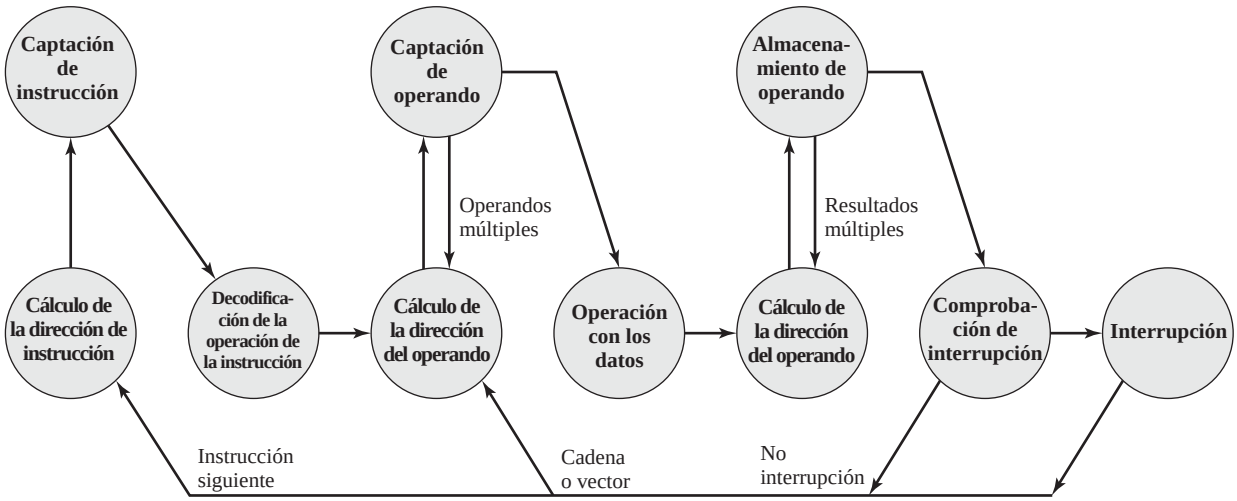


Figura 3.12. Diagrama de estados de un ciclo de instrucción, con interrupciones.

interrupción tiene una prioridad menor, simplemente se retiene, y la ISR de comunicaciones se ejecuta hasta que termina.

Cuando la rutina de comunicaciones termina ($t = 25$), se restaura el estado previo del procesador, que corresponde a la ejecución de la rutina de la impresora. No obstante, antes incluso de que pueda ejecutarse una sola instrucción de esa rutina, el procesador acepta la interrupción, de mayor prioridad, generada por el disco y el control se transfiere a la rutina de disco. Solo cuando esta rutina termina ($t = 35$), la rutina de la impresora puede reanudarse. Cuando termina esa rutina ($t = 40$), el control vuelve finalmente al programa de usuario.

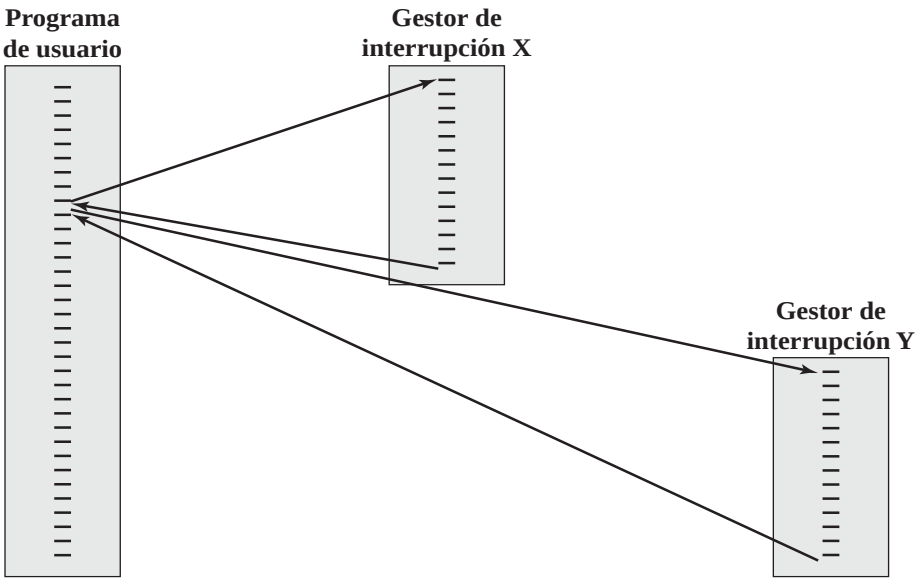
FUNCIONAMIENTO DE LAS E/S

Hasta aquí, hemos discutido el funcionamiento del computador controlado por el procesador, y nos hemos fijado esencialmente en la interacción del procesador y la memoria.

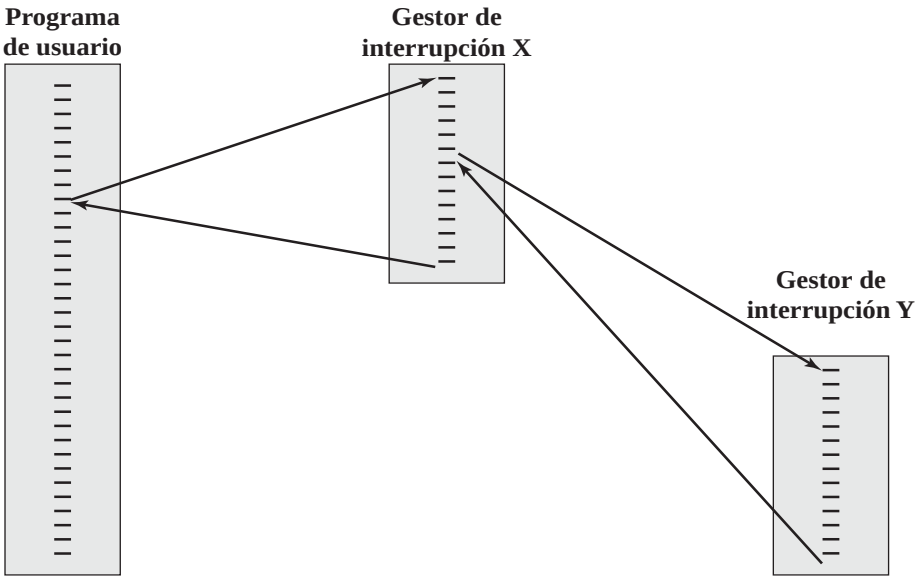
La discusión solo ha aludido al papel de los componentes de E/S. Este papel se discute con detalle en el Capítulo 7, pero es conveniente hacer aquí un breve resumen.

Un módulo de E/S (por ejemplo un controlador de disco) puede intercambiar datos directamente con el procesador. Igual que el procesador puede iniciar una lectura o escritura en memoria, especificando la dirección de una posición concreta de la misma, el procesador también puede leer o escribir datos de (o en) un módulo de E/S. En este último caso, el procesador identifica un dispositivo específico controlado por un módulo de E/S determinado. Por consiguiente, se puede producir una secuencia de instrucciones similar a la de la Figura 3.5, con instrucciones de E/S en lugar de las instrucciones de referencia a memoria.

En algunos casos, es deseable permitir que los intercambios de E/S se produzcan directamente con la memoria. En ese caso, el procesador cede a un módulo de E/S la autoridad para leer de o escribir en memoria, para que así la transferencia E/S-memoria pueda producirse sin la intervención del



(a) Procesamiento de una secuencia de interrupciones



(b) Procesamiento de interrupciones anidadas

Figura 3.13. Transferencia de control con varias interrupciones.

procesador. Durante esas transferencias, el módulo de E/S proporciona a la memoria las órdenes de lectura o escritura, liberando al procesador de cualquier responsabilidad en el intercambio. Esta operación se conoce con el nombre de acceso directo a memoria (DMA**,** *Direct Memory Access*), y se estudiará con detalle en el Capítulo 7.

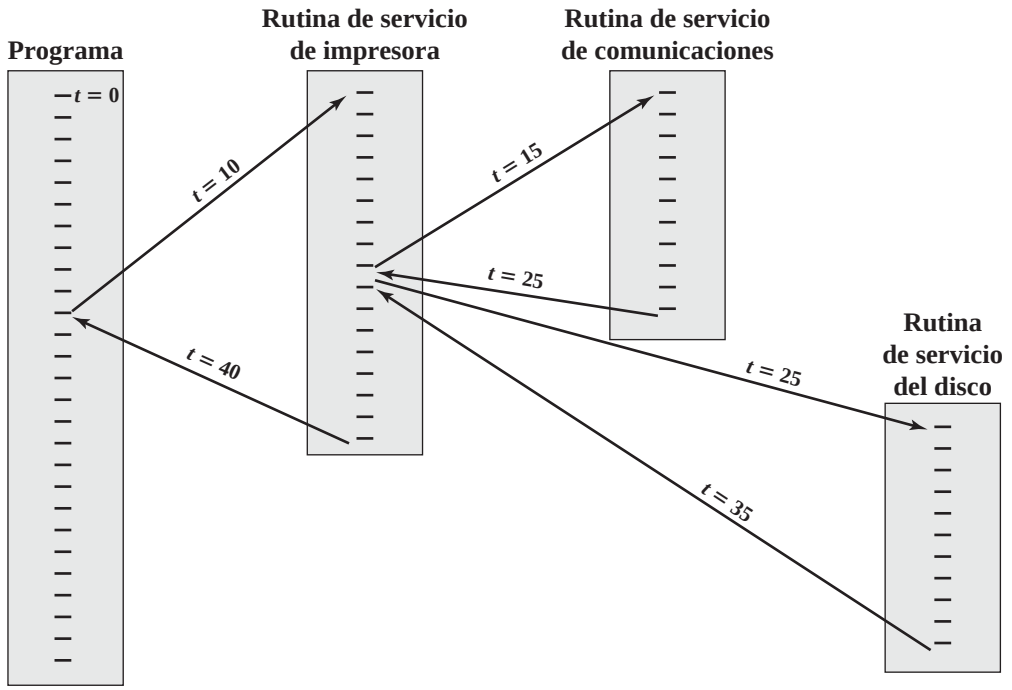


Figura 3.14. Ejemplo de secuencia temporal de varias interrupciones.

3.3. ESTRUCTURAS DE INTERCONEXIÓN

Un computador está constituido por un conjunto de unidades o módulos de tres tipos elementales (procesador, memoria, E/S) que se comunican entre sí. En efecto, un computador es una red de módulos elementales. Por consiguiente, deben existir lineas para interconectar estos módulos.

El conjunto de líneas que conectan los diversos módulos se denomina *estructura de interconexión*. El diseño de dicha estructura dependerá de los intercambios que deban producirse entre los módulos.

La Figura 3.15 sugiere los tipos de intercambios que se necesitan indicando las formas de las entradas y las salidas en cada tipo de módulo:

- **Memoria:** generalmente, un módulo de memoria está constituido por N palabras de la misma longitud. A cada palabra se le asigna una única dirección numérica (0,1,..., $N - 1$). Una palabra de datos puede leerse de o escribirse en la memoria. El tipo de operación se indica mediante las señales de control *Read* (Leer) y *Write* (Escribir). La posición de memoria para la operación se especifica mediante una dirección.
- **Módulo de E/S:** desde un punto de vista interno (al computador), la E/S es funcionalmente similar a la memoria. Hay dos tipos de operaciones, leer y escribir. Además, un módulo de E/S puede controlar más de un dispositivo externo. Nos referiremos a cada una de estas interfaces con un dispositivo externo con el nombre de *puerto (port)*, y se le asignará una dirección a

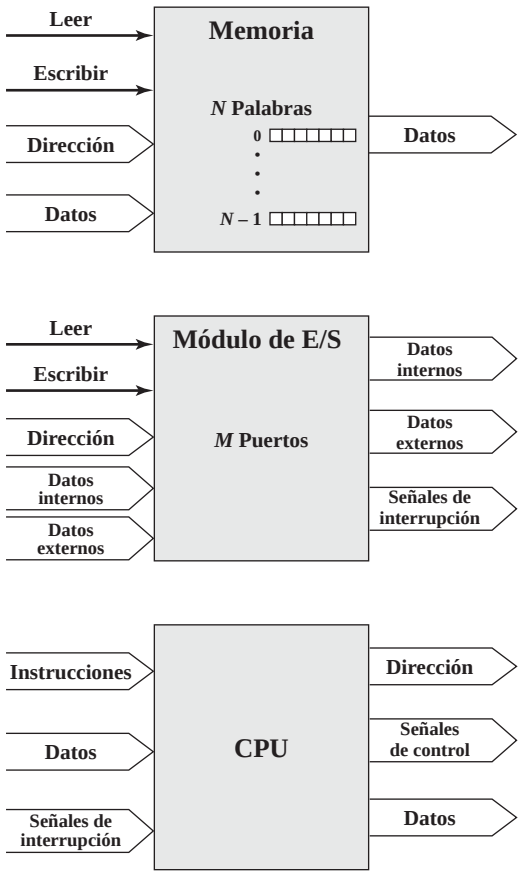


Figura 3.15. Módulos de un computador.

cada uno (0,1,...,M-1). Por otra parte, existen líneas externas de datos para la entrada y la salida de datos por un dispositivo externo. Por último, un módulo de E/S puede enviar señales de interrupción al procesador.

- **Procesador:** el procesador lee instrucciones y datos, escribe datos una vez los ha procesado, y utiliza ciertas señales para controlar el funcionamiento del sistema. También puede recibir señales de interrupción.

La lista precedente especifica los datos que se intercambian. La estructura de interconexión debe dar cobertura a los siguientes tipos de transferencias:

- **Memoria a procesador:** el procesador lee una instrucción o un dato desde la memoria.
- **Procesador a memoria:** el procesador escribe un dato en la memoria.
- **E/S a procesador:** el procesador lee datos de un dispositivo de E/S a través de un módulo de E/S.
- **Procesador a E/S:** el procesador envía datos al dispositivo de E/S.

- **Memoria a E/S y viceversa:** en estos dos casos, un módulo de E/S puede intercambiar datos directamente con la memoria, sin que tengan que pasar a través del procesador, utilizando el acceso directo a memoria (DMA).

A través de los años, se han probado diversas estructuras de interconexión. Las más comunes son, con diferencia, las estructuras de bus y de buses múltiples. El resto de este capítulo se dedica a evaluar las estructuras de buses.

3.4. INTERCONEXIÓN CON BUSES

Un bus es un camino de comunicación entre dos o más dispositivos. Una característica clave de un bus es que se trata de un medio de transmisión compartido. Al bus se conectan varios dispositivos, y cualquier señal transmitida por uno de esos dispositivos está disponible para que los otros dispositivos conectados al bus puedan acceder a ella. Si dos dispositivos transmiten durante el mismo periodo de tiempo, sus señales pueden solaparse y distorsionarse. Consiguientemente, solo un dispositivo puede transmitir con éxito en un momento dado.

Usualmente, un bus está constituido por varios caminos de comunicación, o líneas. Cada línea es capaz de transmitir señales binarias representadas por 1 y por 0. En un intervalo de tiempo, se puede transmitir una secuencia de dígitos binarios a través de una única línea. Se pueden utilizar varias líneas del bus para transmitir dígitos binarios simultáneamente (en paralelo). Por ejemplo, un dato de 8 bits puede transmitirse mediante ocho líneas del bus.

Los computadores poseen diferentes tipos de buses que proporcionan comunicación entre sus componentes a distintos niveles dentro de la jerarquía del sistema. El bus que conecta los componentes principales del computador (procesador, memoria, E/S) se denomina *bus del sistema (system bus)*. Las estructuras de interconexión más comunes dentro de un computador están basadas en el uso de uno o más buses del sistema.

ESTRUCTURA DEL BUS

El bus de sistema está constituido, usualmente, por entre cincuenta y cien líneas. A cada línea se le asigna un significado o una función particular. Aunque existen diseños de buses muy diversos, en todos ellos las líneas se pueden clasificar en tres grupos funcionales (Figura 3.16): líneas de datos, de

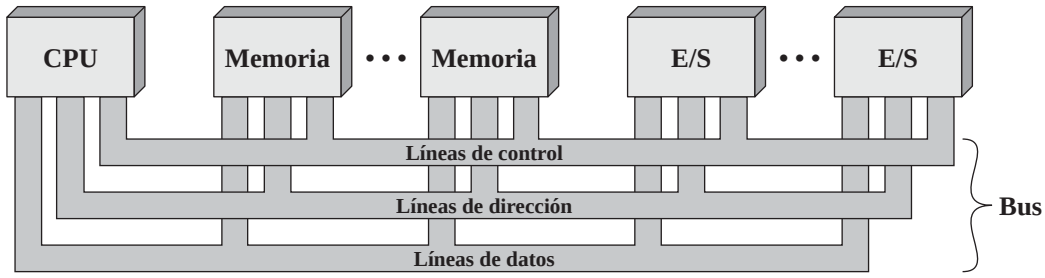


Figura 3.16. Esquema de interconexión mediante un bus.

direcciones y de control. Además, pueden existir líneas de alimentación para suministrar energía a los módulos conectados al bus.

Las *líneas de datos* proporcionan un camino para transmitir datos entre los módulos del sistema. El conjunto constituido por estas líneas se denomina *bus de datos*. El bus de datos puede incluir entre 32 y cientos de líneas, cuyo número se conoce como *anchura* del bus de datos. Puesto que cada línea solo puede transportar un bit cada vez, el número de líneas determina cuántos bits se pueden transferir al mismo tiempo. La anchura del bus es un factor clave a la hora de determinar las prestaciones del conjunto del sistema. Por ejemplo, si el bus de datos tiene una anchura de ocho bits, y las instrucciones son de 16 bits, entonces el procesador debe acceder al módulo de memoria dos veces por cada ciclo de instrucción.

Las *líneas de dirección* se utilizan para designar la fuente o el destino del dato situado en el bus de datos. Por ejemplo, si el procesador desea leer una palabra (8, 16 o 32 bits) de datos de la memoria, sitúa la dirección de la palabra deseada en las líneas de direcciones. Claramente, la anchura del bus de direcciones determina la máxima capacidad de memoria posible en el sistema. Además, las líneas de direcciones generalmente se utilizan también para direccionar los puertos de E/S. Usualmente, los bits de orden más alto se utilizan para seleccionar una posición de memoria o un puerto de E/S dentro de un módulo. Por ejemplo, en un bus de 8 bits, la dirección 01111111 e inferiores harían referencia a posiciones dentro de un módulo de memoria (el módulo 0) con 128 palabras de memoria, y las direcciones 10000000 y superiores designarían dispositivos conectados a un módulo de E/S (módulo 1).

Las **líneas de control** se utilizan para controlar el acceso y el uso de las líneas de datos y de direcciones. Puesto que las líneas de datos y de direcciones son compartidas por todos los componentes, debe existir una forma de controlar su uso. Las señales de control transmiten tanto órdenes como información de temporización entre los módulos del sistema. Las señales de temporización indican la validez de los datos y las direcciones. Las señales de órdenes especifican las operaciones a realizar. Algunas líneas de control típicas son

- **Escritura en memoria (*Memory write*):** hace que el dato del bus se escriba en la posición direccionada.
- **Lectura de memoria (*Memory read*):** hace que el dato de la posición direccionada se sitúe en el bus.
- **Escritura de E/S (*I/O write*):** hace que el dato del bus se transfiera a través del puerto de E/S direccionado.
- **Lectura de E/S (*E/S read*):** hace que el dato del puerto de E/S direccionado se sitúe en el bus.
- **Transferencia reconocida (*Transfer ACK*):** indica que el dato se ha aceptado o se ha situado en el bus.
- **Petición de bus (*Bus request*):** indica que un módulo necesita disponer del control del bus.
- **Cesión de bus (*Bus grant*):** indica que se cede el control del bus a un módulo que lo había solicitado.
- **Petición de interrupción (*Interrupt request*):** indica si hay una interrupción pendiente.
- **Interrupción reconocida (*Interrupt ACK*):** Señala que la interrupción pendiente se ha aceptado.

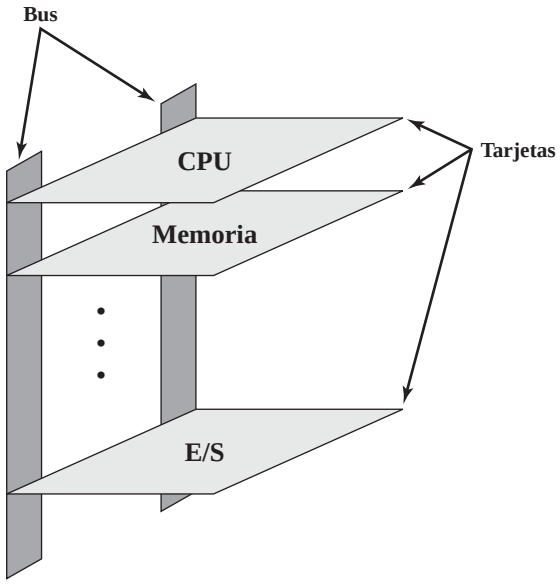


Figura 3.17. Implementación física típica de una arquitectura de bus.

- **Reloj (*clock*):** se utiliza para sincronizar las operaciones.
- **Inicio (*reset*):** pone los módulos conectados en su estado inicial.

El funcionamiento del bus se describe a continuación. Si un módulo desea enviar un dato a otro debe hacer dos cosas: (1) obtener el uso del bus y (2) transferir el dato a través del bus. Si un módulo desea pedir un dato a otro módulo, debe (1) obtener el uso del bus y (2) transferir la petición al otro módulo mediante las líneas de control y dirección apropiadas. Después debe esperar a que el segundo módulo envíe el dato.

Físicamente, el bus de sistema es de hecho un conjunto de conductores eléctricos paralelos. Estos conductores son líneas de metal grabadas en una tarjeta (tarjeta de circuito impreso). El bus se extiende a través de todos los componentes del sistema, cada uno de los cuales se conecta a algunas o a todas las líneas del bus. Una disposición física muy común se muestra en la Figura 3.17. En este ejemplo, el bus consta de dos columnas verticales de conductores. A lo largo de esas columnas, a intervalos regulares, hay puntos de conexión en forma de ranuras (*slots*) dispuestas en sentido horizontal para sostener las tarjetas de circuito impreso. Cada uno de los componentes principales del sistema ocupa una o varias tarjetas y se conecta al bus a través de esas ranuras. El sistema completo se introduce dentro de un chasis. Esta organización puede encontrarse todavía en alguno de los buses del computador. No obstante, los sistemas actuales tienden a tener sus componentes principales en la misma tarjeta y los circuitos integrados incluyen más elementos. Así, el bus que conecta el procesador y la memoria caché se integra en el microprocesador junto con el procesador y la caché (*on-chip*), y el bus que conecta el procesador con la memoria y otros componentes se incluye en la tarjeta (*on-board*).

Esta es la disposición más conveniente. Así se puede adquirir un computador pequeño y expandirlo (ampliar memoria, módulos de E/S) más adelante añadiendo más tarjetas. Si un componente de una tarjeta falla, la tarjeta puede quitarse y sustituirse fácilmente.

JERARQUÍAS DE BUSES MÚLTIPLES

Si se conecta un gran número de dispositivos al bus, las prestaciones pueden disminuir. Hay dos causas principales:

1. En general, a más dispositivos conectados al bus, mayor es el retardo de propagación. Este retardo determina el tiempo que necesitan los dispositivos para coordinarse en el uso del bus. Si el control del bus pasa frecuentemente de un dispositivo a otro, los retardos de propagación pueden afectar sensiblemente a las prestaciones.
2. El bus puede convertirse en un cuello de botella a medida que las peticiones de transferencia acumuladas se aproximan a la capacidad del bus. Este problema se puede resolver en alguna medida incrementando la velocidad a la que el bus puede transferir los datos y utilizando buses más anchos (por ejemplo incrementando el bus de datos de 32 a 64 bits). Sin embargo, puesto que la velocidad de transferencia que necesitan los dispositivos conectados al bus (por ejemplo, controladores de gráficos y de vídeo, interfaces de red) está incrementándose rápidamente, es un hecho que el bus único está destinado a dejar de utilizarse.

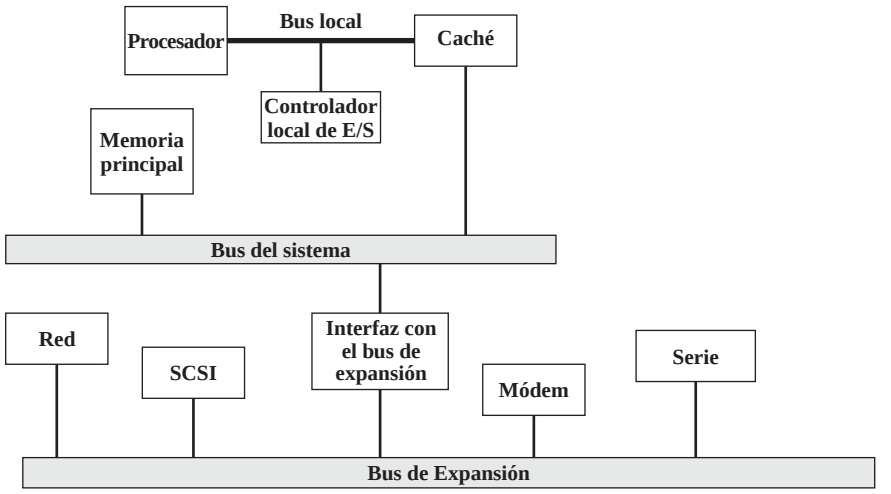
Por consiguiente, la mayoría de los computadores utilizan varios buses, normalmente organizados jerárquicamente. Una estructura típica se muestra en la Figura 3.18a. Hay un bus local que conecta el procesador a una memoria caché y al que pueden conectarse también uno o más dispositivos locales. El controlador de memoria caché conecta la caché no solo al bus local sino también al bus de sistema, donde se conectan todos los módulos de memoria principal. Como se discute en el Capítulo 4, el uso de una caché alivia la exigencia de soportar los accesos frecuentes del procesador a memoria principal. De hecho, la memoria principal puede pasar del bus local al bus de sistema. De esta forma, las transferencias de E/S con la memoria principal a través del bus de sistema no interfieren la actividad del procesador.

Es posible conectar controladores de E/S directamente al bus de sistema. Una solución más eficiente consiste en utilizar uno o más buses de expansión. La interfaz del bus de expansión regula las transferencias de datos entre el bus de sistema y los controladores conectados al bus de expansión. Esta disposición permite conectar al sistema una amplia variedad de dispositivos de E/S y al mismo tiempo aislar el tráfico de información entre la memoria y el procesador del tráfico correspondiente a las E/S.

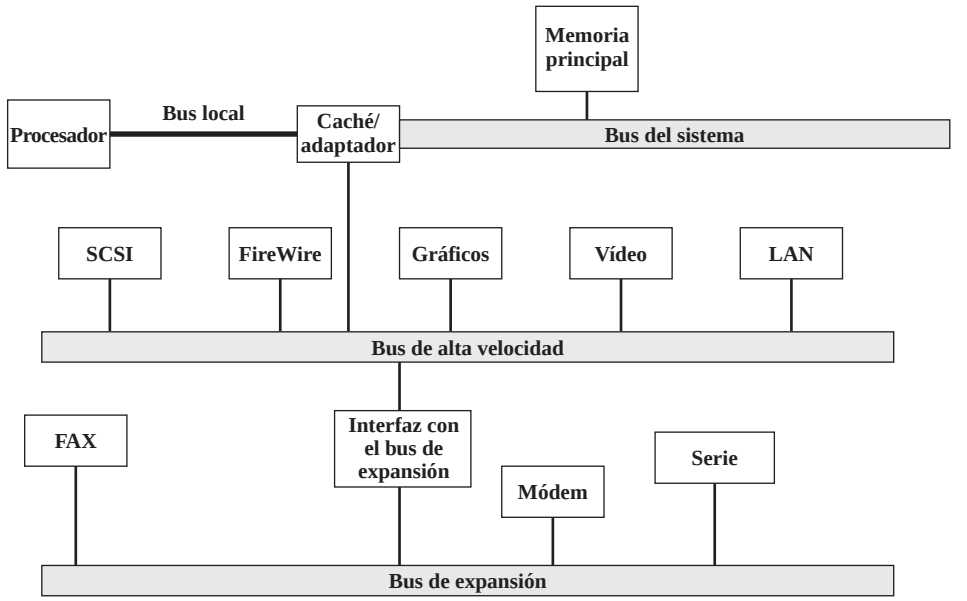
La Figura 3.18a muestra algunos ejemplos típicos de dispositivos de E/S que pueden estar conectados al bus de expansión. Las conexiones a red incluyen conexiones a redes de área local (LAN, *Local Area Networks*) tales como una red Ethernet de diez Mbps y conexiones a redes de area amplia (*Wide Area Networks*) tales como la red de conmutación de paquetes (*packet-switching network*). La interfaz SCSI (*Small Computer System Interface*) es en sí un tipo de bus utilizado para conectar controladores de disco y otros periféricos. El puerto serie puede utilizarse para conectar una impresora o un escáner.

Esta arquitectura de buses tradicional es razonablemente eficiente, pero muestra su debilidad a medida que los dispositivos de E/S ofrecen prestaciones cada vez mayores. La respuesta común a esta situación, por parte de la industria, ha sido proponer un bus de alta velocidad que está estrechamente integrado con el resto del sistema, y requiere solo un adaptador (*bridge*) entre el bus del procesador y el bus de alta velocidad. En algunas ocasiones, esta disposición es conocida como arquitectura de entreplanta (*mezzanine architecture*).

La Figura 3.18b muestra un ejemplo típico de esta aproximación. De nuevo, hay un bus local que conecta el procesador a un controlador de caché, que a su vez está conectado al bus de sistema



(a) Arquitectura de bus tradicional



(b) Arquitectura de altas prestaciones

Figura 3.18. Ejemplos de configuraciones de bus.

que soporta a la memoria principal. El controlador de caché está integrado junto con el adaptador, o dispositivo de acoplo, que permite la conexión al bus de alta velocidad. Este bus permite la conexión de LAN de alta velocidad, tales como Fast Ethernet a cien Mbps, controladores de estaciones de trabajo específicos para aplicaciones gráficas y de vídeo, y también controladores de interfaz para buses de periféricos tales como SCSI y Firewire. Este último es un bus de alta velocidad diseñado específicamente para conectar dispositivos de E/S de alta capacidad. Los dispositivos de velocidad menor pueden conectarse al bus de expansión, que utiliza una interfaz para adaptar el tráfico entre el bus de expansión y el bus de alta velocidad.

La ventaja de esta organización es que el bus de alta velocidad acerca al procesador los dispositivos que exigen prestaciones elevadas y al mismo tiempo es independiente del procesador. Así, se pueden tolerar las diferencias de velocidad entre el procesador y el bus de altas prestaciones y las variaciones en la definición de las líneas de los buses. Los cambios en la arquitectura del procesador no afectan al bus de alta velocidad, y viceversa.

ELEMENTOS DE DISEÑO DE UN BUS

Aunque existe una gran diversidad de diseños de buses, hay unos pocos parámetros o elementos de diseño que sirven para distinguir y clasificar los buses. La Tabla 3.2 enumera los elementos clave.

Tabla 3.2. Elementos de diseño de un bus.

Tipo	**Anchura del bus**
Dedicado	Dirección
Multiplexado	Datos
Método de arbitraje	**Tipo de transferencia de datos**
Centralizado	Lectura
Distribuido	Escritura
Temporización	Lectura-modificación-escritura
Síncrono	Lectura-después de-escritura
Asíncrono	Bloque

Tipos de buses. Las líneas del bus se pueden dividir en dos tipos genéricos: dedicadas y multiplexadas. Una línea de bus dedicada está permanente asignada a una función o a un subconjunto físico de componentes del computador.

Un ejemplo de dedicación funcional, común en muchos buses, es el uso de líneas separadas para direcciones y para datos. Sin embargo, no es esencial. Por ejemplo, la información de dirección y datos podría transmitirse a través del mismo conjunto de líneas si se utiliza una línea de control de Dirección Válida. Al comienzo de la transferencia de datos, la dirección se sitúa en el bus y se activa la línea de Dirección Válida. En ese momento, cada módulo dispone de un periodo de tiempo para copiar la dirección y determinar si es él el módulo direccionado. Después la dirección se quita del bus, y las mismas conexiones se utilizan para la subsecuente tranferencia de lectura o escritura de datos. Este método de uso de las mismas líneas para usos diferentes se llama *multiplexado en el tiempo*.

La ventaja del multiplexado en el tiempo es el uso de menos líneas, cosa que ahorra espacio y, normalmente, costes. La desventaja es que se necesita una circuitería más compleja en cada módulo. Además, existe una posible reducción en las prestaciones debido a que los eventos que deben compartir las mismas líneas no pueden producirse en paralelo.

La *dedicación física* se refiere al uso de múltiples buses, cada uno de los cuales conecta solo un subconjunto de módulos. Un ejemplo típico es el uso de un bus de E/S para interconectar todos los módulos de E/S; este bus a su vez se conecta al bus principal a través de algún tipo de módulo adaptador de E/S. La ventaja potencial de la dedicación física es su elevado rendimiento, debido a que hay

menos conflictos por el acceso al bus (*bus contention*). Una desventaja es el incremento en el tamaño y el costo del sistema.

Método de arbitraje. En todos los sistemas, exceptuando los más simples, más de un módulo puede necesitar el control del bus. Por ejemplo, un módulo de E/S puede necesitar leer o escribir directamente en memoria, sin enviar el dato al procesador. Puesto que en un instante dado solo una unidad puede transmitir a través del bus, se requiere algún método de arbitraje. Los diversos métodos se pueden clasificar aproximadamente como centralizados o distribuidos. En un esquema centralizado, un único dispositivo hardware, denominado *controlador del bus* o *árbitro*, es responsable de asignar tiempos en el bus. El dispositivo puede estar en un módulo separado o ser parte del procesador. En un esquema distribuido, no existe un controlador central. En su lugar, cada módulo dispone de lógica para controlar el acceso y los módulos actúan conjuntamente para compartir el bus. En ambos métodos de arbitraje, el propósito es designar un dispositivo, el procesador o un módulo de E/S como maestro del bus. El maestro podría entonces iniciar una transferencia de datos (lectura o escritura) con otro dispositivo, que actúa como esclavo en este intercambio concreto.

Temporización. El término temporización hace referencia a la forma en la que se coordinan los eventos en el bus. Los buses utilizan temporización síncrona o asíncrona.

Con temporización síncrona, la presencia de un evento en el bus está determinada por un reloj. El bus incluye una línea de reloj a través de la que se transmite una secuencia en la que se alternan intervalos regulares de igual duración a uno y a cero. Un único intervalo a uno seguido de otro a cero se conoce como *ciclo de reloj* o *ciclo de bus* y define un intervalo de tiempo unidad (*time slot*). Todos los dispositivos del bus pueden leer la línea de reloj, y todos los eventos empiezan al principio del ciclo de reloj. La Figura 3.19 muestra el diagrama de tiempos de una operación de lectura síncrona (en el Apéndice 3A se puede consultar una descripción de los diagramas de tiempo). Otras señales del bus pueden cambiar en el flanco de subida de la señal de reloj (reaccionan con un ligero retardo). La mayoría de los eventos se prolongan durante un único ciclo de reloj. En este ejemplo sencillo, la CPU activa una señal de lectura y sitúa una dirección de memoria en las líneas de dirección durante el primer ciclo y puede activar varias líneas de estado. Una vez que las líneas se han estabilizado, el procesador activa una señal de inicio para indicar la presencia de la dirección en el bus. En el caso de una lectura, el procesador pone una orden de lectura al comienzo del segundo ciclo. El módulo de memoria reconoce la dirección y, después de un retardo de un ciclo, sitúa el dato en las líneas de datos. El procesador lee el dato de dichas líneas y quita la señal de lectura. En el caso de una escritura, el procesador pone el dato en las líneas de datos al comienzo del segundo ciclo, y pone la orden de escritura una vez estabilizadas las líneas de datos. El módulo de memoria copia la información de las líneas de datos durante el tercer ciclo de reloj.

Con la temporización asíncrona, la presencia de un evento en el bus es consecuencia y depende de que se produzca un evento previo. En el ejemplo sencillo de la Figura 3.20a, el procesador sitúa las señales de dirección y lectura en el bus. Después de un breve intervalo para que las señales se estabilicen, activa la orden de lectura, indicando la presencia de señales de dirección y control válidas. El módulo de memoria correspondiente decodifica la dirección y responde proporcionando el dato en la línea de datos. Una vez estabilizadas las líneas de datos, el módulo de memoria activa la línea de *reconocimiento* para indicar al procesador que el dato está disponible. Cuando el maestro ha leído el dato de las líneas correspondientes, deshabilita la señal de lectura. Esto hace que el módulo de memoria libere las líneas de datos y reconocimiento. Por último, una vez se ha desactivado la línea de reconocimiento, el procesador quita la información de dirección de las líneas correspondientes.

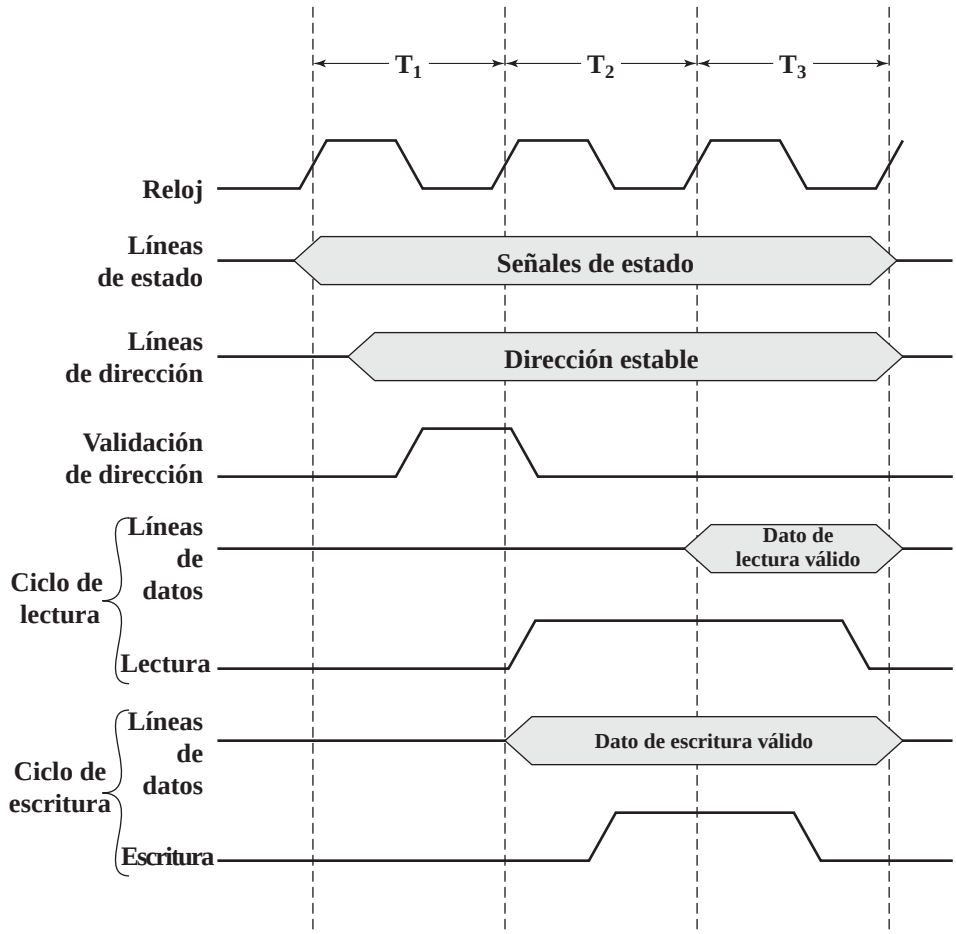
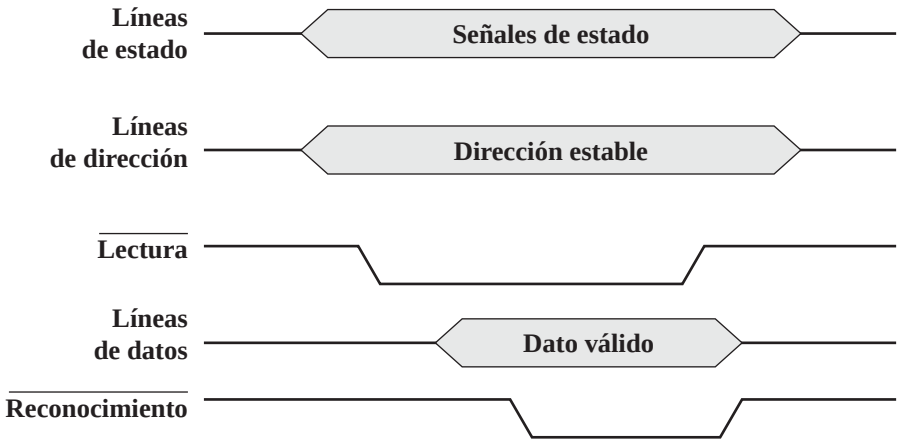


Figura 3.19. Temporización síncrona de las operaciones de bus.

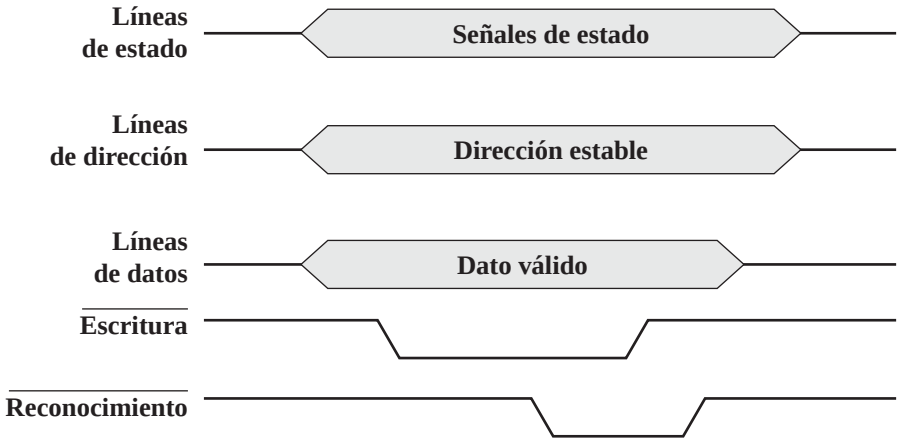
La Figura 3.20b muestra una operación sencilla de escritura asíncrona. En este caso, el maestro sitúa el dato en las líneas de datos al mismo tiempo que la dirección y la información de estado en las líneas correspondientes. El módulo de memoria responde a la orden de escritura copiando el dato de las líneas de datos y activando la línea de reconocimiento. Entonces, el maestro retira la señal de escritura y el módulo de memoria la señal de reconocimiento.

La temporización síncrona es más fácil de implementar y comprobar. Sin embargo, es menos flexible que la temporización asíncrona. Debido a que todos los dispositivos en un bus síncrono deben utilizar la misma frecuencia de reloj, el sistema no puede aprovechar las mejoras en las prestaciones de los dispositivos. Con la temporización asíncrona, pueden compartir el bus una mezcla de dispositivos lentos y rápidos, utilizando tanto las tecnologías más antiguas, como las más recientes.

Anchura del bus. El concepto de anchura del bus se ha presentado ya. La anchura del bus de datos afecta a las prestaciones del sistema: cuanto más ancho es el bus de datos, mayor es el número de bits que se transmiten a la vez. La anchura del bus de direcciones afecta a la capacidad del



(a) Temporización síncrona



(b) Temporización asíncrona

Figura 3.20. Temporización asíncrona de las operaciones de bus. (a) Ciclo de lectura del bus de sistema. (b) Ciclo de eescritura del bus de sistema.

sistema: cuanto más ancho es el bus de direcciones, mayor es el rango de posiciones a las que se puede hacer referencia.

Tipo de tranferencia de datos. Por último, un bus permite varios tipos de transferencias de datos, tal y como ilustra la Figura 3.21. Todos los buses permiten tanto transferencias de escritura (dato de maestro a esclavo) como de lectura (dato de esclavo a maestro). En el caso de un bus con direcciones y datos multiplexados, el bus se utiliza primero para especificar la dirección y luego para transferir el dato. En una operación de lectura, generalmente hay un tiempo de espera mientras el dato se está captando del dispositivo esclavo para situarlo en el bus. Tanto para la lectura como para la escritura, puede haber también un retardo si se necesita utilizar algún procedimiento de arbitraje para acceder al control del bus en el resto de la operación (es decir, tomar el bus para

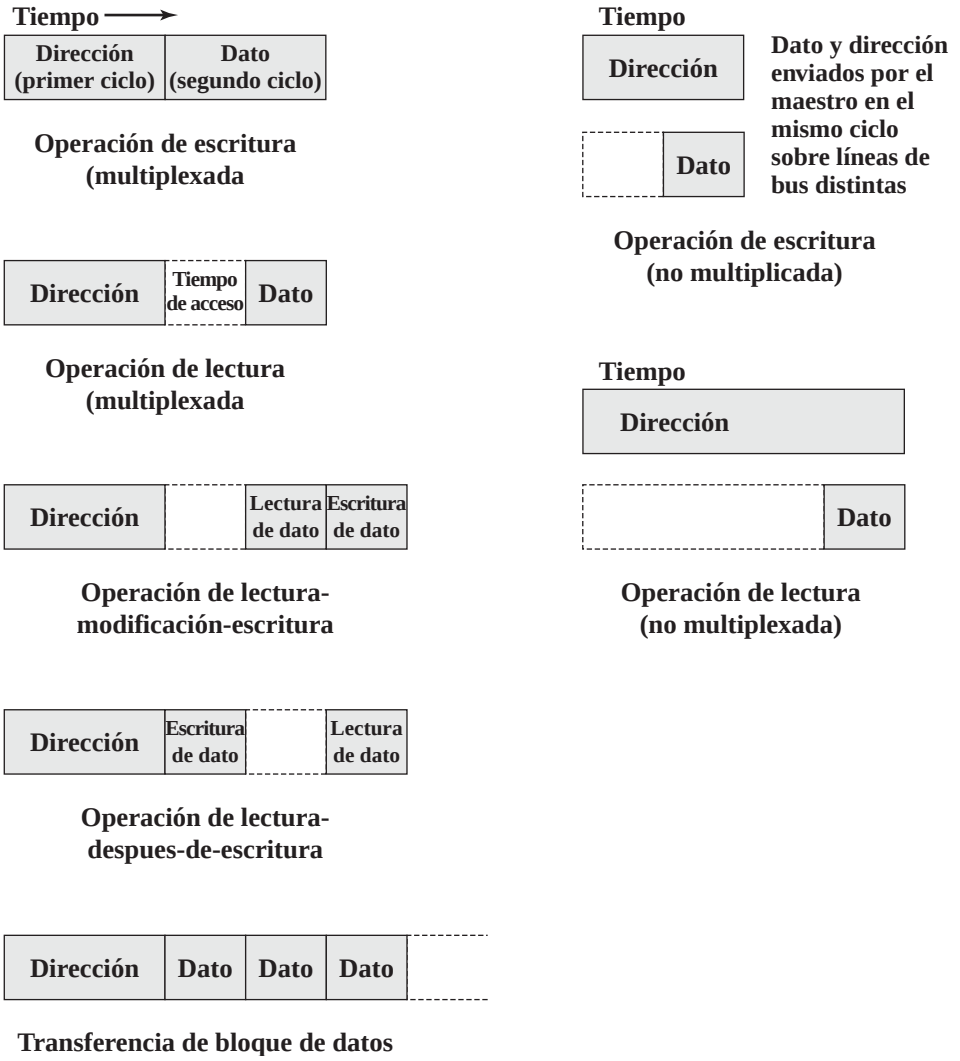


Figura 3.21. Tipos de transferencias de datos en un bus.

solicitar una lectura o una escritura, y después tomar el bus de nuevo para realizar la lectura o la escritura).

En el caso de que haya líneas dedicadas a datos y a direcciones, la dirección se sitúa en el bus de direcciones y se mantiene ahí mientras que el dato se ubica en el bus de datos. En una escritura, el maestro pone el dato en el bus de datos tan pronto como se han estabilizado las líneas de dirección y el esclavo ha podido reconocer su dirección. En una operación de lectura, el esclavo pone el dato en el bus de datos tan pronto como haya reconocido su dirección y disponga del mismo.

En ciertos buses también son posibles algunas operaciones combinadas. Una operación de lectura-modificación-escritura es simplemente una lectura seguida inmediatamente de una escritura en la misma dirección. La dirección se proporciona una sola vez al comienzo de la operación. La operación

completa es generalmente indivisible de cara a evitar cualquier acceso al dato por otros posibles maestros del bus. El objetivo primordial de esta posibilidad es proteger los recursos de memoria compartida en un sistema con multiprogramación (*véase* el Capítulo 8).

La lectura-después de-escritura es una operación indivisible que consiste en una escritura seguida inmediatamente de una lectura en la misma dirección. La operación de lectura se puede realizar con el propósito de comprobar el resultado.

Algunos buses también permiten transferencias de bloques de datos. En este caso, un ciclo de dirección viene seguido por *n* ciclos de datos. El primer dato se transfiere a o desde la dirección especificada; el resto de datos se transfieren a o desde las direcciones siguientes.

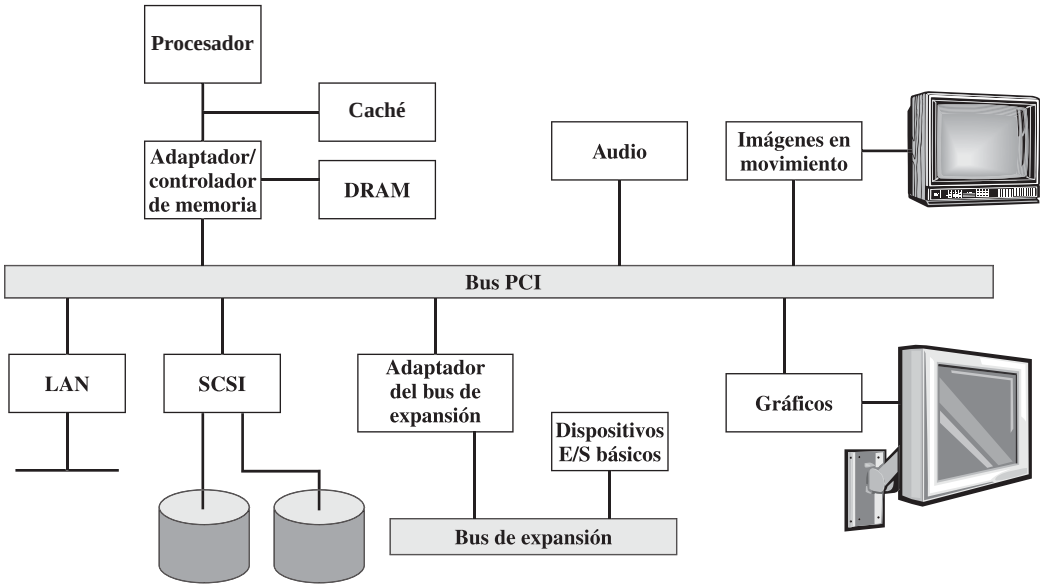
3.5. PCI

El bus PCI (*Peripheral Component Interconnect*, Interconexión de Componente Periférico) es un bus muy popular de ancho de banda elevado, independiente del procesador, que se puede utilizar como bus de periféricos o bus para una arquitectura de entreplanta. Comparado con otras especificaciones comunes de bus, el PCI proporciona mejores prestaciones para los subsistemas de E/S de alta velocidad (por ejemplo, los adaptadores de pantalla gráfica, los controladores de interfaz de red, los controladores de disco, etc.). El estándar actual permite el uso de hasta 64 líneas de datos a 66 MHz, para una velocidad de transferencia de 528 MB, o 4,224 Gbps. Pero no es precisamente su elevada velocidad la que hace atractivo al PCI. El PCI ha sido diseñado específicamente para ajustarse, económicamente a los requisitos de E/S de los sistemas actuales; se implementa con muy pocos circuitos integrados y permite que otros buses se conecten al bus PCI.

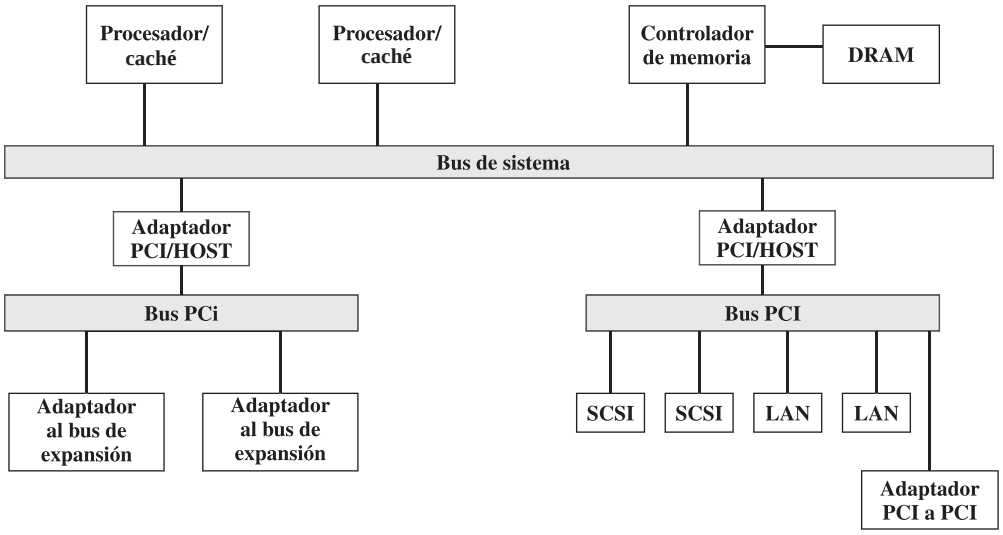
Intel empezó a trabajar en el PCI en 1990 pensando en sus sistemas basados en el Pentium. Muy pronto Intel cedió sus patentes al dominio público y promovió la creación de una asociación industrial, la PCI SIG (de *Special Interest Group*), para continuar el desarrollo y mantener la compatibilidad de las especificaciones del PCI. El resultado ha sido que el PCI ha sido ampliamente adoptado y se está incrementando su uso en los computadores personales, estaciones de trabajo, y servidores. Puesto que las especificaciones son de dominio público y están soportadas por una amplia banda de la industria de procesadores y periféricos, los productos PCI fabricados por compañías diferentes son compatibles.

El PCI está diseñado para permitir una cierta variedad de configuraciones basadas en microprocesadores, incluyendo sistemas tanto de uno como de varios procesadores. Por consiguiente, proporciona un conjunto de funciones de uso general. Utiliza temporización síncrona y un esquema de arbitraje centralizado.

La Figura 3.22a muestra la forma usual de utilizar el bus PCI en un sistema monoprocesador. Un dispositivo que integra el controlador de DRAM y el adaptador al bus PCI proporciona el acoplamiento al procesador y la posibilidad de generar datos a velocidades elevadas. El adaptador actúa como un registro de acoplo (buffer) de datos puesto que la velocidad del bus PCI puede diferir de la capacidad de E/S del procesador. En un sistema multiprocesador (Figura 3.22b), se pueden conectar mediante adaptadores una o varias configuraciones PCI al bus de sistema del procesador. Al bus de sistema se conectan únicamente las unidades procesador/caché, la memoria principal y los adaptadores de PCI. De nuevo, el uso de adaptadores mantiene al PCI independiente de la velocidad del procesador y proporciona la posibilidad de recibir y enviar datos rápidamente.



(a) Sistema de sobremesa típico



(b) Servidor típico

Figura 3.22. Ejemplos de configuraciones PCI.

ESTRUCTURA DEL BUS

El bus PCI puede configurarse como un bus de 32 o 64 bits. La Tabla 3.3 define las 49 líneas de señal obligatorias en el PCI. Se dividen en los grupos funcionales siguientes:

Tabla 3.3. Líneas obligatorias del bus PCI.

Denominación	Tipo	Descripción
		Terminales de sistema
CLK	in	Proporciona la temporización para todas las transacciones y es muestreada en el flanco de subida. Se pueden utilizar frecuencias de reloj de hasta 33 MHz.
RST#	in	Hace que todos los registros, secuenciadores y señales específicas de PCI pasen a su estado de inicio.
		Terminales de direcciones y datos
AD[31:0]	t/s	Líneas multiplexadas para direcciones y datos.
C/BE[3:0]#	t/s	Señales multiplexadas de órdenes del bus y de byte activo (*byte enable*). Durante la fase de datos, las líneas indican cuál de los cuatro grupos de líneas de byte transporta información válida.
PAR	t/s	Proporciona paridad para para las líneas AD y C/BE, un ciclo de reloj después. El maestro proporciona PAR para las fases de dirección y escritura de datos, y el dispositivo de lectura genera PAR en la fase de lectura de datos.
		Terminales de control de interfaz
FRAME#	s/t/s	Suministradas por el maestro actual para indicar el comienzo y la duración de una transferencia. Las activa al comienzo y las desactiva cuando el maestro está preparado para empezar el final de la fase de datos.
IRDY#	s/t/s	Iniciador preparado (*Initiator Ready*). La proporciona el maestro actual del bus (el iniciador de la transacción). Durante una lectura, indica que el maestro está preparado para aceptar datos; durante una escritura indica que el dato válido está en AD.
TRDY#	s/t/s	Dispositivo preparado (*Target Ready*). La proporciona el dispositivo seleccionado. Durante una lectura , el dato válido está en AD; durante una escritura indica que el destino está preparado para aceptar datos.
STOP#	s/t/s	Indica que el dispositivo seleccionado desea que el maestro pare la transacción actual.
LOCK#	s/t/s	Indica una operación atómica indivisible que puede requerir varias transferencias.
IDSEL#	in	Selector de inicio de dispositivo (*Initialization Device Select*). Usada como señal de selección de circuito (*chip select*) durante las lecturas y escrituras de configuración.
DEVSEL#	in	Selector de dispositivo (*Device Select*). Activada por el dispositivo seleccionado cuando ha reconocido su dirección. Indica al maestro actual si se ha seleccionado un dispositivo.
		Terminales de arbitraje
REQ#	t/s	Indica al árbitro que el dispositivo correspondiente solicita utilizar el bus. Es una línea punto-a-punto específica para cada dispositivo.
GNT#	t/s	Indica al dispositivo que el árbitro le ha cedido el acceso al bus. Es una línea punto-a-punto específica para cada dispositivo.
		Terminales para señales de error
PERR#	s/t/s	Error de paridad. Indica que se ha detectado un error de paridad en los datos por parte del dispositivo en el caso de una escritura, o por parte del maestro en el caso de una lectura.
SERR#	o/d	Error del sistema. La puede activar cualquier dispositivo para comunicar errores de paridad en la dirección u otro tipo de errores críticos distintos de la paridad.

- **Terminales («patillas») de sistema:** constituidas por los terminales de reloj y de inicio (reset).
- **Terminales de direcciones y datos:** incluye 32 líneas para datos y direcciones multiplexadas en el tiempo. Las otras líneas del grupo se utilizan para interpretar y validar las líneas de señal correspondientes a los datos y a las direcciones.
- **Terminales de control de la interfaz:** controlan la temporización de las transferencias y proporcionan coordinación entre los que las inician y los destinatarios.
- **Terminales de arbitraje:** a diferencia de las otras líneas de señal del PCI, estas no son líneas compartidas. En cambio, cada maestro del PCI tiene su par propio de líneas que lo conectan directamente al árbitro del bus PCI.
- **Terminales para señales de error:** utilizadas para indicar errores de paridad u otros.

Además, la especificación del PCI define 51 señales opcionales (Tabla 3.4), divididas en los siguientes grupos funcionales:

- **Terminales de interrupción:** para los dispositivos PCI que deben generar peticiones de servicio. Igual que los terminales de arbitraje, no son líneas compartidas sino que cada dispositivo PCI tiene su propia línea o líneas de petición de interrupción a un controlador de interrupciones.
- **Terminales de soporte de caché:** necesarios para permitir memorias caché en el bus PCI asociadas a un procesador o a otro dispositivo. Estos terminales permiten el uso de protocolos de coherencia de caché de sondeo de bus (*snoopy caché*) (en el Capítulo 16 se discuten estos protocolos).
- **Terminales de ampliación a bus de 64 bits:** incluye 32 líneas multiplexadas en el tiempo para direcciones y datos y se combinan con las líneas obligatorias de dirección y datos para constituir un bus de direcciones y datos de 64 bits. Hay otras líneas de este grupo que se utilizan para interpretar y validar las líneas de datos y direcciones. Por último, hay dos líneas que permiten que dos dispositivos PCI se pongan de acuerdo para usar los 64 bits.
- **Terminales de test (JTAG/*Boundary Scan*):** estas señales se ajustan al estándar IEEE 1149.1 para la definición de procedimientos de test.

ÓRDENES DEL PCI

La actividad del bus consiste en transferencias entre elementos conectados al bus, denominándose maestro al que inicia la transferencia. Cuando un maestro del bus adquiere el control del mismo, determina el tipo de transferencia que se producirá a continuación. Durante la fase de direccionamiento de transferencia, se utilizan las líneas C/BE para indicar el tipo de transferencia. Los tipos de órdenes son:

- Reconocimiento de interrupción
- Ciclo especial
- Lectura de E/S
- Escritura en E/S
- Lectura de memoria

Tabla 3.4. Líneas opcionales del bus PCI.

Denominación	Tipo	Descripción
		Terminales de interrupción
INTA#	o/d	Utilizada para pedir una interrupción.
INTB#	o/d	Utilizada para pedir una interrupción; solo tiene significado en un dispositivo multifunción.
INTC#	o/d	Utilizada para pedir una interrupción; solo tiene significado en un dispositivo multifunción.
INTD#	o/d	Utilizada para pedir una interrupción; solo tiene significado en un dispositivo multifunción.
		Terminales de soporte de caché
SBO#	in/out	(*Snoop Backoff*) Indica un acceso a una línea de caché modificada.
SDONE	in/out	(*Snoop Done*) Indica el estado del módulo de sondeo del bus (*snoop*) en el acceso actual a caché. Se activa cuando el sondeo ha terminado.
		Terminales de extensión a bus de 64 bits
AD[63:32]	t/s	Líneas multiplexadas de direcciones y datos para ampliar el bus a 64 bits.
C/BE[7:4]	t/s	Líneas multiplexadas de órdenes y habilitación de byte. Durante la fase de dirección, las líneas proporcionan las órdenes adicionales del bus. Durante la fase de datos, las líneas indican cual de los cuatro bytes del bus ampliado contiene datos válidos.
REQ64#	s/t/s	Utilizada para pedir una transferencia de 64 bits.
ACK64#	s/t/s	Indica que el dispositivo seleccionado está dispuesto para realizar una transferencia de 64 bits.
PAR64	t/s	Proporciona paridad par para las líneas ampliadas AD y para C/BE un ciclo de reloj después.
		Terminales de test (JTAG/*Boundary Scan*)
TCK	in	Reloj de test. Utilizado para sincronizar la entrada y salida de la información de estado y los datos de test del dispositivo testeado (mediante *Boundary Scan*).
TDI	in	Entrada de test. Utilizada para introducir bit a bit los datos y las instrucciones de test en el dispositivo a testear.
TDO	out	Salida de test. Utilizada para obtener bit a bit los datos y las instrucciones de test desde el dispositivo testeado.
TMS	in	Selector de modo de test. Utilizado para establecer el estado del controlador del puerto de acceso para el test.
TRS#	in	Inicicio de test (test reset). Utilizada para iniciar el controlador del puerto de acceso para el test.

in = Señal de entrada
out = Señal de salida
t/s = Señal de E/S, bidireccional, tri-estado
s/t/s = Señal tri-estado activada solo por un dispositivo en cada momento
o/d = Drenador abierto: permite que varios dispositivos lo compartan como en una OR cableada
= La señal se activa en el nivel inferior de tensión (activa en baja)

- Lectura de línea de memoria
- Lectura múltiple de memoria
- Escritura en memoria
- Escritura e invalidación de memoria
- Lectura de configuración
- Escritura de configuración
- Ciclo de dirección dual

El Reconocimineto de Interrupción es una orden de lectura proporcionada por el dispositivo que actúa como controlador de interrupciones en el bus PCI. Las líneas de direcciones no se utilizan en la fase de direccionamiento, y las líneas de byte activo (*byte enable*) indican el tamaño del identificador de interrupción a devolver.

La orden de ciclo especial se utiliza para iniciar la difusión de un mensaje a uno o más destinos.

Las órdenes de lectura de E/S y escritura en E/S se utilizan para intercambiar datos entre el módulo que inicia la transferencia y un controlador de E/S. Cada dispositivo de E/S tiene su propio espacio de direcciones y las líneas de direcciones se utilizan para indicar un dispositivo concreto y para especificar los datos a trasferir a o desde ese dispositivo. El concepto de direcciones de E/S se explora en el Capítulo 7.

Las órdenes de lectura y escritura en memoria se utilizan para especificar la transferencia de una secuencia de datos, utilizando uno o más ciclos de reloj. La interpretación de estas órdenes depende de si el controlador de memoria del bus PCI utiliza el protocolo PCI para transferencias entre memoria y caché, o no. Si lo utiliza, la transferencia de datos a y desde la memoria normalmente se produce en términos de líneas o bloques de caché [2]. El uso de las tres órdenes de lectura de memoria se resume en la Tabla 3.5. La orden de escritura en memoria se utiliza para transferir datos a memoria

Tabla 3.5 Interpretación de las órdenes de Lectura del bus PCI.

Tipo de orden de lectura	Para memoria transferible a caché	Para memoria no transferible a caché
Lectura de memoria	Secuencia de la mitad o menos de una línea	Secuencia de dos o menos ciclos de transferencia de datos
Lectura de línea de memoria	Secuencia de más de media de línea y menos de tres líneas de caché	Secuencia de tres a doce ciclos de transferencia
Lectura de memoria múltiple	Secuencia de más de tres líneas de caché	Secuencia de más de doce ciclos de transferencia de datos

[2] Los principios fundamentales de la memoria caché se describe en el Capítulo 4; los protocolos de caché basados en buses se describen en el Capítulo 18.

en uno o más ciclos de datos. La orden de escritura e invalidación de memoria transfiere datos a memoria en uno o más ciclos. Además, indica que al menos se ha escrito en una línea de caché. Esta orden permite el funcionamiento de la caché con postescritura (*write back*) en memoria.

Las dos órdenes de configuración permiten que un dispositivo maestro lea y actualice los parámetros de configuración de un dispositivo conectado al bus PCI. Cada dispositivo PCI puede disponer de hasta 256 registros internos utilizados para configurar dicho dispositivo durante la inicialización del sistema.

La orden de ciclo de dirección dual se utiliza por el dispositivo que inicia la transferencia para indicar que está utilizando direcciones de 64 bits.

TRANSFERENCIAS DE DATOS

Toda transferencia de datos en el bus PCI es una transacción única que consta de una fase de direccionamiento y una o más fases de datos. En esta discusión, se ilustra una operación de lectura típica; las operaciones de escritura se producen de forma análoga.

La Figura 3.23 muestra la temporización de una operación de lectura. Todos los eventos se sincronizan en las transiciones de bajada del reloj, cosa que sucede a la mitad de cada ciclo de reloj. Los dispositivos del bus interpretan las líneas del bus en los flancos de subida al comienzo del ciclo de bus. A continuación se describen los eventos significativos señalados en el diagrama:

a) Una vez que el maestro del bus ha obtenido el control del bus, debe iniciar la transacción activando FRAME. Esta línea permanece activa hasta que el maestro está dispuesto para terminar la última fase de datos. El maestro también sitúa la dirección de inicio en el bus de direcciones y la orden de lectura en las líneas C/BE.

b) Al comienzo del ciclo de reloj 2, el dispositivo del que se lee reconocerá su dirección en las líneas AD.

c) El maestro deja libres las líneas AD del bus. En todas las líneas de señal que pueden ser activadas por más de un dispositivo se necesita un ciclo de cambio (indicado por las dos flechas circulares) para que la liberación de las líneas de dirección permita que el bus pueda ser utilizado por el dispositivo de lectura. El maestro cambia la información de las líneas C/BE para indicar cuales de las líneas AD se utilizan para transferir el dato direccionado (de 1 a 4 bytes). El maestro también activa IRDY para indicar que está preparado para recibir el primer dato.

d) El dispositivo de lectura seleccionado activa DEVSEL para indicar que ha reconocido las direcciones y va a responder. Sitúa el dato solicitado en las líneas AD y activa TRDY para indicar que hay un dato válido en el bus.

e) El maestro lee el dato al comienzo del ciclo de reloj 4 y cambia las líneas de habilitación de byte según se necesite para la próxima lectura.

f) En este ejemplo, el dispositivo de lectura necesita algún tiempo para preparar el segundo bloque de datos para la transmisión. Por consiguiente, desactiva TRDY para señalar al maestro que no proporcionará un nuevo dato en el próximo ciclo. En consecuencia, el maestro no lee las líneas de datos al comienzo del quinto ciclo de reloj y no cambia la

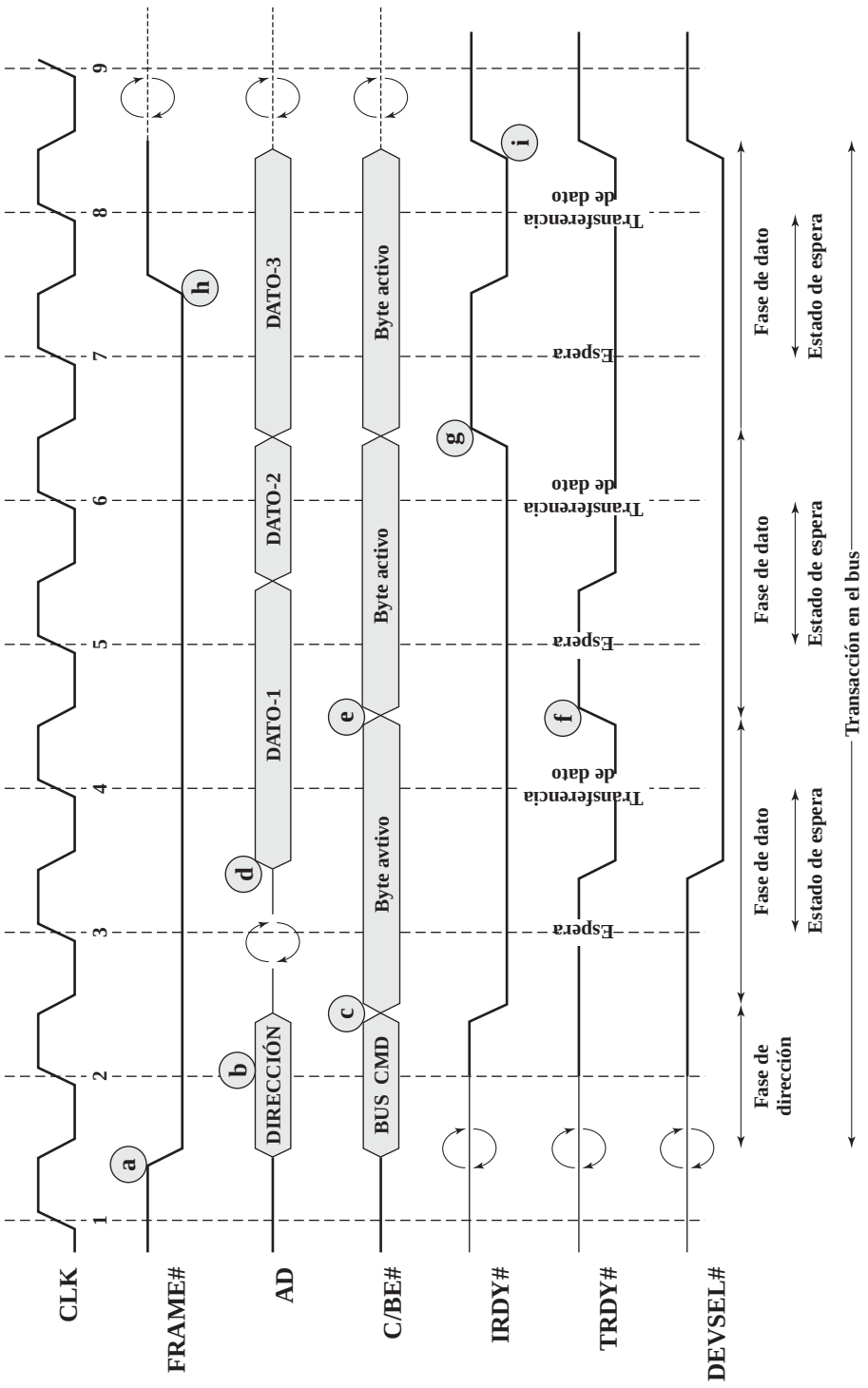


Figura 3.23. Operación de lectura PCI.

señal de habilitación de byte durante ese ciclo. El bloque de datos es leído al comienzo del ciclo de reloj 6.

g) Durante el ciclo 6, el dispositivo de lectura sitúa el tercer dato en el bus. No obstante, en este ejemplo, el maestro todavía no está preparado para leer el dato (por ejemplo, puede tener lleno el registro de almacenamiento temporal). Para indicarlo desactiva IRDY. Esto hará que el dispositivo de lectura mantenga el tercer dato en el bus durante un ciclo de reloj extra.

h) El maestro sabe que el tercer dato es el último, y por eso desactiva FRAME para indicar al dispositivo de lectura que este es el último dato a transferir. Además activa IRDY para indicar que está listo para completar esa transferencia.

i) El maestro desactiva IRDY, haciendo que el bus vuelva a estar libre, y el dispositivo de lectura desactiva TRDY y DEVSEL.

ARBITRAJE

El bus PCI utiliza un esquema de arbitraje centralizado síncrono en el que cada maestro tiene una única señal de petición (REQ) y cesión (GNT) del bus. Estas líneas se conectan a un árbitro central (Figura 3.24) y se utiliza un simple intercambio de las señales de petición y cesión para permitir el acceso al bus.

La especificación PCI no indica un algoritmo particular de arbitraje. El árbitro puede utilizar un procedimiento de primero-en llegar-primero-en servirse, un procedimiento de cesión cíclica (*round-robin*), o cualquier clase de esquema de prioridad. El maestro del PCI establece, para cada transferencia que desee hacer, si tras la fase de dirección sigue una o más fases de datos consecutivas.

La Figura 3.25 es un ejemplo en el que se decide el dispositivo, A o B, al que se cede el bus. Se produce la siguiente secuencia:

a) En algún momento anterior al comienzo del ciclo de reloj 1, A ha activado su señal REQ. El árbitro muestrea esa señal al comienzo del ciclo de reloj 1.

b) Durante el ciclo de reloj 1, B solicita el uso del bus activando su señal REQ.

c) Al mismo tiempo, el árbitro activa GNT-A para ceder el acceso al bus a A.

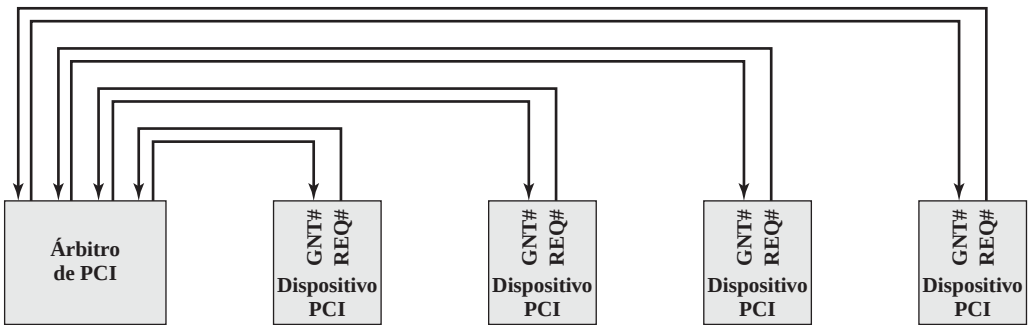


Figura 3.24. Árbitro de bus PCI.

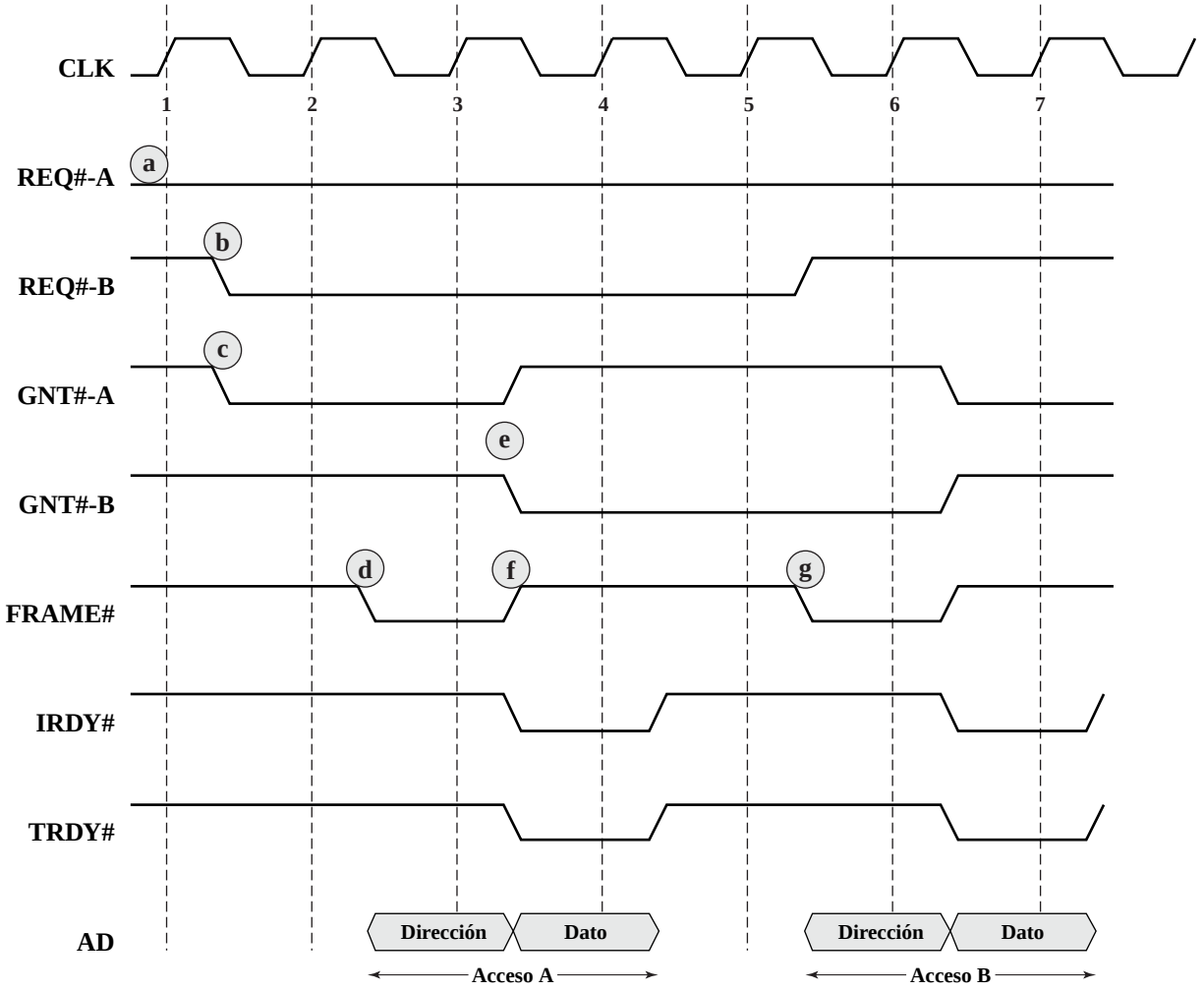


Figura 3.25. Arbitraje del bus PCI entre dos maestros.

d) El maestro del bus A muestrea GNT-A al comienzo del ciclo de reloj 2 y conoce que se le ha cedido el acceso al bus. Además, encuentra IRDY y TRDY desactivados, indicando que el bus está libre. En consecuencia, activa FRAME y coloca la información de dirección en el bus de direcciones y la orden correspondiente en las líneas C/BE (no mostradas). Además mantiene activa REQ-A, puesto que tiene que realizar otra transferencia después de la actual.

e) El árbitro del bus muestrea todas las líneas REQ al comienzo del ciclo 3 y toma la decisión de ceder el bus a B para la siguiente transacción. Entonces activa GNT-B y desactiva GNT-A. B no podrá utilizar el bus hasta que este no vuelva a estar libre.

f) A desactiva FRAME para indicar que la última (y la única) transferencia de dato está en marcha. Pone los datos en el bus de datos y se lo indica al dispositivo destino con IRDY. El dispositivo lee el dato al comienzo del siguiente ciclo de reloj.

g) Al comienzo del ciclo 5, B encuentra IRDY y FRAME desactivados y por consiguiente puede tomar el control del bus activando FRAME. Además desactiva su línea REQ, puesto que solo deseaba realizar una transferencia.

Posteriormente, se cede el acceso al bus al maestro A para que realice su siguiente transferencia.

Hay que resaltar que el arbitraje se produce al mismo tiempo que el maestro del bus actual está realizando una transferencia de datos. Por consiguiente, no se pierden ciclos de bus en realizar el arbitraje. Esto se conoce como *arbitraje oculto o solapado (hidden arbitration)***.**

3.6. LECTURAS Y SITIOS WEB RECOMENDADOS

El libro con la descripción más clara de PCI es [SHAN99]. [ABBO04] también contiene bastante información rigurosa de PCI.

ABBO04 ABBOT, D.: *PCI Bus Desmystified.* New York. Elsevier, 2004.

SHAN99 Shanley, T., y Anderson, D.: *PCI Systems Architecture.* Richardson, TX. Mindshare Press, 1999.



SITIOS WEB RECOMENDADOS

- **PCI Special Interest Group:** información acerca de las especificaciones del bus PCI y productos basados en el mismo
- **PCI Pointers:** enlaces a suministradores y otras fuentes de información sobre PCI.

3.7. PALABRAS CLAVE, CUESTIONES Y PROBLEMAS

PALABRAS CLAVE

anchura del bus	captación de instrucción	interrupción inhabilitada
arbitraje centralizado	ciclo de instrucción	registro de datos de memoria (MBR)
arbitraje del bus	ejecución de instrucción	registro de dirección de memoria (MAR)
arbitraje distribuido	gestor de interrupciones	rutina de servicio de interrupción
bus	interconexión de componente periférico (*peripheral component interconect*, PCI)	temporización asíncrona
bus de datos	interrupción	temporización síncrona
bus de direcciones		
bus de sistema		

CUESTIONES

3.1. ¿Qué tipos generales de funciones especifican las instrucciones de un computador?

3.2. Enumere y defina brevemente los estados posibles que determinan la ejecución de una instrucción.

3.3. Enumere y defina brevemente dos aproximaciones para gestionar las instrucciones múltiples.

3.4. ¿Qué tipos de transferencias debe permitir la estructura de interconexión (por ejemplo, un bus) de un computador?

3.5. ¿Qué ventajas tiene una arquitectura de varios buses frente a otra de bus único?

3.6. Enumere y defina brevemente los grupos de líneas de señal para el bus PCI.

PROBLEMAS

3.1. La máquina hipotética de la Figura 3.4 también tiene dos instrucciones de E/S:

0011 = Cargar AC desde E/S
0111 = Almacenar AC en E/S

En estos casos, la dirección de 12 bits identifica un dispositivo concreto de E/S. Muestre la ejecución del programa (utilizando el formato de la Figura 3.5) para el siguiente programa:

1. Cargar AC desde el dispositivo 5.
2. Sumar el contenido de la posición de memoria 940.
3. Almacenar AC en el dispositivo 6.

Asuma que el siguiente valor obtenido desde el dispositivo 5 es 3 y que la posición 940 almacena el valor 2.

3.2. La ejecución de un programa que se proporciona en la Figura 3.5 se describe en el texto utilizando seis etapas. Amplíe esta descripción indicando la forma en que se utilizan los registros MAR y MBR.

3.3. Considere un hipotético microprocesador de 32 bits cuyas instrucciones de 32 bits están compuestas por dos campos: el primer byte contiene el código de operación (codop) y los restantes un operando inmediato o una dirección de operando.

(a) ¿Cuál es la máxima capacidad de memoria (en bytes) direccionable directamente?

(b) Discuta el impacto que se produciría en la velocidad del sistema si el microprocesador tiene:

1. Un bus de dirección local de 32 bits y un bus de datos local de 16 bits, o
2. un bus de dirección local de 16 bits y un bus de datos local de 16 bits.

(c) ¿Cuántos bits necesitan el contador de programa y el registro de instrucción?

3.4. Considere un microprocesador hipotético que genera direcciones de 16 bits (por ejemplo, suponga que el contador de programa y el registro de dirección son de 16 bits) y tiene un bus de datos de 16 bits.

(a) ¿Cuál es el máximo espacio de direcciones de memoria al que el procesador puede acceder directamente si está conectado a una «memoria de 16 bits»?

(b) ¿Cuál es el máximo espacio de direcciones de memoria al que el procesador puede acceder directamente si está conectado a una «memoria de 8 bits»?

(c) ¿Qué características de la arquitectura permitirán a este procesador acceder a un «espacio de E/S» separado?

(d) Si una instrucción de entrada o de salida pueden especificar un número de puerto de E/S de 8 bits, ¿cuántos puertos de E/S de 8 bits puede soportar el microprocesador? (cuántos puertos de E/S de 16 bits? Explíquelo.

3.5. Considere un microprocesador de 32 bits, con un bus externo de 16 bits, y con una entrada de reloj de 8 MHz. Asuma que el procesador tiene un ciclo de bus cuya duración mínima es igual a cuatro ciclos de reloj. ¿Cuál es la velocidad de transferencia máxima que puede sostener el microprocesador? Para

incrementar sus prestaciones, ¿sería mejor hacer que su bus externo de datos sea de 32 bits o doblar la frecuencia de reloj que se suministra al microprocesador? Establezca las suposiciones que considere y explíquelo. *Ayuda:* determine el número de bytes que pueden transferirse por ciclo de bus.

3.6. Considere un computador que posee un módulo de E/S controlando un teletipo con teclado e impresora. La CPU tiene los siguientes registros conectados directamente al bus de sistema:

INPR:	Registro de Entrada - 8 bits
OUTPR:	Registro de Salida - 8 bits
FGI:	Indicador (Flag) de Entrada - 1 bit
FGO:	Indicador (Flag) de Salida - 1 bit
IEN:	Habilitación de Interrupción - 1 bit

La entrada desde el teclado y la salida a la impresora del teletipo están controlados por el módulo de E/S. El teletipo es capaz de codificar un símbolo alfanumérico mediante una palabra de 8 bits y decodificar una palabra de 8 bits en un símbolo alfanumérico.

(a) Describa cómo el procesador, utilizando el primero de los cuatro registros enumerados anteriormente, puede realizar una E/S con el teletipo.

(b) Describa cómo se puede realizar dicha operación de manera más eficiente si además se utiliza IEN.

3.7. Considere dos microprocesadores con buses de datos externos de 8 y 16 bits, respectivamente. Los dos procesadores son idénticos en todo lo demás y sus ciclos de bus son iguales.

(a) Suponiendo que todas las instrucciones y operandos son de dos bytes, ¿en qué factor difieren las velocidades de transferencia de los dos microprocesadores?

(b) ¿cuál sería la respuesta si en la mitad de los casos los operandos y las instrucciones son de un byte?

3.8. La Figura 3.26 muestra un esquema de arbitraje distribuido que puede utilizarse con un bus actualmente obsoleto denominado Multibus I. Los módulos se conectan en cadena (*daisy-chain*) según su orden de prioridad. El módulo que está más a la izquierda en el diagrama recibe una *prioridad de bus* constante a través de la señal BPRN indicando que ningún módulo de mayor prioridad solicita el bus. Si el módulo no desea utilizar el bus activa su línea de *salida de prioridad del bus (BPRO)***.** Al comienzo de un ciclo de reloj, cualquier módulo puede pedir el control del bus poniendo en baja su línea BPRO. Esto hace que pase a baja la línea BPRN del siguiente módulo de la cadena, que a su vez debe poner a baja su línea BPRO. Así, la señal se propaga a lo largo de la cadena. Al final de esta respuesta en cadena, únicamente hay un módulo cuya BPRN está activada y cuya BPRO no. Este módulo es el que tiene prioridad. Si, al comienzo de un ciclo de bus, el bus no está ocupado (BUSY no activada), el módulo que tiene prioridad puede tomar el control del bus activando la señal BUSY.

Se necesita un cierto intervalo de tiempo para que la señal BPR se propague desde el módulo de prioridad más alta hasta el de más baja. ¿Debe este tiempo ser menor que el ciclo de reloj? Explíquelo.

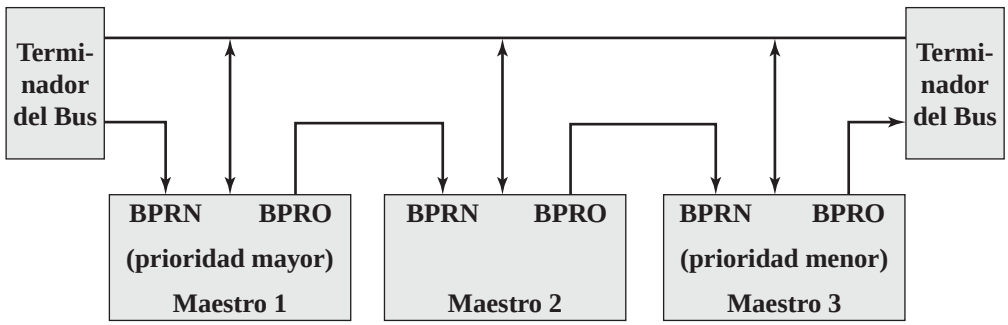


Figura 3.26. Arbitraje distribuido en el Multibus I.

3.9. El bus VAX SBI utiliza un esquema de arbitraje distribuido síncrono. Cada dispositivo SBI (es decir, procesador, memoria, módulo de E/S) tiene una prioridad única y se le asigna una única línea de solicitud de transferencia (TR, de *Transfer Request*). El SBI tiene 16 de dichas líneas (TR0,TR1...TR15), siendo TR0 la de prioridad más elevada. Cuando un dispositivo quiere utilizar el bus, realiza una reserva de un ciclo de bus futuro activando su línea TR durante el ciclo de bus en curso. Al final del ciclo en curso, cada dispositivo con una reserva pendiente examina las líneas TR; el que tiene más alta prioridad utiliza el siguiente ciclo de bus.

Como máximo se pueden conectar al bus 17 dispositivos. El dispositivo con prioridad 16 no tiene línea TR. ¿Por qué?

3.10. En el bus VAX SBI el dispositivo de prioridad más baja usualmente tiene el tiempo de espera medio más bajo. Por esta razón, a la CPU se le da usualmente la prioridad más baja en el SBI. ¿Por qué el dispositivo de prioridad 16 tiene normalmente el menor tiempo de espera medio? ¿En qué circunstancias esto no sería cierto?

3.11. En una operación de lectura síncrona (Figura 3.19), el módulo de memoria debe situar el dato en el bus con la suficiente antelación al flanco de bajada de la señal de lectura para asegurar que las señales se han estabilizado. Si el bus de un microprocesador utiliza un bus de 10 MHz y la señal de lectura empieza a caer en la segunda mitad de T_3:

(a) Determine la longitud del ciclo de instrucción de una lectura de memoria.
(b) ¿Cuándo debería situarse el dato de memoria en el bus como muy tarde? Suponga un tiempo de 20 ns para la estabilización de las líneas de datos.

3.12, Considere un microprocesador con la temporización de lectura de memoria de la Figura 3.19. Después de analizar la situación, un diseñador determina que la memoria no es capaz de proporcionar el dato a leer en un tiempo inferior a 180 ns.

(a) ¿Cuántos estados de espera (ciclos de reloj) deben insertarse para conseguir una lectura adecuada de memoria si el reloj del bus es de 8 MHz?
(b) Para utilizar los estados de espera se añade una línea de estado denominada «Ready». Una vez que el procesador ha activado la señal de lectura debe esperar a que la línea «Ready» se active para intentar leer el dato de las líneas del bus. ¿Durante qué intervalo de tiempo hay que mantener la señal «Ready» en baja para que el procesador inserte los ciclos de espera necesarios?

3.13. Un microprocesador utiliza la temporización que se muestra en la Figura 3.19 para la la escritura en memoria. Su fabricante especifica que la extensión temporal de la señal de escritura viene dada por T-50, donde T es el ciclo de reloj en nanosegundos.

(a) ¿Cuál es la extensión de la señal de escritura si la frecuencia de reloj del bus es de 5 MHz?
(b) La hoja de características del microprocesador especifica que, tras el flanco de bajada de la señal de escritura, los datos se mantienen válidos durante 20 ns. ¿Durante cuánto tiempo se mantienen válidos los datos que se van a escribir en memoria?
(c) ¿Cuántos estados de espera deberían insertarse si la memoria necesita que los datos se mantengan válidos durante al menos 190 ns.?

3.14. Un microprocesador tiene una instrucción de incremento de memoria directo que suma 1 al valor almacenado en una posición de memoria. La instrucción tiene cinco etapas: captación del código de operación (4 ciclos de reloj), captación de la dirección del operando (3 ciclos), captación del operando (3 ciclos), suma de 1 al operando (3 ciclos), y almacenar el operando (3 ciclos).

(a) ¿En qué porcentaje se incrementa la direción de la instrucción si hemos de insertar dos ciclos de bus de espera en cada operación de lectura o escritura de memoria?
(b) Repita el apartado anterior para el caso en que la operación de incremento necesite 13 ciclos en lugar de 3.

3.15. El microprocesador 8088 de Intel tiene una temporización de bus para la lectura similar a la de la Figura 3.19, pero necesita cuatro ciclos de reloj del procesador. El dato se mantiene válido en el bus

durante un tiempo que se extiende hasta el cuarto ciclo de reloj del procesador. Considere que la frecuencia de reloj del procesador es de 8 MHz.

(a) ¿Cuál es la velocidad máxima de transferencia de datos?

(b) ¿Y en el caso de que haya que insertar un estado de espera por byte transferido?

3.16. El microprocesador 8086 de Intel es un procesador similar en muchos aspectos al microprocesador de 8 bits 8088. El 8086 utiliza un bus de 16 bits que puede transferir 2 bytes a la vez, siendo par la dirección del byte menos significativo. Sin embargo, el 8086 permite palabras de operandos alineadas tanto en direcciones pares (*even-aligned*) como impares (*odd-aligned*). Si se hace referencia a una palabra alineada en una dirección impar se necesitan dos ciclos, cada uno de cuatro ciclos de reloj de bus, para transferir la palabra.Considere una instrucción del 8086 que utiliza dos operandos de 16 bits. ¿Cuánto tiempo se tarda en captar los dos operandos según las distintas posibilidades? Considere que la frecuencia de reloj es de 4 MHz y no hay estados de espera.

3.17. Considere un microprocesador de 32 bits cuyo ciclo de bus tiene la misma duración que el de un procesador de 16 bits. Asuma que, en promedio, el 20 por ciento de los operandos e instrucciones son de 32 bits, el 40 por ciento son de 16 bits, y el 40 por ciento son de solo 8 bits. Calcule la mejora que se consigue en la captación de instrucciones y operandos con el microprocesador de 32 bits.

3.18. El microprocesador del problema 3.14 inicia una etapa de captación de operando en la instrucción de incremento de memoria directo al mismo tiempo que el teclado activa una línea de petición de interrupción. ¿Después de cuánto tiempo entra el procesador en el ciclo de procesamiento de interrupción? Considere una frecuencia de reloj de 10 MHz para el bus.

3.19. Dibuje y explique un diagrama de tiempos para una operación de escritura en un bus PCI (similar a la Figura 3.23).

APÉNDICE 3A. DIAGRAMAS DE TIEMPO

En este capítulo, los diagramas de tiempo se utilizan para ilustrar las secuencias de eventos y las dependencias entre eventos. Para el lector que no esté familiarizado con los diagramas de tiempo, este apéndice proporciona una breve explicación.

La comunicación entre los dispositivos conectados a un bus se produce a través de un conjunto de líneas capaces de transmitir señales. Se pueden transmitir dos niveles diferentes de señal (niveles de tensión), representando un 0 binario y un 1 binario. Un diagrama de tiempo muestra el nivel de la señal en una línea en función del tiempo (Figura 3.27a). Por convención, el nivel 1 de la señal binaria se representa por un nivel más alto que el nivel binario 0. Usualmente, el 0 binario es el valor por defecto (o implícito). Es decir, si no se está transmitiendo un dato ni ninguna otra señal, entonces el nivel en la línea es el que representa al 0 binario. Una transición de 0 a 1 en la señal se denomina frecuentemente *flanco de subida de la señal (leading edge)*; una transición de 1 a 0 se denomina *flanco de bajada (trailing edge)*. Por claridad, a menudo las transiciones en las señales se dibujan como si ocurrieran instantáneamente. De hecho, una transición requiere un tiempo no nulo, pero este tiempo de transición es usualmente pequeño comparado con la duración de un nivel en la señal. En un diagrama de tiempos, puede ocurrir que transcurra una cantidad de tiempo variable, o en todo caso irrelevante, entre dos eventos de interés. Esto se representa con un corte en la línea de tiempo.

A veces, la señales se representan agrupadas (Figura 3.27b). Por ejemplo, si se tranfiere un dato de un byte cada vez, se necesitan ocho líneas. Generalmente, no es esencial conocer el valor exacto que se transfiere en dicho grupo sino más bien si las señales están presentes o no.

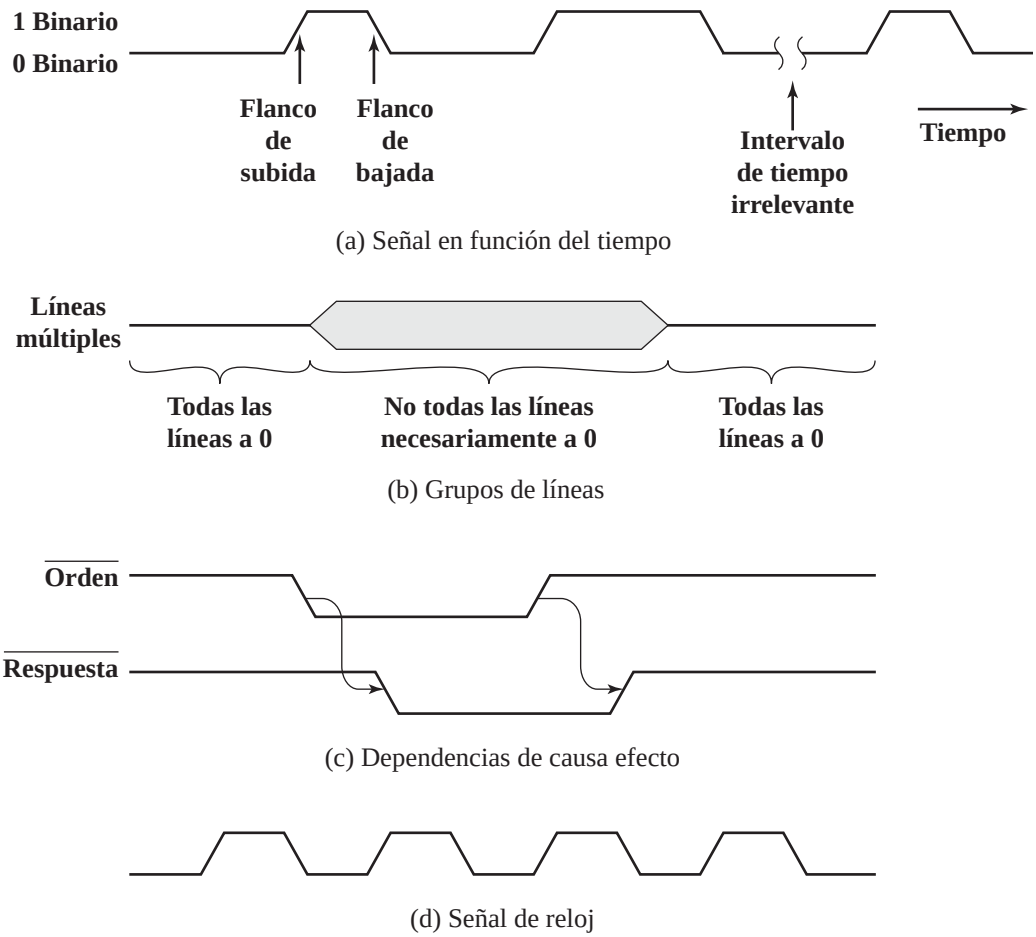


Figura 3.27. Diagramas de tiempo.

Una transición de señal en una línea provoca que un dispositivo conectado ocasione cambios de señal en otras líneas. Por ejemplo, si un módulo de memoria detecta una señal de control de lectura (transición a 0 o a 1), situará las señales correspondientes a los datos en las líneas de datos. Estas relaciones de causa-efecto dan lugar a secuencias de sucesos. Las flechas se utilizan para mostrar estas dependencias en los diagramas de tiempo (Figura 3.27c).

En la Figura 3.27c, una línea sobre el nombre de la señal indica que la señal está activa cuando está en baja. Por ejemplo, $\overline{orden}$ está activa a 0 voltios. Esto significa que $\overline{orden} = 0$ se interpreta como un 1 lógico.

A menudo el bus de sistema contiene una línea de reloj. Un reloj electrónico se conecta a la línea de reloj y proporciona una secuencia repetitiva y repetitiva de transiciones (Figura 3.27d). Puede haber otros eventos sincronizados con la señal de reloj.

1

INTRODUCCIÓN

Una computadora moderna consta de uno o más procesadores, una memoria principal, discos, impresoras, un teclado, un ratón, una pantalla o monitor, interfaces de red y otros dispositivos de entrada/salida. En general es un sistema complejo. Si todos los programadores de aplicaciones tuvieran que comprender el funcionamiento de todas estas partes, no escribirían código alguno. Es más: el trabajo de administrar todos estos componentes y utilizarlos de manera óptima es una tarea muy desafiante. Por esta razón, las computadoras están equipadas con una capa de software llamada **sistema operativo**, cuyo trabajo es proporcionar a los programas de usuario un modelo de computadora mejor, más simple y pulcro, así como encargarse de la administración de todos los recursos antes mencionados. Los sistemas operativos son el tema de este libro.

La mayoría de los lectores habrán tenido cierta experiencia con un sistema operativo como Windows, Linux, FreeBSD o Mac OS X, pero las apariencias pueden ser engañosas. El programa con el que los usuarios generalmente interactúan se denomina **shell**, cuando está basado en texto, y **GUI** (*Graphical User Interface*; Interfaz gráfica de usuario**)** cuando utiliza elementos gráficos o iconos. En realidad no forma parte del sistema operativo, aunque lo utiliza para llevar a cabo su trabajo.

La figura 1-1 presenta un esquema general de los componentes principales que aquí se analizan. En la parte inferior se muestra el hardware, que consiste en circuitos integrados (chips), tarjetas, discos, un teclado, un monitor y objetos físicos similares. Por encima del hardware se encuentra el software. La mayoría de las computadoras tienen dos modos de operación: modo kernel y modo usuario. El sistema operativo es la pieza fundamental del software y se ejecuta en **modo kernel** (también conocido como **modo supervisor**). En este modo, el sistema operativo tiene acceso

completo a todo el hardware y puede ejecutar cualquier instrucción que la máquina sea capaz de ejecutar. El resto del software se ejecuta en **modo usuario**, en el cual sólo un subconjunto de las instrucciones de máquina es permitido. En particular, las instrucciones que afectan el control de la máquina o que se encargan de la E/S (entrada/salida) están prohibidas para los programas en modo usuario. Volveremos a tratar las diferencias entre el modo kernel y el modo usuario repetidamente a lo largo de este libro.

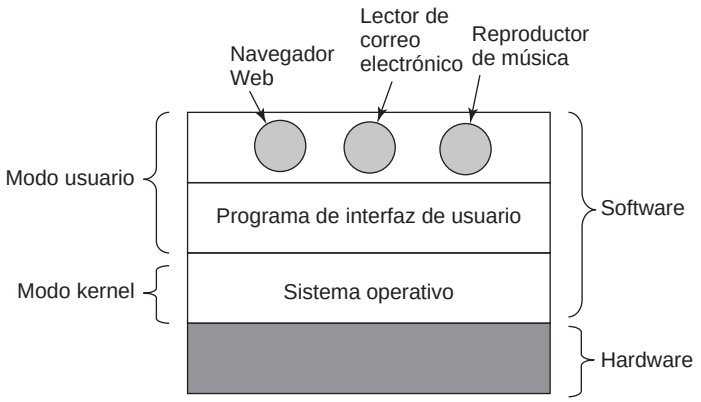


Figura 1-1. Ubicación del sistema operativo.

El programa de interfaz de usuario, shell o GUI, es el nivel más bajo del software en modo usuario y permite la ejecución de otros programas, como un navegador Web, lector de correo electrónico o reproductor de música. Estos programas también utilizan en forma intensiva el sistema operativo.

La ubicación del sistema operativo se muestra en la figura 1-1. Se ejecuta directamente sobre el hardware y proporciona la base para las demás aplicaciones de software.

Una distinción importante entre el sistema operativo y el software que se ejecuta en modo usuario es que, si a un usuario no le gusta, por ejemplo, su lector de correo electrónico, es libre de conseguir otro o incluso escribir el propio si así lo desea; sin embargo, no es libre de escribir su propio manejador de interrupciones de reloj, que forma parte del sistema operativo y está protegido por el hardware contra cualquier intento de modificación por parte de los usuarios.

Algunas veces esta distinción no es clara en los sistemas integrados (a los que también se conoce como integrados o incrustados, y que podrían no tener modo kernel) o en los sistemas interpretados (como los sistemas operativos basados en Java que para separar los componentes utilizan interpretación y no el hardware).

Además, en muchos sistemas hay programas que se ejecutan en modo de usuario, pero ayudan al sistema operativo o realizan funciones privilegiadas. Por ejemplo, a menudo hay un programa que permite a los usuarios cambiar su contraseña. Este programa no forma parte del sistema operativo y no se ejecuta en modo kernel, pero sin duda lleva a cabo una función delicada y tiene que proteger-

se de una manera especial. En ciertos sistemas, la idea se lleva hasta el extremo y partes de lo que tradicionalmente se considera el sistema operativo (por ejemplo, el sistema de archivos) se ejecutan en el espacio del usuario. En dichos sistemas es difícil trazar un límite claro. Todo lo que se ejecuta en modo kernel forma, sin duda, parte del sistema operativo, pero podría decirse que algunos programas que se ejecutan fuera de este modo también forman parte del mismo sistema, o por lo menos están estrechamente asociados a él.

Los sistemas operativos difieren de los programas de usuario (es decir, de aplicación) en varias cuestiones además del lugar en el que residen. En particular, son enormes, complejos y de larga duración. El código fuente de un sistema operativo como Linux o Windows contiene cerca de cinco millones de líneas de código. Para tener una idea de lo que esto significa, considere el trabajo de imprimir cinco millones de líneas en un formato de libro: con 50 líneas por página y 1000 páginas por volumen, se requerirían 100 volúmenes para listar un sistema operativo de este tamaño; es decir, todo un librero. Imagine el lector que tiene un trabajo como encargado de dar mantenimiento a un sistema operativo y que en su primer día su jefe le presenta un librero con el código y le dice: "Apréndase todo esto". Y ésta sólo sería la parte que se ejecuta en el kernel. Los programas de usuario como la interfaz gráfica, las bibliotecas y el software de aplicación básico (como el Explorador de Windows) pueden abarcar fácilmente de 10 a 20 veces esa cantidad.

En este punto, el lector debe tener una idea clara de por qué los sistemas operativos tienen una larga vida: es muy difícil escribir uno y, por lo tanto, el propietario se resiste a tirarlo y empezar de nuevo. En vez de ello, evolucionan durante periodos extensos. Windows 95/98/Me es, esencialmente, un sistema operativo distinto de Windows NT/2000/XP/Vista, su sucesor. Tienen una apariencia similar para los usuarios, ya que Microsoft se aseguró bien de ello, sin embargo, tuvo muy buenas razones para deshacerse de Windows 98, las cuales describiremos cuando estudiemos Windows con detalle en el capítulo 11.

El otro ejemplo principal que utilizaremos a lo largo de este libro (además de Windows) es UNIX, con sus variantes y clones. También ha evolucionado a través de los años con versiones tales como System V, Solaris y FreeBSD que se derivan del sistema original, mientras que Linux tiene una base de código nueva, modelada estrechamente de acuerdo con UNIX y altamente compatible con él. Utilizaremos ejemplos de UNIX a lo largo de este libro y analizaremos Linux con detalle en el capítulo 10.

En este capítulo hablaremos brevemente sobre varios aspectos clave de los sistemas operativos, incluyendo en síntesis qué son, cuál es su historia, cuáles son los tipos que existen, algunos de los conceptos básicos y su estructura. En capítulos posteriores volveremos a hablar sobre muchos de estos tópicos importantes con más detalle.

1.1 ¿QUÉ ES UN SISTEMA OPERATIVO?

Es difícil definir qué es un sistema operativo aparte de decir que es el software que se ejecuta en modo kernel (además de que esto no siempre es cierto). Parte del problema es que los sistemas operativos realizan dos funciones básicas que no están relacionadas: proporcionar a los programadores de aplicaciones (y a los programas de aplicaciones, naturalmente) un conjunto abstracto de recursos simples, en vez de los complejos conjuntos de hardware; y administrar estos recursos de hard-

ware. Dependiendo de quién se esté hablando, el lector podría escuchar más acerca de una función o de la otra. Ahora analizaremos ambas.

1.1.1 El sistema operativo como una máquina extendida

La **arquitectura** (conjunto de instrucciones, organización de memoria, E/S y estructura de bus) de la mayoría de las computadoras a nivel de lenguaje máquina es primitiva y compleja de programar, en especial para la entrada/salida. Para hacer este punto más concreto, considere la forma en que se lleva a cabo la E/S de disco flexible mediante los dispositivos controladores (*device controllers*) compatibles NEC PD765 que se utilizan en la mayoría de las computadoras personales basadas en Intel (a lo largo de este libro utilizaremos los términos "disco flexible" y "diskette" indistintamente). Utilizamos el disco flexible como un ejemplo debido a que, aunque obsoleto, es mucho más simple que un disco duro moderno. El PD765 tiene 16 comandos, cada uno de los cuales se especifica mediante la carga de 1 a 9 bytes en un registro de dispositivo. Estos comandos son para leer y escribir datos, desplazar el brazo del disco y dar formato a las pistas, así como para inicializar, detectar, restablecer y recalibrar el dispositivo controlador y las unidades.

Los comandos más básicos son `read` y `write` (lectura y escritura), cada uno de los cuales requiere 13 parámetros, empaquetados en 9 bytes. Estos parámetros especifican elementos tales como la dirección del bloque de disco a leer, el número de sectores por pista, el modo de grabación utilizado en el medio físico, el espacio de separación entre sectores y lo que se debe hacer con una marca de dirección de datos eliminados. Si el lector no comprende estos tecnicismos, no se preocupe: ése es precisamente el punto, pues se trata de algo bastante oscuro. Cuando la operación se completa, el chip del dispositivo controlador devuelve 23 campos de estado y error, empaquetados en 7 bytes. Como si esto no fuera suficiente, el programador del disco flexible también debe estar constantemente al tanto de si el motor está encendido o apagado. Si el motor está apagado, debe encenderse (con un retraso largo de arranque) para que los datos puedan ser leídos o escritos. El motor no se debe dejar demasiado tiempo encendido porque se desgastará. Por lo tanto, el programador se ve obligado a lidiar con el problema de elegir entre tener retrasos largos de arranque o desgastar los discos flexibles (y llegar a perder los datos).

Sin entrar en los detalles *reales*, debe quedar claro que el programador promedio tal vez no desee involucrarse demasiado con la programación de los discos flexibles (o de los discos duros, que son aún más complejos). En vez de ello, lo que desea es una abstracción simple de alto nivel que se encargue de lidiar con el disco. En el caso de los discos, una abstracción común sería que el disco contiene una colección de archivos con nombre. Cada archivo puede ser abierto para lectura o escritura, después puede ser leído o escrito y, por último, cerrado. Los detalles, tales como si la grabación debe utilizar o no la modulación de frecuencia y cuál es el estado del motor en un momento dado, no deben aparecer en la abstracción que se presenta al programador de aplicaciones.

La abstracción es la clave para lidiar con la complejidad. Las buenas abstracciones convierten una tarea casi imposible en dos tareas manejables. La primera de éstas es definir e implementar las abstracciones; la segunda, utilizarlas para resolver el problema en cuestión. Una abstracción que casi cualquier usuario de computadora comprende es el archivo: es una pieza útil de información, como una fotografía digital, un mensaje de correo electrónico almacenado o una página Web. Es más fácil lidiar con fotografías, correos electrónicos y páginas Web que con los detalles de los discos,

como en el caso del disco flexible descrito. El trabajo del sistema operativo es crear buenas abstracciones para después implementar y administrar los objetos abstractos entonces creados. En este libro hablaremos mucho acerca de las abstracciones, dado que son claves para comprender los sistemas operativos.

Este punto es tan importante que vale la pena repetirlo en distintas palabras. Con el debido respeto a los ingenieros industriales que diseñaron la Macintosh, el hardware es feo. Los procesadores, memorias, discos y otros dispositivos reales son muy complicados y presentan interfaces difíciles, enredadas, muy peculiares e inconsistentes para las personas que tienen que escribir software para utilizarlos. Algunas veces esto se debe a la necesidad de tener compatibilidad con el hardware anterior; otras, a un deseo de ahorrar dinero, y otras más, a que los diseñadores de hardware no tienen idea (o no les importa) qué tan grave es el problema que están ocasionando para el software. Una de las principales tareas del sistema operativo es ocultar el hardware y presentar a los programas (y a sus programadores) abstracciones agradables, elegantes, simples y consistentes con las que puedan trabajar. Los sistemas operativos ocultan la parte fea con la parte hermosa, como se muestra en la figura 1-2.

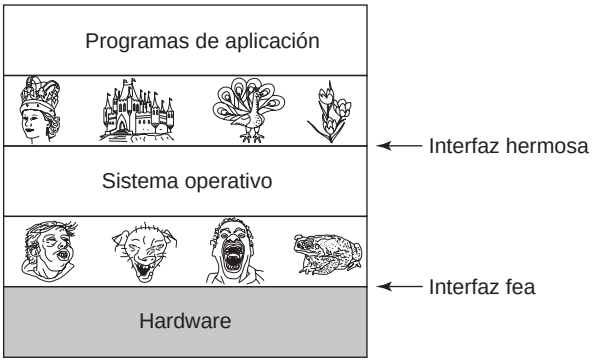


Figura 1-2. Los sistemas operativos ocultan el hardware feo con abstracciones hermosas.

Hay que recalcar que los verdaderos clientes del sistema operativo son los programas de aplicación (a través de los programadores de aplicaciones, desde luego). Son los que tratan directamente con el sistema operativo y sus abstracciones. En contraste, los usuarios finales tienen que lidiar con las abstracciones que proporciona la interfaz de usuario, ya sea un shell de línea de comandos o una interfaz gráfica. Aunque las abstracciones en la interfaz de usuario pueden ser similares a las que proporciona el sistema operativo, éste no siempre es el caso. Para aclarar este punto, considere el escritorio normal de Windows y el indicador para comandos orientado a texto. Ambos son programas que se ejecutan en el sistema operativo Windows y utilizan las abstracciones que este sistema proporciona, pero ofrecen interfaces de usuario muy distintas. De manera similar, un usuario de Linux que ejecuta Gnome o KDE ve una interfaz muy distinta a la que ve un usuario de Linux que trabaja directamente encima del Sistema X Window subyacente (orientado a texto), pero las abstracciones del sistema operativo subyacente son las mismas en ambos casos.

En este libro estudiaremos detalladamente las abstracciones que se proporcionan a los programas de aplicación, pero trataremos muy poco acerca de las interfaces de usuario, que es un tema bastante extenso e importante, pero que sólo está relacionado con la periferia de los sistemas operativos.

1.1.2 El sistema operativo como administrador de recursos

El concepto de un sistema operativo cuya función principal es proporcionar abstracciones a los programas de aplicación responde a una perspectiva de arriba hacia abajo. La perspectiva alterna, de abajo hacia arriba, sostiene que el sistema operativo está presente para administrar todas las piezas de un sistema complejo. Las computadoras modernas constan de procesadores, memorias, temporizadores, discos, ratones, interfaces de red, impresoras y una amplia variedad de otros dispositivos. En la perspectiva alterna, el trabajo del sistema operativo es proporcionar una asignación ordenada y controlada de los procesadores, memorias y dispositivos de E/S, entre los diversos programas que compiten por estos recursos.

Los sistemas operativos modernos permiten la ejecución simultánea de varios programas. Imagine lo que ocurriría si tres programas que se ejecutan en cierta computadora trataran de imprimir sus resultados en forma simultánea en la misma impresora. Las primeras líneas de impresión podrían provenir del programa 1, las siguientes del programa 2, después algunas del programa 3, y así en lo sucesivo: el resultado sería un caos. El sistema operativo puede imponer orden al caos potencial, guardando en búferes en disco toda la salida destinada para la impresora. Cuando termina un programa, el sistema operativo puede entonces copiar su salida, previamente almacenada, del archivo en disco a la impresora, mientras que al mismo tiempo el otro programa puede continuar generando más salida, ajeno al hecho de que la salida en realidad no se está enviando a la impresora todavía.

Cuando una computadora (o red) tiene varios usuarios, la necesidad de administrar y proteger la memoria, los dispositivos de E/S y otros recursos es cada vez mayor; de lo contrario, los usuarios podrían interferir unos con otros. Además, los usuarios necesitan con frecuencia compartir no sólo el hardware, sino también la información (archivos o bases de datos, por ejemplo). En resumen, esta visión del sistema operativo sostiene que su tarea principal es llevar un registro de qué programa está utilizando qué recursos, de otorgar las peticiones de recursos, de contabilizar su uso y de mediar las peticiones en conflicto provenientes de distintos programas y usuarios.

La administración de recursos incluye el **multiplexaje** (compartir) de recursos en dos formas distintas: en el tiempo y en el espacio. Cuando un recurso se multiplexa en el tiempo, los distintos programas o usuarios toman turnos para utilizarlo: uno de ellos obtiene acceso al recurso, después otro, y así en lo sucesivo. Por ejemplo, con sólo una CPU y varios programas que desean ejecutarse en ella, el sistema operativo primero asigna la CPU a un programa y luego, una vez que se ha ejecutado por el tiempo suficiente, otro programa obtiene acceso a la CPU, después otro, y en un momento dado el primer programa vuelve a obtener acceso al recurso. La tarea de determinar cómo se multiplexa el recurso en el tiempo (quién sigue y durante cuánto tiempo) es responsabilidad del sistema operativo. Otro ejemplo de multiplexaje en el tiempo es la compartición de la impresora. Cuando hay varios trabajos en una cola de impresión, para imprimirlos en una sola impresora, se debe tomar una decisión en cuanto a cuál trabajo debe imprimirse a continuación.

El otro tipo de multiplexaje es en el espacio. En vez de que los clientes tomen turnos, cada uno obtiene una parte del recurso. Por ejemplo, normalmente la memoria principal se divide entre varios programas en ejecución para que cada uno pueda estar residente al mismo tiempo (por ejemplo, para poder tomar turnos al utilizar la CPU). Suponiendo que hay suficiente memoria como para contener varios programas, es más eficiente contener varios programas en memoria a la vez, en vez de proporcionar a un solo programa toda la memoria, en especial si sólo necesita una pequeña fracción. Desde luego que esto genera problemas de equidad y protección, por ejemplo, y corresponde al sistema operativo resolverlos. Otro recurso que se multiplexa en espacio es el disco duro. En muchos sistemas, un solo disco puede contener archivos de muchos usuarios al mismo tiempo. Asignar espacio en disco y llevar el registro de quién está utilizando cuáles bloques de disco es una tarea típica de administración de recursos común del sistema operativo.

1.2 HISTORIA DE LOS SISTEMAS OPERATIVOS

Los sistemas operativos han ido evolucionando a través de los años. En las siguientes secciones analizaremos brevemente algunos de los hitos más importantes. Como los sistemas operativos han estado estrechamente relacionados a través de la historia con la arquitectura de las computadoras en las que se ejecutan, analizaremos generaciones sucesivas de computadoras para ver cómo eran sus sistemas operativos. Esta vinculación de generaciones de sistemas operativos con generaciones de computadoras es un poco burda, pero proporciona cierta estructura donde de cualquier otra forma no habría.

La progresión que se muestra a continuación es en gran parte cronológica, aunque el desarrollo ha sido un tanto accidentado. Cada fase surgió sin esperar a que la anterior terminara completamente. Hubo muchos traslapes, sin mencionar muchos falsos inicios y callejones sin salida. El lector debe tomar esto como guía, no como la última palabra.

La primera computadora digital verdadera fue diseñada por el matemático inglés Charles Babbage (de 1792 a 1871). Aunque Babbage gastó la mayor parte de su vida y fortuna tratando de construir su “máquina analítica”, nunca logró hacer que funcionara de manera apropiada, debido a que era puramente mecánica y la tecnología de su era no podía producir las ruedas, engranes y dientes con la alta precisión que requería. Por supuesto, la máquina analítica no tenía un sistema operativo.

Como nota histórica interesante, Babbage se dio cuenta de que necesitaba software para su máquina analítica, por lo cual contrató a una joven llamada Ada Lovelace, hija del afamado poeta británico Lord Byron, como la primera programadora del mundo. El lenguaje de programación Ada® lleva su nombre.

1.2.1 La primera generación (1945 a 1955): tubos al vacío

Después de los esfuerzos infructuosos de Babbage, no hubo muchos progresos en la construcción de computadoras digitales sino hasta la Segunda Guerra Mundial, que estimuló una explosión de esta actividad. El profesor John Atanasoff y su estudiante graduado Clifford Berry construyeron lo que ahora se conoce como la primera computadora digital funcional en Iowa State University. Utilizaba 300 tubos de vacío (bulbos). Aproximadamente al mismo tiempo, Konrad Zuse en Berlín

construyó la computadora Z3 a partir de relevadores. En 1944, la máquina Colossus fue construida por un equipo de trabajo en Bletchley Park, Inglaterra; la Mark I, por Howard Aiken en Harvard, y la ENIAC, por William Mauchley y su estudiante graduado J. Presper Eckert en la Universidad de Pennsylvania. Algunas fueron binarias, otras utilizaron bulbos, algunas eran programables, pero todas eran muy primitivas y tardaban segundos en realizar incluso hasta el cálculo más simple.

En estos primeros días, un solo grupo de personas (generalmente ingenieros) diseñaban, construían, programaban, operaban y daban mantenimiento a cada máquina. Toda la programación se realizaba exclusivamente en lenguaje máquina o, peor aún, creando circuitos eléctricos mediante la conexión de miles de cables a tableros de conexiones (*plugboards*) para controlar las funciones básicas de la máquina. Los lenguajes de programación eran desconocidos (incluso se desconocía el lenguaje ensamblador). Los sistemas operativos también se desconocían. El modo usual de operación consistía en que el programador trabajaba un periodo dado, registrándose en una hoja de firmas, y después entraba al cuarto de máquinas, insertaba su tablero de conexiones en la computadora e invertía varias horas esperando que ninguno de los cerca de 20,000 bulbos se quemara durante la ejecución. Prácticamente todos los problemas eran cálculos numéricos bastante simples, como obtener tablas de senos, cosenos y logaritmos.

A principios de la década de 1950, la rutina había mejorado un poco con la introducción de las tarjetas perforadas. Entonces fue posible escribir programas en tarjetas y leerlas en vez de usar tableros de conexiones; aparte de esto, el procedimiento era el mismo.

1.2.2 La segunda generación (1955 a 1965): transistores y sistemas de procesamiento por lotes

La introducción del transistor a mediados de la década de 1950 cambió radicalmente el panorama. Las computadoras se volvieron lo bastante confiables como para poder fabricarlas y venderlas a clientes dispuestos a pagar por ellas, con la expectativa de que seguirían funcionando el tiempo suficiente como para poder llevar a cabo una cantidad útil de trabajo. Por primera vez había una clara separación entre los diseñadores, constructores, operadores, programadores y el personal de mantenimiento.

Estas máquinas, ahora conocidas como **mainframes**, estaban encerradas en cuartos especiales con aire acondicionado y grupos de operadores profesionales para manejarlas. Sólo las empresas grandes, universidades o agencias gubernamentales importantes podían financiar el costo multimillonario de operar estas máquinas. Para ejecutar un **trabajo** (es decir, un programa o conjunto de programas), el programador primero escribía el programa en papel (en FORTRAN o en ensamblador) y después lo pasaba a tarjetas perforadas. Luego llevaba el conjunto de tarjetas al cuarto de entrada de datos y lo entregaba a uno de los operadores; después se iba a tomar un café a esperar a que los resultados estuvieran listos.

Cuando la computadora terminaba el trabajo que estaba ejecutando en un momento dado, un operador iba a la impresora y arrancaba las hojas de resultados para llevarlas al cuarto de salida de datos, para que el programador pudiera recogerlas posteriormente. Entonces, el operador tomaba uno de los conjuntos de tarjetas que se habían traído del cuarto de entrada y las introducía en la máquina. Si se necesitaba el compilador FORTRAN, el operador tenía que obtenerlo de un gabinete

de archivos e introducirlo a la máquina. Se desperdiciaba mucho tiempo de la computadora mientras los operadores caminaban de un lado a otro del cuarto de la máquina.

Dado el alto costo del equipo, no es sorprendente que las personas buscaran rápidamente formas de reducir el tiempo desperdiciado. La solución que se adoptó en forma general fue el **sistema de procesamiento por lotes**. La idea detrás de este concepto era recolectar una bandeja llena de trabajos en el cuarto de entrada de datos y luego pasarlos a una cinta magnética mediante el uso de una pequeña computadora relativamente económica, tal como la IBM 1401, que era muy adecuada para leer las tarjetas, copiar cintas e imprimir los resultados, pero no tan buena para los cálculos numéricos. Para llevar a cabo los cálculos numéricos se utilizaron otras máquinas mucho más costosas, como la IBM 7094. Este procedimiento se ilustra en la figura 1-3.

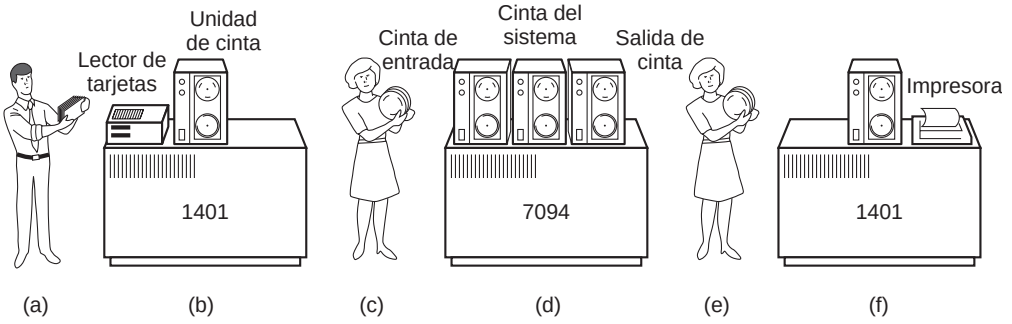


Figura 1-3. Uno de los primeros sistemas de procesamiento por lotes. a) Los programadores llevan las tarjetas a la 1401. b) La 1401 lee los lotes de trabajos y los coloca en cinta. c) El operador lleva la cinta de entrada a la 7094. d) La 7094 realiza los cálculos. e) El operador lleva la cinta de salida a la 1401. f) La 1401 imprime los resultados.

Después de aproximadamente una hora de recolectar un lote de trabajos, las tarjetas se leían y se colocaban en una cinta magnética, la cual se llevaba al cuarto de máquinas, en donde se montaba en una unidad de cinta. Después, el operador cargaba un programa especial (el ancestro del sistema operativo de hoy en día), el cual leía el primer trabajo de la cinta y lo ejecutaba. Los resultados se escribían en una segunda cinta, en vez de imprimirlos. Después de que terminaba cada trabajo, el sistema operativo leía de manera automática el siguiente trabajo de la cinta y empezaba a ejecutarlo. Cuando se terminaba de ejecutar todo el lote, el operador quitaba las cintas de entrada y de salida, reemplazaba la cinta de entrada con el siguiente lote y llevaba la cinta de salida a una 1401 para imprimir **fuera de línea** (es decir, sin conexión con la computadora principal).

En la figura 1-4 se muestra la estructura típica de un trabajo de entrada ordinario. Empieza con una tarjeta $JOB, especificando el tiempo máximo de ejecución en minutos, el número de cuenta al que se va a cargar y el nombre del programador. Después se utiliza una tarjeta $FORTRAN, indicando al sistema operativo que debe cargar el compilador FORTRAN de la cinta del sistema. Después le sigue inmediatamente el programa que se va a compilar y luego una tarjeta $LOAD, que indica al sistema operativo que debe cargar el programa objeto que acaba de compilar (a menudo, los programas compilados se escribían en cintas reutilizables y tenían que cargarse en forma explícita). Después se utiliza la tarjeta $RUN, la cual indica al sistema operativo que debe ejecutar el

programa con los datos que le suceden. Por último, la tarjeta $END marca el final del trabajo. Estas tarjetas de control primitivas fueron las precursoras de los shells e intérpretes de línea de comandos modernos.

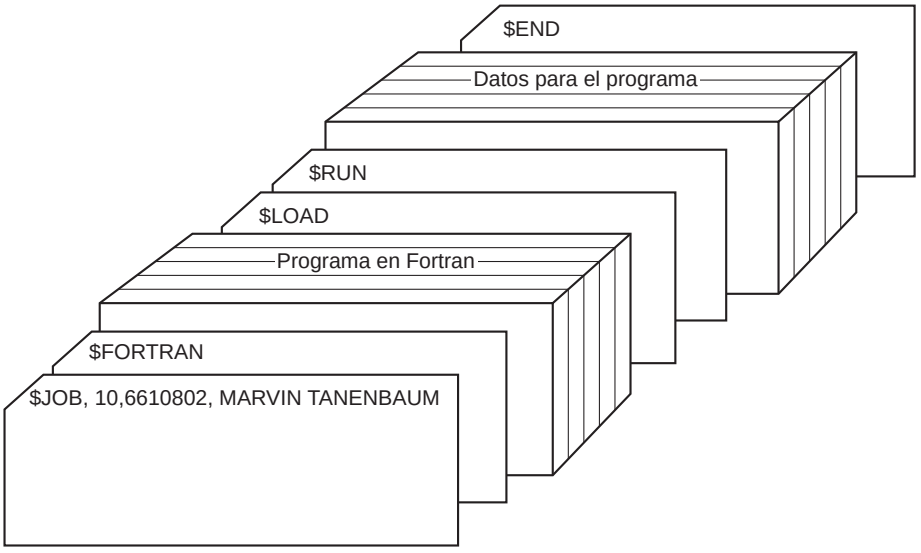


Figura 1-4. Estructura de un trabajo típico de FMS.

Las computadoras grandes de segunda generación se utilizaron principalmente para cálculos científicos y de ingeniería, tales como resolver ecuaciones diferenciales parciales que surgen a menudo en física e ingeniería. En gran parte se programaron en FORTRAN y lenguaje ensamblador. Los sistemas operativos típicos eran FMS (*Fortran Monitor System*) e IBSYS, el sistema operativo de IBM para la 7094.

1.2.3 La tercera generación (1965 a 1980): circuitos integrados y multiprogramación

A principio de la década de 1960, la mayoría de los fabricantes de computadoras tenían dos líneas de productos distintas e incompatibles. Por una parte estaban las computadoras científicas a gran escala orientadas a palabras, como la 7094, que se utilizaban para cálculos numéricos en ciencia e ingeniería. Por otro lado, estaban las computadoras comerciales orientadas a caracteres, como la 1401, que se utilizaban ampliamente para ordenar cintas e imprimir datos en los bancos y las compañías de seguros.

Desarrollar y dar mantenimiento a dos líneas de productos completamente distintos era una propuesta costosa para los fabricantes. Además, muchos nuevos clientes de computadoras necesitaban al principio un equipo pequeño, pero más adelante ya no era suficiente y deseaban una máquina más grande que pudiera ejecutar todos sus programas anteriores, pero con mayor rapidez.

IBM intentó resolver ambos problemas de un solo golpe con la introducción de la línea de computadoras System/360. La 360 era una serie de máquinas compatibles con el software, que variaban desde un tamaño similar a la 1401 hasta algunas que eran más potentes que la 7094. Las máquinas sólo diferían en el precio y rendimiento (máxima memoria, velocidad del procesador, número de dispositivos de E/S permitidos, etcétera). Como todas las máquinas tenían la misma arquitectura y el mismo conjunto de instrucciones, los programas escritos para una máquina podían ejecutarse en todas las demás, por lo menos en teoría. Lo que es más, la 360 se diseñó para manejar tanto la computación científica (es decir, numérica) como comercial. Por ende, una sola familia de máquinas podía satisfacer las necesidades de todos los clientes. En los años siguientes, mediante el uso de tecnología más moderna, IBM ha desarrollado sucesores compatibles con la línea 360, a los cuales se les conoce como modelos 370, 4300, 3080 y 3090. La serie zSeries es el descendiente más reciente de esta línea, aunque diverge considerablemente del original.

La IBM 360 fue la primera línea importante de computadoras en utilizar **circuitos integrados** (**ICs**) (a pequeña escala), con lo cual se pudo ofrecer una mayor ventaja de precio/rendimiento en comparación con las máquinas de segunda generación, las cuales fueron construidas a partir de transistores individuales. Su éxito fue inmediato y la idea de una familia de computadoras compatibles pronto fue adoptada por todos los demás fabricantes importantes. Los descendientes de estas máquinas se siguen utilizando hoy día en centros de cómputo. En la actualidad se utilizan con frecuencia para manejar bases de datos enormes (por ejemplo, para sistemas de reservaciones de aerolíneas) o como servidores para sitios de World Wide Web que deben procesar miles de solicitudes por segundo.

La mayor fortaleza de la idea de "una sola familia" fue al mismo tiempo su mayor debilidad. La intención era que todo el software, incluyendo al sistema operativo **OS/360**, funcionara en todos los modelos. Debía ejecutarse en los sistemas pequeños, que por lo general sólo reemplazaban a la 1401s, que copiaba tarjetas a cinta, y en los sistemas muy grandes, que a menudo reemplazaban a la 7094s, que realizaba predicciones del clima y otros cálculos pesados. Tenía que ser bueno en sistemas con pocos dispositivos periféricos y en sistemas con muchos. Tenía que funcionar en ambos entornos comerciales y científicos. Por encima de todo, tenía que ser eficiente para todos estos usos distintos.

No había forma en que IBM (o cualquier otra) pudiera escribir una pieza de software que cumpliera con todos estos requerimientos en conflicto. El resultado fue un enorme y extraordinariamente complejo sistema operativo, tal vez de dos a tres órdenes de magnitud más grande que el FMS. Consistía en millones de líneas de lenguaje ensamblador escrito por miles de programadores, con miles de errores, los cuales requerían un flujo continuo de nuevas versiones en un intento por corregirlos. Cada nueva versión corregía algunos errores e introducía otros, por lo que probablemente el número de errores permanecía constante en el tiempo.

Fred Brooks, uno de los diseñadores del OS/360, escribió posteriormente un libro ingenioso e incisivo (Brooks, 1996) que describía sus experiencias con el OS/360. Aunque sería imposible resumir este libro aquí, basta con decir que la portada muestra una manada de bestias prehistóricas atrapadas en un pozo de brea. La portada de Silberschatz y coautores (2005) muestra un punto de vista similar acerca de que los sistemas operativos son como dinosaurios.

A pesar de su enorme tamaño y sus problemas, el OS/360 y los sistemas operativos similares de tercera generación producidos por otros fabricantes de computadoras en realidad dejaban razo-

nablemente satisfechos a la mayoría de sus clientes. También popularizaron varias técnicas clave ausentes en los sistemas operativos de segunda generación. Quizá la más importante de éstas fue la **multiprogramación**. En la 7094, cuando el trabajo actual se detenía para esperar a que se completara una operación con cinta u otro dispositivo de E/S, la CPU simplemente permanecía inactiva hasta terminar la operación de E/S. Con los cálculos científicos que requieren un uso intensivo de la CPU, la E/S no es frecuente, por lo que este tiempo desperdiciado no es considerable. Con el procesamiento de datos comerciales, el tiempo de espera de las operaciones de E/S puede ser a menudo de 80 a 90 por ciento del tiempo total, por lo que debía hacerse algo para evitar que la (costosa) CPU esté inactiva por mucho tiempo.

La solución que surgió fue particionar la memoria en varias piezas, con un trabajo distinto en cada partición, como se muestra en la figura 1-5. Mientras que un trabajo esperaba a que se completara una operación de E/S, otro podía estar usando la CPU. Si pudieran contenerse suficientes trabajos en memoria principal al mismo tiempo, la CPU podía estar ocupada casi 100 por ciento del tiempo. Para tener varios trabajos de forma segura en memoria a la vez, se requiere hardware especial para proteger cada trabajo y evitar que los otros se entrometan y lo malogren; el 360 y los demás sistemas de tercera generación estaban equipados con este hardware.

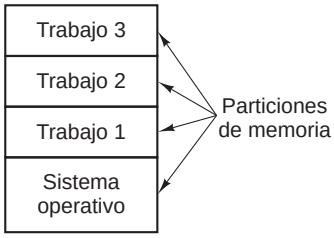


Figura 1-5. Un sistema de multiprogramación con tres trabajos en memoria.

Otra característica importante de los sistemas operativos de tercera generación fue la capacidad para leer trabajos en tarjetas y colocarlos en el disco tan pronto como se llevaban al cuarto de computadoras. Así, cada vez que terminaba un trabajo en ejecución, el sistema operativo podía cargar un nuevo trabajo del disco en la partición que entonces estaba vacía y lo ejecutaba. A esta técnica se le conoce como **spooling** (de *Simultaneous Peripheral Operation On Line*, operación periférica simultánea en línea) y también se utilizó para las operaciones de salida. Con el spooling, las máquinas 1401 no eran ya necesarias y desapareció la mayor parte del trabajo de transportar las cintas.

Aunque los sistemas operativos de tercera generación eran apropiados para los cálculos científicos extensos y las ejecuciones de procesamiento de datos comerciales masivos, seguían siendo en esencia sistemas de procesamiento por lotes. Muchos programadores añoraban los días de la primera generación en los que tenían toda la máquina para ellos durante unas cuantas horas, por lo que podían depurar sus programas con rapidez. Con los sistemas de tercera generación, el tiempo que transcurría entre enviar un trabajo y recibir de vuelta la salida era comúnmente de varias horas, por lo que una sola coma mal colocada podía ocasionar que fallara la compilación, y el programador desperdiciara la mitad del día.

Este deseo de obtener un tiempo rápido de respuesta allanó el camino para el **tiempo compartido** (*timesharing*), una variante de la multiprogramación donde cada usuario tenía una terminal en

línea. En un sistema de tiempo compartido, si 20 usuarios están conectados y 17 de ellos están pensando en dar un paseo o tomar café, la CPU se puede asignar por turno a los tres trabajos que desean ser atendidos. Como las personas que depuran programas generalmente envían comandos cortos (por ejemplo, compilar un procedimiento de cinco hojas[†]) en vez de largos (por ejemplo, ordenar un archivo con un millón de registros), la computadora puede proporcionar un servicio rápido e interactivo a varios usuarios y, tal vez, también ocuparse en trabajos grandes por lotes en segundo plano, cuando la CPU estaría inactiva de otra manera. El primer sistema de tiempo compartido de propósito general, conocido como **CTSS** (*Compatible Time Sharing System*, Sistema compatible de tiempo compartido), se desarrolló en el M.I.T. en una 7094 modificada en forma especial (Corbató y colaboradores, 1962). Sin embargo, en realidad el tiempo compartido no se popularizó sino hasta que el hardware de protección necesario se empezó a utilizar ampliamente durante la tercera generación.

Después del éxito del sistema CTSS, el M.I.T., Bell Labs y General Electric (que en ese entonces era un importante fabricante de computadoras) decidieron emprender el desarrollo de una "utilería para computadora", una máquina capaz de servir a varios cientos de usuarios simultáneos de tiempo compartido. Su modelo fue el sistema de electricidad: cuando se necesita energía, sólo hay que conectar un contacto a la pared y, dentro de lo razonable, toda la energía que se requiera estará ahí. Los diseñadores del sistema conocido como **MULTICS** (*MULTiplexed Information and Computing Service*; Servicio de Información y Cómputo MULTiplexado), imaginaron una enorme máquina que proporcionaba poder de cómputo a todos los usuarios en el área de Boston. La idea de que, sólo 40 años después, se vendieran por millones máquinas 10,000 veces más rápidas que su mainframe GE-645 (a un precio muy por debajo de los 1000 dólares) era pura ciencia ficción. Algo así como la idea de que en estos días existiera un transatlántico supersónico por debajo del agua.

MULTICS fue un éxito parcial. Se diseñó para dar soporte a cientos de usuarios en una máquina que era sólo un poco más potente que una PC basada en el Intel 386, aunque tenía mucho más capacidad de E/S. Esto no es tan disparatado como parece, ya que las personas sabían cómo escribir programas pequeños y eficientes en esos días, una habilidad que se ha perdido con el tiempo. Hubo muchas razones por las que MULTICS no acaparó la atención mundial; una de ellas fue el que estaba escrito en PL/I y el compilador de PL/I se demoró por años, además de que apenas funcionaba cuando por fin llegó. Aparte de eso, MULTICS era un sistema demasiado ambicioso para su época, algo muy parecido a la máquina analítica de Charles Babbage en el siglo diecinueve.

Para resumir esta larga historia, MULTICS introdujo muchas ideas seminales en la literatura de las computadoras, pero convertirlas en un producto serio y con éxito comercial importante era algo mucho más difícil de lo que cualquiera hubiera esperado. Bell Labs se retiró del proyecto y General Electric dejó el negocio de las computadoras por completo. Sin embargo, el M.I.T. persistió y logró hacer en un momento dado que MULTICS funcionara. Al final, la compañía que compró el negocio de computadoras de GE (Honeywell) lo vendió como un producto comercial y fue instalado por cerca de 80 compañías y universidades importantes a nivel mundial. Aunque en número pequeño, los usuarios de MULTICS eran muy leales. Por ejemplo, General Motors, Ford y la Agencia de Seguridad Nacional de los Estados Unidos desconectaron sus sistemas MULTICS sólo hasta

[†] En este libro utilizaremos los términos "procedimiento", "subrutina" y "función" de manera indistinta.

finales de la década de 1990, 30 años después de su presentación en el mercado y de tratar durante años de hacer que Honeywell actualizara el hardware.

Por ahora, el concepto de una "utilería para computadora" se ha disipado, pero tal vez regrese en forma de servidores masivos de Internet centralizados a los que se conecten máquinas de usuario relativamente "tontas", donde la mayoría del trabajo se realice en los servidores grandes. Es probable que la motivación en este caso sea que la mayoría de las personas no desean administrar un sistema de cómputo cada vez más complejo y melindroso, y prefieren delegar esa tarea a un equipo de profesionales que trabajen para la compañía que opera el servidor. El comercio electrónico ya está evolucionando en esta dirección, en donde varias compañías operan centros comerciales electrónicos en servidores multiprocesador a los que se conectan las máquinas cliente simples, algo muy parecido al diseño de MULTICS.

A pesar de la carencia de éxito comercial, MULTICS tuvo una enorme influencia en los sistemas operativos subsecuentes. Se describe en varios artículos y en un libro (Corbató y colaboradores, 1972; Corbató y Vyssotsky, 1965; Daley y Dennis, 1968; Organick, 1972; y Staltzer, 1974). También tuvo (y aún tiene) un sitio Web activo, ubicado en *www.multicians.org*, con mucha información acerca del sistema, sus diseñadores y sus usuarios.

Otro desarrollo importante durante la tercera generación fue el increíble crecimiento de las minicomputadoras, empezando con la DEC PDP-1 en 1961. La PDP-1 tenía sólo 4K de palabras de 18 bits, pero a $120,000 por máquina (menos de 5 por ciento del precio de una 7094) se vendió como pan caliente. Para cierta clase de trabajo no numérico, era casi tan rápida como la 7094 y dio origen a una nueva industria. A esta minicomputadora le siguió rápidamente una serie de otras PDP (a diferencia de la familia de IBM, todas eran incompatibles), culminando con la PDP-11.

Posteriormente, Ken Thompson, uno de los científicos de cómputo en Bell Labs que trabajó en el proyecto MULTICS, encontró una pequeña minicomputadora PDP-7 que nadie estaba usando y se dispuso a escribir una versión simple de MULTICS para un solo usuario. Más adelante, este trabajo se convirtió en el sistema operativo **UNIX**®, que se hizo popular en el mundo académico, las agencias gubernamentales y muchas compañías.

La historia de UNIX ya ha sido contada en muchos otros libros (por ejemplo, Salus, 1994). En el capítulo 10 hablaremos sobre parte de esa historia. Por ahora baste con decir que, debido a que el código fuente estaba disponible ampliamente, varias organizaciones desarrollaron sus propias versiones (incompatibles entre sí), lo cual produjo un caos. Se desarrollaron dos versiones principales: **System V** de AT&T y **BSD** (*Berkeley Software Distribution*, Distribución de Software de Berkeley) de la Universidad de California en Berkeley. Estas versiones tenían también variantes menores. Para que fuera posible escribir programas que pudieran ejecutarse en cualquier sistema UNIX, el IEEE desarrolló un estándar para UNIX conocido como **POSIX**, con el que la mayoría de las versiones de UNIX actuales cumplen. POSIX define una interfaz mínima de llamadas al sistema a la que los sistemas UNIX deben conformarse. De hecho, algunos de los otros sistemas operativos también admiten ahora la interfaz POSIX.

Como agregado, vale la pena mencionar que en 1987 el autor liberó un pequeño clon de UNIX conocido como **MINIX**, con fines educativos. En cuanto a su funcionalidad, MINIX es muy similar a UNIX, incluyendo el soporte para POSIX. Desde esa época, la versión original ha evolucionado en MINIX 3, que es altamente modular y está enfocada a presentar una muy alta confiabilidad. Tiene la habilidad de detectar y reemplazar módulos con fallas o incluso inutilizables (como los dis-

positivos controladores de dispositivos de E/S) al instante, sin necesidad de reiniciar y sin perturbar a los programas en ejecución. También hay disponible un libro que describe su operación interna y contiene un listado del código fuente en un apéndice (Tanenbaum y Woodhull, 2006). El sistema MINIX 3 está disponible en forma gratuita (incluyendo todo el código fuente) a través de Internet, en *www.minix3.org*.

El deseo de una versión de producción (en vez de educativa) gratuita de MINIX llevó a un estudiante finlandés, llamado Linus Torvalds, a escribir **Linux**. Este sistema estaba inspirado por MINIX, además de que fue desarrollado en este sistema y originalmente ofrecía soporte para varias características de MINIX (por ejemplo, el sistema de archivos de MINIX). Desde entonces se ha extendido en muchas formas, pero todavía retiene cierta parte de su estructura subyacente común para MINIX y UNIX. Los lectores interesados en una historia detallada sobre Linux y el movimiento de código fuente abierto tal vez deseen leer el libro de Glyn Moody (2001). La mayor parte de lo que se haya dicho acerca de UNIX en este libro se aplica también a System V, MINIX, Linux y otras versiones y clones de UNIX.

1.2.4 La cuarta generación (1980 a la fecha): las computadoras personales

Con el desarrollo de los circuitos LSI (*Large Scale Integration*, Integración a gran escala), que contienen miles de transistores en un centímetro cuadrado de silicio (chip), nació la era de la computadora personal. En términos de arquitectura, las computadoras personales (que al principio eran conocidas como **microcomputadoras**) no eran del todo distintas de las minicomputadoras de la clase PDP-11, pero en términos de precio sin duda eran distintas. Mientras que la minicomputadora hizo posible que un departamento en una compañía o universidad tuviera su propia computadora, el chip microprocesador logró que un individuo tuviera su propia computadora personal.

Cuando Intel presentó el microprocesador 8080 en 1974 (la primera CPU de 8 bits de propósito general), deseaba un sistema operativo, en parte para poder probarlo. Intel pidió a uno de sus consultores, Gary Kildall, que escribiera uno. Kildall y un amigo construyeron primero un dispositivo controlador para el disco flexible de 8 pulgadas de Shugart Associates que recién había sido sacado al mercado, y conectaron el disco flexible con el 8080, con lo cual produjeron la primera microcomputadora con un disco. Después Kildall escribió un sistema operativo basado en disco conocido como **CP/M** (*Control Program for Microcomputers*; Programa de Control para Microcomputadoras) para esta CPU. Como Intel no pensó que las microcomputadoras basadas en disco tuvieran mucho futuro, cuando Kildall pidió los derechos para CP/M, Intel le concedió su petición. Después Kildall formó una compañía llamada Digital Research para desarrollar y vender el CP/M.

En 1977, Digital Research rediseñó el CP/M para adaptarlo de manera que se pudiera ejecutar en todas las microcomputadoras que utilizaban los chips 8080, Zilog Z80 y otros. Se escribieron muchos programas de aplicación para ejecutarse en CP/M, lo cual le permitió dominar por completo el mundo de la microcomputación durante un tiempo aproximado de 5 años.

A principios de la década de 1980, IBM diseñó la IBM PC y buscó software para ejecutarlo en ella. La gente de IBM se puso en contacto con Bill Gates para obtener una licencia de uso de su intérprete de BASIC. También le preguntaron si sabía de un sistema operativo que se ejecutara en la PC. Gates sugirió a IBM que se pusiera en contacto con Digital Research, que en ese entonces era la compañía con dominio mundial en el área de sistemas operativos. Kildall rehusó a reunirse con IBM y envió a uno de sus subordinados, a lo cual se le considera sin duda la peor decisión de negocios de la historia. Para empeorar más aún las cosas, su abogado se rehusó a firmar el contrato de no divulgación de IBM sobre la PC, que no se había anunciado todavía. IBM regresó con Gates para ver si podía proveerles un sistema operativo.

Cuando IBM regresó, Gates se había enterado de que un fabricante local de computadoras, Seattle Computer Products, tenía un sistema operativo adecuado conocido como **DOS** (*Disk Operating System*; Sistema Operativo en Disco). Se acercó a ellos y les ofreció comprarlo (supuestamente por 75,000 dólares), a lo cual ellos accedieron de buena manera. Después Gates ofreció a IBM un paquete con DOS/BASIC, el cual aceptó. IBM quería ciertas modificaciones, por lo que Gates contrató a la persona que escribió el DOS, Tim Paterson, como empleado de su recién creada empresa de nombre Microsoft, para que las llevara a cabo. El sistema rediseñado cambió su nombre a **MS-DOS** (*Microsoft Disk Operating System;* Sistema Operativo en Disco de MicroSoft) y rápidamente llegó a dominar el mercado de la IBM PC. Un factor clave aquí fue la decisión de Gates (que en retrospectiva, fue en extremo inteligente) de vender MS-DOS a las empresas de computadoras para que lo incluyeran con su hardware, en comparación con el intento de Kildall por vender CP/M a los usuarios finales, uno a la vez (por lo menos al principio). Después de que se supo todo esto, Kildall murió en forma repentina e inesperada debido a causas que aún no han sido reveladas por completo.

Para cuando salió al mercado en 1983 la IBM PC/AT, sucesora de la IBM PC, con la CPU Intel 80286, MS-DOS estaba muy afianzado y CP/M daba sus últimos suspiros. Más adelante, MS-DOS se utilizó ampliamente en el 80386 y 80486. Aunque la versión inicial de MS-DOS era bastante primitiva, las versiones siguientes tenían características más avanzadas, incluyendo muchas que se tomaron de UNIX. (Microsoft estaba muy al tanto de UNIX e inclusive vendía una versión de este sistema para microcomputadora, conocida como XENIX, durante los primeros años de la compañía).

CP/M, MS-DOS y otros sistemas operativos para las primeras microcomputadoras se basaban en que los usuarios escribieran los comandos mediante el teclado. Con el tiempo esto cambió debido a la investigación realizada por Doug Engelbart en el Stanford Research Institute en la década de 1960. Engelbart inventó la **Interfaz Gráfica de Usuario GUI**, completa con ventanas, iconos, menús y ratón. Los investigadores en Xerox PARC adoptaron estas ideas y las incorporaron en las máquinas que construyeron.

Un día, Steve Jobs, que fue co-inventor de la computadora Apple en su cochera, visitó PARC, vio una GUI y de inmediato se dio cuenta de su valor potencial, algo que la administración de Xerox no hizo. Esta equivocación estratégica de gigantescas proporciones condujo a un libro titulado *Fumbling the Future* (Smith y Alexander, 1988). Posteriormente, Jobs emprendió el proyecto de construir una Apple con una GUI. Este proyecto culminó en Lisa, que era demasiado costosa y fracasó comercialmente. El segundo intento de Jobs, la Apple Macintosh, fue un enorme éxito, no sólo debido a que era mucho más económica que Lisa, sino también porque era **amigable para el**

usuario (*user friendly*), lo cual significaba que estaba diseñada para los usuarios que no sólo no sabían nada acerca de las computadoras, sino que además no tenían ninguna intención de aprender. En el mundo creativo del diseño gráfico, la fotografía digital profesional y la producción de video digital profesional, las Macintosh son ampliamente utilizadas y sus usuarios son muy entusiastas sobre ellas.

Cuando Microsoft decidió crear un sucesor para el MS-DOS estaba fuertemente influenciado por el éxito de la Macintosh. Produjo un sistema basado en GUI llamado Windows, el cual en un principio se ejecutaba encima del MS-DOS (es decir, era más como un shell que un verdadero sistema operativo). Durante cerca de 10 años, de 1985 a 1995, Windows fue sólo un entorno gráfico encima de MS-DOS. Sin embargo, a partir de 1995 se liberó una versión independiente de Windows, conocida como Windows 95, que incorporaba muchas características de los sistemas operativos y utilizaba el sistema MS-DOS subyacente sólo para iniciar y ejecutar programas de MS-DOS antiguos. En 1998, se liberó una versión ligeramente modificada de este sistema, conocida como Windows 98. Sin embargo, tanto Windows 95 como Windows 98 aún contenían una gran cantidad de lenguaje ensamblador para los procesadores Intel de 16 bits.

Otro de los sistemas operativos de Microsoft es **Windows NT** (NT significa Nueva Tecnología), que es compatible con Windows 95 en cierto nivel, pero fue completamente rediseñado en su interior. Es un sistema completo de 32 bits. El diseñador en jefe de Windows NT fue David Cutler, quien también fue uno de los diseñadores del sistema operativo VMS de VAX, por lo que hay algunas ideas de VMS presentes en NT. De hecho, había tantas ideas de VMS presentes que el propietario de VMS (DEC) demandó a Microsoft. El caso se resolvió en la corte por una cantidad de muchos dígitos. Microsoft esperaba que la primera versión de NT acabara con MS-DOS y todas las demás versiones de Windows, ya que era un sistema muy superior, pero fracasó. No fue sino hasta Windows NT 4.0 que finalmente empezó a tener éxito, en especial en las redes corporativas. La versión 5 de Windows NT cambió su nombre a Windows 2000 a principios de 1999. Estaba destinada a ser el sucesor de Windows 98 y de Windows NT 4.0.

Esto tampoco funcionó como se esperaba, por lo que Microsoft preparó otra versión de Windows 98 conocida como **Windows Me** (*Millennium edition*). En el 2001 se liberó una versión ligeramente actualizada de Windows 2000, conocida como Windows XP. Esa versión duró mucho más en el mercado (6 años), reemplazando a casi todas las versiones anteriores de Windows. Después, en enero del 2007 Microsoft liberó el sucesor para Windows XP, conocido como Windows Vista. Tenía una interfaz gráfica nueva, Aero, y muchos programas de usuario nuevos o actualizados. Microsoft espera que sustituya a Windows XP por completo, pero este proceso podría durar casi toda una década.

El otro competidor importante en el mundo de las computadoras personales es UNIX (y todas sus variantes). UNIX es más fuerte en los servidores tanto de redes como empresariales, pero también está cada vez más presente en las computadoras de escritorio, en especial en los países que se desarrollan con rapidez, como India y China. En las computadoras basadas en Pentium, Linux se está convirtiendo en una alternativa popular para Windows entre los estudiantes y cada vez más usuarios corporativos. Como agregado, a lo largo de este libro utilizaremos el término “Pentium” para denotar al Pentium I, II, III y 4, así como sus sucesores tales como el Core 2 Duo. El término **x86** también se utiliza algunas veces para indicar el rango completo de CPU Intel partiendo desde el 8086, mientras que utilizaremos “Pentium” para indicar todas las CPU desde el

Pentium I. Admitimos que este término no es perfecto, pero no hay disponible uno mejor. Uno se pregunta qué genio de mercadotecnia en Intel desperdició una marca comercial (Pentium) que la mitad del mundo conocía bien y respetaba, sustituyéndola con términos como "Core 2 duo" que muy pocas personas comprenden; ¿qué significan "2" y "duo"? Tal vez "Pentium 5" (o "Pentium 5 dual core", etc.) eran demasiado difíciles de recordar. **FreeBSD** es también un derivado popular de UNIX, que se originó del proyecto BSD en Berkeley. Todas la computadoras modernas Macintosh utilizan una versión modificada de FreeBSD. UNIX también es estándar en las estaciones de trabajo operadas por chips RISC de alto rendimiento, como los que venden Hewlett-Packard y Sun Microsystems.

Muchos usuarios de UNIX, en especial los programadores experimentados, prefieren una interfaz de línea de comandos a una GUI, por lo que casi todos los sistemas UNIX presentan un sistema de ventanas llamado **X Window System** (también conocido como **X11**), producido en el M.I.T. Este sistema se encarga de la administración básica de las ventanas y permite a los usuarios crear, eliminar, desplazar y cambiar el tamaño de las ventanas mediante el uso de un ratón. Con frecuencia hay disponible una GUI completa, como **Gnome** o **KDE**, para ejecutarse encima de X11, lo cual proporciona a UNIX una apariencia parecida a la Macintosh o a Microsoft Windows, para aquellos usuarios de UNIX que desean algo así.

Un interesante desarrollo que empezó a surgir a mediados de la década de 1980 es el crecimiento de las redes de computadoras personales que ejecutan **sistemas operativos en red** y **sistemas operativos distribuidos** (Tanenbaum y Van Steen, 2007). En un sistema operativo en red, los usuarios están conscientes de la existencia de varias computadoras, y pueden iniciar sesión en equipos remotos y copiar archivos de un equipo a otro. Cada equipo ejecuta su propio sistema operativo local y tiene su propio usuario (o usuarios) local.

Los sistemas operativos en red no son fundamentalmente distintos de los sistemas operativos con un solo procesador. Es obvio que necesitan un dispositivo controlador de interfaz de red y cierto software de bajo nivel para controlarlo, así como programas para lograr el inicio de una sesión remota y el acceso remoto a los archivos, pero estas adiciones no cambian la estructura esencial del sistema operativo.

En contraste, un sistema operativo distribuido se presenta a sus usuarios en forma de un sistema tradicional con un procesador, aun cuando en realidad está compuesto de varios procesadores. Los usuarios no tienen que saber en dónde se están ejecutando sus programas o en dónde se encuentran sus archivos; el sistema operativo se encarga de todo esto de manera automática y eficiente.

Los verdaderos sistemas operativos distribuidos requieren algo más que sólo agregar un poco de código a un sistema operativo con un solo procesador, ya que los sistemas distribuidos y los centralizados difieren en varios puntos críticos. Por ejemplo, los sistemas distribuidos permiten con frecuencia que las aplicaciones se ejecuten en varios procesadores al mismo tiempo, lo que requiere algoritmos de planificación del procesador más complejos para poder optimizar la cantidad de paralelismo.

Los retrasos de comunicación dentro de la red implican a menudo que estos (y otros) algoritmos deban ejecutarse con información incompleta, obsoleta o incluso incorrecta. Esta situación es muy distinta a la de un sistema con un solo procesador, donde el sistema operativo tiene información completa acerca del estado del sistema.

1.3 REVISIÓN DEL HARDWARE DE COMPUTADORA

Un sistema operativo está íntimamente relacionado con el hardware de la computadora sobre la que se ejecuta. Extiende el conjunto de instrucciones de la computadora y administra sus recursos. Para trabajar debe conocer muy bien el hardware, por lo menos en lo que respecta a cómo aparece para el programador. Por esta razón, revisaremos brevemente el hardware de computadora como se encuentra en las computadoras personales modernas. Después de eso, podemos empezar a entrar en los detalles acerca de qué hacen los sistemas operativos y cómo funcionan.

Conceptualmente, una computadora personal simple se puede abstraer mediante un modelo como el de la figura 1-6. La CPU, la memoria y los dispositivos de E/S están conectados mediante un bus del sistema y se comunican entre sí a través de este bus. Las computadoras personales modernas tienen una estructura más complicada en la que intervienen varios buses, los cuales analizaremos más adelante; por ahora basta con este modelo. En las siguientes secciones analizaremos brevemente estos componentes y examinaremos algunas de las cuestiones de hardware que son de relevancia para los diseñadores de sistemas operativos; sobra decir que será un resumen muy compacto. Se han escrito muchos libros acerca del tema del hardware de computadora y su organización. Dos libros muy conocidos acerca de este tema son el de Tanenbaum (2006) y el de Patterson y Hennessy (2004).

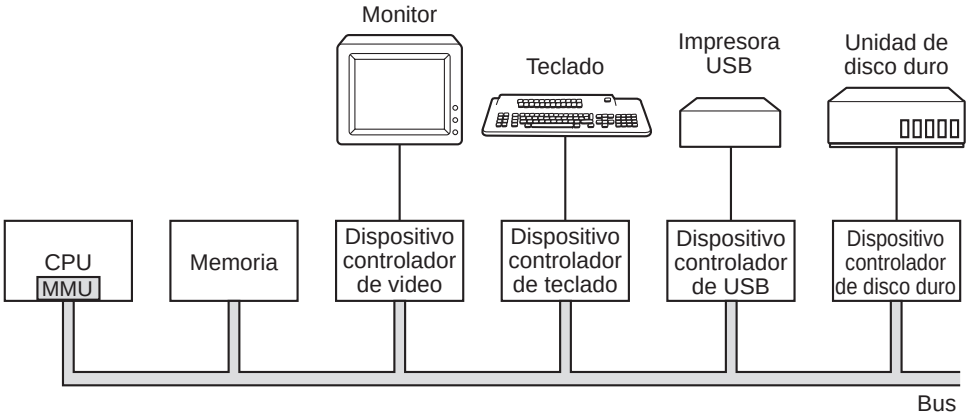


Figura 1-6. Algunos de los componentes de una computadora personal simple.

1.3.1 Procesadores

El “cerebro” de la computadora es la CPU, que obtiene las instrucciones de la memoria y las ejecuta. El ciclo básico de toda CPU es obtener la primera instrucción de memoria, decodificarla para determinar su tipo y operandos, ejecutarla y después obtener, decodificar y ejecutar las instrucciones subsiguientes. El ciclo se repite hasta que el programa termina. De esta forma se ejecutan los programas.

Cada CPU tiene un conjunto específico de instrucciones que puede ejecutar. Así, un Pentium no puede ejecutar programas de SPARC y un SPARC no puede ejecutar programas de Pentium. Como el acceso a la memoria para obtener una instrucción o palabra de datos requiere mucho más tiempo que ejecutar una instrucción, todas las CPU contienen ciertos registros en su interior para contener las variables clave y los resultados temporales. Debido a esto, el conjunto de instrucciones generalmente contiene instrucciones para cargar una palabra de memoria en un registro y almacenar una palabra de un registro en la memoria. Otras instrucciones combinan dos operandos de los registros, la memoria o ambos en un solo resultado, como la operación de sumar dos palabras y almacenar el resultado en un registro o la memoria.

Además de los registros generales utilizados para contener variables y resultados temporales, la mayoría de las computadoras tienen varios registros especiales que están visibles para el programador. Uno de ellos es el **contador de programa** (*program counter*), el cual contiene la dirección de memoria de la siguiente instrucción a obtener. Una vez que se obtiene esa instrucción, el contador de programa se actualiza para apuntar a la siguiente.

Otro registro es el **apuntador de pila** (*stack pointer*), el cual apunta a la parte superior de la pila (*stack*) actual en la memoria. La pila contiene un conjunto de valores por cada procedimiento al que se ha entrado pero del que todavía no se ha salido. El conjunto de valores en la pila por procedimiento contiene los parámetros de entrada, las variables locales y las variables temporales que no se mantienen en los registros.

Otro de los registros es **PSW** (*Program Status Word*; Palabra de estado del programa). Este registro contiene los bits de código de condición, que se asignan cada vez que se ejecutan las instrucciones de comparación, la prioridad de la CPU, el modo (usuario o kernel) y varios otros bits de control. Los programas de usuario pueden leer normalmente todo el PSW pero por lo general sólo pueden escribir en algunos de sus campos. El PSW juega un papel importante en las llamadas al sistema y en las operaciones de E/S.

El sistema operativo debe estar al tanto de todos los registros. Cuando la CPU se multiplexa en el tiempo, el sistema operativo detiene con frecuencia el programa en ejecución para (re)iniciar otro. Cada vez que detiene un programa en ejecución, el sistema operativo debe guardar todos los registros para poder restaurarlos cuando el programa continúe su ejecución.

Para mejorar el rendimiento, los diseñadores de CPUs abandonaron desde hace mucho tiempo el modelo de obtener, decodificar y ejecutar una instrucción a la vez. Muchas CPUs modernas cuentan con medios para ejecutar más de una instrucción al mismo tiempo. Por ejemplo, una CPU podría tener unidades separadas de obtención, decodificación y ejecución, de manera que mientras se encuentra ejecutando la instrucción n, también podría estar decodificando la instrucción $n + 1$ y obteniendo la instrucción $n + 2$. A dicha organización se le conoce como **canalización** (*pipeline*); la figura 1-7(a) ilustra una canalización de tres etapas. El uso de canalizaciones más grandes es común. En la mayoría de los diseños de canalizaciones, una vez que se ha obtenido una instrucción y se coloca en la canalización, se debe ejecutar aún si la instrucción anterior era una bifurcación condicional que se tomó. Las canalizaciones producen grandes dolores de cabeza a los programadores de compiladores y de sistemas operativos, ya que quedan al descubierto las complejidades de la máquina subyacente.

Aún más avanzada que el diseño de una canalización es la CPU **superescalar**, que se muestra en la figura 1-7(b). En este diseño hay varias unidades de ejecución; por ejemplo, una para la arit-

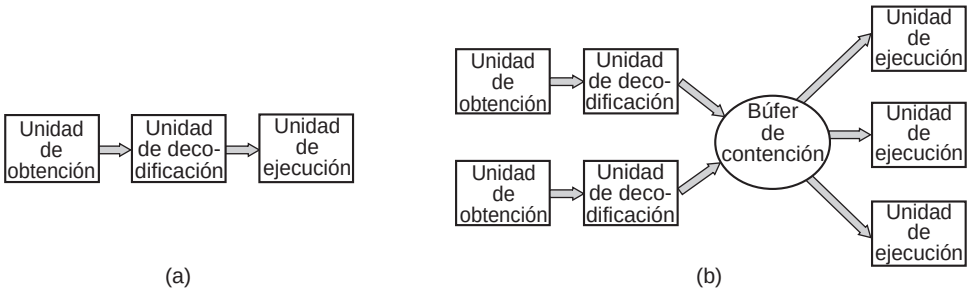


Figura 1-7. (a) Canalización de tres etapas; (b) CPU superescalar.

mética de enteros, una para la aritmética de punto flotante y otra para las operaciones Booleanas. Dos o más instrucciones se obtienen a la vez, se decodifican y se vacían en un búfer de contención hasta que puedan ejecutarse. Tan pronto como una unidad de ejecución se encuentre libre, busca en el búfer de contención para ver si hay una instrucción que pueda manejar; de ser así, saca la instrucción del búfer y la ejecuta. Una consecuencia de este diseño es que con frecuencia las instrucciones del programa se ejecutan en forma desordenada. En gran parte, es responsabilidad del hardware asegurarse de que el resultado producido sea el mismo que hubiera producido una implementación secuencial, pero una cantidad molesta de complejidad es impuesta al sistema operativo, como veremos más adelante.

La mayoría de las CPU, con excepción de las extremadamente simples que se utilizan en los sistemas integrados, tienen dos modos: modo kernel y modo usuario, como dijimos antes. Por lo general, un bit en el PSW controla el modo. Al operar en modo kernel, la CPU puede ejecutar cualquier instrucción de su conjunto de instrucciones y utilizar todas las características del hardware. El sistema operativo opera en modo kernel, lo cual le da acceso al hardware completo.

En contraste, los programas de usuario operan en modo de usuario, el cual les permite ejecutar sólo un subconjunto de las instrucciones y les da acceso sólo a un subconjunto de las características. En general, no se permiten las instrucciones que implican operaciones de E/S y protección de la memoria en el modo usuario. Desde luego que también está prohibido asignar el bit de modo del PSW para entrar al modo kernel.

Para obtener servicios del sistema operativo, un programa usuario debe lanzar una **llamada al sistema** (*system call*), la cual se atrapa en el kernel e invoca al sistema operativo. La instrucción `TRAP` cambia del modo usuario al modo kernel e inicia el sistema operativo. Cuando se ha completado el trabajo, el control se devuelve al programa de usuario en la instrucción que va después de la llamada al sistema. Más adelante en este capítulo explicaremos los detalles acerca del mecanismo de llamadas al sistema, pero por ahora piense en él como un tipo especial de instrucción de llamada a procedimiento que tiene la propiedad adicional de cambiar del modo usuario al modo kernel. Como indicación sobre la tipografía, utilizaremos el tipo de letra Helvetica en minúsculas para indicar las llamadas al sistema en el texto del libro, como se muestra a continuación: `read`.

Vale la pena indicar que las computadoras tienen otros traps aparte de la instrucción para ejecutar una llamada al sistema. La mayoría de los demás traps son producidos por el hardware para advertir acerca de una situación excepcional, tal como un intento de dividir entre 0 o un subdesbor-

damiento de punto flotante. En cualquier caso, el sistema operativo obtiene el control y debe decidir qué hacer. Algunas veces el programa debe terminarse con un error. Otras veces el error se puede ignorar (un número que provoque un subdesbordamiento puede establecerse en 0). Por último, cuando el programa anuncia por adelantado que desea manejar ciertos tipos de condiciones, puede devolvérsele el control para dejarlo resolver el problema.

Chips con multihilamiento y multinúcleo

La ley de Moore establece que el número de transistores en un chip se duplica cada 18 meses. Esta "ley" no es ningún tipo de ley de física, como la de la conservación del momento, sino una observación hecha por Gordon Moore, cofundador de Intel, acerca de la velocidad con la que los ingenieros de procesos en las compañías de semiconductores pueden reducir sus transistores. La ley de Moore ha estado vigente durante tres décadas hasta hoy y se espera que siga así durante al menos una década más.

La abundancia de transistores está ocasionando un problema: ¿qué se debe hacer con todos ellos? En párrafos anteriores vimos una solución: las arquitecturas superescalares, con múltiples unidades funcionales. Pero a medida que se incrementa el número de transistores, se puede hacer todavía más. Algo obvio por hacer es colocar cachés más grandes en el chip de la CPU y eso está ocurriendo, pero en cierto momento se llega al punto de rendimiento decreciente.

El siguiente paso obvio es multiplicar no sólo las unidades funcionales, sino también parte de la lógica de control. El Pentium 4 y algunos otros chips de CPU tienen esta propiedad, conocida como **multihilamiento** (*multithreading*) o **hiperhilamiento** (*hyperthreading*) (el nombre que puso Intel al multihilamiento). Para una primera aproximación, lo que hace es permitir que la CPU contenga el estado de dos hilos de ejecución (*threads*) distintos y luego alterne entre uno y otro con una escala de tiempo en nanosegundos (un hilo de ejecución es algo así como un proceso ligero, que a su vez es un programa en ejecución; veremos los detalles sobre esto en el capítulo 2). Por ejemplo, si uno de los procesos necesita leer una palabra de memoria (que requiere muchos ciclos de reloj), una CPU con multihilamiento puede cambiar a otro hilo. El multihilamiento no ofrece un verdadero paralelismo. Sólo hay un proceso en ejecución a la vez, pero el tiempo de cambio entre un hilo y otro se reduce al orden de un nanosegundo.

El multihilamiento tiene consecuencias para el sistema operativo, debido a que cada hilo aparece para el sistema operativo como una CPU separada. Considere un sistema con dos CPU reales, cada una con dos hilos. El sistema operativo verá esto como si hubiera cuatro CPU. Si hay suficiente trabajo sólo para mantener ocupadas dos CPU en cierto punto en el tiempo, podría planificar de manera inadvertida dos hilos en la misma CPU, mientras que la otra CPU estaría completamente inactiva. Esta elección es mucho menos eficiente que utilizar un hilo en cada CPU. El sucesor del Pentium 4, conocido como arquitectura Core (y también Core 2), no tiene hiperhilamiento, pero Intel ha anunciado que el sucesor del Core lo tendrá nuevamente.

Más allá del multihilamiento, tenemos chips de CPU con dos, cuatro o más procesadores completos, o **núcleos** (*cores*) en su interior. Los chips de multinúcleo (*multicore*) de la figura 1-8 contienen efectivamente cuatro minichips en su interior, cada uno con su propia CPU independiente (más adelante hablaremos sobre las cachés). Para hacer uso de dicho chip multinúcleo se requiere en definitiva un sistema operativo multiprocesador.

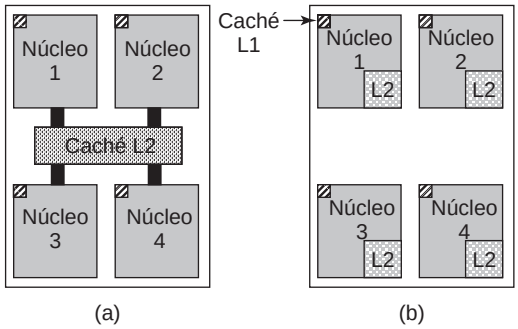


Figura 1-8. (a) Un chip de cuatro núcleos (*quad-core*) con una caché L2 compartida. (b) Un chip de cuatro núcleos con cachés L2 separadas.

1.3.2 Memoria

El segundo componente importante en cualquier computadora es la memoria. En teoría, una memoria debe ser en extremo rápida (más rápida que la velocidad de ejecución de una instrucción, de manera que la memoria no detenga a la CPU), de gran tamaño y muy económica. Ninguna tecnología en la actualidad cumple con todos estos objetivos, por lo que se adopta una solución distinta. El sistema de memoria está construido como una jerarquía de capas, como se muestra en la figura 1-9. Las capas superiores tienen mayor velocidad, menor capacidad y mayor costo por bit que las capas inferiores, a menudo por factores de mil millones o más.

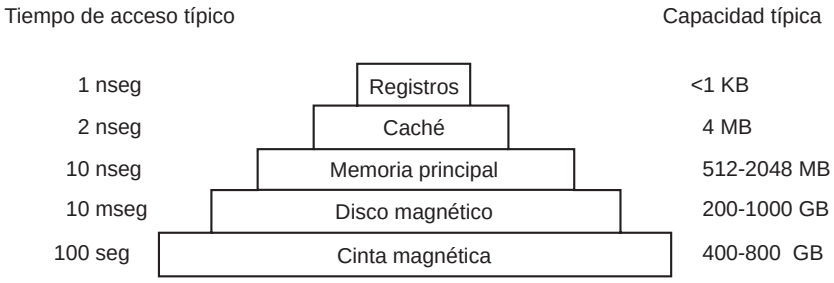


Figura 1-9. Una común jerarquía de memoria. Los números son sólo aproximaciones.

La capa superior consiste en los registros internos de la CPU. Están compuestos del mismo material que la CPU y, por ende, tienen la misma rapidez. En consecuencia no hay retraso a la hora de utilizarlos. La capacidad de almacenamiento disponible en estos registros es generalmente de 32×32 bits en una CPU de 32 bits y de 64×64 bits en una CPU de 64 bits. Menos de 1 KB en ambos casos. Los programas deben administrar los registros (es decir, decidir qué deben guardar en ellos) por su cuenta, en el software.

El siguiente nivel es la memoria caché, que el hardware controla de manera parcial. La memoria principal se divide en **líneas de caché**, que por lo general son de 64 bytes, con direcciones de 0 a 63 en la línea de caché 0, direcciones de 64 a 127 en la línea de caché 1 y así sucesivamente. Las líneas de caché que se utilizan con más frecuencia se mantienen en una caché de alta velocidad, ubicada dentro o muy cerca de la CPU. Cuando el programa necesita leer una palabra de memoria, el hardware de la caché comprueba si la línea que se requiere se encuentra en la caché. Si es así (a lo cual se le conoce como **acierto de caché**), la petición de la caché se cumple y no se envía una petición de memoria a través del bus hacia la memoria principal. Los aciertos de caché por lo general requieren un tiempo aproximado de dos ciclos de reloj. Los fallos de caché tienen que ir a memoria, con un castigo considerable de tiempo. La memoria caché está limitada en tamaño debido a su alto costo. Algunas máquinas tienen dos o incluso tres niveles de caché, cada una más lenta y más grande que la anterior.

El uso de cachés juega un papel importante en muchas áreas de las ciencias computacionales, no sólo en la caché de las líneas de RAM. Cada vez que hay un recurso extenso que se puede dividir en piezas, algunas de las cuales se utilizan con mucho más frecuencia que otras, a menudo se invoca a la caché para mejorar el rendimiento. Los sistemas operativos la utilizan todo el tiempo. Por ejemplo, la mayoría de los sistemas operativos mantienen (piezas de) los archivos que se utilizan con frecuencia en la memoria principal para evitar tener que obtenerlos del disco en forma repetida. De manera similar, los resultados de convertir nombres de rutas extensas tales como

/home/ast/proyectos/minix3/src/kernel/reloj.c

a la dirección en disco donde se encuentra el archivo, se pueden colocar en la caché para evitar búsquedas repetidas. Por último, cuando una dirección de una página Web (URL) se convierte en una dirección de red (dirección IP), el resultado se puede poner en la caché para usarlo a futuro (existen muchos otros usos).

En cualquier sistema de caché surgen con rapidez varias preguntas, incluyendo:

1. Cuándo se debe poner un nuevo elemento en la caché.
2. En qué línea de caché se debe poner el nuevo elemento.
3. Qué elemento se debe eliminar de la caché cuando se necesita una posición.
4. Dónde se debe poner un elemento recién desalojado en la memoria de mayor tamaño.

No todas las preguntas son relevantes para cada situación de uso de la caché. Para poner líneas de la memoria principal en la caché de la CPU, por lo general se introduce un nuevo elemento en cada fallo de caché. La línea de caché a utilizar se calcula generalmente mediante el uso de algunos de los bits de mayor orden de la dirección de memoria a la que se hace referencia. Por ejemplo, con 4096 líneas de caché de 64 bytes y direcciones de 32 bits, los bits del 6 al 17 podrían utilizarse para especificar la línea de caché, siendo los bits del 0 al 5 el byte dentro de la línea de la caché. En este caso, el elemento a quitar es el mismo en el que se colocan los nuevos datos, pero en otros sistemas podría ser otro. Por último, cuando se vuelve a escribir una línea de caché en la memoria principal (si se ha modificado desde la última vez que se puso en caché), la posición en memoria donde se debe volver a escribir se determina únicamente por la dirección en cuestión.

Las cachés son una idea tan útil que las CPUs modernas tienen dos de ellas. La **caché L1** o de primer nivel está siempre dentro de la CPU, y por lo general alimenta las instrucciones decodificadas al motor de ejecución de la CPU. La mayoría de los chips tienen una segunda caché L1 para las

palabras de datos que se utilizan con frecuencia. Por lo general, las cachés L1 son de 16 KB cada una. Además, a menudo hay una segunda caché, conocida como **caché L2**, que contiene varios megabytes de palabras de memoria utilizadas recientemente. La diferencia entre las cachés L1 y L2 está en la velocidad. El acceso a la caché L1 se realiza sin ningún retraso, mientras que el acceso a la caché L2 requiere un retraso de uno o dos ciclos de reloj.

En los chips multinúcleo, los diseñadores deben decidir dónde deben colocar las cachés. En la figura 1-8(a) hay una sola caché L2 compartida por todos los núcleos; esta metodología se utiliza en los chips multinúcleo de Intel. En contraste, en la figura 1-8(b) cada núcleo tiene su propia caché L2; AMD utiliza esta metodología. Cada estrategia tiene sus pros y sus contras. Por ejemplo, la caché L2 compartida de Intel requiere un dispositivo controlador de caché más complicado, pero la manera en que AMD utiliza la caché hace más difícil la labor de mantener las cachés L2 consistentes.

La memoria principal viene a continuación en la jerarquía de la figura 1-9. Es el "caballo de batalla" del sistema de memoria. Por lo general a la memoria principal se le conoce como **RAM** (*Random Access Memory*, Memoria de Acceso Aleatorio). Los usuarios de computadora antiguos algunas veces la llaman **memoria de núcleo** debido a que las computadoras en las décadas de 1950 y 1960 utilizaban pequeños núcleos de ferrita magnetizables para la memoria principal. En la actualidad, las memorias contienen desde cientos de megabytes hasta varios gigabytes y su tamaño aumenta con rapidez. Todas las peticiones de la CPU que no se puedan satisfacer desde la caché pasan a la memoria principal.

Además de la memoria principal, muchas computadoras tienen una pequeña cantidad de memoria de acceso aleatorio no volátil. A diferencia de la RAM, la memoria no volátil no pierde su contenido cuando se desconecta la energía. La **ROM** (*Read Only Memory*, Memoria de sólo lectura) se programa en la fábrica y no puede modificarse después. Es rápida y económica. En algunas computadoras, el cargador de arranque (*bootstrap loader*) que se utiliza para iniciar la computadora está contenido en la ROM. Además, algunas tarjetas de E/S vienen con ROM para manejar el control de los dispositivos de bajo nivel.

La **EEPROM** (*Electrically Erasable PROM*, PROM eléctricamente borrable) y la **memoria flash** son también no volátiles, pero en contraste con la ROM se pueden borrar y volver a escribir datos en ellas. Sin embargo, para escribir en este tipo de memorias se requiere mucho más tiempo que para escribir en la RAM, por lo cual se utilizan en la misma forma que la ROM, sólo con la característica adicional de que ahora es posible corregir los errores en los programas que contienen, mediante la acción de volver a escribir datos en ellas en el campo de trabajo.

La memoria flash también se utiliza comúnmente como el medio de almacenamiento en los dispositivos electrónicos portátiles. Sirve como película en las cámaras digitales y como el disco en los reproductores de música portátiles, para nombrar sólo dos usos. La memoria flash se encuentra entre la RAM y el disco en cuanto a su velocidad. Además, a diferencia de la memoria en disco, si se borra demasiadas veces, se desgasta.

Otro tipo más de memoria es CMOS, la cual es volátil. Muchas computadoras utilizan memoria CMOS para guardar la fecha y hora actuales. La memoria CMOS y el circuito de reloj que incrementa la hora en esta memoria están energizados por una pequeña batería, por lo que la hora se actualiza en forma correcta aun cuando la computadora se encuentre desconectada. La memoria CMOS también puede contener los parámetros de configuración, como el disco del cual se debe iniciar el sistema. Se utiliza CMOS debido a que consume tan poca energía que la batería instalada en

la fábrica dura varios años. Sin embargo, cuando empieza a fallar puede parecer como si la computadora tuviera la enfermedad de Alzheimer, olvidando cosas que ha sabido durante años, como desde cuál disco duro se debe iniciar el sistema.

1.3.3 Discos

El siguiente lugar en la jerarquía corresponde al disco magnético (disco duro). El almacenamiento en disco es dos órdenes de magnitud más económico que la RAM por cada bit, y a menudo es dos órdenes de magnitud más grande en tamaño también. El único problema es que el tiempo para acceder en forma aleatoria a los datos en ella es de cerca de tres órdenes de magnitud más lento. Esta baja velocidad se debe al hecho de que un disco es un dispositivo mecánico, como se muestra en la figura 1-10.

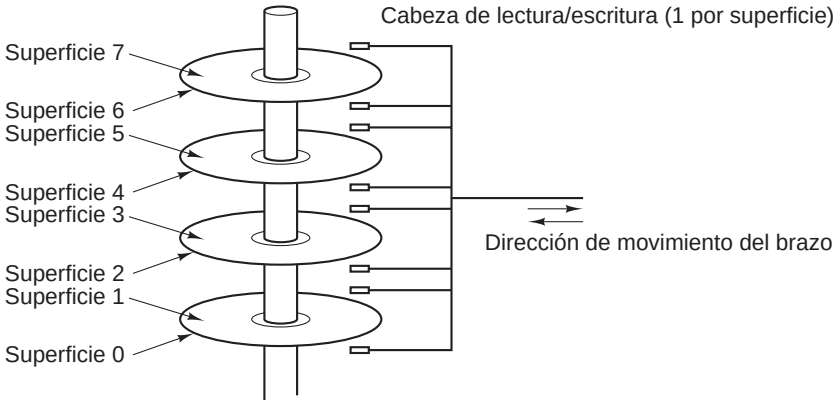


Figura 1-10. Estructura de una unidad de disco.

Un disco consiste en uno o más platos que giran a 5400, 7200 o 10,800 rpm. Un brazo mecánico, con un punto de giro colocado en una esquina, se mueve sobre los platos de manera similar al brazo de la aguja en un viejo tocadiscos. La información se escribe en el disco en una serie de círculos concéntricos. En cualquier posición dada del brazo, cada una de las cabezas puede leer una región anular conocida como **pista** (*track*). En conjunto, todas las pistas para una posición dada del brazo forman un **cilindro** (*cylinder*).

Cada pista se divide en cierto número de sectores, por lo general de 512 bytes por sector. En los discos modernos, los cilindros exteriores contienen más sectores que los interiores. Para desplazar el brazo de un cilindro al siguiente se requiere aproximadamente 1 milisegundo. Para desplazar el brazo a un cilindro aleatoriamente se requieren por lo general de 5 a 10 milisegundos, dependiendo de la unidad. Una vez que el brazo se encuentra en la pista correcta, la unidad debe esperar a que el sector necesario gire hacia abajo de la cabeza, con un retraso adicional de 5 a 10 milisegundos, dependiendo de las rpm de la unidad. Una vez que el sector está bajo la cabeza, la lectura o escritura ocurre a una velocidad de 50 MB/seg en los discos de bajo rendimiento hasta de 160 MB/seg en los discos más rápidos.

Muchas computadoras presentan un esquema conocido como **memoria virtual** (*virtual memory*), el cual describiremos hasta cierto punto en el capítulo 3. Este esquema hace posible la

ejecución de programas más grandes que la memoria física al colocarlos en el disco y utilizar la memoria principal como un tipo de caché para las partes que se ejecutan con más frecuencia. Este esquema requiere la reasignación de direcciones de memoria al instante, para convertir la dirección que el programa generó en la dirección física en la RAM en donde se encuentra la palabra. Esta asignación se realiza mediante una parte de la CPU conocida como **MMU** (*Memory Management Unit*, Unidad de Administración de Memoria), como se muestra en la figura 1-6.

La presencia de la caché y la MMU pueden tener un gran impacto en el rendimiento. En un sistema de multiprogramación, al cambiar de un programa a otro (lo que se conoce comúnmente como **cambio de contexto** o *context switch*), puede ser necesario vaciar todos los bloques modificados de la caché y modificar los registros de asignación en la MMU. Ambas operaciones son costosas y los programadores se esfuerzan bastante por evitarlas. Más adelante veremos algunas de las consecuencias de sus tácticas.

1.3.4 Cintas

La última capa de la jerarquía en la memoria es la cinta magnética. Este medio se utiliza con frecuencia como respaldo para el almacenamiento en disco y para contener conjuntos de datos muy extensos. Para acceder a una cinta, primero debe colocarse en un lector de cinta, ya sea que lo haga una persona o un robot (el manejo automatizado de las cintas es común en las instalaciones con bases de datos enormes). Después la cinta tal vez tenga que embobinarse hacia delante para llegar al bloque solicitado. En general, este proceso podría tardar varios minutos. La gran ventaja de la cinta es que es en extremo económica por bit y removible, lo cual es importante para las cintas de respaldo que se deben almacenar fuera del sitio de trabajo para que puedan sobrevivir a los incendios, inundaciones, terremotos y otros desastres.

La jerarquía de memoria que hemos descrito es la común, pero algunas instalaciones no tienen todas las capas o tienen unas cuantas capas distintas (como el disco óptico). Aún así, a medida que se desciende por todos las capas en la jerarquía, el tiempo de acceso aleatorio se incrementa en forma dramática, la capacidad aumenta de igual forma y el costo por bit baja considerablemente. En consecuencia, es probable que las jerarquías de memoria se utilicen por varios años más.

1.3.5 Dispositivos de E/S

La CPU y la memoria no son los únicos recursos que el sistema operativo debe administrar. Los dispositivos de E/S también interactúan mucho con el sistema operativo. Como vimos en la figura 1-6, los dispositivos de E/S generalmente constan de dos partes: un dispositivo controlador y el dispositivo en sí. El dispositivo controlador es un chip o conjunto de chips que controla físicamente el dispositivo. Por ejemplo, acepta los comandos del sistema operativo para leer datos del dispositivo y los lleva a cabo.

En muchos casos, el control del dispositivo es muy complicado y detallado, por lo que el trabajo del chip o los chips del dispositivo controlador es presentar una interfaz más simple al sistema operativo (pero de todas formas sigue siendo muy complejo). Por ejemplo, un controlador de disco podría aceptar un comando para leer el sector 11,206 del disco 2; después, tiene que convertir este número de

sector lineal en un cilindro, sector y cabeza. Esta conversión se puede complicar por el hecho de que los cilindros exteriores tienen más sectores que los interiores y que algunos sectores defectuosos se han reasignado a otros. Posteriormente, el dispositivo controlador tiene que determinar en cuál cilindro se encuentra el brazo y darle una secuencia de pulsos para desplazarse hacia dentro o hacia fuera el número requerido de cilindros; tiene que esperar hasta que el sector apropiado haya girado bajo la cabeza, y después empieza a leer y almacenar los bits a medida que van saliendo de la unidad, eliminando el preámbulo y calculando la suma de verificación. Por último, tiene que ensamblar los bits entrantes en palabras y almacenarlos en la memoria. Para hacer todo este trabajo, a menudo los dispositivos controladores consisten en pequeñas computadoras incrustadas que se programan para realizar su trabajo.

La otra pieza es el dispositivo en sí. Los dispositivos tienen interfaces bastante simples, debido a que no pueden hacer mucho y también para estandarizarlas. Esto último es necesario de manera que cualquier dispositivo controlador de disco IDE pueda manejar cualquier disco IDE, por ejemplo. **IDE** (*Integrated Drive Electronics*) significa **Electrónica de unidades integradas** y es el tipo estándar de disco en muchas computadoras. Como la interfaz real del dispositivo está oculta detrás del dispositivo controlador, todo lo que el sistema operativo ve es la interfaz para el dispositivo controlador, que puede ser bastante distinta de la interfaz para el dispositivo.

Como cada tipo de dispositivo controlador es distinto, se requiere software diferente para controlar cada uno de ellos. El software que se comunica con un dispositivo controlador, que le proporciona comandos y acepta respuestas, se conoce como **driver** (controlador). Cada fabricante de dispositivos controladores tiene que suministrar un driver específico para cada sistema operativo en que pueda funcionar. Así, un escáner puede venir, por ejemplo, con drivers para Windows 2000, Windows XP, Vista y Linux.

Para utilizar el driver, se tiene que colocar en el sistema operativo de manera que pueda ejecutarse en modo kernel. En realidad, los drivers se pueden ejecutar fuera del kernel, pero sólo unos cuantos sistemas actuales admiten esta posibilidad debido a que se requiere la capacidad para permitir que un driver en espacio de usuario pueda acceder al dispositivo de una manera controlada, una característica que raras veces se admite. Hay tres formas en que el driver se pueda colocar en el kernel: la primera es volver a enlazar el kernel con el nuevo driver y después reiniciar el sistema (muchos sistemas UNIX antiguos trabajan de esta manera); la segunda es crear una entrada en un archivo del sistema operativo que le indique que necesita el driver y después reinicie el sistema, para que en el momento del arranque, el sistema operativo busque los drivers necesarios y los cargue (Windows funciona de esta manera); la tercera forma es que el sistema operativo acepte nuevos drivers mientras los ejecuta e instala al instante, sin necesidad de reiniciar. Esta última forma solía ser rara, pero ahora se está volviendo mucho más común. Los dispositivos conectables en caliente *(hot-pluggable*), como los dispositivos USB e IEEE 1394 (que se describen a continuación) siempre necesitan drivers que se cargan en forma dinámica.

Todo dispositivo controlador tiene un número pequeño de registros que sirven para comunicarse con él. Por ejemplo, un dispositivo controlador de disco con las mínimas características podría tener registros para especificar la dirección de disco, dirección de memoria, número de sectores e instrucción (lectura o escritura). Para activar el dispositivo controlador, el driver recibe un comando del sistema operativo y después lo traduce en los valores apropiados para escribirlos en los registros del dispositivo. La colección de todos los registros del dispositivo forma el **espacio de puertos de E/S**, un tema al que regresaremos en el capítulo 5.

En ciertas computadoras, los registros de dispositivo tienen una correspondencia con el espacio de direcciones del sistema operativo (las direcciones que puede utilizar), de modo que se puedan leer y escribir en ellas como si fuera en palabras de memoria ordinarias. En dichas computadoras no se requieren instrucciones de E/S especiales y los programas de usuario pueden aislarse del hardware al no colocar estas direcciones de memoria dentro de su alcance (por ejemplo, mediante el uso de registros base y límite). En otras computadoras, los registros de dispositivo se colocan en un espacio de puertos de E/S especial, donde cada registro tiene una dirección de puerto. En estas máquinas hay instrucciones `IN` y `OUT` especiales disponibles en modo kernel que permiten a los drivers leer y escribir en los registros. El primer esquema elimina la necesidad de instrucciones de E/S especiales, pero utiliza parte del espacio de direcciones. El segundo esquema no utiliza espacio de direcciones, pero requiere instrucciones especiales. Ambos sistemas se utilizan ampliamente.

Las operaciones de entrada y salida se pueden realizar de tres maneras distintas. En el método más simple, un programa de usuario emite una llamada al sistema, que el kernel posteriormente traduce en una llamada al procedimiento para el driver apropiado. Después el driver inicia la E/S y permanece en un ciclo estrecho, sondeando en forma continua al dispositivo para ver si ha terminado (por lo general hay un bit que indica si el dispositivo sigue ocupado). Una vez terminada la E/S, el driver coloca los datos (si los hay) en donde se necesitan y regresa. Después el sistema operativo devuelve el control al llamador. A este método se le conoce como **espera ocupada** y tiene la desventaja de que mantiene ocupada la CPU sondeando al dispositivo hasta que éste termina.

El segundo método consiste en que el driver inicie el dispositivo y le pida generar una interrupción cuando termine. En este punto el driver regresa. Luego, el sistema operativo bloquea el programa llamador si es necesario y busca otro trabajo por hacer. Cuando el dispositivo controlador detecta el final de la transferencia, genera una **interrupción** para indicar que la operación se ha completado.

Las interrupciones son muy importantes en los sistemas operativos, por lo cual vamos a examinar la idea con más detalle. En la figura 1-11(a) podemos ver un proceso de tres pasos para la E/S. En el paso 1, el driver indica al dispositivo controlador de disco lo que debe hacer, al escribir datos en sus registros de dispositivo. Después el dispositivo controlador inicia el dispositivo; cuando ha terminado de leer o escribir el número de bytes que debe transferir, alerta al chip controlador de interrupciones mediante el uso de ciertas líneas de bus en el paso 2. Si el controlador de interrupciones está preparado para aceptar la interrupción (lo cual podría no ser cierto si está ocupado con una de mayor prioridad), utiliza un pin en el chip de CPU para informarlo, en el paso 3. En el paso 4, el controlador de interrupciones coloca el número del dispositivo en el bus, para que la CPU pueda leerlo y sepa cuál dispositivo acaba de terminar (puede haber muchos dispositivos funcionando al mismo tiempo).

Una vez que la CPU ha decidido tomar la interrupción, el contador de programa y el PSW son típicamente agregados (*pushed*) en la pila actual y la CPU cambia al modo kernel. El número de dispositivo se puede utilizar como un índice en parte de la memoria para encontrar la dirección del manejador (*handler*) de interrupciones para este dispositivo. Esta parte de la memoria se conoce como **vector de interrupción**. Una vez que el manejador de interrupciones (parte del driver para el dispositivo que está realizando la interrupción) ha iniciado, quita el contador de programa y el PSW de la pila y los guarda, para después consultar al dispositivo y conocer su estado. Cuando el manejador de interrupciones termina, regresa al programa de usuario que se estaba ejecutando previamente a la primera instrucción que no se había ejecutado todavía. Estos pasos se muestran en la figura 1-11(b).

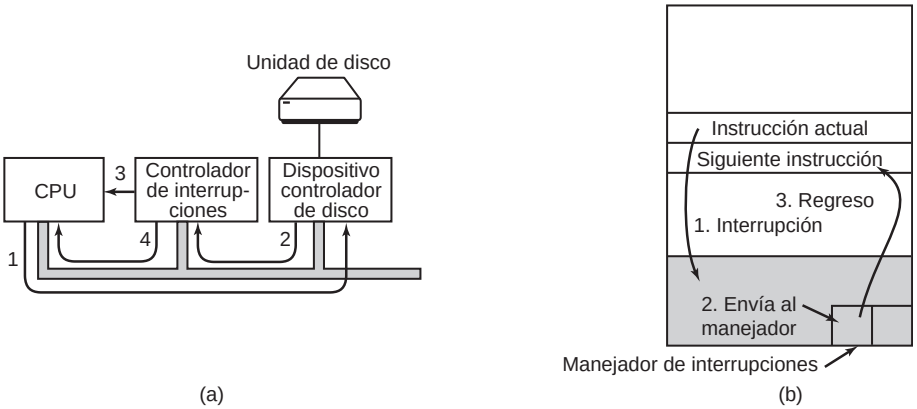


Figura 1-11. (a) Los pasos para iniciar un dispositivo de E/S y obtener una interrupción. (b) El procesamiento de interrupciones involucra tomar la interrupción, ejecutar el manejador de interrupciones y regresar al programa de usuario.

El tercer método para realizar operaciones de E/S hace uso de un chip especial llamado **DMA** (*Direct Memory Access;* Acceso directo a memoria) que puede controlar el flujo de bits entre la memoria y un dispositivo controlador sin la intervención constante de la CPU. La CPU configura el chip DMA, le indica cuántos bytes debe transferir, las direcciones de dispositivo y de memoria involucradas, la instrucción y deja que haga su trabajo. Cuando el chip DMA termina genera una interrupción, la cual se maneja de la manera antes descrita. En el capítulo 5 discutiremos con más detalle sobre el hardware de DMA y de E/S, en general.

A menudo, las interrupciones pueden ocurrir en momentos muy inconvenientes, por ejemplo mientras otro manejador de interrupciones se está ejecutando. Por esta razón, la CPU tiene una forma para deshabilitar las interrupciones y rehabilitarlas después. Mientras las interrupciones están deshabilitadas, cualquier dispositivo que termine continúa utilizando sus señales de interrupción, pero la CPU no se interrumpe sino hasta que se vuelven a habilitar las interrupciones. Si varios dispositivos terminan mientras las interrupciones están habilitadas, el controlador de interrupciones decide cuál debe dejar pasar primero, lo cual se basa generalmente en prioridades estáticas asignadas a cada dispositivo. El dispositivo de mayor prioridad gana.

1.3.6 Buses

La organización de la figura 1-6 se utilizó en las minicomputadoras durante años y también en la IBM PC original. Sin embargo, a medida que los procesadores y las memorias se hicieron más veloces, la habilidad de un solo bus (y sin duda, del bus de la IBM PC) de manejar todo el tráfico se forzaba hasta el punto de quiebre. Algo tenía que ceder. Como resultado se agregaron más buses, tanto para dispositivos de E/S más rápidos como para el tráfico entre la CPU y la memoria. Como consecuencia de esta evolución, un sistema Pentium extenso tiene actualmente una apariencia similar a la figura 1-12.

El sistema tiene ocho buses (caché, local, memoria, PCI, SCSI, USB, IDE e ISA), cada uno con una velocidad de transferencia y función distintas. El sistema operativo debe estar al tanto de todos

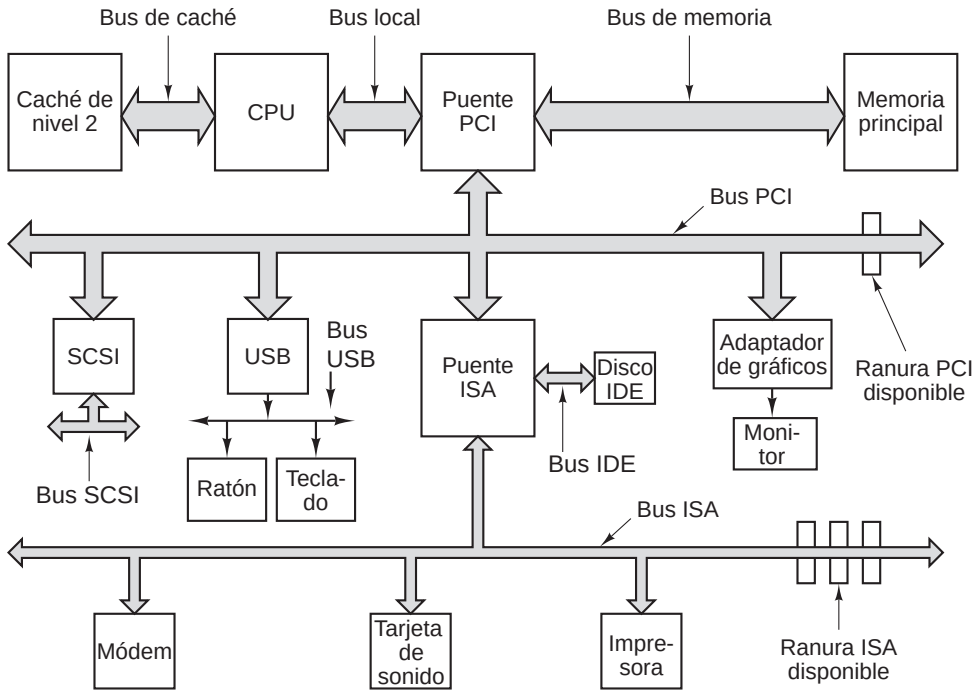


Figura 1-12. La estructura de un sistema Pentium extenso.

estos buses para su configuración y administración. Los dos buses principales son el bus **ISA** (*Industry Standard Architecture*, Arquitectura estándar de la industria) de la IBM PC original y su sucesor, el bus **PCI** (*Peripheral Component Interconnect*, Interconexión de componentes periféricos). El bus ISA (el bus original de la IBM PC/AT) opera a 8.33 MHz y puede transferir 2 bytes a la vez, para una velocidad máxima de 16.67 MB/seg. Se incluye para mantener compatibilidad hacia atrás con las tarjetas de E/S antiguas y lentas. Los sistemas modernos lo omiten con frecuencia, pues ya es obsoleto. El bus PCI fue inventado por Intel como sucesor para el bus ISA. Puede operar a 66 MHz y transferir 8 bytes a la vez, para lograr una velocidad de transferencia de datos de 528 MB/seg. La mayoría de los dispositivos de E/S de alta velocidad utilizan el bus PCI en la actualidad. Incluso algunas computadoras que no emplean procesadores Intel usan el bus PCI, debido al extenso número de tarjetas de E/S disponibles para este bus. Las nuevas computadoras están saliendo al mercado con una versión actualizad del bus PCI, conocida como **PCI Express**.

En esta configuración, la CPU se comunica con el chip puente PCI a través del bus local y el chip puente PCI se comunica con la memoria a través de un bus de memoria dedicado, que normalmente opera a 100 MHz. Los sistemas Pentium tienen una caché de nivel 1 en el chip y una caché de nivel 2 mucho mayor fuera del chip, conectada a la CPU mediante el bus de caché.

Además, este sistema contiene tres buses especializados: IDE, USB y SCSI. El bus IDE sirve para conectar dispositivos periféricos tales como discos y CD-ROM al sistema. El bus IDE es

fruto de la interfaz controladora de disco en la PC/AT y ahora es estándar en casi todos los sistemas basados en Pentium para el disco duro y a menudo para el CD-ROM.

El **USB** (*Universal Serial Bus*; Bus serial universal) se inventó para conectar a la computadora todos los dispositivos de E/S lentos, como el teclado y el ratón. Utiliza un pequeño conector con cuatro cables, dos de los cuales suministran energía eléctrica a los dispositivos USB. El USB es un bus centralizado en el que un dispositivo raíz sondea los dispositivos de E/S cada 1 milisegundo para ver si tienen tráfico. USB 1.0 podía manejar una carga agregada de 1.5 MB/seg, pero el más reciente USB 2.0 puede manejar 60 MB/seg. Todos los dispositivos USB comparten un solo dispositivo controlador USB, lo que hace innecesario instalar un nuevo controlador para cada nuevo dispositivo USB. En consecuencia, pueden agregarse dispositivos USB a la computadora sin necesidad de reiniciar.

El bus **SCSI** (*Small Computer System Interface*, Interfaz para sistemas de cómputo pequeños) es un bus de alto rendimiento, diseñado para discos, escáneres y otros dispositivos veloces que necesitan de un ancho de banda considerable. Puede operar a una velocidad de transferencia de hasta 160 MB/seg. Ha estado presente en los sistemas Macintosh desde que se inventaron y también es popular en UNIX y en ciertos sistemas basados en Intel.

Hay otro bus (que no se muestra en la figura 1-12) conocido como **IEEE 1394**. Algunas veces se le conoce como FireWire, aunque hablando en sentido estricto, FireWire es el nombre que utiliza Apple para su implementación del 1394. Al igual que el USB, el IEEE 1394 es un bus de bits en serie, pero está diseñado para transferencias empaquetadas de hasta 100 MB/seg., lo que lo hace conveniente para conectar a una computadora cámaras de video digitales y dispositivos multimedia similares. A diferencia del USB, el IEE 1394 no tiene un dispositivo controlador central.

Para trabajar en un entorno tal como el de la figura 1-12, el sistema operativo tiene que saber qué dispositivos periféricos están conectados a la computadora y cómo configurarlos. Este requerimiento condujo a Intel y Microsoft a diseñar un sistema de PC conocido como **plug and play** basado en un concepto similar que se implementó por primera vez en la Apple Macintosh. Antes de plug and play, cada tarjeta de E/S tenía un nivel de petición de interrupción fijo y direcciones fijas para sus registros de E/S. Por ejemplo, el teclado tenía la interrupción 1 y utilizaba las direcciones de E/S 0x60 a 0x64, el dispositivo controlador de disco flexible tenía la interrupción 6 y utilizaba las direcciones de E/S 0x3F0 a 0x3F7, la impresora tenía la interrupción 7 y utilizaba las direcciones de E/S 0x378 a 0x37A, y así sucesivamente.

Hasta aquí todo está bien. El problema llegó cuando el usuario compraba una tarjeta de sonido y una tarjeta de módem que utilizaban la misma interrupción, por ejemplo, la 4. Instaladas juntas serían incapaces de funcionar. La solución fue incluir interruptores DIP o puentes (*jumpers*) en cada tarjeta de E/S e indicar al usuario que por favor los configurara para seleccionar un nivel de interrupción y direcciones de dispositivos de E/S que no estuvieran en conflicto con las demás tarjetas en el sistema del usuario. Los adolescentes que dedicaron sus vidas a las complejidades del hardware de la PC podían algunas veces hacer esto sin cometer errores. Por desgracia nadie más podía hacerlo, lo cual provocó un caos.

La función de plug and play es permitir que el sistema recolecte automáticamente la información acerca de los dispositivos de E/S, asigne los niveles de interrupción y las direcciones de E/S de manera central, para que después indique a cada tarjeta cuáles son sus números. Este trabajo está íntimamente relacionado con el proceso de arranque de la computadora, por lo que a continuación analizaremos este proceso nada trivial.

1.3.7 Arranque de la computadora

En forma muy breve, el proceso de arranque del Pentium es el siguiente. Cada Pentium contiene una tarjeta madre (*motherboard*). En la tarjeta madre o padre hay un programa conocido como **BIOS** (*Basic Input Output System*, Sistema básico de entrada y salida) del sistema. El BIOS contiene software de E/S de bajo nivel, incluyendo procedimientos para leer el teclado, escribir en la pantalla y realizar operaciones de E/S de disco, entre otras cosas. Hoy en día está contenido en una RAM tipo flash que es no volátil pero el sistema operativo puede actualizarla cuando se encuentran errores en el BIOS.

Cuando se arranca la computadora, el BIOS inicia su ejecución. Primero hace pruebas para ver cuánta RAM hay instalada y si el teclado junto con otros dispositivos básicos están instalados y responden en forma correcta. Empieza explorando los buses ISA y PCI para detectar todos los dispositivos conectados a ellos. Comúnmente, algunos de estos dispositivos son **heredados** (es decir, se diseñaron antes de inventar la tecnología plug and play), además de tener valores fijos para los niveles de interrupciones y las direcciones de E/S (que posiblemente se establecen mediante interruptores o puentes en la tarjeta de E/S, pero que el sistema operativo no puede modificar). Estos dispositivos se registran; y los dispositivos plug and play también. Si los dispositivos presentes son distintos de los que había cuando el sistema se inició por última vez, se configuran los nuevos dispositivos.

Después, el BIOS determina el dispositivo de arranque, para lo cual prueba una lista de dispositivos almacenada en la memoria CMOS. El usuario puede cambiar esta lista si entra a un programa de configuración del BIOS, justo después de iniciar el sistema. Por lo general, se hace un intento por arrancar del disco flexible, si hay uno presente. Si eso falla, se hace una consulta a la unidad de CD-ROM para ver si contiene un CD-ROM que se pueda arrancar. Si no hay disco flexible ni CD-ROM que puedan iniciarse, el sistema se arranca desde el disco duro. El primer sector del dispositivo de arranque se lee y se coloca en la memoria, para luego ejecutarse. Este sector contiene un programa que por lo general examina la tabla de particiones al final del sector de arranque, para determinar qué partición está activa. Después se lee un cargador de arranque secundario de esa partición. Este cargador lee el sistema operativo de la partición activa y lo inicia.

Luego, el sistema operativo consulta al BIOS para obtener la información de configuración. Para cada dispositivo, comprueba si tiene el driver correspondiente. De no ser así, pide al usuario que inserte un CD-ROM que contenga el driver (suministrado por el fabricante del dispositivo). Una vez que tiene los drivers de todos los dispositivos, el sistema operativo los carga en el kernel. Después inicializa sus tablas, crea los procesos de segundo plano que se requieran, y arranca un programa de inicio de sesión o GUI.

1.4 LOS TIPOS DE SISTEMAS OPERATIVOS

Los sistemas operativos han estado en funcionamiento durante más de medio siglo. Durante este tiempo se ha desarrollado una variedad bastante extensa de ellos, no todos se conocen ampliamente. En esta sección describiremos de manera breve nueve. Más adelante en el libro regresaremos a ver algunos de estos distintos tipos de sistemas.

1.4.1 Sistemas operativos de mainframe

En el extremo superior están los sistemas operativos para las mainframes, las computadoras del tamaño de un cuarto completo que aún se encuentran en los principales centros de datos corporativos. La diferencia entre estas computadoras y las personales está en su capacidad de E/S. Una mainframe con 1000 discos y millones de gigabytes de datos no es poco común; una computadora personal con estas especificaciones sería la envidia de los amigos del propietario. Las mainframes también están volviendo a figurar en el ámbito computacional como servidores Web de alto rendimiento, servidores para sitios de comercio electrónico a gran escala y servidores para transacciones de negocio a negocio.

Los sistemas operativos para las mainframes están profundamente orientados hacia el procesamiento de muchos trabajos a la vez, de los cuales la mayor parte requiere muchas operaciones de E/S. Por lo general ofrecen tres tipos de servicios: procesamiento por lotes, procesamiento de transacciones y tiempo compartido. Un sistema de procesamiento por lotes procesa los trabajos de rutina sin que haya un usuario interactivo presente. El procesamiento de reclamaciones en una compañía de seguros o el reporte de ventas para una cadena de tiendas son actividades que se realizan comúnmente en modo de procesamiento por lotes. Los sistemas de procesamiento de transacciones manejan grandes cantidades de pequeñas peticiones, por ejemplo: el procesamiento de cheques en un banco o las reservaciones en una aerolínea. Cada unidad de trabajo es pequeña, pero el sistema debe manejar cientos o miles por segundo. Los sistemas de tiempo compartido permiten que varios usuarios remotos ejecuten trabajos en la computadora al mismo tiempo, como consultar una gran base de datos. Estas funciones están íntimamente relacionadas; a menudo los sistemas operativos de las mainframes las realizan todas. Un ejemplo de sistema operativo de mainframe es el OS/390, un descendiente del OS/360. Sin embargo, los sistemas operativos de mainframes están siendo reemplazados gradualmente por variantes de UNIX, como Linux.

1.4.2 Sistemas operativos de servidores

En el siguiente nivel hacia abajo se encuentran los sistemas operativos de servidores. Se ejecutan en servidores, que son computadoras personales muy grandes, estaciones de trabajo o incluso mainframes. Dan servicio a varios usuarios a la vez a través de una red y les permiten compartir los recursos de hardware y de software. Los servidores pueden proporcionar servicio de impresión, de archivos o Web. Los proveedores de Internet operan muchos equipos servidores para dar soporte a sus clientes y los sitios Web utilizan servidores para almacenar las páginas Web y hacerse cargo de las peticiones entrantes. Algunos sistemas operativos de servidores comunes son Solaris, FreeBSD, Linux y Windows Server 200x.

1.4.3 Sistemas operativos de multiprocesadores

Una manera cada vez más común de obtener poder de cómputo de las grandes ligas es conectar varias CPU en un solo sistema. Dependiendo de la exactitud con la que se conecten y de lo que se comparta, estos sistemas se conocen como computadoras en paralelo, multicomputadoras o multiprocesadores. Necesitan sistemas operativos especiales, pero a menudo son variaciones de los sistemas operativos de servidores con características especiales para la comunicación, conectividad y consistencia.

Con la reciente llegada de los chips multinúcleo para las computadoras personales, hasta los sistemas operativos de equipos de escritorio y portátiles convencionales están empezando a lidiar con multiprocesadores de al menos pequeña escala y es probable que el número de núcleos aumente con el tiempo. Por fortuna, se conoce mucho acerca de los sistemas operativos de multiprocesadores gracias a los años de investigación previa, por lo que el uso de este conocimiento en los sistemas multinúcleo no debe presentar dificultades. La parte difícil será hacer que las aplicaciones hagan uso de todo este poder de cómputo. Muchos sistemas operativos populares (incluyendo Windows y Linux) se ejecutan en multiprocesadores.

1.4.4 Sistemas operativos de computadoras personales

La siguiente categoría es el sistema operativo de computadora personal. Todos los sistemas operativos modernos soportan la multiprogramación, con frecuencia se inician docenas de programas al momento de arrancar el sistema. Su trabajo es proporcionar buen soporte para un solo usuario. Se utilizan ampliamente para el procesamiento de texto, las hojas de cálculo y el acceso a Internet. Algunos ejemplos comunes son Linux, FreeBSD, Windows Vista y el sistema operativo Macintosh. Los sistemas operativos de computadora personal son tan conocidos que tal vez no sea necesario presentarlos con mucho detalle. De hecho, muchas personas ni siquiera están conscientes de que existen otros tipos de sistemas operativos.

1.4.5 Sistemas operativos de computadoras de bolsillo

Continuando con los sistemas cada vez más pequeños, llegamos a las computadoras de bolsillo (*handheld*). Una computadora de bolsillo o **PDA** (*Personal Digital Assitant*, Asistente personal digital) es una computadora que cabe en los bolsillos y realiza una pequeña variedad de funciones, como libreta de direcciones electrónica y bloc de notas. Además, hay muchos teléfonos celulares muy similares a los PDAs, con la excepción de su teclado y pantalla. En efecto, los PDAs y los teléfonos celulares se han fusionado en esencia y sus principales diferencias se observan en el tamaño, el peso y la interfaz de usuario. Casi todos ellos se basan en CPUs de 32 bits con el modo protegido y ejecutan un sofisticado sistema operativo.

Los sistemas operativos que operan en estos dispositivos de bolsillo son cada vez más sofisticados, con la habilidad de proporcionar telefonía, fotografía digital y otras funciones. Muchos de ellos también ejecutan aplicaciones desarrolladas por terceros. De hecho, algunos están comenzando a asemejarse a los sistemas operativos de computadoras personales de hace una década. Una de las principales diferencias entre los dispositivos de bolsillo y las PCs es que los primeros no tienen discos duros de varios cientos de gigabytes, lo cual cambia rápidamente. Dos de los sistemas operativos más populares para los dispositivos de bolsillo son Symbian OS y Palm OS.

1.4.6 Sistemas operativos integrados

Los sistemas integrados (*embedded*), que también se conocen como incrustados o embebidos, operan en las computadoras que controlan dispositivos que no se consideran generalmente como computadoras, ya que no aceptan software instalado por el usuario. Algunos ejemplos comunes son los hornos

de microondas, las televisiones, los autos, los grabadores de DVDs, los teléfonos celulares y los reproductores de MP3. La propiedad principal que diferencia a los sistemas integrados de los dispositivos de bolsillo es la certeza de que nunca se podrá ejecutar software que no sea confiable. No se pueden descargar nuevas aplicaciones en el horno de microondas; todo el software se encuentra en ROM. Esto significa que no hay necesidad de protección en las aplicaciones, lo cual conlleva a cierta simplificación. Los sistemas como QNX y VxWorks son populares en este dominio.

1.4.7 Sistemas operativos de nodos sensores

Las redes de pequeños nodos sensores se están implementando para varios fines. Estos nodos son pequeñas computadoras que se comunican entre sí con una estación base, mediante el uso de comunicación inalámbrica. Estas redes de sensores se utilizan para proteger los perímetros de los edificios, resguardar las fronteras nacionales, detectar incendios en bosques, medir la temperatura y la precipitación para el pronóstico del tiempo, deducir información acerca del movimiento de los enemigos en los campos de batalla y mucho más.

Los sensores son pequeñas computadoras con radios integrados y alimentadas con baterías. Tienen energía limitada y deben trabajar durante largos periodos al exterior y desatendidas, con frecuencia en condiciones ambientales rudas. La red debe ser lo bastante robusta como para tolerar fallas en los nodos individuales, que ocurren con mayor frecuencia a medida que las baterías empiezan a agotarse.

Cada nodo sensor es una verdadera computadora, con una CPU, RAM, ROM y uno o más sensores ambientales. Ejecuta un sistema operativo pequeño pero real, por lo general manejador de eventos, que responde a los eventos externos o realiza mediciones en forma periódica con base en un reloj interno. El sistema operativo tiene que ser pequeño y simple debido a que los nodos tienen poca RAM y el tiempo de vida de las baterías es una cuestión importante. Además, al igual que con los sistemas integrados, todos los programas se cargan por adelantado; los usuarios no inician repentinamente programas que descargaron de Internet, lo cual simplifica el diseño en forma considerable. TinyOS es un sistema operativo bien conocido para un nodo sensor.

1.4.8 Sistemas operativos en tiempo real

Otro tipo de sistema operativo es el sistema en tiempo real. Estos sistemas se caracterizan por tener el tiempo como un parámetro clave. Por ejemplo, en los sistemas de control de procesos industriales, las computadoras en tiempo real tienen que recolectar datos acerca del proceso de producción y utilizarlos para controlar las máquinas en la fábrica. A menudo hay tiempos de entrega estrictos que se deben cumplir. Por ejemplo, si un auto se desplaza sobre una línea de ensamblaje, deben llevarse a cabo ciertas acciones en determinados instantes. Si un robot soldador realiza su trabajo de soldadura antes o después de tiempo, el auto se arruinará. Si la acción *debe* ocurrir sin excepción en cierto momento (o dentro de cierto rango), tenemos un **sistema en tiempo real duro**. Muchos de estos sistemas se encuentran en el control de procesos industriales, en aeronáutica, en la milicia y en áreas de aplicación similares. Estos sistemas deben proveer garantías absolutas de que cierta acción ocurrirá en un instante determinado.

Otro tipo de sistema en tiempo real es el **sistema en tiempo real suave**, en el cual es aceptable que muy ocasionalmente se pueda fallar a un tiempo predeterminado. Los sistemas de audio digital o de multimedia están en esta categoría. Los teléfonos digitales también son ejemplos de sistema en tiempo real suave.

Como en los sistemas en tiempo real es crucial cumplir con tiempos predeterminados para realizar una acción, algunas veces el sistema operativo es simplemente una biblioteca enlazada con los programas de aplicación, en donde todo está acoplado en forma estrecha y no hay protección entre cada una de las partes del sistema. Un ejemplo de este tipo de sistema en tiempo real es e-Cos.

Las categorías de sistemas para computadoras de bolsillo, sistemas integrados y sistemas en tiempo real se traslapan en forma considerable. Casi todos ellos tienen por lo menos ciertos aspectos de tiempo real suave. Los sistemas integrados y de tiempo real sólo ejecutan software que colocan los diseñadores del sistema; los usuarios no pueden agregar su propio software, lo cual facilita la protección. Los sistemas de computadoras de bolsillo y los sistemas integrados están diseñados para los consumidores, mientras que los sistemas en tiempo real son más adecuados para el uso industrial. Sin embargo, tienen ciertas características en común.

1.4.9 Sistemas operativos de tarjetas inteligentes

Los sistemas operativos más pequeños operan en las tarjetas inteligentes, que son dispositivos del tamaño de una tarjeta de crédito que contienen un chip de CPU. Tienen varias severas restricciones de poder de procesamiento y memoria. Algunas se energizan mediante contactos en el lector en el que se insertan, pero las tarjetas inteligentes sin contactos se energizan mediante inducción, lo cual limita en forma considerable las cosas que pueden hacer. Algunos sistemas de este tipo pueden realizar una sola función, como pagos electrónicos; otros pueden llevar a cabo varias funciones en la misma tarjeta inteligente. A menudo éstos son sistemas propietarios.

Algunas tarjetas inteligentes funcionan con Java. Lo que esto significa es que la ROM en la tarjeta inteligente contiene un intérprete para la Máquina virtual de Java (JVM). Los applets de Java (pequeños programas) se descargan en la tarjeta y son interpretados por el intérprete de la JVM. Algunas de estas tarjetas pueden manejar varias applets de Java al mismo tiempo, lo cual conlleva a la multiprogramación y a la necesidad de planificarlos. La administración de los recursos y su protección también se convierten en un problema cuando hay dos o más applets presentes al mismo tiempo. El sistema operativo (que por lo general es en extremo primitivo) presente en la tarjeta es el encargado de manejar estas cuestiones.

1.5 CONCEPTOS DE LOS SISTEMAS OPERATIVOS

La mayoría de los sistemas operativos proporcionan ciertos conceptos básicos y abstracciones tales como procesos, espacios de direcciones y archivos, que son la base para comprender su funcionamiento. En las siguientes secciones analizaremos algunos de estos conceptos básicos en forma breve, como una introducción. Más adelante en el libro volveremos a analizar cada uno de ellos con mayor detalle. Para ilustrar estos conceptos, de vez en cuando utilizaremos ejemplos que por lo general se basan en UNIX. No obstante, por lo general existen también ejemplos similares en otros sistemas, además de que en el capítulo 11 estudiaremos Windows Vista con detalle.

1.5.1 Procesos

Un concepto clave en todos los sistemas operativos es el **proceso**. Un proceso es en esencia un programa en ejecución. Cada proceso tiene asociado un **espacio de direcciones**, una lista de ubicaciones de memoria que va desde algún mínimo (generalmente 0) hasta cierto valor máximo, donde el proceso puede leer y escribir información. El espacio de direcciones contiene el programa ejecutable, los datos del programa y su pila. También hay asociado a cada proceso un conjunto de recursos, que comúnmente incluye registros (el contador de programa y el apuntador de pila, entre ellos), una lista de archivos abiertos, alarmas pendientes, listas de procesos relacionados y toda la demás información necesaria para ejecutar el programa. En esencia, un proceso es un recipiente que guarda toda la información necesaria para ejecutar un programa.

En el capítulo 2 volveremos a analizar el concepto de proceso con más detalle, pero por ahora la manera más fácil de que el lector se dé una buena idea de lo que es un proceso es pensar en un sistema de multiprogramación. El usuario puede haber iniciado un programa de edición de video para convertir un video de una hora a un formato específico (algo que puede tardar horas) y después irse a navegar en la Web. Mientras tanto, un proceso en segundo plano que despierta en forma periódica para comprobar los mensajes entrantes puede haber empezado a ejecutarse. Así tenemos (cuando menos) tres procesos activos: el editor de video, el navegador Web y el lector de correo electrónico. Cada cierto tiempo, el sistema operativo decide detener la ejecución de un proceso y empezar a ejecutar otro; por ejemplo, debido a que el primero ha utilizado más tiempo del que le correspondía de la CPU en el último segundo.

Cuando un proceso se suspende en forma temporal como en el ejemplo anterior, debe reiniciarse después exactamente en el mismo estado que tenía cuando se detuvo. Esto significa que toda la información acerca del proceso debe guardarse en forma explícita en alguna parte durante la suspensión. Por ejemplo, el proceso puede tener varios archivos abiertos para leerlos al mismo tiempo. Con cada uno de estos archivos hay un apuntador asociado que proporciona la posición actual (es decir, el número del byte o registro que se va a leer a continuación). Cuando un proceso se suspende en forma temporal, todos estos apuntadores deben guardarse de manera que una llamada a `read` que se ejecute después de reiniciar el proceso lea los datos apropiados. En muchos sistemas operativos, toda la información acerca de cada proceso (además del contenido de su propio espacio de direcciones) se almacena en una tabla del sistema operativo, conocida como la **tabla de procesos**, la cual es un arreglo (o lista enlazada) de estructuras, una para cada proceso que se encuentre actualmente en existencia.

Así, un proceso (suspendido) consiste en su espacio de direcciones, que se conoce comúnmente como **imagen de núcleo** (en honor de las memorias de núcleo magnético utilizadas antaño) y su entrada en la tabla de procesos, que guarda el contenido de sus registros y muchos otros elementos necesarios para reiniciar el proceso más adelante.

Las llamadas al sistema de administración de procesos clave son las que se encargan de la creación y la terminación de los procesos. Considere un ejemplo común. Un proceso llamado **intérprete de comandos** o **shell** lee comandos de una terminal. El usuario acaba de escribir un comando, solicitando la compilación de un programa. El shell debe entonces crear un proceso para ejecutar el compilador. Cuando ese proceso ha terminado la compilación, ejecuta una llamada al sistema para terminarse a sí mismo.

Si un proceso puede crear uno o más procesos aparte (conocidos como **procesos hijos**) y estos procesos a su vez pueden crear procesos hijos, llegamos rápidamente la estructura de árbol de procesos de la figura 1-13. Los procesos relacionados que cooperan para realizar un cierto trabajo a menudo necesitan comunicarse entre sí y sincronizar sus actividades. A esta comunicación se le conoce como **comunicación entre procesos**, que veremos con detalle en el capítulo 2.

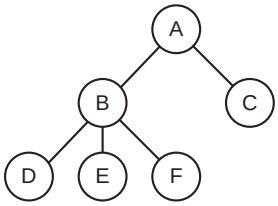


Figura 1-13. Un árbol de proceso. El proceso *A* creó dos proceso hijos, *B* y *C*. El proceso *B* creó tres procesos hijos, *D*, *E* y *F*.

Hay otras llamadas al sistema de procesos disponibles para solicitar más memoria (o liberar la memoria sin utilizar), esperar a que termine un proceso hijo y superponer su programa con uno distinto.

En algunas ocasiones se tiene la necesidad de transmitir información a un proceso en ejecución que no está esperando esta información. Por ejemplo, un proceso que se comunica con otro, en una computadora distinta, envía los mensajes al proceso remoto a través de una red de computadoras. Para protegerse contra la posibilidad de que se pierda un mensaje o su contestación, el emisor puede solicitar que su propio sistema operativo le notifique después de cierto número de segundos para que pueda retransmitir el mensaje, si no se ha recibido aún la señal de aceptación. Después de asignar este temporizador, el programa puede continuar realizando otro trabajo.

Cuando ha transcurrido el número especificado de segundos, el sistema operativo envía una **señal de alarma** al proceso. La señal provoca que el proceso suspenda en forma temporal lo que esté haciendo, almacene sus registros en la pila y empiece a ejecutar un procedimiento manejador de señales especial, por ejemplo, para retransmitir un mensaje que se considera perdido. Cuando termina el manejador de señales, el proceso en ejecución se reinicia en el estado en el que se encontraba justo antes de la señal. Las señales son la analogía en software de las interrupciones de hardware y se pueden generar mediante una variedad de causas además de la expiración de los temporizadores. Muchas traps detectadas por el hardware, como la ejecución de una instrucción ilegal o el uso de una dirección inválida, también se convierten en señales que se envían al proceso culpable.

Cada persona autorizada para utilizar un sistema recibe una **UID** (*User Identification*, Identificación de usuario) que el administrador del sistema le asigna. Cada proceso iniciado tiene el UID de la persona que lo inició. Un proceso hijo tiene el mismo UID que su padre. Los usuarios pueden ser miembros de grupos, cada uno de los cuales tiene una **GID** (*Group Identification*, Identificación de grupo).

Una UID conocida como **superusuario** (*superuser* en UNIX) tiene poder especial y puede violar muchas de las reglas de protección. En instalaciones extensas, sólo el administrador del sistema conoce la contraseña requerida para convertirse en superusuario, pero muchos de los usuarios ordi-

narios (en especial los estudiantes) dedican un esfuerzo considerable para tratar de encontrar fallas en el sistema que les permitan convertirse en superusuario sin la contraseña.

En el capítulo 2 estudiaremos los procesos, la comunicación entre procesos y las cuestiones relacionadas.

1.5.2 Espacios de direcciones

Cada computadora tiene cierta memoria principal que utiliza para mantener los programas en ejecución. En un sistema operativo muy simple sólo hay un programa a la vez en la memoria. Para ejecutar un segundo programa se tiene que quitar el primero y colocar el segundo en la memoria.

Los sistemas operativos más sofisticados permiten colocar varios programas en memoria al mismo tiempo. Para evitar que interfieran unos con otros (y con el sistema operativo), se necesita cierto mecanismo de protección. Aunque este mecanismo tiene que estar en el hardware, es controlado por el sistema operativo.

El anterior punto de vista se relaciona con la administración y protección de la memoria principal de la computadora. Aunque diferente, dado que la administración del espacio de direcciones de los procesos está relacionada con la memoria, es una actividad de igual importancia. Por lo general, cada proceso tiene cierto conjunto de direcciones que puede utilizar, que generalmente van desde 0 hasta cierto valor máximo. En el caso más simple, la máxima cantidad de espacio de direcciones que tiene un proceso es menor que la memoria principal. De esta forma, un proceso puede llenar su espacio de direcciones y aún así habrá suficiente espacio en la memoria principal para contener todo lo necesario.

Sin embargo, en muchas computadoras las direcciones son de 32 o 64 bits, con lo cual se obtiene un espacio de direcciones de 2^{32} o 2^{64} bytes, respectivamente. ¿Qué ocurre si un proceso tiene más espacio de direcciones que la memoria principal de la computadora, y desea usarlo todo? En las primeras computadoras, dicho proceso simplemente no podía hacer esto. Hoy en día existe una técnica llamada memoria virtual, como se mencionó antes, en la cual el sistema operativo mantiene una parte del espacio de direcciones en memoria principal y otra parte en el disco, moviendo pedazos de un lugar a otro según sea necesario. En esencia, el sistema operativo crea la abstracción de un espacio de direcciones como el conjunto de direcciones al que puede hacer referencia un proceso. El espacio de direcciones se desacopla de la memoria física de la máquina, pudiendo ser mayor o menor que la memoria física. La administración de los espacios de direcciones y la memoria física forman una parte importante de lo que hace un sistema operativo, por lo cual el capítulo 3 se dedica a este tema.

1.5.3 Archivos

Otro concepto clave de casi todos los sistemas operativos es el sistema de archivos. Como se dijo antes, una de las funciones principales del sistema operativo es ocultar las peculiaridades de los discos y demás dispositivos de E/S, presentando al programador un modelo abstracto limpio y agradable de archivos independientes del dispositivo. Sin duda se requieren las llamadas al sistema para crear los archivos, eliminarlos, leer y escribir en ellos. Antes de poder leer un archivo, debe locali-

zarse en el disco para abrirse y una vez que se ha leído información del archivo debe cerrarse, por lo que se proporcionan llamadas para hacer estas cosas.

Para proveer un lugar en donde se puedan mantener los archivos, la mayoría de los sistemas operativos tienen el concepto de un **directorio** como una manera de agrupar archivos. Por ejemplo, un estudiante podría tener un directorio para cada curso que esté tomando (para los programas necesarios para ese curso), otro directorio para su correo electrónico y otro más para su página de inicio en World Wide Web. Así, se necesitan llamadas al sistema para crear y eliminar directorios. También se proporcionan llamadas para poner un archivo existente en un directorio y para eliminar un archivo de un directorio. Las entradas de directorio pueden ser archivos u otros directorios. Este modelo también da surgimiento a una jerarquía (el sistema de archivos) como se muestra en la figura 1-14.

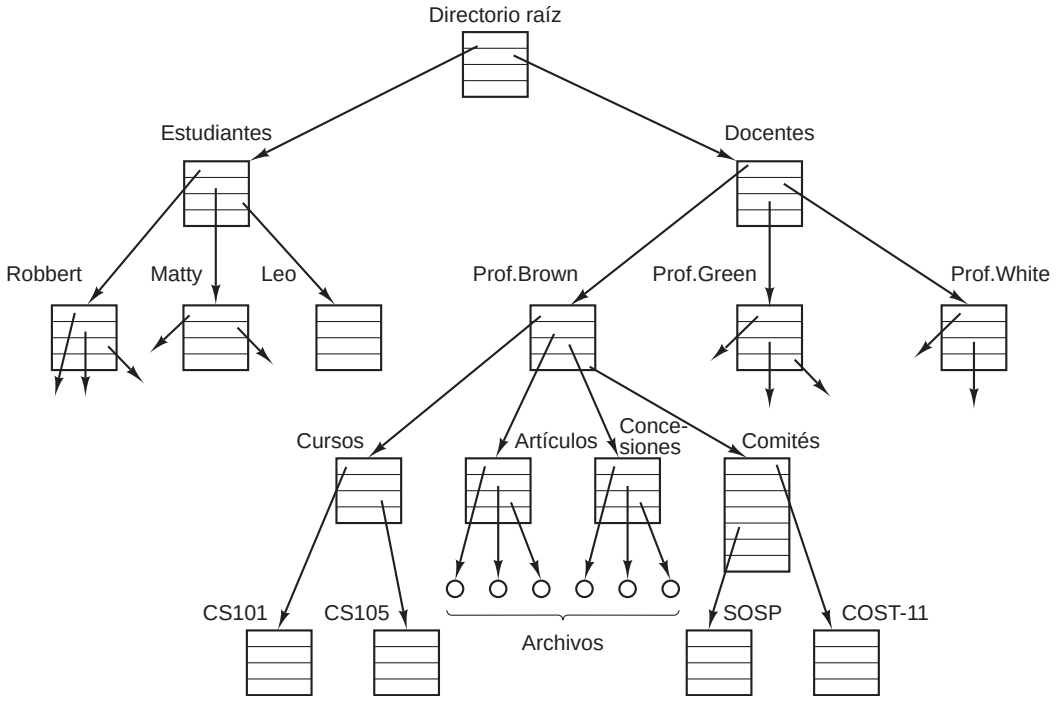


Figura 1-14. Un sistema de archivos para un departamento universitario.

Las jerarquías de procesos y de archivos están organizadas en forma de árboles, pero la similitud se detiene ahí. Por lo general, las jerarquías de procesos no son muy profundas (más de tres niveles es algo inusual), mientras que las jerarquías de archivos son comúnmente de cuatro, cinco o incluso más niveles de profundidad. Es común que las jerarquías de procesos tengan un tiempo de vida corto, por lo general de minutos a lo más, mientras que la jerarquía de directorios puede existir por años. La propiedad y la protección también difieren para los procesos y los archivos. Por lo común, sólo un proceso padre puede controlar o incluso acceder a un proceso hijo, pero casi siem-

pre existen mecanismos para permitir que los archivos y directorios sean leídos por un grupo aparte del propietario.

Para especificar cada archivo dentro de la jerarquía de directorio, se proporciona su **nombre de ruta** de la parte superior de la jerarquía de directorios, el **directorio raíz**. Dichos nombres de ruta absolutos consisten de la lista de directorios que deben recorrerse desde el directorio raíz para llegar al archivo, y se utilizan barras diagonales para separar los componentes. En la figura 1-14, la ruta para el archivo *CS101* es */Docentes/Prof.Brown/Cursos/CS101*. La primera barra diagonal indica que la ruta es absoluta, es decir, que empieza en el directorio raíz. Como una observación adicional, en MS-DOS y Windows se utiliza el carácter de barra diagonal inversa (\) como separador en vez del carácter de barra diagonal (/), por lo que la ruta del archivo antes mostrado podría escribirse como *\Docentes\Prof.Brown\Cursos\CS101*. A lo largo de este libro utilizaremos generalmente la convención de UNIX para las rutas.

En cada instante, cada proceso tiene un **directorio de trabajo** actual, en el que se buscan los nombres de ruta que no empiecen con una barra diagonal. Como ejemplo, en la figura 1-14 si */Docentes/Prof.Brown* fuera el directorio de trabajo, entonces el uso del nombre de ruta *Cursos/CS101* produciría el mismo archivo que el nombre de ruta absoluto antes proporcionado. Los procesos pueden modificar su directorio de trabajo mediante una llamada al sistema que especifique el nuevo directorio de trabajo.

Antes de poder leer o escribir en un archivo se debe abrir y en ese momento se comprueban los permisos. Si está permitido el acceso, el sistema devuelve un pequeño entero conocido como **descriptor de archivo** para usarlo en las siguientes operaciones. Si el acceso está prohibido, se devuelve un código de error.

Otro concepto importante en UNIX es el sistema de archivos montado. Casi todas las computadoras personales tienen una o más unidades ópticas en las que se pueden insertar los CD-ROMs y los DVDs. Casi siempre tienen puertos USB, a los que se pueden conectar memorias USB (en realidad son unidades de estado sólido), y algunas computadoras tienen discos flexibles o discos duros externos. Para ofrecer una manera elegante de lidiar con estos medios removibles, UNIX permite adjuntar el sistema de archivos en un CD-ROM o DVD al árbol principal. Considere la situación de la figura 1-15(a). Antes de la llamada `mount` (montar), el **sistema de archivos raíz** en el disco duro y un segundo sistema de archivos en un CD-ROM están separados y no tienen relación alguna.

Sin embargo, el sistema de archivo en el CD-ROM no se puede utilizar, debido a que no hay forma de especificar los nombres de las rutas en él. UNIX no permite colocar prefijos a los nombres de rutas basados en un nombre de unidad o un número; ese sería precisamente el tipo de dependencia de dispositivos que los sistemas operativos deben eliminar. En vez de ello, la llamada al sistema `mount` permite adjuntar el sistema de archivos en CD-ROM al sistema de archivos raíz en donde el programa desea que esté. En la figura 1-15(b) el sistema de archivos en el CD-ROM se ha montado en el directorio *b*, con lo cual se permite el acceso a los archivos */b/x* y */b/y*. Si el directorio *b* tuviera archivos, éstos no estarían accesibles mientras el CD-ROM estuviera montado, debido a que */b* haría referencia al directorio raíz del CD-ROM (el hecho de no poder acceder a estos archivos no es tan grave como parece: los sistemas de archivos casi siempre se montan en directorios vacíos). Si un sistema contiene varios discos duros, todos se pueden montar en un solo árbol también.

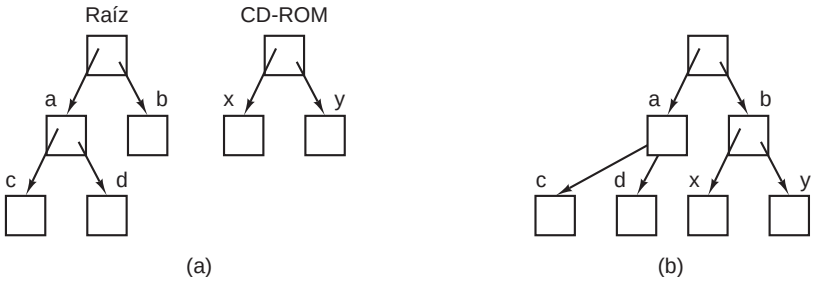


Figura 1.15. (a) Antes de montarse, los archivos en el CD-ROM no están accesibles. (b) Después de montarse, forman parte de la jerarquía de archivos.

Otro concepto importante en UNIX es el **archivo especial**. Los archivos especiales se proporcionan para poder hacer que los dispositivos de E/S se vean como archivos. De esta forma se puede leer y escribir en ellos utilizando las mismas llamadas al sistema que se utilizan para leer y escribir en archivos. Existen dos tipos de archivos especiales: **archivos especiales de bloque** y **archivos especiales de carácter**. Los archivos especiales de bloque se utilizan para modelar dispositivos que consisten en una colección de bloques direccionables al azar, tales como los discos. Al abrir un archivo especial de bloque y leer, por decir, el bloque 4, un programa puede acceder de manera directa al cuarto bloque en el dispositivo sin importar la estructura del sistema de archivos que contenga. De manera similar, los archivos especiales de carácter se utilizan para modelar impresoras, módems y otros dispositivos que aceptan o producen como salida un flujo de caracteres. Por convención, los archivos especiales se mantienen en el directorio */dev*. Por ejemplo, */dev/lp* podría ser la impresora (a la que alguna vez se le llamó impresora de línea).

La última característica que veremos en esta descripción general está relacionada con los procesos y los archivos: los canales. Un **canal (**pipe**)** es un tipo de pseudoarchivo que puede utilizarse para conectar dos procesos, como se muestra en la figura 1-16. Si los procesos *A* y *B* desean comunicarse mediante el uso de un canal, deben establecerlo por adelantado. Cuando el proceso *A* desea enviar datos al proceso *B*, escribe en el canal como si fuera un archivo de salida. De hecho, la implementación de un canal es muy parecida a la de un archivo. El proceso *B* puede leer los datos a través del canal, como si fuera un archivo de entrada. Por ende, la comunicación entre procesos en UNIX tiene una apariencia muy similar a las operaciones comunes de lectura y escritura en los archivos. Y por si fuera poco, la única manera en que un proceso puede descubrir que el archivo de salida en el que está escribiendo no es en realidad un archivo sino un canal, es mediante una llamada al sistema especial. Los sistemas de archivos son muy importantes. En los capítulos 4, 10 y 11 hablaremos mucho más sobre ellos.

1.5.4 Entrada/salida

Todas las computadoras tienen dispositivos físicos para adquirir entrada y producir salida. Después de todo, ¿qué tendría de bueno una computadora si los usuarios no pudieran indicarle qué debe hacer y no pudieran obtener los resultados una vez que realizara el trabajo solicitado? Existen muchos

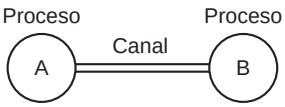


Figura 1-16. Dos procesos conectados mediante un canal.

tipos de dispositivos de entrada y de salida, incluyendo teclados, monitores, impresoras, etcétera. Es responsabilidad del sistema operativo administrar estos dispositivos.

En consecuencia, cada sistema operativo tiene un subsistema de E/S para administrar sus dispositivos de E/S. Parte del software de E/S es independiente de los dispositivos, es decir, se aplica a muchos o a todos los dispositivos de E/S por igual. Otras partes del software, como los drivers de dispositivos, son específicas para ciertos dispositivos de E/S. En el capítulo 5 analizaremos el software de E/S.

1.5.5 Protección

Las computadoras contienen grandes cantidades de información que los usuarios comúnmente desean proteger y mantener de manera confidencial. Esta información puede incluir mensajes de correo electrónico, planes de negocios, declaraciones fiscales y mucho más. Es responsabilidad del sistema operativo administrar la seguridad del sistema de manera que los archivos, por ejemplo, sólo sean accesibles para los usuarios autorizados.

Como un ejemplo simple, sólo para tener una idea de cómo puede funcionar la seguridad, considere el sistema operativo UNIX. Los archivos en UNIX están protegidos debido a que cada uno recibe un código de protección binario de 9 bits. El código de protección consiste en tres campos de 3 bits, uno para el propietario, uno para los demás miembros del grupo del propietario (el administrador del sistema divide a los usuarios en grupos) y uno para todos los demás. Cada campo tiene un bit para el acceso de lectura, un bit para el acceso de escritura y un bit para el acceso de ejecución. Estos 3 bits se conocen como los **bits rwx**. Por ejemplo, el código de protección *rwxr-x--x* indica que el propietario puede leer (**r**), escribir (**w**) o ejecutar (**x**) el archivo, otros miembros del grupo pueden leer o ejecutar (pero no escribir) el archivo y todos los demás pueden ejecutarlo (pero no leer ni escribir). Para un directorio, *x* indica el permiso de búsqueda. Un guión corto indica que no se tiene el permiso correspondiente.

Además de la protección de archivos, existen muchas otras cuestiones de seguridad. Una de ellas es proteger el sistema de los intrusos no deseados, tanto humanos como no humanos (por ejemplo, virus). En el capítulo 9 analizaremos varias cuestiones de seguridad.

1.5.6 El shell

El sistema operativo es el código que lleva a cabo las llamadas al sistema. Los editores, compiladores, ensambladores, enlazadores e intérpretes de comandos en definitiva no forman parte del sistema operativo, aun cuando son importantes y útiles. Con el riesgo de confundir un poco las cosas, en esta sección describiremos brevemente el intérprete de comandos de UNIX, conocido como

shell. Aunque no forma parte del sistema operativo, utiliza con frecuencia muchas características del mismo y, por ende, sirve como un buen ejemplo de la forma en que se pueden utilizar las llamadas al sistema. También es la interfaz principal entre un usuario sentado en su terminal y el sistema operativo, a menos que el usuario esté usando una interfaz gráfica de usuario. Existen muchos shells, incluyendo *sh*, *csh*, *ksh* y *bash*. Todos ellos soportan la funcionalidad antes descrita, que se deriva del shell original (*sh*).

Cuando cualquier usuario inicia sesión, se inicia un shell. El shell tiene la terminal como entrada estándar y salida estándar. Empieza por escribir el **indicador de comandos (prompt)**, un carácter tal como un signo de dólar, que indica al usuario que el shell está esperando aceptar un comando. Por ejemplo, si el usuario escribe

```
date
```

el shell crea un proceso hijo y ejecuta el programa *date* como el hijo. Mientras se ejecuta el proceso hijo, el shell espera a que termine. Cuando el hijo termina, el shell escribe de nuevo el indicador y trata de leer la siguiente línea de entrada.

El usuario puede especificar que la salida estándar sea redirigida a un archivo, por ejemplo:

```
date >archivo
```

De manera similar, la entrada estándar se puede redirigir, como en:

```
sort <archivo1 >archivo2
```

con lo cual se invoca el programa sort con la entrada que se recibe del *archivo1* y la salida se envía al *archivo2*.

La salida de un programa se puede utilizar como entrada para otro, si se conectan mediante un canal. Así:

```
cat archivo1 archivo2 archivo3 | sort >/dev/lp
```

invoca al programa *cat* para con*cat*enar tres archivos y enviar la salida a *sort* para ordenar todas las líneas en orden alfabético. La salida de *sort* se redirige al archivo */dev/lp*, que por lo general es la impresora.

Si un usuario coloca el signo & después de un comando, el shell no espera a que se complete. En vez de ello, proporciona un indicador de comandos de inmediato. En consecuencia:

```
cat archivo1  archivo2  archivo3  |  sort >/dev/lp &
```

inicia el comando sort como un trabajo en segundo plano y permite al usuario continuar su trabajo de manera normal mientras el ordenamiento se lleva a cabo. El shell tiene otras características interesantes, que no describiremos aquí por falta de espacio. La mayoría de los libros en UNIX describen el shell hasta cierto grado (por ejemplo, Kernighan y Pike, 1984; Kochan y Wood, 1990; Medinets, 1999; Newham y Rosenblatt, 1998; y Robbins, 1999).

Actualmente, muchas computadoras personales utilizan una GUI. De hecho, la GUI es sólo un programa que se ejecuta encima del sistema operativo, como un shell. En los sistemas Linux, este hecho se hace obvio debido a que el usuario tiene una selección de (por lo menos) dos GUIs: Gnome y KDE o ninguna (se utiliza una ventana de terminal en X11). En Windows también es posible

reemplazar el escritorio estándar de la GUI (*Windows Explorer*) con un programa distinto, para lo cual se modifican ciertos valores en el registro, aunque pocas personas hacen esto.

1.5.7 La ontogenia recapitula la filogenia

Después de que se publicó el libro de Charles Darwin titulado *El origen de las especies*, el zoólogo alemán Ernst Haeckel declaró que "la ontogenia recapitula la filogenia". Lo que quiso decir fue que el desarrollo de un embrión (ontogenia) repite (es decir, recapitula) la evolución de las especies (filogenia). En otras palabras, después de la fertilización un óvulo humano pasa a través de las etapas de ser un pez, un cerdo y así en lo sucesivo, hasta convertirse en un bebé humano. Los biólogos modernos consideran esto como una simplificación burda, pero aún así tiene cierto grado de verdad.

Algo análogo ha ocurrido en la industria de las computadoras. Cada nueva especie (mainframe, minicomputadora, computadora personal, computadora de bolsillo, computadora de sistema integrado, tarjeta inteligente, etc.) parece pasar a través del desarrollo que hicieron sus ancestros, tanto en hardware como en software. A menudo olvidamos que la mayor parte de lo que ocurre en el negocio de las computadoras y en muchos otros campos está controlado por la tecnología. La razón por la que los antiguos romanos no tenían autos no es que les gustara caminar mucho; se debió a que no sabían construirlos. Las computadoras personales existen *no* debido a que millones de personas tienen un deseo reprimido durante siglos de poseer una computadora, sino a que ahora es posible fabricarlas a un costo económico. A menudo olvidamos qué tanto afecta la tecnología a nuestra visión de los sistemas y vale la pena reflexionar sobre este punto de vez en cuando.

En especial, con frecuencia ocurre que un cambio en la tecnología hace que una idea se vuelva obsoleta y desaparece con rapidez. Sin embargo, otro cambio en la tecnología podría revivirla de nuevo. Esto es en especial verdadero cuando el cambio tiene que ver con el rendimiento relativo de distintas partes del sistema. Por ejemplo, cuando las CPUs se volvieron mucho más rápidas que las memorias, las cachés tomaron importancia para agilizar la memoria "lenta". Si algún día la nueva tecnología de memoria hace que las memorias sean mucho más rápidas que las CPUs, las cachés desaparecerán. Y si una nueva tecnología de CPUs las hace más rápidas que las memorias de nuevo, las cachés volverán a aparecer. En biología, la extinción es para siempre, pero en la ciencia computacional, algunas veces sólo es durante unos cuantos años.

Como consecuencia de esta transitoriedad, en este libro analizaremos de vez en cuando conceptos "obsoletos", es decir, ideas que no son óptimas con la tecnología actual. Sin embargo, los cambios en la tecnología pueden traer de nuevo algunos de los denominados "conceptos obsoletos". Por esta razón, es importante comprender por qué un concepto es obsoleto y qué cambios en el entorno pueden hacer que vuelva de nuevo.

Para aclarar aún más este punto consideremos un ejemplo simple. Las primeras computadoras tenían conjuntos de instrucciones cableados de forma fija. Las instrucciones se ejecutaban directamente por el hardware y no se podían modificar. Después llegó la microprogramación (primero se introdujo en gran escala con la IBM 360), en la que un intérprete subyacente ejecutaba las "instrucciones de hardware" en el software. La ejecución de instrucciones fijas se volvió obsoleta. Pero esto no era lo bastante flexible. Después se inventaron las computadoras RISC y la microprogramación

(es decir, la ejecución interpretada) se volvió obsoleta, debido a que la ejecución directa era más veloz. Ahora estamos viendo el resurgimiento de la interpretación en forma de applets de Java que se envían a través de Internet y se interpretan al momento de su llegada. La velocidad de ejecución no siempre es crucial, debido a que los retrasos en la red son tan grandes que tienden a dominar. Por ende, el péndulo ha oscilado varias veces entre la ejecución directa y la interpretación y puede volver a oscilar de nuevo en el futuro.

Memorias extensas

Ahora vamos a examinar algunos desarrollos históricos en el hardware y la forma en que han afectado al software repetidas veces. Las primeras mainframes tenían memoria limitada. Una IBM 7090 o 7094 completamente equipada, que fungió como rey de la montaña desde finales de 1959 hasta 1964, tenía cerca de 128 KB de memoria. En su mayor parte se programaba en lenguaje ensamblador y su sistema operativo estaba escrito en lenguaje ensamblador también para ahorrar la valiosa memoria.

A medida que pasaba el tiempo, los compiladores para lenguajes como FORTRAN y COBOL se hicieron lo bastante buenos como para que el lenguaje ensamblador se hiciera obsoleto. Pero cuando se liberó al mercado la primera minicomputadora comercial (PDP-1), sólo tenía 4096 palabras de 18 bits de memoria y el lenguaje ensamblador tuvo un regreso sorpresivo. Con el tiempo, las microcomputadoras adquirieron más memoria y los lenguajes de alto nivel prevalecieron.

Cuando las microcomputadoras llegaron a principios de 1980, las primeras tenían memorias de 4 KB y la programación en lenguaje ensamblador surgió de entre los muertos. A menudo, las computadoras embebidas utilizaban los mismos chips de CPU que las microcomputadoras (8080, Z80 y posteriormente 8086) y también se programaban en ensamblador al principio. Ahora sus descendientes, las computadoras personales, tienen mucha memoria y se programan en C, C++ y Java, además de otros lenguajes de alto nivel. Las tarjetas inteligentes están pasando por un desarrollo similar, aunque más allá de un cierto tamaño, a menudo tienen un intérprete de Java y ejecutan los programas de Java en forma interpretativa, en vez de que se compile Java al lenguaje máquina de la tarjeta inteligente.

Hardware de protección

Las primeras mainframes (como la IBM 7090/7094) no tenían hardware de protección, por lo que sólo ejecutaban un programa a la vez. Un programa con muchos errores podía acabar con el sistema operativo y hacer que la máquina fallara con facilidad. Con la introducción de la IBM 360, se hizo disponible una forma primitiva de protección de hardware y estas máquinas podían de esta forma contener varios programas en memoria al mismo tiempo, y dejarlos que tomaran turnos para ejecutarse (multiprogramación). La monoprogramación se declaró obsoleta.

Por lo menos hasta que apareció la primera minicomputadora (sin hardware de protección) la multiprogramación no fue posible. Aunque la PDP-1 y la PDP-8 no tenían hardware de protección, en cierto momento se agregó a la PDP-11 dando entrada a la multiprogramación y con el tiempo a UNIX.

Cuando se construyeron las primeras microcomputadoras, utilizaban el chip de CPU 8080 de Intel, que no tenía protección de hardware, por lo que regresamos de vuelta a la monoprogramación. No fue sino hasta el Intel 80286 que se agregó hardware de protección y se hizo posible la multiprogramación. Hasta la fecha, muchos sistemas integrados no tienen hardware de protección y ejecutan un solo programa.

Ahora veamos los sistemas operativos. Al principio, las primeras mainframes no tenían hardware de protección ni soporte para la multiprogramación, por lo que ejecutaban sistemas operativos simples que se encargaban de un programa cargado en forma manual a la vez. Más adelante adquirieron el soporte de hardware y del sistema operativo para manejar varios programas a la vez, después capacidades completas de tiempo compartido.

Cuando aparecieron las minicomputadoras por primera vez, tampoco tenían hardware de protección y ejecutaban un programa cargado en forma manual a la vez, aun cuando la multiprogramación estaba bien establecida en el mundo de las mainframes para ese entonces. Gradualmente adquirieron hardware de protección y la habilidad de ejecutar dos o más programas a la vez. Las primeras microcomputadoras también fueron capaces de ejecutar sólo un programa a la vez, pero más adelante adquirieron la habilidad de la multiprogramación. Las computadoras de bolsillo y las tarjetas inteligentes se fueron por la misma ruta.

En todos los casos, el desarrollo de software se rigió en base a la tecnología. Por ejemplo, las primeras microcomputadoras tenían cerca de 4 KB de memoria y no tenían hardware de protección. Los lenguajes de alto nivel y la multiprogramación eran demasiado para que un sistema tan pequeño pudiera hacerse cargo. A medida que las microcomputadoras evolucionaron en las computadoras personales modernas, adquirieron el hardware necesario y después el software necesario para manejar características más avanzadas. Es probable que este desarrollo continúe por varios años más. Otros campos también pueden tener esta rueda de reencarnaciones, pero en la industria de las computadoras parece girar con más velocidad.

Discos

Las primeras mainframes estaban en su mayor parte basadas en cinta magnética. Leían un programa de la cinta, lo compilaban, lo ejecutaban y escribían los resultados de vuelta en otra cinta. No había discos ni un concepto sobre el sistema de archivos. Eso empezó a cambiar cuando IBM introdujo el primer disco duro: el RAMAC (*RAndoM Access*, Acceso aleatorio) en 1956. Ocupaba cerca de 4 metros cuadrados de espacio de piso y podía almacenar 5 millones de caracteres de 7 bits, lo suficiente como para una fotografía digital de mediana resolución. Pero con una renta anual de 35,000 dólares, ensamblar suficientes discos como para poder almacenar el equivalente de un rollo de película era en extremo costoso. Con el tiempo los precios disminuyeron y se desarrollaron los sistemas de archivos primitivos.

Uno de los nuevos desarrollos típicos de esa época fue la CDC 6600, introducida en 1964 y considerada durante años como la computadora más rápida del mundo. Los usuarios podían crear "archivos permanentes" al darles un nombre y esperar que ningún otro usuario hubiera decidido también que, por decir, "datos" fuera un nombre adecuado para un archivo. Éste era un directorio de un solo nivel. Con el tiempo las mainframes desarrollaron sistemas de archivos jerárquicos complejos, lo cual probablemente culminó con el sistema de archivos MULTICS.

Cuando las minicomputadoras empezaron a usarse, con el tiempo también tuvieron discos duros. El disco estándar en la PDP-11 cuando se introdujo en 1970 era el disco RK05, con una capacidad de 2.5 MB, aproximadamente la mitad del RAMAC de IBM, pero sólo tenía cerca de 40 cm de diámetro y 5 cm de altura. Pero también tenía al principio un directorio de un solo nivel. Cuando llegaron las microcomputadoras, CP/M fue al principio el sistema operativo dominante y también soportaba un solo directorio en el disco (flexible).

Memoria virtual

La memoria virtual (que se describe en el capítulo 3) proporciona la habilidad de ejecutar programas más extensos que la memoria física de la computadora, llevando y trayendo pedazos entre la RAM y el disco. Pasó por un desarrollo similar, ya que apareció primero en las mainframes, después avanzó a las minis y a las micros. La memoria virtual también permitió la capacidad de ligar dinámicamente un programa a una biblioteca en tiempo de ejecución, en vez de compilarlo. MULTICS fue el primer sistema operativo en tener esta capacidad. Con el tiempo, la idea se propagó descendiendo por toda la línea y ahora se utiliza ampliamente en la mayoría de los sistemas UNIX y Windows.

En todos estos desarrollos vemos ideas que se inventaron en un contexto y más adelante se descartaron cuando cambió el contexto (la programación en lenguaje ensamblador, la monoprogramación, los directorios de un solo nivel, etc.) sólo para reaparecer en un contexto distinto, a menudo una década más tarde. Por esta razón, en este libro algunas veces vemos ideas y algoritmos que pueden parecer atrasados en comparación con las PC de hoy en día con capacidades de gigabytes, pero que pronto pueden volver en las computadoras incrustadas y las tarjetas inteligentes.

1.6 LLAMADAS AL SISTEMA

Hemos visto que los sistemas operativos tienen dos funciones principales: proveer abstracciones a los programas de usuario y administrar los recursos de la computadora. En su mayor parte, la interacción entre los programas de usuario y el sistema operativo se relaciona con la primera función: por ejemplo, crear, escribir, leer y eliminar archivos. La parte de la administración de los recursos es en gran parte transparente para los usuarios y se realiza de manera automática. Por ende, la interfaz entre los programas de usuario y el sistema operativo trata principalmente acerca de cómo lidiar con las abstracciones. Para comprender realmente qué hacen los sistemas operativos, debemos examinar esta interfaz con detalle. Las llamadas al sistema disponibles en la interfaz varían de un sistema operativo a otro (aunque los conceptos subyacentes tienden a ser similares).

Por lo tanto, nos vemos obligados a elegir una opción entre 1) generalidades imprecisas (“los sistemas operativos tienen llamadas al sistema para leer archivos”) y 2) cierto sistema específico (“UNIX tiene una llamada al sistema conocida como `read` con tres parámetros: uno para especificar el archivo, uno para decir en dónde se van a colocar los datos y uno para indicar cuantos bytes se deben leer”).

Hemos optado por la última opción; implica más trabajo, pero proporciona una visión más detallada en cuanto a lo que realmente hacen los sistemas operativos. Aunque este análisis se refiere

en forma específica a POSIX (Estándar internacional 9945-1) y por ende también a UNIX, System V, BSD, Linux, MINIX 3, etc., la mayoría de los demás sistemas operativos modernos tienen llamadas al sistema que realizan las mismas funciones, incluso si difieren los detalles. Como la verdadera mecánica relacionada con la acción de emitir una llamada al sistema es altamente dependiente de la máquina y a menudo debe expresarse en código ensamblador, se proporciona una biblioteca de procedimientos para hacer que sea posible realizar llamadas al sistema desde programas en C y por lo general desde otros lenguajes también.

Es conveniente tener en cuenta lo siguiente. Cualquier computadora con una sola CPU puede ejecutar sólo una instrucción a la vez. Si un proceso está ejecutando un programa de usuario en modo usuario y necesita un servicio del sistema, como leer datos de un archivo, tiene que ejecutar una instrucción de trap para transferir el control al sistema operativo. Después, el sistema operativo averigua qué es lo que quiere el proceso llamador, para lo cual inspecciona los parámetros. Luego lleva a cabo la llamada al sistema y devuelve el control a la instrucción que va después de la llamada al sistema. En cierto sentido, realizar una llamada al sistema es como realizar un tipo especial de llamada a un procedimiento, sólo que las llamadas al sistema entran al kernel y las llamadas a procedimientos no.

Para hacer más entendible el mecanismo de llamadas al sistema, vamos a dar un vistazo rápido a la llamada al sistema `read`. Como dijimos antes, tiene tres parámetros: el primero especifica el archivo, el segundo apunta al búfer y el tercero proporciona el número de bytes a leer. Al igual que casi todas las llamadas al sistema, se invoca desde programas en C mediante una llamada a un procedimiento de la biblioteca con el mismo nombre que la llamada al sistema: *read*. Una llamada desde un programa en C podría tener la siguiente apariencia:

```
cuenta=read(fd, bufer, nbytes);
```

La llamada al sistema (y el procedimiento de biblioteca) devuelve el número de bytes que se leen en *cuenta*. Por lo general este valor es el mismo que *nbytes* pero puede ser más pequeño si, por ejemplo, se encuentra el fin de archivo al estar leyendo.

Si la llamada al sistema no se puede llevar a cabo, ya sea debido a un parámetro inválido o a un error del disco, *cuenta* se establece a -1 y el número de error se coloca en una variable global llamada *errno*. Los programas siempre deben comprobar los resultados de una llamada al sistema para ver si ocurrió un error.

Las llamadas al sistema se llevan a cabo en una serie de pasos. Para que este concepto quede más claro, vamos a examinar la llamada `read` antes descrita. En su preparación para llamar al procedimiento de biblioteca *read*, que es quien realmente hace la llamada al sistema `read`, el programa llamador primero mete los parámetros en la pila, como se muestra en los pasos 1 a 3 de la figura 1-17.

Los compiladores de C y C++ meten los parámetros en la pila en orden inverso por razones históricas (esto tiene que ver con hacer que el primer parámetro para *printf*, la cadena del formato, aparezca en la parte superior de la pila). Los parámetros primero y tercero se pasan por valor, pero el segundo parámetro se pasa por referencia, lo cual significa que se pasa la dirección del búfer (lo cual se indica mediante &), no el contenido del mismo. Después viene la llamada al procedimiento de biblioteca (paso 4). Esta instrucción es la instrucción de llamada a procedimiento normal utilizada para llamar a todos los procedimientos.

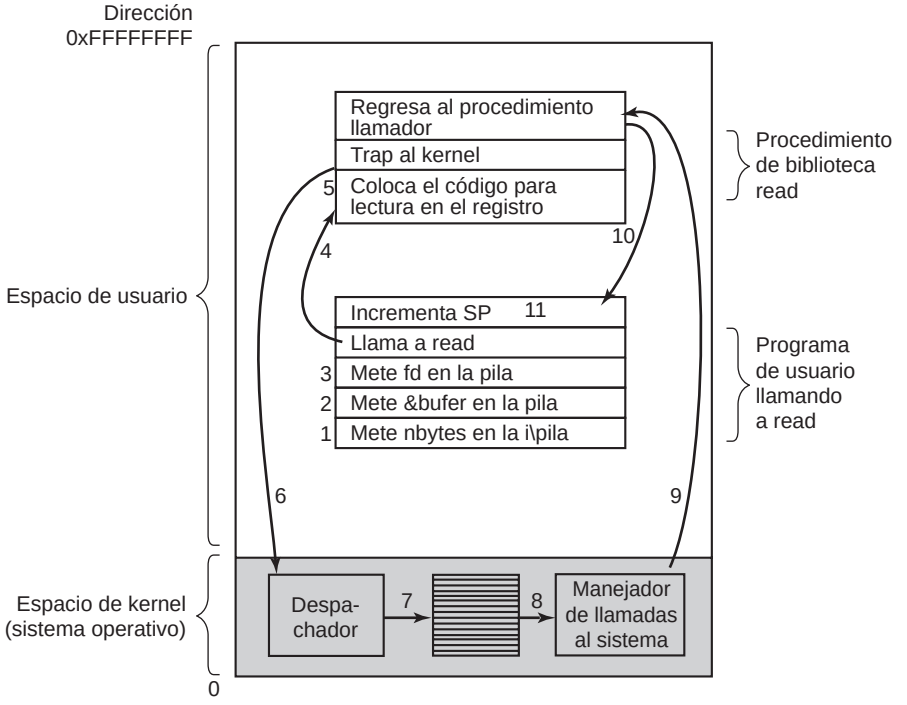


Figura 1-17. Los 11 pasos para realizar la llamada al sistema `read(fd, bufer, nbytes)`.

El procedimiento de biblioteca (probablemente escrito en lenguaje ensamblador) coloca por lo general el número de la llamada al sistema en un lugar en el que el sistema operativo lo espera, como en un registro (paso 5). Después ejecuta una instrucción `TRAP` para cambiar del modo usuario al modo kernel y empezar la ejecución en una dirección fija dentro del núcleo (paso 6). La instrucción `TRAP` en realidad es muy similar a la instrucción de llamada a procedimiento en el sentido en que la instrucción que le sigue se toma de una ubicación distante y la dirección de retorno se guarda en la pila para un uso posterior.

Sin embargo, la instrucción `TRAP` también difiere de la instrucción de llamada a un procedimiento en dos formas básicas. En primer lugar, como efecto secundario, cambia a modo kernel. La instrucción de llamada al procedimiento no cambia el modo. En segundo lugar, en vez de dar una dirección relativa o absoluta en donde se encuentra el procedimiento, la instrucción `TRAP` no puede saltar a una dirección arbitraria. Dependiendo de la arquitectura, salta a una ubicación fija, hay un campo de 8 bits en la instrucción que proporciona el índice a una tabla en memoria que contiene direcciones de salto, o su equivalente.

El código de kernel que empieza después de la instrucción `TRAP` examina el número de llamada al sistema y después la pasa al manejador correspondiente de llamadas al sistema, por lo general a través de una tabla de apuntadores a manejadores de llamadas al sistema, indexados en base al

número de llamada al sistema (paso 7). En ese momento se ejecuta el manejador de llamadas al sistema (paso 8). Una vez que el manejador ha terminado su trabajo, el control se puede regresar al procedimiento de biblioteca que está en espacio de usuario, en la instrucción que va después de la instrucción `TRAP` (paso 9). Luego este procedimiento regresa al programa de usuario en la forma usual en que regresan las llamadas a procedimientos (paso 10).

Para terminar el trabajo, el programa de usuario tiene que limpiar la pila, como lo hace después de cualquier llamada a un procedimiento (paso 11). Suponiendo que la pila crece hacia abajo, como es comúnmente el caso, el código compilado incrementa el apuntador de la pila lo suficiente como para eliminar los parámetros que se metieron antes de la llamada a *read*. Ahora el programa es libre de hacer lo que quiera a continuación.

En el paso 9 anterior, dijimos que "se puede regresar al procedimiento de biblioteca que está en espacio de usuario …" por una buena razón. La llamada al sistema puede bloquear al procedimiento llamador, evitando que continúe. Por ejemplo, si trata de leer del teclado y no se ha escrito nada aún, el procedimiento llamador tiene que ser bloqueado. En este caso, el sistema operativo buscará a su alrededor para ver si se puede ejecutar algún otro proceso a continuación. Más adelante, cuando esté disponible la entrada deseada, este proceso recibirá la atención del sistema y se llevarán a cabo los pasos 9 a 11.

En las siguientes secciones examinaremos algunas de las llamadas al sistema POSIX de uso más frecuente, o dicho en forma más específica, los procedimientos de biblioteca que realizan esas llamadas al sistema. POSIX tiene aproximadamente 100 llamadas a procedimientos. Algunas de las más importantes se listan en la figura 1-18, agrupadas por conveniencia en cuatro categorías. En el texto examinaremos brevemente cada llamada para ver cuál es su función.

En mayor grado, los servicios ofrecidos por estas llamadas determinan la mayor parte de la labor del sistema operativo, ya que la administración de recursos en las computadoras personales es una actividad mínima (por lo menos si se le compara con los equipos grandes que tienen muchos usuarios). Los servicios incluyen acciones tales como crear y terminar procesos, crear, eliminar, leer y escribir en archivos, administrar directorios y realizar operaciones de entrada y salida.

Al margen, vale la pena mencionar que la asignación de las llamadas a procedimientos POSIX a llamadas al sistema no es de uno a uno. El estándar POSIX especifica varios procedimientos que debe suministrar un sistema que se conforme a este estándar, pero no especifica si deben ser llamadas al sistema, llamadas a una biblioteca, o algo más. Si un procedimiento puede llevarse a cabo sin necesidad de invocar una llamada al sistema (es decir, sin atrapar en el kernel), por lo general se realizará en espacio de usuario por cuestión de rendimiento. Sin embargo, la mayoría de los procedimientos POSIX invocan llamadas al sistema, en donde por lo general un procedimiento se asigna directamente a una llamada al sistema. En unos cuantos casos, en especial en donde los procedimientos requeridos son sólo pequeñas variaciones de algún otro procedimiento, una llamada al sistema maneja más de una llamada a la biblioteca.

1.6.1 Llamadas al sistema para la administración de procesos

El primer grupo de llamadas en la figura 1-18 se encarga de la administración de los procesos. `fork` es un buen lugar para empezar este análisis. `fork` es la única manera de crear un nuevo proceso en POSIX. Crea un duplicado exacto del proceso original, incluyendo todos los descriptores de archivos,

Administración de procesos

Llamada	Descripción
pid = fork()	Crea un proceso hijo, idéntico al padre
pid = waitpid(pid, &statloc, opciones)	Espera a que un hijo termine
s = execve(nombre, argv, entornp)	Reemplaza la imagen del núcleo de un proceso
exit(estado)	Termina la ejecución de un proceso y devuelve el estado

Administración de archivos

Llamada	Descripción
fd = open(archivo, como, …)	Abre un archivo para lectura, escritura o ambas
s = close(fd)	Cierra un archivo abierto
n = read(fd, bufer, nbytes)	Lee datos de un archivo y los coloca en un búfer
n = write(fd, bufer, nbytes)	Escribe datos de un búfer a un archivo
posicion = lseek(fd, desplazamiento, dedonde)	Desplaza el apuntador del archivo
s = stat(nombre, &buf)	Obtiene la información de estado de un archivo

Administración del sistema de directorios y archivos

Llamada	Descripción
s = mkdir(nombre, modo)	Crea un nuevo directorio
s = rmdir(nombre)	Elimina un directorio vacío
s = link(nombre1, nombre2)	Crea una nueva entrada llamada nombre2, que apunta a nombre1
s = unlink(nombre)	Elimina una entrada de directorio
s = mount(especial, nombre, bandera)	Monta un sistema de archivos
s = umount(especial)	Desmonta un sistema de archivos

Llamadas varias

Llamada	Descripción
s = chdir(nombredir)	Cambia el directorio de trabajo
s = chmod(nombre, modo)	Cambia los bits de protección de un archivo
s = kill(pid, senial)	Envía una señal a un proceso
segundos = tiempo(&segundos)	Obtiene el tiempo transcurrido desde Ene 1, 1970

Figura 1-18. Algunas de las principales llamadas al sistema POSIX. El código de retorno *s* es −1 si ocurrió un error. Los códigos de retorno son: *pid* es un id de proceso, *fd* es un descriptor de archivo, *n* es una cuenta de bytes, *posicion* es un desplazamiento dentro del archivo y *segundos* es el tiempo transcurrido. Los parámetros se explican en el texto.

registros y todo lo demás. Después de `fork`, el proceso original y la copia (el padre y el hijo) se van por caminos separados. Todas las variables tienen valores idénticos al momento de la llamada a fork, pero como los datos del padre se copian para crear al hijo, los posteriores cambios en uno de ellos no afectarán al otro (el texto del programa, que no se puede modificar, se comparte entre el padre y el hijo). La llamada a `fork` devuelve un valor, que es cero en el hijo e igual al identificador del proceso (`PID`) hijo en el padre. Mediante el uso del PID devuelto, los dos procesos pueden ver cuál es el proceso padre y cuál es el proceso hijo.

En la mayoría de los casos, después de una llamada a `fork` el hijo tendrá que ejecutar código distinto al del padre. Considere el caso del shell: lee un comando de la terminal, llama a fork para crear un proceso hijo, espera a que el hijo ejecute el comando y después lee el siguiente comando cuando el hijo termina. Para esperar a que el hijo termine, el padre ejecuta una llamada al sistema `waitpid`, la cual sólo espera hasta que el hijo termine (cualquier hijo, si existe más de uno). `Waitpid` puede esperar a un hijo específico o a cualquier hijo anterior si establece el primer parámetro a −1. Cuando `waitpid` se completa, la dirección a la que apunta el segundo parámetro (*statloc*) se establece al estado de salida del hijo (terminación normal o anormal, con el valor de `exit`). También se proporcionan varias opciones, especificadas por el tercer parámetro.

Ahora considere la forma en que el shell utiliza a `fork`. Cuando se escribe un comando, el shell crea un nuevo proceso usando `fork`. Este proceso hijo debe ejecutar el comando de usuario. Para ello utiliza la llamada al sistema `execve`, la cual hace que toda su imagen de núcleo completa se sustituya por el archivo nombrado en su primer parámetro. (En realidad, la llamada al sistema en sí es `exec`, pero varios procedimientos de biblioteca lo llaman con distintos parámetros y nombres ligeramente diferentes. Aquí trataremos a estas llamadas como si fueran llamadas al sistema.) En la figura 1-19 se muestra un shell muy simplificado que ilustra el uso de `fork`, `waitpid` y `execve`.

```
#define TRUE 1

while (TRUE) {                                  /* se repite en forma indefinida */
        type_prompt();                          /* muestra el indicador de comando en la pantalla */
        read_command(command, parameters);      /* lee la entrada de la terminal */

        if (fork() !=0) {                       /* usa fork para el proceso hijo */
                /* Codigo del padre. */
                waitpid(-1, &status, 0);        /* espera a que el hijo termine */
        } else {
                /* Codigo del hijo. */
                execve(command, parameters, 0); /* ejecuta el comando */
        }
}
```

Figura 1-19. Una versión simplificada del shell. En este libro supondremos que *TRUE* se define como 1.

En el caso más general, `execve` tiene tres parámetros: el nombre del archivo que se va a ejecutar, un apuntador al arreglo de argumentos y un apuntador al arreglo del entorno. En breve describiremos estos parámetros. Se proporcionan varias rutinas de biblioteca incluyendo a *execl*, *execv*,

execle y *execve*, para permitir la omisión de los parámetros o para especificarlos de varias formas. En este libro utilizaremos el nombre `exec` para representar la llamada al sistema que se invoca mediante cada una de estas rutinas.

Consideremos el caso de un comando tal como

```
cp archivo1 archivo2
```

que se utiliza para copiar *archivo1* a *archivo2*. Una vez que el shell se ha bifurcado mediante `fork`, el proceso hijo localiza y ejecuta el archivo *cp* y le pasa los nombres de los archivos de origen y destino.

El programa principal de *cp* (y el programa principal de la mayoría de los otros programas en C) contienen la siguiente declaración:

```
main(argc, argv, envp)
```

en donde *argc* es una cuenta del número de elementos en la línea de comandos, incluyendo el nombre del programa. Para el ejemplo anterior, *argc* es 3.

El segundo parámetro, *argv*, es un apuntador a un arreglo. El elemento *i* de ese arreglo es un apuntador a la *i*-ésima cadena en la línea de comandos. En nuestro ejemplo, *argv*[0] apuntaría a la cadena "cp", *argv*[1] apuntaría a la cadena "archivo1" y *argv*[2] apuntaría a la cadena "archivo2".

El tercer parámetro de *main*, *envp*, es un apuntador al entorno, un arreglo de cadenas que contiene asignaciones de la forma *nombre = valor* que se utilizan para pasar información, tal como el tipo de terminal y el nombre del directorio de inicio, a los programas. Hay procedimientos de biblioteca que los programas pueden llamar para obtener las variables de entorno, que a menudo se utilizan para personalizar la forma en que un usuario desea realizar ciertas tareas (por ejemplo, la impresora predeterminada que desea utilizar). En la figura 1-19 no se pasa un entorno al hijo, por lo que el tercer parámetro de *execve* es un cero.

Si `exec` parece complicado, no se desanime; es (en sentido semántico) la más compleja de todas las llamadas al sistema POSIX. Todas las demás son mucho más simples. Como ejemplo de una llamada simple considere a `exit`, que los procesos deben utilizar cuando terminan su ejecución. Tiene un parámetro, el estado de `exit` (0 a 255), que se devuelve al padre mediante *statloc* en la llamada al sistema `waitpid`.

En UNIX los procesos tienen su memoria dividida en tres segmentos: el **segmento de texto** (es decir, el código del programa), el **segmento de datos** (es decir, las variables) y el **segmento de pila**. El segmento de datos crece hacia arriba y la pila crece hacia abajo, como se muestra en la figura 1-20. Entre ellos hay un espacio libre de direcciones sin utilizar. La pila crece hacia ese espacio de manera automática, según sea necesario, pero la expansión del segmento de datos se realiza de manera explícita mediante una llamada al sistema (`brk`), la cual especifica la nueva dirección en donde debe terminar el segmento de datos. Sin embargo, esta llamada no está definida en el estándar de POSIX, ya que se recomienda a los programadores utilizar el procedimiento de biblioteca *malloc* para asignar espacio de almacenamiento en forma dinámica y la implementación subyacente de *malloc* no se consideró como un tema adecuado para su estandarización, ya que pocos programadores lo utilizan directamente y es improbable que alguien se dé cuenta siquiera que brk no está en POSIX.

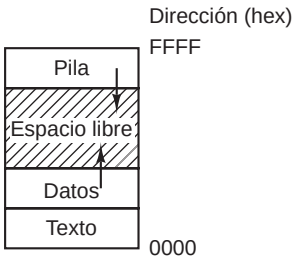


Figura 1-20. Los procesos tienen tres segmentos: de texto, de datos y de pila.

1.6.2 Llamadas al sistema para la administración de archivos

Muchas llamadas al sistema se relacionan con el sistema de archivos. En esta sección analizaremos las llamadas que operan con archivos individuales; en la siguiente sección examinaremos las llamadas que implican el uso de directorios o el sistema de archivos como un todo.

Para leer o escribir en un archivo, éste debe primero abrirse mediante open. Esta llamada especifica el nombre del archivo que se va a abrir, ya sea como un nombre de ruta absoluto o relativo al directorio de trabajo, y un código de *O_RDONLY*, *O_WRONLY* o *O_RDWR*, que significa abrir para lectura, escritura o ambos. Para crear un nuevo archivo se utiliza el parámetro *O_CREAT*. Después se puede utilizar el descriptor de archivo devuelto para leer o escribir. Al terminar, el archivo se puede cerrar mediante `close`, que hace que el descriptor de archivo esté disponible para reutilizarlo en una llamada a `open` posterior.

Las llamadas de uso más frecuente son sin duda `read` y `write`. Anteriormente vimos a `read`. `Write` tiene los mismos parámetros.

Aunque la mayoría de los programas leen y escriben archivos en forma secuencial, para ciertas aplicaciones los programas necesitan la capacidad de acceder a cualquier parte del archivo en forma aleatoria. Con cada archivo hay un apuntador asociado, el cual indica la posición actual en el archivo. Al leer (escribir) en forma secuencial, por lo general apunta al siguiente byte que se va a leer (escribir). La llamada a `lseek` cambia el valor del apuntador de posición, de manera que las siguientes llamadas a `read` o `write` puedan empezar en cualquier parte del archivo.

`Lseek` tiene tres parámetros: el primero es el descriptor del archivo, el segundo es una posición en el archivo y el tercero indica si la posición en el archivo es relativa al inicio del mismo, a la posición actual o al final del archivo. El valor devuelto por `lseek` es la posición absoluta en el archivo (en bytes) después de modificar el apuntador.

Para cada archivo, UNIX lleva el registro del modo del archivo (archivo regular, especial, directorio, etcétera), su tamaño, la hora de la última modificación y demás información. Para ver esta información, los programas pueden utilizar la llamada al sistema `stat`. El primer parámetro especifica el archivo que se va a inspeccionar; el segundo es un apuntador a una estructura en donde se va a colocar la información. Las llamadas al sistema fstat hacen lo mismo para un archivo abierto.

1.6.3 Llamadas al sistema para la administración de directorios

En esta sección analizaremos algunas llamadas al sistema que se relacionan más con los directorios o con el sistema de archivos como un todo, en vez de relacionarse sólo con un archivo específico, como en la sección anterior. Las primeras dos llamadas, `mkdir` y `rmdir`, crean y eliminan directorios vacíos, respectivamente. La siguiente llamada es `link`. Su propósito es permitir que aparezca el mismo archivo bajo dos o más nombres, a menudo en distintos directorios. Un uso común es para permitir que varios miembros del mismo equipo de programación compartan un archivo común, en donde cada uno de ellos puede ver el archivo en su propio directorio, posiblemente bajo distintos nombres. No es lo mismo compartir un archivo que proporcionar a cada miembro del equipo una copia privada; tener un archivo compartido significa que los cambios que realice cualquier miembro del equipo serán visibles de manera instantánea para los otros miembros; sólo hay un archivo. Cuando se realizan copias de un archivo, los cambios subsiguientes que se realizan en una copia no afectan a las demás.

Para ver cómo funciona `link`, considere la situación de la figura 1-21(a). Aquí hay dos usuarios, *ast* y *jim*, y cada uno tiene su propio directorio con algunos archivos. Si *ast* ejecuta ahora un programa que contenga la llamada al sistema

```
link("/usr/jim/memo", "/usr/ast/nota");
```

el archivo *memo* en el directorio de *jim* se introduce en el directorio de *ast* bajo el nombre *nota*. De aquí en adelante, */usr/jim/memo* y */usr/ast/nota* harán referencia al mismo archivo. Para complementar, hay que tener en cuenta que el lugar en el que se mantengan los directorios de los usuarios, ya sea en */usr*, */user*, */home* o en alguna otra parte es simplemente una decisión que realiza el administrador del sistema local.

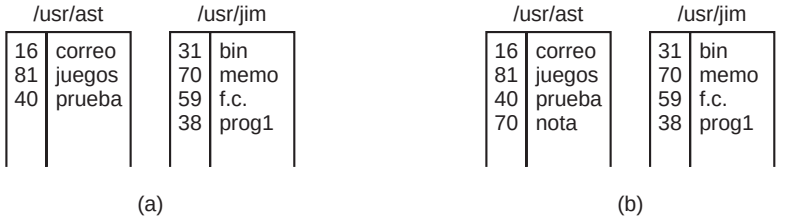


Figura 1-21. a) Dos directorios antes de enlazar */usr/jim/memo* al directorio de *ast*. b) Los mismos directorios después del enlace.

Entendiendo cómo funciona `link` probablemente nos aclare lo que hace. Todo archivo en UNIX tiene un número único, su número-i, que lo identifica. Este número-i es un índice en una tabla de **nodos-i**, uno por archivo, que indican quién es propietario del archivo, en dónde están sus bloques de disco, etcétera. Un directorio es simplemente un archivo que contiene un conjunto de pares (número-i, nombre ASCII). En las primeras versiones de UNIX, cada entrada de directorio era de 16 bytes: 2 bytes para el número-i y 14 bytes para el nombre. Ahora se necesita una estructura más complicada para soportar los nombres de archivo extensos, pero en concepto un directorio sigue siendo un conjunto de pares (número-i, nombre ASCII). En la figura 1-21, *correo* tiene el número-i 16, y así sucesivamente. Lo que `link` hace es tan sólo crear una nueva entrada de directorio con un nombre

(posiblemente nuevo), usando el número-i de un archivo existente. En la figura 1-21(b), dos entradas tienen el mismo número-i (70) y por ende se refieren al mismo archivo. Si más adelante se elimina una de las dos mediante la llamada al sistema `unlink`, la otra sigue vigente. Si se eliminan ambas, UNIX ve que no existen entradas para el archivo (un campo en el nodo-i lleva la cuenta del número de entradas de directorio que apuntan al archivo), por lo que el archivo se elimina del disco.

Como dijimos antes, la llamada al sistema `mount` permite combinar dos sistemas de archivos en uno. Una situación común es hacer que el sistema de archivos raíz contenga las versiones binarias (ejecutables) de los comandos comunes y otros archivos de uso frecuente, en un disco duro. Así, el usuario puede insertar un disco de CD-ROM con los archivos que se van a leer en la unidad de CD-ROM.

Al ejecutar la llamada al sistema `mount`, el sistema de archivos de CD-ROM se puede adjuntar al sistema de archivos raíz, como se muestra en la figura 1-22. Una instrucción común en C para realizar el montaje es

```
mount("/dev/fd0", "/mnt", 0);
```

donde el primer parámetro es el nombre de un archivo especial de bloque para la unidad 0, el segundo parámetro es la posición en el árbol en donde se va a montar y el tercer parámetro indica que el sistema de archivo se va a montar en modo de lectura-escritura o de sólo escritura.



Figura 1-22. (a) Sistema de archivos antes del montaje. (b) Sistema de archivos después del montaje.

Después de la llamada a `mount`, se puede tener acceso a un archivo en la unidad 0 con sólo utilizar su ruta del directorio raíz o del directorio de trabajo, sin importar en cuál unidad se encuentre. De hecho, las unidades segunda, tercera y cuarta también se pueden montar en cualquier parte del árbol. La llamada a `mount` hace posible integrar los medios removibles en una sola jerarquía de archivos integrada, sin importar en qué dispositivo se encuentra un archivo. Aunque este ejemplo involucra el uso de CD-ROMs, también se pueden montar porciones de los discos duros (que a menudo se les conoce como **particiones** o **dispositivos menores**) de esta forma, así como los discos duros y memorias USB externos. Cuando ya no se necesita un sistema de archivos, se puede desmontar mediante la llamada al sistema `umount`.

1.6.4 Miscelánea de llamadas al sistema

También existe una variedad de otras llamadas al sistema. Sólo analizaremos cuatro de ellas aquí. La llamada a `chdir` cambia el directorio de trabajo actual. Después de la llamada

```
chdir("/usr/ast/prueba");
```

una instrucción para abrir el archivo *xyz* abrirá */usr/ast/prueba/xyz*. El concepto de un directorio de trabajo elimina la necesidad de escribir nombres de ruta absolutos (extensos) todo el tiempo.

En UNIX, cada archivo tiene un modo que se utiliza por protección. El modo incluye los bits leer-escribir-ejecutar para el propietario, grupo y los demás. La llamada al sistema chmod hace posible modificar el modo de un archivo. Por ejemplo, para que un archivo sea de sólo lectura para todos excepto el propietario, podríamos ejecutar

```
chmod("archivo", 0644);
```

La llamada al sistema kill es la forma en que los usuarios y los procesos de usuario envían señales. Si un proceso está preparado para atrapar una señal específica, y luego ésta llega, se ejecuta un manejador de señales. Si el proceso no está preparado para manejar una señal, entonces su llegada mata el proceso (de aquí que se utilice ese nombre para la llamada).

POSIX define varios procedimientos para tratar con el tiempo. Por ejemplo, time sólo devuelve la hora actual en segundos, en donde 0 corresponde a Enero 1, 1970 a medianoche (justo cuando el día empezaba y no terminaba). En las computadoras que utilizan palabras de 32 bits, el valor máximo que puede devolver time es de $2^{32} - 1$ segundos (suponiendo que se utiliza un entero sin signo). Este valor corresponde a un poco más de 136 años. Así, en el año 2106 los sistemas UNIX de 32 bits se volverán locos, algo parecido al famoso problema del año 2000 (Y2K) que hubiera causado estragos con las computadoras del mundo en el 2000, si no fuera por el masivo esfuerzo que puso la industria de IT para corregir el problema. Si actualmente usted tiene un sistema UNIX de 32 bits, se le recomienda que lo intercambie por uno de 64 bits antes del año 2106.

1.6.5 La API Win32 de Windows

Hasta ahora nos hemos enfocado principalmente en UNIX. Es tiempo de dar un vistazo breve a Windows. Windows y UNIX difieren de una manera fundamental en sus respectivos modelos de programación. Un programa de UNIX consiste en código que realiza una cosa u otra, haciendo llamadas al sistema para realizar ciertos servicios. En contraste, un programa de Windows es por lo general manejado por eventos. El programa principal espera a que ocurra cierto evento y después llama a un procedimiento para manejarlo. Los eventos comunes son las teclas que se oprimen, el ratón que se desplaza, un botón de ratón que se oprime o un CD-ROM que se inserta. Después, los manejadores se llaman para procesar el evento, actualizar la pantalla y actualizar el estado interno del programa. En todo, esto produce un estilo algo distinto de programación que con UNIX, pero debido a que el enfoque de este libro es acerca de la función y la estructura del sistema operativo, estos distintos modelos de programación no nos preocuparán por mucho.

Desde luego que Windows también tiene llamadas al sistema. Con UNIX, hay casi una relación de uno a uno entre las llamadas al sistema (por ejemplo, `read`) y los procedimientos de biblioteca (por ejemplo, *read*) que se utilizan para invocar las llamadas al sistema. En otras palabras, para cada llamada al sistema, es raro que haya un procedimiento de biblioteca que sea llamado para invocarlo, como se indica en la figura 1-17. Lo que es más, POSIX tiene aproximadamente 100 llamadas a procedimientos.

Con Windows, la situación es bastante distinta. Para empezar, las llamadas a la biblioteca y las llamadas al sistema están muy desacopladas. Microsoft ha definido un conjunto de procedimientos conocidos como **API Win32** (*Application Program Interface*, Interfaz de programación de aplicaciones) que los programadores deben utilizar para obtener los servicios del sistema operativo. Esta interfaz se proporciona (parcialmente) en todas las versiones de Windows, desde Windows 95. Al desacoplar la interfaz de las llamadas actuales al sistema, Microsoft retiene la habilidad de modificar las llamadas al sistema en el tiempo (incluso de versión en versión) sin invalidar los programas existentes. Lo que constituye realmente a Win32 es también ligeramente ambiguo, debido a que Windows 2000, Windows XP y Windows Vista tienen muchas nuevas llamadas que antes no estaban disponibles. En esta sección Win32 significa la interfaz que soportan todas las versiones de Windows.

El número de llamadas a la API Win32 es muy grande, alrededor de los miles. Lo que es más, aunque muchas de ellas invocan llamadas al sistema, un número considerable de las mismas se lleva a cabo completamente en espacio de usuario. Como consecuencia, con Windows es imposible ver lo que es una llamada al sistema (el kernel se encarga de ello) y qué es simplemente una llamada a la biblioteca en espacio de usuario. De hecho, lo que es una llamada al sistema en una versión de Windows se puede realizar en espacio de usuario en una versión diferente y viceversa. Cuando hablemos sobre las llamadas al sistema Windows en este libro, utilizaremos los procedimientos de Win32 (en donde sea apropiado), ya que Microsoft garantiza que éstos serán estables a través del tiempo. Pero vale la pena recordar que no todos ellos son verdaderas llamadas al sistema (es decir, traps al kernel).

La API Win32 tiene un gran número de llamadas para administrar ventanas, figuras geométricas, texto, tipos de letras, barras de desplazamiento, cuadros de diálogo, menús y otras características de la GUI. Hasta el grado de en que el subsistema de gráficos se ejecute en el kernel (lo cual es cierto en algunas versiones de Windows, pero no en todas), éstas son llamadas al sistema; en caso contrario, sólo son llamadas a la biblioteca. ¿Debemos hablar sobre estas llamadas en este libro o no? Como en realidad no se relacionan con la función de un sistema operativo, hemos decidido que no, aun cuando el kernel puede llevarlas a cabo. Los lectores interesados en la API Win32 pueden consultar uno de los muchos libros acerca del tema (por ejemplo, Hart, 1997; Rector y Newcomer, 1997; y Simon, 1997).

Incluso está fuera de cuestión introducir todas las llamadas a la API Win32 aquí, por lo que nos restringiremos a las llamadas que apenas si corresponden a la funcionalidad de las llamadas de Unix que se listan en la figura 1-18. Éstas se listan en la figura 1-23.

Ahora daremos un breve repaso a la lista de la figura 1-23. `CreateProcess` crea un proceso. Realiza el trabajo combinado de `fork` y `execve` en UNIX. Tiene muchos parámetros que especifican las propiedades del proceso recién creado. Windows no tiene una jerarquía de procesos como UNIX, por lo que no hay un concepto de un proceso padre y un proceso hijo. Una vez que se crea un proceso, el creador y el creado son iguales. `WaitForSingleObject` se utiliza para esperar un evento. Se pueden esperar muchos eventos posibles. Si el parámetro especifica un proceso, entonces el proceso llamador espera a que el proceso especificado termine, lo cual se hace mediante `ExitProcess`.

Las siguientes seis llamadas operan con archivo y tienen una funcionalidad similar a sus contrapartes de UNIX, aunque difieren en cuanto a los parámetros y los detalles. Aún así, los archivos

UNIX	Win32	Descripción
fork	CreateProcess	Crea un nuevo proceso
waitpid	WaitForSingleObject	Puede esperar a que un proceso termine
execve	(ninguno)	CreateProces = fork + execve
exit	ExitProcess	Termina la ejecución
open	CreateFile	Crea un archivo o abre uno existente
close	CloseHandle	Cierra un archivo
read	ReadFile	Lee datos de un archivo
write	WriteFile	Escribe datos en un archivo
lseek	SetFilePointer	Desplaza el apuntador del archivo
stat	GetFileAttributesEx	Obtiene varios atributos de un archivo
mkdir	CreateDirectory	Crea un nuevo directorio
rmdir	RemoveDirectory	Elimina un directorio vacío
link	(ninguno)	Win32 no soporta los enlaces
unlink	DeleteFile	Destruye un archivo existente
mount	(ninguno)	Win32 no soporta el montaje
umount	(ninguno)	Win32 no soporta el montaje
chdir	SetCurrentDirectory	Cambia el directorio de trabajo actual
chmod	(ninguno)	Win32 no soporta la seguridad (aunque NT sí)
kill	(ninguno)	Win32 no soporta las señales
time	GetLocalTime	Obtiene la hora actual

Figura 1-23. Las llamadas a la API Win32 que apenas corresponden a las llamadas de UNIX de la figura 1-18.

se pueden abrir, cerrar, leer y escribir en ellos en forma muy parecida a UNIX. Las llamadas `SetFilePointer` y `GetFileAttributesEx` establecen la posición de un archivo y obtienen algunos de sus atributos.

Windows tiene directorios, que se crean y eliminan con las llamadas a la API `CreateDirectory` y `RemoveDirectory`, respectivamente. También hay una noción de un directorio actual, el cual se establece mediante `SetCurrentDirectory`. La hora actual del día se adquiere mediante el uso de `GetLocalTime`.

La interfaz de Win32 no tiene enlaces a archivos, sistemas de archivos montados, seguridad ni señales, por lo que las llamadas correspondientes a las de UNIX no existen. Desde luego que Win32 tiene un gran número de otras llamadas que UNIX no tiene, en especial para administrar la GUI. Y Windows Vista tiene un elaborado sistema de seguridad, además de que también soporta los enlaces de archivos.

Tal vez vale la pena hacer una última observación acerca de Win32: no es una interfaz tan uniforme o consistente. La principal culpabilidad de esto fue la necesidad de tener compatibilidad hacia atrás con la interfaz de 16 bits anterior utilizada en Windows 3.x.

1.7 ESTRUCTURA DE UN SISTEMA OPERATIVO

Ahora que hemos visto la apariencia exterior de los sistemas operativos (es decir, la interfaz del programador), es tiempo de dar un vistazo a su interior. En las siguientes secciones analizaremos seis estructuras distintas que se han probado, para poder darnos una idea del espectro de posibilidades. De ninguna manera quiere esto decir que sean exhaustivas, pero nos dan una idea de algunos diseños que se han probado en la práctica. Los seis diseños son: sistemas monolíticos, sistemas de capas, microkernels, sistemas cliente-servidor, máquinas virtuales y exokernels.

1.7.1 Sistemas monolíticos

En este diseño, que hasta ahora se considera como la organización más común, todo el sistema operativo se ejecuta como un solo programa en modo kernel. El sistema operativo se escribe como una colección de procedimientos, enlazados entre sí en un solo programa binario ejecutable extenso. Cuando se utiliza esta técnica, cada procedimiento en el sistema tiene la libertad de llamar a cualquier otro, si éste proporciona cierto cómputo útil que el primero necesita. Al tener miles de procedimientos que se pueden llamar entre sí sin restricción, con frecuencia se produce un sistema poco manejable y difícil de comprender.

Para construir el programa objeto actual del sistema operativo cuando se utiliza este diseño, primero se compilan todos los procedimientos individuales (o los archivos que contienen los procedimientos) y luego se vinculan en conjunto para formar un solo archivo ejecutable, usando el enlazador del sistema. En términos de ocultamiento de información, en esencia no hay nada: todos los procedimientos son visibles para cualquier otro procedimiento (en contraste a una estructura que contenga módulos o paquetes, en donde la mayor parte de la información se oculta dentro de módulos y sólo los puntos de entrada designados de manera oficial se pueden llamar desde el exterior del módulo).

Sin embargo, hasta en los sistemas monolíticos es posible tener cierta estructura. Para solicitar los servicios (llamadas al sistema) que proporciona el sistema operativo, los parámetros se colocan en un lugar bien definido (por ejemplo, en la pila) y luego se ejecuta una instrucción de trap. Esta instrucción cambia la máquina del modo usuario al modo kernel y transfiere el control al sistema operativo, lo cual se muestra como el paso 6 en la figura 1-17. Después el sistema operativo obtiene los parámetros y determina cuál es la llamada al sistema que se va a llevar a cabo. Después la indiza en una tabla que contiene en la ranura *k* un apuntador al procedimiento que lleva a cabo la llamada al sistema *k* (paso 7 en la figura 1-17).

Esta organización sugiere una estructura básica para el sistema operativo:

1. Un programa principal que invoca el procedimiento de servicio solicitado.
2. Un conjunto de procedimientos de servicio que llevan a cabo las llamadas al sistema.
3. Un conjunto de procedimientos utilitarios que ayudan a los procedimientos de servicio.

En este modelo, para cada llamada al sistema hay un procedimiento de servicio que se encarga de la llamada y la ejecuta. Los procedimientos utilitarios hacen cosas que necesitan varios procedi-

mientos de servicio, como obtener datos de los programas de usuario. Esta división de los procedimientos en tres niveles se muestra en la figura 1-24.

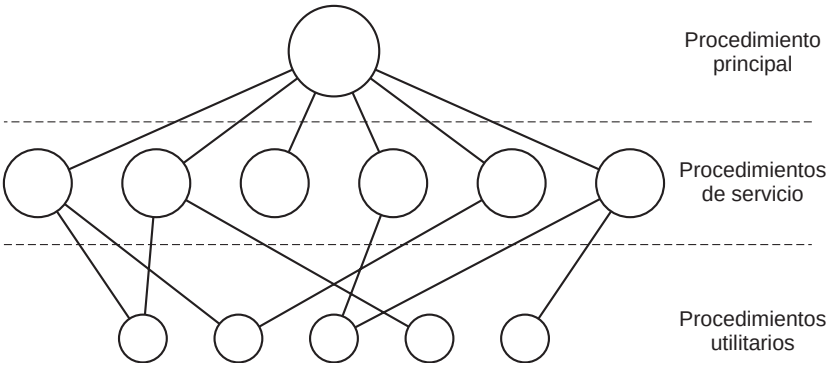


Figura 1-24. Un modelo de estructuración simple para un sistema monolítico.

Además del núcleo del sistema operativo que se carga al arrancar la computadora, muchos sistemas operativos soportan extensiones que se pueden cargar, como los drivers de dispositivos de E/S y sistemas de archivos. Estos componentes se cargan por demanda.

1.7.2 Sistemas de capas

Una generalización del diseño de la figura 1-24 es organizar el sistema operativo como una jerarquía de capas, cada una construida encima de la que tiene abajo. El primer sistema construido de esta forma fue el sistema THE, construido en Technische Hogeschool Eindhoven en Holanda por E. W. Dijkstra (1968) y sus estudiantes. El sistema THE era un sistema simple de procesamiento por lotes para una computadora holandesa, la Electrologica X8, que tenía 32K de palabras de 27 bits (los bits eran costosos en aquel entonces).

El sistema tenía seis capas, como se muestra en la figura 1-25. El nivel 0 se encargaba de la asignación del procesador, de cambiar entre un proceso y otro cuando ocurrían interrupciones o expiraban los temporizadores. Por encima del nivel 0, el sistema consistía en procesos secuenciales, cada uno de los cuales e podía programar sin necesidad de preocuparse por el hecho de que había varios procesos en ejecución en un solo procesador. En otras palabras, el nivel 0 proporcionaba la multiprogramación básica de la CPU.

La capa 1 se encargaba de la administración de la memoria. Asignaba espacio para los procesos en la memoria principal y en un tambor de palabras de 512 K que se utilizaba para contener partes de procesos (páginas), para los que no había espacio en la memoria principal. Por encima de la capa 1, los procesos no tenían que preocuparse acerca de si estaban en memoria o en el tambor; el software de la capa 1 se encargaba de asegurar que las páginas se llevaran a memoria cuando se requerían.

Capa	Función
5	El operador
4	Programas de usuario
3	Administración de la entrada/salida
2	Comunicación operador-proceso
1	Administración de memoria y tambor
0	Asignación del procesador y multiprogramación

Figura 1-25. Estructura del sistema operativo THE.

La capa 2 se encargaba de la comunicación entre cada proceso y la consola del operador (es decir, el usuario). Encima de esta capa, cada proceso tenía en efecto su propia consola de operador. La capa 3 se encargaba de administrar los dispositivos de E/S y de guardar en búferes los flujos de información dirigidos para y desde ellos. Encima de la capa 3, cada proceso podía trabajar con los dispositivos abstractos de E/S con excelentes propiedades, en vez de los dispositivos reales con muchas peculiaridades. La capa 4 era en donde se encontraban los programas de usuario. No tenían que preocuparse por la administración de los procesos, la memoria, la consola o la E/S. El proceso operador del sistema se encontraba en el nivel 5.

Una mayor generalización del concepto de capas estaba presente en el sistema MULTICS. En vez de capa, MULTICS se describió como una serie de anillos concéntricos, en donde los interiores tenían más privilegios que los exteriores (que en efecto viene siendo lo mismo). Cuando un procedimiento en un anillo exterior quería llamar a un procedimiento en un anillo interior, tenía que hacer el equivalente de una llamada al sistema; es decir, una instrucción `TRAP` cuyos parámetros se comprobara cuidadosamente que fueran válidos antes de permitir que continuara la llamada. Aunque todo el sistema operativo era parte del espacio de direcciones de cada proceso de usuario en MULTICS, el hardware hizo posible que se designaran procedimientos individuales (en realidad, segmentos de memoria) como protegidos contra lectura, escritura o ejecución.

Mientras que en realidad el esquema de capas de THE era sólo una ayuda de diseño, debido a que todas las partes del sistema estaban enlazadas entre sí en un solo programa ejecutable, en MULTICS el mecanismo de los anillos estaba muy presente en tiempo de ejecución y el hardware se encargaba de implementarlo. La ventaja del mecanismo de los anillos es que se puede extender fácilmente para estructurar los subsistemas de usuario. Por ejemplo, un profesor podría escribir un programa para evaluar y calificar los programas de los estudiantes, ejecutando este programa en el anillo n, mientras que los programas de los estudiantes se ejecutaban en el anillo $n + 1$ y por ende no podían cambiar sus calificaciones.

1.7.3 Microkernels

Con el diseño de capas, los diseñadores podían elegir en dónde dibujar el límite entre kernel y usuario. Tradicionalmente todos las capas iban al kernel, pero eso no es necesario. De hecho, puede tener mucho sentido poner lo menos que sea posible en modo kernel, debido a que los errores en el

kernel pueden paralizar el sistema de inmediato. En contraste, los procesos de usuario se pueden configurar para que tengan menos poder, por lo que un error en ellos tal vez no sería fatal.

Varios investigadores han estudiado el número de errores por cada 1000 líneas de código (por ejemplo, Basilli y Perricone, 1984; y Ostrand y Weyuker, 2002). La densidad de los errores depende del tamaño del módulo, su tiempo de vida y más, pero una cifra aproximada para los sistemas industriales formales es de diez errores por cada mil líneas de código. Esto significa que es probable que un sistema operativo monolítico de cinco millones de líneas de código contenga cerca de 50,000 errores en el kernel. Desde luego que no todos estos son fatales, ya que algunos errores pueden ser cosas tales como emitir un mensaje de error incorrecto en una situación que ocurre raras veces. Sin embargo, los sistemas operativos tienen tantos errores que los fabricantes de computadoras colocan botones de reinicio en ellas (a menudo en el panel frontal), algo que los fabricantes de televisiones, estéreos y autos no hacen, a pesar de la gran cantidad de software en estos dispositivos.

La idea básica detrás del diseño de microkernel es lograr una alta confiabilidad al dividir el sistema operativo en módulos pequeños y bien definidos, sólo uno de los cuales (el microkernel) se ejecuta en modo kernel y el resto se ejecuta como procesos de usuario ordinarios, sin poder relativamente. En especial, al ejecutar cada driver de dispositivo y sistema de archivos como un proceso de usuario separado, un error en alguno de estos procesos puede hacer que falle ese componente, pero no puede hacer que falle todo el sistema. Así, un error en el driver del dispositivo de audio hará que el sonido sea confuso o se detenga, pero la computadora no fallará. En contraste, en un sistema monolítico con todos los drivers en el kernel, un driver de audio con errores puede hacer fácilmente referencia a una dirección de memoria inválida y llevar a todo el sistema a un alto rotundo en un instante.

Se han implementado y desplegado muchos microkernels (Accetta y colaboradores, 1986; Haertig y colaboradores, 1997; Heiser y colaboradores, 2006; Herder y colaboradores, 2006; Hildebrand, 1992; Kirsch y colaboradores, 2005; Liedtke, 1993, 1995, 1996; Pike y colaboradores, 1992; y Zuberi y colaboradores, 1999). Son en especial comunes en las aplicaciones en tiempo real, industriales, aeronáuticas y militares que son de misión crítica y tienen requerimientos de confiabilidad muy altos. Algunos de los microkernels mejor conocidos son Integrity, K42, L4, PikeOS, QNX, Symbian y MINIX 3. Ahora veremos en forma breve las generalidades acerca de MINIX 3, que ha llevado la idea de la modularidad hasta el límite, dividiendo la mayor parte del sistema operativo en varios procesos independientes en modo usuario. MINIX 3 es un sistema de código fuente abierto en conformidad con POSIX, disponible sin costo en *www.minix3.org* (Herder y colaboradores, 2006a; Herder y colaboradores, 2006b).

El microkernel MINIX 3 sólo tiene cerca de 3200 líneas de C y 800 líneas de ensamblador para las funciones de muy bajo nivel, como las que se usan para atrapar interrupciones y conmutar proceso. El código de C administra y planifica los procesos, se encarga de la comunicación entre procesos (al pasar mensajes entre procesos) y ofrece un conjunto de aproximadamente 35 llamadas al kernel para permitir que el resto del sistema operativo realice su trabajo. Estas llamadas realizan funciones tales como asociar los drivers a las interrupciones, desplazar datos entre espacios de direcciones e instalar nuevos mapas de memoria para los procesos recién creados. La estructura de procesos de MINIX 3 se muestra en la figura 1-26, en donde los manejadores de las llamadas al kernel se etiquetan como *Sys*. El manejador de dispositivo para el reloj también está

en el kernel, debido a que el planificador interactúa de cerca con él. Todos los demás dispositivos controladores se ejecutan como procesos de usuario separados.

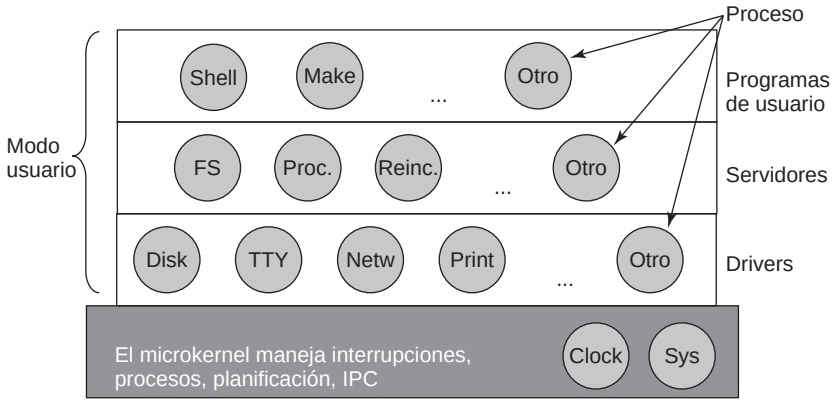


Figura 1-26. Estructura del sistema MINIX 3.

Fuera del kernel, el sistema se estructura como tres capas de procesos, todos se ejecutan en modo usuario. La capa más inferior contiene los drivers de dispositivos. Como todos se ejecutan en modo usuario, no tienen acceso físico al espacio de puertos de E/S y no pueden emitir comandos de E/S directamente. En vez de ello, para programar un dispositivo de E/S el driver crea una estructura para indicarle qué valores debe escribir en cuáles puertos de E/S y realiza una llamada al kernel para indicarle que realice la escritura. Esta metodología permite que el kernel compruebe que el driver esté escribiendo (o leyendo) de la E/S que está autorizado a utilizar. En consecuencia (y a diferencia de un diseño monolítico), un driver de audio defectuoso no puede escribir accidentalmente en el disco.

Encima de los drivers hay otra capa en modo usuario que contiene los servidores, que realizan la mayor parte del trabajo del sistema operativo. Uno o más servidores de archivos administran el (los) sistema(s) de archivos, el administrador de procesos crea, destruye y administra los procesos y así sucesivamente. Los programas de usuario obtienen servicios del sistema operativo mediante el envío de mensajes cortos a los servidores, pidiéndoles las llamadas al sistema POSIX. Por ejemplo, un proceso que necesite realizar una llamada `read` envía un mensaje a uno de los servidores de archivos para indicarle qué debe leer.

Un servidor interesante es el **servidor de reencarnación**, cuyo trabajo es comprobar si otros servidores y drivers están funcionando en forma correcta. En caso de que se detecte uno defectuoso, se reemplaza automáticamente sin intervención del usuario. De esta forma, el sistema es autocorregible y puede lograr una alta confiabilidad.

El sistema tiene muchas restricciones que limitan el poder de cada proceso. Como dijimos antes, los drivers sólo pueden utilizar los puertos de E/S autorizados, pero el acceso a las llamadas al kernel también está controlado dependiendo del proceso, al igual que la habilidad de enviar mensajes a otros procesos. Además, los procesos pueden otorgar un permiso limitado a otros procesos para hacer que el kernel acceda a sus espacios de direcciones. Como ejemplo, un sistema de archivos

puede otorgar permiso al dispositivo controlador de disco para dejar que el kernel coloque un bloque de disco recién leído en una dirección específica dentro del espacio de direcciones del sistema de archivos. El resultado de todas estas restricciones es que cada driver y servidor tiene el poder exacto para realizar su trabajo y no más, con lo cual se limita en forma considerable el daño que puede ocasionar un componente defectuoso.

Una idea que está en parte relacionada con tener un kernel mínimo es colocar el **mecanismo** para hacer algo en el kernel, pero no la **directiva**. Para aclarar mejor este punto, considere la planificación de los proceso. Un algoritmo de planificación relativamente simple sería asignar una prioridad a cada proceso y después hacer que el kernel ejecute el proceso de mayor prioridad que sea ejecutable. El mecanismo, en el kernel, es buscar el proceso de mayor prioridad y ejecutarlo. La directiva, asignar prioridades a los procesos, puede realizarse mediante los procesos en modo usuario. De esta forma, la directiva y el mecanismo se pueden desacoplar y el kernel puede reducir su tamaño.

1.7.4 Modelo cliente-servidor

Una ligera variación de la idea del microkernel es diferenciar dos clases de procesos: los **servidores**, cada uno de los cuales proporciona cierto servicio, y los **clientes**, que utilizan estos servicios. Este modelo se conoce como **cliente-servidor**. A menudo la capa inferior es un microkernel, pero eso no es requerido. La esencia es la presencia de procesos cliente y procesos servidor.

La comunicación entre clientes y servidores se lleva a cabo comúnmente mediante el paso de mensajes. Para obtener un servicio, un proceso cliente construye un mensaje indicando lo que desea y lo envía al servicio apropiado. Después el servicio hace el trabajo y envía de vuelta la respuesta. Si el cliente y el servidor se ejecutan en el mismo equipo se pueden hacer ciertas optimizaciones, pero en concepto estamos hablando sobre el paso de mensajes.

Una generalización obvia de esta idea es hacer que los clientes y los servidores se ejecuten en distintas computadoras, conectadas mediante una red de área local o amplia, como se describe en la figura 1-27. Como los clientes se comunican con los servidores mediante el envío de mensajes, no necesitan saber si los mensajes se manejan en forma local en sus propios equipos o si se envían a través de una red a servidores en un equipo remoto. En cuanto a lo que al cliente concierne, lo mismo ocurre en ambos casos: se envían las peticiones y se regresan las respuestas. Por ende, el modelo cliente-servidor es una abstracción que se puede utilizar para un solo equipo o para una red de equipos.

Cada vez hay más sistemas que involucran a los usuarios en sus PCs domésticas como clientes y equipos más grandes que operan en algún otro lado como servidores. De hecho, la mayor parte de la Web opera de esta forma. Una PC envía una petición de una página Web al servidor y la página Web se envía de vuelta. Éste es un uso común del modelo cliente-servidor en una red.

1.7.5 Máquinas virtuales

Las versiones iniciales del OS/360 eran, en sentido estricto, sistemas de procesamiento por lotes. Sin embargo, muchos usuarios del 360 querían la capacidad de trabajar de manera interactiva en una terminal, por lo que varios grupos, tanto dentro como fuera de IBM, decidieron escribir siste-

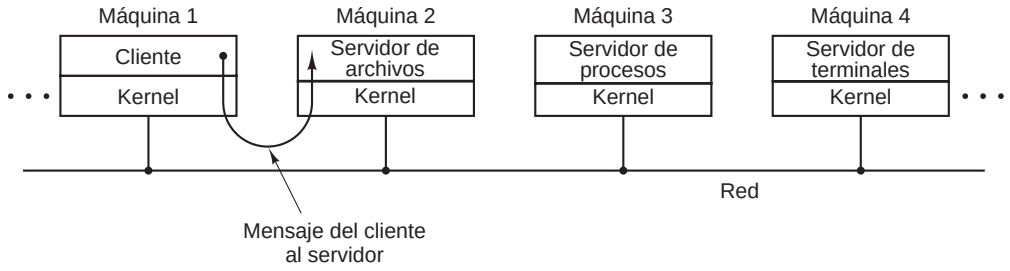


Figura 1-27. El modelo cliente-servidor sobre una red.

mas de tiempo compartido para este sistema. El sistema de tiempo compartido oficial de IBM, conocido como TSS/360, se liberó después de tiempo y cuando por fin llegó era tan grande y lento que pocos sitios cambiaron a este sistema. En cierto momento fue abandonado, una vez que su desarrollo había consumido cerca de 50 millones de dólares (Graham, 1970). Pero un grupo en el Scientific Center de IBM en Cambridge, Massachusetts, produjo un sistema radicalmente distinto que IBM aceptó eventualmente como producto. Un descendiente lineal de este sistema, conocido como **z/VM**, se utiliza ampliamente en la actualidad, en las mainframes de IBM (zSeries) que se utilizan mucho en centros de datos corporativos extensos, por ejemplo, como servidores de comercio electrónico que manejan cientos o miles de transacciones por segundo y utilizan bases de datos cuyos tamaños llegan a ser hasta de varios millones de gigabytes.

VM/370

Este sistema, que en un principio se llamó CP/CMS y posteriormente cambió su nombre a VM/370 (Seawright y MacKinnon, 1979), estaba basado en una astuta observación: un sistema de tiempo compartido proporciona (1) multiprogramación y (2) una máquina extendida con una interfaz más conveniente que el hardware por sí solo. La esencia de VM/370 es separar por completo estas dos funciones.

El corazón del sistema, que se conoce como **monitor de máquina virtual**, se ejecuta en el hardware solamente y realiza la multiprogramación, proporcionando no una, sino varias máquinas virtuales a la siguiente capa hacia arriba, como se muestra en la figura 1-28. Sin embargo, a diferencia de otros sistemas operativos, estas máquinas virtuales no son máquinas extendidas, con archivos y otras características adecuadas. En vez de ello, son copias *exactas* del hardware, incluyendo el modo kernel/ usuario, la E/S, las interrupciones y todo lo demás que tiene la máquina real.

Como cada máquina virtual es idéntica al verdadero hardware, cada una puede ejecutar cualquier sistema operativo que se ejecute directamente sólo en el hardware. Distintas máquinas virtuales pueden (y con frecuencia lo hacen) ejecutar distintos sistemas operativos. En el sistema VM/370 original, algunas ejecutaban OS/360 o uno de los otros sistemas operativos extensos de procesamiento por lotes o de procesamiento de transacciones, mientras que otros ejecutaban un sistema interactivo de un solo usuario llamado **CMS** (Conversational Monitor System; **Sistema monitor conversacional**) para los usuarios interactivos de tiempo compartido. Este último fue popular entre los programadores.

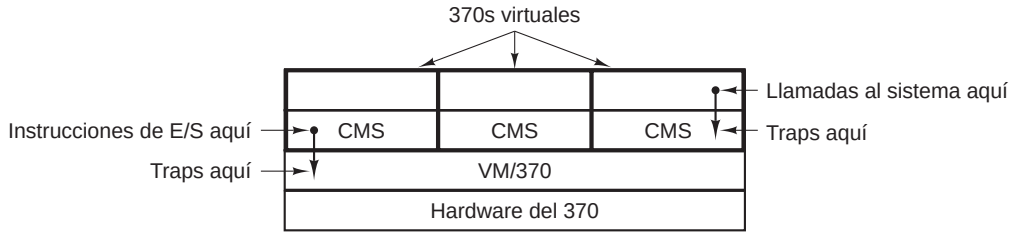


Figura 1-28. La estructura de VM/370 con CMS.

Cuando un programa de CMS ejecutaba una llamada al sistema, ésta quedaba atrapada para el sistema operativo en su propia máquina virtual, no para VM/370, de igual forma que si se ejecutara en una máquina real, en vez de una virtual. Después, CMS emitía las instrucciones normales de E/S de hardware para leer su disco virtual o lo que fuera necesario para llevar a cabo la llamada. Estas instrucciones de E/S eran atrapadas por la VM/370, que a su vez las ejecutaba como parte de su simulación del hardware real. Al separar por completo las funciones de multiprogramación y proporcionar una máquina extendida, cada una de las piezas podían ser más simples, más flexibles y mucho más fáciles de mantener.

En su encarnación moderna, z/VM se utiliza por lo general para ejecutar varios sistemas operativos completos, en vez de sistemas simplificados de un solo usuario como CMS. Por ejemplo, la serie zSeries es capaz de ejecutar una o más máquinas virtuales de Linux junto con los sistemas operativos tradicionales de IBM.

Redescubrimiento de las máquinas virtuales

Mientras que IBM ha tenido un producto de máquina virtual disponible durante cuatro décadas, y unas cuantas compañías más como Sun Microsystems y Hewlett-Packard han agregado recientemente el soporte de máquinas virtuales a sus servidores empresariales de alto rendimiento, la idea de la virtualización se había ignorado por mucho tiempo en el mundo de la PC, hasta hace poco. Pero en los últimos años, se han combinado nuevas necesidades, nuevo software y nuevas tecnologías para convertirla en un tema de moda.

Primero hablaremos sobre las necesidades. Muchas compañías han ejecutado tradicionalmente sus servidores de correo, servidores Web, servidores FTP y otros servidores en computadoras separadas, algunas veces con distintos sistemas operativos. Consideran la virtualización como una forma de ejecutarlos todos en la misma máquina, sin que una falla de un servidor haga que falle el resto.

La virtualización también es popular en el mundo del hospedaje Web. Sin ella, los clientes de hospedaje Web se ven obligados a elegir entre el **hospedaje compartido** (que les ofrece sólo una cuenta de inicio de sesión en un servidor Web, pero ningún control sobre el software de servidor) y hospedaje dedicado (que les ofrece su propia máquina, lo cual es muy flexible pero no es costeable para los sitios Web de pequeños a medianos). Cuando una compañía de hospedaje Web ofrece la renta de máquinas virtuales, una sola máquina física puede ejecutar muchas máquinas virtuales,

cada una de las cuales parece ser una máquina completa. Los clientes que rentan una máquina virtual pueden ejecutar cualesquier sistema operativo y software que deseen, pero a una fracción del costo de un servidor dedicado (debido a que la misma máquina física soporta muchas máquinas virtuales al mismo tiempo).

Otro uso de la virtualización es para los usuarios finales que desean poder ejecutar dos o más sistemas operativos al mismo tiempo, por decir Windows y Linux, debido a que algunos de sus paquetes de aplicaciones favoritos se ejecutan en el primero y algunos otros en el segundo. Esta situación se ilustra en la figura 1-29(a), en donde el término "monitor de máquinas virtuales" ha cambiado su nombre por el de **hipervisor** de tipo 1 en años recientes.

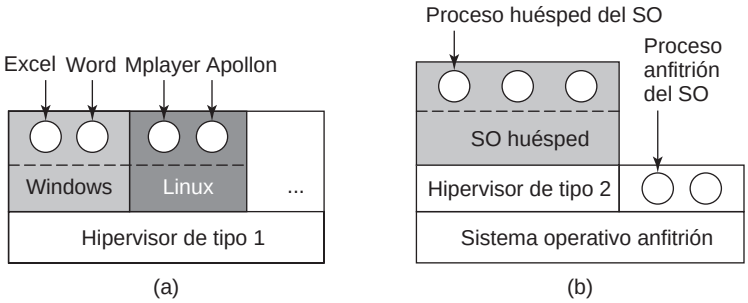


Figura 1-29. (a) Un hipervisor de tipo 1. (b) Un hipervisor de tipo 2.

Ahora pasemos al software. Aunque nadie disputa lo atractivo de las máquinas virtuales, el problema está en su implementación. Para poder ejecutar software de máquina virtual en una computadora, su CPU debe ser virtualizable (Popek y Goldberg, 1974). En síntesis, he aquí el problema. Cuando un sistema operativo que opera en una máquina virtual (en modo usuario) ejecuta una instrucción privilegiada, tal como para modificar el PSW o realizar una operación de E/S, es esencial que el hardware la atrape para el monitor de la máquina virtual, de manera que la instrucción se pueda emular en el software. En algunas CPUs (como el Pentium, sus predecesores y sus clones) los intentos de ejecutar instrucciones privilegiadas en modo de usuario simplemente se ignoran. Esta propiedad hacía que fuera imposible tener máquinas virtuales en este hardware, lo cual explica la falta de interés en el mundo de la PC. Desde luego que había intérpretes para el Pentium que se ejecutaban en el Pentium, pero con una pérdida de rendimiento de por lo general 5x a 10x, no eran útiles para un trabajo serio.

Esta situación cambió como resultado de varios proyectos de investigación académicos en la década de 1990, como Disco en Stanford (Bugnion y colaboradores, 1997), que ocasionaron el surgimiento de productos comerciales (por ejemplo, VMware Workstation) y que reviviera el interés en las máquinas virtuales. VMware Workstation es un hipervisor de tipo 2, el cual se muestra en la figura 1-29(b). En contraste con los hipervisores de tipo 1, que se ejecutaban en el hardware directo, los hipervisores de tipo 2 se ejecutan como programas de aplicación encima de Windows, Linux o algún otro sistema operativo, conocido como **sistema operativo anfitrión**. Una vez que se inicia un hipervisor de tipo 2, lee el CD-ROM de instalación para el **sistema operativo huésped** elegido

y lo instala en un disco virtual, que es tan sólo un gran archivo en el sistema de archivos del sistema operativo anfitrión.

Cuando se arrancar el sistema operativo huésped, realiza lo mismo que en el hardware real; por lo general inicia algunos procesos en segundo plano y después una GUI. Algunos hipervisores traducen los programas binarios del sistema operativo huésped bloque por bloque, reemplazando ciertas instrucciones de control con llamadas al hipervisor. Después, los bloques traducidos se ejecutan y se colocan en caché para su uso posterior.

Un enfoque distinto en cuanto al manejo de las instrucciones de control es el de modificar el sistema operativo para eliminarlas. Este enfoque no es una verdadera virtualización, sino **paravirtualización**. En el capítulo 8 hablaremos sobre la virtualización con más detalle.

La máquina virtual de Java

Otra área en donde se utilizan máquinas virtuales, pero de una manera algo distinta, es para ejecutar programas de Java. Cuando Sun Microsystems inventó el lenguaje de programación Java, también inventó una máquina virtual (es decir, una arquitectura de computadora) llamada **JVM** (*Java Virtual Machine*, Máquina virtual de Java). El compilador de Java produce código para la JVM, que a su vez típicamente se ejecuta mediante un software intérprete de JVM. La ventaja de este método es que el código de la JVM se puede enviar a través de Internet, a cualquier computadora que tenga un intérprete de JVM y se ejecuta allí. Por ejemplo, si el compilador hubiera producido programas binarios para SPARC o Pentium, no se podrían haber enviado y ejecutado en cualquier parte con la misma facilidad. (Desde luego que Sun podría haber producido un compilador que produjera binarios de SPARC y después distribuir un intérprete de SPARC, pero JVM es una arquitectura mucho más simple de interpretar.) Otra ventaja del uso de la JVM es que, si el intérprete se implementa de manera apropiada, que no es algo completamente trivial, se puede comprobar que los programas de JVM entrantes sean seguros para después ejecutarlos en un entorno protegido, de manera que no puedan robar datos ni realizar algún otro daño.

1.7.6 Exokernels

En vez de clonar la máquina actual, como se hace con las máquinas virtuales, otra estrategia es particionarla; en otras palabras, a cada usuario se le proporciona un subconjunto de los recursos. Así, una máquina virtual podría obtener los bloques de disco del 0 al 1023, la siguiente podría obtener los bloques de disco del 1024 al 2047 y así sucesivamente.

En la capa inferior, que se ejecuta en el modo kernel, hay un programa llamado **exokernel** (Engler y colaboradores, 1995). Su trabajo es asignar recursos a las máquinas virtuales y después comprobar los intentos de utilizarlos, para asegurar que ninguna máquina trate de usar los recursos de otra. Cada máquina virtual de nivel de usuario puede ejecutar su propio sistema operativo, al igual que en la VM/370 y las Pentium 8086 virtuales, con la excepción de que cada una está restringida a utilizar sólo los recursos que ha pedido y que le han sido asignados.

La ventaja del esquema del exokernel es que ahorra una capa de asignación. En los otros diseños, cada máquina virtual piensa que tiene su propio disco, con bloques que van desde 0 hasta cier-

to valor máximo, por lo que el monitor de la máquina virtual debe mantener tablas para reasignar las direcciones del disco (y todos los demás recursos). Con el exokernel, esta reasignación no es necesaria. El exokernel sólo necesita llevar el registro para saber a cuál máquina virtual se le ha asignado cierto recurso. Este método sigue teniendo la ventaja de separar la multiprogramación (en el exokernel) del código del sistema operativo del usuario (en espacio de usuario), pero con menos sobrecarga, ya que todo lo que tiene que hacer el exokernel es mantener las máquinas virtuales separadas unas de las otras.

1.8 EL MUNDO SEGÚN C

Por lo general los sistemas operativos son extensos programas en C (o algunas veces C++) que consisten de muchas piezas escritas por muchos programadores. El entorno que se utiliza para desarrollar sistemas operativos es muy distinto a lo que están acostumbrados los individuos (como los estudiantes) al escribir pequeños programas en Java. Esta sección es un intento de proporcionar una muy breve introducción al mundo de la escritura de sistemas operativos para los programadores inexpertos de Java.

1.8.1 El lenguaje C

Ésta no es una guía para el uso de C, sino un breve resumen de algunas de las diferencias clave entre C y Java. Java está basado en C, por lo que existen muchas similitudes entre los dos. Ambos son lenguajes imperativos con tipos de datos, variables e instrucciones de control, por ejemplo. Los tipos de datos primitivos en C son enteros (incluyendo cortos y largos), caracteres y números de punto flotante. Los tipos de datos compuestos se pueden construir mediante el uso de arreglos, estructuras y uniones. Las instrucciones de control en C son similares a las de Java, incluyendo las instrucciones if, switch, for y while. Las funciones y los parámetros son aproximadamente iguales en ambos lenguajes.

Una característica de C que Java no tiene son los apuntadores explícitos. Un **apuntador** es una variable que apunta a (es decir, contiene la dirección de) una variable o estructura de datos. Considere las siguientes instrucciones:

```
char c1, c2, *p;
c1 = 'x';
p = &c1;
c2 = *p;
```

que declaran a *c1* y *c2* como variables de tipo carácter y a *p* como una variable que apunta a (es decir, contiene la dirección de) un carácter. La primera asignación almacena el código ASCII para el carácter 'c' en la variable *c1*. La segunda asigna la dirección de *c1* a la variable apuntador *p*. La tercera asigna el contenido de la variable a la que apunta *p*, a la variable *c2*, por lo que después de ejecutar estas instrucciones, *c2* también contiene el código ASCII para 'c'. En teoría, los apuntadores están tipificados, por lo que no se debe asignar la dirección de un número de punto flotante a un

apuntador tipo carácter, pero en la práctica los compiladores aceptan dichas asignaciones, aunque algunas veces con una advertencia. Los apuntadores son una construcción muy potente, pero también una gran fuente de errores cuando se utilizan sin precaución.

Algunas cosas que C no tiene son: cadenas integradas, hilos, paquetes, clases, objetos, seguridad de tipos y recolección de basura. La última es un gran obstáculo para los sistemas operativos. Todo el almacenamiento en C es estático o el programador lo asigna y libera de manera explícita, por lo general con las funciones de biblioteca *malloc* y *free*. Esta última propiedad (control total del programador sobre la memoria) junto con los apuntadores explícitos que hacen de C un lenguaje atractivo para escribir sistemas operativos. En esencia, los sistemas operativos son sistemas en tiempo real hasta cierto punto, incluso los de propósito general. Cuando ocurre una interrupción, el sistema operativo sólo puede tener unos cuantos microsegundos para realizar cierta acción o de lo contrario, puede perder información crítica. Es intolerable que el recolector de basura entre en acción en un momento arbitrario.

1.8.2 Archivos de encabezado

Por lo general, un proyecto de sistema operativo consiste en cierto número de directorios, cada uno de los cuales contiene muchos archivos *.c* que contienen el código para cierta parte del sistema, junto con varios archivos de encabezado *.h* que contienen declaraciones y definiciones utilizadas por uno o más archivos de código. Los archivos de encabezado también pueden incluir **macros** simples, tales como:

```
#define TAM_BUFER 4096
```

las cuales permiten que el programador nombre constantes, de manera que cuando *TAM_BUFER* se utilice en el código, se reemplace durante la compilación por el número 4096. Una buena práctica de programación de C es nombrar cada constante excepto 0, 1 y -1, y algunas veces también éstas se nombran. Las macros pueden tener parámetros, tales como:

```
#define max(a, b) (a > b ? a : b)
```

con lo cual el programador puede escribir:

```
i = max(j, k+1)
```

y obtener:

```
i = (j > k+1 ? j : k + 1)
```

para almacenar el mayor de *j* y $k + 1$ en *i*. Los encabezados también pueden contener compilación condicional, por ejemplo:

```
#ifdef PENTIUM
intel_int_ack();
#endif
```

lo cual se compila en una llamada a la función *intel_int_ack* si la macro *PENTIUM* está definida y no hace nada en caso contrario. La compilación condicional se utiliza con mucha frecuencia para

aislar el código dependiente de la arquitectura, de manera que cierto código se inserte sólo cuando se compile el sistema en el Pentium, que se inserte otro código sólo cuando el sistema se compile en una SPARC y así sucesivamente. Un archivo *.c* puede incluir físicamente cero o más archivos de encabezado mediante la directiva *#include*. También hay muchos archivos de encabezado comunes para casi cualquier archivo *.c*, y se almacenan en un directorio central.

1.8.3 Proyectos de programación extensos

Para generar el sistema operativo, el compilador de C compila cada archivo *.c* en un **archivo de código objeto**. Estos archivos, que tienen el sufijo *.o*, contienen instrucciones binarias para el equipo de destino. Posteriormente la CPU los ejecutará directamente. No hay nada como el código byte de Java en el mundo de C.

La primera pasada del compilador de C se conoce como **preprocesador de C**. A medida que lee cada archivo *.c*, cada vez que llega a una directiva *#include* va y obtiene el archivo de encabezado cuyo nombre contiene y lo procesa, expandiendo las macros, manejando la compilación condicional (y varias otras cosas) y pasando los resultados a la siguiente pasada del compilador, como si se incluyeran físicamente.

Como los sistemas operativos son muy extensos (es común que tengan cerca de cinco millones de líneas de código), sería imposible tener que recompilar todo cada vez que se modifica un archivo. Por otro lado, para cambiar un archivo de encabezado clave que se incluye en miles de otros archivos, hay que volver a compilar esos archivos. Llevar la cuenta de qué archivos de código objeto depende de cuáles archivos de encabezado es algo que no se puede manejar sin ayuda.

Por fortuna, las computadoras son muy buenas precisamente en este tipo de cosas. En los sistemas UNIX hay un programa llamado *make* (con numerosas variantes tales como *gmake*, *pmake*, etcétera) que lee el *Makefile*, un archivo que indica qué archivos son dependientes de cuáles otros archivos. Lo que hace *make* es ver qué archivos de código objeto se necesitan para generar el archivo binario del sistema operativo que se necesita en un momento dado y para cada uno comprueba si alguno de los archivos de los que depende (el código y los encabezados) se ha modificado después de la última vez que se creó el archivo de código objeto. De ser así, ese archivo de código objeto se tiene que volver a compilar. Cuando *make* ha determinado qué archivos *.c* se tienen que volver a compilar, invoca al compilador de C para que los recompile, con lo cual se reduce el número de compilaciones al mínimo. En los proyectos extensos, la creación del *Makefile* está propensa a errores, por lo que hay herramientas que lo hacen de manera automática.

Una vez que están listos todos los archivos *.o*, se pasan a un programa conocido como **enlazador** para combinarlos en un solo archivo binario ejecutable. Cualquier función de biblioteca que sea llamada también se incluye en este punto, se resuelven las referencias dentro de las funciones y se reasignan las direcciones de la máquina según sea necesario. Cuando el enlazador termina, el resultado es un programa ejecutable, que por tradición se llama *a.out* en los sistemas UNIX. Los diversos componentes de este proceso se ilustran en la figura 1-30 para un programa con tres archivos de C y dos archivos de encabezado. Aunque hemos hablado aquí sobre el desarrollo de sistemas operativos, todo esto se aplica al desarrollo de cualquier programa extenso.

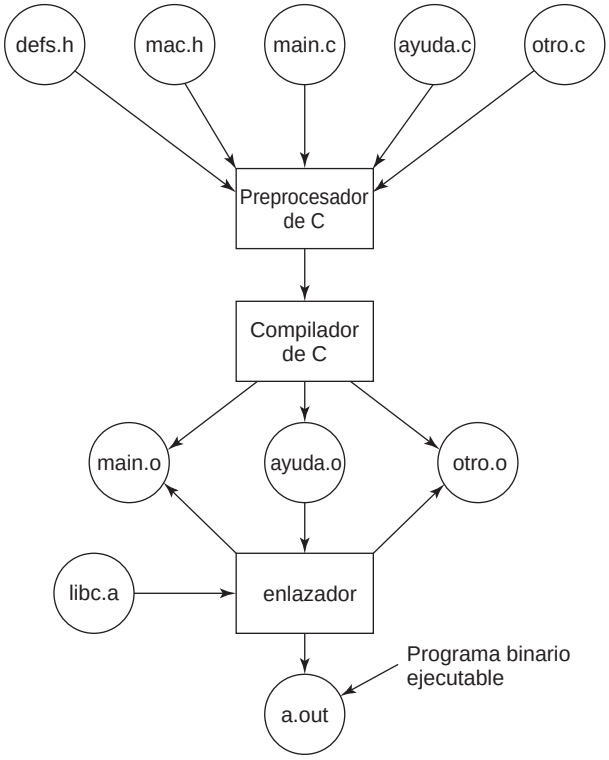


Figura 1-30. El proceso de compilar archivos de C y de encabezado para crear un archivo ejecutable.

1.8.4 El modelo del tiempo de ejecución

Una vez que se ha enlazado el archivo binario del sistema operativo, la computadora puede reiniciarse con el nuevo sistema operativo. Una vez en ejecución, puede cargar piezas en forma dinámica que no se hayan incluido de manera estática en el binario, como los drivers de dispositivos y los sistemas de archivos. En tiempo de ejecución, el sistema operativo puede consistir de varios segmentos, para el texto (el código del programa), los datos y la pila. Por lo general, el segmento de texto es inmutable y no cambia durante la ejecución. El segmento de datos empieza con cierto tamaño y se inicializa con ciertos valores, pero puede cambiar y crecer según lo necesite. Al principio la pila está vacía, pero crece y se reduce a medida que se hacen llamadas a funciones y se regresa de ellas. A menudo el segmento de texto se coloca cerca de la parte final de la memoria, el segmento de datos justo encima de éste, con la habilidad de crecer hacia arriba y el segmento de pila en una dirección virtual superior, con la habilidad de crecer hacia abajo, pero los distintos sistemas trabajan de manera diferente.

En todos los casos, el código del sistema operativo es ejecutado directamente por el hardware, sin intérprete y sin compilación justo-a-tiempo, como se da el caso con Java.

1.9 INVESTIGACIÓN ACERCA DE LOS SISTEMAS OPERATIVOS

La ciencia computacional es un campo que avanza con rapidez y es difícil predecir hacia dónde va. Los investigadores en universidades y los laboratorios de investigación industrial están desarrollando constantemente nuevas ideas, algunas de las cuales no van a ningún lado, pero otras se convierten en la piedra angular de futuros productos y tienen un impacto masivo en la industria y en los usuarios. Saber qué ideas tendrán éxito es más fácil en retrospección que en tiempo real. Separar el trigo de la paja es en especial difícil, debido a que a menudo se requieren de 20 a 30 años para que una idea haga impacto.

Por ejemplo, cuando el presidente Eisenhower estableció la Agencia de Proyectos Avanzados de Investigación (ARPA) del Departamento de Defensa en 1958, trataba de evitar que el Ejército predominara sobre la Marina y la Fuerza Aérea en relación con el presupuesto de investigación. No estaba tratando de inventar Internet. Pero una de las cosas que hizo ARPA fue patrocinar cierta investigación universitaria sobre el entonces oscuro tema de la conmutación de paquetes, lo cual condujo a la primera red experimental de conmutación de paquetes, ARPANET. Entró en funcionamiento en 1969. Poco tiempo después, otras redes de investigación patrocinadas por ARPA estaban conectadas a ARPANET,y nació Internet, que en ese entonces era utilizada felizmente por los investigadores académicos para enviarse correo electrónico unos con otros durante 20 años. A principios de la década de 1990, Tim Berners-Lee inventó la World Wide Web en el laboratorio de investigación CERN en Ginebra y Marc Andreesen escribió el código de un navegador gráfico para esta red en la Universidad de Illinois. De repente, Internet estaba llena de adolescentes conversando entre sí. Probablemente el presidente Eisenhower esté revolcándose en su tumba.

La investigación en los sistemas operativos también ha producido cambios dramáticos en los sistemas prácticos. Como vimos antes, los primeros sistemas computacionales comerciales eran todos sistemas de procesamiento por lotes, hasta que el M.I.T. inventó el tiempo compartido interactivo a principios de la década de 1960. Todas las computadoras eran basadas en texto, hasta que Doug Engelbart inventó el ratón y la interfaz gráfica de usuario en el Stanford Research Institute a finales de la década de 1960. ¿Quién sabe lo que vendrá a continuación?

En esta sección y en otras secciones equivalentes en el resto de este libro, daremos un breve vistazo a una parte de la investigación sobre los sistemas operativos que se ha llevado a cabo durante los últimos 5 a 10 años, sólo para tener una idea de lo que podría haber en el horizonte. En definitiva, esta introducción no es exhaustiva y se basa en gran parte en los artículos que se han publicado en los principales diarios de investigación y conferencias, debido a que estas ideas han sobrevivido por lo menos a un proceso de revisión personal riguroso para poder publicarse. La mayoría de los artículos citados en las secciones de investigación fueron publicados por la ACM, la IEEE Computer Society o USENIX, que están disponibles por Internet para los miembros (estudiantes) de estas organizaciones. Para obtener más información acerca de estas organizaciones y sus bibliotecas digitales, consulte los siguientes sitios Web:

ACM	http://www.acm.org
IEEE Computer Society	http://www.computer.org
USENIX	http://www.usenix.org

Casi todos los investigadores de sistemas operativos consideran que los sistemas operativos actuales son masivos, inflexibles, no muy confiables, inseguros y están cargados de errores, evidentemente algunos más que otros (*no divulgamos los nombres para no dañar a los aludidos*). En consecuencia, hay mucha investigación acerca de cómo construir mejores sistemas operativos. En fechas recientes se ha publicado trabajo acerca de los nuevos sistemas operativos (Krieger y colaboradores, 2006), la estructura del sistema operativo (Fassino y colaboradores, 2002), la rectitud del sistema operativo (Elphinstone y colaboradores, 2007; Kumar y Li, 2002; y Yang y colaboradores, 2006), la confiabilidad del sistema operativo (Swift y colaboradores, 2006; y LeVasseur y colaboradores, 2004), las máquinas virtuales (Barham y colaboradores, 2003; Garfinkel y colaboradores, 2003; King y colaboradores, 2003; y Whitaker y colaboradores, 2002), los virus y gusanos (Costa y colaboradores, 2005; Portokalidis y colaboradores, 2006; Tucek y colaboradores, 2007; y Vrable y colaboradores, 2005), los errores y la depuración (Chou y colaboradores, 2001; y King y colaboradores, 2005), el hiperhilamiento y el multihilamiento (Fedorova, 2005; y Bulpin y Pratt, 2005), y el comportamiento de los usuarios (Yu y colaboradores, 2006), entre muchos otros temas.

1.10 DESCRIPCIÓN GENERAL SOBRE EL RESTO DE ESTE LIBRO

Ahora hemos completado nuestra introducción y un breve visión del sistema operativo. Es tiempo de entrar en detalles. Como dijimos antes, desde el punto de vista del programador, el propósito principal de un sistema operativo es proveer ciertas abstracciones clave, siendo las más importantes los procesos y hilos, los espacios de direcciones y los archivos. De manera acorde, los siguientes tres capítulos están dedicados a estos temas cruciales.

El capítulo 2 trata acerca de los procesos y hilos. Describe sus propiedades y la manera en que se comunican entre sí. También proporciona varios ejemplos detallados acerca de la forma en que funciona la comunicación entre procesos y cómo se pueden evitar algunos obstáculos.

En el capítulo 3 estudiaremos los espacios de direcciones y su administración adjunta de memoria con detalle. Examinaremos el importante tema de la memoria virtual, junto con los conceptos estrechamente relacionados, como la paginación y la segmentación.

Después, en el capítulo 4 llegaremos al importantísimo tema de los sistemas de archivos. Hasta cierto punto, lo que el usuario ve es en gran parte el sistema de archivos. Analizaremos la interfaz del sistema de archivos y su implementación.

En el capítulo 5 se cubre el tema de las operaciones de entrada/salida. Hablaremos sobre los conceptos de independencia de dispositivos y dependencia de dispositivos. Utilizaremos como ejemplos varios dispositivos importantes, como los discos, teclados y pantallas.

El capítulo 6 trata acerca de los interbloqueos. Anteriormente en este capítulo mostramos en forma breve qué son los interbloqueos, pero hay mucho más por decir. Hablaremos también sobre las formas de prevenirlos o evitarlos.

En este punto habremos completado nuestro estudio acerca de los principios básicos de los sistemas operativos con una sola CPU. Sin embargo, hay más por decir, en especial acerca de los temas avanzados. En el capítulo 7 examinaremos los sistemas de multimedia, que tienen varias

propiedades y requerimientos distintos de los sistemas operativos convencionales. Entre otros elementos, la planificación de tareas y el sistema de archivos se ven afectados por la naturaleza de la multimedia. Otro tema avanzado es el de los sistemas con múltiples procesadores, incluyendo los multiprocesadores, las computadoras en paralelo y los sistemas distribuidos. Cubriremos estos temas en el capítulo 8.

Un tema de enorme importancia es la seguridad de los sistemas operativos, que se cubre en el capítulo 9. Entre los temas descritos en este capítulo están las amenazas (es decir, los virus y gusanos), los mecanismos de protección y los modelos de seguridad.

A continuación veremos algunos ejemplos prácticos de sistemas operativos reales. Éstos son Linux (capítulo 10), Windows Vista (capítulo 11) y Symbian (capítulo 12). El libro concluye con ciertos consejos y pensamientos acerca del diseño de sistemas operativos en el capítulo 13.

1.11 UNIDADES MÉTRICAS

Para evitar cualquier confusión, vale la pena declarar en forma explícita que en este libro, como en la ciencia computacional en general, se utilizan unidades métricas en vez de las tradicionales unidades del sistema inglés. Los principales prefijos métricos se listan en la figura 1-31. Por lo general, estos prefijos se abrevian con sus primeras letras y las unidades mayores de 1 están en mayúsculas. Así, una base de datos de 1 TB ocupa 10^{12} bytes de almacenamiento, y un reloj de 100 pseg (o 100 ps) emite un pulso cada 10^{-10} segundos. Como mili y micro empiezan con la letra “m”, se tuvo que hacer una elección. Por lo general, “m” es para mili y “μ” (la letra griega mu) es para micro.

Exp.	Explícito	Prefijo	Exp.	Explícito	Prefijo
10^{-3}	0.001	mili	10^{3}	1,000	Kilo
10^{-6}	0.000001	micro	10^{6}	1,000,000	Mega
10^{-9}	0.000000001	nano	10^{9}	1,000,000,000	Giga
10^{-12}	0.000000000001	pico	10^{12}	1,000,000,000,000	Tera
10^{-15}	0.000000000000001	femto	10^{15}	1,000,000,000,000,000	Peta
10^{-18}	0.000000000000000001	atto	10^{18}	1,000,000,000,000,000,000	Exa
10^{-21}	0.000000000000000000001	zepto	10^{21}	1,000,000,000,000,000,000,000	Zetta
10^{-24}	0.000000000000000000000001	yocto	10^{24}	1,000,000,000,000,000,000,000,000	Yotta

Figura 1-31. Los principales prefijos métricos.

También vale la pena recalcar que para medir los tamaños de las memorias, en la práctica común de la industria, las unidades tienen significados ligeramente diferentes. Ahí, Kilo significa 2^{10} (1024) en vez de 10^3 (1000), debido a que las memorias siempre utilizan una potencia de dos. Por ende, una memoria de 1 KB contiene 1024 bytes, no 1000 bytes. De manera similar, una memoria de 1 MB contiene 2^{20} (1,048,576) bytes y una memoria de 1 GB contiene 2^{30} (1,073,741,824) bytes. Sin embargo, una línea de comunicación de 1 Kbps transmite 1000 bits por segundo y una LAN de 10 Mbps opera a 10,000,000 bits/seg, debido a que estas velocidades no son potencias de dos. Por desgracia, muchas personas tienden a mezclar estos dos sistemas, en especial para los tamaños

de los discos. Para evitar ambigüedades, en este libro utilizaremos los símbolos KB, MB y GB para 2^{10}, 2^{20} y 2^{30} bytes, respectivamente y los símbolos Kbps, Mbps y Gbps para 10^3, 10^6 y 10^9 bits/seg, respectivamente.

1.12 RESUMEN

Los sistemas operativos se pueden ver desde dos puntos de vista: como administradores de recursos y como máquinas extendidas. En el punto de vista correspondiente al administrador de recursos, la función del sistema operativo es administrar las distintas partes del sistema en forma eficiente. En el punto de vista correspondiente a la máquina extendida, la función del sistema operativo es proveer a los usuarios abstracciones que sean más convenientes de usar que la máquina actual. Estas abstracciones incluyen los procesos, espacios de direcciones y archivos.

Los sistemas operativos tienen una larga historia, empezando desde los días en que reemplazaron al operador, hasta los sistemas modernos de multiprogramación. Entre los puntos importantes tenemos a los primeros sistemas de procesamiento por lotes, los sistemas de multiprogramación y los sistemas de computadora personal.

Como los sistemas operativos interactúan de cerca con el hardware, es útil tener cierto conocimiento del hardware de computadora parea comprenderlos. Las computadoras están compuestas de procesadores, memorias y dispositivos de E/S. Estas partes se conectan mediante buses.

Los conceptos básicos en los que se basan todos los sistemas operativos son los procesos, la administración de memoria, la administración de E/S, el sistema de archivos y la seguridad. Cada uno de estos conceptos se tratará con detalle en un capítulo posterior.

El corazón de cualquier sistema operativo es el conjunto de llamadas al sistema que puede manejar. Estas llamadas indican lo que realmente hace el sistema operativo. Para UNIX, hemos visto cuatro grupos de llamadas al sistema. El primer grupo de llamadas se relaciona con la creación y terminación de procesos. El segundo grupo es para leer y escribir en archivos. El tercer grupo es para administrar directorios. El cuarto grupo contiene una miscelánea de llamadas.

Los sistemas operativos se pueden estructurar en varias formas. Las más comunes son como un sistema monolítico, una jerarquía de capas, microkernel, cliente-servidor, máquina virtual o exokernel.

PROBLEMAS

1. ¿Qué es la multiprogramación?

2. ¿Qué es spooling? ¿Cree usted que las computadoras personales avanzadas tendrán spooling como característica estándar en el futuro?

3. En las primeras computadoras, cada byte de datos leídos o escritos se manejaba mediante la CPU (es decir, no había DMA). ¿Qué implicaciones tiene esto para la multiprogramación?

4. La idea de una familia de computadoras fue introducida en la década de 1960 con las mainframes IBM System/360. ¿Está muerta ahora esta idea o sigue en pie?

5. Una razón por la cual las GUI no se adoptaron con rapidez en un principio fue el costo del hardware necesario para darles soporte. ¿Cuánta RAM de video se necesita para dar soporte a una pantalla de texto monocromático de 25 líneas x 80 caracteres? ¿Cuánta se necesita para un mapa de bits de 1024 $\times$ 768 píxeles y colores 24 bits? ¿Cuál fue el costo de esta RAM con precios de 1980 (5 dólares/KB)? ¿Cuánto vale ahora?

6. Hay varias metas de diseño a la hora de crear un sistema operativo, por ejemplo: la utilización de recursos, puntualidad, que sea robusto, etcétera. De un ejemplo de dos metas de diseño que puedan contradecirse entre sí.

7. ¿Cuál de las siguientes instrucciones debe permitirse sólo en modo kernel?

a) Deshabilitar todas las interrupciones.
b) Leer el reloj de la hora del día.
c) Establecer el reloj de la hora del día.
d) Cambiar el mapa de memoria.

8. Considere un sistema con dos CPUs y que cada CPU tiene dos hilos (hiperhilamiento). Suponga que se inician tres programas *P0*, *P1* y *P2* con tiempos de ejecución de 5, 10 y 20 mseg, respectivamente. ¿Cuánto se tardará en completar la ejecución de estos programas? Suponga que los tres programas están 100% ligados a la CPU, que no se bloquean durante la ejecución y no cambian de CPU una vez que se les asigna.

9. Una computadora tiene una canalización con cuatro etapas. Cada etapa requiere el mismo tiempo para hacer su trabajo, a saber, 1 nseg. ¿Cuántas instrucciones por segundo puede ejecutar esta máquina?

10. Considere un sistema de cómputo con memoria caché, memoria principal (RAM) y disco, y que el sistema operativo utiliza memoria virtual. Se requieren 2 nseg para acceder a una palabra desde la caché, 10 nseg para acceder a una palabra desde la RAM y 10 ms para acceder a una palabra desde el disco. Si la proporción de aciertos de caché es de 95% y la proporción de aciertos de memoria (después de un fallo de caché) es de 99%, ¿cuál es el tiempo promedio para acceder a una palabra?

11. Un revisor alerta observa un error de ortografía consistente en el manuscrito del libro de texto de sistemas operativos que está a punto de ser impreso. El libro tiene cerca de 700 páginas, cada una con 50 líneas de 80 caracteres. ¿Cuánto tiempo se requerirá para digitalizar en forma electrónica el texto, para el caso en que la copia maestra se encuentre en cada uno de los niveles de memoria de la figura 1-9? Para los métodos de almacenamiento interno, considere que el tiempo de acceso dado es por carácter, para los discos suponga que el tiempo es por bloque de 1024 caracteres y para la cinta suponga que el tiempo dado es para el inicio de los datos, con un acceso posterior a la misma velocidad que el acceso al disco.

12. Cuando un programa de usuario realiza una llamada al sistema para leer o escribir en un archivo en disco, proporciona una indicación de qué archivo desea, un apuntador al búfer de datos y la cuenta. Después, el control se transfiere al sistema operativo, el cual llama al driver apropiado. Suponga que el driver inicia el disco y termina hasta que ocurre una interrupción. En el caso de leer del disco, es obvio que el procedimiento que hizo la llamada tiene que ser bloqueado (debido a que no hay datos para leer). ¿Qué hay sobre el caso de escribir en el disco? ¿Necesita ser bloqueado el procedimiento llamador, para esperar a que se complete la transferencia del disco?

13. ¿Qué es una instrucción de trap? Explique su uso en los sistemas operativos.

14. ¿Cuál es la diferencia clave entre un trap y una interrupción?

15. ¿Por qué se necesita la tabla de procesos en un sistema de tiempo compartido? ¿Se necesita también en los sistemas de computadora personal en los que sólo existe un proceso, y ese proceso ocupa toda la máquina hasta que termina?

16. ¿Existe alguna razón por la que sería conveniente montar un sistema de archivos en un directorio no vacío? De ser así, ¿cuál es?

17. ¿Cuál es el propósito de una llamada al sistema en un sistema operativo?

18. Para cada una de las siguientes llamadas al sistema, proporcione una condición que haga que falle: `fork`, `exec` y `unlink`.

19. ¿Podría la llamada

```
cuenta = write(fd, bufer, nbytes);
```

devolver algún valor en *cuenta* distinto de *nbytes*? Si es así, ¿por qué?

20. Un archivo cuyo descriptor es *fd* contiene la siguiente secuencia de bytes: 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5. Se realizan las siguientes llamadas al sistema:

```
lseek(fd, 3, SEEK_SET);
read(fd, &bufer, 4);
```

en donde la llamada `lseek` realiza una búsqueda en el byte 3 del archivo. ¿Qué contiene *bufer* después de completar la operación de lectura?

21. Suponga que un archivo de 10 MB se almacena en un disco, en la misma pista (pista #: 50) en sectores consecutivos. El brazo del disco se encuentra actualmente situado en la pista número 100. ¿Cuánto tardará en recuperar este archivo del disco? Suponga que para desplazar el brazo de un cilindro al siguiente se requiere aproximadamente 1 ms y se requieren aproximadamente 5 ms para que el sector en el que está almacenado el inicio del archivo gire bajo la cabeza. Suponga además que la lectura ocurre a una velocidad de 100 MB/s.

22. ¿Cuál es la diferencia esencial entre un archivo especial de bloque y un archivo especial de carácter?

23. En el ejemplo que se da en la figura 1-17, el procedimiento de biblioteca se llama *read* y la misma llamada al sistema se llama `read`. ¿Es esencial que ambos tengan el mismo nombre? Si no es así, ¿cuál es más importante?

24. El modelo cliente-servidor es popular en los sistemas distribuidos. ¿Puede utilizarse también en un sistema de una sola computadora?

25. Para un programador, una llamada al sistema se ve igual que cualquier otra llamada a un procedimiento de biblioteca. ¿Es importante que un programador sepa cuáles procedimientos de biblioteca resultan en llamadas al sistema? ¿Bajo qué circunstancias y por qué?

26. La figura 1-23 muestra que varias llamadas al sistema de UNIX no tienen equivalentes en la API Win32. Para cada una de las llamadas que se listan y no tienen un equivalente en Win32, ¿cuáles son las consecuencias de que un programador convierta un programa de UNIX para que se ejecute en Windows?

27. Un sistema operativo portátil se puede portar de la arquitectura de un sistema a otro, sin ninguna modificación. Explique por qué no es factible construir un sistema operativo que sea completamente portátil. Describa dos capas de alto nivel que tendrá al diseñar un sistema operativo que sea altamente portátil.

28. Explique cómo la separación de la directiva y el mecanismo ayuda a construir sistemas operativos basados en microkernel.

29. He aquí algunas preguntas para practicar las conversiones de unidades:

(a) ¿A cuántos segundos equivale un microaño?
(b) A los micrómetros se les conoce comúnmente como micrones. ¿Qué tan largo es un gigamicron?
(c) ¿Cuántos bytes hay en una memoria de 1 TB?
(d) La masa de la Tierra es de 6000 yottagramos. ¿Cuánto es eso en kilogramos?

30. Escriba un shell que sea similar a la figura 1-19, pero que contenga suficiente código como para que pueda funcionar y lo pueda probar. También podría agregar algunas características como la redirección de la entrada y la salida, los canales y los trabajos en segundo plano.

31. Si tiene un sistema personal parecido a UNIX (Linux, MINIX, FreeBSD, etcétera) disponible que pueda hacer fallar con seguridad y reiniciar, escriba una secuencia de comandos de shell que intente crear un número ilimitado de procesos hijos y observe lo que ocurre. Antes de ejecutar el experimento, escriba `sync` en el shell para vaciar los búferes del sistema de archivos al disco y evitar arruinar el sistema de archivos. **Nota:** No intente esto en un sistema compartido sin obtener primero permiso del administrador del sistema. Las consecuencias serán obvias de inmediato, por lo que es probable que lo atrapen y sancionen.

32. Examine y trate de interpretar el contenido de un directorio tipo UNIX o Windows con una herramienta como el programa *od* de UNIX o el programa *DEBUG* de MS-DOS. *Sugerencia:* La forma en que haga esto dependerá de lo que permita el SO. Un truco que puede funcionar es crear un directorio en un disco flexible con un sistema operativo y después leer los datos puros del disco usando un sistema operativo distinto que permita dicho acceso.

3

ADMINISTRACIÓN DE MEMORIA

La memoria principal (RAM) es un importante recurso que debe administrarse con cuidado. Aunque actualmente una computadora doméstica promedio tiene 10,000 veces más memoria que la IBM 7094, la computadora más grande en el mundo a principios de la década de 1960, los programas están creciendo con más rapidez que las memorias. Parafraseando la ley de Parkinson diría que "los programas se expanden para llenar la memoria disponible para contenerlos". En este capítulo estudiaremos la forma en que los sistemas operativos crean abstracciones de la memoria y cómo las administran.

Lo que todo programador quisiera es una memoria privada, de tamaño y rapidez infinitas, que también sea no volátil, es decir, que no pierda su contenido cuando se desconecta de la potencia eléctrica. Y ya que estamos en ello, ¿por qué no hacerla barata también? Por desgracia, la tecnología no proporciona tales memorias en estos momentos. Tal vez usted descubra cómo hacerlo.

¿Cuál es la segunda opción? A través de los años se ha elaborado el concepto de **jerarquía de memoria**, de acuerdo con el cual, las computadoras tienen unos cuantos megabytes de memoria caché, muy rápida, costosa y volátil, unos cuantos gigabytes de memoria principal, de mediana velocidad, a precio mediano y volátil, unos cuantos terabytes de almacenamiento en disco lento, económico y no volátil, y el almacenamiento removible, como los DVDs y las memorias USB. El trabajo del sistema operativo es abstraer esta jerarquía en un modelo útil y después administrarla.

La parte del sistema operativo que administra (parte de) la jerarquía de memoria se conoce como **administrador de memoria**. Su trabajo es administrar la memoria con eficiencia: llevar el registro de cuáles partes de la memoria están en uso, asignar memoria a los procesos cuando la necesiten y desasignarla cuando terminen.

En este capítulo investigaremos varios esquemas distintos de administración de memoria, que varían desde muy simples hasta muy sofisticados. Como generalmente el hardware es el que se encarga de administrar el nivel más bajo de memoria caché, en este capítulo nos concentraremos en el modelo del programador de la memoria principal y en cómo se puede administrar bien. Las abstracciones y la administración del almacenamiento permanente, el disco, son el tema del siguiente capítulo. Analizaremos primero los esquemas más simples posibles y después progresaremos de manera gradual hacia esquemas cada vez más elaborados.

3.1 SIN ABSTRACCIÓN DE MEMORIA

La abstracción más simple de memoria es ninguna abstracción. Las primeras computadoras mainframe (antes de 1960), las primeras minicomputadoras (antes de 1970) y las primeras computadoras personales (antes de 1980) no tenían abstracción de memoria. Cada programa veía simplemente la memoria física. Cuando un programa ejecutaba una instrucción como

```
MOV REGISTRO1, 1000
```

la computadora sólo movía el contenido de la ubicación de memoria física 1000 a *REGISTRO1*. Así, el modelo de programación que se presentaba al programador era simplemente la memoria física, un conjunto de direcciones desde 0 hasta cierto valor máximo, en donde cada dirección correspondía a una celda que contenía cierto número de bits, comúnmente ocho.

Bajo estas condiciones, no era posible tener dos programas ejecutándose en memoria al mismo tiempo. Si el primer programa escribía un nuevo valor en, por ejemplo, la ubicación 2000, esto borraría cualquier valor que el segundo programa estuviera almacenando ahí. Ambos programas fallarían de inmediato.

Incluso cuando el modelo de memoria consiste en sólo la memoria física hay varias opciones posibles. En la figura 3-1 se muestran tres variaciones. El sistema operativo puede estar en la parte inferior de la memoria en la RAM (*Random Access Memory*, Memoria de acceso aleatorio), como se muestra en la figura 3-1(a), puede estar en la ROM (*Read Only Memory*, Memoria de sólo lectura) en la parte superior de la memoria, como se muestra en la figura 3-1(b), o los controladores de dispositivos pueden estar en la parte superior de la memoria en una ROM y el resto del sistema en RAM más abajo, como se muestra en la figura 3-1(c). El primer modelo se utilizó antes en las mainframes y minicomputadoras, pero actualmente casi no se emplea. El segundo modelo se utiliza en algunas computadoras de bolsillo y sistemas integrados. El tercer modelo fue utilizado por las primeras computadoras personales (por ejemplo, las que ejecutaban MS-DOS), donde la porción del sistema en la ROM se conoce como **BIOS** (*Basic Input Output System*, Sistema básico de entrada y salida). Los modelos (a) y (c) tienen la desventaja de que un error en el programa de usuario puede borrar el sistema operativo, posiblemente con resultados desastrosos (la información del disco podría quedar ininteligible).

Cuando el sistema se organiza de esta forma, por lo general se puede ejecutar sólo un proceso a la vez. Tan pronto como el usuario teclea un comando, el sistema operativo copia el programa solicitado del disco a la memoria y lo ejecuta. Cuando termina el proceso, el sistema operativo mues-

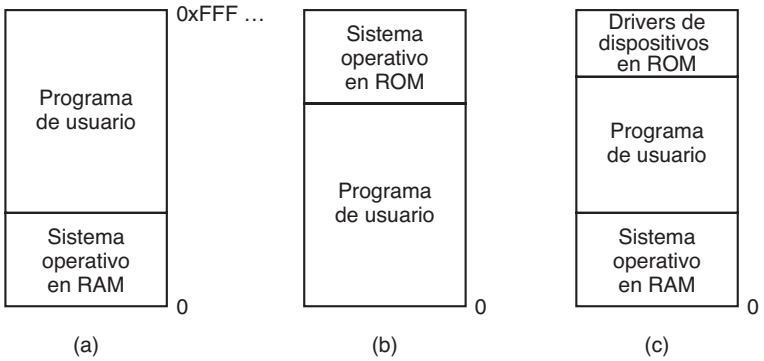


Figura 3-1. Tres formas simples de organizar la memoria con un sistema operativo y un proceso de usuario. También existen otras posibilidades.

tra un carácter indicador de comando y espera un nuevo comando. Cuando recibe el comando, carga un nuevo programa en memoria, sobrescribiendo el primero.

Una forma de obtener cierto grado de paralelismo en un sistema, sin abstracción de memoria, es programar con múltiples hilos. Como se supone que todos los hilos en un proceso ven la misma imagen de memoria, el hecho de que se vean obligados a hacerlo no es un problema. Aunque esta idea funciona, es de uso limitado ya que lo que las personas desean a menudo es que los programas *no relacionados* se ejecuten al mismo tiempo, algo que la abstracción de los hilos no provee. Además, es muy poco probable que un sistema tan primitivo como para no proporcionar una abstracción de memoria proporcione una abstracción de hilos.

Ejecución de múltiple programas sin una abstracción de memoria

No obstante, aun sin abstracción de memoria es posible ejecutar varios programas al mismo tiempo. Lo que el sistema operativo debe hacer es guardar todo el contenido de la memoria en un archivo en disco, para después traer y ejecutar el siguiente programa. Mientras sólo haya un programa a la vez en la memoria no hay conflictos. Este concepto, el intercambio, se analiza a continuación.

Con la adición de cierto hardware especial es posible ejecutar múltiples programas concurrentemente, aun sin intercambio. Los primeros modelos de la IBM 360 resolvieron el problema de la siguiente manera: la memoria estaba dividida en bloques de 2 KB y a cada uno se le asignaba una llave de protección de 4 bits, guardada en registros especiales dentro de la CPU. Un equipo con una memoria de 1 MB sólo necesitaba 512 de estos registros de 4 bits para totalizar 256 bytes de almacenamiento de la llave. El registro **PSW** (*Program Status Word*, Palabra de estado del programa) también contenía una llave de 4 bits. El hardware de la 360 controlaba mediante un trap cualquier intento por parte de un proceso en ejecución de acceder a la memoria con un código de protección distinto del de la llave del PSW. Como sólo el sistema operativo podía modificar las llaves de protección, los procesos de usuario fueron controlados para que no interfirieran unos con otros, ni con el mismo sistema operativo.

Sin embargo, esta solución tenía una gran desventaja, que se ilustra en la figura 3-2. Como muestran las figuras 3-2(a) y (b), se tienen dos programas, cada uno con un tamaño de 16 KB. El primero está sombreado para indicar que tiene una llave de memoria diferente a la del segundo y empieza saltando a la dirección 24, que contiene una instrucción `MOV`; el segundo, saltando a la dirección 28, que contiene una instrucción `CMP`. Las instrucciones que no son relevantes para este análisis no se muestran. Cuando los dos programas se cargan consecutivamente en la memoria, empezando en la dirección 0, tenemos la situación de la figura 3-2(c). Para este ejemplo, suponemos que el sistema operativo está en la parte alta de la memoria y no se muestra.

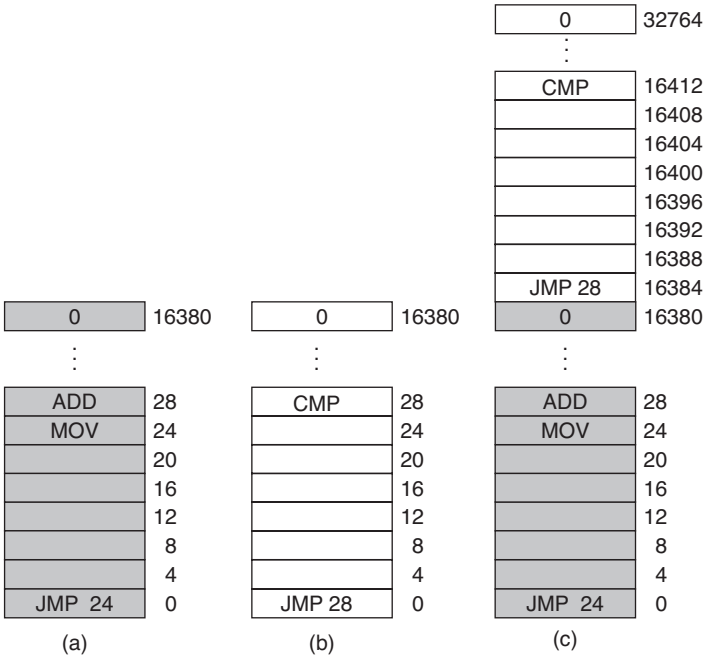


Figura 3-2. Ilustración del problema de reubicación. (a) Un programa de 16 KB. (b) Otro programa de 16 KB. (c) Los dos programas cargados consecutivamente en la memoria.

Una vez que los programas se cargan se pueden ejecutar. Como tienen distintas llaves de memoria, ninguno de los dos puede dañar al otro. Pero el problema es de una naturaleza distinta. Cuando se inicia el primer programa, ejecuta la instrucción `JMP 24`, que salta a la instrucción, como se espera. Este programa funciona de manera normal.

Sin embargo, después de que el primer programa se ha ejecutado el tiempo suficiente, el sistema operativo tal vez decida ejecutar el segundo programa, que se carga encima del primero, en la dirección 16,384. La primera instrucción ejecutada es `JMP 28`, que salta a la instrucción `ADD` en el primer programa, y no a la instrucción `CMP` a la que se supone debe saltar. Es muy probable que el programa falle en menos de 1 segundo.

El problema central aquí es que los dos programas hacen referencia a la memoria física absoluta. Eso, definitivamente, no es lo que queremos; deseamos que cada programa haga referencia a un conjunto privado de direcciones locales para él. En breve le mostraremos cómo se logra esto. Lo que la IBM 360 hacía como solución para salir del paso era modificar el segundo programa al instante, a medida que se cargaba en la memoria, usando una técnica conocida como **reubicación estática**. Esta técnica funcionaba así: cuando se cargaba un programa en la dirección 16,384, se sumaba el valor constante 16,384 a todas las direcciones del programa durante el proceso de carga. Aunque este mecanismo funciona si se lleva a cabo en la forma correcta, no es una solución muy general y reduce la velocidad de la carga. Lo que es más, se requiere información adicional en todos los programas ejecutables para indicar cuáles palabras contienen direcciones (reubicables) y cuáles no. Después de todo, el "28" en la figura 3-2(b) tiene que reubicarse, pero una instrucción como

```
MOV REGISTRO1, 28
```

que mueve el número 28 a *REGISTRO1* no se debe reubicar. El cargador necesita cierta forma de saber qué es una dirección y qué es una constante.

Por último, como recalcamos en el capítulo 1, la historia tiende a repetirse en el mundo de las computadoras. Mientras que el direccionamiento directo de la memoria física está muy lejos de poder aplicarse en las mainframes, minicomputadoras, computadoras de escritorio y notebooks, la falta de una abstracción de memoria sigue siendo común en los sistemas integrados y de tarjeta inteligente. En la actualidad, los dispositivos como las radios, las lavadoras y los hornos de microondas están llenos de software (en ROM), y en la mayoría de los casos el software direcciona memoria absoluta. Esto funciona debido a que todos los programas se conocen de antemano, y los usuarios no tienen la libertad de ejecutar su propio software en su tostador.

Mientras que los sistemas integrados de alta tecnología (como los teléfonos celulares) tienen sistemas operativos elaborados, los más simples no. En algunos casos hay un sistema operativo, pero es sólo una biblioteca ligada con el programa de aplicación y proporciona llamadas al sistema para realizar operaciones de E/S y otras tareas comunes. El popular sistema operativo **e-cos** es un ejemplo común de un sistema operativo como biblioteca.

3.2 UNA ABSTRACCIÓN DE MEMORIA: ESPACIOS DE DIRECCIONES

Con todo, exponer la memoria física a los procesos tiene varias desventajas. En primer lugar, si los programas de usuario pueden direccionar cada byte de memoria, pueden estropear el sistema operativo con facilidad, ya sea intencional o accidentalmente, con lo cual el sistema se detendría en forma súbita (a menos que haya hardware especial como el esquema de bloqueo y llaves de la IBM 360). Este problema existe aun cuando sólo haya un programa de usuario (aplicación) en ejecución. En segundo lugar, con este modelo es difícil tener varios programas en ejecución a la vez (tomando turnos, si sólo hay una CPU). En las computadoras personales es común tener varios programas abiertos a la vez (un procesador de palabras, un programa de correo electrónico y un navegador Web, donde uno de ellos tiene el enfoque actual, pero los demás se reactivan con el clic de un ratón). Como esta situación es difícil de lograr cuando no hay una abstracción de la memoria física, se tuvo que hacer algo.

3.2.1 La noción de un espacio de direcciones

Hay que resolver dos problemas para permitir que haya varias aplicaciones en memoria al mismo tiempo sin que interfieran entre sí: protección y reubicación. Ya analizamos una solución primitiva al primer problema en la IBM 360: etiquetar trozos de memoria con una llave de protección y comparar la llave del proceso en ejecución con la de cada palabra de memoria obtenida por la CPU. Sin embargo, este método por sí solo no resuelve el segundo problema, aunque se puede resolver mediante la reubicación de los programas al momento de cargarlos, pero ésta es una solución lenta y complicada.

Una mejor solución es inventar una nueva abstracción para la memoria: el espacio de direcciones. Así como el concepto del proceso crea un tipo de CPU abstracta para ejecutar programas, el espacio de direcciones crea un tipo de memoria abstracta para que los programas vivan ahí. Un **espacio de direcciones** (*address space*) es el conjunto de direcciones que puede utilizar un proceso para direccionar la memoria. Cada proceso tiene su propio espacio de direcciones, independiente de los que pertenecen a otros procesos (excepto en ciertas circunstancias especiales en donde los procesos desean compartir sus espacios de direcciones).

El concepto de un espacio de direcciones es muy general y ocurre en muchos contextos. Considere los números telefónicos. En los Estados Unidos y en muchos otros países, un número de teléfono local es comúnmente un número de 7 dígitos. En consecuencia, el espacio de direcciones para los números telefónicos varía desde 0,000,000 hasta 9,999,999, aunque algunos números, como los que empiezan con 000, no se utilizan. Con el crecimiento de los teléfonos celulares, módems y máquinas de fax, este espacio se está volviendo demasiado pequeño, en cuyo caso habrá qué utilizar más dígitos. El espacio de direcciones para los puertos de E/S en el Pentium varía desde 0 hasta 16383. Las direcciones IPv4 son números de 32 bits, por lo que su espacio de direcciones varía desde 0 hasta $2^{32} - 1$ (de nuevo, con ciertos números reservados).

Los espacios de direcciones no tienen que ser numéricos. El conjunto de dominios *.com* de Internet es también un espacio de direcciones. Este espacio de direcciones consiste de todas las cadenas de longitud de 2 a 63 caracteres que se puedan formar utilizando letras, números y guiones cortos, seguidas de *.com*. Para estos momentos usted debe de hacerse hecho ya la idea. Es bastante simple.

Algo un poco más difícil es proporcionar a cada programa su propio espacio de direcciones, de manera que la dirección 28 en un programa indique una ubicación física distinta de la dirección 28 en otro programa. A continuación analizaremos una forma simple que solía ser común, pero ha caído en desuso debido a la habilidad de poner esquemas mucho más complicados (y mejores) en los chips de CPU modernos.

Registros base y límite

La solución sencilla utiliza una versión muy simple de la **reubicación dinámica**. Lo que hace es asociar el espacio de direcciones de cada proceso sobre una parte distinta de la memoria física, de una manera simple. La solución clásica, que se utilizaba en máquinas desde la CDC 6600 (la primera supercomputadora del mundo) hasta el Intel 8088 (el corazón de la IBM PC original), es equipar cada CPU con dos registros de hardware especiales, conocidos comúnmente como los registros

base y **límite**. Cuando se utilizan estos registros, los programas se cargan en ubicaciones consecutivas de memoria en donde haya espacio y sin reubicación durante la carga, como se muestra en la figura 3-2(c). Cuando se ejecuta un proceso, el registro base se carga con la dirección física donde empieza el programa en memoria y el registro límite se carga con la longitud del programa. En la figura 3-2(c), los valores base y límite que se cargarían en estos registros de hardware al ejecutar el primer programa son 0 y 16,384, respectivamente. Los valores utilizados cuando se ejecuta el segundo programa son 16,384 y 16,384, respectivamente. Si se cargara un tercer programa de 16 KB directamente por encima del segundo y se ejecutara, los registros base y límite serían 32,768 y 16,384.

Cada vez que un proceso hace referencia a la memoria, ya sea para obtener una instrucción de memoria, para leer o escribir una palabra de datos, el hardware de la CPU suma de manera automática el valor base a la dirección generada por el proceso antes de enviar la dirección al bus de memoria. Al mismo tiempo comprueba si la dirección ofrecida es igual o mayor que el valor resultante de sumar los valores de los registros límite y base, en cuyo caso se genera un fallo y se aborta el acceso. Por ende, en el caso de la primera instrucción del segundo programa en la figura 3-2(c), el proceso ejecuta una instrucción

```
JMP 28
```

pero el hardware la trata como si fuera

```
JMP 16412
```

por lo que llega a la instrucción `CMP`, como se esperaba. Los valores de los registros base y límite durante la ejecución del segundo programa de la figura 3-2(c) se muestran en la figura 3-3.

El uso de registros base y límite es una manera fácil de proporcionar a cada proceso su propio espacio de direcciones privado, ya que a cada dirección de memoria que se genera en forma automática se le suma el contenido del registro base antes de enviarla a memoria. En muchas implementaciones, los registros base y límite están protegidos de tal forma que sólo el sistema operativo puede modificarlos. Éste fue el caso en la CDC 6600, pero no el del Intel 8088, que ni siquiera tenía el registro límite. Sin embargo, tenía varios registros base, lo cual permitía que se reubicaran texto y datos de manera independiente, pero no ofrecía una protección contra las referencias a memoria fuera de rango.

Una desventaja de la reubicación usando los registros base y límite es la necesidad de realizar una suma y una comparación en cada referencia a memoria. Las comparaciones se pueden hacer con rapidez, pero las sumas son lentas debido al tiempo de propagación del acarreo, a menos que se utilicen circuitos especiales de suma.

3.2.2 Intercambio

Si la memoria física de la computadora es lo bastante grande como para contener todos los procesos, los esquemas descritos hasta ahora funcionarán en forma más o menos correcta. Pero en la práctica, la cantidad total de RAM que requieren todos los procesos es a menudo mucho mayor de lo que puede acomodarse en memoria. En un sistema Windows o Linux común, se pueden iniciar

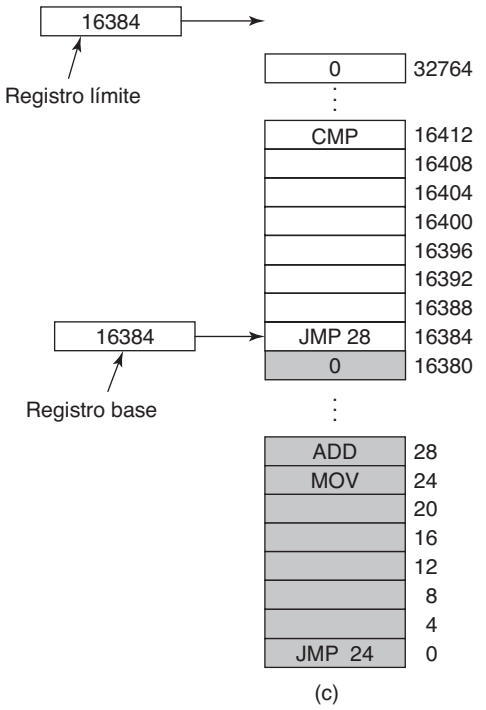


Figura 3-3. Se pueden utilizar registros base y límite para dar a cada proceso un espacio de direcciones separado.

entre 40 y 60 procesos o más cada vez que se inicia la computadora. Por ejemplo, cuando se instala una aplicación Windows, a menudo emite comandos de manera que en los inicios subsiguientes del sistema se inicie un proceso que no haga nada, excepto buscar actualizaciones para la aplicación. Dicho proceso puede ocupar fácilmente de 5 a 10 MB de memoria. Otros procesos en segundo plano comprueban el correo entrante, las conexiones de red entrantes y muchas otras cosas. Y todo esto se hace antes de que se inicie el primer programa de usuario. Hoy en día, los programas de aplicaciones de usuario serios pueden ejecutarse ocupando fácilmente desde 50 a 200 MB y más. En consecuencia, para mantener todos los procesos en memoria todo el tiempo se requiere una gran cantidad de memoria y no puede hacerse si no hay memoria suficiente.

A través de los años se han desarrollado dos esquemas generales para lidiar con la sobrecarga de memoria. La estrategia más simple, conocida como **intercambio**, consiste en llevar cada proceso completo a memoria, ejecutarlo durante cierto tiempo y después regresarlo al disco. Los procesos inactivos mayormente son almacenados en disco, de tal manera que no ocupan memoria cuando no se están ejecutando (aunque algunos de ellos se despiertan periódicamente para realizar su trabajo y después vuelven a quedar inactivos). La otra estrategia, conocida como **memoria virtual**, permite que los programas se ejecuten incluso cuando sólo se encuentran en forma parcial en la memoria. A continuación estudiaremos el intercambio; en la sección 3.3 examinaremos la memoria virtual.

La operación de un sistema de intercambio se ilustra en la figura 3-4. Al principio, sólo el proceso *A* está en la memoria. Después los procesos *B* y *C* se crean o se intercambian desde disco. En la figura 3-4(d), *A* se intercambia al disco. Después llega *D* y sale *B*. Por último, *A* entra de nuevo. Como *A* está ahora en una ubicación distinta, las direcciones que contiene se deben reubicar, ya sea mediante software cuando se intercambia o (más probablemente) mediante hardware durante la ejecución del programa. Por ejemplo, los registros base y límite funcionarían bien en este caso.

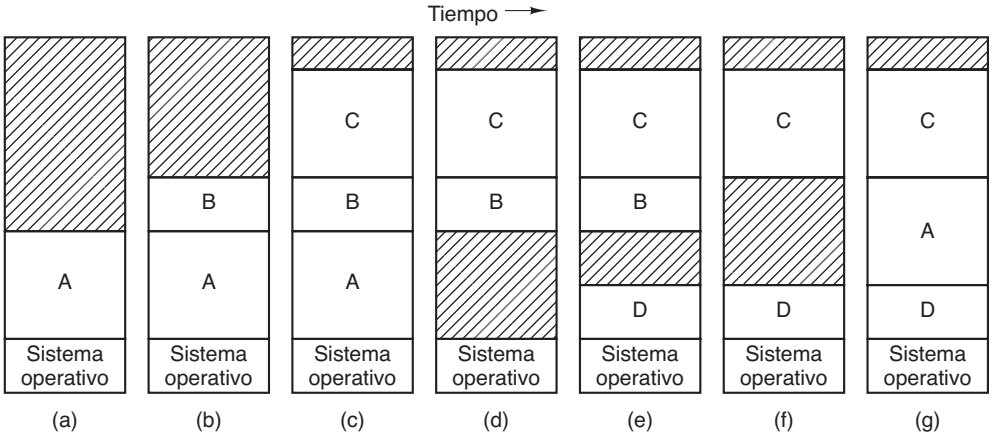


Figura 3-4. La asignación de la memoria cambia a medida que llegan procesos a la memoria y salen de ésta. Las regiones sombreadas son la memoria sin usar.

Cuando el intercambio crea varios huecos en la memoria, es posible combinarlos todos en uno grande desplazando los procesos lo más hacia abajo que sea posible. Esta técnica se conoce como **compactación de memoria**. Por lo general no se realiza debido a que requiere mucho tiempo de la CPU. Por ejemplo, en una máquina con 1 GB que pueda copiar 4 bytes en 20 nseg, se requerirían aproximadamente 5 segundos para compactar toda la memoria.

Un aspecto que vale la pena mencionar es la cantidad de memoria que debe asignarse a un proceso cuando éste se crea o se intercambia. Si los procesos se crean con un tamaño fijo que nunca cambia, entonces la asignación es sencilla: el sistema operativo asigna exactamente lo necesario, ni más ni menos.

No obstante, si los segmentos de datos de los procesos pueden crecer, por ejemplo mediante la asignación dinámica de memoria proveniente de un heap, como en muchos lenguajes de programación, ocurre un problema cuando un proceso trata de crecer. Si hay un hueco adyacente al proceso, puede asignarse y se permite al proceso crecer en el hueco; si el proceso está adyacente a otro proceso, el proceso en crecimiento tendrá que moverse a un hueco en memoria que sea lo bastante grande como para alojarlo, o habrá que intercambiar uno o más procesos para crear un hueco con el tamaño suficiente. Si un proceso no puede crecer en memoria y el área de intercambio en el disco está llena, el proceso tendrá que suspenderse hasta que se libere algo de espacio (o se puede eliminar).

Si se espera que la mayoría de los procesos crezcan a medida que se ejecuten, probablemente sea conveniente asignar un poco de memoria adicional cada vez que se intercambia o se mueve un proceso, para reducir la sobrecarga asociada con la acción de mover o intercambiar procesos que ya no caben en su memoria asignada. No obstante, al intercambiar procesos al disco, se debe intercambiar sólo la memoria que se encuentre en uso; es un desperdicio intercambiar también la memoria adicional. En la figura 3-5(a) podemos ver una configuración de memoria en la cual se ha asignado espacio para que crezcan dos procesos.

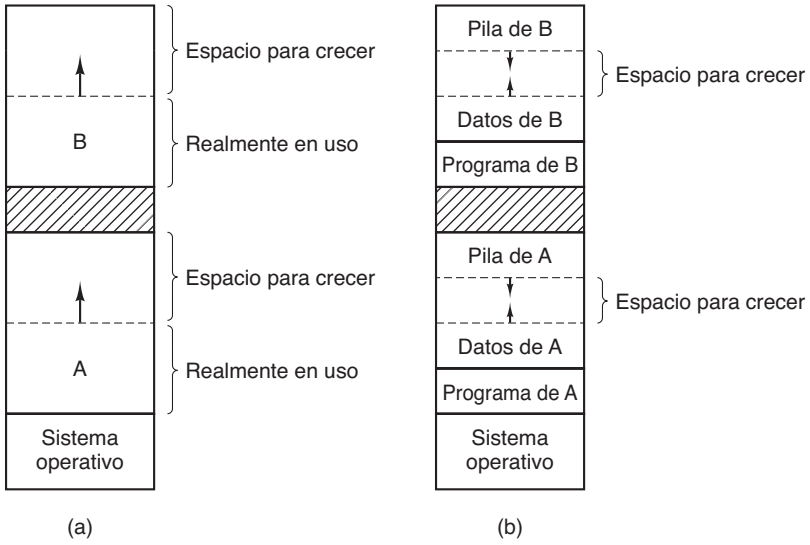


Figura 3-5. (a) Asignación de espacio para un segmento de datos en crecimiento. (b) Asignación de espacio para una pila en crecimiento y un segmento de datos en crecimiento.

Si los procesos pueden tener dos segmentos en crecimiento, por ejemplo, cuando el segmento de datos se utiliza como heap para las variables que se asignan y liberan en forma dinámica y un segmento de pila para las variables locales normales y las direcciones de retorno, un arreglo alternativo se sugiere por sí mismo, a saber, el de la figura 3-5(b). En esta figura podemos ver que cada proceso ilustrado tiene una pila en la parte superior de su memoria asignada, la cual está creciendo hacia abajo, y un segmento de datos justo debajo del texto del programa, que está creciendo hacia arriba. La memoria entre estos segmentos se puede utilizar para cualquiera de los dos. Si se agota, el proceso tendrá que moverse a un hueco con suficiente espacio, intercambiarse fuera de la memoria hasta que se pueda crear un hueco lo suficientemente grande, o eliminarse.

3.2.3 Administración de memoria libre

Cuando la memoria se asigna en forma dinámica, el sistema operativo debe administrarla. En términos generales, hay dos formas de llevar el registro del uso de la memoria: mapas de bits y listas libres. En esta sección y en la siguiente analizaremos estos dos métodos.

Administración de memoria con mapas de bits

Con un mapa de bits, la memoria se divide en unidades de asignación tan pequeñas como unas cuantas palabras y tan grandes como varios kilobytes. Para cada unidad de asignación hay un bit correspondiente en el mapa de bits, que es 0 si la unidad está libre y 1 si está ocupada (o viceversa). La figura 3-6 muestra parte de la memoria y el mapa de bits correspondiente.

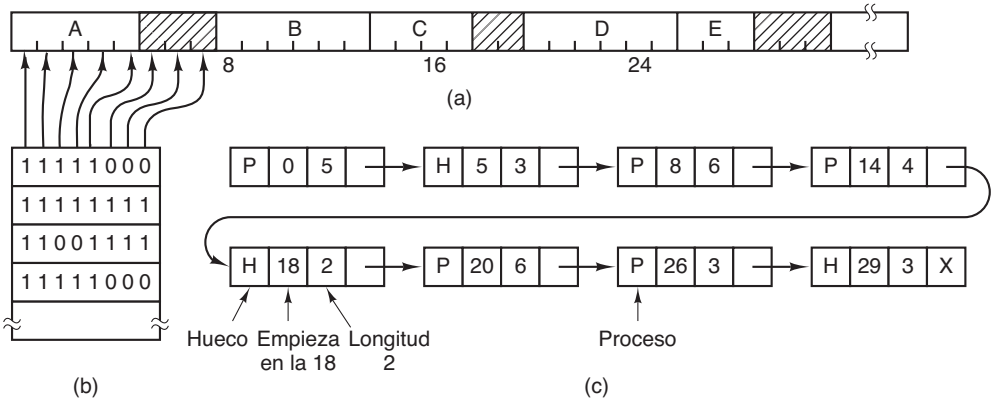


Figura 3-6. (a) Una parte de la memoria con cinco procesos y tres huecos. Las marcas de graduación muestran las unidades de asignación de memoria. Las regiones sombreadas (0 en el mapa de bits) están libres. (b) El mapa de bits correspondiente. (c) La misma información en forma de lista.

El tamaño de la unidad de asignación es una importante cuestión de diseño. Entre más pequeña sea la unidad de asignación, mayor será el mapa de bits. Sin embargo, aun con una unidad de asignación tan pequeña como 4 bytes, 32 bits de memoria sólo requerirán 1 bit del mapa. Una memoria de 32*n* bits utilizará *n* bits del mapa, por lo que el mapa de bits ocupará sólo 1/33 de la memoria. Si la unidad de asignación se elige de manera que sea grande, el mapa de bits será más pequeño pero se puede desperdiciar una cantidad considerable de memoria en la última unidad del proceso si su tamaño no es un múltiplo exacto de la unidad de asignación.

Un mapa de bits proporciona una manera simple de llevar el registro de las palabras de memoria en una cantidad fija de memoria, debido a que el tamaño del mapa de bits sólo depende del tamaño de la memoria y el tamaño de la unidad de asignación. El problema principal es que, cuando se ha decidido llevar un proceso de *k* unidades a la memoria, el administrador de memoria debe buscar en el mapa para encontrar una serie de *k* bits consecutivos con el valor 0 en el mapa de bits. El proceso de buscar en un mapa de bits una serie de cierta longitud es una operación lenta (debido a que la serie puede cruzar los límites entre las palabras en el mapa); éste es un argumento contra los mapas de bits.

Administración de memoria con listas ligadas

Otra manera de llevar el registro de la memoria es mantener una lista ligada de segmentos de memoria asignados y libres, en donde un segmento contiene un proceso o es un hueco vacío entre dos

procesos. La memoria de la figura 3-6(a) se representa en la figura 3-6(c) como una lista ligada de segmentos. Cada entrada en la lista especifica un hueco (H) o un proceso (P), la dirección en la que inicia, la longitud y un apuntador a la siguiente entrada.

En este ejemplo, la lista de segmentos se mantiene ordenada por dirección. Al ordenarla de esta manera, tenemos la ventaja de que cuando termina un proceso o se intercambia, el proceso de actualizar la lista es simple. Un proceso en terminación por lo general tiene dos vecinos (excepto cuando se encuentra en la parte superior o inferior de la memoria). Éstos pueden ser procesos o huecos, lo cual conduce a las cuatro combinaciones de la figura 3-7. En la figura 3-7(a), para actualizar la lista se requiere reemplazar una P por una H. En las figuras 3-7(b) y 3-7(c) dos entradas se fusionan en una sola y la lista se reduce en una entrada. En la figura 3-7(d), se fusionan las tres entradas y dos elementos son eliminados de la lista.

Como la ranura de la tabla de procesos para el proceso en terminación en general apuntará a la entrada en la lista para el mismo proceso, puede ser más conveniente tener la lista como una lista doblemente ligada, en vez de la lista simplemente ligada de la figura 3-6(c). Esta estructura facilita encontrar la entrada anterior y ver si es posible una combinación.

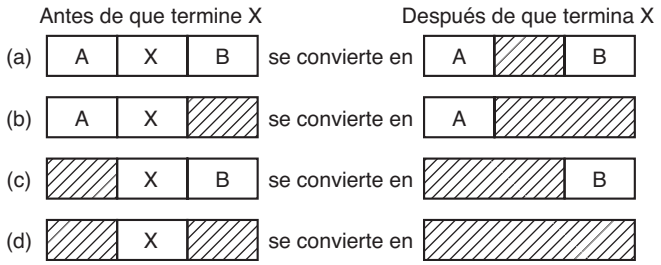


Figura 3-7. Cuatro combinaciones de los vecinos para el proceso en terminación, *X*.

Cuando los procesos y huecos se mantienen en una lista ordenada por dirección, se pueden utilizar varios algoritmos para asignar memoria a un proceso creado (o a un proceso existente que se intercambie del disco). Suponemos que el administrador de memoria sabe cuánta debe asignar. El algoritmo más simple es el de **primer ajuste**: el administrador de memoria explora la lista de segmentos hasta encontrar un hueco que sea lo bastante grande. Después el hueco se divide en dos partes, una para el proceso y otra para la memoria sin utilizar, excepto en el estadísticamente improbable caso de un ajuste exacto. El algoritmo del primer ajuste es rápido debido a que busca lo menos posible.

Una pequeña variación del algoritmo del primer ajuste es el algoritmo del **siguiente ajuste**. Funciona de la misma manera que el primer ajuste, excepto porque lleva un registro de dónde se encuentra cada vez que descubre un hueco adecuado. La siguiente vez que es llamado para buscar un hueco, empieza a buscar en la lista desde el lugar en el que se quedó la última vez, en vez de empezar siempre desde el principio, como el algoritmo del primer ajuste. Las simulaciones realizadas por Bays (1977) muestran que el algoritmo del siguiente ajuste tiene un rendimiento ligeramente peor que el del primer ajuste.

Otro algoritmo muy conocido y ampliamente utilizado es el del **mejor ajuste**. Este algoritmo busca en toda la lista, de principio a fin y toma el hueco más pequeño que sea adecuado. En vez de dividir un gran hueco que podría necesitarse después, el algoritmo del mejor ajuste trata de buscar

un hueco que esté cerca del tamaño actual necesario, que coincida mejor con la solicitud y los huecos disponibles.

Como ejemplo de los algoritmos de primer ajuste y de mejor ajuste, considere de nuevo la figura 3-6. Si se necesita un bloque de tamaño 2, el algoritmo del primer ajuste asignará el hueco en la 5, pero el del mejor ajuste asignará el hueco en la 18.

El algoritmo del mejor ajuste es más lento que el del primer ajuste, ya que debe buscar en toda la lista cada vez que se le llama. De manera sorprendente, también provoca más desperdicio de memoria que los algoritmos del primer ajuste o del siguiente ajuste, debido a que tiende a llenar la memoria con huecos pequeños e inutilizables. El algoritmo del primer ajuste genera huecos más grandes en promedio.

Para resolver el problema de dividir las coincidencias casi exactas en un proceso y en un pequeño hueco, podríamos considerar el algoritmo del **peor ajuste**, es decir, tomar siempre el hueco más grande disponible, de manera que el nuevo hueco sea lo bastante grande como para ser útil. La simulación ha demostrado que el algoritmo del peor ajuste no es muy buena idea tampoco.

Los cuatro algoritmos pueden ser acelerados manteniendo listas separadas para los procesos y los huecos. De esta forma, todos ellos dedican toda su energía a inspeccionar los huecos, no los procesos. El precio inevitable que se paga por esta aceleración en la asignación es la complejidad adicional y la lentitud al desasignar la memoria, ya que un segmento liberado tiene que eliminarse de la lista de procesos e insertarse en la lista de huecos.

Si se mantienen distintas listas para los procesos y los huecos, la lista de huecos se puede mantener ordenada por el tamaño, para que el algoritmo del mejor ajuste sea más rápido. Cuando el algoritmo del mejor ajuste busca en una lista de huecos, del más pequeño al más grande, tan pronto como encuentre un hueco que ajuste, sabrá que el hueco es el más pequeño que se puede utilizar, de aquí que se le denomine el mejor ajuste. No es necesario buscar más, como con el esquema de una sola lista. Con una lista de huecos ordenada por tamaño, los algoritmos del primer ajuste y del mejor ajuste son igual de rápidos, y hace innecesario usar el del siguiente ajuste.

Cuando los huecos se mantienen en listas separadas de los procesos, una pequeña optimización es posible. En vez de tener un conjunto separado de estructuras de datos para mantener la lista de huecos, como se hizo en la figura 3-6(c), la información se puede almacenar en los huecos. La primera palabra de cada hueco podría ser el tamaño del mismo y la segunda palabra un apuntador a la siguiente entrada. Los nodos de la lista de la figura 3-6(c), que requieren tres palabras y un bit (P/H), ya no son necesarios.

Un algoritmo de asignación más es el denominado de **ajuste rápido**, el cual mantiene listas separadas para algunos de los tamaños más comunes solicitados. Por ejemplo, podría tener una tabla con *n* entradas, en donde la primera entrada es un apuntador a la cabeza de una lista de huecos de 4 KB, la segunda entrada es un apuntador a una lista de huecos de 8 KB, la tercera entrada un apuntador a huecos de 12 KB y así sucesivamente. Por ejemplo, los huecos de 21 KB podrían colocarse en la lista de 20 KB o en una lista especial de huecos de tamaño inusual.

Con el algoritmo del ajuste rápido, buscar un hueco del tamaño requerido es extremadamente rápido, pero tiene la misma desventaja que todos los esquemas que se ordenan por el tamaño del hueco: cuando un proceso termina o es intercambiado, buscar en sus vecinos para ver si es posible una fusión es un proceso costoso. Si no se realiza la fusión, la memoria se fragmentará rápidamente en un gran número de pequeños huecos en los que no cabrá ningún proceso.

3.3 MEMORIA VIRTUAL

Mientras que los registros base y límite se pueden utilizar para crear la abstracción de los espacios de direcciones, hay otro problema que se tiene que resolver: la administración del agrandamiento del software (*bloatware*). Aunque el tamaño de las memorias se incrementa con cierta rapidez, el del software aumenta con una mucha mayor. En la década de 1980, muchas universidades operaban un sistema de tiempo compartido con docenas de usuarios (más o menos satisfechos) trabajando simultáneamente en una VAX de 4 MB. Ahora Microsoft recomienda tener por lo menos 512 MB para que un sistema Vista de un solo usuario ejecute aplicaciones simples y 1 GB si el usuario va a realizar algún trabajo serio. La tendencia hacia la multimedia impone aún mayores exigencias sobre la memoria.

Como consecuencia de estos desarrollos, existe la necesidad de ejecutar programas que son demasiado grandes como para caber en la memoria y sin duda existe también la necesidad de tener sistemas que puedan soportar varios programas ejecutándose al mismo tiempo, cada uno de los cuales cabe en memoria, pero que en forma colectiva exceden el tamaño de la misma. El intercambio no es una opción atractiva, ya que un disco SATA ordinario tiene una velocidad de transferencia pico de 100 MB/segundo a lo más, lo cual significa que requiere por lo menos 10 segundos para intercambiar un programa de 1 GB de memoria a disco y otros 10 segundos para intercambiar un programa de 1 GB del disco a memoria.

El problema de que los programas sean más grandes que la memoria ha estado presente desde los inicios de la computación, en áreas limitadas como la ciencia y la ingeniería (para simular la creación del universo o, incluso, para simular una nueva aeronave, se requiere mucha memoria). Una solución que se adoptó en la década de 1960 fue dividir los programas en pequeñas partes, conocidas como **sobrepuestos** (***overlays***). Cuando empezaba un programa, todo lo que se cargaba en memoria era el administrador de sobrepuestos, que de inmediato cargaba y ejecutaba el sobrepuesto 0; cuando éste terminaba, indicaba al administrador de sobrepuestos que cargara el 1 encima del sobrepuesto 0 en la memoria (si había espacio) o encima del mismo (si no había espacio). Algunos sistemas de sobrepuestos eran muy complejos, ya que permitían varios sobrepuestos en memoria a la vez. Los sobrepuestos se mantenían en el disco, intercambiándolos primero hacia adentro de la memoria y después hacia afuera de la memoria mediante el administrador de sobrepuestos.

Aunque el trabajo real de intercambiar sobrepuestos hacia adentro y hacia afuera de la memoria lo realizaba el sistema operativo, el de dividir el programa en partes tenía que realizarlo el programador en forma manual. El proceso de dividir programas grandes en partes modulares más pequeñas consumía mucho tiempo, y era aburrido y propenso a errores. Pocos programadores eran buenos para esto. No pasó mucho tiempo antes de que alguien ideara una forma de pasar todo el trabajo a la computadora.

El método ideado (Fotheringham, 1961) se conoce actualmente como **memoria virtual**. La idea básica detrás de la memoria virtual es que cada programa tiene su propio espacio de direcciones, el cual se divide en trozos llamados **páginas**. Cada página es un rango contiguo de direcciones. Estas páginas se asocian a la memoria física, pero no todas tienen que estar en la memoria física para poder ejecutar el programa. Cuando el programa hace referencia a una parte de su espacio de direcciones que está en la memoria física, el hardware realiza la asociación necesaria al instante. Cuando el programa hace referencia a una parte de su espacio de direcciones que *no* está en la memoria física, el sistema operativo recibe una alerta para buscar la parte faltante y volver a ejecutar la instrucción que falló.

En cierto sentido, la memoria virtual es una generalización de la idea de los registros base y límite. El procesador 8088 tenía registros base separados (pero no tenía registros límite) para texto y datos. Con la memoria virtual, en vez de tener una reubicación por separado para los segmentos de texto y de datos, todo el espacio de direcciones se puede asociar a la memoria física en unidades pequeñas equitativas. A continuación le mostraremos cómo se implementa la memoria virtual.

La memoria virtual funciona muy bien en un sistema de multiprogramación, con bits y partes de muchos programas en memoria a la vez. Mientras un programa espera a que una parte del mismo se lea y coloque en la memoria, la CPU puede otorgarse a otro proceso.

3.3.1 Paginación

La mayor parte de los sistemas de memoria virtual utilizan una técnica llamada **paginación**, que describiremos a continuación. En cualquier computadora, los programas hacen referencia a un conjunto de direcciones de memoria. Cuando un programa ejecuta una instrucción como

```
MOV REG, 1000
```

lo hace para copiar el contenido de la dirección de memoria 1000 a `REG` (o viceversa, dependiendo de la computadora). Las direcciones se pueden generar usando indexado, registros base, registros de segmentos y otras formas más.

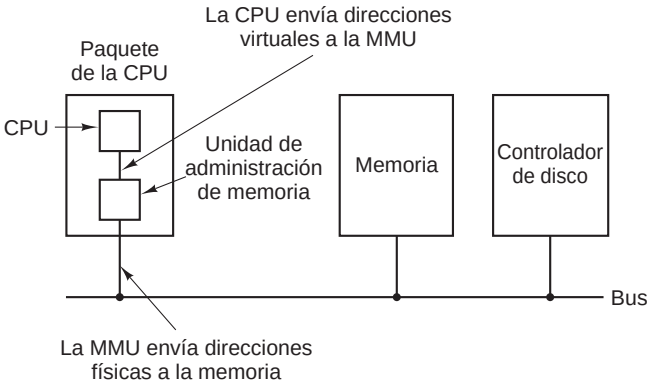


Figura 3-8. La posición y función de la MMU. Aquí la MMU se muestra como parte del chip de CPU, debido a que es común esta configuración en la actualidad. Sin embargo, lógicamente podría ser un chip separado y lo era hace años.

Estas direcciones generadas por el programa se conocen como **direcciones virtuales** y forman el **espacio de direcciones virtuales**. En las computadoras sin memoria virtual, la dirección física se coloca directamente en el bus de memoria y hace que se lea o escriba la palabra de memoria física con la misma dirección. Cuando se utiliza memoria virtual, las direcciones virtuales no van directamente al bus de memoria. En vez de ello, van a una **MMU** (*Memory Managemente Unit*,

Unidad de administración de memoria) que asocia las direcciones virtuales a las direcciones de memoria físicas, como se ilustra en la figura 3-8.

En la figura 3-9 se muestra un ejemplo muy simple de cómo funciona esta asociación. En este ejemplo, tenemos una computadora que genera direcciones de 16 bits, desde 0 hasta 64 K. Éstas son las direcciones virtuales. Sin embargo, esta computadora sólo tiene 32 KB de memoria física. Así, aunque se pueden escribir programas de 64 KB, no se pueden cargar completos en memoria y ejecutarse. No obstante, una copia completa de la imagen básica de un programa, de hasta 64 KB, debe estar presente en el disco para que las partes se puedan traer a la memoria según sea necesario.

El espacio de direcciones virtuales se divide en unidades de tamaño fijo llamadas **páginas**. Las unidades correspondientes en la memoria física se llaman **marcos de página**. Las páginas y los marcos de página por lo general son del mismo tamaño. En este ejemplo son de 4 KB, pero en sistemas reales se han utilizado tamaños de página desde 512 bytes hasta 64 KB. Con 64 KB de espacio de direcciones virtuales y 32 KB de memoria física obtenemos 16 páginas virtuales y 8 marcos de página. Las transferencias entre la RAM y el disco siempre son en páginas completas.

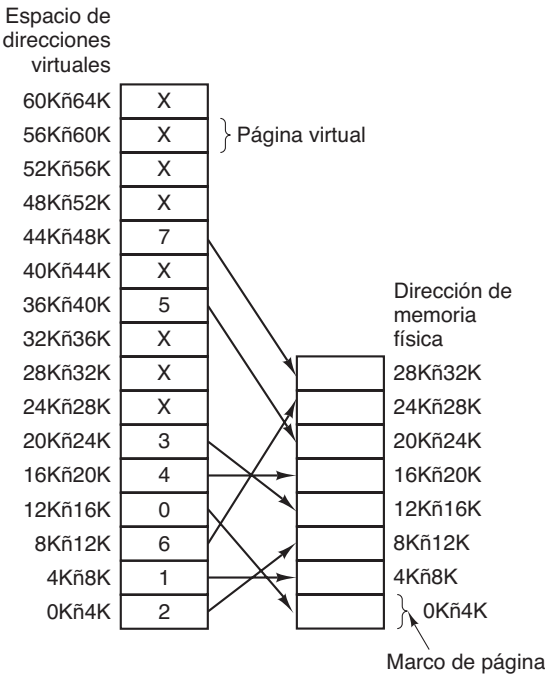


Figura 3-9. La relación entre las direcciones virtuales y las direcciones de memoria física está dada por la tabla de páginas. Cada página empieza en un múltiplo de 4096 y termina 4095 direcciones más arriba, por lo que de 4 K a 8 K en realidad significa de 4096 a 8191 y de 8 K a 12 K significa de 8192 a 12287.

La notación en la figura 3-9 es la siguiente. El rango marcado de 0K a 4 K significa que las direcciones virtuales o físicas en esa página son de 0 a 4095. El rango de 4 K a 8 K se refiere a las

direcciones de 4096 a 8191 y así en lo sucesivo. Cada página contiene exactamente 4096 direcciones que empiezan en un múltiplo de 4096 y terminan uno antes del múltiplo de 4096.

Por ejemplo, cuando el programa trata de acceder a la dirección 0 usando la instrucción

```
MOV REG,0
```

la dirección virtual 0 se envía a la MMU. La MMU ve que esta dirección virtual está en la página 0 (0 a 4095), que de acuerdo con su asociación es el marco de página 2 (8192 a 12287). Así, transforma la dirección en 8192 y envía la dirección 8192 al bus. La memoria no sabe nada acerca de la MMU y sólo ve una petición para leer o escribir en la dirección 8192, la cual cumple. De esta manera, la MMU ha asociado efectivamente todas las direcciones virtuales entre 0 y 4095 sobre las direcciones físicas de 8192 a 12287.

De manera similar, la instrucción

```
MOV REG,8192
```

se transforma efectivamente en

```
MOV REG,24576
```

debido a que la dirección virtual 8192 (en la página virtual 2) se asocia con la dirección 24576 (en el marco de página físico 6). Como tercer ejemplo, la dirección virtual 20500 está a 20 bytes del inicio de la página virtual 5 (direcciones virtuales 20480 a 24575) y la asocia con la dirección física $12288 + 20 = 12308$.

Por sí sola, la habilidad de asociar 16 páginas virtuales a cualquiera de los ocho marcos de página mediante la configuración de la apropiada asociación de la MMU no resuelve el problema de que el espacio de direcciones virtuales sea más grande que la memoria física. Como sólo tenemos ocho marcos de página físicos, sólo ocho de las páginas virtuales en la figura 3-9 se asocian a la memoria física. Las demás, que se muestran con una cruz en la figura, no están asociadas. En el hardware real, un **bit de presente/ausente** lleva el registro de cuáles páginas están físicamente presentes en la memoria.

¿Qué ocurre si el programa hace referencia a direcciones no asociadas, por ejemplo, mediante el uso de la instrucción

```
MOV REG,32780
```

que es el byte 12 dentro de la página virtual 8 (empezando en 32768)? La MMU detecta que la página no está asociada (lo cual se indica mediante una cruz en la figura) y hace que la CPU haga un trap al sistema operativo. A este trap se le llama **fallo de página**. El sistema operativo selecciona un marco de página que se utilice poco y escribe su contenido de vuelta al disco (si no es que ya está ahí). Después obtiene la página que se acaba de referenciar en el marco de página que se acaba de liberar, cambia la asociación y reinicia la instrucción que originó el trap.

Por ejemplo, si el sistema operativo decidiera desalojar el marco de página 1, cargaría la página virtual 8 en la dirección física 8192 y realizaría dos cambios en la asociación de la MMU. Primero, marcaría la entrada de la página virtual 1 como no asociada, para hacer un trap por cualquier acceso a las direcciones virtuales entre 4096 y 8191. Después reemplazaría la cruz en la entrada de

la página virtual 8 con un 1, de manera que al ejecutar la instrucción que originó el trap, asocie la dirección virtual 32780 a la dirección física 4108 (4096 + 12).

Ahora veamos dentro de la MMU para analizar su funcionamiento y por qué hemos optado por utilizar un tamaño de página que sea una potencia de 2. En la figura 3-10 vemos un ejemplo de una dirección virtual, 8196 (0010000000000100 en binario), que se va a asociar usando la asociación de la MMU en la figura 3-9. La dirección virtual entrante de 16 bits se divide en un número de página de 4 bits y en un desplazamiento de 12 bits. Con 4 bits para el número de página, podemos tener 16 páginas y con los 12 bits para el desplazamiento, podemos direccionar todos los 4096 bytes dentro de una página.

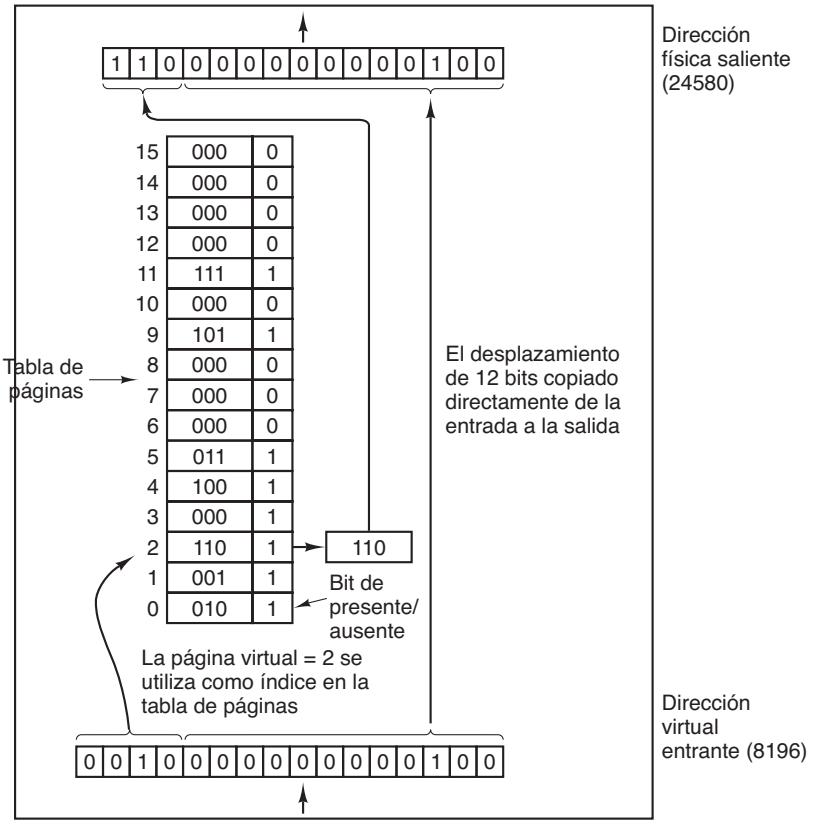


Figura 3-10. La operación interna de la MMU con 16 páginas de 4 KB.

El número de página se utiliza como índice en la **tabla de páginas**, conduciendo al número del marco de página que corresponde a esa página virtual. Si el bit de *presente/ausente* es 0, se provoca un trap al sistema operativo. Si el bit es 1, el número del marco de página encontrado en la tabla de páginas se copia a los 3 bits de mayor orden del registro de salida, junto con el desplazamiento de 12 bits, que se copia sin modificación de la dirección virtual entrante. En conjunto forman una dirección física de 15 bits. Después, el registro de salida se coloca en el bus de memoria como la dirección de memoria física.

3.3.2 Tablas de páginas

En una implementación simple, la asociación de direcciones virtuales a direcciones físicas se puede resumir de la siguiente manera: la dirección virtual se divide en un número de página virtual (bits de mayor orden) y en un desplazamiento (bits de menor orden). Por ejemplo, con una dirección de 16 bits y un tamaño de página de 4 KB, los 4 bits superiores podrían especificar una de las 16 páginas virtuales y los 12 bits inferiores podrían entonces especificar el desplazamiento de bytes (0 a 4095) dentro de la página seleccionada. Sin embargo, también es posible una división con 3, 5 u otro número de bits para la página. Las distintas divisiones implican diferentes tamaños de página.

El número de página virtual se utiliza como índice en la tabla de páginas para buscar la entrada para esa página virtual. En la entrada en la tabla de páginas, se encuentra el número de marco de página (si lo hay). El número del marco de página se adjunta al extremo de mayor orden del desplazamiento, reemplazando el número de página virtual, para formar una dirección física que se pueda enviar a la memoria.

Por ende, el propósito de la tabla de páginas es asociar páginas virtuales a los marcos de página. Hablando en sentido matemático, la tabla de páginas es una función donde el número de página virtual es un argumento y el número de marco físico es un resultado. Utilizando el resultado de esta función, el campo de la página virtual en una dirección virtual se puede reemplazar por un campo de marco de página, formando así una dirección de memoria física.

Estructura de una entrada en la tabla de páginas

Ahora vamos a pasar de la estructura de las tablas de páginas en general a los detalles de una sola entrada en la tabla de páginas. La distribución exacta de una entrada depende en gran parte de la máquina, pero el tipo de información presente es aproximadamente el mismo de una máquina a otra. En la figura 3-11 proporcionamos un ejemplo de una entrada en la tabla de páginas. El tamaño varía de una computadora a otra, pero 32 bits es un tamaño común. El campo más importante es el *número de marco de página*. Después de todo, el objetivo de la asociación de páginas es mostrar este valor. Enseguida de este campo tenemos el bit de *presente/ausente*. Si este bit es 1, la entrada es válida y se puede utilizar. Si es 0, la página virtual a la que pertenece la entrada no se encuentra actualmente en la memoria. Al acceder a una entrada en la tabla de página con este bit puesto en 0 se produce un fallo de página.

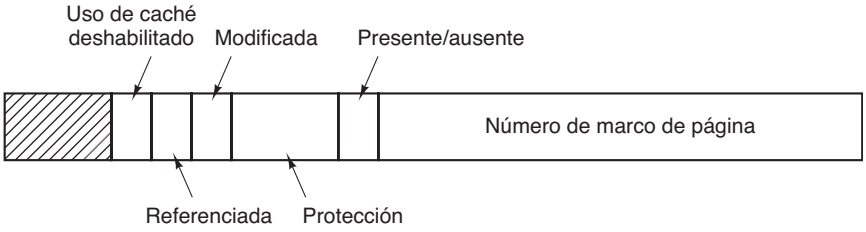


Figura 3-11. Una típica entrada en la tabla de páginas.

Los bits de *protección* indican qué tipo de acceso está permitido. En su forma más simple, este campo contiene 1 bit, con 0 para lectura/escritura y 1 para sólo lectura. Un arreglo más sofisticado es tener 3 bits: uno para habilitar la lectura, otro para la escritura y el tercero para ejecutar la página.

Los bits de *modificada* y *referenciada* llevan el registro del uso de páginas. Cuando se escribe en una página, el hardware establece de manera automática el bit de *modificada*. Este bit es valioso cuando el sistema operativo decide reclamar un marco de página. Si la página en él ha sido modificada (es decir, está "sucia"), debe escribirse de vuelta en el disco. Si no se ha modificado (es decir, está "limpia") sólo se puede abandonar, debido a que la copia del disco es aun válida. A este bit se le conoce algunas veces como **bit sucio**, ya que refleja el estado de la página.

El bit de *referenciada* se establece cada vez que una página es referenciada, ya sea para leer o escribir. Su función es ayudar al sistema operativo a elegir una página para desalojarla cuando ocurre un fallo de página. Las páginas que no se estén utilizando son mejores candidatos que las páginas que se están utilizando y este bit desempeña un importante papel en varios de los algoritmos de reemplazo de páginas que estudiaremos más adelante en este capítulo.

Finalmente, el último bit permite deshabilitar el uso de caché para la página. Esta característica es importante para las páginas que se asocian con los registros de dispositivos en vez de la memoria. Si el sistema operativo está esperando en un ciclo de espera hermético a que cierto dispositivo de E/S responda a un comando que acaba de recibir, es esencial que el hardware siga obteniendo la palabra del dispositivo y no utilice una copia antigua en la caché. Con este bit, el uso de la caché se puede desactivar. Las máquinas que tienen un espacio de E/S separado y no utilizan E/S asociada a la memoria no necesitan este bit.

Observe que la dirección del disco utilizado para guardar la página cuando no está en memoria no forma parte de la tabla de páginas. La razón es simple: la tabla de páginas sólo guarda la información que el hardware necesita para traducir una dirección virtual en una dirección física. La información que necesita el sistema operativo para manejar los fallos de página se mantiene en tablas de software dentro del sistema operativo. El hardware no la necesita.

Antes de analizar más aspectos de la implementación, vale la pena recalcar de nuevo que lo que la memoria virtual hace es crear una abstracción —el espacio de direcciones— que es una abstracción de la memoria física, al igual que un proceso es una abstracción del procesador físico (CPU). Para implementar la memoria virtual, hay que descomponer el espacio de direcciones virtuales en páginas y asociar cada una a cierto marco de página de memoria física o dejarla (temporalmente) sin asociar. Por ende, este capítulo trata fundamentalmente de una abstracción creada por el sistema operativo y la forma en que se administra esa abstracción.

3.3.3 Aceleración de la paginación

Acabamos de ver los fundamentos de la memoria virtual y la paginación. Ahora es tiempo de entrar en más detalle acerca de las posibles implementaciones. En cualquier sistema de paginación hay que abordar dos cuestiones principales:

1. La asociación de una dirección virtual a una dirección física debe ser rápida.

2. Si el espacio de direcciones virtuales es grande, la tabla de páginas será grande.

El primer punto es una consecuencia del hecho de que la asociación virtual-a-física debe realizarse en cada referencia de memoria. Todas las instrucciones deben provenir finalmente de la memoria y muchas de ellas hacen referencias a operandos en memoria también. En consecuencia, es necesario hacer una, dos o algunas veces más referencias a la tabla de páginas por instrucción. Si la ejecución de una instrucción tarda, por ejemplo 1 nseg, la búsqueda en la tabla de páginas debe realizarse en menos de 0.2 nseg para evitar que la asociación se convierta en un cuello de botella importante.

El segundo punto se deriva del hecho de que todas las computadoras modernas utilizan direcciones virtuales de por lo menos 32 bits, donde 64 bits se vuelven cada vez más comunes. Por decir, con un tamaño de página de 4 KB, un espacio de direcciones de 32 bits tiene 1 millón de páginas y un espacio de direcciones de 64 bits tiene más de las que desearíamos contemplar. Con 1 millón de páginas en el espacio de direcciones virtual, la tabla de páginas debe tener 1 millón de entradas. Y recuerde que cada proceso necesita su propia tabla de páginas (debido a que tiene su propio espacio de direcciones virtuales).

La necesidad de una asociación de páginas extensa y rápida es una restricción considerable en cuanto a la manera en que se construyen las computadoras. El diseño más simple (por lo menos en concepto) es tener una sola tabla de páginas que consista en un arreglo de registros de hardware veloces, con una entrada para cada página virtual, indexada por número de página virtual, como se muestra en la figura 3-10. Cuando se inicia un proceso, el sistema operativo carga los registros con la tabla de páginas del proceso, tomada de una copia que se mantiene en la memoria principal. Durante la ejecución del proceso no se necesitan más referencias a memoria para la tabla de páginas. Las ventajas de este método son que es simple y no requiere referencias a memoria durante la asociación. Una desventaja es que es extremadamente costoso que la tabla de páginas sea extensa; otra es que tener que cargar la tabla de páginas completa en cada conmutación de contexto ve afectado el rendimiento.

En el otro extremo, toda la tabla de páginas puede estar en la memoria principal. Así, todo lo que el hardware necesita es un solo registro que apunte al inicio de la tabla de páginas. Este diseño permite cambiar la asociación de direcciones virtuales a direcciones físicas al momento de una conmutación de contexto con sólo recargar un registro. Desde luego, tiene la desventaja de requerir una o más referencias a memoria para leer las entradas en la tabla de páginas durante la ejecución de cada instrucción, con lo cual se hace muy lenta.

Búferes de traducción adelantada

Ahora veamos esquemas implementados ampliamente para acelerar la paginación y manejar espacios de direcciones virtuales extensos, empezando con la aceleración de la paginación. El punto inicial de la mayor parte de las técnicas de optimización es que la tabla de páginas está en la memoria. Potencialmente, este diseño tiene un enorme impacto sobre el rendimiento. Por ejemplo, considere una instrucción de 1 byte que copia un registro a otro. A falta de paginación, esta instrucción hace

sólo una referencia a memoria para obtener la instrucción. Con la paginación se requiere al menos una referencia adicional a memoria para acceder a la tabla de páginas. Como la velocidad de ejecución está comúnmente limitada por la proporción a la que la CPU puede obtener instrucciones y datos de la memoria, al tener que hacer dos referencias a memoria por cada una de ellas se reduce el rendimiento a la mitad. Bajo estas condiciones, nadie utilizaría la paginación.

Los diseñadores de computadoras han sabido acerca de este problema durante años y han ideado una solución que está basada en la observación de que la mayor parte de los programas tienden a hacer un gran número de referencias a un pequeño número de páginas y no viceversa. Por ende, sólo se lee con mucha frecuencia una pequeña fracción de las entradas en la tabla de páginas; el resto se utiliza muy pocas veces.

La solución que se ha ideado es equipar a las computadoras con un pequeño dispositivo de hardware para asociar direcciones virtuales a direcciones físicas sin pasar por la tabla de páginas. El dispositivo, llamado **TLB** (*Translation Lookaside Buffer*, Búfer de traducción adelantada) o algunas veces **memoria asociativa**, se ilustra en la figura 3-12. Por lo general se encuentra dentro de la MMU y consiste en un pequeño número de entradas, ocho en este ejemplo, pero raras veces más de 64. Cada entrada contiene información acerca de una página, incluyendo el número de página virtual, un bit que se establece cuando se modifica la página, el código de protección (permisos de lectura/escritura/ejecución) y el marco de página físico en el que se encuentra la página. Estos campos tienen una correspondencia de uno a uno con los campos en la tabla de páginas, excepto por el número de página virtual, que no se necesita en la tabla de páginas. Otro bit indica si la entrada es válida (es decir, si está en uso) o no.

Válida	Página virtual	Modificada	Protección	Marco de página
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

Figura 3-12. Un TLB para acelerar la paginación.

Un ejemplo que podría generar el TLB de la figura 3-12 es un proceso en un ciclo que abarque las páginas virtuales 19, 20 y 21, de manera que estas entradas en el TLB tengan códigos de protección para lectura y ejecución. Los datos principales que se están utilizando (por decir, un arreglo que se esté procesando) se encuentran en las páginas 129 y 130. La página 140 contiene los índices utilizados en los cálculos del arreglo. Al final, la pila está en las páginas 860 y 861.

Ahora veamos cómo funciona el TLB. Cuando se presenta una dirección virtual a la MMU para que la traduzca, el hardware primero comprueba si su número de página virtual está presente en

el TLB al compararla con todas las entradas en forma simultánea (es decir, en paralelo). Si se encuentra una coincidencia válida y el acceso no viola los bits de protección, el marco de página se toma directamente del TLB, sin pasar por la tabla de páginas. Si el número de página virtual está presente en el TLB, pero la instrucción está tratando de escribir en una página de sólo lectura, se genera un fallo por protección.

El caso interesante es lo que ocurre cuando el número de página virtual no está en el TLB. La MMU detecta que no está y realiza una búsqueda ordinaria en la tabla de páginas. Después desaloja una de las entradas del TLB y la reemplaza con la entrada en la tabla de páginas que acaba de buscar. De esta forma, si esa página se utiliza pronto otra vez, la segunda vez se producirá un acierto en el TLB en vez de un fracaso. Cuando se purga una entrada del TLB, el bit modificado se copia de vuelta a la entrada en la tabla de páginas en memoria. Los otros valores ya están ahí, excepto el bit de referencia. Cuando el TLB se carga de la tabla de páginas, todos los campos se toman de la memoria.

Administración del TLB mediante software

Hasta ahora hemos supuesto que toda máquina con memoria virtual paginada tiene tablas de páginas reconocidas por el hardware, más un TLB. En este diseño, la administración y el manejo de fallas del TLB se realiza por completo mediante el hardware de la MMU. Las traps, o trampas, para el sistema operativo ocurren sólo cuando una página no se encuentra en memoria.

En el pasado, esta suposición era correcta. Sin embargo, muchas máquinas RISC modernas (incluyendo SPARC, MIPS y HP PA) hacen casi toda esta administración de páginas mediante software. En estas máquinas, las entradas del TLB se cargan de manera explícita mediante el sistema operativo. Cuando no se encuentra una coincidencia en el TLB, en vez de que la MMU vaya a las tablas de páginas para buscar y obtener la referencia a la página que se necesita, sólo genera un fallo del TLB y pasa el problema al sistema operativo. El sistema debe buscar la página, eliminar una entrada del TLB, introducir la nueva página y reiniciar la instrucción que originó el fallo. Y, desde luego, todo esto se debe realizar en unas cuantas instrucciones, ya que los fallos del TLB ocurren con mucha mayor frecuencia que los fallos de página.

De manera sorprendente, si el TLB es razonablemente grande (por ejemplo, de 64 entradas) para reducir la proporción de fallos, la administración del TLB mediante software resulta tener una eficiencia aceptable. El beneficio principal aquí es una MMU mucho más simple, lo cual libera una cantidad considerable de área en el chip de CPU para cachés y otras características que pueden mejorar el rendimiento. Uhlig y colaboradores (1994) describen la administración del TLB mediante software.

Se han desarrollado varias estrategias para mejorar el rendimiento en equipos que realizan la administración del TLB mediante software. Un método se enfoca en reducir los fallos del TLB y el costo de un fallo del TLB cuando llega a ocurrir (Bala y colaboradores, 1994). Para reducir los fallos del TLB, algunas veces el sistema operativo averigua de modo “intuitivo” cuáles páginas tienen más probabilidad de ser utilizadas a continuación y precarga entradas para ellas en el TLB. Por ejemplo, cuando un proceso cliente envía un mensaje a un proceso servidor en el mismo equipo, es muy probable que el servidor tenga que ejecutarse pronto. Sabiendo esto al tiempo que procesa el trap para realizar la operación `send`, el sistema también puede comprobar en dónde están las páginas de código, datos y pila del servidor, asociándolas antes de que tengan la oportunidad de producir fallos del TLB.

La forma normal de procesar un fallo del TLB, ya sea en hardware o en software, es ir a la tabla de páginas y realizar las operaciones de indexado para localizar la página referenciada. El problema al realizar esta búsqueda en software es que las páginas que contienen la tabla de páginas tal vez no estén en el TLB, lo cual producirá fallos adicionales en el TLB durante el procesamiento. Estos fallos se pueden reducir al mantener una caché grande en software (por ejemplo, de 4 KB) de entradas en el TLB en una ubicación fija, cuya página siempre se mantenga en el TLB. Al comprobar primero la caché de software, el sistema operativo puede reducir de manera substancial los fallos del TLB.

Cuando se utiliza la administración del TLB mediante software, es esencial comprender la diferencia entre los dos tipos de fallos. Un **fallo suave** ocurre cuando la página referenciada no está en el TLB, sino en memoria. Todo lo que se necesita aquí es que el TLB se actualice. No se necesita E/S de disco. Por lo general, un fallo suave requiere de 10 a 20 instrucciones de máquina y se puede completar en unos cuantos nanosegundos. Por el contrario, un **fallo duro** ocurre cuando la misma página no está en memoria (y desde luego, tampoco en el TLB). Se requiere un acceso al disco para traer la página, lo cual tarda varios milisegundos. Un fallo duro es en definitiva un millón de veces más lento que un fallo suave.

3.3.4 Tablas de páginas para memorias extensas

Los TLBs se pueden utilizar para acelerar las traducciones de direcciones virtuales a direcciones físicas sobre el esquema original de la tabla de páginas en memoria. Pero ése no es el único problema que debemos combatir. Otro problema es cómo lidiar con espacios de direcciones virtuales muy extensos. A continuación veremos dos maneras de hacerlo.

Tablas de páginas multinivel

Como primer método, considere el uso de una **tabla de páginas multinivel**. En la figura 3-13 se muestra un ejemplo simple. En la figura 3-13(a) tenemos una dirección virtual de 32 bits que se particiona en un campo *TP1* de 10 bits, un campo *TP2* de 10 bits y un campo *Desplazamiento* de 12 bits. Como los desplazamientos son de 12 bits, las páginas son de 4 KB y hay un total de 2^{20}.

El secreto del método de la tabla de páginas multinivel es evitar mantenerlas en memoria todo el tiempo, y en especial, aquellas que no se necesitan. Por ejemplo, suponga que un proceso necesita 12 megabytes: los 4 megabytes inferiores de memoria para el texto del programa, lo siguientes 4 megabytes para datos y los 4 megabytes superiores para la pila. Entre la parte superior de los datos y la parte inferior de la pila hay un hueco gigantesco que no se utiliza.

En la figura 3-13(b) podemos ver cómo funciona la tabla de página de dos niveles en este ejemplo. A la izquierda tenemos la tabla de páginas de nivel superior, con 1024 entradas, que corresponden al campo *TP1* de 10 bits. Cuando se presenta una dirección virtual a la MMU, primero extrae el campo *TP1* y utiliza este valor como índice en la tabla de páginas de nivel superior. Cada una de estas 1024 entradas representa 4 M, debido a que todo el espacio de direcciones virtuales de 4 gigabytes (es decir, de 32 bits) se ha dividido en trozos de 4096 bytes.

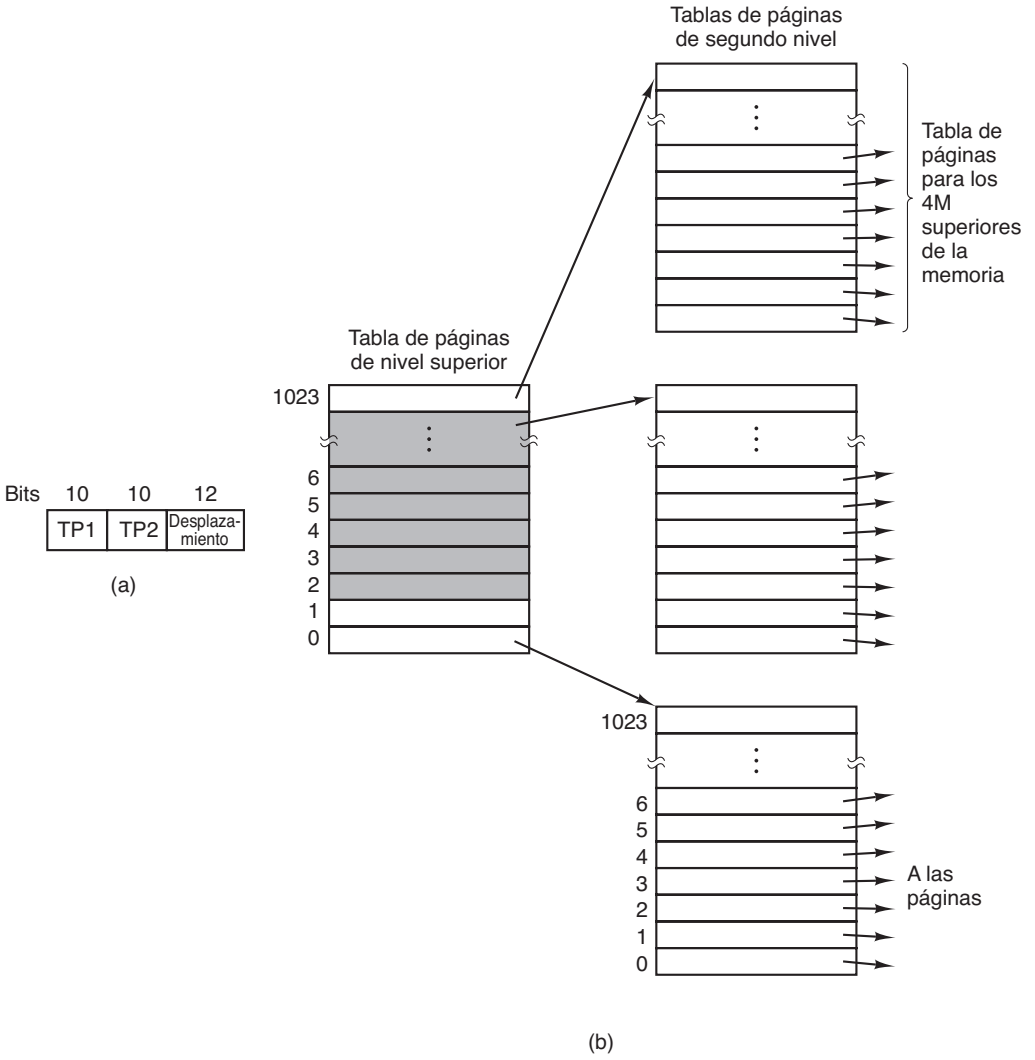


Figura 3-13. (a) Una dirección de 32 bits con dos campos de tablas de páginas. (b) Tablas de páginas de dos niveles.

La entrada que se localiza al indexar en la tabla de páginas de nivel superior produce la dirección (o número de marco de página) de una tabla de páginas de segundo nivel. La entrada 0 de la tabla de páginas de nivel superior apunta a la tabla de páginas para el texto del programa, la entrada 1 apunta a la tabla de páginas para los datos y la entrada 1023 apunta a la tabla de páginas para la pila. Las otras entradas (sombreadas) no se utilizan. Ahora el campo *TP2* se utiliza como índice en la tabla de páginas de segundo nivel seleccionada para buscar el número de marco de página para esta página en sí.

Como ejemplo, considere la dirección virtual de 32 bits 0x00403004 (4,206,596 decimal), que se encuentra 12,292 bytes dentro de los datos. Esta dirección virtual corresponde a *TP1* = 1,

TP2 = 2 y *Desplazamiento* = 4. La MMU utiliza primero a *TP1* para indexar en la tabla de páginas de nivel superior y obtener la entrada 1, que corresponde a las direcciones de 4M a 8M. Después utiliza *PT2* para indexar en la tabla de páginas de segundo nivel que acaba de encontrar y extrae la entrada 3, que corresponde a las direcciones de 12288 a 16383 dentro de su trozo de 4M (es decir, las direcciones absolutas de 4,206,592 a 4,210,687). Esta entrada contiene el número de marco de la página que contiene la dirección virtual 0x00403004. Si esa página no está en la memoria, el bit de *presente/ausente* en la entrada de la tabla de páginas será cero, con lo cual se producirá un fallo de página. Si la página está en la memoria, el número del marco de página que se obtiene de la tabla de páginas de segundo nivel se combina con el desplazamiento (4) para construir la dirección física. Esta dirección se coloca en el bus y se envía a la memoria.

Lo interesante acerca de la figura 3-13 es que, aunque el espacio de direcciones contiene más de un millón de páginas, en realidad sólo cuatro tablas de páginas son requeridas: la tabla de nivel superior y las tablas de segundo nivel de 0 a 4M (para el texto del programa), de 4M a 8M (para los datos) y los 4M superiores (para la pila). Los bits de *presente/ausente* en 1021 entradas de la tabla de páginas de nivel superior están en 0 y obligan a que se produzca un fallo de página si se tratan de utilizar alguna vez. En caso de que esto ocurra, el sistema operativo observará que el proceso está tratando de referenciar memoria que no debe y tomará la acción apropiada, como enviar una señal o eliminarlo. En este ejemplo hemos elegido números redondos para los diversos tamaños y que *TP1* sea igual a *TP2*, pero en la práctica también es posible elegir otros valores.

El sistema de tablas de páginas de dos niveles de la figura 3-13 se puede expandir a tres, cuatro o más niveles. Entre más niveles se obtiene una mayor flexibilidad, pero es improbable que la complejidad adicional sea de utilidad por encima de tres niveles.

Tablas de páginas invertidas

Para los espacios de direcciones virtuales de 32 bits, la tabla de páginas multinivel funciona bastante bien. Sin embargo, a medida que las computadoras de 64 bits se hacen más comunes, la situación cambia de manera drástica. Si el espacio de direcciones es de 2^{64} bytes, con páginas de 4 KB, necesitamos una tabla de páginas con 2^{52} entradas. Si cada entrada es de 8 bytes, la tabla es de más de 30 millones de gigabyes (30 PB). Ocupar 30 millones de gigabytes sólo para la tabla de páginas no es una buena idea por ahora y probablemente tampoco lo sea para el próximo año. En consecuencia, se necesita una solución diferente para los espacios de direcciones virtuales paginados de 64 bits.

Una de esas soluciones es la **tabla de páginas invertida**. En este diseño hay una entrada por cada marco de página en la memoria real, en vez de tener una entrada por página de espacio de direcciones virtuales. Por ejemplo, con direcciones virtuales de 64 bits, una página de 4 KB y 1 GB de RAM, una tabla de páginas invertida sólo requiere 262,144 entradas. La entrada lleva el registro de quién (proceso, página virtual) se encuentra en el marco de página.

Aunque las tablas de página invertidas ahorran grandes cantidades de espacio, al menos cuando el espacio de direcciones virtuales es mucho mayor que la memoria física, tienen una seria desventaja: la traducción de dirección virtual a dirección física se hace mucho más difícil. Cuando el proceso *n* hace referencia a la página virtual *p*, el hardware ya no puede buscar la página física usando *p* como índice en la tabla de páginas. En vez de ello, debe buscar una entrada (*n*, *p*) en toda la

tabla de páginas invertida. Lo que es peor: esta búsqueda se debe realizar en cada referencia a memoria, no sólo en los fallos de página. Buscar en una tabla de 256 K en cada referencia a memoria no es la manera de hacer que la máquina sea deslumbrantemente rápida.

La forma de salir de este dilema es utilizar el TLB. Si el TLB puede contener todas las páginas de uso frecuente, la traducción puede ocurrir con igual rapidez que con las tablas de páginas regulares. Sin embargo, en un fallo de TLB la tabla de páginas invertida tiene que buscarse mediante software. Una manera factible de realizar esta búsqueda es tener una tabla de hash arreglada según el hash de la dirección virtual. Todas las páginas virtuales que se encuentren en memoria y tengan el mismo valor de hash se encadenan en conjunto, como se muestra en la figura 3-14. Si la tabla de hash tiene tantas ranuras como las páginas físicas de la máquina, la cadena promedio tendrá sólo una entrada, con lo cual se acelera de manera considerable la asociación. Una vez que se ha encontrado el número de marco de página, se introduce el nuevo par (virtual, física) en el TLB.

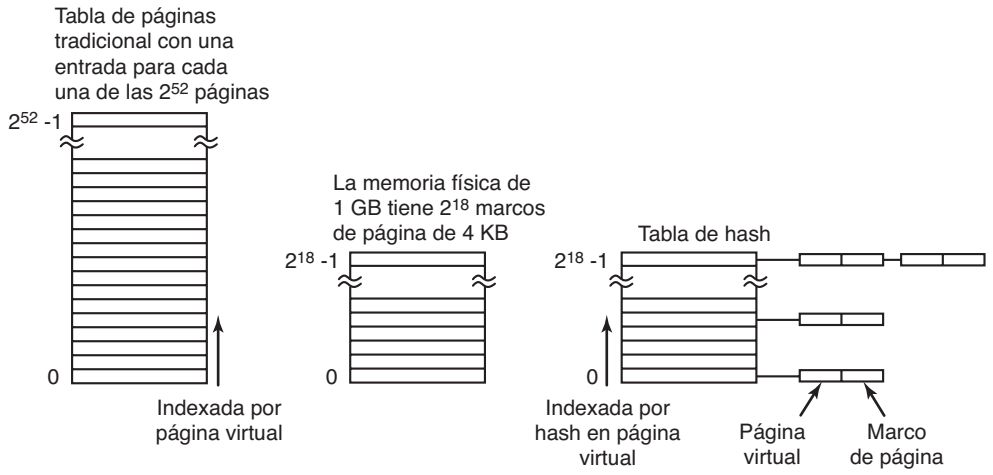


Figura 3-14. Comparación de una tabla de páginas tradicional con una tabla de páginas invertida.

Las tablas de páginas invertidas son comunes en las máquinas de 64 bits ya que incluso con un tamaño de página muy grande, el número de entradas en la tabla de páginas es enorme. Por ejemplo, con páginas de 4 MB y direcciones virtuales de 64 bits, se necesitan 2^{42} entradas en la tabla de páginas. Otros métodos para manejar memorias virtuales extensas se pueden encontrar en Talluri y colaboradores (1995).

3.4 ALGORITMOS DE REEMPLAZO DE PÁGINAS

Cuando ocurre un fallo de página, el sistema operativo tiene que elegir una página para desalojarla (eliminarla de memoria) y hacer espacio para la página entrante. Si la página a eliminar se modificó

mientras estaba en memoria, debe volver a escribirse en el disco para actualizar la copia del mismo. No obstante, si la página no se ha modificado (por ejemplo, si contiene el texto del programa), la copia ya está actualizada y no se necesita rescribir. La página que se va a leer sólo sobrescribe en la página que se va a desalojar.

Aunque sería posible elegir una página al azar para desalojarla en cada fallo de página, el rendimiento del sistema es mucho mayor si se selecciona una página que no sea de uso frecuente. Si se elimina una página de uso frecuente, tal vez tenga que traerse de vuelta rápidamente, lo cual produce una sobrecarga adicional. Se ha realizado mucho trabajo, tanto teórico como experimental en el tema de los algoritmos de reemplazo de páginas. A continuación describiremos algunos de los algoritmos más importantes.

Vale la pena observar que el problema del "reemplazo de páginas" ocurre también en otras áreas del diseño computacional. Por ejemplo, la mayoría de las computadoras tienen una o más memorias cachés que consisten en bloques de memoria de 32 bytes o 64 bytes de uso reciente. Cuando la caché está llena, hay que elegir cierto bloque para eliminarlo. Este problema es precisamente el mismo que la reemplazo de páginas, sólo que en una escala de tiempo menor (tiene que realizarse en unos cuantos nanosegundos, no milisegundos como en el reemplazo de páginas). La razón de tener una menor escala de tiempo es que los fallos de bloques de caché se satisfacen desde la memoria principal, que no tiene tiempo de búsqueda ni latencia rotacional.

Un segundo ejemplo es en un servidor Web. El servidor puede mantener cierto número de páginas Web de uso muy frecuente en su memoria caché. Sin embargo, cuando la memoria caché está llena y se hace referencia a una nueva página, hay que decidir cuál página Web se debe desalojar. Las consideraciones son similares a las de las páginas de memoria virtual, excepto por el hecho de que las páginas Web nunca se modifican en la caché, por lo cual siempre hay una copia fresca "en el disco". En un sistema de memoria virtual, las páginas en memoria principal pueden estar sucias o limpias.

En todos los algoritmos de reemplazo de páginas que analizaremos a continuación, surge cierta cuestión: al desalojar una página de la memoria, ¿tiene que ser una de las propias páginas del proceso fallido o puede ser una página que pertenezca a otro proceso? En el primer caso, estamos limitando en efecto cada proceso a un número fijo de páginas; en el segundo caso no. Ambas son posibilidades. En la sección 3-5.1 volveremos a ver este punto.

3.4.1 El algoritmo de reemplazo de páginas óptimo

El mejor algoritmo de reemplazo de páginas posible es fácil de describir, pero imposible de implementar: al momento en que ocurre un fallo de página, hay cierto conjunto de páginas en memoria y una de éstas se referenciará en la siguiente instrucción (la página que contiene la instrucción); otras páginas tal vez no se referencien sino hasta 10, 100 o tal vez 1000 instrucciones después. Cada página se puede etiquetar con el número de instrucciones que se ejecutarán antes de que se haga referencia por primera vez a esa página.

El algoritmo óptimo de reemplazo de páginas establece que la página con la etiqueta más alta debe eliminarse. Si una página no se va a utilizar durante 8 millones de instrucciones y otra no se va a utilizar durante 6 millones de instrucciones, al eliminar la primera se enviará el fallo de página que la obtendrá de vuelta lo más lejos posible en el futuro. Al igual que las personas, las computadoras tratan de posponer los eventos indeseables el mayor tiempo posible.

El único problema con este algoritmo es que no se puede realizar. Al momento del fallo de página, el sistema operativo no tiene forma de saber cuándo será la próxima referencia a cada una de las páginas (vimos una situación similar antes, con el algoritmo de planificación del trabajo más corto primero: ¿cómo puede saber el sistema cuál trabajo es más corto?). Aún así, al ejecutar un programa en un simulador y llevar la cuenta de todas las referencias a páginas, es posible implementar un algoritmo óptimo de reemplazo de página en la *segunda* corrida, al utilizar la información de referencia de páginas recolectada durante la *primera* corrida.

De esta manera, se puede comparar el rendimiento de los algoritmos realizables con el mejor posible. Si un sistema operativo logra un rendimiento de, por ejemplo, sólo 1 por ciento peor que el algoritmo óptimo, el esfuerzo invertido en buscar un mejor algoritmo producirá cuando mucho una mejora del 1 por ciento.

Para evitar cualquier posible confusión, hay que aclarar que este registro de referencias de páginas se refiere sólo al programa que se acaba de medir y sólo con una entrada específica. Por lo tanto, el algoritmo de reemplazo de páginas que se derive de esto es específico para ese programa y esos datos de entrada. Aunque este método es útil para evaluar los algoritmos de reemplazo de páginas, no es útil en sistemas prácticos. A continuación estudiaremos algoritmos que *son* útiles en sistemas reales.

3.4.2 El algoritmo de reemplazo de páginas: no usadas recientemente

Para permitir que el sistema operativo recolecte estadísticas útiles sobre el uso de páginas, la mayor parte de las computadoras con memoria virtual tienen dos bits de estado asociados a cada página. *R* se establece cada vez que se hace referencia a la página (lectura o escritura); *M* se establece cuando se escribe en la página (es decir, se modifica). Los bits están contenidos en cada entrada de la tabla de páginas, como se muestra en la figura 3-11. Es importante tener en cuenta que estos bits se deben actualizar en cada referencia a la memoria, por lo que es imprescindible que se establezcan mediante el hardware. Una vez que se establece un bit en 1, permanece así hasta que el sistema operativo lo restablece.

Si el hardware no tiene estos bits, pueden simularse de la siguiente forma. Cuando se inicia un proceso, todas sus entradas en la tabla de páginas se marcan como que no están en memoria. Tan pronto como se haga referencia a una página, ocurrirá un fallo de página. Entonces, el sistema operativo establece el bit *R* (en sus tablas internas), cambia la entrada en la tabla de páginas para que apunte a la página correcta, con modo de SÓLO LECTURA, y reinicia la instrucción. Si la página se modifica después, ocurrirá otro fallo de página que permite al sistema operativo establecer el bit *M* y cambiar el modo de la página a LECTURA/ESCRITURA.

Los bits *R* y *M* se pueden utilizar para construir un algoritmo simple de paginación de la siguiente manera. Cuando se inicia un proceso, ambos bits de página para todas sus páginas se establecen en 0 mediante el sistema operativo. El bit *R* se borra en forma periódica (en cada interrupción de reloj) para diferenciar las páginas a las que no se ha hecho referencia recientemente de las que si se han referenciado.

Cuando ocurre un fallo de página, el sistema operativo inspecciona todas las páginas y las divide en 4 categorías con base en los valores actuales de sus bits *R* y *M*:

Clase 0: no ha sido referenciada, no ha sido modificada.

Clase 1: no ha sido referenciada, ha sido modificada.

Clase 2: ha sido referenciada, no ha sido modificada.

Clase 3: ha sido referenciada, ha sido modificada.

Aunque las páginas de la clase 1 parecen a primera instancia imposibles, ocurren cuando una interrupción de reloj borra el bit *R* de una página de la clase 3. Las interrupciones de reloj no borran el bit *M* debido a que esta información se necesita para saber si la página se ha vuelto a escribir en el disco o no. Al borrar *R* pero no *M* se obtiene una página de clase 1.

El algoritmo **NRU** (*Not Recently Used*, No usada recientemente) elimina una página al azar de la clase de menor numeración que no esté vacía. En este algoritmo está implícita la idea de que es mejor eliminar una página modificada a la que no se haya hecho referencia en al menos un pulso de reloj (por lo general, unos 20 mseg) que una página limpia de uso frecuente. La principal atracción del NRU es que es fácil de comprender, moderadamente eficiente de implementar y proporciona un rendimiento que, aunque no es óptimo, puede ser adecuado.

3.4.3 El algoritmo de reemplazo de páginas: Primera en entrar, primera en salir (FIFO)

Otro algoritmo de paginación con baja sobrecarga es el de **Primera en entrar, primera en salir** (*First-In, First-Out*, **FIFO**). Para ilustrar cómo funciona, considere un supermercado con suficientes repisas como para mostrar exactamente *k* productos distintos. Un día, cierta empresa introduce un nuevo alimento de conveniencia: yogurt orgánico instantáneo liofilizado que se puede reconstituir en un horno de microondas. Es un éxito inmediato, por lo que nuestro supermercado finito se tiene que deshacer de un producto antiguo para tenerlo en existencia.

Una posibilidad es buscar el producto que el supermercado haya tenido en existencia por más tiempo (por ejemplo, algo que se empezó a vender hace 120 años) y deshacerse de él por la razón de que nadie está interesado ya. En efecto, el supermercado mantiene una lista ligada de todos los productos que vende actualmente en el orden en el que se introdujeron. El nuevo pasa a la parte final de la lista; el que está al frente de la lista se elimina.

Como un algoritmo de reemplazo de páginas, se aplica la misma idea. El sistema operativo mantiene una lista de todas las páginas actualmente en memoria, en donde la llegada más reciente está en la parte final y la menos reciente en la parte frontal. En un fallo de página, se elimina la página que está en la parte frontal y la nueva página se agrega a la parte final de la lista. Cuando se aplica en las tiendas, FIFO podría eliminar la gomina para el bigote, pero también podría eliminar harina, sal o mantequilla. Cuando se aplica a las computadoras, surge el mismo problema. Por esta razón es raro que se utilice FIFO en su forma pura.

3.4.4 El algoritmo de reemplazo de páginas: segunda oportunidad

Una modificación simple al algoritmo FIFO que evita el problema de descartar una página de uso frecuente es inspeccionar el bit *R* de la página más antigua. Si es 0, la página es antigua y no se ha

utilizado, por lo que se sustituye de inmediato. Si el bit *R* es 1, el bit se borra, la página se pone al final de la lista de páginas y su tiempo de carga se actualiza, como si acabara de llegar a la memoria. Después la búsqueda continúa.

La operación de este algoritmo, conocido como **segunda oportunidad**, se muestra en la figura 3-15. En la figura 3-15(a) vemos que las páginas de la *A* a la *H* se mantienen en una lista ligada y se ordenan con base en el tiempo en que llegaron a la memoria.

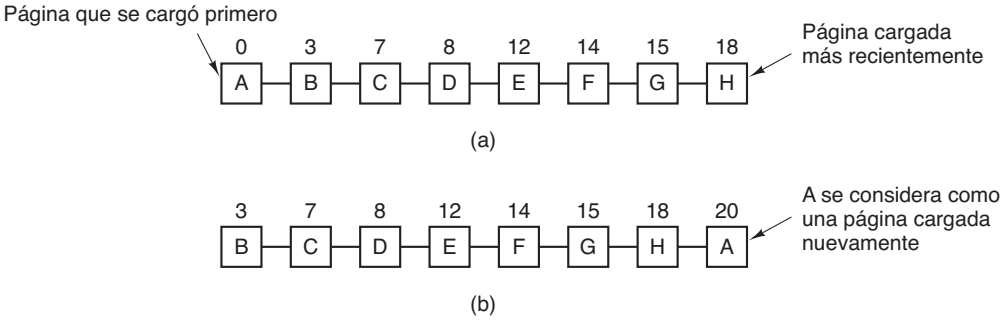


Figura 3-15. Operación del algoritmo de la segunda oportunidad. (a) Páginas ordenadas con base en FIFO. (b) Lista de las páginas si ocurre un fallo de página en el tiempo 20 y *A* tiene su bit *R* activado. Los números encima de las páginas son sus tiempos de carga.

Suponga que ocurre un fallo de página en el tiempo 20. La página más antigua es *A*, que llegó en el tiempo 0, cuando se inició el proceso. Si el bit *R* de *A* esta desactivado, se desaloja de la memoria, ya sea escribiéndola en el disco (si está sucia) o sólo se abandona (si está limpia). Por otro lado, si el bit *R* está activado, *A* se coloca al final de la lista y su "tiempo de carga" se restablece al tiempo actual (20). El bit *R* también esta desactivado. La búsqueda de una página adecuada continúa con *B*.

Lo que el algoritmo de segunda oportunidad está buscando es una página antigua a la que no se haya hecho referencia en el intervalo de reloj más reciente. Si se ha hecho referencia a todas las páginas, el algoritmo segunda oportunidad se degenera y se convierte en FIFO puro. De manera específica, imagine que todas las páginas en la figura 3-15(a) tienen sus bits *R* activados. Una por una, el sistema operativo mueve las páginas al final de la lista, desactivando el bit *R* cada vez que adjunta una página al final de la lista. En un momento dado regresará a la página *A*, que ahora tiene su bit *R* desactivado. En este punto, *A* se desaloja. Por ende, el algoritmo siempre termina.

3.4.5 El algoritmo de reemplazo de páginas: reloj

Aunque el algoritmo segunda oportunidad es razonable, también es innecesariamente ineficiente debido a que está moviendo constantemente páginas en su lista. Un mejor método sería mantener todos los marcos de página en una lista circular en forma de reloj, como se muestra en la figura 3-16. La manecilla apunta a la página más antigua.

Cuando ocurre un fallo de página, la página a la que apunta la manecilla se inspecciona. Si el bit *R* es 0, la página se desaloja, se inserta la nueva página en el reloj en su lugar y la manecilla se avanza

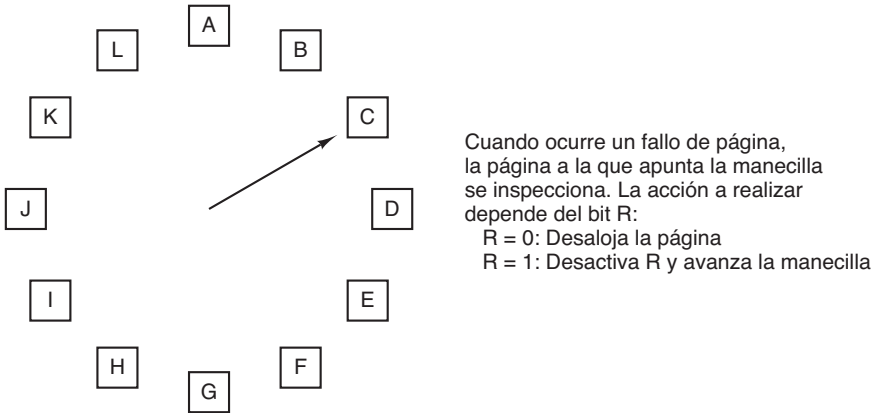


Figura 3-16. El algoritmo de reemplazo de páginas en reloj.

una posición. Si *R* es 1, se borra y la manecilla se avanza a la siguiente página. Este proceso se repite hasta encontrar una página con $R = 0$. No es de sorprender que a este algoritmo se le llame en **reloj**.

3.4.6 El algoritmo de reemplazo de páginas: menos usadas recientemente (LRU)

Una buena aproximación al algoritmo óptimo se basa en la observación de que las páginas que se hayan utilizado con frecuencia en las últimas instrucciones, tal vez se volverán a usar con frecuencia en las siguientes. Por el contrario, las páginas que no se hayan utilizado por mucho tiempo probablemente seguirán sin utilizarse por mucho tiempo más. Esta idea sugiere un algoritmo factible: cuando ocurra un fallo de página, hay que descartar la página que no se haya utilizado durante la mayor longitud de tiempo. A esta estrategia se le conoce como paginación **LRU** (*Least Recently Used*, Menos usadas recientemente).

Aunque en teoría el algoritmo LRU es realizable, no es barato. Para implementar el LRU por completo, es necesario mantener una lista enlazada de todas las páginas en memoria, con la página de uso más reciente en la parte frontal y la de uso menos reciente en la parte final. La dificultad es que la lista debe actualizarse en cada referencia a memoria. Buscar una página en la lista, eliminarla y después pasarla al frente es una operación que consume mucho tiempo, incluso mediante hardware (suponiendo que pudiera construirse dicho hardware).

Sin embargo, hay otras formas de implementar el algoritmo LRU con hardware especial. Consideremos primero la forma más simple. Este método requiere equipar el hardware con un contador de 64 bits, llamado *C*, el cual se incrementa de manera automática después de cada instrucción. Además, cada entrada en la tabla de páginas debe tener también un campo lo bastante grande como para poder guardar el contador. Después de cada referencia a memoria, el valor actual de *C* se almacena en la entrada en la tabla de páginas para la página que se acaba de referenciar. Cuando ocurre un fallo de página, el sistema operativo examina todos los contadores en la tabla de páginas para encontrar el menor. Esa página es la de uso menos reciente.

Ahora veamos un segundo algoritmo LRU mediante hardware. Para una máquina con *n* marcos de página, el hardware del LRU puede mantener una matriz de *n* x *n* bits (inicialmente, todos son 0). Cada vez que se hace referencia a la página *k*, el hardware primero establece todos los bits de la fila *k* en 1 y después todos los bits de la columna k en 0. En cualquier instante, la fila cuyo valor binario sea menor es la de uso menos reciente, la fila cuyo valor sea el siguiente más bajo es la del siguiente uso menos reciente, y así en lo sucesivo. El funcionamiento de este algoritmo se muestra en la figura 3-17 para cuatro marcos de página y referencias a páginas en el siguiente orden:

0 1 2 3 2 1 0 3 2 3

Después de hacer referencia a la página 0, tenemos la situación de la figura 3-17(a). Después de hacer referencia a la página 1, tenemos la situación de la figura 3-17(b), y así en lo sucesivo.

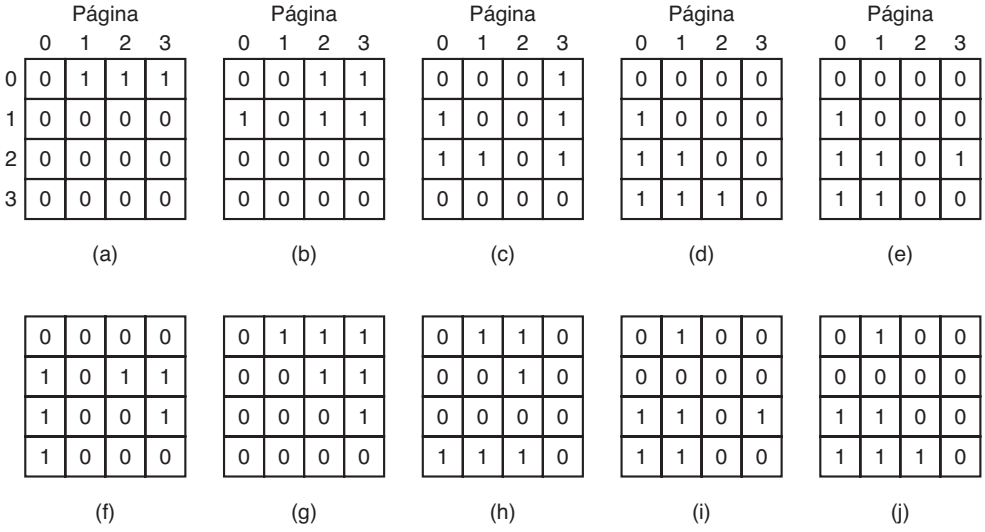


Figura 3-17. LRU usando una matriz cuando se hace referencia a las páginas en el orden 0, 1, 2, 3, 2, 1, 0, 3, 2, 3.

3.4.7 Simulación de LRU en software

Aunque los dos algoritmos LRU anteriores son (en principio) realizables, muy pocas máquinas (si acaso) tienen el hardware requerido. En vez de ello, se necesita una solución que puede implementarse en software. Una posibilidad es el algoritmo **NFU** (*Not Frequently Used*, No utilizadas frecuentemente). Este algoritmo requiere un contador de software asociado con cada página, que al principio es cero. En cada interrupción de reloj, el sistema operativo explora todas las páginas en memoria. Para cada página se agrega el bit *R*, que es 0 o 1, al contador. Los contadores llevan la cuenta aproximada de la frecuencia con que se hace referencia a cada página. Cuando ocurre un fallo de página, se selecciona la página con el contador que sea menor para sustituirla.

El principal problema con el algoritmo NFU es que nunca olvida nada. Por ejemplo, en un compilador con varias pasadas, las páginas que se utilizaron con frecuencia durante la pasada 1 podrían

tener todavía una cuenta alta en las siguientes pasadas. De hecho, si la pasada 1 tiene el tiempo de ejecución más largo de todas las pasadas, las páginas que contienen el código para las pasadas subsiguientes pueden tener siempre cuentas menores que las páginas de la pasada 1. En consecuencia, el sistema operativo eliminará páginas útiles, en vez de páginas que ya no se hayan utilizado.

Por fortuna, una pequeña modificación al algoritmo NFU le permite simular el algoritmo LRU muy bien. La modificación consta de dos partes. Primero, cada uno de los contadores se desplaza a la derecha 1 bit antes de agregar el bit *R*. Después, el bit *R* se agrega al bit de más a la izquierda, en lugar de sumarse al bit de más a la derecha.

La figura 3-18 ilustra cómo funciona el algoritmo modificado, conocido como **envejecimiento** (*aging*). Suponga que después del primer pulso de reloj los bits *R* para las páginas de la 0 a la 5 tienen los valores 1, 0, 1, 0, 1 y 1, respectivamente (la página 0 es 1, la página 1 es 0, la página 2 es 1, etc.). En otras palabras, entre los pulsos 0 y 1 se hizo referencia a las páginas 0, 2, 4 y 5, estableciendo sus bits *R* en 1, mientras que los de las otras páginas se quedaron en 0. Después de que se han desplazado los seis contadores correspondientes y el bit *R* se ha insertado a la izquierda, tienen los valores que se muestran en la figura 3-18(a). Las cuatro columnas restantes muestran los seis contadores después de los siguientes cuatro pulsos de reloj.

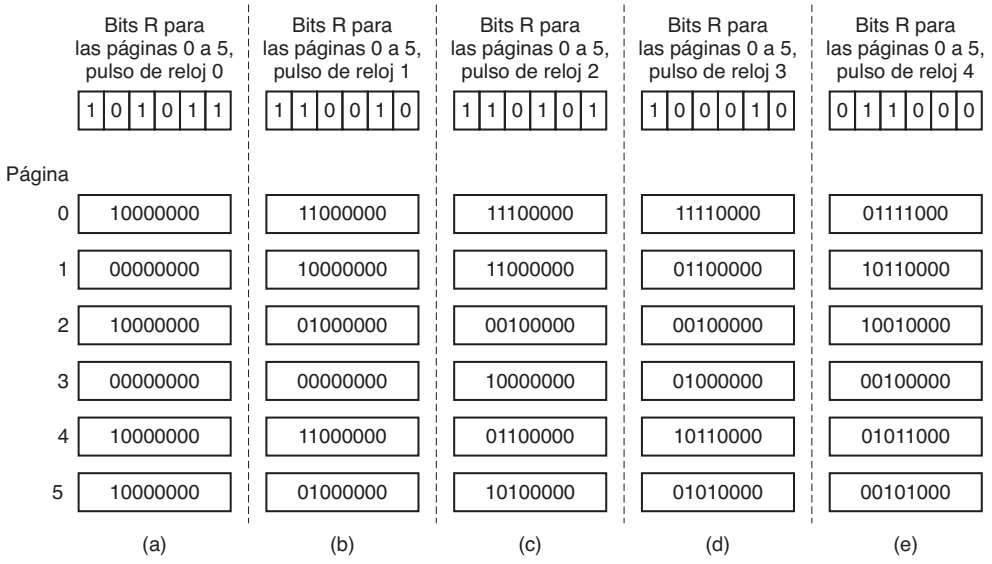


Figura 3-18. El algoritmo de envejecimiento simula el LRU en software. Aquí se muestran seis páginas para cinco pulsos de reloj. Los cinco pulsos de reloj se representan mediante los incisos (a) a (e).

Cuando ocurre un fallo de página, se elimina la página cuyo contador sea el menor. Está claro que una página a la que no se haya hecho referencia durante, por ejemplo, cuatro pulsos de reloj, tendrá cuatro ceros a la izquierda en su contador, y por ende tendrá un valor menor que un contador al que no se haya hecho referencia durante tres pulsos de reloj.

Este algoritmo difiere del de LRU en dos formas. Considere las páginas 3 y 5 en la figura 3-18(e). Ninguna se ha referenciado durante dos pulsos de reloj; ambas se referenciaron en el

pulso anterior a esos dos. De acuerdo con el algoritmo LRU, si hay que sustituir una página, debemos elegir una de estas dos. El problema es que no sabemos a cuál de ellas se hizo referencia por última vez en el intervalo entre el pulso 1 y el 2. Al registrar sólo un bit por cada intervalo de tiempo, hemos perdido la capacidad de diferenciar las referencias anteriores en el intervalo de reloj de las que ocurrieron después. Todo lo que podemos hacer es eliminar la página 3, debido a que también se hizo referencia a la página 5 dos pulsos de reloj antes, y a la página 3 no.

La segunda diferencia entre los algoritmos de LRU y envejecimiento es que en este último los contadores tienen un número finito de bits (8 en este ejemplo), lo cual limita su horizonte pasado. Suponga que dos páginas tienen cada una un valor de 0 en su contador; todo lo que podemos hacer es elegir una de ellas al azar. En realidad, bien podría ser una de las páginas a las que se haya hecho referencia nueve pulsos atrás, y la otra 1000 pulsos atrás. No tenemos forma de determinarlo. Sin embargo, en la práctica bastan 8 bits si un pulso de reloj es de aproximadamente 20 milisegundos. Si no se ha hecho referencia a una página en 160 milisegundos, tal vez no sea tan importante.

3.4.8 El algoritmo de reemplazo de páginas: conjunto de trabajo

En la forma más pura de paginación, los procesos inician sin ninguna de sus páginas en la memoria. Tan pronto como la CPU trata de obtener la primera instrucción, recibe un fallo de página, causando que el sistema operativo tenga que traer la página que contiene la primera instrucción. Por lo general a este fallo le siguen otros fallos de página por las variables globales y la pila. Después de cierto tiempo, el proceso tiene la mayoría de las páginas que necesita y se establece para ejecutarse con relativamente pocos fallos de página. A esta estrategia se le conoce como **paginación bajo demanda**, debido a que las páginas se cargan sólo según la demanda, no por adelantado.

Desde luego que es fácil escribir un programa de prueba que lea de manera sistemática todas las páginas en un espacio de direcciones extenso, produciendo tantos fallos de página que no haya suficiente memoria como para contenerlos todos. Por fortuna, la mayoría de los procesos no trabajan de esta manera. Exhiben una **localidad de referencia**, lo cual significa que durante cualquier fase de ejecución el proceso hace referencia sólo a una fracción relativamente pequeña de sus páginas. Por ejemplo, cada pasada de un compilador con varias pasadas hace referencia sólo a una fracción de todas las páginas y cada vez es una fracción distinta.

El conjunto de páginas que utiliza un proceso en un momento dado se conoce como su **conjunto de trabajo** (Denning, 1968a; Denning, 1980). Si todo el conjunto de trabajo está en memoria, el proceso se ejecutará sin producir muchos fallos hasta que avance a otra fase de ejecución (por ejemplo, la siguiente pasada del compilador). Si la memoria disponible es demasiado pequeña como para contener todo el conjunto de trabajo completo, el proceso producirá muchos fallos de página y se ejecutará lentamente, ya que para ejecutar una instrucción se requieren unos cuantos nanosegundos y para leer una página del disco se requieren en general 10 milisegundos. A una proporción de una o dos instrucciones por cada 10 milisegundos, se requeriría una eternidad para terminar. Se dice que un programa que produce fallos de página cada pocas instrucciones está **sobrepaginando** (***thrashing***) (Denning, 1968b).

En un sistema de multiprogramación, los procesos se mueven con frecuencia al disco (es decir, todas sus páginas se eliminan de la memoria) para dejar que otros procesos tengan oportunidad de

usar la CPU. Aquí surge la pregunta de qué hacer cuando un proceso se regresa una y otra vez. Técnicamente, no hay que hacer nada. El proceso producirá fallos de página sólo hasta que se haya cargado su conjunto de trabajo. El problema es que tener 20, 100 o incluso 1000 fallos de página cada vez que se carga un proceso es algo lento, además de que se desperdicia un tiempo considerable de la CPU, ya que el sistema operativo tarda unos cuantos milisegundos de tiempo de la CPU en procesar un fallo de página.

Por lo tanto, muchos sistemas de paginación tratan de llevar la cuenta del conjunto de trabajo de cada proceso y se aseguran que esté en memoria antes de permitir que el proceso se ejecute. Este método se conoce como **modelo del conjunto de trabajo** (Denning, 1970). Está diseñado para reducir en gran parte la proporción de fallos de página. Al proceso de cargar las páginas *antes* de permitir que se ejecuten los procesos también se le conoce como **prepaginación**. Tenga en cuenta que el conjunto de trabajo cambia con el tiempo.

Desde hace mucho se sabe que la mayor parte de los programas no hacen referencia a su espacio de direcciones de manera uniforme, sino que las referencias tienden a agruparse en un pequeño número de páginas. Una referencia a memoria puede obtener una instrucción, puede obtener datos o puede almacenar datos. En cualquier instante de tiempo *t*, existe un conjunto consistente de todas las páginas utilizadas por las *k* referencias a memoria más recientes. Este conjunto $w(k, t)$, es el de trabajo. Debido a que las $k = 1$ referencias más recientes deben haber utilizado todas las páginas utilizadas por las $k > 1$ referencias más recientes y tal vez otras, $w(k, t)$ es una función de *k* monotónicamente no decreciente. El límite de $w(k, t)$ a medida que *k* crece es finito, debido a que un programa no puede hacer referencia a más páginas de las que contiene su espacio de direcciones, y pocos programas utilizarán cada una de las páginas. La figura 3-19 muestra el tamaño del conjunto de trabajo como una función de *k*.

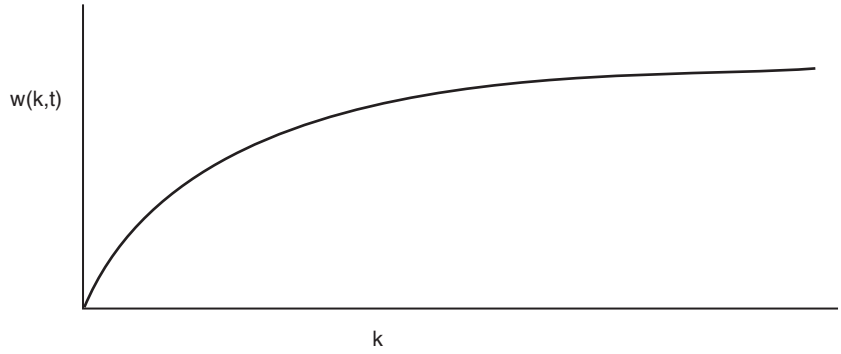


Figura 3-19. El conjunto de trabajo es el conjunto de páginas utilizadas por las *k* referencias a memoria más recientes. La función $w(k, t)$ es el tamaño del conjunto de trabajo en el tiempo *t*.

El hecho de que la mayoría de los programas acceden de manera aleatoria a un pequeño número de páginas, pero que este conjunto cambia lentamente con el tiempo, explica la rápida elevación inicial de la curva y después la lenta elevación para una *k* grande. Por ejemplo, un programa que ejecuta un ciclo que ocupa dos páginas utilizando datos en cuatro páginas podría hacer referencia a las seis páginas cada 1000 instrucciones, pero la referencia más reciente a alguna otra página po-

dría ser un millón de instrucciones antes, durante la fase de inicialización. Debido a este comportamiento asintótico, el contenido del conjunto de trabajo no es sensible al valor elegido de *k*. Dicho en forma distinta, existe un amplio rango de valores *k* para los cuales el conjunto de trabajo no cambia. Debido a que el conjunto de trabajo varía lentamente con el tiempo, es posible realizar una predicción razonable en cuanto a qué páginas se necesitarán cuando el programa se reinicie, con base en su conjunto de trabajo la última vez que se detuvo. La prepaginación consiste en cargar estas páginas antes de reanudar el proceso.

Para implementar el modelo del conjunto de trabajo, es necesario que el sistema operativo lleve la cuenta de cuáles páginas están en el conjunto de trabajo. Tener esta información también nos conduce de inmediato a un posible algoritmo de reemplazo de páginas: cuando ocurra un fallo de página, hay que buscar una página que no se encuentre en el conjunto de trabajo y desalojarla. Para implementar dicho algoritmo se requiere una manera precisa de determinar cuáles páginas están en el conjunto de trabajo. Por definición, el conjunto de trabajo es el conjunto de páginas utilizadas en las *k* referencias a memoria más recientes (algunos autores utilizan las *k* referencias a páginas más recientes, pero la elección es arbitraria). Para implementar cualquier algoritmo de conjunto de trabajo se debe elegir por adelantado cierto valor de *k*. Una vez que se ha seleccionado cierto valor, después de cada referencia a memoria, el conjunto de páginas utilizadas por las *k* referencias a memoria más recientes se determina en forma única.

Desde luego que tener una definición operacional del conjunto de trabajo no significa que haya una forma eficiente de calcularlo durante la ejecución de un programa. Uno podría imaginar un registro de desplazamiento de longitud *k*, donde cada referencia a memoria desplazará el registro una posición a la izquierda e insertará el número de página de referencia más reciente a la derecha. El conjunto de todos los *k* números de página en el registro de desplazamiento sería el conjunto de trabajo. En teoría, en un fallo de página, el contenido del registro de desplazamiento se podría extraer y ordenar; las páginas duplicadas entonces pueden ser eliminadas. El resultado sería el conjunto de trabajo. Sin embargo, el proceso de mantener el registro de desplazamiento y procesarlo en un fallo de página sería en extremo costoso, por lo que esta técnica nunca se utiliza.

En vez de ello, se utilizan varias aproximaciones. Una de uso común es desechar la idea de contar hacia atrás *k* referencias de memoria y usar en su defecto el tiempo de ejecución. Por ejemplo, en vez de definir el conjunto de trabajo como las páginas utilizadas durante los 10 millones de referencias a memoria anteriores, podemos definirlo como el conjunto de páginas utilizadas durante los últimos 100 milisegundos de tiempo de ejecución. En la práctica, dicha definición es igual de conveniente y es mucho más fácil trabajar con ella. Observe que para cada proceso sólo cuenta su propio tiempo de ejecución. Así, si un proceso empieza su ejecución en el tiempo T y ha tenido 40 milisegundos de tiempo de la CPU a un tiempo real de $T + 100$ milisegundos, para los fines del conjunto de trabajo su tiempo es de 40 mseg. La cantidad de tiempo de la CPU que ha utilizado en realidad un proceso desde que empezó se conoce comúnmente como su **tiempo virtual actual**. Con esta aproximación, el conjunto de trabajo de un proceso es el conjunto de páginas a las que se ha hecho referencia durante los últimos τ segundos de tiempo virtual.

Ahora veamos un algoritmo de reemplazo de páginas basado en el conjunto de trabajo. La idea básica es buscar una página que no esté en el conjunto de trabajo y desalojarla. En la figura 3-20 vemos una porción de una tabla de páginas para cierta máquina. Ya que sólo las páginas que están en memoria se consideran como candidatos para el desalojo, este algoritmo ignora las páginas que

no están en la memoria. Cada entrada contiene (al menos) dos elementos clave de información: el tiempo (aproximado) que se utilizó la página por última vez y el bit *R* (referenciada). El rectángulo vacío de color blanco simboliza los demás campos que no necesita este algoritmo, como el número de marco de página, los bits de protección y el bit *M* (modificado).

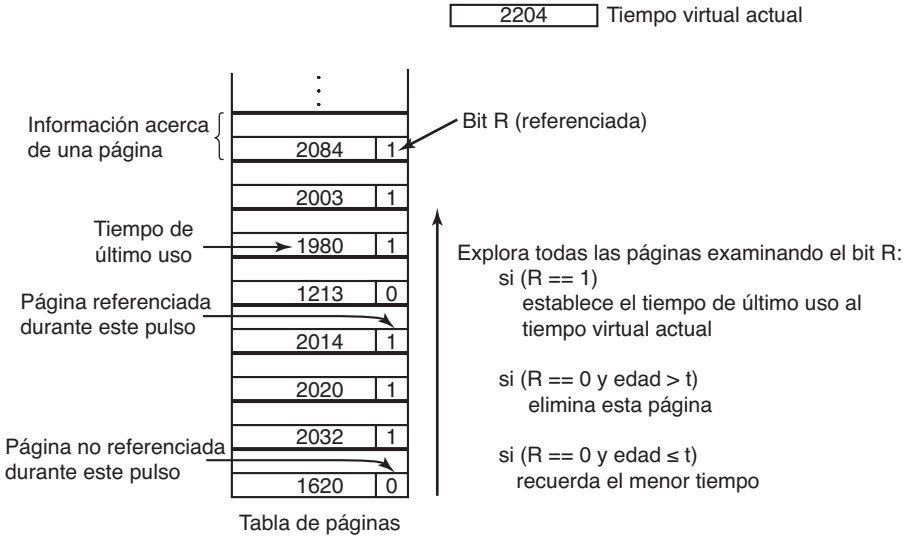


Figura 3-20. El algoritmo del conjunto de trabajo.

El algoritmo funciona de la siguiente manera. Se supone que el hardware debe establecer los bits *R* y *M*, como vimos antes. De manera similar, se supone que una interrupción periódica de reloj hace que se ejecute software para borrar el bit *Referenciada* en cada pulso de reloj. En cada fallo de página se explora la tabla de páginas en busca de una página adecuada para desalojarla.

A medida que se procesa cada entrada, se examina el bit *R*. Si es 1, el tiempo virtual actual se escribe en el campo *Tiempo de último uso* en la tabla de páginas, indicando que la página estaba en uso al momento en que ocurrió el fallo de página. Como se hizo referencia a la página durante el pulso de reloj actual, es evidente que está en el conjunto de trabajo y no es candidata para la eliminación (se supone que τ abarca varios pulsos de reloj).

Si *R* es 0, no se ha hecho referencia a la página durante el pulso de reloj actual y puede ser candidata para la eliminación. Para ver si debe o no eliminarse, se calcula su edad (el tiempo virtual actual menos su *Tiempo de último uso*) y se compara con τ. Si la edad es mayor que τ, la página ya no se encuentra en el conjunto de trabajo y la nueva página la sustituye. La exploración continúa actualizando las entradas restantes.

No obstante, si *R* es 0 pero la edad es menor o igual que τ, la página está todavía en el conjunto de trabajo. La página se reserva temporalmente, pero se apunta la página con la mayor edad (el menor valor de *Tiempo de último uso*). Si toda la tabla completa se explora sin encontrar un candidato para desalojar, eso significa que todas las páginas están en el conjunto de trabajo. En ese caso, si se encontraron una o más páginas con $R = 0$, se desaloja la más antigua. En el peor caso, se ha hecho

referencia a todas las páginas durante el pulso de reloj actual (y por ende, todas tienen $R = 1$), por lo que se selecciona una al azar para eliminarla, de preferencia una página limpia, si es que existe.

3.4.9 El algoritmo de reemplazo de páginas WSClock

Al algoritmo básico del conjunto de trabajo es incómodo ya que exige explorar toda la tabla de páginas en cada fallo de página hasta localizar un candidato adecuado. Un algoritmo mejorado, basado en el algoritmo de reloj pero que también utiliza la información del conjunto de trabajo, se conoce como **WSClock** (Carr y Hennessey, 1981). Debido a su simplicidad de implementación y buen rendimiento, es muy utilizado en la práctica.

La estructura de datos necesaria es una lista circular de marcos de página, como en el algoritmo de reloj, mostrada en la figura 3-21(a). Al principio, esta lista está vacía. Cuando se carga la primera página, se agrega a la lista. A medida que se agregan más páginas, pasan a la lista para formar un anillo. Cada entrada contiene el campo *Tiempo de último uso* del algoritmo básico del conjunto de trabajo, así como el bit *R* (mostrado) y el bit *M* (no mostrado).

Al igual que con el algoritmo de reloj, en cada fallo de página se examina primero la página a la que apunta la manecilla. Si el bit *R* es 1, la página se ha utilizado durante el pulso actual, por lo que no es candidata ideal para la eliminación. Después el bit *R* se establece en 0, la manecilla se avanza a la siguiente página y se repite el algoritmo para esa página. El estado después de esta secuencia de eventos se muestra en la figura 3-21(b).

Ahora considere lo que ocurre si la página a la que apunta la manecilla tiene $R = 0$, como se muestra en la figura 3-21(c). Si la edad es mayor que τ y la página está limpia, significa que no se encuentra en el conjunto de trabajo y existe una copia válida en el disco. El marco de página simplemente se reclama y la nueva página se coloca ahí, como se muestra en la figura 3-21(d). Por otro lado, si la página está sucia no se puede reclamar de inmediato, ya que no hay una copia válida presente en el disco. Para evitar una conmutación de procesos, la escritura al disco se planifica pero la manecilla avanza y el algoritmo continúa con la siguiente página. Después de todo, podría haber una página antigua y limpia más allá de la línea que se pueda utilizar de inmediato.

En principio, todas las páginas se podrían planificar para la E/S de disco en un ciclo alrededor del reloj. Para reducir el tráfico de disco se podría establecer un límite para permitir que se escriban de vuelta un máximo de *n* páginas. Una vez que se llega a este límite, no se planifican nuevas escrituras.

¿Qué ocurre si la manecilla llega otra vez a su punto inicial? Hay dos casos a considerar:

1. Se ha planificado por lo menos una escritura.
2. No se han planificado escrituras.

En el primer caso, la manecilla sólo sigue moviéndose, buscando una página limpia. Como se han planificado una o más escrituras, en algún momento se completará alguna escritura y su página se marcará como limpia. La primera página limpia que se encuentre se desaloja. Esta página no es necesariamente la primera escritura planificada, ya que el controlador de disco puede reordenar escrituras para poder optimizar el rendimiento del disco.

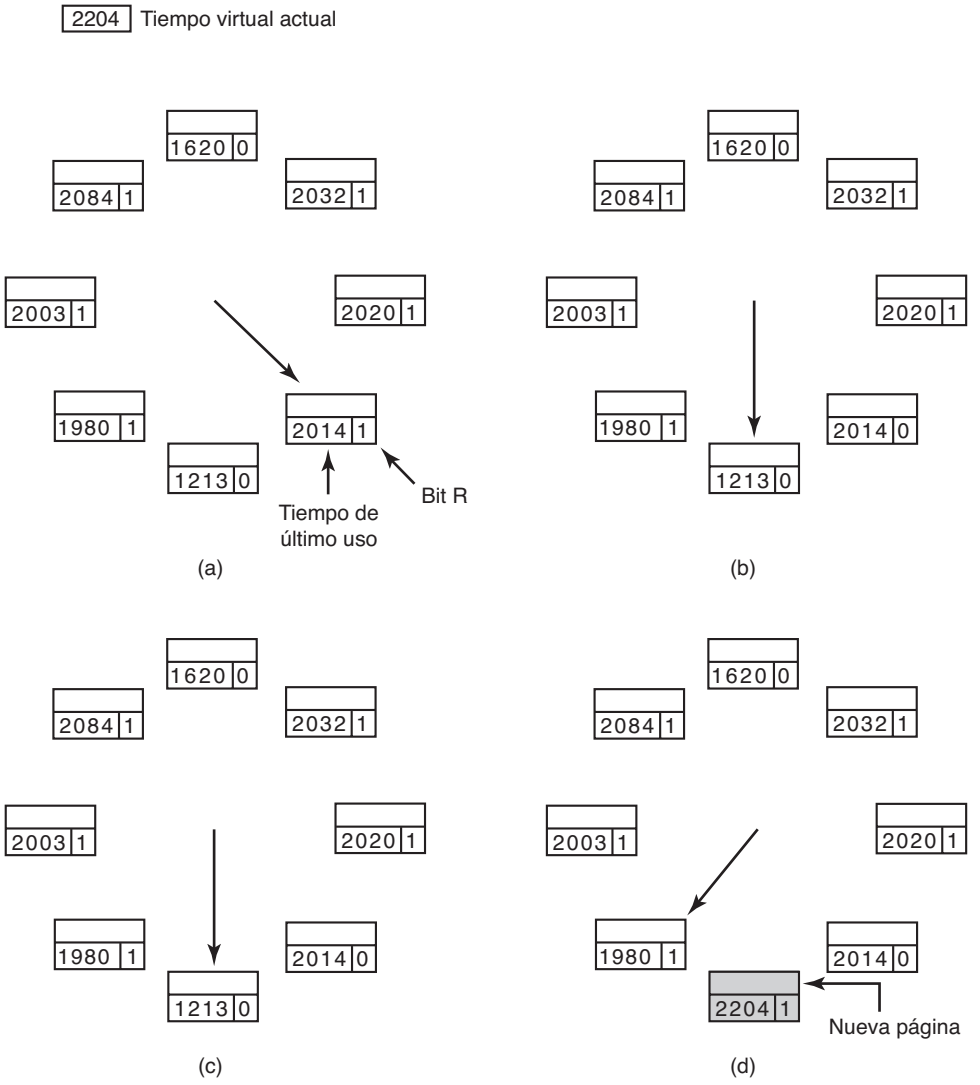


Figura 3-21. Operación del algoritmo WSClock. (a) y (b) dan un ejemplo de lo que ocurre cuando $R = 1$. (c) y (d) dan un ejemplo de cuando $R = 0$.

En el segundo caso, todas las páginas están en el conjunto de trabajo, de otra manera se hubiera planificado por lo menos una escritura. Sin información adicional, lo más simple por hacer es reclamar cualquier página limpia y usarla. La ubicación de una página limpia podría rastrearse durante el barrido. Si no existen páginas limpias, entonces se selecciona la página actual como la víctima y se escribe de vuelta al disco.

3.4.10 Resumen de los algoritmos de reemplazo de páginas

Ahora hemos visto una variedad de algoritmos de reemplazo de páginas. En esta sección mostraremos un breve resumen de ellos. La lista de algoritmos descritos se proporciona en la figura 3-22.

Algoritmo	Comentario
Óptimo	No se puede implementar, pero es útil como punto de comparación
NRU (No usadas recientemente)	Una aproximación muy burda del LRU
FIFO (primera en entrar, primera en salir)	Podría descartar páginas importantes
Segunda oportunidad	Gran mejora sobre FIFO
Reloj	Realista
LRU (menos usadas recientemente)	Excelente, pero difícil de implementar con exactitud
NFU (no utilizadas frecuentemente)	Aproximación a LRU bastante burda
Envejecimiento	Algoritmo eficiente que se aproxima bien a LRU
Conjunto de trabajo	Muy costoso de implementar
WSClock	Algoritmo eficientemente bueno

Figura 3-22. Algoritmos de reemplazo de páginas descritos en el texto.

El algoritmo óptimo desaloja la página a la que se hará referencia en el futuro más lejano. Por desgracia, no hay forma de determinar cuál página es, por lo que en la práctica no se puede utilizar este algoritmo. Sin embargo, es útil como punto de comparación para los demás algoritmos.

El algoritmo NRU divide las páginas en cuatro clases dependiendo del estado de los bits *R* y *M*. Se selecciona una página aleatoria de la clase con menor numeración. Este algoritmo es fácil de implementar, pero muy burdo. Existen mejores.

FIFO lleva la cuenta del orden en el que se cargaron las páginas en memoria al mantenerlas en una lista enlazada. Eliminar la página más antigua se vuelve entonces un proceso trivial, pero como esa página podría estar todavía en uso, FIFO es una mala opción.

El algoritmo de segunda oportunidad es una modificación de FIFO que comprueba si hay una página en uso antes de eliminarla. Si lo está, la página se reserva. Esta modificación mejora de manera considerable el rendimiento. El algoritmo de reloj es simplemente una implementación distinta del algoritmo de segunda oportunidad. Tiene las mismas propiedades de rendimiento, pero toma un poco menos de tiempo para ejecutar el algoritmo.

LRU es un algoritmo excelente, pero no se puede implementar sin hardware especial. NFU es un burdo intento por aproximarse a LRU; sin embargo, el algoritmo de envejecimiento es una mucho mejor aproximación a LRU y se puede implementar con eficiencia. Es una buena opción.

Los últimos dos algoritmos utilizan el conjunto de trabajo. El algoritmo del conjunto de trabajo ofrece un rendimiento razonable, pero es un poco costoso de implementar. WSClock es una variante que no sólo da un buen rendimiento, sino que también es eficiente al implementar.

Con todo, los dos mejores algoritmos son el de envejecimiento y WSClock. Se basan en LRU y el conjunto de trabajo, respectivamente. Ambos dan un buen rendimiento en la paginación y pueden implementarse con eficiencia. Existen varios algoritmos más, pero estos dos son tal vez los más importantes en la práctica.

3.5 CUESTIONES DE DISEÑO PARA LOS SISTEMAS DE PAGINACIÓN

En las secciones anteriores hemos explicado cómo funciona la paginación y hemos analizado unos cuantos de los algoritmos básicos de reemplazo de páginas, además de mostrar cómo modelarlos. Pero no basta con conocer la mecánica básica. Para diseñar un sistema hay que saber mucho más para hacer que funcione bien. Es como la diferencia que hay entre saber mover las piezas de ajedrez y ser un buen jugador. En la siguientes secciones analizaremos otras cuestiones que deben considerar los diseñadores de sistemas operativos para poder obtener un buen rendimiento de un sistema de paginación.

3.5.1 Políticas de asignación local contra las de asignación global

En las secciones anteriores hemos analizado varios algoritmos para seleccionar una página y sustituirla al ocurrir un fallo. Una cuestión importante asociada con esta elección (que hemos dejado de lado cuidadosamente hasta ahora) es cómo se debe asignar la memoria entre los procesos ejecutables en competencia.

Dé un vistazo a la figura 3-23(a). En esta figura, tres procesos (*A*, *B* y *C*) componen el conjunto de procesos ejecutables. Suponga que *A* obtiene un fallo de página. ¿Debe el algoritmo de reemplazo de páginas tratar de encontrar la página de uso menos reciente, considerando sólo las seis páginas que están actualmente asignadas a *A* o debe considerar todas las páginas en memoria? Si sólo examina las páginas asignadas a *A*, la página con el menor valor de edad es *A5*, por lo que obtenemos la situación de la figura 3-23(b).

Por otro lado, si se va a eliminar la página con el menor valor de edad sin importar de quién sea, se elegirá la página *B3* y obtendremos la situación de la figura 3-23(c). Al algoritmo de la figura 3-23(b) se le considera de reemplazo de páginas **local**, mientras que al de la figura 3-23(c), un algoritmo **global**. Los algoritmos locales corresponden de manera efectiva a asignar a cada proceso una fracción fija de la memoria. Los algoritmos globales asignan marcos de página de manera dinámica entre los procesos ejecutables. Así, el número de marcos de página asignados a cada proceso varía en el tiempo.

En general, los algoritmos globales funcionan mejor, en especial cuando el tamaño del conjunto de trabajo puede variar durante el tiempo de vida de un proceso. Si se utiliza un algoritmo local y el conjunto de trabajo crece, se producirá una sobrepaginación, aun cuando haya muchos marcos de página libres. Si el conjunto de trabajo se reduce, los algoritmos locales desperdician memoria. Si se utiliza un algoritmo global, el sistema debe decidir en forma continua cuántos marcos de página asignar a cada proceso. Una manera es supervisar el tamaño del conjunto de trabajo, según lo

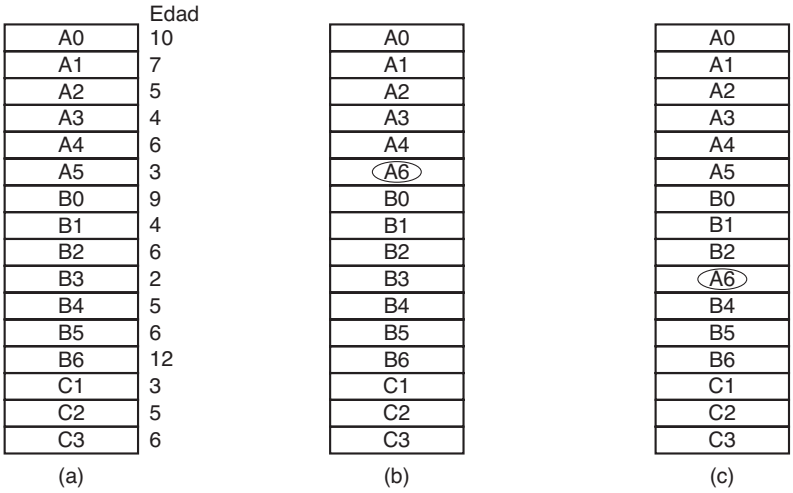


Figura 3-23. Comparación entre el reemplazo de páginas local y el global. (a) Configuración original. (b) Reemplazo de páginas local. (c) Reemplazo de páginas global.

indicado por los bits de envejecimiento, pero este método no necesariamente evita la sobrepaginación. El conjunto de trabajo puede cambiar de tamaño en microsegundos, mientras que los bits de envejecimiento son una medida burda, esparcida a través de varios pulsos de reloj.

Otro método es tener un algoritmo para asignar marcos de página a los procesos. Una manera es determinar periódicamente el número de procesos en ejecución y asignar a cada proceso una parte igual. Así, con 12,416 marcos de página disponibles (es decir, que no son del sistema operativo) y 10 procesos, cada proceso obtiene 1241 marcos. Los seis restantes pasan a una reserva, para utilizarlos cuando ocurran fallos de página.

Aunque este método parece equitativo, tiene poco sentido otorgar partes iguales de la memoria a un proceso de 10 KB y un proceso de 300 KB. En vez de ello, se pueden asignar páginas en proporción al tamaño total de cada proceso, donde un proceso de 300 KB obtiene 30 veces la asignación de un proceso de 10 KB. Probablemente sea prudente dar a cada proceso cierto número mínimo, de manera que se pueda ejecutar sin importar qué tan pequeño sea. Por ejemplo, en algunas máquinas una sola instrucción de dos operandos puede necesitar hasta seis páginas, debido a que la misma instrucción, el operando de origen y el operando de destino pueden extenderse a través de los límites de las páginas. Con una asignación de sólo cinco páginas, los programas que contengan tales instrucciones no se podrán ejecutar.

Si se utiliza un algoritmo global, es posible empezar cada proceso con cierto número de páginas proporcional al tamaño del proceso, pero la asignación se tiene que actualizar dinámicamente a medida que se ejecuten los procesos. Una manera de administrar la asignación es utilizando el algoritmo **PFF** (*Page Fault Frequency*, Frecuencia de fallo de páginas). Este algoritmo indica cuándo se debe incrementar o decrementar la asignación de páginas a un proceso, pero no dice nada acerca de cuál página se debe sustituir en un fallo. Sólo controla el tamaño del conjunto de asignación.

Para una clase extensa de algoritmos de reemplazo de páginas, incluyendo LRU, se sabe que la proporción de fallos disminuye a medida que se asignan más páginas, como lo describimos anteriormente. Ésta es la suposición detrás del algoritmo PFF. Esta propiedad se ilustra en la figura 3-24.

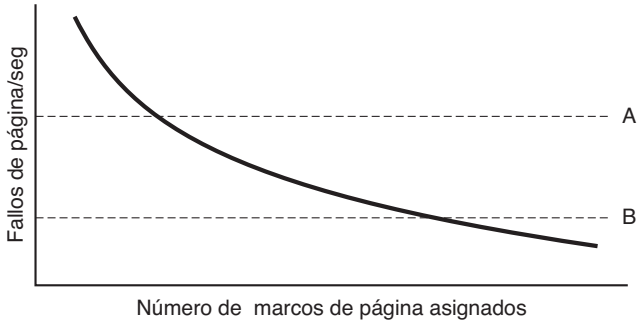


Figura 3-24. Proporción de fallos de página como una función del número de marcos de página asignados.

Medir la proporción de fallos de página es un proceso simple: sólo se cuenta el número de fallos por segundo, posiblemente tomando una media móvil sobre los segundos transcurridos también. Una manera sencilla de hacer esto es sumar el número de fallos de página durante el segundo inmediatamente anterior a la media móvil actual y dividir entre dos. La línea punteada marcada como *A* corresponde a una proporción de fallos de página que es demasiado alta, por lo que el proceso que emitió el fallo recibe más marcos de página para reducir la proporción de fallos. La línea punteada marcada como *B* corresponde a una proporción de fallos de página tan baja que podemos suponer que el proceso tiene demasiada memoria. En este caso se le pueden quitar marcos de página. Así, el algoritmo PFF trata de mantener la proporción de paginación para cada proceso dentro de límites aceptables.

Es importante recalcar que ciertos algoritmos de reemplazo de páginas pueden funcionar con una política de sustitución local o con una global. Por ejemplo, FIFO puede sustituir la página más antigua en toda la memoria (algoritmo global) o la más antigua que posea el proceso actual (algoritmo local). De manera similar, el algoritmo LRU o alguna aproximación a éste puede reemplazar la página usada menos recientemente en toda la memoria (algoritmo global) o la página menos usada recientemente poseída por el proceso actual (algoritmo local). La elección entre local y global es independiente del algoritmo en algunos casos.

Por otro lado, para los demás algoritmos de sustitución de página, sólo tiene sentido una estrategia local. En especial, los algoritmos del conjunto de trabajo y WSClock se refieren a un proceso específico y deben aplicarse en este contexto. En realidad no hay un conjunto de trabajo para la máquina como un todo, y al tratar de usar la unión de todos los conjuntos de trabajo se perdería la propiedad de localidad, y no funcionaría bien.

3.5.2 Control de carga

Aun con el mejor algoritmo de reemplazo de páginas y una asignación global óptima de marcos de página a los procesos, puede ocurrir que el sistema se sobrepagine. De hecho, cada vez que los con-

juntos de trabajo combinados de todos los procesos exceden a la capacidad de la memoria, se puede esperar la sobrepaginación. Un síntoma de esta situación es que el algoritmo PFF indica que algunos procesos necesitan más memoria, pero ningún proceso necesita menos memoria. En este caso no hay forma de proporcionar más memoria a esos procesos que la necesitan sin lastimar a algún otro proceso. La única solución real es deshacerse temporalmente de algunos procesos.

Una buena forma de reducir el número de procesos que compiten por la memoria es intercambiar algunos de ellos enviándolos al disco y liberar todas las páginas que ellos mantienen. Por ejemplo, un proceso puede intercambiarse al disco y sus marcos de página dividirse entre otros procesos que están sobrepaginando. Si el sobrepaginado se detiene, el sistema puede operar de esta forma por un tiempo. Si no se detiene hay que intercambiar otro proceso y así en lo sucesivo hasta que se detenga el sobrepaginado. Por ende, incluso hasta con la paginación se sigue necesitando el intercambio, sólo que ahora se utiliza para reducir la demanda potencial de memoria, en vez de reclamar páginas.

El proceso de intercambiar procesos para liberar la carga en la memoria es semejante a la planificación de dos niveles, donde ciertos procesos se colocan en disco y se utiliza un planificador de corto plazo para planificar los procesos restantes. Sin duda se pueden combinar las dos ideas donde se intercambien, fuera de memoria, sólo los procesos suficientes para hacer que la proporción de fallo de páginas sea aceptable. Periódicamente, ciertos procesos se traen del disco y otros se intercambian hacia el mismo.

Sin embargo, otro factor a considerar es el grado de multiprogramación. Cuando el número de procesos en la memoria principal es demasiado bajo, la CPU puede estar inactiva durante largos periodos. Esta consideración sostiene que no sólo se debe tomar en cuenta el tamaño del proceso y la proporción de paginación al decidir qué proceso se debe intercambiar, sino también sus características, tal como si está ligado a la CPU o a la E/S, así como las características que tienen los procesos restantes.

3.5.3 Tamaño de página

El tamaño de página es un parámetro que a menudo el sistema operativo puede elegir. Incluso si el hardware se ha diseñado, por ejemplo, con páginas de 512 bytes, el sistema operativo puede considerar fácilmente los pares de páginas 0 y 1, 2 y 3, 4 y 5, y así en lo sucesivo, como páginas de 1 KB al asignar siempre dos marcos de página de 512 bytes consecutivos para ellas.

Para determinar el mejor tamaño de página se requiere balancear varios factores competitivos. Como resultado, no hay un tamaño óptimo en general. Para empezar, hay dos factores que están a favor de un tamaño de página pequeño. Un segmento de texto, datos o pila elegido al azar no llenará un número integral de páginas. En promedio, la mitad de la página final estará vacía. El espacio adicional en esa página se desperdicia. A este desperdicio se le conoce como **fragmentación interna**. Con n segmentos en memoria y un tamaño de página de p bytes, se desperdiciarán $np/2$ bytes en fragmentación interna. Este razonamiento está a favor de un tamaño de página pequeño.

Otro argumento para un tamaño de página pequeño se hace aparente si consideramos que un programa consiste de ocho fases secuenciales de 4 KB cada una. Con un tamaño de página de 32 KB, se deben asignar 32 KB al programa todo el tiempo. Con un tamaño de página de 16 KB,

sólo necesita 16 KB. Con un tamaño de página de 4 KB o menor, sólo requiere 4 KB en cualquier instante. En general, un tamaño de página grande hará que haya una parte más grande no utilizada del programa que un tamaño de página pequeño.

Por otro lado, tener páginas pequeñas implica que los programas necesitarán muchas páginas, lo que sugiere la necesidad de una tabla de páginas grande. Un programa de 32 KB necesita sólo cuatro páginas de 8 KB, pero 64 páginas de 512 bytes. Las transferencias hacia y desde el disco son por lo general de una página a la vez, y la mayor parte del tiempo se debe al retraso de búsqueda y al retraso rotacional, por lo que para transferir una página pequeña se requiere casi el mismo tiempo que para transferir una página grande. Se podran requerir 64 x 10 mseg para cargar 64 páginas de 512 bytes, pero sólo 4 x 12 mseg para cargar cuatro páginas de 8 KB.

En algunas máquinas, la tabla de páginas se debe cargar en registros de hardware cada vez que la CPU cambia de un proceso a otro. En estas máquinas, tener un tamaño pequeño de página significa que el tiempo requerido para cargar sus registros aumenta a medida que se hace más pequeña. Además, el espacio ocupado por la tabla de páginas aumenta a medida que se reduce el tamaño de las páginas.

Este último punto se puede analizar matemáticamente. Digamos que el tamaño promedio de un proceso es de s bytes y que el tamaño de página es de p bytes. Además supone que cada entrada de página requiere e bytes. El número aproximado de páginas necesarias por proceso es entonces s/p, ocupando se/p bytes de espacio en la tabla de páginas. La memoria desperdiciada en la última página del proceso debido a la fragmentación interna es $p/2$. Así, la sobrecarga total debido a la tabla de páginas y a la pérdida por fragmentación interna se obtiene mediante la suma de estos dos términos:

$$\text{sobrecarga} = se/p + p/2$$

El primer término (tamaño de la tabla de páginas) es grande cuando el tamaño de página es pequeño. El segundo término (fragmentación interna) es grande cuando el tamaño de página es grande. El valor óptimo debe estar entre estos dos. Al sacar la primera derivada con respecto a p e igualarla a cero, obtenemos la ecuación

$$-se/p^2 + 1/2 = 0$$

De esta ecuación podemos derivar una fórmula que proporcione el tamaño de página óptimo (considerando sólo la memoria gastada en la fragmentación y el tamaño de la tabla de páginas). El resultado es:

$$p = \sqrt{2se}$$

Para $s = 1$ MB y $e = 8$ bytes por cada entrada en la tabla de páginas, el tamaño de página óptimo es de 4 KB. Las computadoras disponibles en forma comercial han utilizado tamaños de página que varían desde 512 bytes hasta 64 KB. Un valor común solía ser 1 KB, pero hoy en día es más común tener 4 KB u 8 KB. A medida que las memorias aumentan su tamaño, el tamaño de página tiende a crecer también (pero no en forma lineal). Cuadruplicar el tamaño de la RAM rara vez duplica siquiera el tamaño de página.

3.5.4 Espacios separados de instrucciones y de datos

La mayor parte de las computadoras tienen un solo espacio de direcciones que contiene tanto programas como datos, como se muestra en la figura 3-25(a). Si este espacio de direcciones es lo bastante grande, todo funciona bien. No obstante, a menudo es demasiado pequeño, lo cual obliga a los programadores a pararse de cabeza tratando de ajustar todo en el espacio de direcciones.

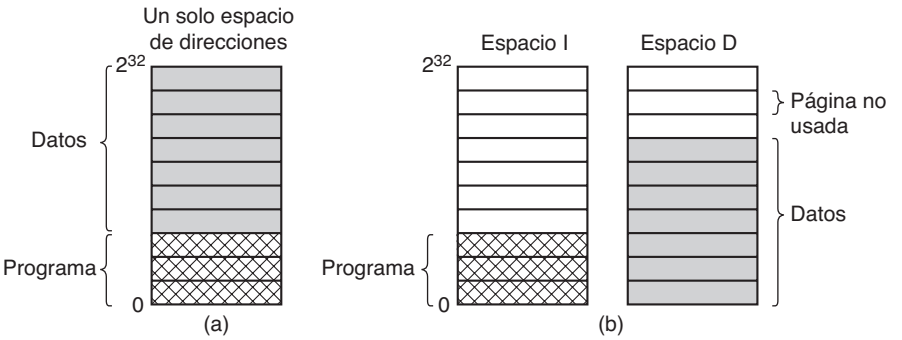


Figura 3-25. (a) Un espacio de direcciones. (b) Espacios I y D separados.

Una solución utilizada por primera vez en la PDP-11 (de 16 bits) es tener espacios de direcciones separados para las instrucciones (texto del programa) y los datos, llamados **espacio I** y **espacio D**, respectivamente, como se muestra en la figura 3-25(b). Cada espacio de direcciones empieza desde 0 hasta cierto valor máximo, por lo general de $2^{16} - 1$ o de $2^{32} - 1$. El enlazador debe saber cuándo se utilizan espacios I y D separados, ya que cuando esto ocurre los datos se reubican a la dirección virtual 0, en vez de empezar después del programa.

En una computadora con este diseño, ambos espacios de direcciones se pueden paginar de manera independiente. Cada uno tiene su propia tabla de páginas, con su propia asignación de páginas virtuales a marcos de páginas físicas. Cuando el hardware desea obtener una instrucción, sabe que debe utilizar el espacio I y la tabla de páginas de este espacio. De manera similar, las referencias a los datos deben pasar a través de la tabla de páginas del espacio D. Aparte de esta distinción, tener espacios I y D separados no introduce ninguna complicación especial y sí duplica el espacio de direcciones disponible.

3.5.5 Páginas compartidas

Otra cuestión de diseño es la compartición. En un sistema de multiprogramación grande, es común que varios usuarios ejecuten el mismo programa a la vez. Evidentemente es más eficiente compartir las páginas para evitar tener dos copias de la misma página en memoria al mismo tiempo. Un problema es que no todas las páginas se pueden compartir. En especial, sólo pueden compartirse las páginas que son de sólo lectura como el texto del programa, pero las páginas de datos no.

Si se admiten espacios I y D separados, es relativamente simple compartir los programas al hacer que dos o más procesos utilicen la misma tabla de páginas para su espacio I pero distintas

tablas de páginas para sus espacios D. Por lo general en una implementación que soporta la compartición de esta forma, las tablas de páginas son estructuras de datos independientes de la tabla de procesos. Entonces, cada proceso tiene dos apuntadores en su tabla de procesos: uno para la tabla de páginas del espacio I y otro para la tabla de páginas del espacio D, como se muestra en la figura 3-26. Cuando el planificador selecciona un proceso para ejecutarlo, utiliza estos apuntadores para localizar las tablas de páginas apropiadas y establece la MMU para que los utilice. Aun sin espacios I y D separados, los procesos pueden compartir programas (o algunas veces bibliotecas) pero el mecanismo es más complicado.

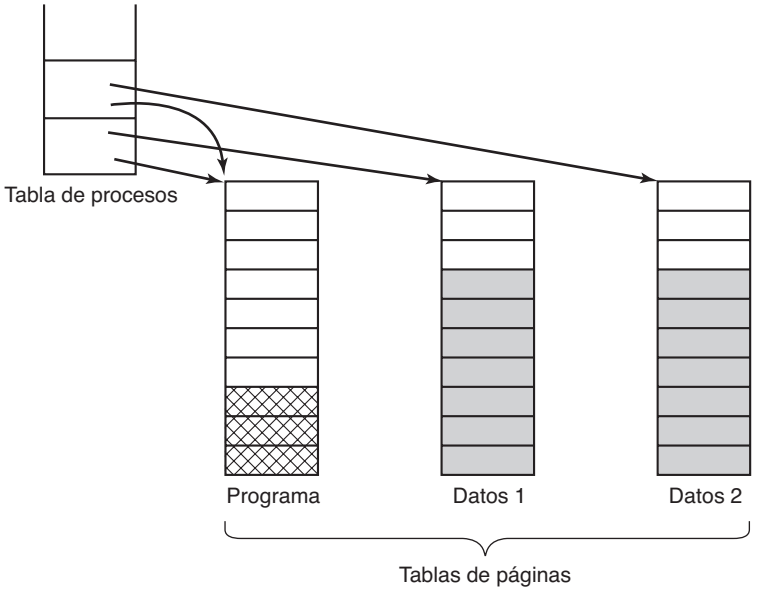


Figura 3-26. Dos procesos comparten el mismo programa compartiendo su tabla de páginas.

Cuando dos o más procesos comparten cierto código, ocurre un problema con las páginas compartidas. Suponga que los procesos *A* y *B* están ejecutando el editor y comparten sus páginas. Si el planificador decide eliminar *A* de la memoria, desalojando todas sus páginas y llenando los marcos de página vacíos con otro programa, causará que *B* genere un gran número de fallos de página para traerlas de vuelta.

De manera similar, cuando *A* termina, es esencial poder descubrir que las páginas aún están en uso, de manera que su espacio en el disco no se libere por accidente. Buscar en todas las tablas de páginas para ver si una página es compartida, frecuentemente es muy caro, por lo que se necesitan estructuras de datos especiales para llevar la cuenta de las páginas compartidas, en especial si la unidad de compartición es la página individual (o serie de páginas) en vez de toda una tabla de páginas completa.

Compartir datos es más complicado que compartir código, pero no imposible. En especial, en UNIX después de una llamada al sistema `fork`, el padre y el hijo tienen que compartir tanto el tex-

to del programa como el texto de los datos. En un sistema paginado, lo que se hace a menudo es dar a cada uno de estos procesos su propia tabla de páginas y hacer que ambos apunten al mismo conjunto de páginas. Así, no se realiza una copia de páginas al momento de la operación `fork`. Sin embargo, todas las páginas de datos son asociadas en ambos procesos de SÓLO LECTURA (READ ONLY).

Mientras que ambos procesos sólo lean sus datos, sin modificarlos, esta situación puede continuar. Tan pronto como cualquiera de los procesos actualiza una palabra de memoria, la violación de la protección de sólo lectura produce un trap al sistema operativo. Después se hace una copia de la página ofensora, para que cada proceso tenga ahora su propia copia privada. Ambas copias se establecen ahora como LECTURA-ESCRITURA (READ-WRITE), para que las siguientes operaciones de escritura en cualquiera de las copias continúen sin lanzar un trap. Esta estrategia significa que aquellas páginas que nunca se modifican (incluyendo todas las del programa) no se necesitan copiar. Este método, conocido como **copiar en escritura**, mejora el rendimiento al reducir el copiado.

3.5.6 Bibliotecas compartidas

La compartición se puede realizar en otros elementos además de las páginas individuales. Si un programa se inicia dos veces, la mayor parte de los sistemas operativos compartirán de manera automática todas las páginas de texto, quedando sólo una copia en la memoria. Las páginas de texto siempre son de sólo lectura, por lo que aquí no hay problema. Dependiendo del sistema operativo, cada proceso puede obtener su propia copia privada de las páginas de datos o se pueden compartir y marcar como de sólo lectura. Si cualquier proceso modifica una página de datos, se realizará una copia privada para éste, esto es, se aplicará la copia en escritura.

En los sistemas modernos hay muchas bibliotecas extensas utilizadas por muchos procesos, por ejemplo, la biblioteca que maneja el diálogo para explorar por archivos que se desean abrir y varias bibliotecas de gráficos. Si se enlazaran en forma estática estas bibliotecas con cada programa ejecutable en el disco se agrandarían aún más.

En vez de ello, una técnica común es utilizar **bibliotecas compartidas** (que se conocen como **DLL**s o **Bibliotecas de enlaces dinámicos** en Windows). Para aclarar la idea de una biblioteca compartida, primero considere el enlazamiento tradicional. Cuando un programa se enlaza, se nombra uno o más archivos de código objeto y posiblemente algunas bibliotecas en el comando para el enlazador, como el siguiente comando de UNIX:

```
ld *.o –lc –lm
```

el cual enlaza todos los *.o* (objeto) en el directorio actual y después explora dos bibliotecas, */usr/lib/libc.a* y */usr/lib/libm.a*. Las funciones a las que se llame en los archivos objeto pero que no estén ahí (por ejemplo, *printf*) se conocen como **externas indefinidas** y se buscan en las bibliotecas. Si se encuentran, se incluyen en el binario ejecutable. Cualquier función a la que llamen pero que no esté aún presente también se convierte en externa indefinida. Por ejemplo, *printf* necesita a *write*, por lo que si *write* no está ya incluida, el enlazador la buscará y la incluirá cuando la encuentre.

Cuando el enlazador termina, se escribe un archivo binario ejecutable en el disco que contiene todas las funciones necesarias. Las funciones presentes en la biblioteca, pero que no se llamaron, no se incluyen. Cuando el programa se carga en memoria y se ejecuta, todas las funciones que necesita están ahí.

Ahora suponga que los programas comunes usan de 20 a 50 MB de funciones de gráficos y de interfaz de usuario. Si se enlazaran de manera estática cientos de programas con todas estas bibliotecas se desperdiciaría una tremenda cantidad de espacio en el disco, además de desperdiciar espacio en la RAM a la hora de cargarlas, ya que el sistema no tendría forma de saber si puede compartirlas. Aquí es donde entran las bibliotecas compartidas. Cuando un programa se vincula con bibliotecas compartidas (que son ligeramente diferentes a las estáticas), en vez de incluir la función a la que se llamó, el vinculador incluye una pequeña rutina auxiliar que se enlaza a la función llamada en tiempo de ejecución. Dependiendo del sistema y los detalles de configuración, las bibliotecas compartidas se cargan cuando se carga el programa o cuando las funciones en ellas se llaman por primera vez. Desde luego que si otro programa ya ha cargado la biblioteca compartida, no hay necesidad de volver a cargarla; éste es el objetivo. Observe que cuando se carga o utiliza una biblioteca compartida, no se lee toda la biblioteca en memoria de un solo golpe. Se pagina una página a la vez según sea necesario, de manera que las funciones que no sean llamadas no se carguen en la RAM.

Además de reducir el tamaño de los archivos ejecutables y ahorrar espacio en memoria, las bibliotecas compartidas tienen otra ventaja: si una función en una biblioteca compartida se actualiza para eliminar un error, no es necesario recompilar los programas que la llaman, pues los antiguos binarios siguen funcionando. Esta característica es en especial importante para el software comercial, donde el código fuente no se distribuye al cliente. Por ejemplo, si Microsoft descubre y corrige un error de seguridad en alguna DLL estándar, *Windows Update* descargará la nueva DLL y sustituirá la anterior, y todos los programas que utilicen la DLL podrán usar de manera automática la nueva versión la próxima vez que se inicien.

Sin embargo, las bibliotecas compartidas tienen un pequeño problema que hay que resolver. El problema se ilustra en la figura 3-27. Aquí vemos dos procesos compartiendo una biblioteca de 20 KB en tamaño (suponiendo que cada cuadro sea de 4 KB). No obstante, la biblioteca está ubicada en una dirección distinta en cada proceso, tal vez debido a que los programas en sí no son del mismo tamaño. En el proceso 1, la biblioteca empieza en la dirección 36K; en el proceso 2 empieza en la 12K. Suponga que lo primero que hace la primera función en la biblioteca es saltar a la dirección 16 en la biblioteca. Si la biblioteca no fuera compartida, podría reubicarse al instante al momento de cargarla, de manera que el salto (en el proceso 1) pudiera ser a la dirección virtual 36K + 16. Observe que la dirección física en la RAM en donde se encuentra la biblioteca no importa, ya que todas las páginas son asociadas de direcciones virtuales a direcciones físicas por el hardware de la MMU.

Pero como la biblioteca es compartida, la reubicación instantánea no funcionará. Después de todo, cuando el proceso 2 llama a la primera función (en la dirección 12K), la instrucción de salto tiene que ir a 12K + 16, no a 36K + 16. Éste es el pequeño problema. Una manera de resolverlo es utilizar la copia en escritura y crear páginas para cada proceso que comparta la biblioteca, reubicándolas instantánemente a medida que se crean, pero es evidente que este esquema no sigue el propósito de compartir la biblioteca.

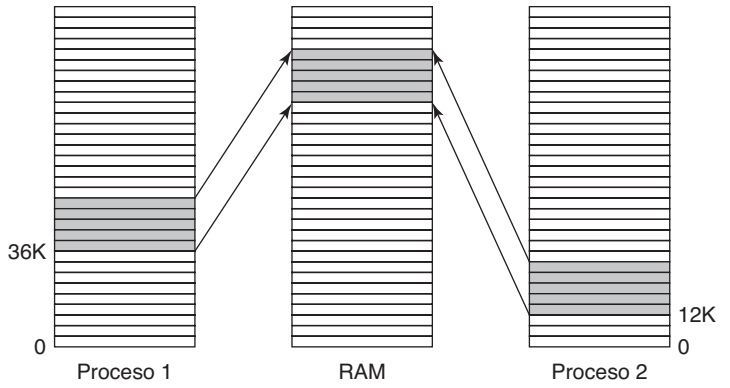


Figura 3-27. Una biblioteca compartida utilizada por dos procesos.

Una mejor solución es compilar las bibliotecas compartidas con una bandera de compilador especial, para indicar al compilador que no debe producir instrucciones que utilicen direcciones absolutas. En vez de ello, sólo se utilizan instrucciones con direcciones relativas. Por ejemplo, hay casi siempre una instrucción para saltar hacia adelante (o hacia atrás) por *n* bytes (en contraste a una instrucción que proporciona una dirección específica a la que se debe saltar). Esta instrucción funciona correctamente sin importar en dónde se coloque la biblioteca compartida en el espacio de direcciones virtuales. Al evitar direcciones absolutas, el problema se puede resolver. El código que utiliza sólo desplazamientos relativos se conoce como **código independiente de la posición**.

3.5.7 Archivos asociados

Las bibliotecas compartidas son realmente un caso de una herramienta más general, conocida como **archivos asociados a memoria**. La idea aquí es que un proceso puede emitir una llamada al sistema para asociar un archivo a una porción de su espacio de direcciones virtuales. En la mayor parte de las implementaciones no se traen páginas al momento de la asociación, sino que a medida que se usan las páginas, se paginan bajo demanda una a la vez, usando el archivo de disco como el almacén de respaldo. Cuando el proceso termina o desasocia en forma explícita el archivo, todas las páginas modificadas se escriben de vuelta en el archivo.

Los archivos asociados proporcionan un modelo alternativo para la E/S. En vez de realizar lecturas y escrituras, el archivo se puede accesar como un gran arreglo de caracteres en la memoria. En algunas situaciones, los programadores encuentran que este modelo es más conveniente.

Si dos o más procesos se asocian al mismo archivo y al mismo tiempo, se pueden comunicar a través de la memoria compartida. Las escrituras realizadas por un proceso en la memoria compartida son inmediatamente visibles cuando el otro lee de la parte de su espacio de direcciones virtuales asociado al archivo. Por lo tanto, este mecanismo proporciona un canal con un gran ancho de banda entre los procesos, y a menudo se utiliza como tal (incluso al grado de asociar un archivo temporal). Ahora debe estar claro que si hay disponibles archivos asociados a memoria, las bibliotecas compartidas pueden usar este mecanismo.

3.5.8 Política de limpieza

La paginación funciona mejor cuando hay muchos marcos de página libres que se pueden reclamar al momento en que ocurran fallos de página. Si cada marco de página está lleno y además modificado, antes de que se pueda traer una nueva página se debe escribir una página anterior en el disco. Para asegurar una provisión abundante de marcos de página libres, muchos sistemas de paginación tienen un proceso en segundo plano conocido como **demonio de paginación**, que está inactivo la mayor parte del tiempo pero se despierta en forma periódica para inspeccionar el estado de la memoria. Si hay muy pocos marcos de página libres, el demonio de paginación empieza a seleccionar páginas para desalojarlas mediante cierto algoritmo de reemplazo de páginas. Si estas páginas han sido modificadas después de haberse cargado, se escriben en el disco.

En cualquier caso se recuerda el contenido anterior de la página. En caso de que una de las páginas desalojadas se necesite otra vez antes de que se sobrescriba su marco, puede reclamarse si se elimina de la reserva de marcos de página libres. Al mantener una provisión de marcos de página a la mano se obtiene un mejor rendimiento que al utilizar toda la memoria y después tratar de encontrar un marco al momento de necesitarlo. Cuando menos el demonio de paginación asegura que todos los marcos libres estén limpios, por lo que no se necesitan escribir en el disco en un apuro a la hora de ser requeridos.

Una manera de implementar esta política de limpieza es mediante un reloj con dos manecillas. La manecilla principal es controlada por el demonio de paginación. Cuando apunta a una página sucia, esa página se escribe de vuelta al disco y la manecilla principal se avanza. Cuando apunta a una página limpia, sólo se avanza. La manecilla secundaria se utiliza para reemplazar páginas, como en el algoritmo de reloj estándar. Sólo que ahora, la probabilidad de que la manecilla secundaria llegue a una página limpia se incrementa debido al trabajo del demonio de paginación.

3.5.9 Interfaz de memoria virtual

Hasta ahora, en todo nuestro análisis hemos supuesto que la memoria virtual es transparente para los procesos y los programadores; es decir, todo lo que ven es un gran espacio de direcciones virtuales en una computadora con una memoria física (más) pequeña. Con muchos sistemas esto es cierto pero, en algunos sistemas avanzados, los programadores tienen cierto control sobre el mapa de memoria y pueden utilizarlo de maneras no tradicionales para mejorar el comportamiento de un programa. En esta sección analizaremos unas cuantas de estas formas.

Una razón por la que se otorga a los programadores el control sobre su mapa de memoria es para permitir que dos o más procesos compartan la misma memoria. Si los programadores pueden nombrar regiones de su memoria, tal vez sea posible para un proceso dar a otro proceso el nombre de una región de memoria, de manera que el proceso también pueda asociarla. Con dos (o más) procesos compartiendo las mismas páginas, la compartición con mucho ancho de banda se hace posible: un proceso escribe en la memoria compartida y otro proceso lee de ella.

La compartición de páginas también se puede utilizar para implementar un sistema de transmisión de mensajes de alto rendimiento. Por lo general, cuando se pasan mensajes los datos se copian de un espacio de direcciones a otro, a un costo considerable. Si los procesos pueden controlar su mapa de páginas, se puede pasar un mensaje al hacer que el proceso emisor desasocie la(s) pági-

na(s) que contiene(n) el mensaje, y el proceso receptor la(s) asocia. Aquí sólo se tienen que copiar los nombres de las páginas, en vez de todos los datos.

Otra técnica más de administración avanzada de memoria es la **memoria compartida distribuida** (Feeley y colaboradores, 1995; Li, 1986; Li y Hudak, 1989; y Zekauskas y colaboradores, 1994). La idea aquí es permitir que varios procesos compartan a través de la red un conjunto de páginas, posiblemente (pero no es necesario) como un solo espacio de direcciones lineal compartido. Cuando un proceso hace referencia a una página que no está asociada, obtiene un fallo de página. El manejador de fallos de página, que puede estar en espacio de kernel o de usuario, localiza entonces la máquina que contiene la página y le envía un mensaje pidiéndole que la desasocie y la envíe a través de la red. Cuando llega la página, se asocia y la instrucción que falló se reinicia. En el capítulo 8 examinaremos la memoria compartida distribuida con más detalle.

3.6 CUESTIONES DE IMPLEMENTACIÓN

Los implementadores de los sistemas de memoria virtual tienen que elegir entre los principales algoritmos teóricos: entre el algoritmo de segunda oportunidad y el de envejecimiento, entre la asignación de páginas local o global, y entre la paginación bajo demanda o la prepaginación. Pero también tienen que estar al tanto de varias cuestiones prácticas de implementación. En esta sección daremos un vistazo a unos cuantos de los problemas comunes y ciertas soluciones.

3.6.1 Participación del sistema operativo en la paginación

Hay cuatro ocasiones en las que el sistema operativo tiene que realizar trabajo relacionado con la paginación: al crear un proceso, al ejecutar un proceso, al ocurrir un fallo de página y al terminar un proceso. Ahora examinaremos brevemente cada una de estas ocasiones para ver qué se tiene que hacer.

Cuando se crea un proceso en un sistema de paginación, el sistema operativo tiene que determinar qué tan grandes serán el programa y los datos (al principio), y crear una tabla de páginas para ellos. Se debe asignar espacio en memoria para la tabla de páginas y se tiene que inicializar. La tabla de páginas no necesita estar residente cuando el proceso se intercambia hacia fuera, pero tiene que estar en memoria cuando el proceso se está ejecutando. Además, se debe asignar espacio en el área de intercambio en el disco, para que cuando se intercambie una página, tenga un lugar a donde ir. El área de intercambio también se tiene que inicializar con el texto del programa y los datos, para que cuando el nuevo proceso empiece a recibir fallos de página, las páginas se puedan traer. Algunos sistemas paginan el texto del programa directamente del archivo ejecutable, con lo cual se ahorra espacio en disco y tiempo de inicialización. Por último, la información acerca de la tabla de páginas y el área de intercambio en el disco se debe registrar en la tabla de procesos.

Cuando un proceso se planifica para ejecución, la MMU se tiene que restablecer para el nuevo proceso y el TLB se vacía para deshacerse de los restos del proceso que se estaba ejecutando antes. La tabla de páginas del nuevo proceso se tiene que actualizar, por lo general copiándola o mediante un apuntador a éste hacia cierto(s) registro(s) de hardware. De manera opcional, algunas o

todas las páginas del proceso se pueden traer a memoria para reducir el número de fallos de página al principio (por ejemplo, es evidente que será necesaria la página a la que apunta la PC).

Cuando ocurre un fallo de página, el sistema operativo tiene que leer los registros de hardware para determinar cuál dirección virtual produjo el fallo. Con base en esta información debe calcular qué página se necesita y localizarla en el disco. Después debe buscar un marco de página disponible para colocar la nueva página, desalojando alguna página anterior si es necesario. Luego debe leer la página necesaria y colocarla en el marco de páginas. Por último, debe respaldar el contador de programa para hacer que apunte a la instrucción que falló y dejar que la instrucción se ejecute de nuevo.

Cuando un proceso termina, el sistema operativo debe liberar su tabla de páginas, sus páginas y el espacio en disco que ocupan las páginas cuando están en disco. Si alguna de las páginas están compartidas con otros procesos, las páginas en memoria y en disco sólo pueden liberarse cuando el último proceso que las utilice haya terminado.

3.6.2 Manejo de fallos de página

Finalmente, estamos en una posición para describir con detalle lo que ocurre en un fallo de página. La secuencia de eventos es la siguiente:

1. El hardware hace un trap al kernel, guardando el contador de programa en la pila. En la mayor parte de las máquinas, se guarda cierta información acerca del estado de la instrucción actual en registros especiales de la CPU.

2. Se inicia una rutina en código ensamblador para guardar los registros generales y demás información volátil, para evitar que el sistema operativo la destruya. Esta rutina llama al sistema operativo como un procedimiento.

3. El sistema operativo descubre que ha ocurrido un fallo de página y trata de descubrir cuál página virtual se necesita. A menudo, uno de los registros de hardware contiene esta información. De no ser así, el sistema operativo debe obtener el contador de programa, obtener la instrucción y analizarla en software para averiguar lo que estaba haciendo cuando ocurrió el fallo.

4. Una vez que se conoce la dirección virtual que produjo el fallo, el sistema comprueba si esta dirección es válida y si la protección es consistente con el acceso. De no ser así, el proceso recibe una señal o es eliminado. Si la dirección es válida y no ha ocurrido un fallo de página, el sistema comprueba si hay un marco de página disponible. Si no hay marcos disponibles, se ejecuta el algoritmo de reemplazo de páginas para seleccionar una víctima.

5. Si el marco de página seleccionado está sucio, la página se planifica para transferirla al disco y se realiza una conmutación de contexto, suspendiendo el proceso fallido y dejando que se ejecute otro hasta que se haya completado la transferencia al disco. En cualquier caso, el marco se marca como ocupado para evitar que se utilice para otro propósito.

6. Tan pronto como el marco de página esté limpio (ya sea de inmediato, o después de escribirlo en el disco), el sistema operativo busca la dirección de disco en donde se encuentra la página necesaria, y planifica una operación de disco para llevarla a memoria. Mientras se está cargando la página, el proceso fallido sigue suspendido y se ejecuta otro proceso de usuario, si hay uno disponible.

7. Cuando la interrupción de disco indica que la página ha llegado, las tablas de páginas se actualizan para reflejar su posición y el marco se marca como en estado normal.

8. La instrucción fallida se respalda al estado en que tenía cuando empezó, y el contador de programa se restablece para apuntar a esa instrucción.

9. El proceso fallido se planifica y el sistema operativo regresa a la rutina (en lenguaje ensamblador) que lo llamó.

10. Esta rutina recarga los registros y demás información de estado, regresando al espacio de usuario para continuar la ejecución, como si no hubiera ocurrido el fallo.

3.6.3 Respaldo de instrucción

Cuando un programa hace referencia a una página que no está en memoria, la instrucción que produjo el fallo se detiene parcialmente y ocurre un trap al sistema operativo. Una vez que el sistema operativo obtiene la página necesaria, debe reiniciar la instrucción que produjo el trap. Es más fácil decir esto que hacerlo.

Para ver la naturaleza del problema en el peor de los casos, considere una CPU que tiene instrucciones con dos direcciones, como el procesador Motorola 680x0, utilizado ampliamente en sistemas integrados. Por ejemplo, la instrucción

```
MOV.L #6(A1),2(A0)
```

es de 6 bytes (vea la figura 3-28). Para poder reiniciar la instrucción, el sistema operativo debe determinar en dónde se encuentra el primer byte de la instrucción. El valor del contador de programa al momento en que ocurre el trap depende de cuál fue el operando que falló y cómo se ha implementado el microcódigo de la CPU.

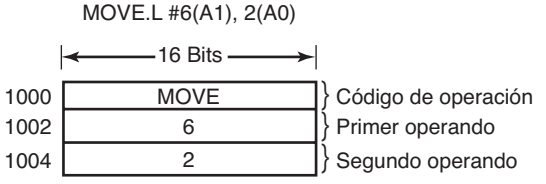


Figura 3-28. Una instrucción que produce un fallo de página.

En la figura 3-28, tenemos una instrucción que empieza en la dirección 1000 y que hace tres referencias a memoria: la palabra de la instrucción en sí y dos desplazamientos para los operandos.

Dependiendo de cuál de estas tres referencias a memoria haya ocasionado el fallo de página, el contador de programa podría ser 1000, 1002 o 1004 al momento del fallo. Con frecuencia es imposible que el sistema operativo determine sin ambigüedad en dónde empezó la instrucción. Si el contador de programa es 1002 al momento del fallo, el sistema operativo no tiene manera de saber si la palabra en 1002 es una dirección de memoria asociada con una instrucción en 1000 (por ejemplo, la ubicación de un operando) o el código de operación de una instrucción.

Tan mal como podría estar este problema, podría ser aún peor. Algunos modos de direccionamiento del 680x0 utilizan el autoincremento, lo cual significa que un efecto secundario de ejecutar la instrucción es incrementar uno o más registros. Las instrucciones que utilizan el modo de autoincremento también pueden fallar. Dependiendo de los detalles del microcódigo, el incremento se puede realizar antes de la referencia a memoria, en cuyo caso el sistema operativo debe decrementar el registro en el software antes de reiniciar la instrucción. O el autoincremento se puede realizar después de la referencia a memoria, en cuyo caso no se habrá realizado al momento del trap y el sistema operativo no deberá deshacerlo. También existe el modo de autodecremento y produce un problema similar. Los detalles precisos de si se han o no realizado los autoincrementos o autodecrementos antes de las correspondientes referencias a memoria pueden diferir de una instrucción a otra, y de un modelo de CPU a otro.

Por fortuna, en algunas máquinas los diseñadores de la CPU proporcionan una solución, por lo general en la forma de un registro interno oculto, en el que se copia el contador de programa justo antes de ejecutar cada instrucción. Estas máquinas también pueden tener un segundo registro que indique cuáles registros se han ya autoincrementado o autodecrementado y por cuánto. Dada esta información, el sistema operativo puede deshacer sin ambigüedad todos los efectos de la instrucción fallida, de manera que se pueda reiniciar. Si esta información no está disponible, el sistema operativo tiene que hacer peripecias para averiguar qué ocurrió y cómo puede repararlo. Es como si los diseñadores del hardware no pudieran resolver el problema y pasaran esa responsabilidad a los escritores del sistema operativo.

3.6.4 Bloqueo de páginas en memoria

Aunque no hemos hablado mucho sobre la E/S en este capítulo, el hecho de que una computadora tenga memoria virtual no significa que estén ausentes las operaciones de E/S. La memoria virtual y la E/S interactúan en formas sutiles. Considere un proceso que acaba de emitir una llamada al sistema para leer algún archivo o dispositivo y colocarlo en un búfer dentro de su espacio de direcciones. Mientras espera a que se complete la E/S, el proceso se suspende y se permite a otro proceso ejecutarse. Este otro proceso recibe un fallo de página.

Si el algoritmo de paginación es global, hay una pequeña probabilidad (distinta de cero) de que la página que contiene el búfer de E/S sea seleccionada para eliminarla de la memoria. Si un dispositivo de E/S se encuentra en el proceso de realizar una transferencia por DMA a esa página, al eliminarla parte de los datos se escribirán en el búfer al que pertenecen y parte sobre la página que se acaba de cargar. Una solución a este problema es bloquear las páginas involucradas en operaciones de E/S en memoria, de manera que no se eliminen. Bloquear una página se conoce a menudo

como **fijada** (*pinning*) en la memoria. Otra solución es enviar todas las operaciones de E/S a búferes del kernel y después copiar los datos a las páginas de usuario.

3.6.5 Almacén de respaldo

En nuestro análisis de los algoritmos de reemplazo de páginas, vimos cómo se selecciona una página para eliminarla. No hemos dicho mucho con respecto a dónde se coloca en el disco cuando se pagina hacia fuera de la memoria. Ahora vamos a describir algunas cuestiones relacionadas con la administración del disco.

El algoritmo más simple para asignar espacio de página en el disco es tener una partición de intercambio especial en el disco o aún mejor es tenerla en un disco separado del sistema operativo (para balancear la carga de E/S). La mayor parte de los sistemas UNIX funcionan así. Esta partición no tiene un sistema de archivos normal, lo cual elimina la sobrecarga de convertir desplazamientos en archivos a direcciones de bloque. En vez de ello, se utilizan números de bloque relativos al inicio de la partición.

Cuando se inicia el sistema, esta partición de intercambio está vacía y se representa en memoria como una sola entrada que proporciona su origen y tamaño. En el esquema más simple, cuando se inicia el primer proceso, se reserva un trozo del área de la partición del tamaño del primer proceso y se reduce el área restante por esa cantidad. A medida que se inician nuevos procesos, se les asigna trozos de la partición de intercambio con un tamaño equivalente al de sus imágenes de núcleo. Al terminar, se libera su espacio en disco. La partición de intercambio se administra como una lista de trozos libres. En el capítulo 10 analizaremos mejores algoritmos.

Con cada proceso está asociada la dirección de disco de su área de intercambio; es decir, en qué parte de la partición de intercambio se mantiene su imagen. Esta información se mantiene en la tabla de procesos. El cálculo la dirección en la que se va a escribir una página es simple: sólo se suma el desplazamiento de la página dentro del espacio de direcciones virtual al inicio del área de intercambio. Sin embargo, antes de que un proceso pueda empezar se debe inicializar el área de intercambio. Una forma de hacerlo es copiar toda la imagen del proceso al área de intercambio, de manera que se pueda traer y colocar *en* la memoria según sea necesario. La otra es cargar todo el proceso en memoria y dejar que se pagine *hacia fuera* según sea necesario.

Sin embargo, este simple modelo tiene un problema: los procesos pueden incrementar su tamaño antes de empezar. Aunque el texto del programa por lo general es fijo, el área de los datos puede crecer algunas veces, y la pila siempre puede crecer. En consecuencia, podría ser mejor reservar áreas de intercambio separadas para el texto, los datos y la pila, permitiendo que cada una de estas áreas consista de más de un trozo en el disco.

El otro extremo es no asignar nada por adelantado y asignar espacio en el disco para cada página cuando ésta se intercambie hacia fuera de la memoria y desasignarlo cuando se vuelva a intercambiar hacia la memoria. De esta forma, los procesos en memoria no acaparan espacio de intercambio. La desventaja es que se necesita una dirección de disco en la memoria para llevar la cuenta de cada página en el disco. En otras palabras, debe haber una tabla por cada proceso que indique en dónde se encuentra cada página en el disco. Las dos alternativas se muestran en la figura 3-29.

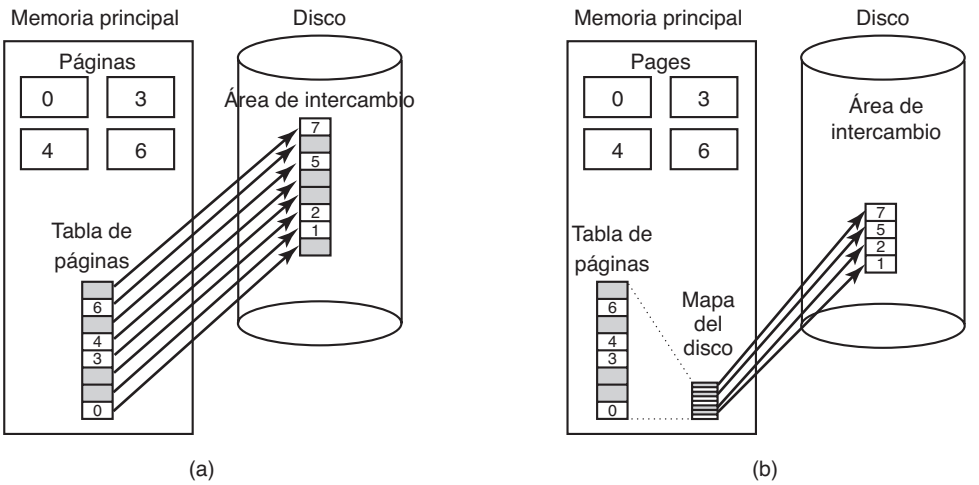


Figura 3-29. (a) Paginación a un área de intercambio estática. (b) Respaldo de páginas en forma dinámica.

En la figura 3-29(a) se ilustra una tabla de páginas con ocho páginas. Las páginas 0, 3, 4 y 6 están en memoria. Las páginas 1, 2, 5 y 7 están en disco. El área de intercambio en el disco es tan grande como el espacio de direcciones virtuales del proceso (ocho páginas), donde cada página tiene una ubicación fija a la cual se escribe cuando se desaloja de la memoria principal. Para calcular esta dirección sólo se requiere saber dónde empieza el área de paginación del proceso, ya que las páginas se almacenan en ella de manera contigua, ordenadas por su número de página virtual. Una página que está en memoria siempre tiene una copia sombra en el disco, pero esta copia puede estar obsoleta si la página se modificó después de haberla cargado. Las páginas sombreadas en la memoria indican páginas que no están presentes en memoria. Las páginas sombreadas en el disco son (en principio) suplantadas por las copias en memoria, aunque si una página de memoria se tiene que intercambiar de vuelta a disco y no se ha modificado desde que se cargó, se utilizará la copia del disco (sombreada).

En la figura 3-29(b), las páginas no tienen direcciones fijas en el disco. Cuando se intercambia una página hacia fuera de la memoria, se selecciona una página vacía en el disco al momento y el mapa de disco (que tiene espacio para una dirección de disco por página virtual) se actualiza de manera acorde. Una página en memoria no tiene copia en el disco. Sus entradas en el mapa de disco contienen una dirección de disco inválida o un bit que las marca como que no están en uso.

No siempre es posible tener una partición de intercambio fija. Por ejemplo, tal vez no haya particiones de disco disponibles. En este caso se pueden utilizar uno o más archivos previamente asignados dentro del sistema de archivos normal. Windows utiliza este método. Sin embargo, aquí se puede utilizar una optimización para reducir la cantidad de espacio en disco necesaria. Como el texto del programa de cada proceso proviene de algún archivo (ejecutable) en el sistema de archivos, este archivo ejecutable se puede utilizar como el área de intercambio. Mejor aún, ya que el texto del programa generalmente es de sólo lectura, cuando la memoria es escasa y se tienen que desalojar páginas del programa de la memoria, sólo se descartan y se vuelven a leer del archivo ejecutable cuando se necesiten. Las bibliotecas compartidas también pueden trabajar de esta forma.

3.6.6 Separación de política y mecanismo

Una importante herramienta para administrar la complejidad de cualquier sistema es separar la política del mecanismo. Este principio se puede aplicar a la administración de la memoria, al hacer que la mayor parte del administrador de memoria se ejecute como un proceso a nivel usuario. Dicha separación se realizó por primera vez en Mach (Young y colaboradores, 1987). El siguiente análisis se basa de manera general en Mach.

En la figura 3-30 se muestra un ejemplo simple de cómo se pueden separar la política y el mecanismo. Aquí, el sistema de administración de memoria se divide en dos partes:

1. Un manejador de la MMU de bajo nivel.
2. Un manejador de fallos de página que forma parte del kernel.
3. Un paginador externo que se ejecuta en espacio de usuario.

Todos los detalles acerca del funcionamiento de la MMU están encapsulados en el manejador de la MMU, que es código dependiente de la máquina y tiene que volver a escribirse para cada nueva plataforma a la que se porte el sistema operativo. El manejador de fallos de página es código independiente de la máquina y contiene la mayor parte del mecanismo para la paginación. La política se determina en gran parte mediante el paginador externo, que se ejecuta como un proceso de usuario.

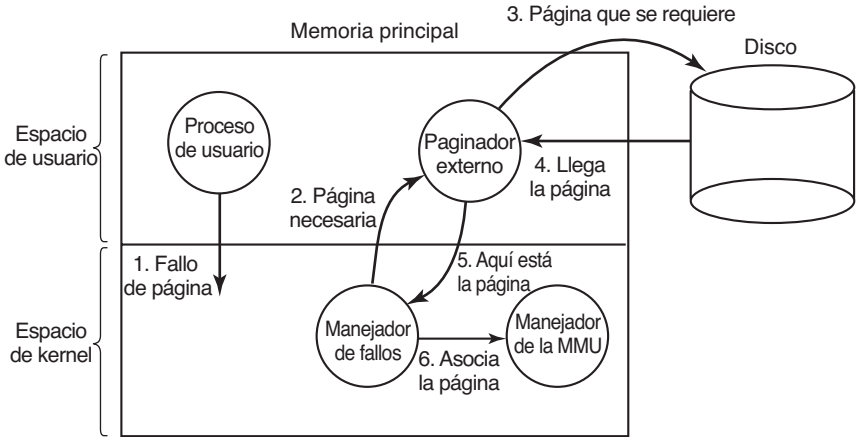


Figura 3-30. Manejo de fallos de página con un paginador externo.

Cuando se inicia un proceso, se notifica al paginador externo para poder establecer el mapa de páginas del proceso y asignar el almacenamiento de respaldo en el disco, si es necesario. A medida que el proceso se ejecuta, puede asignar nuevos objetos en su espacio de direcciones, por lo que se notifica de nuevo al paginador externo.

Una vez que el proceso empieza a ejecutarse, puede obtener un fallo de página. El manejador de fallos averigua cuál página virtual se necesita y envía un mensaje al paginador externo, indicán-

dole el problema. Después el paginador externo lee la página necesaria del disco y la copia a una porción de su propio espacio de direcciones. Después le indica al manejador de fallos en dónde está la página. Luego, el manejador de fallos desasigna la página del espacio de direcciones del paginador externo y pide al manejador de la MMU que la coloque en el espacio de direcciones del usuario, en el lugar correcto. Entonces se puede reiniciar el proceso de usuario.

Esta implementación no deja establecido dónde se va a colocar el algoritmo de reemplazo de páginas. Sería más limpio tenerlo en el paginador externo, pero hay ciertos problemas con este método. El problema principal es que el paginador externo no tiene acceso a los bits *R* y *M* de todas las páginas. Estos bits desempeñan un papel en muchos de los algoritmos de paginación. Por ende, se necesita algún mecanismo para pasar esta información al paginador externo o el algoritmo de reemplazo de páginas debe ir en el kernel. En el último caso, el manejador de fallos indica al paginador externo cuál página ha seleccionado para desalojarla y proporciona los datos, ya sea asignándola al espacio de direcciones eterno del paginador o incluyéndola en un mensaje. De cualquier forma, el paginador externo escribe los datos en el disco.

La principal ventaja de esta implementación es que se obtiene un código más modular y una mayor flexibilidad. La principal desventaja es la sobrecarga adicional de cruzar el límite entre usuario y kernel varias veces, y la sobrecarga de los diversos mensajes que se envían entre las partes del sistema. En estos momentos el tema es muy controversial, pero a medida que las computadoras se hacen cada vez más rápidas, y el software se hace cada vez más complejo, a la larga sacrificar cierto rendimiento por un software más confiable probablemente sea algo aceptable para la mayoría de los implementadores.

3.7 SEGMENTACIÓN

La memoria virtual que hemos analizado hasta ahora es unidimensional, debido a que las direcciones virtuales van desde 0 hasta cierta dirección máxima, una dirección después de la otra. Para muchos problemas, tener dos o más espacios de direcciones virtuales separados puede ser mucho mejor que tener sólo uno. Por ejemplo, un compilador tiene muchas tablas que se generan a medida que procede la compilación, las cuales posiblemente incluyen:

1. El texto del código fuente que se guarda para el listado impreso (en sistemas de procesamiento por lotes).
2. La tabla de símbolos, que contiene los nombres y atributos de las variables.
3. La tabla que contiene todas las constantes enteras y de punto flotante utilizadas.
4. El árbol de análisis sintáctico, que contiene el análisis sintáctico del programa.
5. La pila utilizada para las llamadas a procedimientos dentro del compilador.

Cada una de las primeras cuatro tablas crece en forma continua a medida que procede la compilación. La última crece y se reduce de maneras impredecibles durante la compilación. En una memoria unidimensional, a estas cinco tablas se les tendría que asignar trozos contiguos de espacio de direcciones virtuales, como en la figura 3-31.

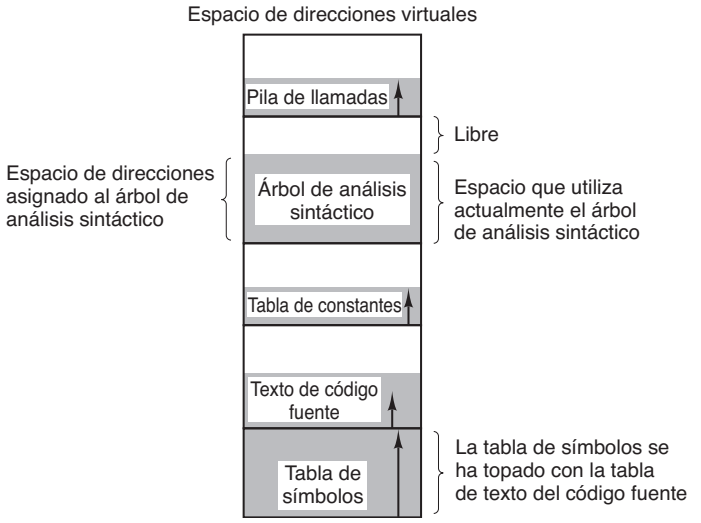


Figura 3-31. En un espacio de direcciones unidimensional con tablas que aumentan de tamaño, una tabla puede toparse con otra.

Considere lo que ocurre si un programa tiene un número variables mucho mayor de lo usual, pero una cantidad normal de todo lo demás. El trozo de espacio de direcciones asignado para la tabla de símbolos se puede llenar, pero puede haber mucho espacio en las otras tablas. Desde luego que el compilador podría simplemente emitir un mensaje indicando que la compilación no puede continuar debido a que hay demasiadas variables, pero esto no parece muy deportivo cuando se deja espacio sin usar en las otras tablas.

Otra posibilidad es jugar a Robin Hood, tomando espacio de las tablas con exceso y dándolo a las tablas con poco espacio. Esta revoltura puede hacerse, pero es similar a cuando uno administra sus propios sobrepuestos; una molestia como mínimo y, en el peor de los casos, mucho trabajo tedioso sin recompensas.

Lo que se necesita realmente es una forma de liberar al programador de tener que administrar las tablas en expansión y contracción, de la misma forma que la memoria virtual elimina la preocupación de tener que organizar el programa en sobrepuestos.

Una solución simple y en extremado general es proporcionar la máquina con muchos espacios de direcciones por completo independientes, llamados **segmentos**. Cada segmento consiste en una secuencia lineal de direcciones, desde 0 hasta cierto valor máximo. La longitud de cada segmento puede ser cualquier valor desde 0 hasta el máximo permitido. Los distintos segmentos pueden tener distintas longitudes (y por lo general así es). Además las longitudes de los segmentos pueden cambiar durante la ejecución. La longitud de un segmento de pila puede incrementarse cada vez que se meta algo a la pila y decrementarse cada vez que se saque algo.

Debido a que cada segmento constituye un espacio de direcciones separado, los distintos segmentos pueden crecer o reducirse de manera independiente, sin afectar unos a otros. Si una pila en

cierto segmento necesita más espacio de direcciones para crecer, puede tenerlo, ya que no hay nada más en su espacio de direcciones con lo que se pueda topar. Desde luego que un segmento se puede llenar, pero por lo general los segmentos son muy grandes, por lo que esta ocurrencia es rara. Para especificar una dirección en esta memoria segmentada o bidimensional, el programa debe suministrar una dirección en dos partes, un número de segmento y una dirección dentro del segmento. La figura 3-32 ilustra el uso de una memoria segmentada para las tablas del compilador que vimos antes. Aquí se muestran cinco segmentos independientes.

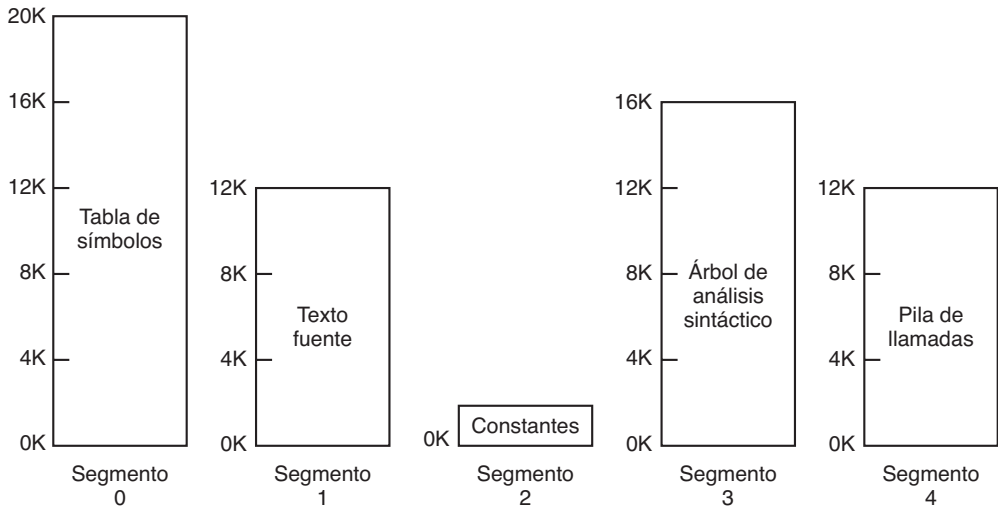


Figura 3-32. Una memoria segmentada permite que cada tabla crezca o se reduzca de manera independiente a las otras tablas.

Enfatizamos que un segmento es una entidad lógica, de la cual el programador está consciente y la utiliza como entidad lógica. Un segmento podría contener un procedimiento, o un arreglo, o una pila, o una colección de variables escalares, pero por lo general no contiene una mezcla de distintos tipos.

Una memoria segmentada tiene otras ventajas además de simplificar el manejo de estructuras de datos que aumentan o reducen su tamaño. Si cada procedimiento ocupa un segmento separado, con la dirección 0 como su dirección inicial, la vinculación de procedimientos que se compilan por separado se simplifica de manera considerable. Después de que se han compilado y vinculado todos los procedimientos que constituyen un programa, una llamada al procedimiento en el segmento *n* utilizará la dirección en dos partes (*n*, 0) para direccionar la palabra 0 (el punto de entrada).

Si el procedimiento en el segmento *n* se modifica y recompila posteriormente, no hay necesidad de cambiar los demás procedimientos (ya que no se han modificado direcciones iniciales), aun si la nueva versión es más grande que la anterior. Con una memoria unidimensional, los procedimientos se empaquetan estrechamente, uno al lado del otro, sin espacio de direcciones entre ellos. En consecuencia, al cambiar el tamaño de un procedimiento se puede afectar la dirección inicial de otros procedimientos (no relacionados). Esto a su vez requiere la modificación de todos los proce-

dimientos que llamen a cualquiera de los procedimientos que se movieron, para poder incorporar sus nuevas direcciones iniciales. Si un programa contiene cientos de procedimientos, este proceso puede ser costoso.

La segmentación también facilita la compartición de procedimientos o datos entre varios procesos. Un ejemplo común es la biblioteca compartida. Las estaciones de trabajo modernas que operan sistemas de ventanas avanzados tienen a menudo bibliotecas gráficas en extremo extensas que se compilan en casi todos los programas. En un sistema segmentado, la biblioteca gráfica se puede colocar en un segmento y varios procesos pueden compartirla, eliminando la necesidad de tenerla en el espacio de direcciones de cada proceso. Aunque también es posible tener bibliotecas compartidas en sistemas de paginación puros, es más complicado. En efecto, estos sistemas lo hacen mediante la simulación de la segmentación.

Como cada segmento forma una entidad lógica de la que el programador está consciente, como un procedimiento, un arreglo o una pila, los distintos segmentos pueden tener diferentes tipos de protección. Un segmento de procedimiento se puede especificar como de sólo ejecución, para prohibir los intentos de leer de él o almacenar en él. Un arreglo de punto flotante se puede especificar como de lectura/escritura pero no como de ejecución, y los intentos de saltar a él se atraparán. Dicha protección es útil para atrapar errores de programación.

El lector debe tratar de comprender por qué la protección es sensible en una memoria segmentada, pero no en una memoria paginada unidimensionalmente. En una memoria segmentada, el usuario está consciente de lo que hay en cada segmento. Por lo general, un segmento no contendría un procedimiento y una pila, por ejemplo, sino uno o el otro, no ambos. Como cada segmento contiene sólo un tipo de objeto, puede tener la protección apropiada para ese tipo específico. La paginación y la segmentación se comparan en la figura 3-33.

El contenido de una página es, en cierto sentido, accidental. El programador ni siquiera está consciente del hecho de que está ocurriendo la paginación. Aunque sería posible poner unos cuantos bits en cada entrada de la tabla de páginas para especificar el acceso permitido, para utilizar esta característica el programador tendría que llevar registro del lugar en el que se encontraran los límites de página en su espacio de direcciones. La paginación se inventó para eliminar precisamente ese tipo de administración. Como el usuario de una memoria segmentada tiene la ilusión de que todos los segmentos se encuentran en memoria principal todo el tiempo (es decir, que puede direccionarlos como si estuvieran) puede proteger cada segmento por separado sin tener que preocuparse con la administración por tener que superponerlos.

3.7.1 Implementación de segmentación pura

La implementación de segmentación difiere de la paginación de una manera esencial: las páginas tienen un tamaño fijo y los segmentos no. La figura 3-34(a) muestra un ejemplo de una memoria física que al principio contiene cinco segmentos. Ahora considere lo que ocurre si el segmento 1 se desaloja y el segmento 7, que es más pequeño, se coloca en su lugar. Obtendremos la configuración de memoria de la figura 3-34(b). Entre el segmento 7 y el 2 hay un área sin uso; es decir, un hueco. Después el segmento 4 se reemplaza por el segmento 5, como en la figura 3-34(c), y el segmento 3 se reemplaza por el segmento 6, como en la figura 3-34(d).

Consideración	Paginación	Segmentación
¿Necesita el programador estar consciente de que se está utilizando esta técnica?	No	Sí
¿Cuántos espacios de direcciones lineales hay?	1	Muchos
¿Puede el espacio de direcciones total exceder al tamaño de la memoria física?	Sí	Sí
¿Pueden los procedimientos y los datos diferenciarse y protegerse por separado?	No	Sí
¿Pueden las tablas cuyo tamaño fluctúa acomodarse con facilidad?	No	Sí
¿Se facilita la compartición de procedimientos entre usuarios?	No	Sí
¿Por qué se inventó esta técnica?	Para obtener un gran espacio de direcciones lineal sin tener que comprar más memoria física	Para permitir a los programas y datos dividirse en espacios de direcciones lógicamente independientes, ayudando a la compartición y la protección

Figura 3-33. Comparación de la paginación y la segmentación.

Una vez que el sistema haya estado en ejecución por un tiempo, la memoria se dividirá en un número de trozos, de los cuales algunos contendrán segmentos y otros huecos. Este fenómeno, llamado **efecto de tablero de ajedrez** o **fragmentación externa**, desperdicia memoria en los huecos. Se puede manejar mediante la compactación, como veremos en la figura 3-34(c).

3.7.2 Segmentación con paginación: MULTICS

Si los segmentos son extensos, puede ser inconveniente (o incluso imposible) mantenerlos completos en memoria principal. Esto nos lleva a la idea de paginarlos, de manera que sólo las páginas que realmente se necesiten tengan que estar presentes. Varios sistemas importantes han soportado segmentos de páginas. En esta sección describiremos el primero: MULTICS. En la siguiente analizaremos uno más reciente: el Intel Pentium.

MULTICS operaba en las máquinas Honeywell 6000 y sus descendientes; proveía a cada programa con una memoria virtual de hasta 2^{18} segmentos (más de 250,000), cada uno de los cuales podría ser de hasta 65,536 palabras (36 bits) de longitud. Para implementar esto, los diseñadores de MULTICS optaron por considerar a cada segmento como una memoria virtual y la paginaron, com-

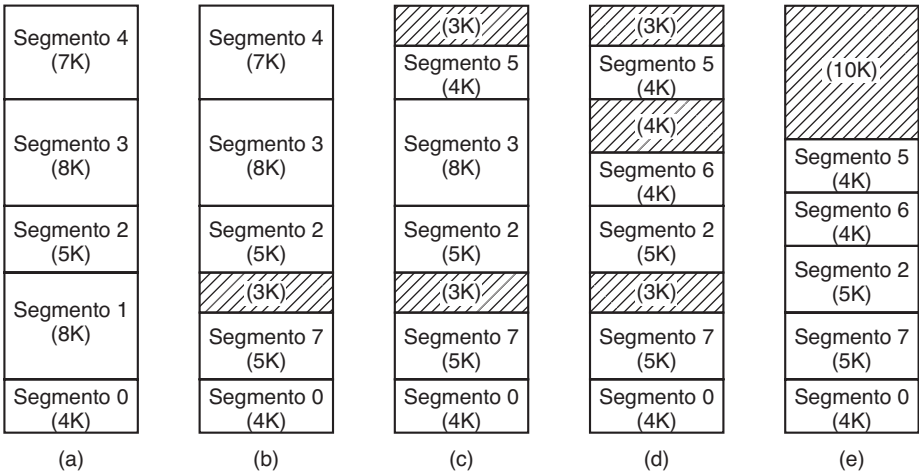


Figura 3-34. (a)-(d) Desarrollo del efecto de tablero de ajedrez. (e) Eliminación del efecto de tablero de ajedrez mediante la compactación.

binando las ventajas de la paginación (tamaño de página uniforme y no tener que mantener todo el segmento en la memoria, si sólo se está utilizando parte de él) con las ventajas de la segmentación (facilidad de programación, modularidad, protección, compartición).

Cada programa de MULTICS tiene una tabla de segmentos, con un descriptor por segmento. Como hay en potencia más de un cuarto de millón de entradas en la tabla, la tabla de segmentos es en sí un segmento y se pagina. Un descriptor de segmentos contiene una indicación acerca de si el segmento está en memoria principal o no. Si cualquier parte del segmento está en memoria, se considera que el segmento está en memoria y su tabla de páginas estará en memoria. Si el segmento está en memoria, su descriptor contiene un apuntador de 18 bits a su tabla de páginas, como en la figura 3-35(a). Como las direcciones físicas son de 24 bits y las páginas se alinean en límites de 64 bytes (implicando que los 6 bits de menor orden de las direcciones de página son 000000), sólo se necesitan 18 bits en el descriptor para almacenar una dirección de la tabla de páginas. El descriptor también contiene el tamaño del segmento, los bits de protección y unos cuantos elementos más. La figura 3-35(b) ilustra un descriptor de segmento de MULTICS. La dirección del segmento en la memoria secundaria no está en el descriptor de segmentos, sino en otra tabla utilizada por el manejador de fallos de segmento.

Cada segmento es un espacio de direcciones virtual ordinario y se pagina de la misma forma que la memoria paginada no segmentada descrita anteriormente en este capítulo. El tamaño normal de página es de 1024 palabras (aunque unos cuantos segmentos utilizados por MULTICS en sí no están paginados o están paginados en unidades de 64 palabras para ahorrar memoria física).

Una dirección en MULTICS consiste en dos partes: el segmento y la dirección dentro del segmento. La dirección dentro del segmento se divide aún más en un número de página y en una palabra dentro de la página, como se muestra en la figura 3-36. Cuando ocurre una referencia a memoria, se lleva a cabo el siguiente algoritmo.

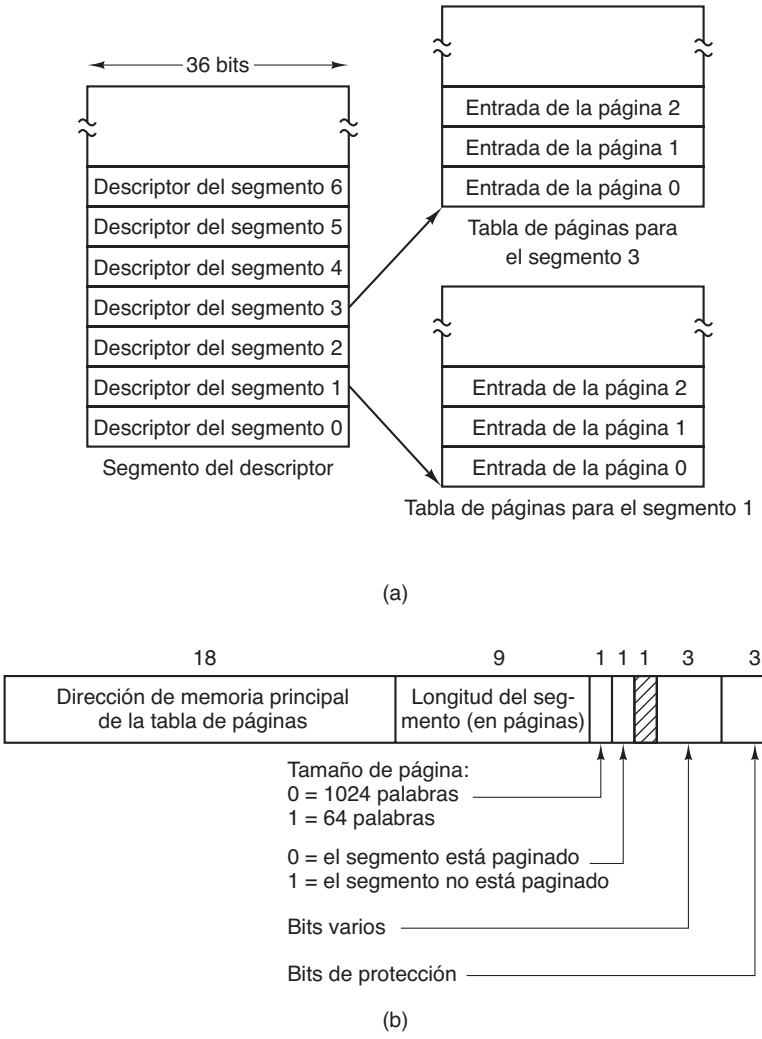


Figura 3-35. La memoria virtual de MULTICS. (a) El segmento del descriptor apunta a las tablas de páginas. (b) Un descriptor de segmento. Los números son las longitudes de los campos.

1. El número de segmento se utiliza para encontrar el descriptor de segmentos.

2. Se realiza una comprobación para ver si la tabla de páginas del segmento está en la memoria. Si la tabla de páginas está en memoria, se localiza. Si no, ocurre un fallo de segmento. Si hay una violación a la protección, ocurre un fallo (trap).

3. La entrada en la tabla de páginas para la página virtual solicitada se examina. Si la página en sí no está en memoria, se dispara un fallo de página. Si está en memoria, la direc-

ción de la memoria principal del inicio de la página se extrae de la entrada en la tabla de páginas.

4. El desplazamiento se agrega al origen de la página para obtener la dirección de memoria principal en donde se encuentra la palabra.

5. Finalmente se lleva a cabo la operación de lectura o almacenamiento.

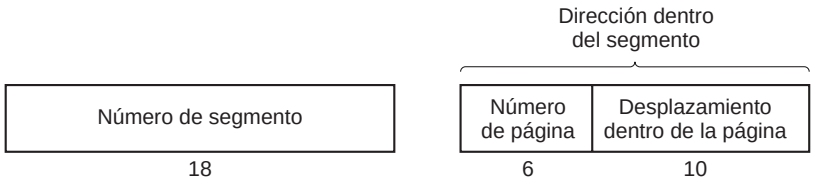


Figura 3-36. Una dirección virtual de MULTICS de 34 bits.

Este proceso se ilustra en la figura 3-37. Por simpleza, omitimos el hecho de que el mismo segmento del descriptor está paginado. Lo que ocurre en realidad es que se utiliza un registro (el registro base del descriptor) para localizar la tabla de páginas del segmento del descriptor, que a su vez apunta a las páginas del segmento del descriptor. Una vez que se ha encontrado el descriptor para el segmento necesario, el direccionamiento procede como se muestra en la figura 3-37.

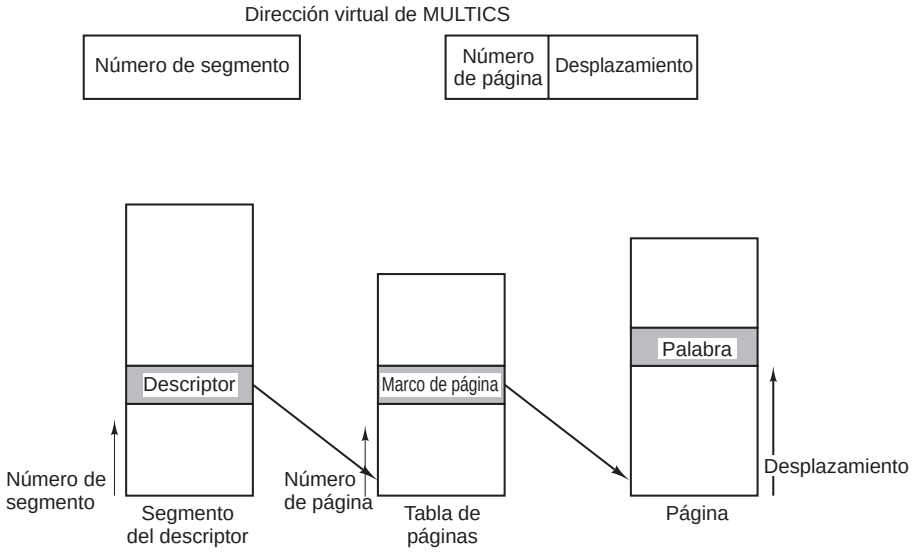


Figura 3-37. Conversión de una dirección MULTICS de dos partes en una dirección de memoria principal.

Como sin duda habrá adivinado para estos momentos, si el sistema operativo llevara a cabo el algoritmo anterior en cada instrucción, los programas no se ejecutarían con mucha rapidez. En realidad, el hardware de MULTICS contiene un TLB de 16 palabras de alta velocidad que puede buscar una llave dada en todas sus entradas en paralelo. Esto se ilustra en la figura 3-38. Cuando se

presenta una dirección a la computadora, el hardware de direccionamiento primero comprueba que la dirección virtual esté en el TLB. De ser así, obtiene el número del marco de página directamente del TLB y forma la dirección actual de la palabra referenciada sin tener que buscar en el segmento del descriptor o en la tabla de páginas.

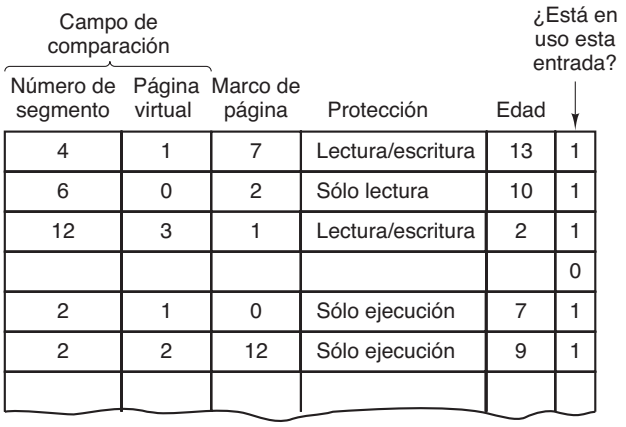


Figura 3-38. Una versión simplificada del TLB de MULTICS. La existencia de dos tamaños de página hace que el TLB real sea más complicado.

Las direcciones de las 16 páginas con referencia más reciente se mantienen en el TLB. Los programas cuyo conjunto de trabajo sea menor que el tamaño del TLB se equilibrarán con las direcciones de todo el conjunto de trabajo en el TLB, y por lo tanto se ejecutarán con eficiencia. Si la página no está en el TLB, se hace referencia al descriptor y las tablas de páginas para encontrar la dirección del marco, el TLB se actualiza para incluir esta página, y la página de uso menos reciente se descarta. El campo edad lleva el registro de cuál entrada es la de uso menos reciente. La razón de usar un TLB es para comparar los números de segmento y de página de todas las entradas en paralelo.

3.7.3 Segmentación con paginación: Intel Pentium

La memoria virtual en el Pentium se asemeja en muchas formas a MULTICS, incluyendo la presencia de segmentación y paginación. Mientras que MULTICS tiene 256K segmentos independientes, cada uno con hasta 64K palabras de 36 bits, el Pentium tiene 16K segmentos independientes, cada uno de los cuales contiene hasta un mil millones de palabras de 32 bits. Aunque hay menos segmentos, entre mayor sea el tamaño del segmento será más importante, ya que pocos programas necesitan más de 1000 segmentos, pero muchos programas necesitan segmentos extensos.

El corazón de la memoria virtual del Pentium consiste en dos tablas, llamadas **LDT** (*Local Descriptor Table*, Tabla de descriptores locales) y **GDT** (*Global Descriptor Table*, Tabla de descrip-

tores globales). Cada programa tiene su propia LDT, pero hay una sola GDT compartida por todos los programas en la computadora. La LDT describe los segmentos locales para cada programa, incluyendo su código, datos, pila, etcétera, mientras que la GDT describe los segmentos del sistema, incluyendo el sistema operativo en sí.

Para acceder a un segmento, un programa del Pentium primero carga un selector para ese segmento en uno de los seis registros de segmento de la máquina. Durante la ejecución, el registro CS contiene el selector para el segmento de código y el registro DS contiene el selector para el segmento de datos. Los demás registros de segmento son menos importantes. Cada selector es un número de 16 bits, como se muestra en la figura 3-39.

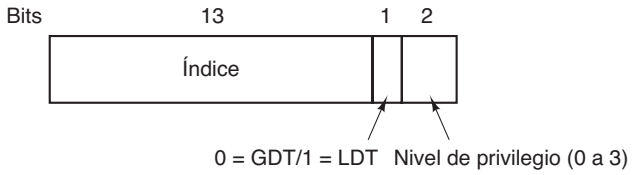


Figura 3-39. Un selector del Pentium.

Uno de los bits del selector indica si el segmento es local o global (es decir, si está en la LDT o en la GDT). Otros trece bits especifican el número de entrada en la LDT o GDT, por lo que estas tablas están restringidas a contener cada una 8K descriptores de segmento. Los otros 2 bits que se relacionan con la protección se describen más adelante. El descriptor 0 está prohibido. Puede cargarse de manera segura en un registro de segmento para indicar que el registro de segmento no está disponible en un momento dado. Produce un trap si se utiliza.

Al momento en que se carga un selector en un registro de segmento, el descriptor correspondiente se obtiene de la LDT o GDT y se almacena en registros de microprograma, por lo que se puede acceder con rapidez. Como se ilustra en la figura 3-40, un descriptor consiste de 8 bytes, incluyendo la dirección base del segmento, su tamaño y demás información.

El formato del selector se ha elegido con inteligencia para facilitar la localización del descriptor. Primero se selecciona la LDT o GDT, con base en el bit 2 del selector. Después el selector se copia a un registro temporal interno, y los 3 bits de menor orden se establecen en 0. Por último, se le suma la dirección de la tabla LDT o GDT para proporcionar un apuntador al descriptor. Por ejemplo, el selector 72 se refiere a la entrada 9 en la GDT, que se encuentra en la dirección GDT + 72.

Vamos a rastrear los pasos mediante los cuales un par (selector, desplazamiento) se convierte en una dirección física. Tan pronto como el microprograma sabe cuál registro de segmento se está utilizando, puede encontrar el descriptor completo que corresponde a ese selector en sus registros internos. Si el segmento no existe (selector 0) o en ese momento se paginó para sacarlo de memoria, se produce un trap.

Después, el hardware utiliza el campo *Límite* para comprobar si el desplazamiento está más allá del final del segmento, en cuyo caso también se produce un trap. Lógicamente debería haber un campo de 32 bits en el descriptor para proporcionar el tamaño del segmento, pero sólo hay 20 bits

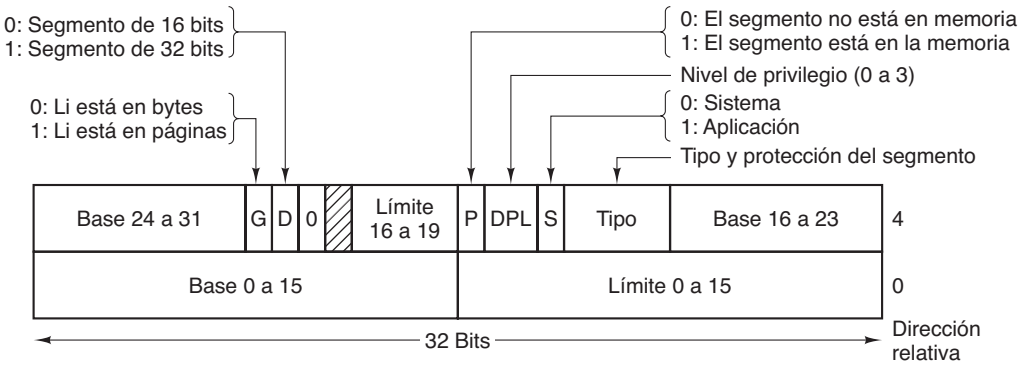


Figura 3-40. Descriptor del segmento de código del Pentium. Los segmentos de datos difieren un poco.

disponibles, por lo que se utiliza un esquema distinto. Si el campo *Gbit* (granularidad) es 0, el campo *Límite* es el tamaño de segmento exacto, hasta 1 MB. Si es 1, el campo *Límite* proporciona el tamaño del segmento en páginas, en vez de bytes. El tamaño de página del Pentium está fijo en 4 KB, por lo que 20 bits son suficientes para segmentos de hasta 2^{32} bytes.

Suponiendo que el segmento está en memoria y que el desplazamiento está en el rango, el Pentium suma el campo *Base* de 32 bits en el descriptor al desplazamiento para formar lo que se conoce como una **dirección lineal**, como se muestra en la figura 3-41. El campo *Base* se divide en tres partes y se esparce por todo el descriptor para tener compatibilidad con el 286, en donde el campo *Base* sólo tiene 24 bits. En efecto, el campo *Base* permite que cada segmento empiece en un lugar arbitrario dentro del espacio de direcciones lineal de 32 bits.

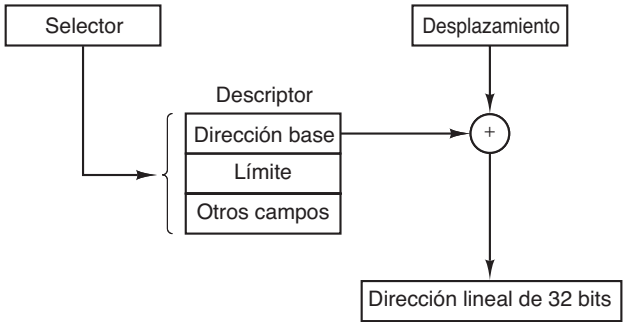


Figura 3-41. Conversión de un par (selector, desplazamiento) en una dirección lineal.

Si se deshabilita la paginación (mediante un bit en un registro de control global), la dirección lineal se interpreta como la dirección física y se envía a la memoria para la lectura o escritura. Por ende, con la paginación deshabilitada tenemos un esquema de segmentación pura, en donde la dirección base de cada segmento se proporciona en su descriptor. No se evita que los segmentos se

traslapen, probablemente debido a que sería demasiado problema y se requeriría mucho tiempo para verificar que todos estuvieran disjuntos.

Por otro lado, si la paginación está habilitada, la dirección lineal se interpreta como una dirección virtual y se asigna a la dirección física usando tablas de páginas, en forma muy parecida a los ejemplos anteriores. La única complicación verdadera es que con una dirección virtual de 32 bits y una página de 4 KB, un segmento podría contener 1 millón de páginas, por lo que se utiliza una asignación de dos niveles para reducir el tamaño de la tabla de páginas para segmentos pequeños.

Cada programa en ejecución tiene un **directorio de páginas** que consiste de 1024 entradas de 32 bits. Se encuentra en una dirección a la que apunta un registro global. Cada entrada en este directorio apunta a una tabla de páginas que también contiene 1024 entradas de 32 bits. Las entradas en la tabla de páginas apuntan a marcos de página. El esquema se muestra en la figura 3-42.

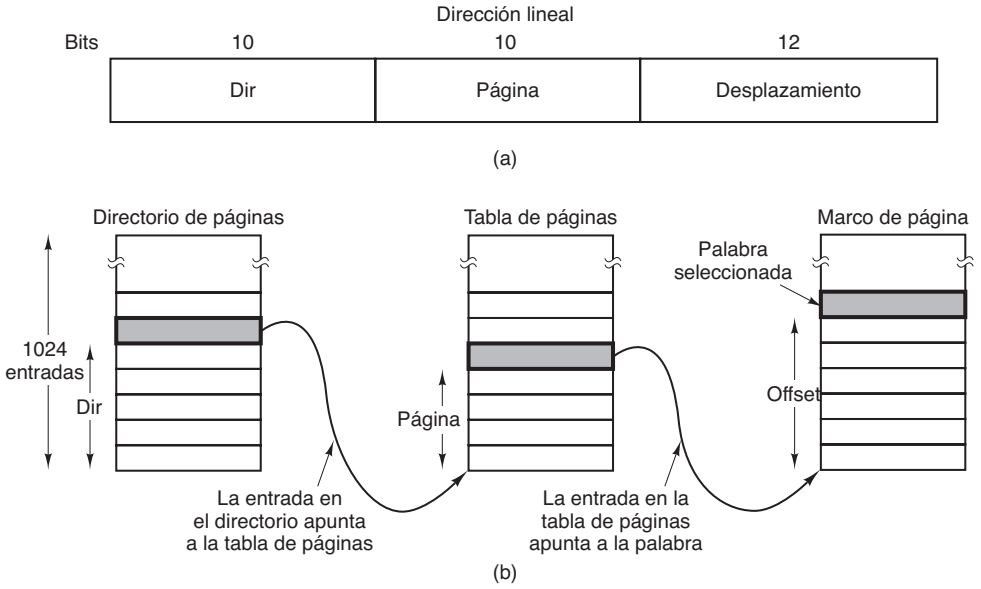


Figura 3-42. Asociación de una dirección lineal a una dirección física.

En la figura 3-42(a) podemos ver una dirección lineal dividida en tres campos: *Dir*, *Página* y *Desplazamiento*. El campo *Dir* se utiliza para indexar en el directorio de páginas y localizar un apuntador a la tabla de páginas apropiada. Después se utiliza el campo *Página* como un índice en la tabla de páginas para buscar la dirección física del marco de página. Por último, *Desplazamiento* se suma a la dirección del marco de página para obtener la dirección física del byte o palabra requerida.

Las entradas en la tabla de páginas son de 32 bits cada una, 20 de las cuales contienen un número de marco de página. Los bits restantes contienen bits de acceso y bits sucios, establecidos por el hardware para el beneficio del sistema operativo, bits de protección y otros bits utilitarios.

Cada tabla de páginas tiene entradas para 1024 marcos de página de 4 KB, por lo que una sola tabla de páginas maneja 4 megabytes de memoria. Un segmento menor de 4M tendrá un directo-

rio de páginas con una sola entrada: un apuntador a su única tabla de páginas. De esta manera, la sobrecarga por los segmentos cortos es sólo de dos páginas, en vez del millón de páginas que se necesitarían en una tabla de páginas de un nivel.

Para evitar realizar referencias repetidas a memoria, el Pentium (al igual que MULTICS) tiene un pequeño TLB que asigna directamente las combinaciones *Dir-Página* de uso más reciente a la dirección física del marco de página. Sólo cuando la combinación actual no está presente en el TLB es cuando se lleva a cabo el mecanismo de la figura 3-42 y se actualiza el TLB. Mientras los fracasos del TLB sean raros, el rendimiento será bueno.

También vale la pena observar que si alguna aplicación no necesita segmentación pero está contenta con un solo espacio de direcciones paginado de 32 bits, ese modelo es posible. Todos los registros de segmento se pueden establecer con el mismo selector, cuyo descriptor tiene *Base* = 0 y *Límite* establecido al máximo. Entonces el desplazamiento de la instrucción será la dirección lineal, con sólo un espacio de direcciones único utilizado, en efecto, una paginación normal. De hecho, todos los sistemas operativos actuales para el Pentium funcionan de esta manera. OS/2 fue el único que utilizó todo el poder de la arquitectura Intel MMU.

Con todo, hay que dar crédito a los diseñadores del Pentium. Dadas las metas conflictivas por implementar la paginación pura, la segmentación pura y los segmentos paginados, al tiempo que debía ser compatible con el 286, y todo esto había que hacerlo con eficiencia. El diseño resultante es sorprendentemente simple y limpio.

Aunque hemos cubierto en forma breve la arquitectura completa de la memoria virtual del Pentium, vale la pena decir unas cuantas palabras acerca de la protección, ya que este tema está estrechamente relacionado con la memoria virtual. Al igual que el esquema de memoria virtual está modelado en forma muy parecida a MULTICS, el sistema de protección también lo está. El Pentium admite cuatro niveles de protección, donde el nivel 0 es el más privilegiado y el 3 el menos privilegiado. Éstos se muestran en la figura 3-43. En cada instante, un programa en ejecución se encuentra en cierto nivel, indicado por un campo de 2 bits en su PSW. Cada segmento en el sistema también tiene un nivel.

Mientras que un programa se restrinja a sí mismo a utilizar segmentos en su propio nivel, todo funcionará bien. Se permiten los intentos de acceder a los datos en un nivel más alto, pero los de acceder a los datos en un nivel inferior son ilegales y producen traps. Los intentos de llamar procedimientos en un nivel distinto (mayor o menor) se permiten, pero de una manera controlada cuidadosamente. Para realizar una llamada entre niveles, la instrucción CALL debe contener un selector en vez de una dirección. Este selector designa a un descriptor llamado **compuerta de llamada**, el cual proporciona la dirección del procedimiento al que se va a llamar. Por ende, no es posible saltar en medio de un segmento de código arbitrario en un nivel distinto. Sólo se pueden utilizar puntos de entrada oficiales. Los conceptos de los niveles de protección y las compuertas de llamada se utilizaron por primera vez en MULTICS, en donde se denominaron **anillos de protección**.

Un uso común para este mecanismo se sugiere en la figura 3-43. En el nivel 0 encontramos el kernel del sistema operativo, que se encarga de la E/S, la administración de memoria y otras cuestiones críticas. En el nivel 1 está presente el manejador de llamadas al sistema. Los programas de usuario pueden llamar procedimientos aquí para llevar a cabo las llamadas al sistema, pero sólo se puede llamar a una lista de procedimientos específicos y protegidos. El nivel 2 contiene procedimientos de biblioteca, posiblemente compartida entre muchos programas en ejecución. Los progra-

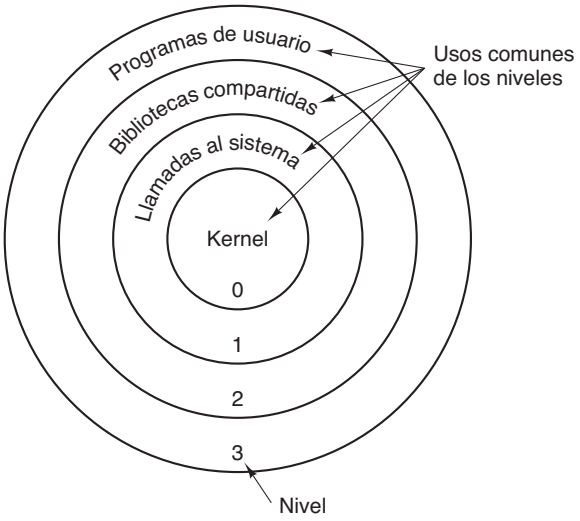


Figura 3-43. La protección en el Pentium.

mas de usuario pueden llamar a estos procedimientos y leer sus datos, pero no pueden modificarlos. Por último, los programas de usuario se ejecutan en nivel 3, que tiene la menor protección.

Los traps y las interrupciones utilizan un mecanismo similar a las compuertas de llamadas. También hacen referencia a los descriptores en vez de direcciones absolutas, y estos descriptores apuntan a los procedimientos específicos que se van a ejecutar. El campo *Tipo* en la figura 3-40 indica las diferencias entre los segmentos de código, los segmentos de datos y los diversos tipos de compuertas.

3.8 INVESTIGACIÓN ACERCA DE LA ADMINISTRACIÓN DE MEMORIA

La administración de memoria, en especial los algoritmos de paginación, fue alguna vez un área fructífera para la investigación, pero la mayor parte de eso parece haber desaparecido desde hace mucho tiempo, por lo menos para los sistemas de propósito general. La mayoría de los sistemas reales tienden a utilizar cierta variación sobre el reloj, debido a que es fácil de implementar y relativamente efectivo. Sin embargo, una excepción reciente es un rediseño del sistema de memoria virtual de BSD 4.4 (Cranor y Prulkar, 1999).

Sin embargo, aún se están realizando investigaciones sobre la paginación en los tipos más recientes de sistemas. Por ejemplo, los teléfonos celulares y los PDAs se han convertido en pequeñas PCs y muchps de ellas paginan la RAM al “disco”, sólo que el disco en un teléfono celular es la memoria flash, que tiene propiedades distintas a las de un disco magnético giratorio. Cierto trabajo reciente se reporta (In y colaboradores, 2007; Joo y colaboradores, 2006; y Park y colaboradores, 2004a). Park y colaboradores (2004b) también han analizado la paginación por demanda consciente de la energía en los dispositivos móviles.

También se están realizando investigaciones acerca del modelado del rendimiento de la paginación (Albers y colaboradores, 2002; Burton y Kelly, 2003; Cascaval y colaboradores, 2005; Panagiotou y Souza, 2006; y Peserico, 2003). También es de interés la administración de memoria para los sistemas multimedia (Dasigenis y colaboradores, 2001; Hand, 1999) y los sistemas de tiempo real (Pizlo y Vitek, 2006).

3.9 RESUMEN

En este capítulo examinamos la administración de memoria. Vimos que los sistemas más simples no realizan intercambios ni paginaciones. Una vez que se carga un programa en la memoria, permanece ahí hasta que termina. Algunos sistemas operativos sólo permiten un proceso a la vez en la memoria, mientras que otros soportan la multiprogramación.

El siguiente paso es el intercambio. Cuando se utiliza esta técnica, el sistema puede manejar más procesos de los que puede alojar en la memoria. Los procesos para los cuales no haya espacio se intercambian hacia el disco. Se puede llevar el registro del espacio libre en la memoria y en el disco con un mapa de bits o una lista de huecos.

A menudo, las computadoras modernas tienen cierta forma de memoria virtual. En su forma más simple, el espacio de direcciones de cada proceso se divide en bloques de tamaño uniforme llamados páginas, que pueden colocarse en cualquier marco de página disponible en la memoria. Hay muchos algoritmos de reemplazo de páginas; dos de los mejores algoritmos son el de envejecimiento y WSClock.

Para modelar los sistemas de paginación hay que abstraer la cadena de referencia de página del programa y utilizar la misma cadena de referencia con distintos algoritmos. Estos modelos se pueden utilizar para hacer algunas predicciones acerca del comportamiento de la paginación.

Para hacer que los sistemas de paginación funcionen bien, no basta con elegir un algoritmo; hay que poner atención en cuestiones tales como determinar el conjunto de trabajo, la política de asignación de memoria y el tamaño de página.

La segmentación ayuda a manejar las estructuras de datos que pueden cambiar de tamaño durante la ejecución, y simplifica la vinculación y la compartición. También facilita la provisión de distintos tipos de protección para distintos segmentos. Algunas veces la segmentación y la paginación se combinan para proporcionar una memoria virtual bidimensional. El sistema MULTICS y el Intel Pentium soportan segmentación y paginación.

PROBLEMAS

1. En la figura 3-3 los registros base y límite contienen el mismo valor, 16,384. ¿Es esto un accidente, o siempre son iguales? Si esto es sólo un accidente, ¿por qué son iguales en este ejemplo?

2. Un sistema de intercambio elimina huecos mediante la compactación. Suponiendo una distribución aleatoria de muchos huecos y muchos segmentos de datos y un tiempo de lectura o escritura en una palabra de memoria de 32 bits de 10 nseg, ¿aproximadamente cuánto tiempo se requiere para compactar 128 MB? Para simplificar, suponga que la palabra 0 es parte de un hueco y que la palabra más alta en la memoria contiene datos válidos.

3. En este problema tiene que comparar el almacenamiento necesario para llevar la cuenta de la memoria libre, utilizando un mapa de bits contra el uso de una lista ligada. La memoria de 128 MB se asigna en unidades de *n* bytes. Para la lista enlazada, suponga que la memoria consiste en una secuencia alternante de segmentos y huecos, cada uno de 64 KB. Suponga también que cada nodo en la lista enlazada necesita una dirección de memoria de 32 bits, una longitud de 16 bits y un campo para el siguiente nodo de 16 bits. ¿Cuánto bytes de almacenamiento se requieren para cada método? ¿Cuál es mejor?

4. Considere un sistema de intercambio en el que la memoria consiste en los siguientes tamaños de hueco, por orden de memoria: 10 KB, 4 KB, 20 KB, 18 KB, 7 KB, 9 KB, 12 KB y 15 KB. ¿Cuál hueco se toma para las siguientes solicitudes de segmento sucesivas:

   a) 12 KB
   b) 10 KB
   c) 9 KB

   para el algoritmo del primer ajuste? Ahora repita la pregunta para el mejor ajuste, peor ajuste y siguiente ajuste.

5. Para cada una de las siguientes direcciones virtuales decimales, calcule el número de página virtual y desplazamiento para una página de 4 KB y para una página de 8 KB: 20000, 32768, 60000.

6. El procesador Intel 8086 no admite memoria virtual. Sin embargo, algunas compañías vendían anteriormente sistemas que contenían una CPU 8086 sin modificaciones y realizaba la paginación. Trate de llegar a una conclusión lógica acerca de cómo lo hicieron. *Sugerencia:* piense acerca de la ubicación lógica de la MMU.

7. Considere el siguiente programa en C:

   ```
   int X[N];
   int paso = M; // M es una constante predefinida
   for (int i = 0; i < N; i += paso) X[i] = X[i] + 1;
   ```

   a) Si este programa se ejecuta en una máquina con un tamaño de página de 4 KB y un TLB con 64 entradas, ¿qué valores de M y N harán que un TLB falle para cada ejecución del ciclo interno?

   b) ¿Sería distinta su respuesta al inciso a) si el ciclo se repitiera muchas veces? Explique.

8. La cantidad de espacio en disco que debe estar disponible para el almacenamiento de páginas está relacionada con el número máximo de procesos *n,*, el número de bytes en el espacio de direcciones virtual *v,* así como con el número de bytes de RAM *r*. Proporcione una expresión para los requerimientos de espacio en disco en el peor de los casos. ¿Qué tan realista es esa cantidad?

9. Una máquina tiene un espacio de direcciones de 32 bits y una página de 8 KB. La tabla de páginas está completamente en el hardware, con una palabra de 32 bits por cada entrada. Cuando se inicia un proceso, la tabla de páginas se copia al hardware desde la memoria, una palabra por cada 100 nseg. Si cada proceso se ejecuta durante 100 mseg (incluyendo el tiempo para cargar la tabla de páginas), ¿qué fracción del tiempo de la CPU se dedica a cargar las tablas de páginas?

10. Suponga que una máquina tiene direcciones virtuales de 48 bits y direcciones físicas de 32 bits.

   a) Si las páginas son de 4 KB, ¿Cuántas entradas hay en la tabla de páginas si sólo hay un nivel? Explique.

b) Suponga que el mismo sistema tiene un TLB (Búfer de traducción adelantada) con 32 entradas. Además, suponga que un programa contiene instrucciones que caben en una página y lee secuencialmente elementos enteros largos de un arreglo que abarca miles de páginas. ¿Qué tan efectivo será el TLB para este caso?

11. Suponga que una máquina tiene direcciones virtuales de 38 bits y direcciones físicas de 32 bits.

a) ¿Cuál es la principal ventaja de una tabla de páginas de multinivel sobre una tabla de páginas de un solo nivel?

b) Con una tabla de páginas de dos niveles, páginas de 16 KB y entradas de 4 bytes, ¿cuántos bits se deben asignar para el campo de la tabla de páginas de nivel superior y cuántos para el campo de la tabla de páginas del siguiente nivel? Explique.

12. Una computadora con una dirección de 32 bits utiliza una tabla de páginas de dos niveles. Las direcciones virtuales se dividen en un campo de la tabla de páginas de nivel superior de 9 bits, un campo de la tabla de páginas de segundo nivel de 11 bits y un desplazamiento. ¿Qué tan grandes son las páginas y cuántas hay en el espacio de direcciones?

13. Suponga que una dirección virtual de 32 bits se divide en cuatro campos: *a*, *b*, *c* y *d*. Los primeros tres se utilizan para un sistema de tablas de páginas de tres niveles. El cuarto campo (*d*) es el desplazamiento. ¿Depende el número de páginas de los tamaños de los cuatro campos? Si no es así, ¿cuáles importan y cuáles no?

14. Una computadora tiene direcciones virtuales de 32 bits y páginas de 4 KB. El programa y los datos caben juntos en la página más baja (0 a 4095). La pila cabe en la página más alta. ¿Cuántas entradas se necesitan en la tabla de páginas si se utiliza la paginación tradicional (un nivel)? ¿Cuántas entradas en la tabla de páginas se necesitan para la paginación de dos niveles, con 10 bits en cada parte?

15. Una computadora cuyos procesos tienen 1024 páginas en sus espacios de direcciones mantiene sus tablas de páginas en memoria. La sobrecarga requerida para leer una palabra de la tabla de páginas es 5 nseg. Para reducir esta sobrecarga, la computadora tiene un TLB que contiene 32 pares (página virtual, marco de página física) y puede realizar una búsqueda en 1 nseg. ¿Qué proporción de aciertos necesita para reducir la sobrecarga promedio a 2 nseg?

16. El TLB en la VAX no contiene un bit *R*. ¿Por qué?

17. ¿Cómo puede implementarse en hardware el dispositivo de memoria asociativa necesario para implementar una TLB y cuáles son las implicaciones de dicho diseño para que sea expandible?

18. Una máquina tiene direcciones virtuales de 48 bits y direcciones físicas de 32 bits. Las páginas son de 8 KB. ¿Cuántas entradas se necesitan para la tabla de páginas?

19. Una computadora con una página de 8 KB, una memoria principal de 256 KB y un espacio de direcciones virtuales de 64 GB utiliza una tabla de páginas invertida para implementar su memoria virtual. ¿Qué tan grande debe ser la tabla de hash para asegurar una cadena de hash de una longitud promedio menor a 1? Suponga que el tamaño de la tabla de hash es una potencia de dos.

20. Un estudiante en un curso de diseño de compiladores propone al profesor un proyecto de escribir un compilador que produzca una lista de referencias a páginas que se puedan utilizar para implementar el algoritmo de reemplazo de páginas óptimo. ¿Es esto posible? ¿Por qué sí o por qué no? ¿Hay algo que pudiera hacerse para mejorar la eficiencia de la paginación en tiempo de ejecución?

21. Suponga que el flujo de referencia de páginas virtuales contiene repeticiones de largas secuencias de referencias a páginas, seguidas ocasionalmente por una referencia a una página aleatoria. Por ejemplo, la secuencia 0, 1, …, 511, 431, 0, 1, …, 511, 332, 0, 1, … consiste en repeticiones de la secuencia 0, 1, …, 511 seguida de una referencia aleatoria a las páginas 431 y 332.

a) ¿Por qué no serían efectivos los algoritmos de sustitución estándar (LRU, FIFO, Reloj) al manejar esta carga de trabajo para una asignación de página que sea menor que la longitud de la secuencia?
b) Si a este programa se le asignaran 500 marcos de página, describa un método de sustitución de página que tenga un rendimiento mucho mejor que los algoritmos LRU, FIFO o Reloj.

22. Si se utiliza el algoritmo FIFO de reemplazo de páginas con cuatro marcos de página y ocho páginas, ¿cuántos fallos de página ocurrirán con la cadena de referencia 0172327103 si los cuatro marcos están vacíos al principio? Ahora repita este problema para el algoritmo LRU.

23. Considere la secuencia de páginas de la figura 3-15(b). Suponga que los bits *R* para las páginas de la *B* a la *A* son 11011011, respectivamente, ¿Cuál página eliminará el algoritmo de segunda oportunidad?

24. Una pequeña computadora tiene cuatro marcos de página. En el primer pulso de reloj, los bits *R* son 0111 (la página 0 es 0, el resto son 1). En los siguientes pulsos de reloj, los valores son 1011, 1010, 1101, 0010, 1010, 1100 y 0001. Si se utiliza el algoritmo de envejecimiento con un contador de 8 bits, proporcione los valores de los cuatro contadores después del último pulso.

25. Dé un ejemplo simple de una secuencia de referencias a páginas en donde la primera página seleccionada para la sustitución sea diferente para los algoritmos de reemplazo de páginas de reloj y LRU. Suponga que a un proceso se le asignan 3 marcos y que la cadena de referencia contiene números de página del conjunto 0, 1, 2, 3.

26. En el algoritmo WSClock de la figura 3-21(c), la manecilla apunta a una página con $R = 0$. Si $\tau = 400$, ¿se eliminará esta página? ¿Qué pasa si $\tau = 1000$?

27. ¿Cuánto tiempo se requiere para cargar un programa de 64 KB de un disco cuyo tiempo de búsqueda promedio es de 10 mseg, cuyo tiempo de rotación es de 10 mseg y cuyas pistas contienen 32 KB

a) para un tamaño de página de 2 KB?
b) para un tamaño de página de 4 KB?

Las páginas están esparcidas de manera aleatoria alrededor del disco y el número de cilindros es tan grande que la probabilidad de que dos páginas se encuentren en el mismo cilindro es insignificante.

28. Una computadora tiene cuatro marcos de página. El tiempo de carga, tiempo del último acceso y los bits *R* y *M* para cada página se muestran a continuación (los tiempos están en pulsos de reloj):

Página	Cargada	Última referencia	R	M
0	126	280	1	0
1	230	265	0	1
2	140	270	0	0
3	110	285	1	1

a) ¿Cuál página reemplazará el algoritmo NRU?
b) ¿Cuál página reemplazará el algoritmo FIFO?
c) ¿Cuál página reemplazará el algoritmo LRU?
d) ¿Cuál página reemplazará el algoritmo de segunda oportunidad?

29. Considere el siguiente arreglo bidimensional:

```
int X[64][64];
```

Suponga que un sistema tiene cuatro marcos de página y que cada marco es de 128 palabras (un entero ocupa una palabra). Los programas que manipulan el arreglo X caben exactamente en una página y siempre ocupan la página 0. Los datos se intercambian hacia dentro y hacia fuera de los otros tres marcos. El arreglo *X* se almacena en orden de importancia por filas (es decir, *X*[0][1] va después de *X*[0][0] en la memoria). ¿Cuál de los dos fragmentos de código que se muestran a continuación generarán el menor número de fallos de página? Explique y calcule el número total de fallos de página.

Fragmento A

```
for (int j = 0; j < 64; j++)
  for (int i = 0; i < 64; i++) X[i][j] = 0;
```

Fragmento B

```
for (int i = 0; i < 64; i++)
  for (int j = 0; j < 64; j++) X[i][j] = 0;
```

30. Una de las primeras máquinas de tiempo compartido (la PDP-1) tenía una memoria de 4K palabras de 18 bits. Contenía un proceso a la vez en la memoria. Cuando el planificador de proceso decidía ejecutar otro proceso, el proceso en memoria se escribía en un tambor de paginación, con 4K palabras de 18 bits alrededor de la circunferencia del tambor, el cual podía empezar a escribir (o leer) en cualquier palabra, en vez de hacerlo sólo en la palabra 0. ¿Supone usted que este tambor fue seleccionado?

31. Una computadora proporciona a cada proceso 65,536 bytes de espacio de direcciones, dividido en páginas de 4096 bytes. Un programa específico tiene un tamaño de texto de 32,768 bytes, un tamaño de datos de 16,386 bytes y un tamaño de pila de 15,870 bytes. ¿Cabrá este programa en el espacio de direcciones? Si el tamaño de página fuera de 512 bytes, ¿cabría? Recuerde que una página no puede contener partes de dos segmentos distintos.

32. ¿Puede una página estar en dos conjuntos de trabajo al mismo tiempo? Explique.

33. Se ha observado que el número de instrucciones ejecutadas entre fallos de página es directamente proporcional al número de marcos de página asignados a un programa. Si la memoria disponible se duplica, el intervalo promedio entre los fallos de página también se duplica. Suponga que una instrucción normal requiere 1 microsegundo, pero si ocurre un fallo de página, requiere 2001 μseg (es decir, 2 mseg para hacerse cargo del fallo). Si un programa requiere 60 segundos para ejecutarse, tiempo durante el cual obtiene 15,000 fallos de página, ¿cuánto tiempo requeriría para ejecutarse si hubiera disponible el doble de memoria?

34. Un grupo de diseñadores de sistemas operativos para la Compañía de Computadoras Frugal están ideando maneras de reducir la cantidad de almacenamiento de respaldo necesario en su nuevo sistema operativo. El jefe de ellos ha sugerido que no se deben preocupar por guardar el texto del programa en el área de intercambio, sino sólo paginarla directamente desde el archivo binario cada vez

que se necesite. ¿Bajo qué condiciones, si las hay, funciona esta idea para el texto del programa? ¿Bajo qué condiciones, si las hay, funciona para los datos?

35. Una instrucción en lenguaje máquina para cargar una palabra de 32 bits en un registro contiene la dirección de 32 bits de la palabra que se va a cargar. ¿Cuál es el número máximo de fallos de página que puede provocar esta instrucción?

36. Cuando se utilizan la segmentación y la paginación, como en MULTICS, primero se debe buscar el descriptor del segmento y después el descriptor de página. ¿Funciona el TLB también de esta manera, con dos niveles de búsqueda?

37. Consideremos un programa que tiene los dos segmentos que se muestran a continuación, los cuales consisten de instrucciones en el segmento 0 y datos de lectura/escritura en el segmento 1. El segmento 0 tiene protección de lectura/ejecución y el segmento 1 tiene protección de lectura/escritura. El sistema de memoria es un sistema de memoria virtual con paginación bajo demanda, con direcciones virtuales que tienen un número de página de 4 bits y un desplazamiento de 10 bits. Las tablas de páginas y la protección son las siguientes (todos los números en la tabla están en decimal):

Segmento 0		**Segmento 1**	
Lectura/ejecución		**Lectura/escritura**	
# de página virtual	**# de marco de página**	**# delpágina virtual**	**# de marco de página**
0	2	0	En disco
1	En disco	1	14
2	11	2	9
3	5	3	6
4	En disco	4	En disco
5	En disco	5	13
6	4	6	8
7	3	7	12

Para cada uno de los siguientes casos, proporcione la dirección de memoria real (actual) que resulta de la traducción de direcciones dinámicas o identifique el tipo de fallo que ocurre (ya sea fallo de página o de protección).

a) Obtener del segmento 1, página 1, desplazamiento 3
b) Almacenar en segmento 0, página 0, desplazamiento 16
c) Obtener del segmento 1, página 4, desplazamiento 28
d) Saltar a la ubicación en el segmento 1, página 3, desplazamiento 32

38. ¿Puede pensar en alguna situación en donde el soporte de la memoria virtual fuera una mala idea y qué se ganaría al no tener que soportar la memoria virtual? Explique.

39. Trace un histograma y calcule la media y la mediana de los tamaños de los archivos binarios ejecutables en una computadora a la que tenga acceso. En un sistema Windows, analice todos los archivos .exe y .dll; en un sistema UNIX analice todos los archivos ejecutables en */bin*, */usr/bin* y

/local/bin que no sean secuencias de comandos (o utilice la herramienta *file* para buscar todos los ejecutables). Determine el tamaño de página óptimo para esta computadora, considerando sólo el código (no los datos). Considere la fragmentación interna y el tamaño de la tabla de páginas, haciendo alguna suposición razonable acerca del tamaño de una entrada en la tabla de páginas. Suponga que todos los programas tienen la misma probabilidad de ejecutarse, y por ende deben considerarse con el mismo peso.

40. Los pequeños programas para MS-DOS se pueden compilar como archivos *.COM*. Estos archivos siempre se cargan en la dirección 0x100 en un solo segmento de memoria que se utilice para código, datos y pila. Las instrucciones que transfieren el control de la ejecución, como **JMP** y **CALL**, o que acceden a datos estáticos desde direcciones fijas hacen que las instrucciones se compilen en el código objeto. Escriba un programa que pueda reubicar dicho archivo de programa para ejecutarlo empezando en una dirección arbitraria. Su programa debe explorar el código en busca de códigos objeto para instrucciones que hagan referencia a direcciones de memoria fijas, después debe modificar esas direcciones que apunten a ubicaciones de memoria dentro del rango a reubicar. Encontrará los códigos objeto en un libro de programación en lenguaje ensamblador. Tenga en cuenta que hacer esto perfectamente sin información adicional es, en general, una tarea imposible debido a que ciertas palabras de datos pueden tener valores similares a los códigos objeto de las instrucciones.

41. Escriba un programa que simule un sistema de paginación utilizando el algoritmo de envejecimiento. El número de marcos de página es un parámetro. La secuencia de referencias a páginas debe leerse de un archivo. Para un archivo de entrada dado, dibuje el número de fallos de página por cada 1000 referencias a memoria como una función del número de marcos de página disponibles.

42. Escriba un programa para demostrar el efecto de los fallos del TLB en el tiempo de acceso efectivo a la memoria, midiendo el tiempo por cada acceso que se requiere para recorrer un arreglo extenso.

a) Explique los conceptos principales detrás del programa y describa lo que espera que muestre la salida para alguna arquitectura de memoria virtual práctica.
b) Ejecute el programa en una computadora y explique qué tan bien se ajustan los datos a sus expectativas.
c) Repita la parte b) pero para una computadora más antigua con una arquitectura distinta y explique cualquier diferencia importante en la salida.

43. Escriba un programa que demuestre la diferencia entre el uso de una política de reemplazo de páginas local y una global para el caso simple de dos procesos. Necesitará una rutina que pueda generar una cadena de referencias a páginas basado en un modelo estadístico. Este modelo tiene *N* estados enumerados de 0 a N-1, los cuales representan cada una de las posibles referencias a páginas y una probabilidad p_1 asociada con cada estado *i* que represente la probabilidad de que la siguiente referencia sea a la misma página. En caso contrario, la siguiente referencia a una página será a una de las otras páginas con igual probabilidad.

a) Demuestre que la rutina de generación de la cadena de referencias a páginas se comporta en forma apropiada para cierta *N* pequeña.
b) Calcule la proporción de fallos de página para un pequeño ejemplo en el que hay un proceso y un número fijo de marcos de página. Explique por qué es correcto el comportamiento.
c) Repita la parte b) con dos procesos con secuencias de referencias a páginas independientes, y el doble de marcos de página que en la parte (b).
d) Repita la parte c) utilizando una política global en vez de una local. Además, compare la proporción de fallos de página por proceso con la del método de política local.

8

SISTEMAS DE MÚLTIPLES PROCESADORES

Desde su inicio, la industria de las computadoras se ha orientado fundamentalmente a buscar sin pausa un poder de cómputo cada vez mayor. La ENIAC podía realizar 300 operaciones por segundo —es decir, era 1000 veces más rápida que cualquier calculadora anterior a ella— pero aún así las personas no estaban satisfechas. Las máquinas actuales son millones de veces más rápidas que la ENIAC y sigue existiendo una demanda de mayor potencia de cómputo. Los astrónomos están tratando de entender el universo, los biólogos tratan de comprender las implicaciones del genoma humano y los ingenieros aeronáuticos están interesados en construir aeronaves más seguras y eficientes, y todos ellos requieren más ciclos de la CPU. No importa cuánto poder de cómputo haya disponible, nunca será suficiente.

En el pasado, la solución era siempre hacer que el reloj operara a mayor velocidad. Por desgracia, estamos comenzando a toparnos con ciertos límites fundamentales sobre la velocidad del reloj. De acuerdo con la teoría especial de la relatividad de Einstein, ninguna señal eléctrica se puede propagar más rápido que la velocidad de la luz, que es de aproximadamente 30 cm/nseg en el vacío y 20 cm/nseg en el alambre de cobre o la fibra óptica. Esto significa que en una computadora con un reloj de 10 GHz, las señales no pueden viajar más de 2 cm en total. Para una computadora de 100 GHz la longitud total de la ruta es cuando mucho de 2 mm. Una computadora de 1 THz (1000 GHz) tendría que ser más pequeña que 100 micrones, sólo para dejar que la señal pase de un extremo al otro y regrese una vez, dentro de un solo ciclo de reloj.

Tal vez sea posible fabricar computadoras así de pequeñas, pero entonces llegamos a otro problema fundamental: la disipación del calor. Entre más rápido opera la computadora, más calor genera y entre más pequeña sea, más difícil es deshacerse de este calor. De antemano en los sistemas

Pentium de alto rendimiento, el enfriador de la CPU es más grande que la misma CPU. Con todo, para pasar de 1 MHz a 1 GHz se requirió sólo una ingeniería cada vez mejor del proceso de fabricación del chip; para pasar de 1 GHz a 1 THz se va a requerir una metodología radicalmente distinta.

Una metodología para obtener una mayor velocidad es a través de las computadoras masivamente en paralelo. Estas máquinas consisten en muchas CPUs, cada una de las cuales opera a una velocidad "normal" (lo que eso signifique en cierto año específico), pero que en conjunto tienen mucho más poder de cómputo que una sola CPU. Ahora hay sistemas con 1000 CPUs disponibles comercialmente. Es probable que se construyan sistemas con 1 millón de CPUs en la siguiente década. Aunque hay otras metodologías potenciales para obtener mayor velocidad, como las computadoras biológicas, en este capítulo nos enfocaremos en los sistemas con varias CPUs convencionales.

Las computadoras altamente paralelas se utilizan con frecuencia para el cálculo numérico intensivo. Problemas como la predicción del clima, el modelado del flujo de aire alrededor del ala de una aeronave, la simulación de la economía mundial o la comprensión de las interacciones de los receptores de fármacos en el cerebro son actividades que requieren de cómputo intensivo. Sus soluciones requieren largas ejecuciones en muchas CPUs a la vez. Los sistemas de múltiples procesadores que analizaremos en este capítulo se utilizan muchos para estos problemas y otros similares en la ciencia y en la ingeniería, entre otras áreas.

Otro desarrollo relevante es el increíblemente rápido crecimiento de Internet. Se diseñó en su origen como un prototipo para un sistema de control militar tolerante a fallas, después se hizo popular entre los científicos computacionales universitarios, y desde hace tiempo adquirió muchos usos nuevos. Uno de ellos es vincular miles de computadoras en todo el mundo para que trabajen en conjunto en problemas científicos grandes. En cierto grado, un sistema que consiste en 1000 computadoras esparcidas por todo el mundo no es distinto a uno que consiste en 1000 computadoras en un solo cuarto, aunque el retraso y otras características técnicas son diferentes. También consideraremos estos sistemas en este capítulo.

Es fácil colocar 1 millón de computadoras no relacionadas en una habitación, siempre y cuando se tenga el dinero suficiente… y una habitación lo bastante grande. Esparcir 1 millón de computadoras no relacionadas alrededor del mundo es incluso más sencillo, ya que no se tiene el segundo problema. Las dificultades empiezan cuando queremos que las computadoras se comuniquen entre sí para que trabajen en conjunto en un solo problema. Como consecuencia, se ha realizado mucho trabajo en cuanto a la tecnología de interconexión, y distintas tecnologías de interconexión han producido tipos de sistemas con diferente calidad y distintas organizaciones de software.

En última instancia, toda la comunicación entre los componentes electrónicos (u ópticos) se basa en el envío de mensajes (cadenas de bits bien definidas) entre ellos. Las diferencias están en la escala de tiempo, la escala de distancia y la organización lógica involucrada. En un extremo se encuentran los multiprocesadores de memoria compartida, en los cuales cierta cantidad de CPUs (entre 2 y aproximadamente 1000) se comunican a través de una memoria compartida. En este modelo, cada CPU tiene el mismo acceso a toda la memoria física, y puede leer y escribir palabras individuales mediante instrucciones `LOAD` y `STORE`. Para acceder a una palabra de memoria por lo general se requieren de 2 a 10 nseg. Aunque este modelo, que se ilustra en la figura 8-1(a), parece simple, en realidad no es tan sencillo implementarlo y por lo general implica el paso de muchos mensajes

de manera interna, como explicaremos en breve. Sin embargo, este proceso de paso de mensajes es invisible para los programadores.

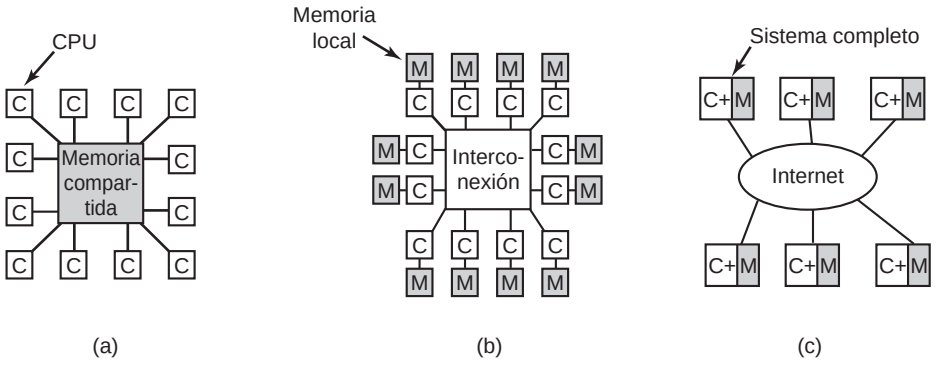


Figura 8-1. (a) Un multiprocesador con memoria compartida. (b) Una multicomputadora con paso de mensajes. (c) Un sistema distribuido de área amplia.

A continuación tenemos el sistema de la figura 8-1(b), en el cual varios pares de CPU-memoria se conectan a una interconexión de alta velocidad. A este tipo de sistema se le conoce como multicomputadora con paso de mensajes. Cada memoria es local para una sola CPU y puede ser utilizada sólo por esa CPU. Las CPUs se comunican enviando mensajes de varias palabras a través de la interconexión. Con una buena interconexión, un mensaje corto se puede enviar en un tiempo de 10 a 50 µseg, pero aún es más largo que el tiempo de acceso a la memoria de la figura 8-1(a). No hay memoria global compartida en este diseño. Las multicomputadoras (es decir, sistemas de paso de mensajes) son mucho más fáciles de construir que los multiprocesadores (memoria compartida), pero son más difíciles de programar. Por lo tanto, cada género tiene sus admiradores.

El tercer modelo, que se ilustra en la figura 8-1(c), conecta sistemas de cómputo completos a través de una red de área amplia como Internet, para formar un **sistema distribuido**. Cada uno de estos sistemas tiene su propia memoria, y los sistemas se comunican mediante el paso de mensajes. La única diferencia real entre la figura 8-1(b) y la figura 8-1(c) es que en esta última se utilizan computadoras completas, y los tiempos de los mensajes son comúnmente entre 10 y 100 mseg. Este largo retraso obliga a que estos sistemas **débilmente acoplados** se utilicen de maneras distintas a los sistemas **fuertemente acoplados** de la figura 8-1(b). Los tres tipos de sistemas difieren en sus retrasos por una cantidad aproximada a los tres órdenes de magnitud. Ésa es la diferencia entre un día y tres años.

Este capítulo consta de cuatro secciones principales, que corresponden a los tres modelos de la figura 8-1 más una sección sobre la virtualización, que es una manera en software de crear la apariencia de más CPUs. En cada sección empezaremos con una breve introducción al hardware relevante. Después pasaremos al software, en especial las cuestiones relacionadas con el sistema operativo para ese tipo de sistema. Como veremos, en cada caso hay diferentes cuestiones presentes y se requieren distintas metodologías.

8.1 MULTIPROCESADORES

Un **multiprocesador con memoria compartida** (o sólo multiprocesador, de aquí en adelante) es un sistema de cómputo en el que dos o más CPUs comparten todo el acceso a una RAM común. Un programa que se ejecuta en cualquiera de las CPUs ve un espacio normal de direcciones virtuales (por lo general paginadas). La única propiedad inusual que tiene este sistema es que la CPU puede escribir cierto valor en una palabra de memoria y después puede volver a leer esa palabra y obtener un valor distinto (tal vez porque otra CPU lo cambió). Si se organiza en forma correcta, esta propiedad forma la base de la comunicación entre procesadores: una CPU escribe ciertos datos en la memoria y otra lee esos datos.

En su mayor parte, los sistemas operativos multiprocesadores son sólo sistemas operativos regulares. Manejan las llamadas al sistema, administran la memoria, proveen un sistema de archivos y administran los dispositivos de E/S. Sin embargo, hay ciertas áreas en las que tienen características únicas. Estas áreas son la sincronización de procesos, la administración de recursos y la programación de tareas. A continuación analizaremos con brevedad el hardware de los multiprocesadores y después pasaremos a ver las cuestiones relacionadas con estos sistemas operativos.

8.1.1 Hardware de multiprocesador

Aunque todos los multiprocesadores tienen la propiedad de que cada CPU puede direccionar toda la memoria, algunos tienen la característica adicional de que cada palabra de memoria se puede leer con la misma velocidad que cualquier otra palabra de memoria. Estas máquinas se conocen como multiprocesadores **UMA** (*Uniform Memory Access*, Acceso uniforme a la memoria). Por el contrario, los multiprocesadores **NUMA** (*Non-uniform Memory Access*, Acceso no uniforme a la memoria) no tienen esta propiedad. Más adelante aclararemos por qué existe esta diferencia. Primero examinaremos los multiprocesadores UMA y después los NUMA.

Multiprocesadores UMA con arquitecturas basadas en bus

Los multiprocesadores más simples se basan en un solo bus, como se ilustra en la figura 8-2(a). Dos o más CPUs y uno o más módulos de memoria utilizan el mismo bus para comunicarse. Cuando una CPU desea leer una palabra de memoria, primero comprueba si el bus está ocupado. Si está inactivo, la CPU coloca la dirección de la palabra que desea en el bus, declara unas cuantas señales de control y espera hasta que la memoria coloque la palabra deseada en el bus.

Si el bus está ocupado cuando una CPU desea leer o escribir en la memoria, la CPU espera sólo hasta que el bus esté inactivo. Aquí es donde se encuentra el problema con este diseño. Con dos o tres CPUs, la contención por el bus será manejable; con 32 o 64 será imposible. El sistema estará totalmente limitado por el ancho de banda del bus, y la mayoría de las CPUs estarán inactivas la mayor parte del tiempo.

La solución a este problema es agregar una caché a cada CPU, como se ilustra en la figura 8.2(b). La caché puede estar dentro del chip de la CPU, a un lado de ésta, en el tablero del procesador o puede ser alguna combinación de las tres opciones anteriores. Como esto permite satisfacer

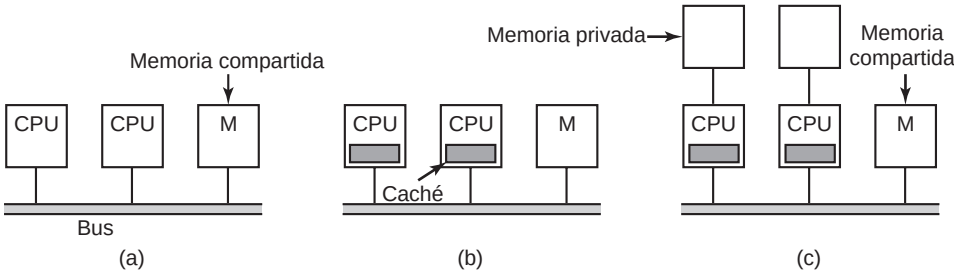


Figura 8-2. Tres multiprocesadores basados en bus. (a) Sin caché. (b) Con caché. (c) Con caché y memorias privadas.

muchas operaciones de lectura desde la caché local, habrá mucho menos tráfico en el bus y el sistema podrá soportar más CPUs. En general, el uso de caché no se realiza por cada palabra individual, sino por bloques de 32 o 64 bytes. Cuando se hace referencia a una palabra, se obtiene su bloque completo (conocido como **línea de caché**) y se coloca en la caché de la CPU que está en contacto con ella.

Cada bloque de la caché se marca como de sólo lectura (en cuyo caso puede estar presente en varias cachés al mismo tiempo) o como de lectura-escritura (no puede estar presente en otras cachés). Si una CPU trata de escribir una palabra que está en una o más cachés remotas, el hardware del bus detecta la escritura y coloca una señal en el bus para informar a todas las demás cachés sobre la operación de escritura. Si otras cachés tienen una copia "limpia" (es decir, una copia exacta de lo que hay en memoria), sólo tienen que descartar sus copias y dejar que la CPU que va a escribir obtenga el bloque de la caché de la memoria antes de modificarla. Si alguna otra caché tiene una copia "sucia" (es decir, modificada), debe escribirla de vuelta a la memoria para que pueda continuar la escritura, o transferirla directamente a la CPU que va a realizar la operación de escritura a través del bus. A este conjunto de reglas se le conoce como **protocolo de coherencia de cachés** y es uno de muchos.

Otra posibilidad es el diseño de la figura 8-2(c), donde cada CPU tiene no sólo una caché, sino también una memoria privada local que utiliza mediante un bus dedicado (privado). Para utilizar esta configuración de manera óptima, el compilador debe colocar todo el texto del programa, las cadenas, constantes y demás datos de sólo lectura, las pilas y las variables locales en las memorias privadas. De esta forma, la memoria compartida se utiliza sólo para escribir variables compartidas. En la mayoría de los casos, esta cuidadosa colocación reducirá de manera considerable el tráfico en el bus, pero requiere de una cooperación activa por parte del compilador.

Multiprocesadores UMA que utilizan interruptores de barras cruzadas

Aun con la mejor caché, el uso de un solo bus limita el tamaño de un multiprocesador UMA a 16 o 32 CPUs, aproximadamente. Para lograr algo más se necesita un tipo distinto de interconexión. El circuito más simple para conectar *n* CPUs a *k* memorias es el **interruptor (conmutador) de barras cruzadas**, que se muestra en la figura 8-3. Los interruptores de barras cruzadas se han utilizado por décadas en los puntos de intercambio de los conmutadores telefónicos para conectar un grupo de líneas entrantes a un conjunto de líneas salientes de manera arbitraria.

En cada intersección de una línea horizontal (entrante) y una vertical (saliente) hay un **punto de cruce**, el cual es un pequeño interruptor que puede estar abierto o cerrado en sentido eléctrico, dependiendo de si las líneas horizontal y vertical van a conectarse o no. En la figura 8-3(a) podemos ver tres puntos de cruce cerrados al mismo tiempo, con lo cual se permiten tres conexiones entre los pares (CPU, memoria) (010, 000), (101, 101) y (110, 010) al mismo tiempo. También son posibles muchas otras combinaciones. De hecho, el número de combinaciones es igual al número de las distintas formas en que se pueden colocar ocho torres en un tablero de ajedrez.

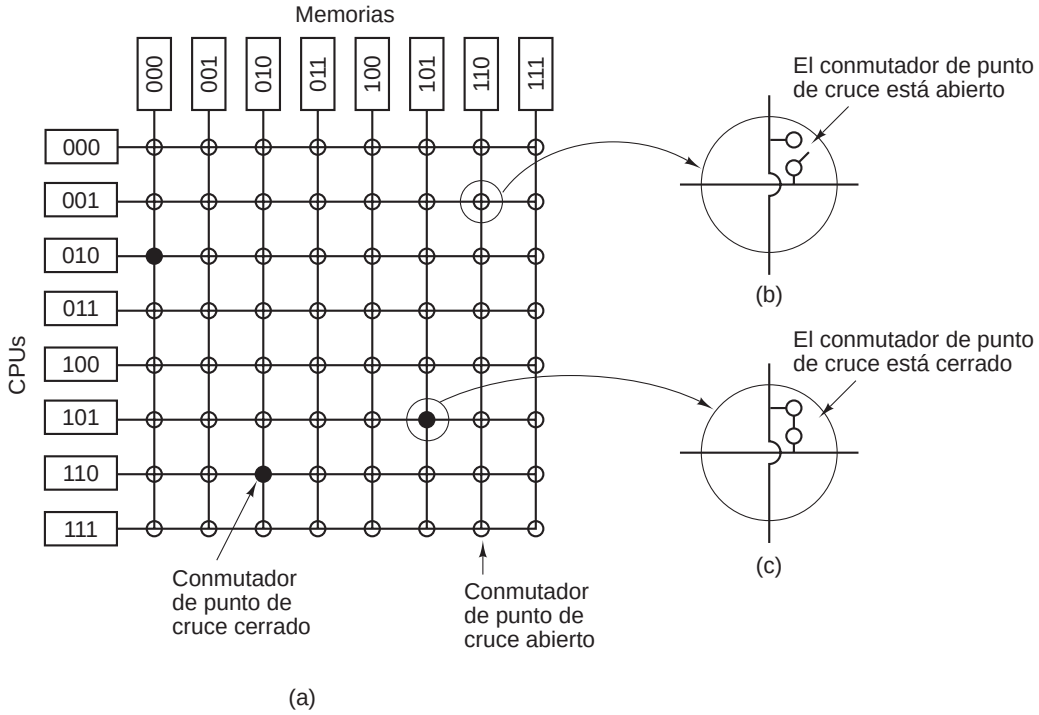


Figura 8-3. (a) Un conmutador de barras cruzadas de 8 × 8. (b) Un punto de cruce abierto. (c) Un punto de cruce cerrado.

Una de las propiedades más agradables de un interruptor de barras cruzadas es que es una **red sin bloqueos**, lo cual significa que a ninguna CPU se le niega nunca la conexión que necesita, debido a que algún punto de cruce o una línea ya estén ocupados (suponiendo que el módulo de memoria en sí esté disponible). Además, no se necesita una planeación por adelantado. Aun si ya hay siete conexiones arbitrarias establecidas, siempre es posible conectar la CPU restante a la memoria restante.

Desde luego que también es posible la contención de memoria, si dos CPUs desean acceder al mismo módulo, al mismo tiempo. Sin embargo, al particionar la memoria en *n* unidades, la contención se reduce por un factor de *n* en comparación con el modelo de la figura 8-2.

Una de las peores propiedades del conmutador de barras cruzadas es el hecho de que el número de puntos de cruce aumenta con base en n^2. Con 1000 CPUs y 1000 módulos de memoria, necesitamos un millón de puntos de cruce. No es factible un interruptor de barras cruzadas de este tamaño. Sin embargo, para los sistemas de tamaño mediano, se puede utilizar el diseño de las barras cruzadas.

Multiprocesadores UMA que utilizan redes de conmutación multietapa

Un diseño completamente distinto de multiprocesador se basa en el humilde conmutador de 2 × 2 que se muestra en la figura 8-4(a). Este interruptor tiene dos entradas y dos salidas. Los mensajes que llegan en cualquiera de las líneas de entrada se pueden conmutar a cualquiera de las líneas de salida. Para nuestros fines, los mensajes deben contener hasta cuatro partes, como se muestra en la figura 8-4(b). El campo *Módulo* indica qué memoria utilizar. *Dirección* especifica una dirección dentro de un módulo. *CódigoOp* proporciona la operación, como `READ` o `WRITE`. Por último, el campo *Valor* opcional puede contener un operando, como una palabra de 32 bits a escribir mediante una operación `WRITE`. El interruptor inspecciona el campo *Módulo* y lo utiliza para determinar si el mensaje se debe enviar en *X* o en *Y*.

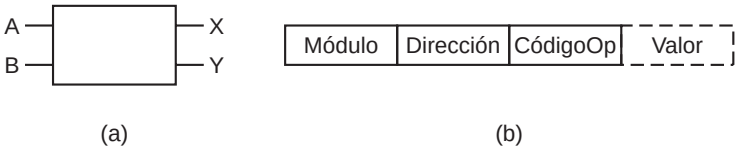


Figura 8-4. (a) Un conmutador de 2 × 2 con dos líneas de entrada (*A* y *B*) y dos líneas de salida (*X* y *Y*). (b) Un formato de mensaje.

Nuestros conmutadores de 2 × 2 se pueden organizar de muchas formas para construir **redes de conmutación multietapa** más grandes (Adams y colaboradores, 1987; Bhuyan y colaboradores, 1989; Kumar y Reddy, 1987). Una posibilidad es la **red omega** de clase económica sin adornos, que se ilustra en la figura 8-5. Aquí hemos conectado ocho CPUs a ocho memorias, utilizando 12 conmutadores. De manera más general, para n CPUs y n memorias necesitaríamos $\log_2 n$ etapas, con $n/2$ conmutadores por etapa, para un total de $(n/2)\log_2 n$ conmutadores, que es mucho mejor que n^2 puntos de cruce, en especial para valores grandes de n.

El patrón de cableado de la red omega se conoce comúnmente como **barajado perfecto**, ya que la mezcla de las señales en cada etapa se asemeja al proceso de cortar a la mitad un mazo de cartas y después mezclarlas, carta por carta. Para ver cómo funciona la red omega, suponga que la CPU 011 desea leer una palabra del módulo de memoria 110. La CPU envía un mensaje `READ` al interruptor 1D que contiene el valor 110 en el campo *Módulo*. El interruptor toma el primer bit (de más a la izquierda) de 110 y lo utiliza para el enrutamiento. Un 0 enruta a la salida superior y un 1 a la inferior. Como este bit es un 1, el mensaje se enruta a través de la salida inferior hacia 2D.

Todos los interruptores de la segunda etapa (incluyendo a 2D) utilizan el segundo bit para el enrutamiento. Éste también es un 1, por lo que ahora el mensaje se reenvía a través de la salida inferior hacia 3D. Aquí se evalúa el tercer bit y se encuentra que es 0. En consecuencia, el mensaje

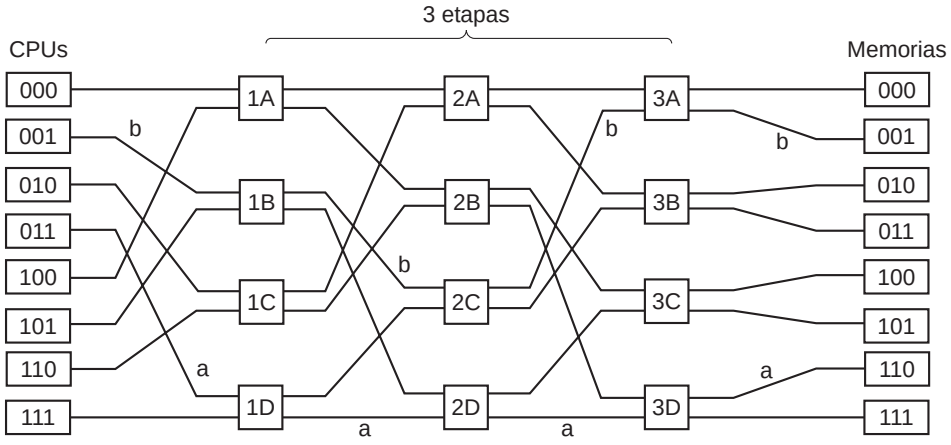


Figura 8-5. Una red de conmutación omega.

pasa por la salida superior y llega a la memoria 110, como se desea. La ruta seguida por este mensaje está marcada en la figura 8-5 por la letra *a*.

A medida que el mensaje pasa a través de la red de conmutación, los bits en el extremo izquierdo del número de módulo ya no se necesitan. Se les puede dar un buen uso al registrar ahí el número de línea entrante, para que la respuesta pueda encontrar su camino de regreso. Para la ruta *a*, las líneas entrantes son 0 (entrada superior a 1D), 1 (entrada inferior a 2D) y 1 (entrada inferior a 3D), respectivamente. La respuesta se enruta de vuelta utilizando 011, sólo que esta vez se lee de derecha a izquierda.

Al mismo tiempo que ocurre todo esto, la CPU 001 desea escribir una palabra en el módulo de memoria 001. Aquí ocurre un proceso análogo, donde el mensaje se enruta a través de las salidas superior, superior e inferior, respectivamente, marcadas por la letra *b*. Cuando llega, en su campo *Módulo* se lee 001, lo cual representa la ruta que tomó. Como estas dos peticiones no utilizan ninguno de los mismos conmutadores, líneas o módulos de memoria, pueden proceder en paralelo.

Ahora considere lo que ocurriría si la CPU 000 quisiera acceder al mismo tiempo al módulo de memoria 000. Su petición entraría en conflicto con la petición de la CPU 001 en el interruptor 3A. Una de ellas tendría que esperar. A diferencia del conmutador de barras cruzadas, la red omega es una **red con bloqueo**. No todos los conjuntos de peticiones pueden procesarse al mismo tiempo. Puede haber conflictos sobre el uso de un cable o de un conmutador, así como entre las peticiones *a* la memoria y las respuestas *de* la memoria.

Sin duda, es conveniente esparcir las referencias a memoria de manera uniforme a través de los módulos. Una técnica común es utilizar los bits de menor orden como el número de módulo. Por ejemplo, considere un espacio de direcciones orientado a bytes para una computadora que por lo general utiliza palabras completas de 32 bits. Los 2 bits de menor orden por lo general serán 00, pero los siguientes 3 bits estarán distribuidos de manera uniforme. Al utilizar estos 3 bits como el número de módulo, las palabras consecutivas estarán en módulos consecutivos. Se dice que un sistema de memoria en el que las palabras consecutivas están en distintos módulos es **entrelazado**. Las memorias entrelazadas maximizan el paralelismo, debido a que la mayoría de las referencias a memo-

ria son a direcciones consecutivas. También es posible diseñar redes de conmutación que sean sin bloqueo y que ofrezcan varias rutas desde cada CPU a cada módulo de memoria, para esparcir mejor el tráfico.

Multiprocesadores NUMA

Los multiprocesadores UMA de un solo bus por lo general están limitados a no más de unas cuantas docenas de CPUs, y los multiprocesadores de barras cruzadas o de conmutadores necesitan mucho hardware (costoso) y no son tan grandes. Para poder tener más de 100 CPUs, hay que ceder algo. Por lo general, lo que cede es la idea de que todos los módulos de memoria tienen el mismo tiempo de acceso. Esta concesión nos lleva a la idea de los multiprocesadores NUMA, como dijimos antes. Al igual que sus parientes UMA, proveen un solo espacio de direcciones a través de todas las CPUs, pero a diferencia de las máquinas UMA, el acceso a los módulos de memoria locales es más rápido que el acceso a los remotos. Por ende, todos los programas UMA se ejecutarán sin cambios en las máquinas NUMA, pero el rendimiento será peor que en una máquina UMA a la misma velocidad de reloj.

Las máquinas NUMA poseen tres características clave que, en conjunto, las distinguen de otros multiprocesadores:

1. Hay un solo espacio de direcciones visible para todas las CPUs.

2. El acceso a la memoria remota es mediante instrucciones `LOAD` y `STORE`.

3. El acceso a la memoria remota es más lento que el acceso a la memoria local.

Cuando el tiempo de acceso a la memoria remota no está oculto (debido a que no hay caché), al sistema se le llama **NC-NUMA** (*No Cache NUMA*, NUMA sin caché). Cuando hay cachés coherentes presentes, al sistema se le llama **CC-NUMA** (*Cache-Coherent NUMA*, NUMA con cachés coherentes).

El método más popular para construir grandes multiprocesadores CC-NUMA en la actualidad es el **multiprocesador basado en directorios**. La idea es mantener una base de datos que indique dónde está cada línea de caché y cuál es su estado. Cuando se hace referencia a una línea de caché, se consulta la base de datos para averiguar dónde está, y si está limpia o sucia (modificada). Como esta base de datos se debe consultar en cada instrucción que hace referencia a la memoria, debe mantenerse en un hardware de propósito especial, en extremo veloz, que pueda responder en una fracción de un ciclo del bus.

Para que la idea sobre un multiprocesador basado en directorios sea algo más concreta, vamos a considerar un ejemplo simple (hipotético): un sistema con 256 nodos donde cada nodo consista en una CPU y 16 MB de RAM conectados a la CPU a través de un bus local. La memoria total es de 2^{32} bytes, que se divide en 2^{26} líneas de caché de 64 bytes cada una. La memoria se asigna en forma estática entre los nodos, con 0 a 16M en el nodo 0, 16M a 32M en el nodo 1, y así en lo sucesivo. Los nodos se conectan mediante una red de interconexión, como se muestra en la figura 8-6(a). Cada nodo también contiene las entradas de directorio para las 2^{18} líneas de caché de 64 bytes que

conforman su memoria de 2^{24} bytes. Por el momento, vamos a suponer que una línea se puede contener a lo más en una caché.

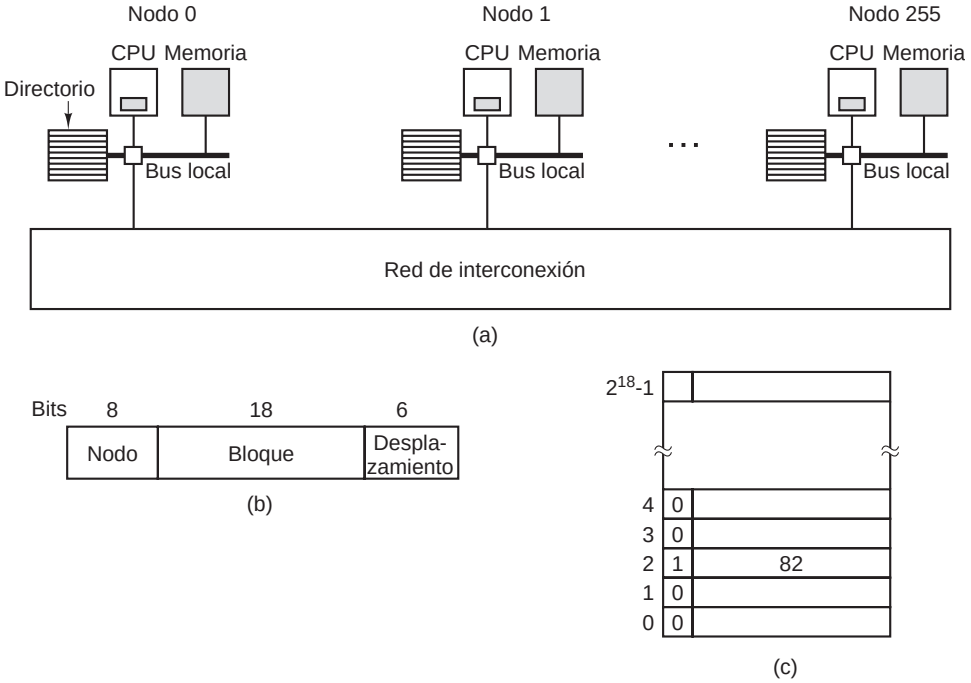


Figura 8.6. (a) Un multiprocesador basado en directorio de 256 nodos. (b) División de una dirección de memoria de 32 bits en campos. (c) El directorio en el nodo 36.

Para ver cómo funciona el directorio, vamos a rastrear una instrucción `LOAD` desde la CPU 20 que hace referencia a una línea en la caché. Primero, la CPU que emite la instrucción la presenta a su MMU, que la traduce en una dirección física; por ejemplo, 0x24000108. La MMU divide esta dirección en las tres partes que se muestran en la figura 8-6(b). En decimal, las tres partes son el nodo 36, la línea 4 y el desplazamiento 8. La MMU ve que la palabra de memoria a la que se hace referencia es del nodo 36, no del nodo 20, por lo que envía un mensaje de petición a través de la red de interconexión al nodo de inicio de la línea (36), en donde le pregunta si su línea 4 está en la caché, y de ser así, en dónde está.

Cuando la petición llega al nodo 36 a través de la red de interconexión, se enruta al hardware de directorio. El hardware la indexa en su tabla de 2^{18} entradas, una para cada una de sus líneas de caché, y extrae la entrada 4. De la figura 8-6(c) podemos ver que la línea no está en la caché, por lo que el hardware obtiene la línea 4 de la RAM local, la envía de vuelta al nodo 20 y actualiza la entrada 4 del directorio para indicar que la línea ahora está en caché en el nodo 20.

Ahora consideremos una segunda petición, esta vez preguntando sobre la línea 2 del nodo 36. En la figura 8-6(c) podemos ver que esta línea está en la caché en el nodo 82. En este punto, el hard-

ware podría actualizar la entrada 2 del directorio para decir que la línea se encuentra ahora en el nodo 20, y después enviar un mensaje al nodo 82 para indicarle que debe pasar la línea al nodo 20 e invalidar su caché. Observe que hasta un "multiprocesador con memoria compartida" realiza muchas operaciones de paso de mensajes de manera interna.

Como una observación rápida adicional, vamos a calcular cuánta memoria está siendo ocupada por los directorios. Cada nodo tiene 16 MB de RAM y 2^{18} entradas de 9 bits para llevar la cuenta de esa RAM. Por ende, la sobrecarga del directorio es de aproximadamente 9×2^{18} bits, dividida entre 16 MB o aproximadamente de 1.76%, que por lo general es aceptable (aunque tiene que ser memoria de alta velocidad, lo que desde luego incrementa su costo). Aun con las líneas de caché de 32 bytes, la sobrecarga sería sólo de 4%. Con líneas de caché de 128 bytes, sería menor de 1%.

Una limitación obvia de este diseño es que una línea se puede colocar en caché sólo en un nodo. Para permitir que se coloquen líneas en caché en varios nodos, necesitaríamos alguna forma de localizarlos a todos, por ejemplo para invalidarlos o actualizarlos en una escritura. Hay varias opciones posibles para permitir el uso de caché en varios nodos al mismo tiempo, pero un análisis de estas opciones es algo que está más allá del alcance de este libro.

Chips multinúcleo

A medida que mejora la tecnología de fabricación de chips, los transistores se están haciendo cada vez más pequeños y es posible colocar cada vez más de ellos en un chip. A esta observación empírica se le conoce algunas veces como **Ley de Moore**, en honor del co-fundador de Intel Gordon Moore, quien la descubrió primero. Los chips en la clase del Intel Core 2 Duo contienen cerca de 300 millones de transistores.

Una pregunta obvia es: "¿Qué se debe hacer con todos esos transistores?". Como vimos en la sección 1.3.1, una opción es agregar megabytes de caché al chip. Esta opción es seria, y los chips con 4 MB de caché fuera del chip ya son comunes, con cachés más grandes en camino. Pero en cierto punto, incrementar el tamaño de la caché tal vez sólo eleve la tasa de aciertos de 99% a 99.5%, lo cual no mejora mucho el rendimiento de las aplicaciones.

La otra opción es colocar dos o más CPUs completas, a las que generalmente se les denomina **núcleos**, en el mismo chip (técnicamente, en la misma **pastilla**). Los chips de doble núcleo y de cuádruple núcleo ya son comunes; se han fabricado chips con 80 núcleos, y los chips con cientos de núcleos ya están en el horizonte.

Aunque las CPUs pueden o no compartir cachés (por ejemplo, vea la figura 1-8), siempre comparten la memoria principal, y ésta es consistente en el sentido de que siempre hay un valor único para cada palabra de memoria. Los circuitos de hardware especiales aseguran que si hay una palabra presente en dos o más cachés, y una de las CPUs modifica la palabra, ésta se remueva en forma automática y atómica de todas las cachés para poder mantener la consistencia. A este proceso se le conoce como ***snooping***.

El resultado de este diseño es que los chips multinúcleo son sólo pequeños multiprocesadores. De hecho, a los chips multinúcleo se les conoce algunas veces como **CMP**s (**Multiprocesadores a nivel de chip**). Desde la perspectiva del software, los CMPs no son en realidad tan distintos de los multiprocesadores basados en bus, o de los multiprocesadores que utilizan redes de conmutación.

Sin embargo, hay algunas diferencias. Para empezar, en un multiprocesador basado en bus cada CPU tiene su propia caché, como en la figura 8-2(b) y también como en el diseño AMD de la figura 1-8(b). El diseño de caché compartida de la figura 1-8(a), que Intel utiliza, no ocurre en otros multiprocesadores. La caché L2 compartida puede afectar el rendimiento. Si un núcleo necesita mucha memoria caché y los otros no, este diseño permite que ese núcleo tome lo que necesite. Por otro lado, la caché compartida también hace posible que un núcleo avaricioso dañe el rendimiento de los demás núcleos.

Otra área en la que los CMPs difieren de sus primos más grandes es la tolerancia a fallas. Como las CPUs tienen una conexión tan estrecha, las fallas en los componentes compartidos pueden afectar a varias CPUs a la vez, algo que es menos probable en los multiprocesadores tradicionales.

Además de los chips multinúcleo simétricos, en donde todos los núcleos son idénticos, hay otra categoría de chip multinúcleo conocida como **sistema en un chip**. Estos chips tienen una o más CPUs principales, pero también tienen núcleos de propósito especial, como decodificadores de video y audio, criptoprocesadores, interfaces de red y demás, con lo cual se obtiene un sistema de cómputo completo en un chip.

Como ha ocurrido con frecuencia en el pasado, el hardware está muy adelantado al software. Aunque los chips multinúcleo ya están aquí, nuestra habilidad para escribir aplicaciones para ellos todavía no. Los lenguajes de programación actuales no están preparados para escribir programas altamente paralelos, y los buenos compiladores y herramientas de depuración son escasos. Pocos programadores han tenido experiencia con la programación en paralelo y la mayoría saben poco acerca de cómo dividir el trabajo en varios paquetes que se puedan ejecutar en paralelo. La sincronización, la eliminación de las condiciones de carrera y la prevención del interbloqueo van a ser pesadillas, y el rendimiento sufrirá de manera considerable como resultado. Los semáforos no son la respuesta. Y más allá de estos problemas iniciales, está muy lejos de ser obvio qué tipo de aplicación realmente necesita cientos de núcleos. El reconocimiento de voz en lenguaje natural probablemente podría absorber mucho poder de cómputo, pero el problema aquí no es la falta de ciclos, sino de algoritmos que funcionen. En resumen, los diseñadores de hardware pueden estar ofreciendo un producto que los diseñadores de software no saben cómo usar, y que los usuarios no quieren.

8.1.2 Tipos de sistemas operativos multiprocesador

Ahora vamos a pasar del hardware de multiprocesador al software de multiprocesador, y en especial a los sistemas operativos multiprocesador. Hay varias metodologías posibles. A continuación estudiaremos tres de ellas. Observe que todas se pueden aplicar de igual forma a los sistemas multinúcleo, así como a los sistemas con CPUs discretas.

Cada CPU tiene su propio sistema operativo

La manera más simple posible de organizar un sistema operativo multiprocesador es dividir estáticamente la memoria y su propia copia privada del sistema operativo. En efecto, las *n* CPUs operan entonces como *n* computadoras independientes. Una optimización obvia es permitir que todas las

CPUs compartan el código del sistema operativo y obtengan copias privadas sólo de las estructuras de datos del sistema operativo, como se muestra en la figura 8-7.

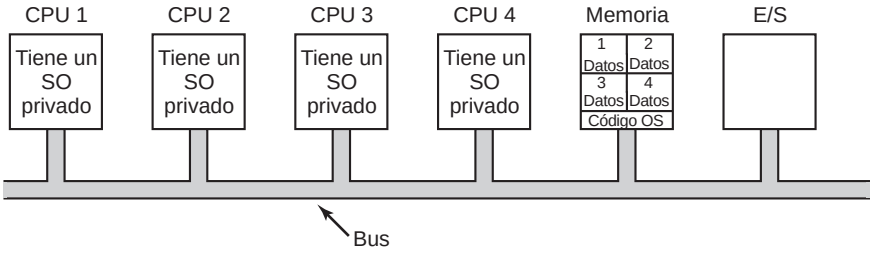


Figura 8-7. Particionamiento de la memoria de un multiprocesador entre cuatro CPUs, pero compartiendo una sola copia del código del sistema operativo. Los cuadros marcados como Datos son los datos privados del sistema operativo para cada CPU.

Este esquema es aún mejor que tener *n* computadoras separadas, ya que permite que todas las máquinas compartan un conjunto de discos y otros dispositivos de E/S, y también permite compartir la memoria con flexibilidad. Por ejemplo, aun con la asignación estática de memoria, una CPU puede recibir una porción extra-grande de la memoria para poder manejar con eficiencia los programas grandes. Además, los procesos pueden comunicarse de manera eficiente unos con otros, al permitir que un productor escriba los datos directamente en la memoria y que un consumidor los obtenga del lugar en el que los escribió el productor. Aún así, desde la perspectiva de un sistema operativo, el que cada CPU tenga su propio sistema operativo es lo más primitivo posible.

Vale la pena mencionar cuatro aspectos de este diseño que tal vez no sean obvios. En primer lugar, cuando un proceso realiza una llamada al sistema, ésta se atrapa y maneja en su propia CPU, usando las estructuras de datos en las tablas de ese sistema operativo.

En segundo lugar, como cada sistema operativo tiene sus propias tablas, también tiene su propio conjunto de procesos que programa por su cuenta. No hay compartición de procesos. Si un usuario se conecta a la CPU 1, todos sus procesos se ejecutan en la CPU 1. Como consecuencia, puede ocurrir que la CPU 1 esté inactiva mientras la CPU 2 está cargada de trabajo.

En tercer lugar, no hay compartición de páginas. Puede ocurrir que la CPU 1 tenga páginas de sobra, mientras que la CPU 2 esté paginando en forma continua. No hay forma de que la CPU 2 pida prestadas algunas páginas a la CPU 1, ya que la asignación de memoria es fija.

En cuarto lugar (y lo que es peor), si el sistema operativo mantiene una caché en búfer de los bloques de disco de uso reciente, cada sistema operativo hace esto de manera independiente a los demás. Por ende, puede ocurrir que haya cierto bloque de disco presente y sucio en varias cachés de búfer al mismo tiempo, con lo cual se producirán resultados inconsistentes. La única forma de evitar este problema es eliminar las cachés del búfer. Eso no es tan difícil, pero daña el rendimiento en forma considerable.

Por estas razones, es muy raro que se utilice este modelo, aunque en los primeros días de los multiprocesadores sí se utilizaba, cuando el objetivo era portar los sistemas operativos existentes hacia algún multiprocesador nuevo, lo más rápido posible.

Multiprocesadores maestro-esclavo

En la figura 8-8 se muestra un segundo modelo. Aquí, hay una copia del sistema operativo y sus tablas presente en la CPU 1, y nada más ahí. Todas las llamadas al sistema se redirigen a la CPU 1 para procesarlas ahí. La CPU 1 también puede ejecutar proceso de usuario si tiene tiempo de sobra. A este modelo se le conoce como **maestro-esclavo**, ya que la CPU 1 es el maestro y todas los demás son los esclavos.

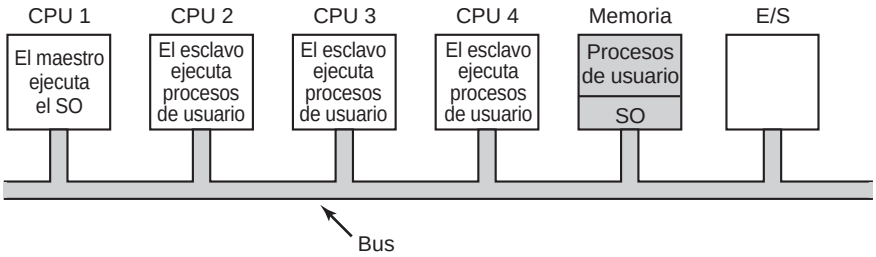


Figura 8.8. Un modelo de multiprocesador maestro-esclavo.

El modelo maestro-esclavo resuelve la mayoría de los problemas del primer modelo. Hay una sola estructura de datos (por ejemplo, una lista o un conjunto de listas con prioridades) que lleva la cuenta de los procesos listos. Cuando una CPU está inactiva, pide al sistema operativo en la CPU 1 un proceso para ejecutarlo, y se le asigna uno. Por ende, nunca puede ocurrir que una CPU esté inactiva mientras que otra esté sobrecargada. De manera similar, se pueden asignar las páginas entre todos los procesos en forma dinámica y sólo hay una caché de búfer, por lo que nunca ocurren inconsistencias.

El problema con este modelo es que con muchas CPUs, el maestro se convertirá en un cuello de botella. Después de todo, debe manejar todas las llamadas al sistema de todas las CPUs. Si, por ejemplo, el 10% del tiempo está manejando llamadas al sistema, entonces con 10 CPUs el maestro casi se saturará y con 20 CPUs estará completamente sobrecargado. Por ende, este modelo es simple y funciona para pequeños multiprocesadores, pero no para los grandes.

Multiprocesadores simétricos

Nuestro tercer modelo, el **SMP** (**Multiprocesador simétrico**), elimina esta asimetría. Hay una copia del sistema operativo en memoria, pero cualquier CPU puede ejecutarlo. Cuando se hace una llamada al sistema, la CPU en la que se hizo la llamada al sistema atrapa para el kernel y procesa la llamada al sistema. El modelo SMP se ilustra en la figura 8-9.

Este modelo equilibra los procesos y la memoria en forma dinámica, ya que sólo hay un conjunto de tablas del sistema operativo. También elimina el cuello de botella de la CPU, ya que no hay maestro, pero introduce sus propios problemas. En especial, si dos o más CPUs están ejecutando código del sistema operativo al mismo tiempo, se puede producir un desastre. Imagine que dos CPUs seleccionan el mismo proceso para ejecutarlo, o que reclaman la misma página de memoria

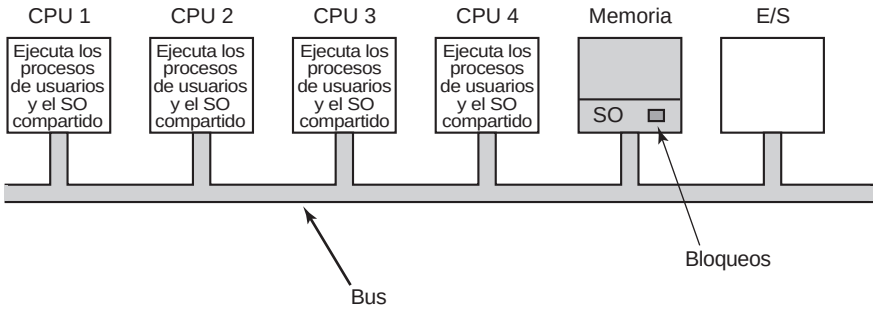


Figura 8-9. El modelo de multiprocesador SMP.

libre. La solución más simple para estos problemas es asociar un mutex (es decir, bloqueo) con el sistema operativo, convirtiendo a todo el sistema en una gran región crítica. Cuando una CPU desea ejecutar código del sistema operativo, debe adquirir primero el mutex. Si el mutex está bloqueado, sólo espera. De esta forma, cualquier CPU puede ejecutar el sistema operativo, pero sólo una a la vez.

Este modelo funciona, pero casi tan mal como el modelo maestro-esclavo. De nuevo, suponga que 10% de todo el tiempo de ejecución se invierte dentro del sistema operativo. Con 20 CPUs, habrá largas colas de CPUs esperando entrar. Por fortuna, es fácil de mejorar. Muchas partes del sistema operativo son independientes unas de otras. Por ejemplo, no habría problema si una CPU ejecutara el planificador mientras que otra CPU manejara una llamada al sistema de archivos, y una tercera CPU estuviera procesando un fallo de página.

Esta observación ocasiona que se divida el sistema operativo en varias regiones críticas independientes que no interactúan entre sí. Cada región crítica está protegida por su propio mutex, por lo que sólo una CPU a la vez puede ejecutarlo. De esta forma se puede lograr mucho más paralelismo. Sin embargo, puede ocurrir que algunas tablas (como las de procesos) sean utilizadas por varias regiones críticas. Por ejemplo, la tabla de procesos es necesaria para la programación, pero también para la llamada al sistema `fork` y para el manejo de señales. Cada tabla que pueda ser utilizada por varias regiones críticas necesita su propio mutex. De esta forma, cada región crítica se puede ejecutar sólo por una CPU a la vez, y cada tabla crítica se puede utilizar sólo por una CPU a la vez.

La mayoría de los multiprocesadores modernos utilizan esta organización. La parte difícil sobre escribir el sistema operativo para una máquina de este tipo no es que el código actual sea tan distinto de un sistema operativo regular. No es así. La parte difícil es dividirlo en regiones críticas que se puedan ejecutar de manera concurrente por distintas CPUs sin interferir unas con otras, ni siquiera en formas sutiles e indirectas. Además, cada tabla utilizada por dos o más regiones críticas debe estar protegida por separado por un mutex, y todo el código que utilice la tabla debe utilizar el mutex en forma correcta.

Además, hay que tener mucho cuidado para evitar los interbloqueos. Si dos regiones críticas necesitan la tabla *A* y la tabla *B*, y una de ellas reclama la tabla *A* primero y la otra reclama la tabla *B* primero, tarde o temprano ocurrirá un interbloqueo y nadie sabrá por qué. En teoría, a todas las

tablas se les podrían asignar valores enteros y todas las regiones críticas podrían requerir la adquisición de tablas en orden ascendente. Esta estrategia evita los interbloqueos, pero requiere que el programador piense con mucho cuidado sobre cuáles tablas necesita cada región crítica, y realice las peticiones en el orden correcto.

A medida que el código evoluciona con el tiempo, una región crítica puede llegar a necesitar una nueva tabla que no necesitaba antes. Si el programador es nuevo y no comprende la lógica completa del sistema, entonces la tentación será sólo colocar el mutex en la tabla, en el punto en el que se necesite y liberarlo cuando ya no sea necesario. Sin importar qué tan razonable pueda parecer esto, se pueden producir interbloqueos, que el usuario percibirá como si el sistema se paralizara. No es fácil hacerlo bien, y mantenerlo así durante un periodo de años frente a los cambios de programadores es muy difícil.

8.1.3 Sincronización de multiprocesadores

Con frecuencia, las CPUs en un multiprocesador necesitan sincronizarse. Acabamos de ver el caso en el que las regiones críticas del kernel y las tablas se tienen que proteger mediante mutexes. Ahora vamos a analizar con más detalle la forma en que funciona esta sincronización en un multiprocesador. No carece de importancia, como pronto veremos.

Para empezar, realmente se necesitan primitivas de sincronización apropiadas. Si un proceso en una máquina uniprocesador (sólo una CPU) realiza una llamada al sistema que requiera acceder a cierta tabla crítica del kernel, el código del kernel sólo tiene que deshabilitar las interrupciones antes de tocar la tabla. Después puede hacer su trabajo, sabiendo que podrá terminar sin que ningún otro proceso se entrometa y toque la tabla antes de que haya terminado. En un sistema multiprocesador, deshabilitar las interrupciones sólo afecta a esa CPU, desshilitándola. Las demás CPUs se siguen ejecutando y aún pueden tocar la tabla crítica. Como consecuencia, se debe utilizar un protocolo de mutex apropiado, el cual debe ser respetado por todas las CPUs para garantizar que funcione la exclusión mutua.

El corazón de cualquier protocolo de mutex práctico es una instrucción especial que permite inspeccionar una palabra de memoria y establecerla en una operación indivisible. En la figura 2-22 vimos cómo se utilizaba `TSL` (Probar y establecer bloqueo) para implementar regiones críticas. Como vimos antes, lo que hace esta instrucción es leer una palabra de memoria y almacenarla en un registro. Al mismo tiempo, escribe un 1 (o cualquier otro valor distinto de cero) en la palabra de memoria. Desde luego que se requieren dos ciclos de bus para realizar las operaciones de lectura y escritura de memoria. En un uniprocesador, mientras que la instrucción no se pueda descomponer a mitad de ejecución, `TSL` siempre funcionará como se espera.

Ahora piense en lo que podría ocurrir en un multiprocesador. En la figura 8-10 podemos ver la sincronización en el peor caso, en donde la palabra de memoria 1000, que se utiliza como bloqueo, en un principio es 0. En el paso 1, la CPU 1 lee la palabra y obtiene un 0. En el paso 2, antes de que la CPU 1 tenga la oportunidad de volver a escribir la palabra para que sea 1, la CPU 2 entra y también lee la palabra como un 0. En el paso 3, la CPU 1 escribe un 1 en la palabra. En el paso 4, la CPU 2 también escribe un 1 en la palabra. Ambas CPUs obtuvieron un 0 de la instrucción `TSL`, por lo que ambas tienen ahora acceso a la región crítica y falla la exclusión mutua.

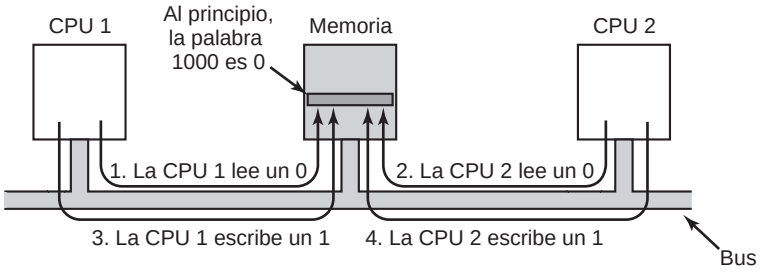


Figura 8-10. La instrucción TSL puede fallar si el bus no se puede bloquear. Estos cuatro pasos muestran una secuencia de eventos en donde se demuestra la falla.

Para evitar este problema, la instrucción TSL debe primero bloquear el bus, para evitar que otras CPUs lo utilicen; después debe realizar ambos accesos a la memoria, y luego desbloquear el bus. Por lo general, para bloquear el bus se hace una petición del bus usando el protocolo de petición de bus común, y después se declara (es decir, se establece a un 1 lógico) cierta línea de bus especial hasta que se hayan completado *ambos* ciclos. Mientras que esté declarada esta línea especial, ninguna otra CPU obtendrá acceso al bus. Esta instrucción puede implementarse sólo en un bus que tenga las líneas necesarias y el protocolo (de hardware) para utilizarlas. Los buses modernos tienen estas facilidades, pero en los primeros que no las tenían, no era posible implementar la instrucción TSL en forma correcta. Ésta es la razón por la que se inventó el protocolo de Peterson: para sincronizar por completo en el software (Peterson, 1981).

Si la instrucción TSL se implementa y utiliza en forma correcta, garantiza que la exclusión mutua pueda llegar a funcionar. Sin embargo, este método de exclusión mutua utiliza un **bloqueo de giro** debido a que la CPU que hace la petición sólo espera en un ciclo estrecho, evaluando el bloqueo lo más rápido que pueda. No sólo se desperdicia por completo el tiempo de la CPU (o CPUs) que hace la petición, sino que también se puede imponer una carga masiva en el bus o la memoria, reduciendo considerablemente a todas las demás CPUs que tratan de realizar su trabajo normal.

En primera instancia, podría parecer que la presencia de la caché debería eliminar el problema de la contención del bus, pero no es así. En teoría, una vez que la CPU que hace la petición ha leído la palabra de bloqueo, debe obtener una copia en su caché. Mientras que ninguna otra CPU intente utilizar el bloqueo, la CPU que hace la petición debe ser capaz de salir de su caché. Cuando la CPU que posee el bloqueo escribe un 1 en la caché para liberarla, el protocolo de la caché invalida de manera automática todas las copias de ésta en las cachés remotas, con lo que se debe volver a obtener el valor correcto.

El problema es que las cachés operan en bloques de 32 o 64 bytes. Por lo general, la CPU que contiene el bloqueo necesita las palabras que lo rodean. Como la instrucción TSL es una escritura (ya que modifica el bloqueo), necesita acceso exclusivo al bloque de la caché que contiene el bloqueo. Por lo tanto, cada instrucción TSL invalida el bloqueo en la caché de la CPU que contiene el bloqueo y obtiene una copia privada y exclusiva para la CPU que hace la petición. Tan pronto como la CPU que contiene el bloqueo toca una palabra adyacente al mismo, el bloque de la caché se

mueve a su máquina. En consecuencia, todo el bloque de la caché que contiene el bloqueo se transporta constantemente entre el propietario del bloqueo y el que solicita el bloqueo, generando aún más tráfico en el bus que las lecturas individuales en la palabra de bloqueo.

Si pudiéramos deshacernos de todas las escrituras inducidas por TSL en el lado que hace la petición, podríamos reducir el *thrashing* de caché en forma considerable. Este objetivo se puede lograr al hacer que la CPU que hace la petición realice primero una lectura pura, para ver si el bloqueo está libre. Sólo si el bloqueo parece estar libre es cuando se realiza una TSL para poder adquirirlo. El resultado de este pequeño cambio es que la mayoría de los sondeos son ahora lecturas en vez de escrituras. Si la CPU que contiene el bloqueo sólo lee las variables en el mismo bloque de caché, cada una de ellas puede tener una copia del bloque de la caché en modo de sólo lectura compartido, eliminando todas las transferencias del bloque de caché. Cuando por fin se libera el bloqueo, el propietario realiza una escritura, la cual requiere acceso exclusivo, con lo cual se invalidan todas las otras copias en las cachés remotas. En la siguiente lectura que realice la CPU que hace la petición, el bloque de la caché se volverá a cargar. Observe que si hay dos o más CPUs contendiendo por el mismo bloqueo, puede ocurrir que ambas vean que está libre al mismo tiempo, y ambas puedan llegar a realizar una operación TSL simultánea para adquirirlo. Sólo una de estas operaciones tendrá éxito, por lo que no hay condición de competencia aquí debido a que la verdadera adquisición la realiza la instrucción TSL, y esta instrucción es atómica. Detectar que el bloqueo está libre y tratar de obtenerlo de inmediato con una operación TSL no garantiza la obtención del mismo. Alguien más podría ganar, pero para que el algoritmo sea correcto, no importa quién lo obtenga. El éxito en la lectura pura es tan sólo una sugerencia de que sería un buen momento de adquirir el bloqueo, pero no constituye una garantía de que la adquisición tendrá éxito.

Otra forma de reducir el tráfico en el bus es mediante el uso del popular algoritmo de *backoff* exponencial binario de Ethernet (Anderson, 1990). En vez de sondear en forma continua como en la figura 2-22, se puede insertar un ciclo de retraso entre los sondeos. Al principio, el retraso es una instrucción. Si el bloqueo sigue ocupado, el retraso se duplica a dos instrucciones, después a cuatro instrucciones, y así hasta llegar a cierto valor máximo. Un máximo bajo ofrece una respuesta rápida cuando se libera el bloqueo, pero desperdicia más ciclos de bus en el *thrashing* de la caché. Un máximo alto reduce el *thrashing* de la caché a expensas de no observar que el bloque está libre con tanta rapidez. El algoritmo de *backoff* exponencial binario se puede utilizar con o sin las lecturas puras que van antes de la instrucción TSL.

Una idea mejor incluso es proporcionar a cada CPU que desea adquirir el mutex su propia variable de bloqueo privada para evaluarla, como se ilustra en la figura 8-11 (Mellor-Crummey y Scott, 1991). La variable debe residir en un bloque de caché no utilizado, para evitar conflictos. El algoritmo funciona al hacer que una CPU que no puede adquirir el bloqueo asigne una variable de bloqueo y se adjunte a sí misma al final de una lista de CPUs que esperan el bloqueo. Cuando la CPU que contiene el bloque sale de la región crítica, libera el bloqueo privado que la primera CPU en la lista está evaluando (en su propia caché). Después, esta CPU entra a la región crítica. Cuando termina, libera el bloqueo que está usando su sucesora, y así en lo sucesivo. Aunque el protocolo es algo complicado (para evitar que dos CPUs se adjunten a sí mismas al final de la lista al mismo tiempo), es eficiente y libre de inanición. Para obtener todos los detalles, los lectores deberán consultar el artículo.

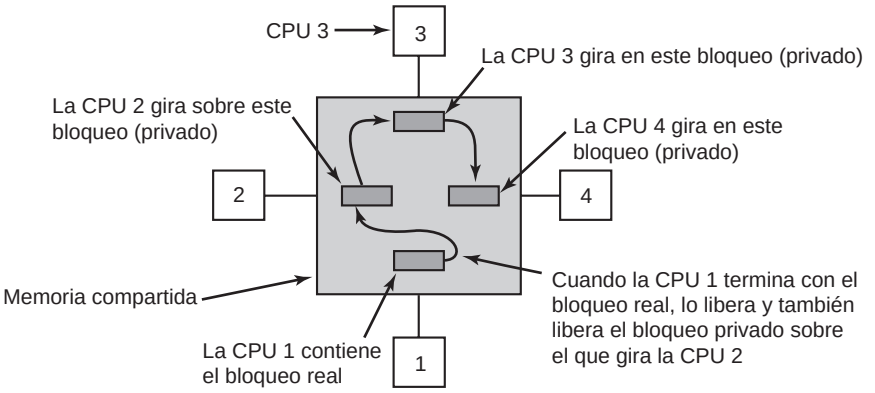


Figura 8-11. Uso de varios bloqueos para evitar el *thrashing* de la caché.

Comparación entre espera activa y conmutación

Hasta ahora hemos asumido que una CPU que necesita un mutex bloqueado sólo lo espera, ya sea mediante un sondeo continuo, un sondeo intermitente o adjuntándose a sí misma a una lista de CPUs en espera. Algunas veces no hay alternativa para que la CPU que hace la petición sólo espere. Por ejemplo, suponga que cierta CPU está inactiva y necesita acceder a la lista compartida de procesos listos para elegir un proceso y ejecutarlo. Si la lista de procesos listos está bloqueada, la CPU no puede sólo decidir suspender lo que está haciendo y ejecutar otro proceso, ya que para ello tendría que leer la lista de procesos listos. *Debe* esperar hasta que pueda adquirir esa lista.

Sin embargo, en otros casos hay una opción. Por ejemplo, si algún hilo en una CPU necesita acceso a la caché del búfer del sistema de archivos y actualmente está bloqueada, la CPU puede decidir cambiar a un hilo distinto en vez de esperar. La cuestión de si debe hacer una espera activa o realizar un cambio de hilo ha sido tema de mucha investigación, parte de la cual analizaremos a continuación. Observe que esta cuestión no ocurre en un uniprocesador, ya que no tiene mucho sentido una espera activa cuando no hay otra CPU para liberar el bloqueo. Si un subproceso trata de adquirir un bloqueo y falla, siempre se bloquea para dar al propietario del bloqueo la oportunidad de ejecutarse y liberar el bloqueo.

Suponiendo que la espera activa y realizar un cambio de hilo son opciones viables, hay que hacer la siguiente concesión. La espera activa desperdicia ciclos de la CPU de manera directa; evaluar un bloqueo en forma repetida no es un trabajo productivo. Sin embargo, cambiar de hilo también desperdicia ciclos de la CPU, ya que se debe guardar el estado del hilo actual, se debe adquirir el bloqueo sobre la lista de procesos listos, hay que seleccionar un hilo, se debe cargar su estado y debe iniciarse. Además, la caché de la CPU contendrá todos los bloques incorrectos, por lo que ocurrirán muchos fallos de caché a medida que el nuevo hilo se empiece a ejecutar. También es probable que ocurran fallos de TLB. En un momento dado se realizará un cambio de vuelta al hilo original, y le seguirán más fallos de caché. Además de los fallos de caché, se desperdician los ciclos invertidos en estos dos cambios de contexto.

Por ejemplo, si se sabe que los mutexes se contienen generalmente durante 50 µseg, y se requiere 1 mseg para cambiar del hilo actual, y 1 mseg para volver a cambiar posteriormente, es más eficiente una espera activa sobre el mutex. Por otra parte, si el mutex promedio se contiene durante 10 mseg, vale la pena el esfuerzo de realizar los dos cambios de contexto. El problema es que las regiones críticas pueden variar de manera considerable en cuanto a su duración, así que, ¿cuál método es mejor?

Un diseño es una espera activa todo el tiempo; un segundo diseño es siempre cambiar. Un tercer diseño es tomar una decisión por separado cada vez que nos encontramos con un mutex bloqueado. Al momento en que se tiene que tomar la decisión, no se sabe si es mejor realizar una espera activa o cambiar, pero para cualquier sistema dado, es posible realizar un rastreo de toda la actividad y analizarla después fuera de línea. Así, podemos saber en retrospectiva cuál decisión fue la mejor y cuánto tiempo se desperdició en el mejor caso. Este algoritmo de retrospección se convierte entonces en un punto de referencia contra el que se pueden medir los algoritmos viables.

Los investigadores han estudiado este problema (Karlin y colaboradores, 1989; Karlin y colaboradores, 1991; Ousterhout, 1982). La mayor parte del trabajo utiliza un modelo en el que un hilo que no puede adquirir un mutex gira durante cierto periodo. Si se excede este umbral, hace el cambio. En algunos casos se corrige el umbral, que por lo general es la sobrecarga conocida por cambiar a otro hilo y después cambiar de vuelta. En otros casos es dinámico, dependiendo de la historia observada del mutex sobre el que se espera.

Los mejores resultados se obtienen cuando el sistema lleva el rastro de los últimos tiempos de espera activa observados, y asume que éste será similar a los anteriores. Por ejemplo, suponiendo un tiempo de cambio de contexto de 1 mseg otra vez, un hilo realiza una espera activa por un máximo de 2 mseg, pero observe cuánto tiempo giró en realidad. Si no puede adquirir un bloqueo y detecta que en las tres ejecuciones anteriores esperó un promedio de 200 µseg, debe hacer una espera activa por 2 mseg antes de cambiar de hilo. Pero si detecta que la espera activa dura los 2 mseg completos en cada uno de los intentos anteriores, debe cambiar de inmediato y no realizar una espera activa. Encontrará más detalles en (Karlin y colaboradores, 1991).

8.1.4 Planificación de multiprocesadores

Antes de analizar la forma en que se realiza la planificación en los multiprocesadores, es necesario determinar *qué* es lo que se va a planificar. En los días de antaño, cuando todos los procesos tenían un solo hilo, los procesos eran los que se planificaban; no había nada más que se pudiera planificar. Todos los sistemas operativos modernos soportan procesos multihilo, lo cual complica aún más la planificación.

Es importante saber si los hilos son de kernel o de usuario. Si la ejecución de hilos se lleva a cabo mediante una biblioteca en espacio de usuario y el kernel no sabe nada acerca de los hilos, entonces la planificación se lleva a cabo con base en cada proceso, como siempre. Si el kernel ni siquiera sabe que los hilos existen, es muy difícil que pueda planificarlos.

Con los hilos del kernel sucede algo distinto. Aquí el kernel está al tanto de todos los hilos y puede elegir y seleccionar de entre los hilos que pertenecen a un proceso. En estos sistemas, la tendencia es que el kernel seleccione un hilo para ejecutarlo, y el proceso al que pertenece sólo

tiene un pequeño papel (o tal vez ninguno) en el algoritmo de selección de hilos. A continuación hablaremos sobre la planificación de hilos, pero desde luego que, en un sistema con procesos con un solo hilo, o con hilos implementados en espacio de usuario, son los procesos los que se programan.

Proceso contra hilo no es la única cuestión de planificación. En un uniprocesador, la planificación es de una dimensión. La única pregunta que hay que responder (repetidas veces) es: "¿Cuál hilo se debe ejecutar a continuación?". En un multiprocesador, la planificación tiene dos dimensiones: el planificador tiene que decidir qué hilo ejecutar y en cuál CPU lo va a ejecutar. Esta dimensión adicional complica de manera considerable la planificación en los multiprocesadores.

Otro factor que complica las cosas es que, en ciertos sistemas, ninguno de los hilos está relacionado, mientras que en otros se dividen en grupos donde todos pertenecen a la misma aplicación y trabajan en conjunto. Un ejemplo del primer caso es un sistema de tiempo compartido, en el que usuarios independientes inician procesos independientes. Los hilos de los distintos procesos no están relacionados, y cada uno de ellos se puede planificar de manera independiente a los demás.

Un ejemplo del segundo caso ocurre con regularidad en los entornos de desarrollo de programas. A menudo, los sistemas extensos consisten de cierto número de archivos de encabezado que contienen macros, definiciones de tipos y declaraciones de variables que utilizan los archivos de código actuales. Cuando se modifica un archivo de encabezado, se deben volver a compilar todos los archivos de código que lo incluyen. El programa *make* se utiliza comúnmente para administrar el desarrollo. Cuando se invoca *make*, inicia la compilación sólo de aquellos archivos de código que se deben volver a compilar debido a las modificaciones realizadas en el encabezado o los archivos de código. Los archivos de código objeto que siguen siendo válidos no se regeneran.

La versión original de *make* realizaba su trabajo en forma secuencial, pero las versiones más recientes diseñadas para multiprocesadores pueden iniciar todas las compilaciones a la vez. Si se necesitan 10 compilaciones, no tiene caso programar 9 de ellas para que se ejecuten de inmediato y dejar la última hasta mucho después, ya que el usuario no percibirá el trabajo como terminado sino hasta que termine la última compilación. En este caso, tiene sentido considerar los hilos que realizan las compilaciones como si fueran un grupo, y tomar eso en cuenta al programarlos.

Tiempo compartido

Vamos a analizar el caso de la programación de hilos independientes; más adelante consideraremos cómo planificar hilos relacionados. El algoritmo de planificación más simple para lidiar con hilos no relacionados es tener una sola estructura de datos a nivel de sistema para los hilos que estén listos, que tal vez sea sólo una lista, pero es mucho más probable que sea un conjunto de listas para los hilos con distintas prioridades, como se ilustra en la figura 8-12(a). Aquí, las 16 CPUs están ocupadas en un momento dado, y hay un conjunto de 14 hilos por orden de prioridad, esperando ser ejecutados. La primera CPU en terminar su trabajo actual (o en hacer que su hilo se bloquee) es la CPU 4, que después bloquea las colas de programación y selecciona el hilo con mayor prioridad (*A*), como se muestra en la figura 8-12(b). A continuación, la CPU 12 queda inactiva y selecciona el hilo *B*, como se ilustra en la figura 8-12(c). Mientras que los hilos no estén relacionados de nin-

guna manera, es una opción razonable llevar a cabo la planificación de esta forma, y es muy simple de implementar con eficiencia.

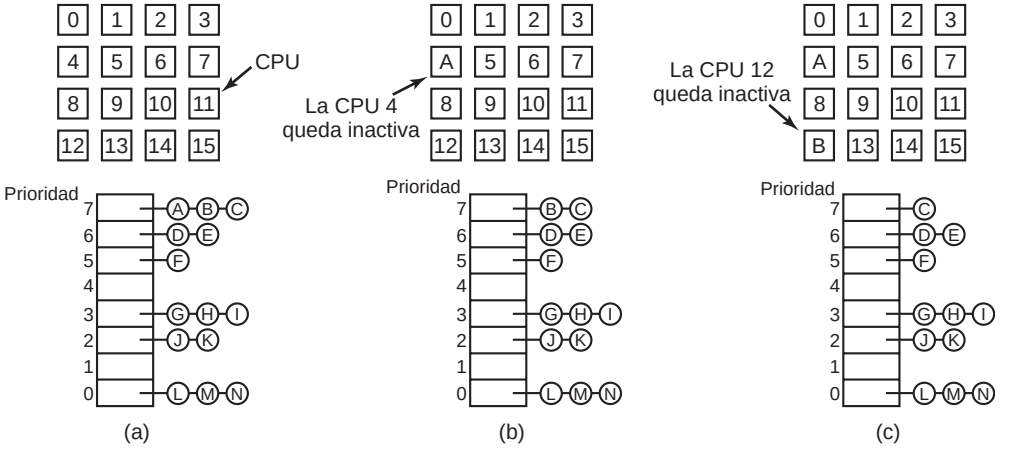


Figura 8-12. Uso de una sola estructura de datos para programar un multiprocesador.

Al tener una sola estructura de datos de planificación utilizada por todas las CPUs, éstas se comparten en el tiempo, de igual forma que como se hace en un sistema uniprocesador. También se proporciona un balance automático de la carga, ya que no puede haber una CPU inactiva mientras otras están sobrecargadas. Dos desventajas de este método son: la potencial contención por la estructura de datos que se utiliza en la programación a medida que aumenta el número de CPUs, y la sobrecarga usual que se produce al realizar un cambio de contexto, cuando un hilo se bloquea en espera de una operación de E/S.

También es posible que ocurra un cambio de contexto al momento en que expire el quantum de un hilo, por ejemplo, en un multiprocesador, que tiene ciertas propiedades que no están presentes en un uniprocesador. Suponga que el hilo contiene un bloqueo de espera activa al momento en que expira su quantum. Las otras CPUs que esperan a que se libere el bloqueo de espera activa sólo desperdician su tiempo en una espera activa hasta que se vuelve a programar ese hilo y se libera el bloqueo. En un uniprocesador, los bloqueos de espera activa se utilizan raras veces, por lo que si un proceso se suspende mientras contiene un mutex, y otro hilo se inicia y trata de adquirir el mutex, se bloqueará de inmediato, por lo que no se desperdicia mucho tiempo.

Para solucionar esta anomalía, algunos sistemas utilizan la **planificación inteligente**, en la cual un hilo que adquiere un bloqueo de espera activa establece una bandera a nivel de proceso, para mostrar que actualmente tiene un bloqueo de espera activa (Zahorjan y colaboradores, 1991). Al liberar el bloqueo, se borra la bandera. Así, el planificador no detiene un hilo que contenga un bloqueo de espera activa, sino que le da un poco más de tiempo para que complete su región crítica y libere el bloqueo.

Otra cuestión que desempeña un papel en la planificación es el hecho de que aunque todas las CPUs son iguales, ciertas CPUs son más iguales que otras. En especial, cuando el subproceso *A* se ha ejecutado durante mucho tiempo en la CPU *k*, la caché de esta CPU estará llena de bloques de *A*. Si *A* se vuelve a ejecutar poco tiempo después, tal vez su rendimiento sea mejor si se ejecuta en

la CPU *k* debido a que la caché de esta CPU tal vez todavía contenga algunos de los bloques de *A*. Cuando se cargan previamente los bloques en la caché se incrementa la proporción de aciertos y, en consecuencia, la velocidad del hilo. Además, se puede dar el caso de que la TLB contenga las páginas correctas, con lo cual se reducen sus fallas.

Algunos multiprocesadores toman en cuenta este efecto y utilizan lo que se conoce como **planificación por afinidad** (Vaswani y Zahorjan, 1991). Aquí, la idea básica es esforzarse en serio por hacer que un hilo se ejecute en la misma CPU en que se ejecutó la última vez. Una manera de crear esta afinidad es mediante el uso de un **algoritmo de planificación de dos niveles**. Cuando se crea un hilo, éste se asigna a una CPU, por ejemplo con base en cuál de las CPUs tiene la menor carga en ese momento. Esta asignación de hilos a CPUs es el nivel superior del algoritmo. Como resultado de esta directiva, cada CPU adquiere su propia colección de hilos.

La planificación actual de los hilos es el nivel inferior del algoritmo. Cada CPU la realiza por separado, mediante el uso de prioridades o de algún otro medio. Al tratar de mantener un hilo en la misma CPU durante todo su tiempo de vida, se maximiza la afinidad de la caché. No obstante, si una CPU no tiene hilos qué ejecutar, obtiene uno de otra CPU en vez de quedar inactiva.

La planificación de dos niveles tiene tres beneficios. En primer lugar, distribuye la carga de una manera casi uniforme sobre las CPUs disponibles. En segundo lugar, se aprovecha la afinidad de la caché siempre que sea posible. En tercer lugar, al dar a cada CPU su propia lista de hilos listos se minimiza la contención por estas listas, ya que es muy poco frecuente que una CPU intente utilizar la lista de hilos listos de otra CPU.

Espacio compartido

El otro método general para la planificación de multiprocesadores se puede utilizar cuando los hilos están relacionados entre sí de alguna manera. Anteriormente mencionamos el ejemplo del programa *make* paralelo como uno de los casos. También ocurre con frecuencia que un solo proceso tenga varios hilos que trabajen en conjunto. Por ejemplo, si los hilos de un proceso se comunican con mucha frecuencia, es conveniente hacer que se ejecuten al mismo tiempo. A la planificación de varios hilos al mismo tiempo, esparcidos sobre varias CPUs, se le conoce como **espacio compartido**.

El algoritmo de espacio compartido más simple funciona de la siguiente manera. Suponga que se crea un grupo completo de hilos relacionados al mismo tiempo. Al momento de su creación, el programador comprueba si hay tantas CPUs libres como el número de hilos. Si las hay, a cada hilo se le proporciona su propia CPU dedicada (es decir, sin multiprogramación) y todos empiezan su ejecución. Si no hay suficientes CPUs, ninguno de los hilos se iniciará, sino hasta que haya suficientes CPUs disponibles. Cada hilo se aferra a su CPU hasta que termina, y en ese momento la CPU se regresa a la reserva de CPUs disponibles. Si un hilo se bloquea en espera de una operación de E/S, de todas formas retiene a la CPU, que simplemente se queda inactiva hasta que el hilo se despierta. Cuando aparece el siguiente lote de hilos, se aplica el mismo algoritmo.

En cualquier instante, el conjunto de CPUs se divide de manera estática en cierto número de particiones, cada una de las cuales ejecuta los hilos de un hilo. Por ejemplo, en la figura 8-13 hay particiones con 4, 6, 8 y 12 CPUs, y hay 2 CPUs sin asignar. Con el tiempo cambiará el número y el tamaño de las particiones, a medida que se creen nuevos hilos y cuando los anteriores terminen de ejecutarse.

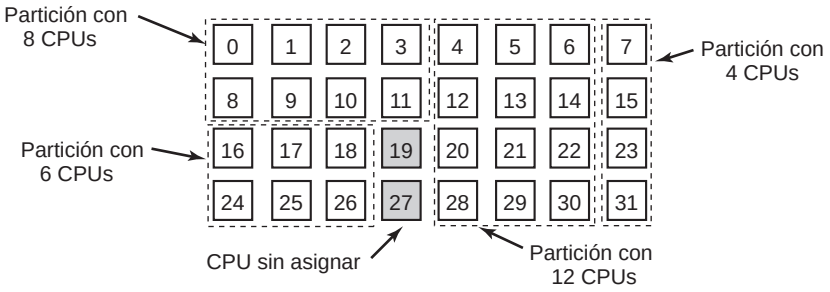


Figura 8-13. Un conjunto de 32 CPUs divididas en cuatro particiones, con dos CPUs disponibles.

De vez en cuando hay que tomar decisiones sobre planificación. En los sistemas con uniprocesador, el algoritmo "el trabajo más corto primero" es un algoritmo popular para la programación por lotes. El algoritmo análogo para un multiprocesador es seleccionar el proceso que necesita el menor número de ciclos de la CPU; es decir, el hilo cuyo valor de la multiplicación "cuenta de la CPU $\times$ tiempo de ejecución" sea el menor de todos. No obstante, en la práctica esta información raras veces está disponible, por lo que es difícil llevar a cabo el algoritmo. De hecho, los estudios han demostrado que, en la práctica, es difícil vencer al algoritmo "primero en llegar, primero en ser atendido" (Krueger y colaboradores, 1994).

En este modelo simple de particionamiento, un hilo sólo pide cierto número de CPUs y los obtiene todos, o tiene que esperar hasta que estén disponibles. Una metodología distinta es que los hilos administren de manera activa el grado de paralelismo. Un método para administrar el paralelismo es tener un servidor central que lleve la cuenta de cuáles hilos se están ejecutando, cuáles desean ejecutarse y cuáles son sus requerimientos de CPU mínimos y máximos (Tucker y Gupta, 1989). De vez en cuando, cada aplicación sondea al servidor central para saber cuántas CPUs puede utilizar. Después aumenta o reduce el número de hilos para ajustarse a lo que haya disponible. Por ejemplo, un servidor Web puede tener 5, 10, 20 o cualquier otro número de hilos ejecutándose en paralelo. Si en un momento dado tiene 10 hilos, aumenta la demanda de CPUs de manera repentina y el servidor Web tiene que reducir su número de hilos a 5, cuando los siguientes 5 hilos terminen su trabajo actual, tendrán que dejar de ejecutarse en vez de recibir más trabajo. Este esquema permite que los tamaños de las particiones varíen en forma dinámica para adaptarse a la carga de trabajo actual de una mejor forma que el sistema fijo de la figura 8-13.

Planificación por pandilla

Una ventaja clara de la compartición de espacio es la eliminación de la multiprogramación, que a su vez elimina la sobrecarga impuesta por el cambio de contexto. Sin embargo, una desventaja también clara es el tiempo que se desperdicia cuando una CPU se bloquea y no tiene nada que hacer hasta que vuelva a estar lista. En consecuencia, las personas han buscado algoritmos que intenten realizar la planificación tanto en tiempo como en espacio, en especial para los hilos que crean varios hilos, que por lo general se necesitan comunicar entre sí.

Para ver el tipo de problema que puede ocurrir cuando los hilos de un proceso se planifican de manera independiente, considere un sistema con los hilos A_0 y A_1 que pertenecen al proceso *A*, y los hilos B_0 y B_1 que pertenecen al proceso *B*. Los hilos A_0 y B_0 comparten el tiempo en la CPU 0; los hilos A_1 y B_1 comparten el tiempo en la CPU 1. Los hilos A_0 y A_1 necesitan comunicarse con frecuencia. El patrón de comunicación es que A_0 envía un mensaje a A_1, y después A_1 envía de regreso una respuesta a A_0, seguida de otra secuencia similar. Suponga que A_0 y B_1 se inician primero, como se muestra en la figura 8-14.

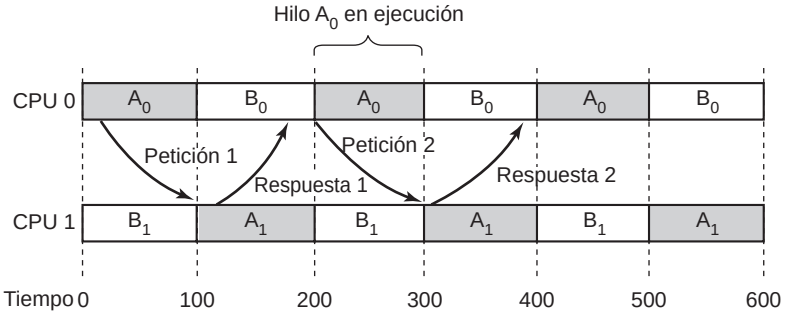


Figura 8-14. Comunicación entre dos hilos que pertenecen al hilo *A*, los cuales se están ejecutando fuera de fase.

En el intervalo 0, A_0 envía una petición a A_1, pero A_1 no la recibe sino hasta que se ejecuta en el intervalo 1, el cual empieza en los 100 mseg. Envía la respuesta de inmediato, pero A_0 no la recibe sino hasta que se vuelve a ejecutar de nuevo, a los 200 mseg. El resultado neto es una secuencia de solicitud-respuesta cada 200 mseg. No es algo muy bueno.

La solución a este problema es la **planificación por pandilla**, que es una consecuencia de la **co-planificación** (Ousterhout, 1982). La planificación por pandilla consta de tres partes:

1. Los grupos de hilos relacionados se programan como una unidad, o pandilla.
2. Todos los miembros de una plantilla se ejecutan en forma simultánea, en distintas CPUs de tiempo compartido.
3. Todos los miembros de la pandilla inician y terminan sus intervalos en conjunto.

El truco que hace funcionar la planificación por pandilla es que todas las CPUs se planifican en forma asíncrona. Esto significa que el tiempo se divide en quantums discretos, como en la figura 8-14. Al inicio de cada nuevo quantum *todas* las CPUs se reprograman, y se inicia un nuevo hilo en cada CPU. Al inicio del siguiente quantum ocurre otro evento de planificación. En el intervalo no se realiza ninguna planificación. Si un hilo se bloquea, su CPU permanece inactiva hasta el final del quantum.

En la figura 8-15 se muestra un ejemplo del funcionamiento de la planificación por pandilla. Aquí tenemos un multiprocesador con seis CPUs que son utilizadas por cinco procesos (*A* a *E*), con un total de 24 hilos listos para ejecutarse. Durante la ranura de tiempo 0, se programan los hilos A_0

a A_6 y se ejecutan. Durante la ranura de tiempo 1, se programan los hilos B_0, B_1, B_2, C_0, C_1 y C_2, y se ejecutan. Durante la ranura de tiempo 2, se ejecutan los cinco hilos de D y E_0. Los seis hilos restantes que pertenecen al hilo E se ejecutan en la ranura de tiempo 3. Después se repite el ciclo, donde la ranura 4 es igual a la ranura 0, y así en lo sucesivo.

Ranura de tiempo	CPU 0	CPU 1	CPU 2	CPU 3	CPU 4	CPU 5
0	A_0	A_1	A_2	A_3	A_4	A_5
1	B_0	B_1	B_2	C_0	C_1	C_2
2	D_0	D_1	D_2	D_3	D_4	E_0
3	E_1	E_2	E_3	E_4	E_5	E_6
4	A_0	A_1	A_2	A_3	A_4	A_5
5	B_0	B_1	B_2	C_0	C_1	C_2
6	D_0	D_1	D_2	D_3	D_4	E_0
7	E_1	E_2	E_3	E_4	E_5	E_6

Figura 8-15. Planificación por pandilla.

La idea de la planificación por pandilla es que todos los hilos de un hilo se ejecuten en conjunto, de manera que si uno de ellos envía una petición a otro, recibirá el mensaje casi de inmediato y podrá responder también casi de inmediato. En la figura 8-15, como todos los hilos de *A* se están ejecutando en conjunto, durante un quantum pueden enviar y recibir una gran cantidad de mensajes, con lo cual se elimina el problema de la figura 8-14.

8.2 MULTICOMPUTADORAS

Los multiprocesadores son populares y atractivos debido a que ofrecen un modelo de comunicación simple: todas las CPUs comparten una memoria común. Los procesos pueden escribir mensajes en la memoria, para que después otros procesos los lean. La sincronización se puede realizar mediante el uso de mutexes, semáforos, monitores y otras técnicas bien establecidas. El único contratiempo es la dificultad de construir multiprocesadores grandes, además de que es un proceso costoso.

Para resolver estos problemas se ha investigado mucho sobre las **multicomputadoras**: CPUs con acoplamiento fuerte que no comparten memoria. Cada una tiene su propia memoria, como se muestra en la figura 8-1(b). Estos sistemas también son conocidos por una variedad de nombres, incluyendo **clúster de computadoras** y **COWS** (*Clusters of Workstations*, Clústeres de estaciones de trabajo).

Es fácil construir las multicomputadoras, ya que el componente básico es una PC que tiene sólo los componentes esenciales, además de una tarjeta de interfaz de red de alto rendimiento. Desde luego que el secreto para obtener un alto rendimiento es diseñar de manera inteligente la red de in-

terconexión y la tarjeta de interfaz. Este problema es completamente análogo al de construir la memoria compartida en un multiprocesador. Sin embargo, el objetivo es enviar mensajes en una escala de tiempo en microsegundos, en vez de acceder a la memoria en una escala de tiempo de nanosegundos, por lo que es más simple, económico y fácil de lograr.

En las siguientes secciones, primero analizaremos con brevedad el hardware de una multicomputadora, en especial el hardware de interconexión. Luego pasaremos al software: empezaremos con el software de comunicación de bajo nivel y continuaremos con el de comunicación de alto nivel. También analizaremos una manera en que se puede obtener memoria compartida en sistemas que no la tienen. Por último examinaremos la planificación y el equilibrio de carga.

8.2.1 Hardware de una multicomputadora

El nodo básico de una multicomputadora consiste en una CPU, la memoria, una interfaz de red y algunas veces un disco duro. El nodo puede estar empaquetado en un gabinete de PC estándar, pero el adaptador de gráficos, el monitor, el teclado y el ratón casi nunca están presentes. En algunos casos la PC contiene un tablero de multiprocesador de 2 o 4 vías, en donde cada nodo posiblemente tiene un chip de doble o cuádruple núcleo en vez de una sola CPU, pero por cuestión de simplicidad vamos a suponer que cada nodo tiene una CPU. A menudo se conectan cientos, o incluso miles de nodos entre sí, para formar una multicomputadora. A continuación hablaremos un poco acerca de la organización de este hardware.

Tecnología de interconexión

Cada nodo tiene una tarjeta de interfaz de red, de la cual salen uno o dos cables (o fibras). Estos cables se conectan a otros nodos o a switches. En un sistema pequeño, puede haber un switch al que están conectados todos los nodos en la topología de estrella de la figura 8-16(a). Las redes Ethernet modernas con switches utilizan esta topología.

Como alternativa al diseño de un solo switch, los nodos pueden formar un anillo con dos cables que provienen de la tarjeta de interfaz de red: uno que entra al nodo por la izquierda y otro que sale del nodo por la derecha, como se muestra en la figura 8-16(b). En esta topología no se necesitan switches, por lo cual no se muestra ninguno.

La **rejilla** o **malla** de la figura 8-16(c) es un diseño bidimensional que se ha utilizado en muchos sistemas comerciales. Es muy regular y tiene la facilidad de poder escalar a tamaños mayores. Tiene un **diámetro**, que es la ruta más larga entre dos nodos cualesquiera, y que aumenta sólo con base en la raíz cuadrada del número de nodos. Una variante de la malla es el **doble toroide** de la figura 8-16(d), una rejilla con los bordes conectados. No sólo es más tolerante a fallas que la rejilla, sino que el diámetro es también menor debido a que las esquinas opuestas ahora se pueden comunicar en sólo dos saltos.

El **cubo** de la figura 8-16(e) constituye una topología tridimensional regular. Hemos ilustrado un cubo de $2 \times 2 \times 2$, pero en el caso más general podría ser un cubo de $k \times k \times k$. En la figura 8-16(f) tenemos un cubo de cuatro dimensiones, construido a partir de dos cubos tridimensionales con los correspondientes nodos conectados. Podríamos hacer un cubo de cinco dimensiones si

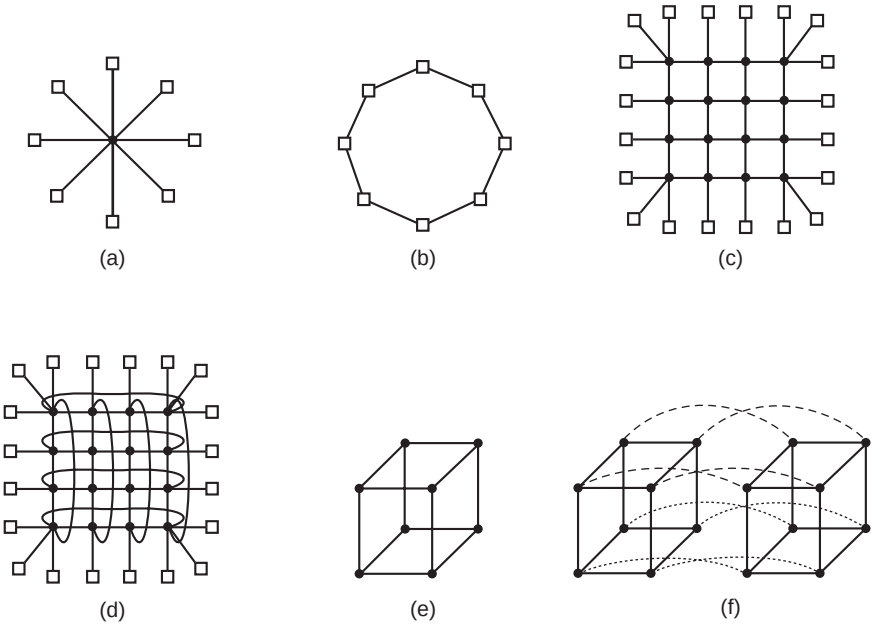


Figura 8-16. Varias topologías de interconexión. (a) Un solo switch. (b) Un anillo. (c) Una rejilla. (d) Un dobel toroide. (e) Un cubo. (f) Un hipercubo 4D.

clonamos la estructura de la figura 8-16(f) y conectamos los nodos correspondientes, para formar un bloque de cuatro cubos. Para obtener seis dimensiones, podríamos duplicar el bloque de cuatro cubos e interconectar los nodos correspondientes, y así en lo sucesivo. A un cubo de *n* dimensiones que se forma así se le conoce como **hipercubo**. Muchas computadoras paralelas utilizan esta topología debido a que el diámetro aumenta en proporción lineal con las dimensiones. Dicho de otra forma, el diámetro es el logaritmo de base 2 del número de nodos; por ejemplo, un hipercubo de 10 dimensiones tiene 1024 nodos pero un diámetro de sólo 10, con lo cual se obtienen excelentes propiedades de retraso. Observe que por el contrario, si se organizan 1024 nodos como una rejilla de 32×32 se tendrá un diámetro de 62, lo cual es más de seis veces peor que el hipercubo. La desventaja de un diámetro más pequeño es que el factor de salida (*fanout*) y por ende el número de vínculos (y el costo), son mucho mayores para el hipercubo.

En las multicomputadoras se utilizan dos tipos de esquemas de conmutación. En el primero, cada mensaje primero se divide (ya sea que lo haga el usuario o la interfaz de red) en un trozo de cierta longitud máxima, conocido como **paquete**. El esquema de conmutación, conocido como **almacenamiento de conmutación de paquetes y retransmisión**, consiste en inyectar el paquete en el primer switch mediante la tarjeta de interfaz de red del nodo de origen, como se muestra en la figura 8-17(a). Los bits llegan uno a la vez, y cuando llega todo el paquete en un búfer de entrada, se copia a la línea que conduce al siguiente switch a lo largo de la ruta, como se muestra en la figura 8-17(b). Cuando llega el paquete al switch conectado al nodo de destino, como se muestra en la

figura 8-17(c), el paquete se copia a la tarjeta de interfaz de red de ese nodo, y en un momento dado también se copia a la RAM.

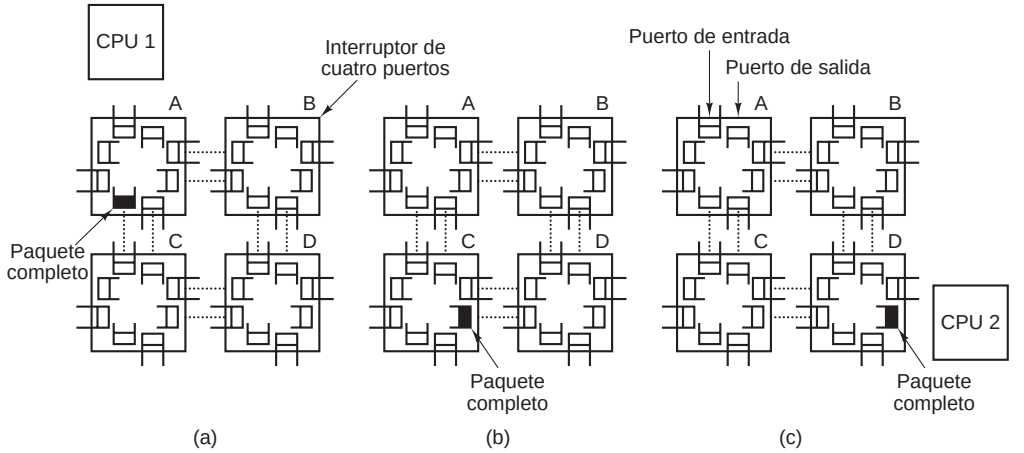


Figura 8-17. Almacenamiento de conmutación de paquetes y retransmisión.

Aunque el almacenamiento de conmutación de paquetes y retransmisión es flexible y eficiente, tiene el problema de que la latencia (retraso) aumenta a través de la red de interconexión. Suponga que el tiempo para que un paquete dé un salto en la figura 8-17 es de *T* nseg. Como el paquete se debe copiar cuatro veces para ir de la CPU 1 a la CPU 2 (de *A* a *C*, luego a *D* y finalmente a la CPU de destino), y no se puede iniciar ninguna copia sino hasta que haya terminado la copia anterior, la latencia a través de la red de interconexión es de 4*T*. Una solución es diseñar una red en la que un paquete se pueda dividir lógicamente en unidades más pequeñas. Tan pronto como llega la primera unidad a un switch, ésta se puede retransmitir, incluso antes de que haya llegado la parte final. Es posible que la unidad llegue a ser tan pequeña como 1 bit.

El otro régimen de conmutación, conocido como **conmutación de circuitos**, consiste en que el primer switch establece primero una ruta a través de todos los switches, hasta el switch de destino. Una vez que se ha establecido esa ruta, los bits se inyectan desde el origen hasta el destino, sin detenerse y con la mayor rapidez posible. No hay búfer en los switches intermedios. La conmutación de circuitos requiere una fase de configuración, que requiere cierto tiempo, pero es más rápida una vez que se ha completado esta fase. Al enviarse el paquete, la ruta se debe deshacer. Hay una variación de la conmutación de circuitos, conocida como **enrutamiento segmentado** (*wormhole routing*), que divide cada paquete en subpaquetes y permite que el primer subpaquete empiece a fluir, incluso antes de que se haya construido la ruta completa.

Interfaces de red

Todos los nodos en una multicomputadora tienen una tarjeta insertable que contiene la conexión del nodo hacia la red de interconexión, la cual mantiene unida toda la multicomputadora. La forma en que están construidas estas tarjetas y la manera en que se conectan a la CPU principal y la

RAM tienen consecuencias considerables para el sistema operativo. Ahora analizaremos brevemente algunas de las cuestiones relacionadas. Parte de este material se basa en (Bhoedjang, 2000).

En casi todas las multicomputadoras, la tarjeta de interfaz contiene una cantidad considerable de RAM para contener los paquetes salientes y entrantes. Por lo general, un paquete saliente se tiene que copiar a la RAM de la tarjeta de interfaz antes de poder transmitirlo al primer switch. La razón de este diseño es que muchas redes de interconexión son asíncronas, por lo que una vez que se inicia la transmisión de un paquete, los bits deben seguir fluyendo a un ritmo constante. Si el paquete está en la RAM principal no se puede garantizar este flujo continuo hacia la red, debido al tráfico adicional en el bus de memoria. Para eliminar este problema se utiliza una RAM dedicada en la tarjeta de interfaz. Este diseño se muestra en la figura 8-18.

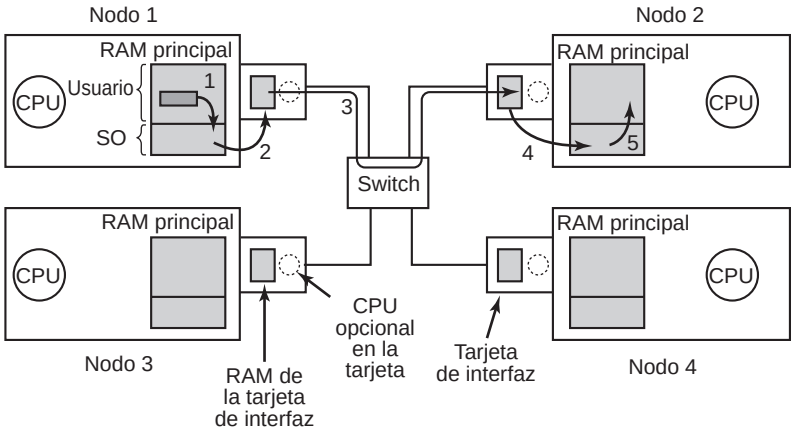


Figura 8-18. Posición de las tarjetas de interfaz de red en una multicomputadora.

Con los paquetes entrantes se produce el mismo problema. Los bits llegan de la red a una velocidad constante y, a menudo, en extremo alta. Si la tarjeta de interfaz de red no puede almacenarlos en tiempo real a medida que vayan llegando, se perderán los datos. Es demasiado riesgoso tratar otra vez aquí de pasar por el bus del sistema (por ejemplo, el bus PCI) a la RAM principal. Como por lo general la tarjeta de red se inserta en el bus PCI, ésta es la única conexión que tiene con la RAM principal, por lo que es inevitable competir por este bus con el disco y cualquier otro dispositivo de E/S. Es más seguro almacenar los paquetes entrantes en la RAM privada de la tarjeta de interfaz, y después copiarlos a la RAM principal más adelante.

La tarjeta de interfaz puede tener uno o más canales de DMA, o incluso puede contener una CPU completa (o tal vez hasta varias CPUs). Los canales de DMA pueden copiar los paquetes entre la tarjeta de interfaz y la RAM principal a una velocidad alta, para lo cual solicitan transferencias de bloques en el bus del sistema, y así se transfieren varias palabras sin tener que solicitar el bus para cada palabra por separado. Sin embargo, es precisamente por este tipo de transferencia de

bloques (que mantiene ocupado el bus del sistema por varios ciclos de bus) que se necesita la RAM de la tarjeta de interfaz.

Muchas tarjetas de interfaz contienen una CPU completa, posiblemente además de uno o más canales de DMA. A estas CPUs se les conoce como **procesadores de red** y se están haciendo cada vez más potentes. Este diseño indica que la CPU principal puede descargar parte de su trabajo y pasarlo a la tarjeta de red, como el manejo de una transmisión confiable (en caso de que el hardware subyacente pueda perder paquetes), la multitransmisión (enviar un paquete a más de un destino), la compresión/descompresión, el cifrado/descifrado y el manejo de la protección en un sistema con varios procesos. Sin embargo, tener dos CPUs significa que deben estar sincronizadas para evitar condiciones de competencia, lo cual agrega más sobrecarga y más trabajo para el sistema operativo.

8.2.2 Software de comunicación de bajo nivel

El enemigo de la comunicación de alto rendimiento en los sistemas de multicomputadora es el copiado de paquetes en exceso. En el mejor caso, habrá una copia de la RAM a la tarjeta de interfaz en el nodo de origen, una copia de la tarjeta de interfaz de origen a la tarjeta de interfaz de destino (si no hay almacenamiento y retransmisión a lo largo de la ruta), y una copia de ahí a la RAM de destino: un total de tres copias. Sin embargo, en muchos sistemas es aún peor. En especial, si la tarjeta de interfaz está asignada al espacio de direcciones virtuales del kernel y no al espacio de direcciones virtuales de usuario, un proceso de usuario sólo puede enviar un paquete si emite una llamada al sistema que se atrape en el kernel. Tal vez los kernels tengan que copiar los paquetes a su propia memoria, tanto en la entrada como en la salida; por ejemplo, para evitar los fallos de página mientras transmiten por la red. Además, el kernel receptor tal vez no sepa en dónde debe colocar los paquetes entrantes sino hasta que haya tenido la oportunidad de examinarlos. Estos cinco pasos de copia se ilustran en la figura 8-18.

Si las copias que van hacia la RAM y que provienen de ella son el cuello de botella, las copias adicionales hacia/desde el kernel pueden duplicar el retraso de un extremo al otro, y reducir la tasa de transferencia a la mitad. Para evitar este impacto en el rendimiento, muchas multicomputadoras asignan la tarjeta de interfaz directamente al espacio de usuario y permiten que el proceso de usuario coloquen los paquetes en la tarjeta directamente, sin que el kernel se involucre. Aunque en definitiva este método ayuda al rendimiento, introduce dos problemas.

En primer lugar, ¿qué tal si hay varios procesos en ejecución en el nodo y necesitan acceso a la red para enviar paquetes? ¿Cuál obtiene la tarjeta de interfaz en su espacio de direcciones? Es problemático hacer una llamada al sistema para asignar la tarjeta y desasignarla de un espacio de direcciones virtuales, pero si sólo un proceso obtiene la tarjeta, ¿cómo envían paquetes los demás procesos? ¿Y qué ocurre si la tarjeta está asignada al espacio de direcciones virtuales del proceso *A* y llega un paquete para el proceso *B*, en especial si *A* y *B* tienen distintos propietarios, ninguno de los cuales desea hacer un esfuerzo por ayudar al otro?

Una solución es asignar la tarjeta de interfaz a todos los procesos que la necesitan, pero entonces se requiere un mecanismo para evitar condiciones de competencia. Por ejemplo, si *A* reclama un búfer en la tarjeta de interfaz y después, debido a un intervalo, se ejecuta *B* y reclama el mismo

búfer, se produce un desastre. Se necesita algún tipo de mecanismo de sincronización, pero estos mecanismos (como los mutexes) sólo funcionan cuando se asume que los procesos van a cooperar. En un entorno de tiempo compartido con varios usuarios, en donde todos tienen prisa por realizar su trabajo, tal vez un usuario podría bloquear el mutex asociado con la tarjeta y nunca liberarlo. La conclusión aquí es que el proceso de asignar la tarjeta de interfaz al espacio de usuario funciona bien nada más cuando hay un solo proceso de usuario en ejecución en cada nodo, a menos que se tomen precauciones especiales (por ejemplo, que distintos procesos asignen distintas porciones de la RAM de la interfaz a sus espacios de direcciones).

El segundo problema es que tal vez el kernel necesite acceso a la red de interconexión en sí, por ejemplo, para acceder al sistema de archivos en un nodo remoto. No es conveniente que el kernel comparta la tarjeta de interfaz con los usuarios, ni siquiera cuando hay tiempo compartido. Suponga que mientras se asigna la tarjeta al espacio de usuario llega un paquete del kernel. O suponga que el proceso de usuario envió un paquete a una máquina remota, pretendiendo ser el kernel. La conclusión es que el diseño más simple es tener dos tarjetas de interfaz de red, una asignada al espacio de usuario para el tráfico de las aplicaciones, y la otra asignada al espacio del kernel para que la utilice el sistema operativo. Muchas multicomputadoras hacen precisamente esto.

Interfaz de comunicación de nodo a red

Otra de las cuestiones es cómo llevar los paquetes a la interfaz de red. La manera más rápida es utilizar el chip de DMA en la tarjeta sólo para copiar los paquetes de la RAM. El problema con este método es que el DMA utiliza direcciones físicas en vez de virtuales, y se ejecuta de manera independiente a la CPU. Para empezar, aunque sin duda un proceso de usuario conoce la dirección virtual de cada paquete que desea enviar, por lo general no conoce la dirección física. No es conveniente hacer una llamada al sistema para realizar la asignación de dirección virtual a dirección física, ya que el objetivo de colocar la tarjeta de interfaz en espacio de usuario en primer lugar era para evitar tener que hacer una llamada al sistema para cada paquete que se va a enviar.

Además, si el sistema operativo decide sustituir una página mientras el chip de DMA copia un paquete de esta página, se transmitirán los datos incorrectos. Peor aún, si el sistema operativo sustituye una página mientras el chip de DMA copia un paquete entrante a esta página, no sólo se perderá el paquete entrante, sino que se arruinará también una página de memoria inocente.

Para evitar estos problemas hay que hacer llamadas al sistema con el fin de marcar y desmarcar páginas en la memoria, marcándolas para indicar que no se pueden paginar temporalmente. Sin embargo, es muy problemático tener que hacer una llamada al sistema para marcar la página que contenga cada uno de los paquetes salientes y después tener que hacer otra llamada para desmarcarla. Si los paquetes son pequeños (por ejemplo, de 64 bytes o menores), la sobrecarga por marcar y desmarcar cada búfer es demasiado alta. Para paquetes grandes (por ejemplo, de 1 KB o más) puede ser tolerable. Para los tamaños intermedios, depende de los detalles del hardware. Además de impactar en el rendimiento, el proceso de marcar y desmarcar páginas hace que el software sea más complejo.

8.2.3 Software de comunicación a nivel de usuario

Para comunicarse, los procesos en distintas CPUs en una multicomputadora se envían mensajes entre sí. En su forma más simple, este proceso de paso de mensajes está expuesto para los procesos de usuario. En otras palabras, el sistema operativo proporciona la forma de enviar y recibir mensajes, y los procedimientos de biblioteca hacen que estas llamadas subyacentes estén disponibles para los procesos de usuario. En una forma más sofisticada, el proceso actual para el paso de mensajes está oculto de los usuarios, al hacer que la comunicación remota sea como una llamada a un procedimiento. A continuación estudiaremos ambos métodos.

Enviar y recibir

En su mínima expresión, los servicios de comunicación que se proporcionen se pueden reducir a dos llamadas (de biblioteca), una para enviar mensajes y otra para recibirlos. La llamada para enviar un mensaje podría ser

```
send(dest, &mptr);
```

y la llamada para recibir un mensaje podría ser

```
receive(direc, &mptr);
```

La primera llamada envía el mensaje al que apunta *mptr* a un proceso identificado por *dest*, y hace que el proceso que hizo la llamada se bloquee hasta que se haya enviado el mensaje. La segunda llamada hace que el proceso que hizo la llamada se bloquee hasta que llegue un mensaje. Cuando pasa esto, el mensaje se copia al búfer al que apunta *mptr* y el proceso que hizo la llamada se desbloquea. El parámetro *direc* especifica la dirección en la que el receptor está escuchando. Hay muchas variantes posibles de estos dos procedimientos y sus parámetros.

Una de las cuestiones es cómo se realiza el direccionamiento. Como las multicomputadoras son estáticas, con un número fijo de CPUs, la manera más sencilla de manejar el direccionamiento es hacer que *addr* sea una dirección compuesta por dos partes: un número de CPU y un número de proceso o de puerto en la CPU direccionada. De esta forma, cada CPU puede administrar sus propias direcciones sin conflictos potenciales.

Comparación entre llamadas con bloqueo y sin bloqueo

Las llamadas antes descritas son **llamadas con bloqueo** (que algunas veces se les conoce como **llamadas síncronas**). Cuando un proceso llama a *send*, especifica un destino y un búfer para enviarlo a ese destino. Mientras se está enviando el mensaje, el proceso emisor se bloquea (es decir, se suspende). La instrucción que sigue a la llamada a *send* no se ejecuta sino hasta que el mensaje se haya enviado por completo, como se muestra en la figura 8-19(a). De manera similar, una llamada a *receive* no devuelve el control sino hasta que se haya recibido en realidad un mensaje y se haya

colocado en el búfer al que apunta el parámetro. El proceso permanece suspendido en *receive* hasta que llega un mensaje, aunque tarde horas en llegar. En algunos sistemas, el receptor puede especificar de qué proceso desea recibir, en cuyo caso permanece bloqueado hasta que llega un mensaje de ese emisor.

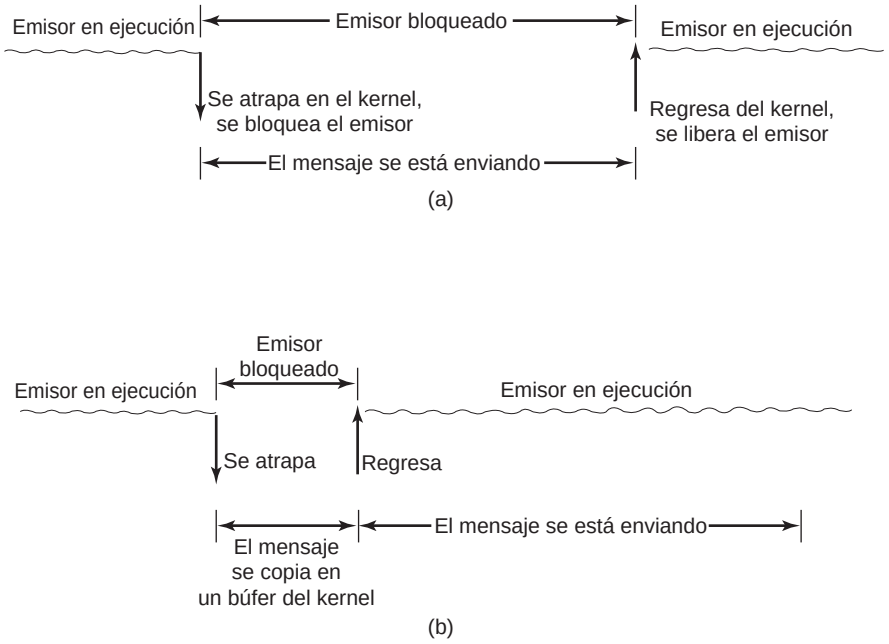


Figura 8-19. (a) Una llamada de envío con bloqueo. (b) Una llamada de envío sin bloqueo.

Una alternativa para las llamadas con bloqueo es el uso de **llamadas sin bloqueo** (a las que algunas veces se les conoce como **llamadas asíncronas**). Si *send* es sin bloqueo, devuelve de inmediato el control al proceso que hizo la llamada, antes de enviar el mensaje. La ventaja de este esquema es que el proceso emisor puede seguir operando en paralelo con la transmisión del mensaje, en vez de que la CPU quede inactiva (suponiendo que no se pueda ejecutar ningún otro proceso). Por lo general, los diseñadores del sistema son los que eligen utilizar primitivas con bloqueo o sin bloqueo (es decir, sólo hay disponible una de las dos primitivas), aunque en unos cuantos sistemas están disponibles ambas primitivas, y los usuarios pueden elegir su favorita.

Sin embargo, la ventaja de rendimiento que ofrecen las primitivas sin bloqueo es contrarrestada por una seria desventaja: el emisor no puede modificar el búfer del mensaje sino hasta que éste se haya enviado. Las consecuencias de que el proceso sobrescriba el mensaje durante la transmisión son demasiado horribles como para contemplarlas. Peor aún, el proceso emisor no tiene idea de cuándo terminará la transmisión, por lo que nunca sabrá cuándo es seguro volver a utilizar el búfer. Es muy difícil que pueda evitar usarlo de manera indefinida.

Hay tres posibles soluciones. La primera es hacer que el kernel copie el mensaje a un búfer interno del kernel y que después permita que continúe el proceso, como se muestra en la figura 8-19(b).

Desde el punto de vista del emisor, este esquema es igual que una llamada con bloqueo: tan pronto como recibe el control de vuelta, puede reutilizar el búfer. Desde luego que el mensaje todavía no se habrá enviado, pero al emisor no le estorba este hecho. La desventaja de este método es que cada mensaje saliente tiene que copiarse del espacio de usuario al espacio del kernel. Con muchas interfaces de red, el mensaje se tendrá que copiar posteriormente a un búfer de transmisión de hardware de todas formas, por lo que básicamente se desperdicia la primera copia. La copia adicional puede reducir el rendimiento del sistema en forma considerable.

La segunda solución es interrumpir al emisor cuando se haya enviado el mensaje por completo, para informarle que el búfer está disponible otra vez. En este caso no se requiere copia, lo cual ahorra tiempo, pero las interrupciones a nivel de usuario hacen que la programación sea engañosa, difícil y que esté sujeta a condiciones de competencia, lo que las hace irreproducibles y casi imposibles de depurar.

La tercera solución es hacer que el búfer copie al escribir; es decir, se marca como de sólo lectura hasta que se haya enviado el mensaje. Si el búfer se vuelve a utilizar antes de enviar el mensaje, se obtiene una copia. El problema con esta solución es que, a menos que el búfer esté aislado en su propia página, las escrituras de las variables cercanas obligarán a que se obtenga una copia. Además se requiere una administración adicional, debido a que el hecho de enviar un mensaje ahora afecta de manera implícita al estado de lectura/escritura de la página. Por último, tarde o temprano es probable que se vuelva a escribir en la página, con lo cual se activará una copia que tal vez ya no sea necesaria.

En consecuencia, las opciones en el lado emisor son:

1. Envío con bloqueo (la CPU queda inactiva durante la transmisión del mensaje).
2. Envío sin bloqueo con copia (se desperdicia el tiempo de la CPU por la copia adicional).
3. Envío sin bloqueo con interrupción (dificulta la programación).
4. Copia al escribir (probablemente se requiera una copia adicional en un momento dado).

Bajo condiciones normales, la primera opción es la mejor, en especial si hay varios hilos disponibles, para que mientras un hilo se bloquee tratando de enviar, los demás hilos puedan continuar su trabajo. Además, no se requiere la administración de búferes del kernel. Lo que es más, como se puede ver al comparar la figura 8-19(a) con la figura 8-19(b), por lo general el mensaje saldrá con más rapidez si no se requiere una copia.

Para que conste, nos gustaría recalcar que algunos autores utilizan un criterio distinto para diferenciar las primitivas asíncronas de las asíncronas. En la opinión alternativa, una llamada es síncrona sólo si el emisor se bloquea hasta que se haya recibido el mensaje y se haya enviado un reconocimiento de vuelta (Andrews, 1991). En el mundo de la comunicación en tiempo real, la palabra “síncrona” tiene otro significado, que por desgracia puede provocar una confusión.

Tanto *send* como *receive* pueden ser con bloqueo o sin bloqueo. Una llamada con bloqueo sólo suspende al emisor hasta que haya llegado un mensaje. Si hay varios hilos disponibles, éste es un método simple. De manera alternativa, una llamada *receive* sin bloqueo sólo indica al kernel en dónde está el búfer, y devuelve el control casi de inmediato. Se puede utilizar una interrupción para indicar que ha llegado un mensaje. Sin embargo, las interrupciones son difíciles de programar además

de ser muy lentas, por lo que tal vez sea preferible que el receptor realice sondeos en busca de mensajes entrantes mediante el uso de un procedimiento (*poll*) que indique si hay mensajes en espera. De ser así, el emisor puede llamar a *get_message*, que devuelve el primer mensaje que haya llegado. En algunos sistemas, el compilador puede insertar llamadas de sondeo en el código en lugares apropiados, aunque es difícil saber con qué frecuencia hay que sondear.

Otra opción más es un esquema en el que la llegada de un mensaje provoca la creación de un nuevo hilo de manera espontánea, en el espacio de direcciones del proceso receptor. A dicho hilo se le conoce como **hilo emergente**. Este hilo ejecuta un procedimiento que se especifica por adelantado y cuyo parámetro es un apuntador al mensaje entrante. Después de procesar el mensaje, simplemente termina y se destruye de manera automática.

Una variante de esta idea es ejecutar el código del receptor directamente en el manejador de interrupciones, para evitar la dificultad de crear un hilo emergente. Para que este esquema sea aún más rápido, el mensaje en sí contiene la dirección del manejador, para que cuando llegue un mensaje se pueda llamar al manejador con unas cuantas instrucciones. La gran ventaja aquí es que no se necesita ningún tipo de copia. El manejador recibe el mensaje de la tarjeta de interfaz y lo procesa al instante. A este esquema se le conoce como **mensajes activos** (Von Eicken y colaboradores, 1992). Como cada mensaje contiene la dirección del manejador, los mensajes activos sólo funcionan cuando los emisores y receptores confían entre sí por completo.

8.2.4 Llamada a procedimiento remoto

Aunque el modelo de paso de mensajes ofrece una manera conveniente de estructurar un sistema operativo de multicomputadora, tiene una falla incorregible: el paradigma básico sobre el que se basa toda la comunicación es la entrada/salida. En esencia, los procedimientos *send* y *receive* se dedican a realizar operaciones de E/S, y muchas personas creen que la E/S es el modelo de programación incorrecto.

Este problema se conoce desde hace mucho tiempo, pero no se había hecho mucho al respecto sino hasta que Birrel y Nelson (1984) escribieron un artículo en el que introdujeron una forma completamente distinta de atacarlo. Aunque la idea es gratificante y simple (ya que alguien pensó en ella), a menudo las implicaciones son sutiles. En esta sección examinaremos el concepto, su implementación, sus puntos fuertes y sus puntos débiles.

En resumen, lo que Birrell y Nelson sugirieron fue permitir que los programas llamen a los procedimientos que se encuentran en otras CPUs. Cuando un proceso en la máquina 1 llama a un procedimiento en la máquina 2, el proceso que hizo la llamada en la máquina 1 se suspende, y se lleva a cabo la ejecución del procedimiento al que llamó en la máquina 2. La información se puede transportar del proceso que hizo la llamada al procedimiento que llamó mediante los parámetros, y puede devolverse en el resultado del procedimiento. El programador no puede ver el paso de mensajes ni las operaciones de E/S. A esta técnica se le conoce como **RPC** (*Remote Procedure Call,* Llamada a procedimiento remoto) y se ha convertido en la base de una gran cantidad de software de multicomputadora. Por tradición, el procedimiento que hace la llamada se denomina cliente y el procedimiento al que se llamó se denomina servidor; en este libro también utilizaremos esos nombres.

La idea detrás de RPC es hacer que una llamada a un procedimiento remoto sea lo más parecida a una llamada local. En su forma más simple, para llamar a un procedimiento remoto, el programa

cliente se debe enlazar con un pequeño procedimiento de biblioteca conocido como **resguardo** (**stub**) **del cliente**, el cual representa al procedimiento del servidor en el espacio de direcciones del cliente. De manera similar, el servidor se enlaza con un procedimiento conocido como **resguardo del servidor**. Estos procedimientos ocultan el hecho de que la llamada al procedimiento desde el cliente al servidor no es local.

En la figura 8-20 se muestran los pasos para llevar a cabo una RPC. El paso 1 es cuando el cliente llama al resguardo del cliente. Esta llamada es una llamada a un procedimiento local, donde los parámetros se meten en la pila de la manera usual. El paso 2 es cuando el resguardo del cliente empaqueta los parámetros en un mensaje y realiza una llamada al sistema para enviarlo. Al proceso de empaquetar los parámetros se le conoce como ***marshaling***. El paso 3 es cuando el kernel envía el mensaje de la máquina cliente a la máquina servidor. El paso 4 es cuando el kernel pasa el paquete entrante al resguardo del servidor (que por lo general llama a *receive* desde antes). Por último, el paso 5 es cuando el resguardo del servidor llama al procedimiento del servidor. La respuesta sigue la misma ruta en dirección opuesta.

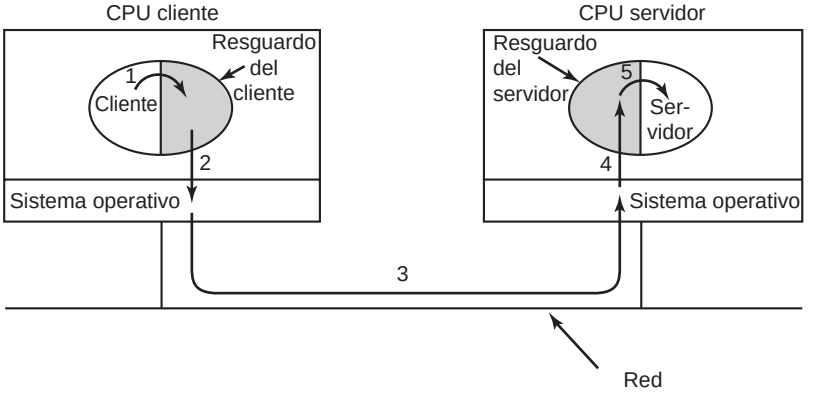


Figura 8-20. Pasos para realizar una llamada a un procedimiento remoto. Los resguardos están sombreados en color gris.

El elemento clave aquí es que el procedimiento cliente (escrito por el usuario) sólo hace una llamada a procedimiento normal (es decir, local) al resguardo del cliente, que tiene el mismo nombre que el procedimiento del servidor. Como el procedimiento del cliente y el resguardo del cliente están en el mismo espacio de direcciones, los parámetros se pasan de la manera usual. De manera similar, un procedimiento llama al procedimiento del servidor en su espacio de direcciones, con los parámetros que espera. Para el procedimiento del servidor no hay nada fuera de lo común. De esta forma, en vez de realizar operaciones de E/S mediante *send* y *receive*, se lleva a cabo la comunicación remota al fingir una llamada a un procedimiento normal.

Aspectos de implementación

A pesar de la elegancia conceptual de la RPC, hay unos cuantos problemas a tratar. Uno de los más importantes es el uso de parámetros tipo apuntador. Por lo general no hay problema al pasar un

apuntador a un procedimiento. El procedimiento al que se llama puede utilizar el apuntador de la misma forma que lo hace el procedimiento que hace la llamada, debido a que los dos procedimientos residen en el mismo espacio de direcciones virtuales. Con la RPC es imposible pasar apuntadores, ya que el cliente y el servidor se encuentran en distintos espacios de direcciones.

En algunos casos es posible emplear trucos para que sea posible pasar apuntadores. Suponga que el primer parámetro es un apuntador a un entero *k*. El resguardo del cliente puede empaquetar a *k* y enviarlo hacia el servidor. Después, el resguardo del servidor crea un apuntador a *k* y lo pasa al procedimiento del servidor, como es de esperarse. Cuando el procedimiento del servidor devuelve el control al resguardo del servidor, éste envía a *k* de vuelta al cliente, en donde el nuevo *k* se copia sobre el anterior, sólo en caso de que el servidor lo haya cambiado. En efecto, la secuencia de llamada estándar de la llamada por referencia se ha sustituido por copia-restauración. Por desgracia, este truco no siempre funciona; por ejemplo, si el apuntador apunta a un grafo u otra estructura de datos compleja. Por esta razón, hay que imponer algunas restricciones sobre los parámetros para los procedimientos a los que se llama de manera remota.

Un segundo problema es que en los lenguajes con tipificación débil como C, es perfectamente válido escribir un procedimiento que calcule el producto interno de dos vectores (arreglos) sin especificar la longitud de ninguno. Cada vector podría terminar por un valor especial conocido sólo para el procedimiento que hizo la llamada y el procedimiento al que llamó. Bajo esas circunstancias, en esencia es imposible que el talón del cliente pueda empaquetar los parámetros, ya que no tiene manera de determinar qué tan grandes son.

Un tercer problema es que no siempre es posible deducir los tipos de los parámetros, ni siquiera a partir de una especificación formal o del mismo código. La función *printf* es un ejemplo, ya que puede tener cualquier número de parámetros (por lo menos uno) y pueden ser una mezcla arbitraria de enteros, enteros cortos, enteros largos, caracteres, cadenas, números de punto flotante de varias longitudes y otros tipos. Prácticamente sería imposible tratar de llamar a *printf* como un procedimiento remoto, ya que C no es tan permisivo. Sin embargo, una regla que diga que se puede utilizar RPC siempre y cuando el lenguaje de programación no sea C (o C++) no sería tan popular.

Un cuarto problema se relaciona con el uso de las variables globales. Por lo general, los procedimientos que hacen las llamadas y aquéllos a los que se llama se pueden comunicar mediante el uso de variables globales, además de hacerlo mediante parámetros. Si el procedimiento al que se llama se cambia a una máquina remota, el código fallará debido a que las variables globales ya no están compartidas.

Estos problemas no indican que RPC no tenga remedio. De hecho se utiliza ampliamente, pero hay que tener cuidado y considerar ciertas restricciones para que funcione bien en la práctica.

8.2.5 Memoria compartida distribuida

Aunque RPC tiene sus atractivos, muchos programadores aún prefieren un modelo de memoria compartida y les gustaría utilizarlo también en una multicomputadora. Para nuestra sorpresa, es posible preservar muy bien la ilusión de la memoria compartida (aun cuando en realidad no existe) mediante el uso de una técnica llamada **DSM** (*Distributed Shared Memory*, Memoria compartida distribuida) (Li, 1986; Li y Hudak, 1989). Con la DSM, cada página se localiza en una de las

memorias de la figura 8-1. Cada máquina tiene su propia memoria virtual y sus propias tablas de páginas. Cuando una CPU realiza una operación LOAD o STORE en una página que no tiene, se produce una interrupción que pasa al sistema operativo. Después, el sistema operativo localiza la página y pide a la CPU que la contiene que la desasigne y la envíe a través de la red de interconexión. Al llegar, la página se asigna y se reinicia la instrucción que fracasó. En efecto, el sistema operativo sólo está dando servicio a los fallos de página desde la RAM remota, en vez de hacerlo desde el disco local. Para el usuario, parece como si la máquina tuviera memoria compartida.

En la figura 8-21 se ilustra la diferencia entre la verdadera memoria compartida y DSM. En la figura 8-21(a) podemos ver un verdadero multiprocesador, en el cual la memoria compartida física se implementa mediante el hardware. En la figura 821(b) podemos ver la DSM implementada por el sistema operativo. En la figura 8-21(c) podemos ver otra forma de memoria compartida, que se implementa mediante niveles más altos de software. Más adelante en este capítulo regresaremos a esta tercera opción, pero por ahora nos concentraremos en la DSM.

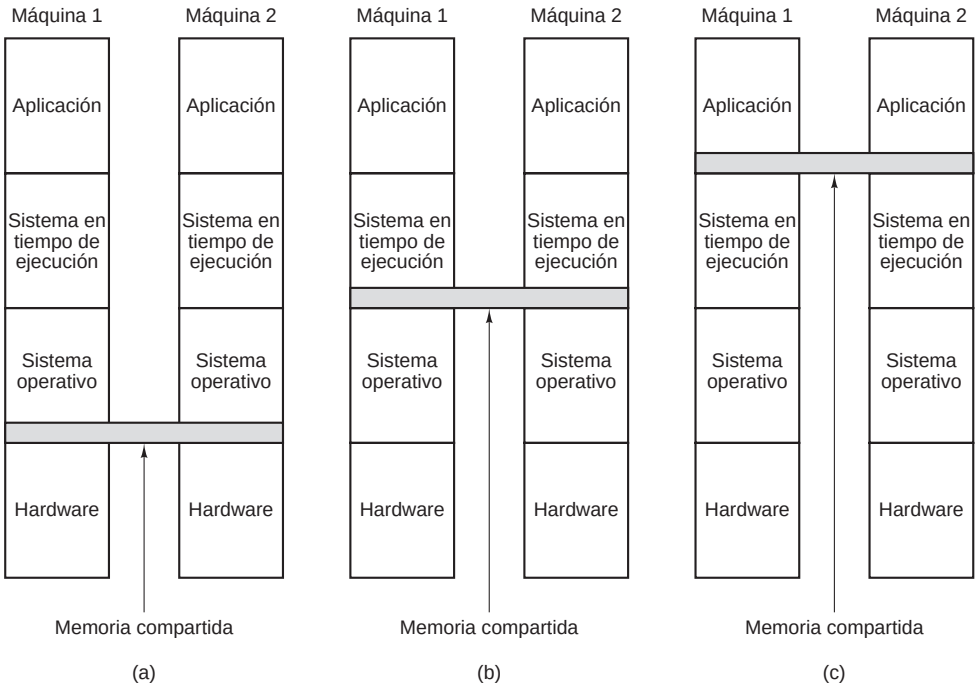


Figura 8-21. Varios niveles en donde se puede implementar la memoria compartida. (a) El hardware. (b) El sistema operativo. (c) Software a nivel de usuario.

Ahora veamos con cierto detalle la forma en que funciona la DSM. En un sistema de DSM el espacio de direcciones se divide en páginas, las cuales se esparcen por todos los nodos en el sistema. Cuando una CPU hace referencia a una dirección que no es local se produce una trampa, el software de DSM obtiene la página que contiene la dirección y reinicia la instrucción fallida, que ahora

se completa con éxito. Este concepto se ilustra en la figura 8-22(a) para un espacio de direcciones con 16 páginas y cuatro nodos, cada uno de los cuales es capaz de contener seis páginas.

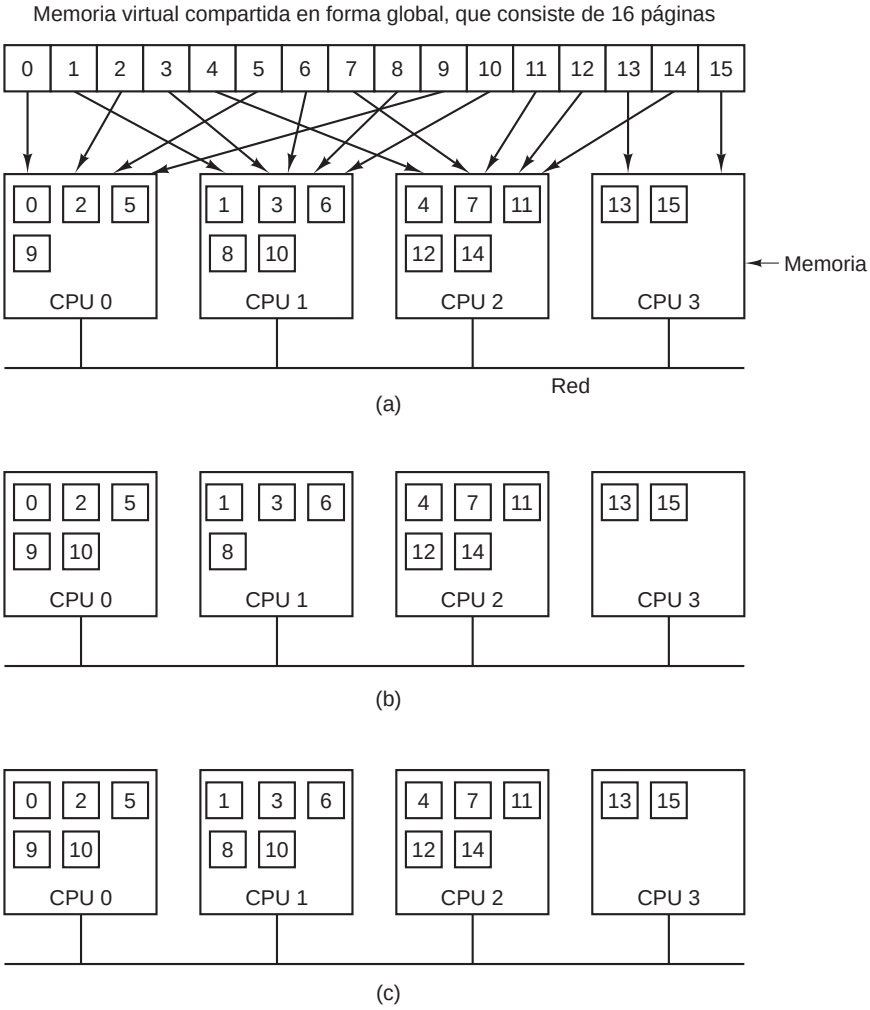


Figura 8-22. (a) Páginas del espacio de direcciones distribuidas entre cuatro máquinas. (b) La situación después de que la CPU 1 hace referencia a la página 10 y ésta se mueve ahí. (c) La situación que ocurre si sólo se lee la página 10 y se utiliza la duplicación.

En este ejemplo, si la CPU 0 hace referencia a instrucciones o datos en las páginas 0, 2, 5 o 9, las referencias se resuelven en forma local. Las referencias a otras páginas producen trampas. Por ejemplo, una referencia a una dirección en la página 10 producirá una trampa al software de DSM, que entonces moverá la página 10 del nodo 1 al nodo 0, como se muestra en la figura 8-22(b).

Duplicación

Una mejora al sistema básico que puede mejorar el rendimiento en forma considerable es la duplicación de páginas que sean de sólo lectura; por ejemplo: texto del programa, constantes de sólo lectura u otras estructuras de datos de sólo lectura. Por ejemplo, si la página 10 en la figura 8-22 es una sección de texto del programa, cuando la CPU 0 la utilice existe la probabilidad de que se envíe una copia a la CPU 0 sin perturbar el texto original en la memoria de la CPU 1, como se muestra en la figura 8-22(c). De esta forma, las CPUs 0 y 1 pueden hacer referencia a la página 10 todas las veces que lo requieran, sin producir trampas para obtener la memoria faltante.

Otra posibilidad es duplicar no sólo las páginas de sólo lectura, sino también todas las páginas. Mientras se estén realizando lecturas, en realidad no hay ninguna diferencia entre duplicar una página de sólo lectura y duplicar una página de lectura-escritura. Pero si se modifica en forma repentina una página duplicada, hay que realizar cierta acción especial para evitar tener varias copias inconsistentes. En las siguientes secciones veremos cómo evitar la inconsistencia.

Compartición falsa

Los sistemas de DSM son similares a los multiprocesadores en ciertos puntos clave. En ambos sistemas, cuando se hace referencia a una palabra de memoria que no sea local, se obtiene un trozo de memoria que contiene la palabra de su ubicación actual y se coloca en la máquina que está haciendo la referencia (memoria principal o caché, respectivamente). Una cuestión importante de diseño es qué tan grande debe ser el tamaño del trozo. En los multiprocesadores, el tamaño del bloque de caché es por lo general de 32 o 64 bytes, para evitar ocupar demasiado tiempo el bus con la transferencia. En los sistemas de DSM, la unidad tiene que ser un múltiplo del tamaño de la página (debido a que la MMU funciona con páginas), pero puede ser de 1, 2, 4 o más páginas. En efecto, al hacer esto se simula un tamaño de página mayor.

Hay ventajas y desventajas en cuanto a un tamaño de página más grande para la DSM. La mayor ventaja es que, como el tiempo de inicio para una transferencia de red es bastante sustancial, en realidad no se requiere mucho más tiempo para transferir 4096 bytes que para transferir 1024 bytes. Al transferir datos en unidades grandes, cuando hay que mover una pieza grande del espacio de direcciones, a menudo se puede reducir el número de transferencias. Esta propiedad es en especial importante debido a que muchos programas exhiben una localidad de referencia, lo cual significa que si un programa ha hecho referencia a una palabra en una página, es probable que haga referencia a otras palabras en la misma página en el futuro inmediato.

Por otra parte, la red estará ocupada más tiempo con una transferencia más grande, y bloqueará las otras fallas provocadas por los otros procesos. Además, con un tamaño efectivo de página demasiado grande se presenta un nuevo problema, conocido como **compartición falsa**, el cual se ilustra en la figura 8-23. Aquí tenemos una página que contiene dos variables compartidas que no están relacionadas (*A* y *B*). El procesador 1 utiliza de manera intensiva la variable *A* en operaciones de lectura y escritura. De manera similar, el proceso 2 utiliza la variable *B* con frecuencia. Bajo estas circunstancias, la página que contiene ambas variables estará viajando constantemente entre una máquina y otra.

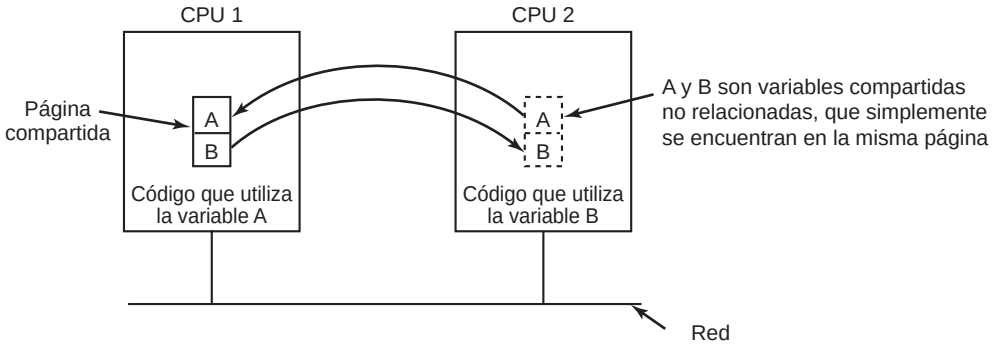


Figura 8-23. Compartición falsa de una página que contiene dos variables no relacionadas.

El problema aquí es que, aunque las variables no están relacionadas, aparecen por casualidad en la misma página, por lo que cuando un proceso utiliza una de ellas, también obtiene la otra. La compartición falsa ocurrirá con más frecuencia entre mayor sea el tamaño efectivo de la página, y por el contrario, ocurrirá con menos frecuencia entre menor sea éste. En los sistemas de memoria virtual ordinarios no hay nada que sea análogo a este fenómeno.

Los compiladores inteligentes que comprenden el problema y colocan las variables en el espacio de direcciones según sea necesario pueden ayudar a reducir la compartición falsa y a mejorar el rendimiento. Sin embargo, decirlo es más fácil que hacerlo. Lo que es más, si la compartición falsa consiste en que el nodo 1 utilice un elemento de un arreglo y el nodo 2 utilice un elemento distinto del mismo arreglo, ni siquiera un compilador inteligente podrá hacer mucho por eliminar el problema.

Cómo lograr una consistencia secuencial

Si no se duplican las páginas en las que se pueden escribir datos, no hay problema para lograr la consistencia. Sólo hay una copia de cada página en la que se puede escribir, y ésta se puede trasladar de manera dinámica, según sea necesario. Como no siempre es posible ver por adelantado cuáles son las páginas en las que se puede escribir, en muchos sistemas DMS cuando un proceso trata de leer una página remota, se crea una copia local y tanto ésta como la copia remota se establecen en sus MMUs respectivas como de sólo lectura. Mientras que todas las referencias sean lecturas, todo está bien.

No obstante, si cualquier proceso intenta escribir en una página duplicada, surge un problema de consistencia potencial debido a que no es aceptable modificar una copia y dejar las demás sin modificar. Esta situación es similar a lo que ocurre en un multiprocesador, cuando una CPU trata de modificar una palabra que está presente en varias cachés. La solución en este caso es que la CPU que está a punto de realizar la escritura coloque primero una señal en el bus, para indicar a todas las demás CPUs que deben descartar su copia del bloque de caché. Por lo general, los sistemas de DMS

funcionan de la misma forma. Antes de poder escribir en una página compartida, se envía un mensaje a todas las demás CPUs que contienen una copia de la página para indicarles que deben desasignar y descartar la página. Una vez que todas estas CPUs hayan respondido que han terminado de realizar la desasignación, la CPU original puede proceder con la escritura.

También es posible tolerar varias copias de páginas en las que se pueda escribir, bajo circunstancias cuidadosamente restringidas. Una forma es permitir que un proceso adquiera un bloqueo sobre una porción del espacio de direcciones virtuales, y que después realice varias operaciones de lectura y escritura en la memoria bloqueada. Al momento en que se libere el bloqueo se podrán propagar los cambios a las demás copias. Mientras que sólo haya una CPU que pueda bloquear una página en un momento dado, este esquema podrá preservar la consistencia.

De manera alternativa, cuando se escribe por primera vez en una página en la que se pueden llegar a escribir datos, se realiza una copia limpia y se guarda en la CPU que está realizando la escritura. Así se pueden adquirir los bloqueos en la página, se actualiza ésta y se liberan los bloqueos. Más adelante, cuando un proceso en una máquina remota trate de adquirir un bloqueo en la página, la CPU que escribió en ella antes compara el estado actual de la página con la copia limpia y crea un mensaje con una lista de todas las palabras que se han modificado. Después, esta lista se envía a la CPU que adquirió el bloqueo para que actualice su copia en vez de invalidarla (Keleher y colaboradores, 1994).

8.2.6 Planificación de multicomputadoras

En un multiprocesador, todos los procesos residen en la misma memoria. Cuando una CPU termina su tarea actual, selecciona un proceso y lo ejecuta. En un principio, todos los procesos son candidatos potenciales. En una multicomputadora, esta situación es bastante distinta. Cada nodo tiene su propia memoria y su propio conjunto de procesos. La CPU 1 no puede decidir de manera repentina ejecutar un proceso localizado en el nodo 4, sin primero realizar una cantidad considerable de trabajo para obtenerlo. Esta diferencia significa que es más fácil la planificación en las multicomputadoras, pero es más importante la asignación de los procesos a los nodos. A continuación estudiaremos estas cuestiones.

La planificación de multicomputadoras es un proceso similar a la planificación de multiprocesadores, pero no todos los algoritmos de la primera se aplican a la segunda. Sin embargo, el algoritmo de multiprocesador más simple (mantener una sola lista central de procesos listos) no funciona debido a que cada proceso sólo se puede ejecutar en la CPU en la que se encuentra ubicado. Sin embargo, al crear un nuevo proceso podemos elegir en dónde colocarlo, por ejemplo para balancear la carga.

Como cada nodo tiene sus propios procesos, se puede utilizar cualquier algoritmo de planificación local. No obstante, también es posible utilizar la planificación de pandilla de los multiprocesadores, ya que tan sólo requiere un acuerdo inicial sobre qué proceso se debe ejecutar en qué ranura de tiempo, y alguna forma de coordinar el inicio de las ranuras de tiempo.

8.2.7 Balanceo de carga

En realidad hay muy poco qué decir sobre la planificación de multicomputadoras, ya que una vez que se asigna un proceso a un nodo se puede utilizar cualquier algoritmo de planificación local, a

menos que se esté utilizando la planificación de pandilla. Sin embargo, y precisamente porque hay muy poco control una vez que se asigna un proceso a un nodo, es importante saber decidir en qué nodo debe ir cada proceso. Esto es lo contrario a los sistemas de multiprocesadores, en donde todos los proceso viven en la misma memoria y se pueden programar en cualquier CPU que se desee. En consecuencia, es conveniente analizar la forma en que se pueden asignar los procesos a los nodos de una manera efectiva. Los algoritmos y la heurística para realizar esta asignación se conocen como **algoritmos de asignación de procesador**.

A través de los años se han propuesto muchos algoritmos de asignación de procesador (es decir, nodo). Estos algoritmos difieren en cuanto a lo que se supone que saben y cuál es el objetivo. Las propiedades que podrían conocerse sobre un proceso incluyen los requerimientos de la CPU, el uso de la memoria y cuánta comunicación con todos los demás procesos. Los posibles objetivos incluyen: minimizar los ciclos desperdiciados de la CPU debido a que no hay trabajo local, minimizar el ancho de banda de comunicación total y asegurar que haya una equidad para los usuarios y los procesos. A continuación examinaremos algunos algoritmos para tener una idea sobre lo que se puede hacer.

Algoritmo determinístico basado en teoría de grafos

Hay una clase de algoritmos que se estudia mucho para los sistemas que consisten en procesos con requerimientos de CPU y memoria conocidos, y una matriz conocida que proporciona la cantidad promedio de tráfico entre cada par de procesos. Si el número de procesos es mayor que el número de CPUs (k), habrá que asignar varios procesos a cada CPU. La idea es realizar esta asignación para minimizar el tráfico de red.

El sistema se puede representar como un grafo ponderado donde cada vértice es un proceso y cada arco representa el flujo de mensajes entre dos procesos. En sentido matemático, el problema se reduce a buscar una forma de particionar (es decir, recortar) el grafo en k subgrafos separados, sujetos a ciertas restricciones (por ejemplo, los requerimientos totales de CPU y memoria por debajo de ciertos límites para cada subgrafo). Para cada solución que cumpla con las restricciones, los arcos que se encuentran por completo dentro de un solo subgrafo representan la comunicación entre máquinas, y se pueden ignorar. Los arcos que van de un subgrafo a otro representan el tráfico de la red. El objetivo entonces es encontrar el particionamiento que minimice el tráfico de red, al tiempo que cumpla con todas las restricciones. Como ejemplo, en la figura 8-24 se muestra un sistema de nueve procesos (A a I), en donde cada arco está etiquetado con la carga de comunicación media entre los dos procesos correspondientes (por ejemplo, en Mbps).

En la figura 8-24(a) hemos particionado el grafo con los procesos A, E y G en el nodo 1, los procesos B, F y H en el nodo 2 y los procesos C, D e I en el nodo 3. El tráfico total de la red es la suma de los arcos intersecados por los cortes (las líneas punteadas), o 30 unidades. En la figura 8-24(b) tenemos un particionamiento distinto, que sólo tiene 28 unidades de tráfico de red. Suponiendo que cumpla con todas las restricciones de memoria y de CPU, ésta es una mejor opción debido a que requiere menos comunicación.

Por intuición, lo que estamos haciendo es buscar clústeres que tengan un acoplamiento fuerte (flujo alto de tráfico entre clústeres), pero que interactúen poco con otros clústeres (flujo bajo de tráfico entre clústeres). Algunos de los primeros artículos que analizan este problema son (Chow y Abraham, 1982; Lo, 1984; Stone y Bokhari, 1978).

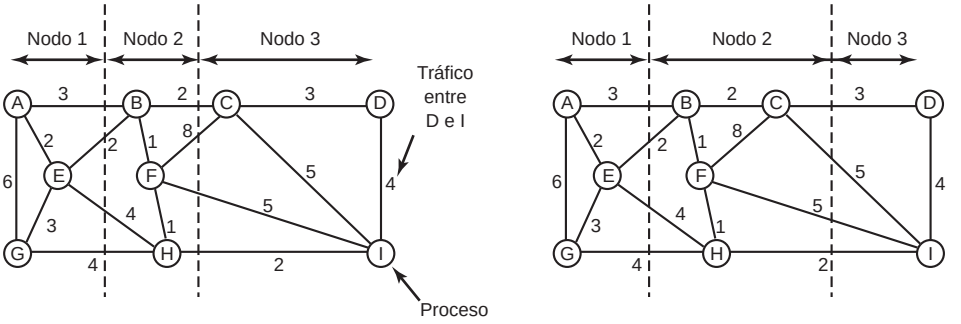


Figura 8-24. Dos formas de asignar procesos a tres nodos.

Un algoritmo heurístico iniciado por el emisor

Ahora veamos algunos algoritmos distribuidos. Un algoritmo dice que cuando se crea un proceso, se ejecuta en el nodo que lo creó a menos que ese nodo esté sobrecargado. La métrica para sobrecarga podría implicar demasiados procesos, un conjunto de trabajo total muy grande o alguna otra métrica. Si el nodo está sobrecargado, selecciona otro nodo al azar y le pregunta sobre su carga (usando la misma métrica). Si la carga de ese nodo es menor a cierto valor de umbral, el nuevo proceso se envía a ese nodo (Eager y colaboradores, 1986). Si no es menor, se selecciona otra máquina para el sondeo. Este proceso de sondeo no es indefinido. Si no se encuentra un host adecuado en *N* sondeos o menos, el algoritmo termina y el proceso se ejecuta en la máquina que lo originó. La idea es que los nodos con mucha carga traten de deshacerse del exceso de trabajo, como se muestra en la figura 8-25(a), que ilustra el balanceo de carga iniciado por el emisor.

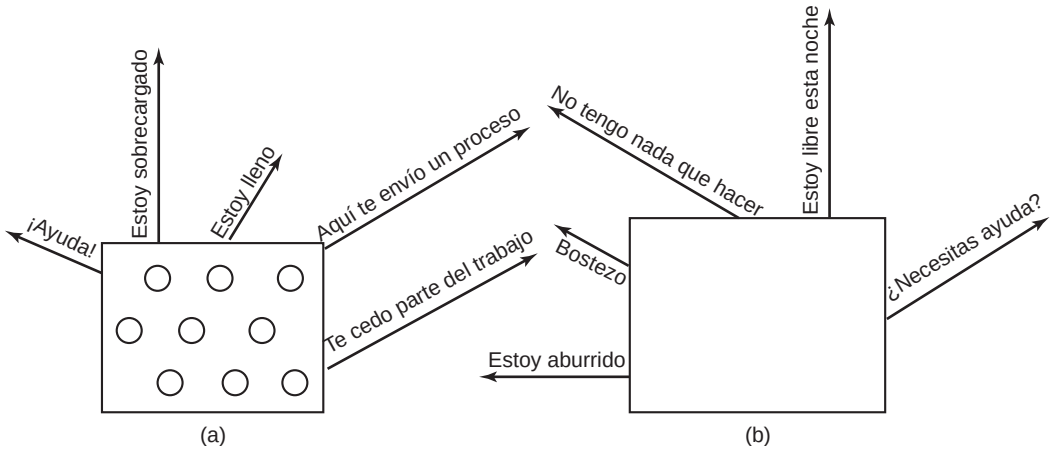


Figura 8-25. (a) Un nodo sobrecargado busca a un nodo con poca carga para darle algunos procesos. (b) Un nodo vacío busca algo que hacer.

Eager y sus colaboradores (1986) construyeron un modelo de puesta en cola analítico de este algoritmo. Mediante el uso de este modelo se estableció que el algoritmo tiene un buen comportamiento y es estable bajo un amplio rango de parámetros, incluyendo diversos valores de umbral, costos de transferencia y límites de sondeo.

Sin embargo, hay que tener en cuenta que bajo condiciones de mucha carga, todas las máquinas enviarán sondas constantemente a otras máquinas, en un intento inútil por encontrar una que esté dispuesta a aceptar más trabajo. Se descargarán pocos procesos hacia la otra máquina, además de que se puede producir una considerable sobrecarga al tratar de hacerlo.

Un algoritmo heurístico distribuido, iniciado por el receptor

Hay un algoritmo complementario para el que analizamos en la sección anterior (que lo inicia un emisor sobrecargado), el cual lo inicia un receptor con poca carga, como se muestra en la figura 8-25(b). Con este algoritmo, cada vez que un proceso termina, el sistema comprueba si tiene suficiente trabajo. De no ser así, selecciona una máquina al azar y le pide trabajo. Si esa máquina no tiene nada que ofrecer, selecciona una segunda máquina y después una tercera. Si no se encuentra trabajo después de *N* sondeos, el nodo deja de pedir trabajo temporalmente, realiza cualquier trabajo que se haya puesto en cola e intenta de nuevo cuando termina el siguiente proceso. Si no hay trabajo disponible, la máquina queda inactiva. Después de cierto intervalo fijo, empieza a sondear de nuevo.

Una ventaja de este algoritmo es que no impone una carga adicional sobre el sistema en momentos críticos. El algoritmo iniciado por el emisor realiza muchos sondeos precisamente cuando el sistema tiene menos capacidad de tolerarlos: cuando tiene mucha carga. Con el algoritmo iniciado por el receptor, cuando el sistema tiene mucha carga hay muy poca probabilidad de que una máquina no tenga suficiente trabajo. Sin embargo, cuando ocurra esto será fácil encontrar trabajo por hacer. Desde luego que cuando hay poco trabajo por hacer, el algoritmo iniciado por el receptor crea un tráfico de sondeo considerable, ya que todas las máquinas desempleadas están buscando trabajo con desesperación. No obstante, es mucho mejor que aumente la sobrecarga cuando el sistema tiene poca carga que cuando está sobrecargado.

También es posible combinar ambos algoritmos, para que las máquinas traten de deshacerse de trabajo cuando tengan mucho, y traten de adquirir trabajo cuando no tengan suficiente. Además, las máquinas tal vez puedan mejorar el sondeo al azar si mantienen un historial de sondeos anteriores, para determinar si tienen un problema crónico de poca carga o de mucha carga. Se puede probar uno de estos algoritmos primero, dependiendo de si el iniciador está tratando de deshacerse de trabajo, o de adquirir más.

8.3 VIRTUALIZACIÓN

En ciertas situaciones, una empresa tiene una multicomputadora pero en realidad no la quiere. Un ejemplo común es cuando una empresa tiene un servidor de correo electrónico, un servidor Web, un servidor FTP, algunos servidores de comercio electrónico, y otros servidores más. Todos estos servidores se ejecutan en distintas computadoras del mismo bastidor de equipos, y todos están conec-

tados por una red de alta velocidad; en otras palabras, es una multicomputadora. En algunos casos, todos estos servidores se ejecutan en máquinas separadas debido a que una sola máquina no puede manejar la carga, pero en muchos otros casos la razón principal para no ejecutar todos estos servicios como procesos en la misma máquina es la confiabilidad: la administración simplemente no confía en que el sistema operativo se ejecute las 24 horas del día, los 365 o 366 días del año sin fallas. Al colocar cada servicio en una computadora separada, si falla uno de los servidores por lo menos los demás no se verán afectados. Aunque de esta forma se logra la tolerancia a fallas, esta solución es costosa y difícil de administrar debido a que hay muchas máquinas involucradas.

¿Qué se debe hacer? Se ha propuesto como solución la tecnología de las máquinas virtuales, que a menudo se conoce sólo como **virtualización** y tiene más de 40 años, como vimos en la sección 1.7.5. Esta tecnología permite que una sola computadora contenga varias máquinas virtuales, cada una de las cuales puede llegar a ejecutar un sistema operativo distinto. La ventaja de este método es que una falla en una máquina virtual no ocasiona que las demás fallen de manera automática. En un sistema virtualizado, se pueden ejecutar distintos servidores en diferentes máquinas virtuales, con lo cual se mantiene el modelo parcial de fallas que tiene una computadora, pero a un costo mucho menor y con una administración más sencilla.

Claro que consolidar los servidores de esta forma es como poner todos los huevos en una canasta. Si falla el servidor que ejecuta todas las máquinas virtuales, el resultado es aún más catastrófico que cuando falla un solo servidor dedicado. Sin embargo, la razón por la que la virtualización puede funcionar es que la mayoría de las fallas en los servicios no se deben a un hardware defectuoso, sino al software presuntuoso, poco confiable y lleno de errores, en especial los sistemas operativos. Con la tecnología de máquinas virtuales, el único software que se ejecuta en el modo del kernel es el hipervisor, el cual tiene 100 veces menos líneas de código que un sistema operativo completo, y por ende tiene 100 veces menos errores.

La ejecución de software en las máquinas virtuales tiene otras ventajas además de un sólido aislamiento. Una de ellas es que al tener menos máquinas físicas hay un ahorro en hardware y electricidad, y se ocupa menos espacio en la oficina. Para una compañía como Amazon, Yahoo, Microsoft o Google, que puede tener cientos de miles de servidores que realizan una enorme variedad de tareas distintas, la reducción de las demandas físicas en sus centros de datos representa un enorme ahorro en los costos. Por lo general, en las empresas grandes los departamentos individuales o grupos piensan en una idea interesante y después van y compran un servidor para implementarla. Si la idea tiene éxito y se requieren cientos o miles de servidores, se expande el centro de datos corporativo. A menudo es difícil mover el software a las máquinas existentes, debido a que cada aplicación necesita con frecuencia una versión distinta del sistema operativo, sus propias bibliotecas, archivos de configuración y demás. Con las máquinas virtuales, cada aplicación puede tener su propio entorno.

Otra ventaja de las máquinas virtuales es que es mucho más fácil usar puntos de comprobación y migrar datos entre una máquina virtual y otra (por ejemplo, para balancear la carga entre varios servidores) que migrar procesos que se ejecutan en un sistema operativo normal. En este último caso, se mantiene una cantidad considerable de información crítica de estado sobre cada proceso en las tablas del sistema operativo, incluyendo la información relacionada con la apertura de archivos, las alarmas, los manejadores de señales y demás. Al migrar una máquina virtual, todo lo que hay que mover es la imagen de memoria, ya que todas las tablas del sistema operativo se mueven también.

Otro uso para las máquinas virtuales es para ejecutar aplicaciones heredadas en los sistemas operativos (o en versiones de los sistemas operativos) que ya no tienen soporte o que no funcionan en el hardware actual. Estas aplicaciones heredadas se pueden ejecutar al mismo tiempo y en el mismo hardware que las aplicaciones actuales. De hecho, la habilidad de ejecutar al mismo tiempo aplicaciones que utilizan distintos sistemas operativos es un gran argumento a favor de las máquinas virtuales.

El desarrollo de software es otro uso aún más importante de las máquinas virtuales. Un programador que quiera asegurarse que su software funcione en Windows 98, Windows 2000, Windows XP, Windows Vista, varias versiones de Linux, FreeBSD, OpenBSD, NetBSD y Mac OS X ya no tiene que conseguir una docena de computadoras e instalar distintos sistemas operativos en todas ellas. Lo único que tiene que hacer es crear una docena de máquinas virtuales en una sola computadora e instalar distintos sistemas operativos en cada máquina virtual. Desde luego que el programador podría haber particionado el disco duro para instalar un sistema operativo distinto en cada partición, pero este método es más dificultoso. En primer lugar, las PCs estándar sólo soportan cuatro particiones primarias de disco, sin importar qué tan grande sea. En segundo lugar, aunque se podría instalar un programa de multiarranque en el bloque de arranque, sería necesario reiniciar la computadora para trabajar en otro sistema operativo. Con las máquinas virtuales todos los sistemas operativos se pueden ejecutar al mismo tiempo, ya que en realidad sólo son procesos glorificados.

8.3.1 Requerimientos para la virtualización

Como vimos en el capítulo 1, hay dos métodos para la virtualización. En la figura 1-29(a) se muestra un tipo de hipervisor, denominado **hipervisor de tipo 1** (o **monitor de máquina virtual**). En realidad es el sistema operativo, ya que es el único programa que se ejecuta en modo del kernel. Su trabajo es soportar varias copias del hardware actual, conocidas como **máquinas virtuales**, de una manera similar a los procesos que soporta un sistema operativo normal. Por el contrario, un **hipervisor de tipo 2** [que se muestra en la figura 1-29(b)] es algo completamente distinto. Es sólo un programa de usuario que se ejecuta (por decir) en Windows o Linux e "interpreta" el conjunto de instrucciones de la máquina, el cual también crea una máquina virtual. Pusimos "interpreta" entre comillas debido a que por lo general, los trozos de código se procesan de cierta forma para después colocarlos en la caché y ejecutarlos directamente para mejorar el rendimiento, pero la interpretación completa funcionaría en principio, aunque con lentitud. El sistema operativo que se ejecuta encima del hipervisor en ambos casos se denomina **sistema operativo invitado**. En el caso de un hipervisor de tipo 2, el sistema operativo que se ejecuta en el hardware se denomina **sistema operativo anfitrión**.

Es importante tener en cuenta que en ambos casos las máquinas virtuales deben actuar justo de igual forma que el hardware real. Específicamente, debe ser posible iniciarlas como máquinas reales e instalar cualquier sistema operativo en ellas, justo como lo que se puede hacer con el hardware real. La tarea del hipervisor es brindar esta ilusión con eficiencia (sin ser un intérprete completo).

La razón de que haya dos tipos de hipervisores se debe a ciertos defectos en la arquitectura del Intel 386 que se acarrearon servilmente a las nuevas CPUs durante 20 años, para mantener la compatibilidad inversa. En síntesis, cada CPU con modo de kernel y modo de usuario tiene un conjun-

to de instrucciones que sólo se pueden ejecutar en modo de kernel, como las instrucciones que realizan operaciones de E/S, las que modifican las opciones de la MMU, etcétera. Popek y Goldberg (1974) desarrollaron un trabajo clásico sobre virtualización, en el cual a estas instrucciones les llamaron **instrucciones sensibles**. También hay un conjunto de instrucciones que producen una trampa (interrupción) si se ejecutan en modo de usuario. A estas instrucciones, Popek y Goldberg les llamaron **instrucciones privilegiadas**. En su artículo declararon por primera vez que una máquina se puede virtualizar sólo si las instrucciones sensibles son un subconjunto de las instrucciones privilegiadas. Para explicarlo en un lenguaje más simple, si usted trata de hacer algo en modo de usuario que no deba hacer en este modo, el hardware deberá producir una interrupción. A diferencia de la IBM/370, que tenía esta propiedad, el Intel 386 no la tiene. Se ignoraban muchas instrucciones sensibles del 386 si se ejecutaba en modo de usuario. Por ejemplo, la instrucción `POPF` sustituye el registro de banderas, que modifica el bit que habilita/deshabilita las interrupciones. En modo de usuario, este bit simplemente no se modifica. Como consecuencia, el 386 y sus sucesores no se podían virtualizar, por lo que no podían soportar un hipervisor de tipo 1.

En realidad la situación es un poco peor de lo que se muestra. Además de los problemas con las instrucciones que no se pueden atrapar en modo de usuario, hay instrucciones que pueden leer el estado sensible en modo de usuario sin producir una interrupción. Por ejemplo, en el Pentium un programa puede determinar si se está ejecutando en modo de usuario o en modo de kernel con sólo leer su selector del segmento de código. Un sistema operativo que hiciera esto y descubriera que se encuentra en modo de usuario podría tomar una decisión incorrecta con base en esta información.

Este problema se resolvió cuando Intel y AMD introdujeron la virtualización en sus CPUs, empezando en el 2005. En las CPUs Intel Core 2 se conoce como **VT** (**Tecnología de virtualización**); en las CPUs AMD Pacific se conoce como **SVM** (**Máquina virtual segura**). A continuación utilizaremos el término VT en un sentido genérico. Ambas tecnologías se inspiraron en el trabajo de la IBM VM/370, pero tienen unas cuantas diferencias. La idea básica es crear contenedores en los que se puedan ejecutar máquinas virtuales. Cuando se inicia un sistema operativo invitado en un contenedor, se sigue ejecutando ahí hasta que produce una excepción y se atrapa en el hipervisor, por ejemplo, mediante la ejecución de una instrucción de E/S. El conjunto de operaciones que se atrapan se controla mediante un mapa de bits de hardware establecido por el hipervisor. Con estas extensiones, es posible utilizar el clásico método de atrapar y emular de una máquina virtual.

8.3.2 Hipervisores de tipo 1

La capacidad de virtualización es una cuestión importante, por lo que la examinaremos con más detalle. En la figura 8-26 podemos ver un hipervisor de tipo 1 que soporta una máquina virtual. Al igual que todos los hipervisores de tipo 1, se ejecuta en modo de kernel. La máquina virtual se ejecuta como un proceso de usuario en modo de usuario y, como tal, no puede ejecutar instrucciones sensibles. La máquina virtual ejecuta un sistema operativo invitado que piensa que se encuentra en modo de kernel, aunque desde luego se encuentra en modo de usuario. A éste le llamaremos **modo de kernel virtual**. La máquina virtual también ejecuta procesos de usuario, los cuales creen que se encuentran en modo de usuario (y en realidad así es).

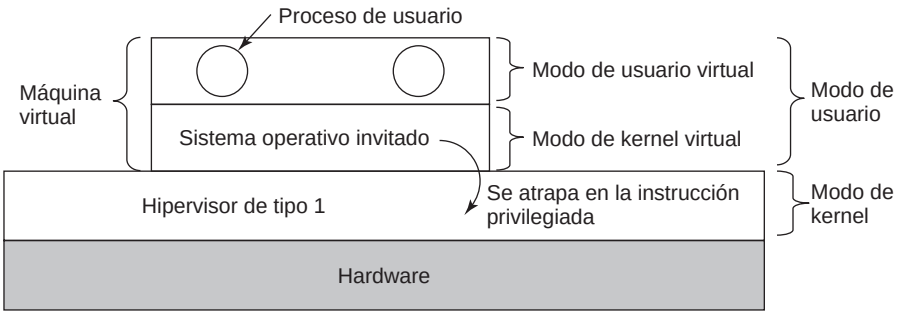


Figura 8-26. Cuando el sistema operativo en una máquina virtual ejecuta una instrucción que sólo se puede ejecutar en modo de kernel, se atrapa en el hipervisor si hay tecnología de virtualización presente.

¿Qué ocurre cuando el sistema operativo (el cual cree que se encuentra en modo de kernel) ejecuta una instrucción sensible (una que sólo se permite en modo de kernel)? En las CPUs sin VT, la instrucción falla y por lo general también lo hace el sistema operativo. Esto hace que la verdadera virtualización sea imposible. Sin duda podríamos argumentar que todas las instrucciones sensibles siempre se deben atrapar al ejecutarse en modo de usuario, pero esa no es la forma en que trabajaba el 386 y sus sucesores que no utilizaban la tecnología VT.

En las CPUs con VT, cuando el sistema operativo invitado ejecuta una instrucción sensible se produce una interrupción en el kernel, como se muestra en la figura 8-26. Así, el hipervisor puede inspeccionar la instrucción para ver si el sistema operativo anfitrión la emitió en la máquina virtual, o si fue un programa de usuario en la máquina virtual. En el primer caso, hace las preparaciones para que se ejecute la instrucción; en el segundo caso, emula lo que haría el hardware real al confrontarlo con una instrucción sensible que se ejecuta en modo de usuario. Si la máquina virtual no tiene VT, por lo general la instrucción se ignora; si tiene VT, se atrapa en el sistema operativo invitado que se ejecuta en la máquina virtual.

8.3.3 Hipervisores de tipo 2

El proceso de crear un sistema de máquina virtual es bastante simple cuando hay VT disponible, pero ¿qué hacían las personas antes de eso? Sin duda, no se podría ejecutar un sistema operativo completo en una máquina virtual debido a que sólo se ignorarían (algunas de) las instrucciones sensibles, y el sistema fallaría. En vez de ello, lo que ocurrió fue la invención de lo que ahora se conoce como **hipervisores de tipo 2**, como se muestra en la figura 1-29(b). El primero de estos hipervisores fue **VMware** (Adams y Agesen, 2006; y Waldspurger, 2002), el fruto del proyecto de investigación DISCO en la Universidad de Stanford (Bugnion y colaboradores, 1997). VMware se ejecuta como un programa de usuario ordinario encima de un sistema operativo anfitrión como Windows o Linux. Cuando se inicia por primera vez, actúa como una computadora que se acaba de iniciar y espera encontrar en la unidad de CD-ROM un disco que contenga un sistema operativo.

Después instala el sistema operativo en su **disco virtual** (que en realidad sólo es un archivo de Windows o Linux), para lo cual ejecuta el programa de instalación que se encuentra en el CD-ROM. Una vez que se instala el sistema operativo invitado en el disco virtual, se puede iniciar al ejecutarlo.

Ahora veamos cómo funciona VMware con un poco más de detalle. Al ejecutar un programa binario del Pentium, que puede obtener del CD-ROM de instalación o del disco virtual, primero explora el código para buscar **bloques básicos**; es decir, ejecuciones de instrucciones seguidas que terminan en un salto, una llamada, una interrupción o alguna instrucción que modifica el flujo de control. Por definición, ningún bloque básico contiene una instrucción que modifique el contador del programa, excepto el último. Se inspecciona el bloque básico para ver si contiene instrucciones sensibles (en el sentido de Popek y Goldberg). De ser así, cada una de estas instrucciones se sustituye con una llamada a un procedimiento de VMware que la maneja. La instrucción final también se sustituye con una llamada a VMware.

Una vez que se realicen estos pasos, el bloque básico se coloca en la caché de VMware y después se ejecuta. Un bloque básico que no contenga instrucciones sensibles se ejecutará con la misma rapidez bajo VMware que en la máquina básica (ya que se está ejecutando en esa máquina). Las instrucciones sensitivas se atrapan y emulan de esta manera. A esta técnica se le conoce como **traducción binaria**.

Una vez que el bloque básico haya completado su ejecución, el control se regresa a VMware, que localiza a su sucesor. Si ya se ha traducido el sucesor, se puede ejecutar de inmediato. Si no, primero se traduce, se coloca en la caché y después se ejecuta. En un momento dado, la mayor parte del programa estará en la caché y se ejecutará casi a la velocidad completa. Se utilizan varias optimizaciones; por ejemplo, si un bloque básico termina al saltar (o llamar) a otro, se puede sustituir la instrucción final mediante un salto o llamada directamente al bloque básico traducido, con lo cual se elimina toda la sobrecarga asociada con la búsqueda del bloque sucesor. Además, no hay necesidad de sustituir las instrucciones sensibles en los programas de usuario; el hardware las ignorará de todas formas.

Ahora debe estar claro por qué funcionan los hipervisores de tipo 2, incluso en hardware que no se puede virtualizar: todas las instrucciones sensibles se sustituyen mediante llamadas a procedimientos que emulan estas instrucciones. El verdadero hardware nunca ejecuta las instrucciones sensibles que emite el sistema operativo invitado. Se convierten en llamadas al hipervisor, quien a su vez las emula.

Tal vez podríamos esperar que las CPUs con VT tuvieran un rendimiento mucho mayor que las técnicas de software que se utilizan en los hipervisores de tipo 2, pero las mediciones muestran otra cosa (Adams y Agesen, 2006). Resulta ser que el método de atrapar y emular que utiliza el hardware con VT genera muchas interrupciones, y éstas son muy costosas en el hardware moderno, debido a que arruinan las cachés de las CPUs, los TLBs y las tablas de predicción de bifurcación internas para la CPU. Por el contrario, cuando las instrucciones sensibles se sustituyen mediante llamadas a procedimientos de VMware dentro del proceso que está en ejecución, no se produce sobrecarga debido a este cambio de contexto. Como muestran Adams y Agesen, algunas veces el software vence al hardware, dependiendo de la carga de trabajo. Por esta razón, algunos hipervisores de tipo 1 realizan la traducción binaria por cuestiones de rendimiento, aun cuando el software se ejecute de manera correcta sin ella.

8.3.4 Paravirtualización

Los hipervisores de tipo 1 y tipo 2 funcionan con sistemas operativos invitados que no estén modificados, pero tienen que hacer un gran esfuerzo por obtener un rendimiento razonable. Un método distinto que se está haciendo popular es modificar el código fuente del sistema operativo invitado, de manera que en vez de ejecutar instrucciones sensibles, realiza **llamadas al hipervisor**. En efecto, el sistema operativo invitado actúa como un programa de usuario que realiza llamadas al sistema operativo (el hipervisor). Cuando se utiliza este método, el hipervisor debe definir una interfaz que consiste en un conjunto de llamadas a procedimientos que los sistemas operativos invitados puedan utilizar. Este conjunto de llamadas forma en efecto una **API** (**Interfaz de programación de aplicaciones**), aun cuando es una interfaz para que la utilicen los sistemas operativos invitados, y no los programas de aplicación.

Si vamos un paso más allá, al eliminar todas las instrucciones sensibles del sistema operativo para que sólo haga llamadas al hipervisor para obtener servicios del sistema como la E/S, hemos convertido al hipervisor en un microkernel como el de la figura 1-26. Se dice que un sistema operativo invitado para el que se han eliminado de manera intencional (algunas) instrucciones sensibles está **paravirtualizado** (Barham y colaboradores, 2003; Whitaker y colaboradores, 2002). La emulación de instrucciones peculiares del hardware es una tarea desagradable y que consume mucho tiempo. Requiere una llamada al hipervisor, para después emular la semántica exacta de una instrucción complicada. Es mucho mejor tan sólo hacer que el sistema operativo invitado llame al hipervisor (o microkernel) para realizar las operaciones de E/S, y así en lo sucesivo. La principal razón por la que los primeros hipervisores sólo emulaban la máquina completa era por la falta de disponibilidad de código fuente para el sistema operativo invitado (por ejemplo, Windows), o debido a la gran cantidad de variantes (como Linux). Tal vez en el futuro se estandarizará la API del hipervisor/microkernel y los próximos sistemas operativos se diseñarán para llamar a esa API, en vez de utilizar instrucciones sensibles. Esto facilitaría el soporte y uso de la tecnología de las máquinas virtuales.

En la figura 8-27 se ilustra la diferencia entre la verdadera virtualización y la paravirtualización. Aquí tenemos dos máquinas virtuales que se soportan mediante hardware de VT. A la izquierda hay una versión sin modificar de Windows como el sistema operativo invitado. Cuando se ejecuta una instrucción sensible, el hardware hace que se atrape en el hipervisor, que a su vez la emula y regresa. A la derecha hay una versión de Linux modificada de manera que ya no contiene instrucciones sensibles. Así, cuando necesita realizar operaciones de E/S o modificar registros internos críticos (como el que apunta a las tablas de páginas), realiza una llamada al hipervisor para realizar el trabajo, de igual forma que cuando un programa de aplicación hace una llamada al sistema en Linux estándar.

En la figura 8-27 hemos mostrado que el hipervisor se divide en dos partes separadas por una línea punteada. En realidad sólo hay un programa que se ejecuta en el hardware. Una parte de este programa es responsable de interpretar las instrucciones sensibles que se atrapan, que en este caso son de Windows. La otra parte del programa sólo lleva a cabo las llamadas al hipervisor. En la figura, la segunda parte del programa se etiqueta como “microkernel”. Si el hipervisor sólo ejecuta sistemas operativos invitados paravirtualizados, no hay necesidad de emular instrucciones sensibles y tenemos un verdadero microkernel que sólo proporciona servicios muy básicos, como despachar

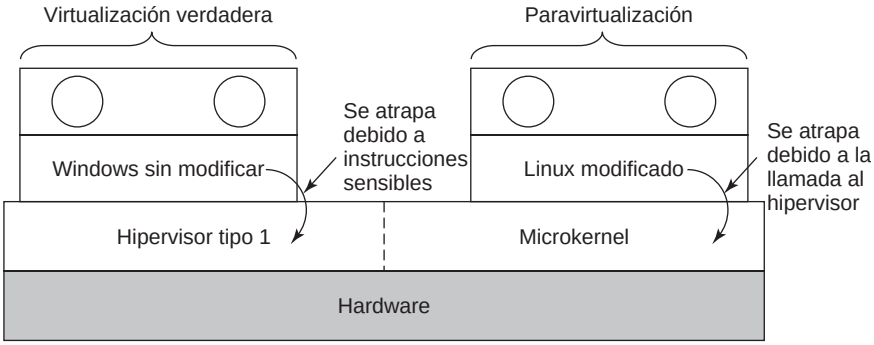


Figura 8-27. Un hipervisor que soporta la virtualización verdadera y la paravirtualización.

procesos y administrar la MMU. El límite entre un hipervisor de tipo 1 y un microkernel es de por sí impreciso, y empeorará aún más a medida que los hipervisores adquieran cada vez más funcionalidad y llamadas, según parece. Este tema es controversial, pero cada vez es más claro que el programa que se ejecute en modo de kernel en el hardware básico debe ser pequeño y confiable, y debe consistir de miles de líneas de código, no de millones de líneas. Varios investigadores han hablado sobre este tema (Hand y colaboradores, 2005; Heiser y colaboradores, 2006; Hohmuth y colaboradores, 2004; Roscoe y colaboradores, 2007).

Al paravirtualizar el sistema operativo invitado, surgen varias cuestiones. En primer lugar, si las instrucciones sensibles se sustituyen con llamadas al hipervisor, ¿cómo se puede ejecutar el sistema operativo en el hardware nativo? Después de todo, el hardware no comprende estas llamadas al hipervisor. Y en segundo lugar, ¿qué pasa si hay varios hipervisores disponibles en el mercado como VMware, el Xen de código fuente abierto (diseñado originalmente por la Universidad de Cambridge) y Microsoft Viridian, todos ellos con APIs de hipervisor algo distintas? ¿Cómo se puede modificar el kernel para ejecutarse en todos ellos?

Amsden y sus colaboradores (2006) han propuesto una solución. En su modelo, el kernel se modifica para llamar a ciertos procedimientos especiales cada vez que necesita hacer algo sensible. En conjunto, estos procedimientos conocidos como **VMI** (*Virtual Machina Interface*, Interfaz de máquina virtual) forman una capa de bajo nivel que actúa como interfaz para el hardware o hipervisor. Estos procedimientos están diseñados para ser genéricos y no estar enlazados al hardware o a un hipervisor en especial.

En la figura 8-28 se muestra un ejemplo de esta técnica para una versión paravirtualizada de Linux, conocida como VMI Linux (VMIL). Cuando VMI Linux se ejecuta en el hardware básico, tiene que vincularse con una biblioteca que emite la instrucción actual (sensible) necesaria para realizar el trabajo, como se muestra en la figura 8.28(a). Al ejecutarse en un hipervisor (por ejemplo, en VMware o Xen), el sistema operativo invitado se vincula con distintas bibliotecas que componen las llamadas apropiadas (y distintas) al hipervisor para el hipervisor subyacente. De esta forma, el núcleo del sistema operativo sigue siendo portable, amigable para los hipervisores y eficiente.

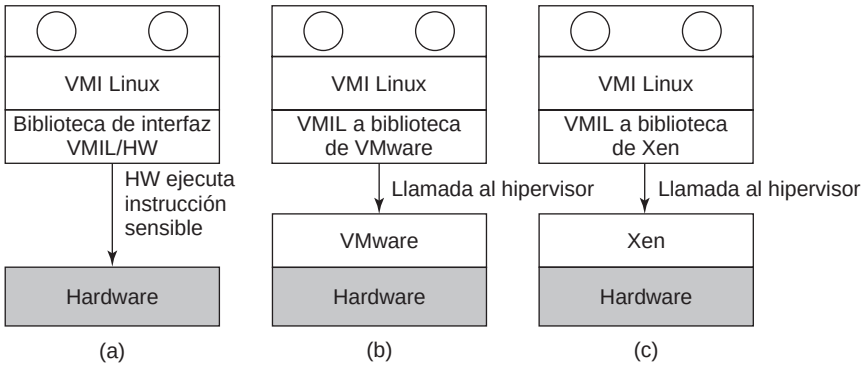


Figura 8-28. VMI Linux ejecutándose en (a) el hardware básico (b) VMware (c) Xen.

También se han propuesto otras técnicas para la interfaz de la máquina virtual. Una de las más populares es **paravirt ops**. Esta idea es similar en concepto a lo que hemos visto antes, pero difiere en algunos detalles.

8.3.5 Virtualización de la memoria

Hasta ahora sólo hemos tratado con la cuestión sobre cómo virtualizar la CPU. Pero un sistema de cómputo tiene más componentes aparte de la CPU. También tiene memoria y dispositivos de E/S. Éstos también se tienen que virtualizar. Veamos ahora cómo se hace esto.

Casi todos los sistemas operativos modernos soportan la memoria virtual, que en esencia es una asignación de las páginas en el espacio de direcciones virtuales a las páginas de la memoria física. Esta asignación se define mediante tablas de páginas (multinivel). Por lo general la asignación se establece en movimiento, al hacer que el sistema operativo establezca un registro de control en la CPU que apunte a la tabla de páginas de nivel superior. La virtualización complica de manera considerable la administración de la memoria.

Por ejemplo, suponga que se está ejecutando una máquina virtual y que el sistema operativo invitado en ella decide asignar sus páginas virtuales 7, 4 y 3 a las páginas físicas 10, 11 y 12, respectivamente. Crea tablas de páginas que contienen esta asignación y carga un registro de hardware para que apunte a la página de tablas de nivel superior. Esta instrucción es sensible. En una CPU con VT, se producirá una interrupción; con VMware producirá una llamada a un procedimiento de VMware; en un sistema operativo paravirtualizado generará una llamada al hipervisor. En aras de la simplicidad, vamos a suponer que se atrapa en un hipervisor de tipo 1, pero el problema es el mismo en los tres casos. ¿Qué hace ahora el hipervisor? Una solución es asignar las páginas físicas 10, 11 y 12 a esta máquina virtual y establecer las páginas de tablas actuales para que asignen las páginas virtuales 7, 4 y 3 de la máquina virtual para utilizarlas. Hasta ahora todo va bien.

Ahora suponga que se inicia una segunda máquina virtual y que asigna sus páginas virtuales 4, 5 y 6 a las páginas físicas 10, 11 y 12, y que carga el registro de control para que apunte a sus tablas de páginas. El hipervisor atrapa la interrupción, pero ¿qué debe hacer? No puede utilizar esta asignación debido a que las páginas físicas 10, 11 y 12 ya están en uso. Puede buscar páginas libres, por ejemplo 20, 21 y 22, y utilizarlas, pero primero tiene que crear nuevas páginas de tablas para asignar los páginas virtuales 4, 5 y 6 de la máquina virtual 2 a las páginas físicas 20, 21 y 22. Si se inicia otra máquina virtual y trata de utilizar las páginas físicas 10, 11 y 12, tiene que crear una asignación para ella. En general, para cada máquina virtual el hipervisor necesita crear una **tabla de páginas oculta** (**shadow**) que asigne las páginas virtuales que utiliza la máquina virtual a las páginas actuales que el hipervisor le otorgó.

Lo que es peor aún, cada vez que el sistema operativo invitado cambie sus tablas de páginas, el hipervisor también tiene que cambiar las tablas de páginas ocultas. Por ejemplo, si el SO invitado reasigna la página virtual 7 a lo que ve como la página virtual 200 (en vez de 10), el hipervisor tiene que saber acerca de este cambio. El problema es que el sistema operativo invitado puede cambiar sus tablas de páginas con sólo escribir en la memoria. No se requieren operaciones sensibles, por lo que el hipervisor ni siquiera se entera sobre el cambio y, en definitiva, no puede actualizar las tablas de páginas ocultas que utiliza el hardware actual.

Una posible solución (pero burda) es que el hipervisor lleve el registro sobre cuál página en la memoria virtual del invitado contiene la tabla de páginas de nivel superior. Puede obtener esta información la primera vez que el invitado intente cargar el registro de software al que apunta, ya que esta instrucción es sensible y produce una interrupción. El hipervisor puede crear una tabla de páginas oculta en este punto, y también puede asignar la tabla de páginas de nivel superior y las tablas de páginas a las que apunta para que sean de sólo lectura. Los intentos subsiguientes del sistema operativo invitado por modificar cualquiera de estas páginas producirán un fallo de página y, en consecuencia, otorgarán el control al hipervisor, que puede analizar el flujo de instrucciones, averiguar qué es lo que está tratando de hacer el SO invitado y actualizar las tablas de páginas ocultas según sea necesario. No es bonito, pero se puede hacer en principio.

Ésta es un área en la que las futuras versiones de VT podrían ofrecer asistencia al realizar una asignación de dos niveles en el hardware. Primero, el hardware podría asignar la página virtual a la idea que tiene el invitado sobre la página física, y después podría asignar esa dirección (que el hardware ve como una dirección virtual) a la dirección física, todo sin que se produzcan interrupciones. De esta forma no habría que marcar ninguna tabla de páginas como de sólo lectura, y el hipervisor simplemente tendría que proveer una asignación entre el espacio de direcciones virtuales de cada invitado y la memoria física. Al cambiar de máquinas virtuales, sólo cambiaría esta asignación de la misma forma en que un sistema operativo normal cambia la asignación al cambiar de procesos.

En un sistema operativo paravirtualizado, la situación es diferente. Aquí, el SO paravirtualizado en el invitado sabe que cuando termine de cambiar la tabla de páginas de cierto proceso, debe informar al hipervisor. En consecuencia, primero cambia la tabla de páginas por completo y después emite una llamada al hipervisor para indicarle sobre la nueva tabla de páginas. Así, en vez de obtener un fallo de protección en cada actualización a la tabla de páginas, hay una llamada al hipervisor cuando se han realizado todas las actualizaciones. Ésta es una manera más eficiente de hacer las cosas.

8.3.6 Virtualización de la E/S

Ya que analizamos la virtualización de la CPU y de la memoria, el siguiente paso es examinar la virtualización de la E/S. Por lo general, el sistema operativo invitado empezará con un sondeo del hardware para averiguar qué tipos de dispositivos de E/S están conectados. Estas sondas producirán interrupciones en el hipervisor. ¿Qué debe hacer el hipervisor? Un método es que reporte de vuelta que los discos, las impresoras y demás dispositivos son los que realmente tiene el hardware. Después el invitado cargará drivers para estos dispositivos y tratará de utilizarlos. Cuando los drivers de los dispositivos traten de realizar operaciones de E/S, leerán y escribirán en los registros de dispositivo del hardware del dispositivo correspondiente. Estas instrucciones son sensibles y se atraparán en el hipervisor, que podría entonces copiar los valores necesarios que entran y salen de los registros de hardware, según sea necesario.

Pero aquí también tenemos un problema. Cada SO invitado piensa que posee toda una partición de disco, y puede haber muchas máquinas virtuales más (cientos) que particiones de disco. La solución usual es que el hipervisor cree un archivo o región en el disco para el disco físico de cada máquina virtual. Como el SO invitado está tratando de controlar un disco que el hardware real tiene (y que el hipervisor conoce), puede convertir el número de bloque al que se está accediendo en un desplazamiento en el archivo o región que se está utilizando para el almacenamiento, y así puede realizar la operación de E/S.

También es posible que el disco que utiliza el invitado sea diferente del disco real. Por ejemplo, si el disco actual es un disco nuevo de alto rendimiento (o RAID) con una nueva interfaz, el hipervisor podría anunciar al SO invitado que tiene un simple disco IDE antiguo, para permitirle que instale un driver de disco IDE. Cuando este driver emite los comandos de disco IDE, el hipervisor los convierte en comandos para controlar el nuevo disco. Esta estrategia se puede utilizar para actualizar el hardware sin tener que cambiar el software. De hecho, esta habilidad de las máquinas virtuales para reasignar dispositivos de hardware fue una de las razones por las que la VM/370 se hizo popular: las empresas querían comprar hardware nuevo y más veloz, pero no querían cambiar su software. Esto fue posible gracias a la tecnología de las máquinas virtuales.

Otro problema de E/S que se debe resolver de alguna manera es el uso de DMA, que utiliza direcciones de memoria absolutas. Como podría esperarse, el hipervisor tiene que intervenir aquí y reasignar las direcciones antes de que inicie el DMA. Sin embargo, está empezando a aparecer el hardware con una **MMU de E/S**, que virtualiza la E/S de la misma forma en que la MMU virtualiza la memoria. Este hardware elimina el problema del DMA.

Un método distinto para manejar la E/S es dedicar una de las máquinas virtuales para que ejecute un sistema operativo estándar y que le refleje todas las llamadas a la E/S de las demás máquinas virtuales. Este método se mejora cuando se utiliza la paravirtualización, por lo que el comando que se emite al hipervisor en realidad indica lo que el SO invitado desea (por ejemplo, leer el bloque 1403 del disco 1), en vez de que sea una serie de comandos para escribir a los registros de dispositivo, en cuyo caso el hipervisor tiene que jugar a ser Sherlock Holmes para averiguar qué es lo que está tratando de hacer. Xen utiliza este método para la E/S, y la máquina virtual que se encarga de la E/S se llama **dominio 0**.

La virtualización de la E/S es un área en la que los hipervisores de tipo 2 tienen una ventaja práctica sobre los hipervisores de tipo 1: el sistema operativo anfitrión contiene los drivers de dis-

positivos para todos los extraños y maravillosos dispositivos de E/S conectados a la computadora. Cuando un programa de aplicación intenta acceder a un dispositivo E/S extraño, el código traducido puede llamar al driver de dispositivo existente para que realice el trabajo. Con un hipervisor de tipo 1, el hipervisor debe contener el driver o debe realizar una llamada a un driver en el dominio 0, que es algo similar a un sistema operativo anfitrión. A medida que madure la tecnología de las máquinas virtuales, es probable que el futuro hardware permita que los programas de aplicación accedan al hardware directamente de una manera segura, lo cual significa que los drivers de dispositivos podrán vincularse directamente con el código de la aplicación, o se podrán colocar en servidores separados en modo de usuario, con lo cual se eliminará el problema.

8.3.7 Dispositivos virtuales

Las máquinas virtuales ofrecen una interesante solución a un problema que desde hace mucho tiempo perturba a los usuarios, en especial a los del software de código fuente abierto: cómo instalar nuevos programas de aplicación. El problema es que muchas aplicaciones dependen de otras tantas aplicaciones y bibliotecas, que a su vez dependen de muchos otros paquetes de software, y así en lo sucesivo. Lo que es más, puede haber dependencias en versiones específicas de los compiladores, lenguajes de secuencias de comandos y en el sistema operativo.

Ahora que están disponibles las máquinas virtuales, un desarrollador de software puede construir con cuidado una máquina virtual, cargarla con el sistema operativo, compiladores, bibliotecas y código de aplicación requeridos, y congelar la unidad completa, lista para ejecutarse. Esta imagen de la máquina virtual se puede colocar en un CD-ROM o en un sitio Web para que los clientes la instalen o descarguen. Este método implica que sólo el desarrollador de software tiene que conocer todas las dependencias. Los clientes reciben un paquete completo que funciona de verdad, sin importar qué sistema operativo estén ejecutando ni qué otro software, paquetes y bibliotecas tengan instalados. A estas máquinas virtuales "empaquetadas y listas para usarse" se les conoce comúnmente como **dispositivos virtuales**.

8.3.8 Máquinas virtuales en CPUs de multinúcleo

La combinación de las máquinas virtuales y las CPUs de multinúcleo abre todo un nuevo mundo, en donde el número de CPUs disponibles se puede establecer en el software. Por ejemplo, si hay cuatro núcleos y cada uno de ellos se puede utilizar para ejecutar, digamos, hasta ocho máquinas virtuales, se puede configurar una sola CPU (escritorio) como una multicomputadora de 32 nodos si es necesario, pero también puede tener menos CPUs dependiendo de las necesidades del software. Nunca antes había sido posible que un diseñador de aplicaciones seleccionara primero cuántas CPUs quiere, para después escribir el software de manera acorde. Sin duda, esto representa una nueva fase en la computación.

Aunque todavía no es tan común, sin duda es concebible que las máquinas virtuales pudieran compartir la memoria. Todo lo que hay que hacer es asignar las páginas físicas en los espacios

direcciones de varias máquinas virtuales. Si se puede hacer esto, una sola computadora se puede convertir en un multiprocesador virtual. Como todos los núcleos en un chip multinúcleo comparten la misma RAM, un solo chip con cuádruple núcleo se podría configurar fácilmente como un multiprocesador de 32 nodos o como una multicomputadora de 32 nodos, según sea necesario.

La combinación de multinúcleo, máquinas virtuales, hipervisores y microkernels va a cambiar de manera radical la forma en que las personas piensan sobre los sistemas de cómputo. El software actual no puede lidiar con la idea de que el programador determine cuántas CPUs se necesitan, si se deben establecer como una multicomputadora o como un multiprocesador, y qué papel desempeñan los kernels mínimos de un tipo u otro. El software futuro tendrá que lidiar con estas cuestiones.

8.3.9 Cuestiones sobre licencias

Las licencias de la mayoría del software son por cada CPU. En otras palabras, cuando usted compra un programa, tiene el derecho de ejecutarlo sólo en una CPU. ¿Acaso este contrato le otorga el derecho de ejecutar el software en varias máquinas virtuales, todas las cuales se ejecutan en la misma máquina? Muchos distribuidores de software están algo inseguros sobre lo que deben hacer aquí.

El problema es mucho peor en las empresas que tienen una licencia que les permite tener *n* máquinas ejecutando el software al mismo tiempo, en especial cuando aumenta y disminuye la demanda de las máquinas virtuales.

En algunos casos, los distribuidores de software han colocado una cláusula explícita en la licencia, en la que se prohíbe al concesionario ejecutar el software en una máquina virtual o en una máquina virtual no autorizada. Está por verse si alguna de estas restricciones será válida en una corte y la manera en que los usuarios responderán a ellas.

8.4 SISTEMAS DISTRIBUIDOS

Ahora que hemos completado nuestro estudio de los multiprocesadores, las multicomputadoras y las máquinas virtuales, es tiempo de analizar el último tipo de sistema de múltiples procesadores: el **sistema distribuido**. Estos sistemas son similares a las multicomputadoras en cuanto a que cada nodo tiene su propia memoria privada, sin memoria física compartida en el sistema. Sin embargo, los sistemas distribuidos tienen un acoplamiento aún más débil que las multicomputadoras.

Para empezar, los nodos de una multicomputadora comúnmente tienen una CPU, RAM, una interfaz de red y tal vez un disco duro para la paginación. Por el contrario, cada nodo en un sistema distribuido es una computadora completa, con todo un complemento de periféricos. Además, los nodos de una multicomputadora por lo general están en un solo cuarto, de manera que se pueden comunicar mediante una red dedicada de alta velocidad, mientras que los nodos de un sistema distribuido pueden estar esparcidos en todo el mundo. Por último, todos los nodos de una computadora ejecutan el mismo sistema operativo, comparten un solo sistema de archivos y están bajo una administración común, mientras que cada uno de los nodos de un sistema distribuido puede ejecutar un sistema operativo diferente, cada uno con su propio sistema de archivos, y pueden estar

bajo una administración diferente. Un ejemplo típico de una multicomputadora es: 512 nodos en un solo cuarto en una empresa o universidad, trabajando (por decir) en el modelado farmacéutico, mientras que un sistema distribuido típico consiste en miles de máquinas que cooperan libremente a través de Internet. En la figura 8-29 se comparan los multiprocesadores, las multicomputadoras y los sistemas distribuidos en los puntos antes mencionados.

Elemento	Multiprocesador	Multicomputadora	Sistema distribuido
Configuración de nodo	CPU	CPU, RAM, interfaz de red	Computadora completa
Periféricos de nodo	Todos compartidos	Compartidos, excepto tal vez el disco	Conjunto completo por nodo
Ubicación	Mismo bastidor	Mismo cuarto	Posiblemente a nivel mundial
Comunicación entre nodos	RAM compartida	Interconexión dedicada	Red tradicional
Sistemas operativos	Uno, compartido	Varios, igual	Posiblemente todos distintos
Sistemas de archivos	Uno, compartido	Uno, compartido	Cada nodo tiene el suyo
Administración	Una organización	Una organización	Muchas organizaciones

Figura 8-29. Comparación de los tres tipos de sistemas de múltiples CPUs.

Es evidente que las multicomputadoras se encuentran en la parte media de esta métrica. Una pregunta interesante sería: "¿Son las multicomputadoras más parecidas a los multiprocesadores o a los sistemas distribuidos?". Aunque parezca mentira, la respuesta depende en gran parte de la perspectiva de cada quien. Desde una perspectiva técnica, los multiprocesadores tienen memoria compartida y los otros dos sistemas no. Esta diferencia produce distintos modelos de programación y diferentes pensamientos. Sin embargo, desde la perspectiva de las aplicaciones, los multiprocesadores y las multicomputadoras son sólo bastidores de equipo en un cuarto de máquinas. Ambos sistemas se utilizan para resolver problemas que requieren cálculos intensivos, mientras que un sistema distribuido que conecta computadoras a través de Internet está por lo general mucho más involucrado en la comunicación que en la computación, y se utiliza de una manera distinta.

Hasta cierto grado, el débilmente acoplamiento de las computadoras en un sistema distribuido es tanto una ventaja como una desventaja. Es una ventaja debido a que las computadoras se pueden utilizar para una gran variedad de aplicaciones, pero también es una desventaja ya que es difícil programar estas aplicaciones debido a que no existe un modelo subyacente común.

Las aplicaciones comunes de Internet incluyen el acceso a computadoras remotas (mediante el uso de *telnet*, *ssh* y *rlogin*), el acceso a información remota (mediante el uso de World Wide Web y FTP, el Protocolo de Transferencia de Archivos), la comunicación de persona a persona (mediante el correo electrónico y los programas de chat) y muchas aplicaciones emergentes (por ejemplo, el comercio electrónico, la telemedicina y el aprendizaje a distancia). El problema con todas estas aplicaciones

es que cada una tiene que reinventar la rueda. Por ejemplo, el correo electrónico, FTP y World Wide Web fundamentalmente desplazan archivos del punto *A* al punto *B*, pero cada aplicación tiene su propia manera de hacerlo, junto con sus propias convenciones de nomenclatura, protocolos de transferencia, técnicas de duplicación y todo lo demás. Aunque muchos navegadores Web ocultan al usuario promedio estas diferencias, los mecanismos subyacentes son completamente distintos. Ocultarlos a nivel de la interfaz de usuario es como hacer que una persona en un sitio Web de una agencia de viajes de servicio completo ordene un viaje de Nueva York a San Francisco, y que no pueda averiguar sino hasta después si compró un boleto de avión, de tren o de autobús.

Lo que agregan los sistemas distribuidos a la red subyacente es cierto paradigma (modelo) común que ofrece una manera uniforme de ver todo el sistema. La intención del sistema distribuido es convertir un grupo de máquinas con una conexión débil en un sistema coherente, con base en un concepto. Algunas veces el paradigma es simple y otras es más elaborado, pero la idea es siempre proveer algo que unifique al sistema.

En UNIX podemos encontrar un ejemplo simple de un paradigma unificador en un contexto ligeramente distinto, en donde todos los dispositivos de E/S se crean de manera que parezcan archivos. Al hacer que los teclados, impresoras y líneas en serie operen de la misma forma, con las mismas primitivas, se facilita el proceso de lidiar con ellos, en vez de que todos sean distintos en concepto.

Una de las formas en que un sistema distribuido puede obtener cierta medida de uniformidad frente a los distintos sistemas operativos y el hardware subyacente es tener un nivel de software encima del sistema operativo. Este nivel, conocido como **middleware**, se ilustra en la figura 8-30. Este nivel provee ciertas estructuras de datos y operaciones que permiten a los procesos y usuarios que están en máquinas remotas interoperar de una manera consistente.

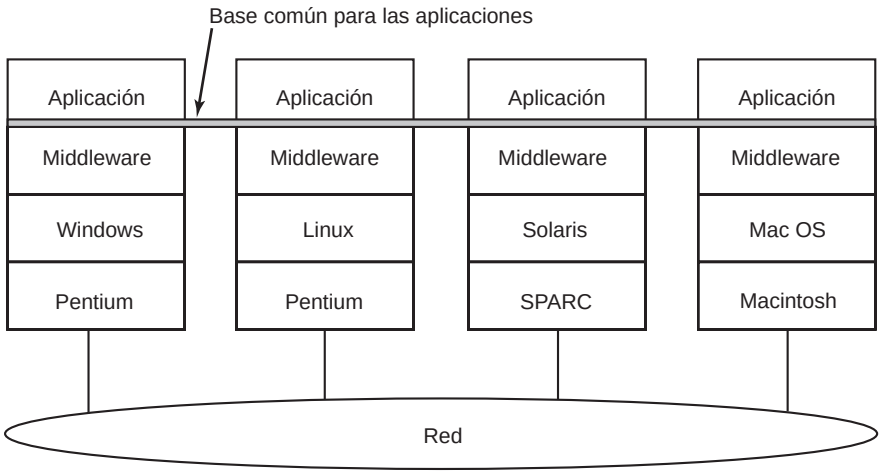


Figura 8-30. Posicionamiento del middleware en un sistema distribuido.

En un sentido, el middleware es como el sistema operativo de un sistema distribuido. Esto explica por qué lo estamos analizando en un libro sobre sistemas operativos. Por otra parte, en realidad *no* es un sistema operativo por lo que no realizaremos un análisis con mucho detalle. Para un tratamiento extenso de los sistemas distribuidos, consulte la obra *Sistemas distribuidos* (Tanenbaum

y van Steen, 2006). En el resto del capítulo analizaremos brevemente el hardware que se utiliza en un sistema distribuido (es decir, la red de computadoras subyacente) y después su software de comunicación (los protocolos de red). Después consideraremos una variedad de paradigmas que se utilizan en estos sistemas.

8.4.1 Hardware de red

Como los sistemas distribuidos se construyen sobre las redes de computadoras, veremos una pequeña introducción a este tema. Las redes se clasifican en dos variedades principales: **LANs** (**Redes de área local**), que cubren un edificio o un campus, y las **WANs** (**Redes de área amplia**), que pueden ser a nivel de ciudad, del país o incluso a nivel mundial. El tipo de LAN más importante es Ethernet, por lo que la examinaremos como un ejemplo de LAN. Analizaremos Internet como un ejemplo de WAN, aun cuando técnicamente Internet no es una red, sino una federación de miles de redes separadas. Sin embargo, para nuestros fines basta con considerarla como una WAN.

Ethernet

La Ethernet clásica, que se describe en el Estándar IEEE 802.3, consiste en un cable coaxial al que se conectan varias computadoras. A este cable se le conoce como **Ethernet**, en relación al *éter luminífero* por medio del cual se pensaba hace tiempo que se propagaba la radiación electromagnética. (Cuando el físico inglés James Clerk Maxwell del siglo diecinueve descubrió que la radiación electromagnética se podía describir mediante una ecuación de onda, los científicos asumieron que este espacio debía llenarse con algún medio etéreo en el que se propagara la radiación. No fue sino hasta después del famoso experimento de Michelson-Morley en 1887, en el que no se pudo detectar el éter, que los físicos se dieron cuenta de que la radiación se podía propagar en un vacío).

En la primera versión de Ethernet, para conectar una computadora al cable literalmente se perforaba un agujero a mitad del cable y se atornillaba un cable que iba a la computadora. A esto se le conocía como ***vampire tap*** (**punción de vampiro**) y se muestra de manera simbólica en la figura 8-31(a). Era difícil hacer las punciones, por lo que se utilizaban conectores largos apropiados para ello. Sin embargo, en relación con la electricidad, todas las computadoras estaban conectadas como si los cables en sus tarjetas de interfaz de red estuvieran soldados.

Para enviar un paquete en una red Ethernet, una computadora primero escucha en el cable para ver si hay otra computadora que esté transmitiendo en ese momento. Si no es así, sólo empieza a transmitir un paquete, el cual consiste en un encabezado corto seguido de una carga útil de 0 a 1500 bytes. Si el cable está en uso, la computadora sólo espera hasta que termine la transmisión actual y después empieza a enviar el paquete.

Si dos computadoras empiezan a transmitir al mismo tiempo se produce una colisión, que ambas detectan. Las dos computadoras responden terminando sus transmisiones, esperan una cantidad aleatoria de tiempo entre 0 y T μseg, y después empiezan de nuevo. Si ocurre otra colisión, todas las computadoras que participan en esa colisión esperan una cantidad aleatoria de tiempo en un intervalo de 0 a $2T$ μseg, y después intentan de nuevo. En cada colisión subsiguiente, el intervalo

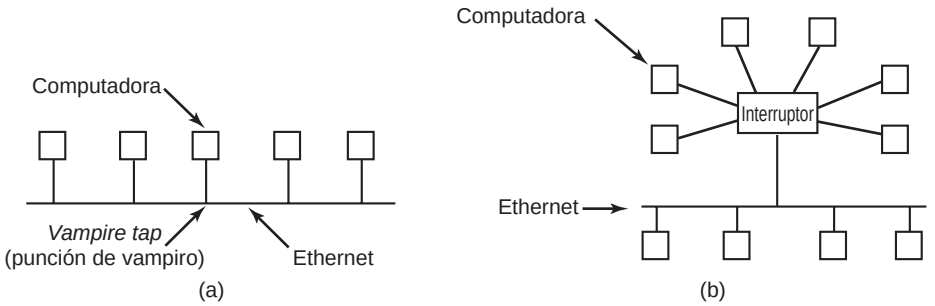


Figura 8-31. (a) Ethernet clásica. (b) Ethernet conmutada.

máximo de espera se duplica, con lo cual se reduce la probabilidad de que haya más colisiones. Este algoritmo se conoce como **backoff exponencial binario**. En secciones anteriores vimos cómo se utiliza este algoritmo para reducir la sobrecarga del sondeo en los bloqueos.

Una red Ethernet tiene una longitud máxima de cable y también un número máximo de computadoras que se pueden conectar. Para exceder cualquiera de estos límites, un edificio o campus grande se puede cablear con varias redes Ethernet, que a su vez se conectan mediante dispositivos conocidos como **puentes** (**bridges**). Un puente permite que el tráfico pase de una red Ethernet a otra cuando el origen está en un lado y el destino en el otro.

Para evitar el problema de las colisiones, las redes Ethernet modernas utilizan switches, como se muestra en la figura 8-31(b). Cada switch tiene cierto número de puertos, a los cuales se puede conectar una computadora, una red Ethernet u otro switch. Cuando un paquete evita con éxito todas las colisiones y llega al switch, se coloca en el búfer y se envía al puerto en el que vive la máquina de destino. Al dar a cada computadora su propio puerto se pueden eliminar todas las colisiones, a expensas de requerir switches más grandes. También son posibles los compromisos con sólo unas cuantas computadoras por puerto. En la figura 8-31(b), una red Ethernet clásica con varias computadoras conectadas a un cable mediante punciones de vampiro está conectada a uno de los puertos del switch.

Internet

Internet evolucionó de ARPANET, una red de conmutación de paquetes experimental, fundada por la Agencia de Proyectos Avanzados de Investigación del Departamento de Defensa de los EE.UU. Se puso en operación en diciembre de 1969, con tres computadoras en California y una en Utah. Se diseñó durante la Guerra Fría como una red con alta tolerancia a las fallas, que continuara transmitiendo el tráfico militar inclusive en caso de que la red sufriera golpes nucleares directos en varias partes, mediante el re-enrutamiento automático del tráfico alrededor de las máquinas inutilizadas.

ARPANET creció con rapidez en la década de 1970, y llegó a tener cientos de computadoras. Después se conectaron una red de radiopaquetes, una red satelital y finalmente miles de redes Ethernet, con lo cual surgió la federación de redes que ahora conocemos como Internet.

Internet consiste en dos tipos de computadoras: hosts y enrutadores. Los **hosts** son PCs, notebooks, dispositivos de bolsillo, servidores, mainframes y otras computadoras que poseen individuos o empresas que se desean conectar a Internet. Los **enrutadores** son computadoras de conmutación especializadas, que aceptan los paquetes entrantes de una de varias líneas entrantes, y los envían a lo largo de muchas líneas salientes. Un enrutador es similar al interruptor de la figura 8-31(b), pero también tiene ciertas diferencias que no son de nuestra incumbencia por el momento. Los enrutadores se conectan entre sí en grandes redes, en donde cada enrutador tiene cables o fibras que van a muchos otros enrutadores y hosts. Las compañías telefónicas y los ISPs (Proveedores de Servicio de Internet) operan extensas redes de enrutadores nacionales o mundiales para sus clientes.

En la figura 8-32 se muestra una porción de Internet. En la parte superior tenemos uno de los backbones, controlado comúnmente por un operador de backbone. Consiste en un número de enrutadores conectados mediante fibra óptica de alta capacidad de ancho de banda, con conexiones a los backbones operados por otras compañías telefónicas (competidoras). Por lo general ningún host se conecta directamente al backbone, con la excepción de las máquinas de mantenimiento y prueba que opera la compañía telefónica.

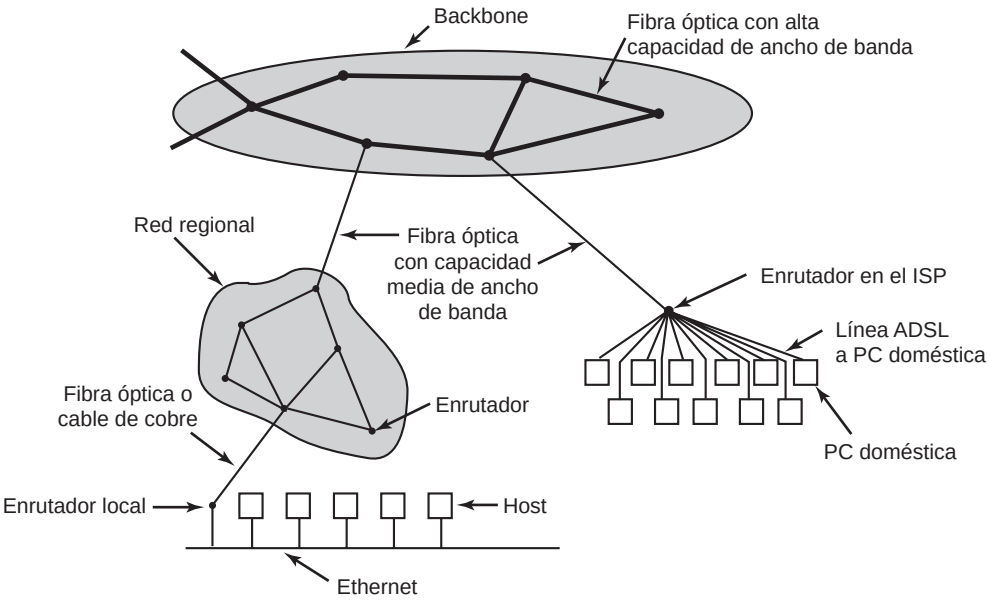


Figura 8-32. Una porción de Internet.

Las redes regionales y los enrutadores en los ISPs se conectan a los enrutadores del backbone mediante fibra óptica de velocidad media. A su vez, cada una de las redes Ethernet corporativas con-

tiene un enrutador, y estos enrutadores se conectan a los enrutadores de las redes regionales. Los enrutadores en los ISPs se conectan a los bancos de módems utilizados por los clientes del ISP. De esta forma, cada host en Internet tiene por lo menos una ruta, y a menudo muchas rutas, hacia cada uno de los otros hosts.

Todo el tráfico en Internet se envía en forma de paquetes. Cada paquete lleva en su interior la dirección de destino, y esta dirección es la que se utiliza para el enrutamiento. Cuando llega un paquete a un enrutador, éste extrae la dirección de destino y la busca (parte de ella) en una tabla para saber en qué línea saliente debe enviar el paquete, y por ende a cuál enrutador. Este procedimiento se repite hasta que el paquete llega al host de destino. Las tablas de enrutamiento son muy dinámicas y se actualizan en forma continua, ya que los enrutadores y los enlaces interrumpen y reanudan su conexión a medida que cambian las condiciones del tráfico.

8.4.2 Protocolos y servicios de red

Todas las redes de computadoras ofrecen ciertos servicios a sus usuarios (hosts y procesos), que implementan mediante el uso de ciertas reglas sobre los intercambios de mensajes legales. A continuación veremos una breve introducción a estos temas.

Servicios de red

Las redes de computadoras proveen servicios a los hosts y procesos que las utilizan. El **servicio orientado a conexión** se modela con base en el sistema telefónico. Para hablar con alguien, usted toma el teléfono, marca el número, habla y después cuelga. De manera similar, para utiliza un servicio de red orientado a conexión, el usuario del servicio establece primero una conexión, la utiliza y después la libera. El aspecto esencial de una conexión es que actúa como un tubo: el emisor mete los objetos (bits) en un extremo y el receptor los saca en el mismo orden del otro extremo.

Por el contrario, el **servicio orientado a no conexión** se modela con base en el sistema postal. Cada mensaje (carta) lleva la dirección de destino completa, y se enruta a través del sistema de manera independiente a los otros mensajes. Por lo general, cuando se envían dos mensajes al mismo destino, el primero que se envíe será el primero en llegar. No obstante, es posible que el primero que se envíe tenga un retraso y el segundo llegue primero. Con un servicio orientado a conexión, esto es imposible.

Cada servicio se puede caracterizar con base en la **calidad del servicio**. Algunos servicios son confiables en cuanto a que nunca pierden datos. Por lo general, para implementar un servicio confiable el receptor debe confirmar la recepción de cada mensaje, para lo cual envía de vuelta un **paquete de reconocimiento** especial, de manera que el emisor esté seguro de que llegó el mensaje. El proceso de reconocimiento introduce sobrecarga y retrasos, los cuales son necesarios para detectar la pérdida de paquetes, pero disminuyen la velocidad de operación.

Una situación común en la que es apropiado un servicio orientado a conexión confiable es la transferencia de un archivo. El propietario del archivo desea estar seguro de que los bits lleguen en forma correcta y en el mismo orden en el que se enviaron. Muy pocos clientes de transferencias de

archivos preferirían un servicio que en ocasiones revolviera o perdiera unos cuantos bits, aunque fuera más rápido.

El servicio orientado a conexión confiable tiene dos variantes menores: secuencias de mensajes y flujos de bytes. En la primera variante, se preservan los límites de los mensajes. Cuando se envían dos mensajes de 1 KB, llegan como dos mensajes distintos de 1 KB cada uno, y nunca como un solo mensaje de 2 KB. En la segunda variante, la conexión es simplemente un flujo de bytes, sin límites en los mensajes. Cuando llegan 2K bytes al receptor, no hay forma de saber si se enviaron como un mensaje de 2 KB, dos mensajes de 1 KB cada uno o 2048 mensajes de 1 byte. Si se envían las páginas de un libro a través de una red a un compositor de imagen como mensajes separados, tal vez sea importante preservar los límites de los mensajes. Por otra parte, cuando una terminal se conecta a un sistema remoto de tiempo compartido, todo lo que se necesita es un flujo de bytes de la terminal a la computadora.

Para algunas aplicaciones, los retrasos que se introducen debido a los reconocimientos son inaceptables. Una de esas aplicaciones es el tráfico de voz. Es preferible que los usuarios de teléfonos escuchen un poco de ruido en la línea o una palabra confusa de vez en cuando, que introducir un retraso para esperar los reconocimientos.

No todas las aplicaciones requieren conexiones. Por ejemplo, para evaluar la red todo lo que se necesita es una manera de enviar un solo paquete que tenga una alta probabilidad de llegar, pero que no haya garantía. A menudo, al servicio sin conexión no confiable (lo que significa que no hay reconocimientos) se le conoce como **servicio de datagramas** debido a su similitud con el servicio de telegramas, que tampoco envía una señal de reconocimiento de vuelta al emisor.

En otras situaciones es conveniente no tener que establecer una conexión para enviar un mensaje corto, pero la confiabilidad es esencial. Para estas aplicaciones se puede proporcionar el **servicio de datagramas con reconocimiento**. Es como enviar una carta registrada y solicitar un acuse de recibo. Cuando el acuse de recibo regrese, el emisor estará absolutamente seguro de que la carta se entregó a la parte interesada y no se perdió en el camino.

Otro de los servicios es el **servicio de solicitud-respuesta**. En este servicio, el emisor transmite un solo datagrama que contiene una solicitud; esta petición contiene la respuesta. Por ejemplo, en esta categoría entra una consulta en la biblioteca local para saber en dónde se habla el idioma Uigur. El servicio de solicitud-respuesta se utiliza comúnmente para implementar la comunicación en el modelo cliente-servidor: el cliente emite una solicitud y el servidor le responde. En la figura 8-33 se sintetizan los tipos de servicios antes descritos.

Protocolos de red

Todas las redes tienen reglas muy especializadas para establecer qué mensajes se pueden enviar y qué respuestas se pueden devolver en relación con estos mensajes. Por ejemplo, bajo ciertas circunstancias (como la transferencia de un archivo), cuando un mensaje se envía de un origen a un destino, se requiere que el destino envíe un reconocimiento de vuelta para indicar que recibió el mensaje correctamente. Bajo otras circunstancias (como la telefonía digital) no se espera dicho reconocimiento. El conjunto de reglas mediante el cual se comunican computadoras específicas se conoce como **protocolo**. Existen muchos protocolos, incluyendo los de enrutador a enrutador, los de

	Servicio	Ejemplo
Orientado a conexión	Flujo de mensajes confiable	Secuencia de páginas de un libro
	Flujo de bytes confiable	Inicio de sesión remoto
	Conexión no confiable	Voz digitalizada
Orientado a no conexión	Datagrama no confiable	Paquetes de prueba de red
	Datagrama reconocido	Correo registrado
	Solicitud-respuesta	Consulta de base de datos

Figura 8-33. Seis tipos distintos de servicio de red.

host a host y varios más. Para un tratamiento detallado de las redes de computadoras y sus protocolos, consulte la obra *Computer Networks* (Tanenbaum, 2003).

Todas las redes modernas utilizan una **pila de protocolos** para distribuir los distintos protocolos en niveles, uno encima de otro. En cada nivel se lidia con distintas cuestiones. Por ejemplo, en el nivel inferior los protocolos definen la forma de saber en dónde empieza y termina un paquete en el flujo de bits. En un nivel más alto, los protocolos definen cómo enrutar los paquetes a través de redes complejas, del origen al destino. Y a un nivel todavía más alto, se aseguran de que todos los paquetes en un mensaje compuesto por varios paquetes hayan llegado en forma correcta y en el orden apropiado.

Como la mayoría de los sistemas distribuidos utilizan Internet como base, los protocolos base que utilizan estos sistemas son los dos protocolos principales de Internet: IP y TCP. **IP** (**Protocolo de Internet**) es un protocolo de datagramas, en el cual un emisor inyecta un datagrama de hasta 64 KB en la red, y espera que llegue. No se dan garantías. El datagrama se puede fragmentar en paquetes más pequeños a medida que pasa por Internet. Estos paquetes viajan de manera independiente, tal vez a lo largo de distintas rutas. Cuando todas las piezas llegan a su destino, se ensamblan en el orden correcto y se entregan.

En la actualidad se utilizan dos versiones de IP: v4 y v6. Aquí vamos a analizar la versión v4, que aún domina el mercado, pero v6 está ganando terreno. Cada paquete v4 empieza con un encabezado de 40 bytes que contiene una dirección de origen de 32 bits y una dirección de destino de 32 bits, entre otros campos. A éstas se les conoce como **direcciones IP** y forman la base del enrutamiento en Internet. Por convención se escriben como cuatro números decimales en el rango de 0 a 255 separados por puntos, como en 192.31.231.65. Cuando un paquete llega a un enrutador, éste extrae la dirección IP de destino y la utiliza para enrutar el paquete.

Como los datagramas IP no utilizan señales de reconocimiento, no es suficiente el protocolo IP para una comunicación confiable en Internet. Para proveer una comunicación confiable, por lo general se utiliza otro protocolo conocido como **TCP** (**Protocolo de control de transmisión**) que se coloca en un nivel encima de IP. TCP utiliza a IP para proveer flujos orientados a conexión. Para utilizar TCP, un proceso establece primero una conexión a un proceso remoto. El proceso requerido se especifica mediante la dirección IP de una máquina y un número de puerto en esa máquina,

en el que escuchan los procesos interesados en recibir conexiones entrantes. Una vez que se ha realizado esto, sólo se inyectan bytes en la conexión y se garantiza que saldrán por el otro extremo sin daños y en el orden correcto. Para lograr esta garantía, la implementación de TCP utiliza números de secuencia, sumas de comprobación y retransmisiones de los paquetes que se reciben en forma incorrecta. Todo esto es transparente para los procesos emisor y receptor. Éstos sólo ven una comunicación confiable entre procesos, justo igual que una canalización en UNIX.

Para ver cómo interactúan todos estos protocolos, considere el caso más simple de un mensaje muy pequeño que no se necesita fragmentar en ningún nivel. El host se encuentra en una red Ethernet conectada a Internet. ¿Qué ocurre con exactitud? El proceso de usuario genera el mensaje y realiza una llamada al sistema para enviarlo en una conexión TCP que se haya establecido de antemano. La pila de protocolos del kernel agrega un encabezado TCP y después un encabezado IP en la parte frontal. Después pasa al driver de Ethernet, que agrega un encabezado Ethernet para dirigir al paquete al enrutador en Ethernet. Después este enrutador inyecta el paquete en Internet, como se ilustra en la figura 8-34.

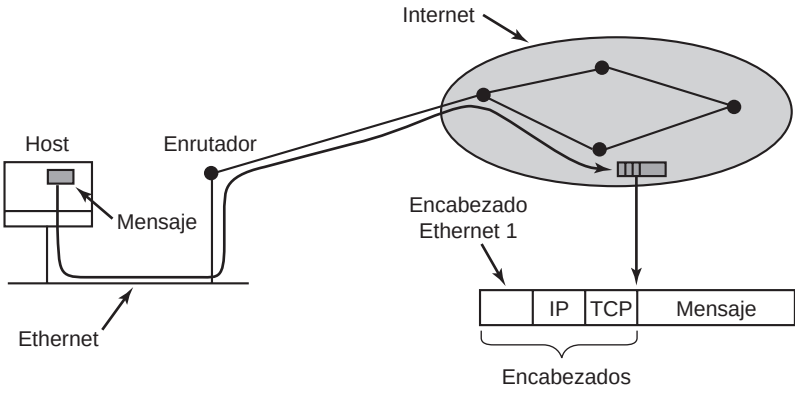


Figura 8-34. Acumulación de encabezados de paquetes.

Para establecer una conexión con un host remoto (o incluso para enviarle un datagrama) es necesario conocer su dirección IP. Como es inconveniente para las personas administrar listas de direcciones IP de 32 bits, se inventó un esquema llamado **DNS** (**Sistema de nombres de dominio**) como una base de datos para asignar nombres ASCII para los hosts a sus direcciones IP. Por ende, es posible utilizar el nombre DNS *star.cs.vu.nl* en vez de la correspondiente dirección IP 130.37.24.6. Los nombres DNS son muy conocidos debido a que las direcciones de correo electrónico en Internet son de la forma *nombre-usuario@nombre-host-DNS*. Este sistema de nomenclatura permite que el programa de correo en el host emisor busque la dirección IP del host de destino en la base de datos DNS, que establezca una conexión TCP con el proceso demonio de correo ahí y envíe el mensaje como un archivo. El *nombre-usuario* se envía también para identificar en qué bandeja de correo se debe colocar el mensaje.

8.4.3 Middleware basado en documentos

Ahora que tenemos ciertos antecedentes sobre las redes y los protocolos, podemos empezar a analizar los distintos niveles de middleware que se pueden superponer en la red básica para producir un paradigma constante para las aplicaciones y los usuarios. Empezaremos con un ejemplo simple, pero muy conocido: World Wide Web. Tim Berners-Lee inventó el servicio Web en 1989 en CERN, el Centro Europeo de Investigación de Física Nuclear, y desde entonces se ha esparcido como fuego arrasador por todo el mundo.

El paradigma original detrás de Web era muy simple: cada computadora podía contener uno o más documentos, llamados **páginas Web**. Cada página Web contiene texto, imágenes, iconos, sonidos, películas y demás, así como **hipervínculos** (apuntadores) a otras páginas Web. Cuando un usuario solicita una página Web mediante el uso de un programa llamado **navegador Web**, la página se muestra en la pantalla. Al hacer clic en un vínculo, la página actual se sustituye en la pantalla por la página a la que apunta el vínculo. Aunque en años recientes se han agregado muchos complementos al Web, es evidente que el paradigma subyacente aún sigue presente: Web es un gran grafo dirigido de documentos que pueden apuntar a otros documentos, como se muestra en la figura 8-35.

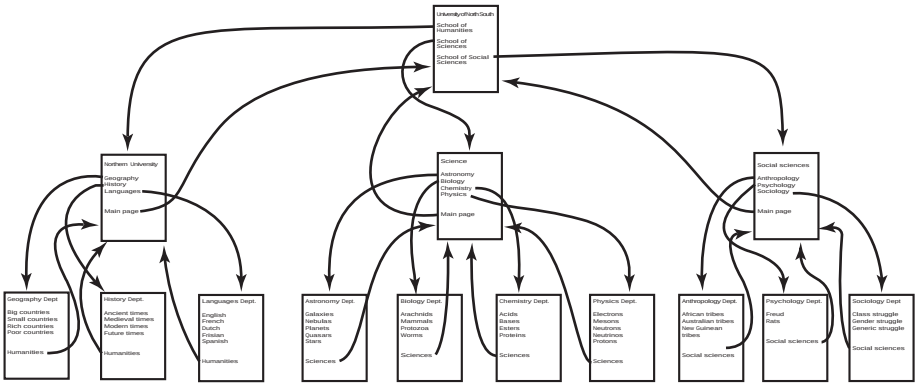


Figura 8-35. Web es un gran grafo dirigido de documentos.

Cada página Web tiene una dirección única, conocida como **URL** (*Uniform Resource Lovator,* Localizador uniforme de recursos), de la forma *protocolo://nombre-DNS/nombre-archivo*. El protocolo más común es *http* (Protocolo de transferencia de hipertexto), pero también existen otros protocolos como *ftp*. Después se utiliza el nombre DNS del host que contiene el archivo. Por último, hay un nombre de archivo local que indica cuál es el archivo que se necesita.

A continuación veremos la forma en que funciona el sistema completo. En esencia, Web es un sistema cliente-servidor en donde el usuario es el cliente y el sitio Web es el servidor. Cuando el usuario proporciona un URL al navegador, ya sea escribiéndolo o haciendo clic en un hipervínculo en la página actual, el navegador realiza ciertos pasos para obtener la página Web solicitada. Como

un ejemplo simple, suponga que el URL que se proporciona es *http://www.minix3.org/doc/faq.html*. Después, el navegador realiza los siguientes pasos para obtener la página:

1. El navegador pide al DNS la dirección IP de *www.minix3.org*.
2. DNS responde con 130.37.20.20.
3. El navegador realiza una conexión TCP al puerto 80 en 130.37.20.20.
4. Después envía una petición por el archivo *doc/faq.html*.
5. El servidor *www.acm.org* envía el archivo *doc/faq.html*.
6. Se libera la conexión TCP.
7. El navegador muestra todo el texto en *doc/faq.html*.
8. El navegador obtiene y muestra todas las imágenes en *doc/faq.html*.

En principio, ésta es la base de Web y la forma en que funciona. Desde entonces se han agregado muchas otras características al Web básico, incluyendo hojas de estilo en cascada, páginas Web dinámicas que se generan al instante, páginas Web que contienen pequeños programas o secuencias de comandos que se ejecutan en la máquina cliente y mucho más, pero esto queda fuera del alcance de este análisis.

8.4.4 Middleware basado en sistemas de archivos

La idea básica detrás de la Web es hacer que un sistema distribuido tenga la apariencia de una colección gigante de documentos hipervinculados. Un segundo método sería hacer que un sistema distribuido tenga la apariencia de un gran sistema de archivos. En esta sección analizaremos algunas de las cuestiones involucradas en el diseño de un sistema de archivos a nivel mundial.

Utilizar un modelo de sistema de archivos para un sistema distribuido significa que hay un solo sistema de archivos global, en el que usuarios de todo el mundo tienen autorización de leer y escribir en archivos. Para lograr la comunicación, un proceso tiene que escribir datos en un archivo y otros procesos tienen que leer los datos de vuelta. Aquí surgen muchos de los problemas estándar con los sistemas de archivos, pero también aparecen nuevos problemas relacionados con la distribución.

Modelo de transferencia

La primera cuestión es la elección entre el **modelo de envío/descarga** y el **modelo de acceso remoto**. En el primer modelo, que se muestra en la figura 8-36(a), para que un proceso pueda utilizar un archivo debe copiarlo primero del servidor remoto en el que se encuentra. Si el archivo sólo se va a leer, se lee entonces de manera local, para obtener un alto rendimiento. Si el archivo se va a escribir, se escribe de manera local. Cuando el proceso termina de usar el archivo, coloca el archivo actualizado de vuelta en el servidor. Con el modelo de acceso remoto, el archivo permanece en el servidor y el cliente le envía comandos para realizar el trabajo ahí, como se muestra en la figura 8-36(b).

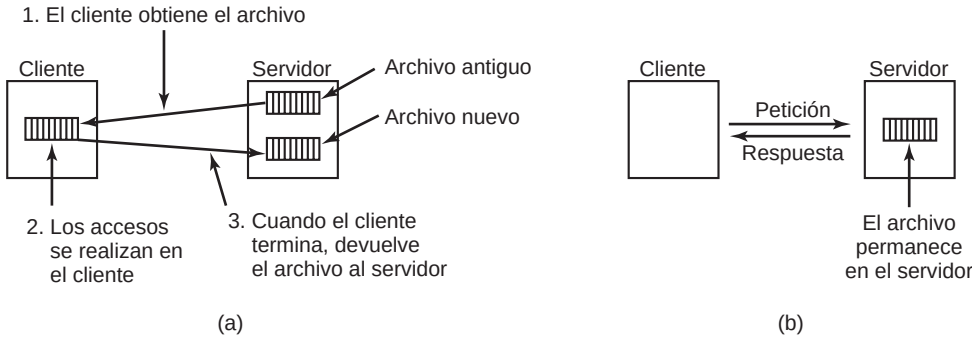


Figura 8-36. (a) El modelo de envío/descarga. (b) El modelo de acceso remoto.

Las ventajas del modelo de envío/descarga son su simplicidad y el hecho de que es más eficiente transferir todos los datos entre archivos a la vez que transferirlos en piezas pequeñas. Las desventajas son que debe haber suficiente espacio de almacenamiento para todo el archivo en forma local, desplazar el archivo completo es un desperdicio si sólo se necesitan piezas de éste, y surgen problemas de consistencia si hay varios usuarios concurrentes.

La jerarquía de directorios

Los archivos sólo son parte de la historia; la otra parte es el sistema de directorios. Todos los sistemas de archivos distribuidos aceptan directorios que contienen varios archivos. El siguiente problema de diseño es si todos los clientes tienen la misma vista de la jerarquía de directorios. Como ejemplo de lo que queremos, considere la figura 8-37. En la figura 8-37(a) mostramos dos servidores de archivos, cada uno de los cuales contiene tres directorios y ciertos archivos. En la figura 8-37(b) tenemos un sistema en el que todos los clientes (y otras máquinas) tienen la misma vista del sistema de archivos distribuido. Si la ruta */D/E/x* es válida en una máquina, es válida en todos ellos.

En contraste, en la figura 8-37(c) hay distintas máquinas que pueden tener distintas vistas del sistema de archivos. Para repetir el ejemplo anterior, la ruta */D/E/x* podría ser válida en el cliente 1, pero no en el cliente 2. En los sistemas que administran varios sistemas de archivos mediante el montaje remoto, la figura 8-37(c) es la norma. Es flexible y simple de implementar, pero tiene la desventaja de que no hace que todo el sistema se comporte como un sistema individual de tiempo compartido a la antigua. En un sistema de tiempo compartido todos los procesos ven el mismo sistema de archivos, como en el modelo de la figura 8-37(b). Esta propiedad facilita la programación y comprensión del sistema.

Una pregunta muy relacionada es si debe haber o no un directorio raíz global, que todas las máquinas reconozcan como el directorio raíz. Una manera de tener un directorio raíz global es hacer que la raíz contenga una entrada para cada servidor y nada más. Bajo estas circunstancias, las rutas toman la forma */servidor/ruta*, lo cual tiene sus propias desventajas, pero al menos es lo mismo en cualquier parte del sistema.

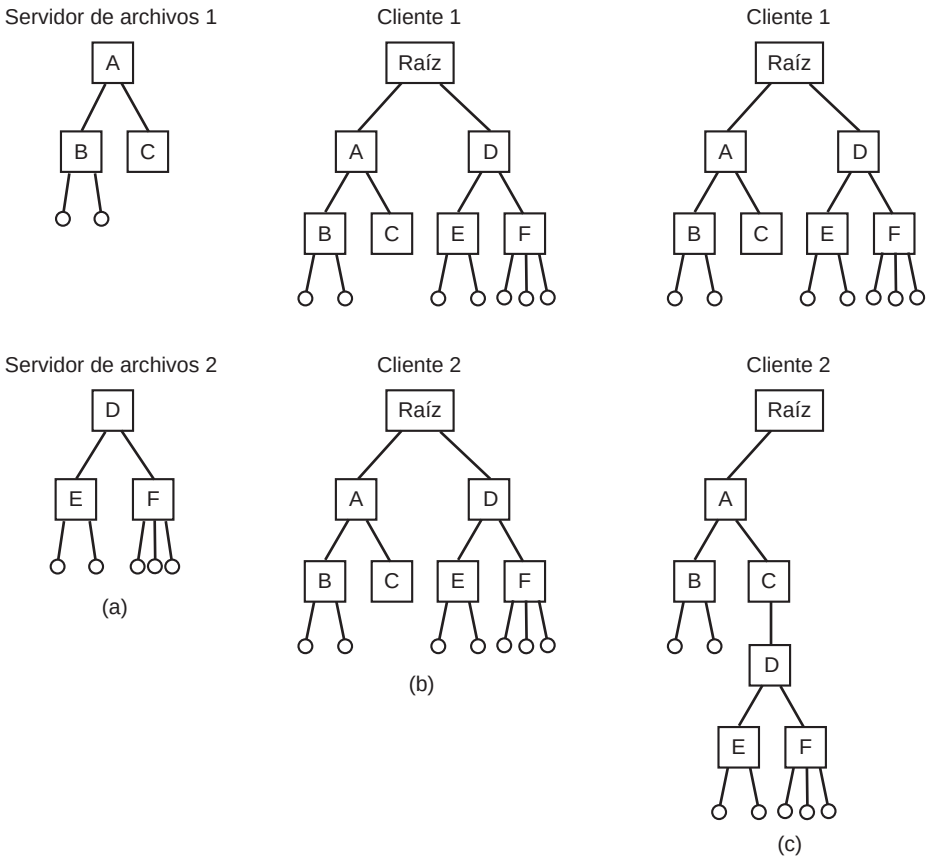


Figura 8-37. (a) Dos servidores de archivos. Los cuadros son directorios y los círculos son archivos. (b) Un sistema en el que todos los clientes tienen la misma vista del sistema de archivos. (c) Un sistema en el que distintos clientes pueden tener distintas vistas del sistema de archivos.

Transparencia de nomenclatura

El principal problema con esta forma de nomenclatura es que no es totalmente transparente. Hay dos formas de transparencia relevantes en este contexto que vale la pena distinguir. En la primera, conocida como **transparencia de localización**, el nombre de ruta no da ninguna pista sobre la ubicación del archivo. Una ruta como */servidor1/dir1/dir2/x* indica que el archivo *x* está ubicado en el servidor 1, pero no indica en dónde se encuentra ese servidor. El servidor se puede mover a cualquier parte en la red sin tener que cambiar el nombre de la ruta. Por ende, este sistema tiene transparencia de ubicación.

Ahora suponga que el archivo *x* es muy grande y que el espacio está restringido en el servidor 1. Además, suponga que hay mucho espacio en el servidor 2. El sistema podría mover el archivo *x* al

servidor 2 de manera automática. Por desgracia, cuando el primer componente de todos los nombres de ruta es el servidor, el sistema no puede mover el archivo al otro servidor de manera automática, aun si *dir1* y *dir2* existen en ambos servidores. El problema es que al mover el archivo de manera automática se cambia su nombre de ruta de */servidor1/dir1/dir2/x* a */servidor2/dir1/dir2/x*. Los programas que contienen la primera cadena dejarán de funcionar si cambia la ruta. Se dice que un sistema en el que se pueden mover archivos sin cambiar sus nombres tiene **independencia de ubicación**. Un sistema distribuido que incrusta los nombres de máquinas o de servidores en los nombres de las rutas en definitiva no es independiente de la ubicación. Un sistema basado en el montaje remoto tampoco lo es, ya que no es posible mover un archivo de un grupo de archivos (la unidad de montaje) a otro y poder seguir utilizando el nombre de ruta anterior. No es fácil lograr la independencia de la ubicación, pero es una propiedad conveniente de tener en un sistema distribuido.

Para resumir lo que dijimos antes, hay tres métodos comunes para nombrar archivos y directorios en un sistema distribuido:

1. Usar la nomenclatura de máquina + ruta, como */máquina/ruta* o *máquina:ruta*.
2. Montar los sistemas de archivos remotos en la jerarquía de archivos local.
3. Un solo espacio de nombres que sea igual para todas las máquinas.

Los primeros dos métodos son fáciles de implementar, en especial como una forma de conectar los sistemas existentes que no fueron diseñados para un uso distribuido. El tercer método es difícil y requiere un diseño cuidadoso, pero facilita la vida a los programadores y usuarios.

Semántica de la compartición de archivos

Cuando dos o más usuarios comparten el mismo archivo, es necesario definir la semántica de leer y escribir con precisión para evitar problemas. En los sistemas con un solo procesador, la semántica por lo general establece que cuando una llamada al sistema `read` va después de una llamada al sistema `write`, la llamada a `read` devuelve el valor que se acaba de escribir, como se muestra en la figura 8-38(a). De manera similar, cuando se realizan dos llamadas a `write` en sucesión rápida, seguidas de una llamada a `read`, el valor leído es el valor almacenado por la última escritura. En efecto, el sistema implementa un orden en todas las llamadas al sistema, y todos los procesadores ven el mismo orden. A este modelo le llamaremos **consistencia secuencial**.

En un sistema distribuido, la consistencia secuencial se puede obtener fácilmente mientras haya sólo un servidor de archivos y los clientes no coloquen archivos en la caché. Todas las llamadas a `read` y `write` van directamente al servidor de archivos, que las procesa estrictamente en forma secuencial.

Sin embargo, en la práctica el rendimiento de un sistema distribuido en el que todas las peticiones de archivos deben enviarse a un solo servidor es, con frecuencia, pobre. A menudo, este problema se resuelve al permitir que los clientes mantengan copias locales de archivos de uso frecuente en sus cachés privadas. No obstante, si el cliente 1 modifica de manera local un archivo en caché y, poco tiempo después, el cliente 2 lee el archivo del servidor, el segundo cliente obtendrá un archivo obsoleto, como se ilustra en la figura 8-38(b).

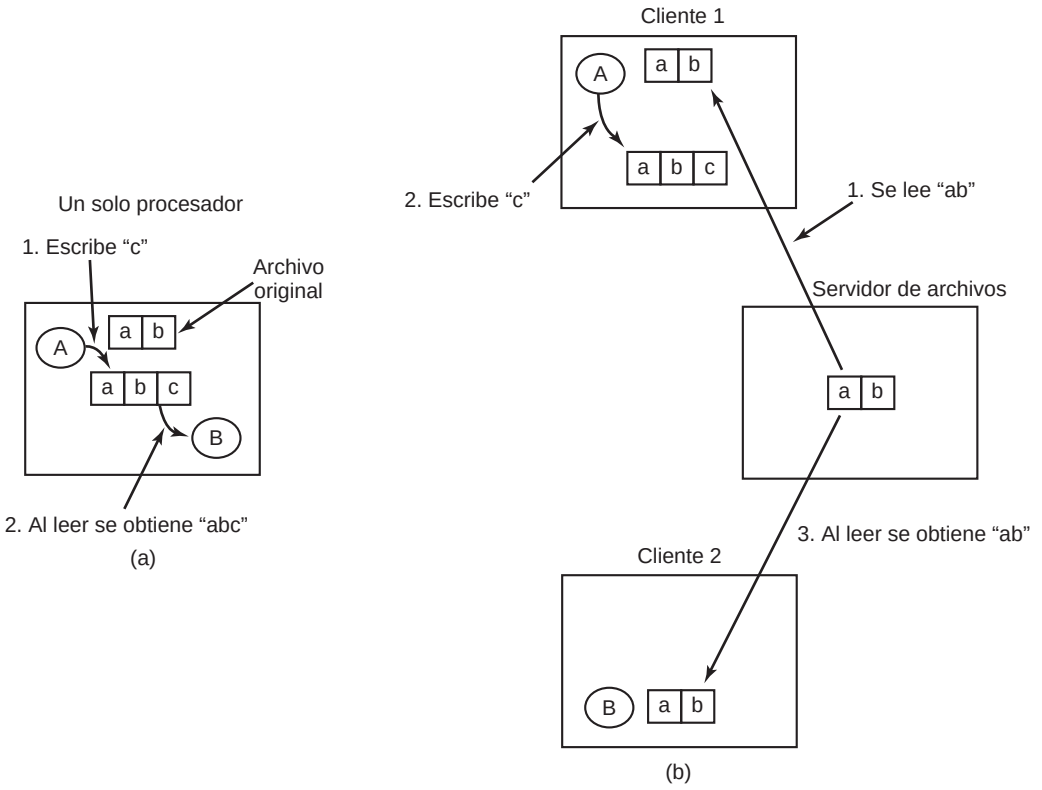


Figura 8-38. (a) Consistencia secuencial. (b) En un sistema distribuido con caché, al leer un archivo se puede obtener un valor obsoleto.

Una manera de solucionar esta dificultad es propagar todos los cambios a los archivos en caché de vuelta al servidor de manera inmediata. Aunque este método es simple en concepto, es ineficiente. Una solución alternativa es relajar la semántica de la compartición de archivos. En vez de requerir una llamada a `read` para ver los efectos de todas las llamadas anteriores a `write`, podemos tener una nueva regla que establezca lo siguiente: “Los cambios a un archivo abierto son visibles en un principio sólo para el proceso que los hizo. Sólo hasta que se cierre el archivo podrán ser visibles los cambios para los demás procesos”. La adopción de dicha regla no cambia lo que ocurre en la figura 8-38(b), pero sí redefine el comportamiento actual (*B* recibe el valor original del archivo) como correcto. Cuando el cliente 1 cierra el archivo, envía una copia de vuelta al servidor para que las llamadas subsiguientes a `read` obtengan el nuevo valor, como se requiere. En efecto, éste es el modelo de envío/descarga de la figura 8-36. Esta regla semántica se implementa ampliamente, y se conoce como **semántica de sesión**.

Al utilizar la semántica de sesión surge la cuestión de lo que ocurre si dos o más clientes están usando caché y modificando el mismo archivo a la vez. Una solución es que a medida que se vaya cerrando cada archivo, su valor se envíe de vuelta al servidor, por lo que el resultado final dependerá

de cuál archivo cierre al último. Una alternativa menos placentera pero un poco más fácil de implementar es que el resultado final sea uno de los candidatos, pero dejar sin especificar la opción de cuál archivo se debe elegir.

Un método alternativo a la semántica de sesión es utilizar el modelo de envío/descarga, pero bloquear de manera automática un archivo que se haya descargado. Cuando otros clientes intenten descargar el archivo, se quedarán en espera hasta que el primer cliente lo haya regresado. Si hay mucha demanda por el archivo, el servidor podría enviar mensajes al cliente que contiene el archivo para pedirle que se apure, y tal vez eso ayude o tal vez no. En general, es difícil obtener la semántica correcta de los archivos compartidos sin soluciones elegantes y eficientes.

8.4.5 Middleware basado en objetos

Ahora vamos a analizar un tercer paradigma. En vez de decir que todo es un documento o que todo es un archivo, decimos que todo es un objeto. Un **objeto** es una colección de variables que se incluyen con un conjunto de procedimientos de acceso, llamados **métodos**. No se permite que los procesos accedan directamente a las variables. En vez de ello, tienen que invocar los métodos.

Algunos lenguajes de programación como C++ y Java son orientados a objetos, pero éstos son objetos a nivel de lenguaje en vez de objetos en tiempo de ejecución. Un sistema muy conocido que se basa en objetos en tiempo de ejecución es **CORBA** (*Common Object Request Broker Architechture*, Arquitectura común de intermediarios en peticiones a objetos) (Vinoski, 1997). CORBA es un sistema cliente-servidor, en el que los procesos cliente en máquinas cliente pueden invocar operaciones en objetos ubicados en máquinas servidores (posiblemente remotos). CORBA se diseñó para un sistema heterogéneo que ejecuta una variedad de plataformas de hardware y sistema operativos, programado en una variedad de lenguajes. Para que sea posible que un cliente en una plataforma invo que a un servidor en una plataforma distinta, se interponen **ORB**s *Object Request Broker* (Intermediarios en peticiones a objetos) entre el cliente y el servidor para permitir que coincidan. Los ORBs desempeñan un papel tan importante en CORBA que hasta incluyen su nombre en el sistema.

Cada objeto CORBA se define mediante una definición de interfaz en un lenguaje conocido como **IDL** (*Interfaz Definition Language*, Lenguaje de definición de interfaz), el cual indica qué métodos debe exportar el objeto y qué tipos de parámetros espera cada método. La especificación del IDL se puede compilar en un procedimiento de resguardo del cliente y se puede almacenar en una biblioteca. Si un proceso cliente sabe de antemano que necesitará acceder a cierto objeto, se vincula con el código de resguardo del cliente del objeto. La especificación IDL también se puede compilar en un procedimiento **esqueleto** que se utiliza en el lado servidor. Si un proceso no sabe de antemano cuáles objetos CORBA va a necesitar también es posible la invocación dinámica, pero su funcionamiento es algo que está más allá del alcance de este libro.

Cuando se crea un objeto CORBA también se crea una referencia a este objeto y se devuelve al proceso que lo creó. Esta referencia es la forma en que el proceso identifica al objeto para las subsiguientes invocaciones de sus métodos. La referencia se puede pasar a otros procesos, o se puede almacenar en un directorio de objetos.

Para que un proceso cliente pueda invocar a un método sobre un objeto, debe adquirir primero una referencia al objeto. La referencia puede provenir directamente del proceso que lo creó, pero es

más probable que los busque por nombre o por función en algún tipo de directorio. Una vez que está disponible la referencia al objeto, el proceso cliente organiza los parámetros para las llamadas a los métodos en una estructura conveniente y después se pone en contacto con el ORB cliente. A su vez, el ORB cliente envía un mensaje al ORB servidor, el cual invoca al método del objeto. Todo el mecanismo es similar a RPC.

La función de los ORBs es ocultar al código cliente y servidor todos los detalles de distribución y comunicación de bajo nivel. En especial, los ORBs ocultan al cliente la ubicación del servidor, si éste es un programa binario o una secuencia de comandos, en qué hardware y sistema operativo se ejecuta el servidor, si el objeto está activo en ese momento y cómo se comunican los dos ORBs (por ejemplo, TCP/IP, RPC, memoria compartida, etcétera.).

En la primera versión de CORBA, no se especificaba el protocolo entre el ORB cliente y el ORB servidor. Como resultado, cada distribuidor de ORBs utilizaba un protocolo distinto y los ORBs de un distribuidor no se podían comunicar con los de los otros distribuidores. En la versión 2.0 se especificó el protocolo. Para comunicarse a través de Internet, el protocolo se llama **IIOP** (*Internet InterOrb Protocol*, Protocolo InterOrb de Internet).

Para que sea posible utilizar en los sistemas CORBA objetos que no se hayan escrito para CORBA, cada objeto se puede equipar con un **adaptador de objeto**. Esto es una envoltura que maneja las tareas como el registro del objeto, la generación de referencias al objeto y su activación, si se invoca cuando no esté activo. La distribución de todas estas partes de CORBA se muestra en la figura 8-39.

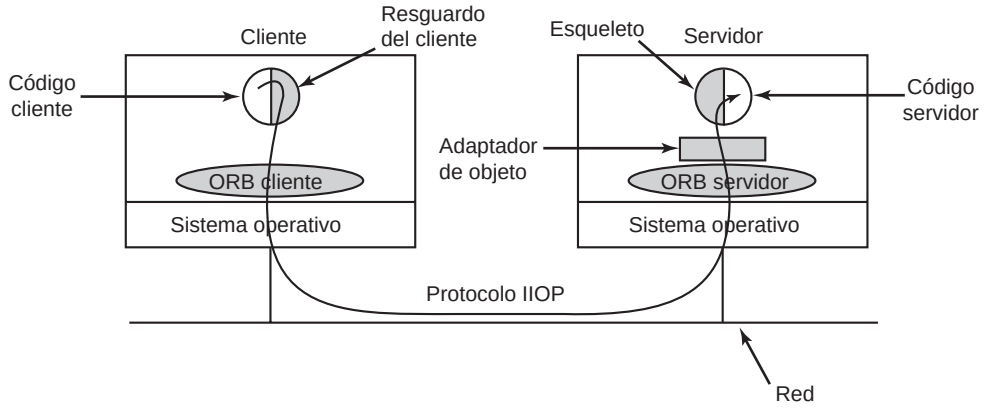


Figura 8-39. Los principales elementos de un sistema distribuido basado en CORBA. Las partes de CORBA se muestran en color gris.

Un problema grave con CORBA es que cada objeto se encuentra sólo en un servidor, lo cual significa que el rendimiento será terrible para los objetos que se utilizan con mucha frecuencia en máquinas cliente por todo el mundo. En la práctica, CORBA sólo funciona de manera aceptable en los sistemas de pequeña escala, como los que se utilizan para conectar procesos en una computadora, una LAN o dentro de una sola empresa.

8.4.6 Middleware basado en coordinación

Nuestro último paradigma para un sistema distribuido se llama **middleware basado en coordinación**. Empezaremos con el sistema Linda, un proyecto académico de investigación que inició todo el campo, y después analizaremos dos ejemplos comerciales que se inspiraron en gran parte en el sistema Linda: publicar/suscribir y Jini.

Linda

David Gelernter y su estudiante Nick Carriero (Carriero y Gelernter, 1986; Carriero y Gelernter, 1989; Gelernter, 1985) desarrollaron en la Universidad de Yale el sistema **Linda**, el cual es un sistema novel para comunicación y sincronización. En Linda, los procesos independientes se comunican mediante un **espacio de tuplas** abstracto. El espacio de tuplas es global para todo el sistema, y los procesos en cualquier máquina pueden insertar o quitar tuplas en el espacio de tuplas sin importar en dónde estén almacenadas. Para el usuario, el espacio de tuplas parece una gran memoria compartida global, como hemos visto en varias formas anteriores, como en la figura 8-21(c).

Una **tupla** es como una estructura en C o Java. Consiste en uno o más campos, cada uno de los cuales es un valor de cierto tipo, soportado por el lenguaje base (para implementar a Linda se agrega una biblioteca a un lenguaje existente, como C). Para C-Linda, los tipos de campos pueden ser enteros, enteros largos y números de punto flotante, y también tipos compuestos como arreglos (incluyendo cadenas) y estructuras (pero no otras tuplas). A diferencia de los objetos, las tuplas son sólo datos; no tienen métodos asociados. En la figura 8-40 se muestran tres ejemplos de tuplas.

```
("abc",2,5)
("matriz-1",1, 6, 3.14)
("familia", "es-hermana", "Stephany", "Roberta")
```

Figura 8-40. Tres tuplas en Linda.

Las tuplas pueden tener cuatro operaciones. La primera (*out*) coloca una tupla en el espacio de tuplas. Por ejemplo,

```
out("abc", 2,5);
```

coloca la tupla ("abc", 2, 5) en el espacio de tuplas. Por lo general, los campos de *out* son constantes, variables o expresiones, como en

```
out("matriz-1", i, j, 3.14);
```

que envía una tupla con cuatro campos, de los cuales el segundo y el tercero se determinan con base en los valores actuales de las variables *i* y *j*.

Para obtener tuplas del espacio de tuplas se utiliza la primitiva *in*. Se tratan por contenido y no por nombre o dirección. Los campos de *in* pueden ser expresiones de parámetros formales. Por ejemplo, considere

```
in("abc", 2, ?i);
```

Esta operación "busca" en el espacio de tuplas una tupla que consista en la cadena "abc", el entero 2 y un tercer campo que contenga cualquier entero (suponiendo que *i* sea un entero). Si la encuentra, la tupla se elimina del espacio de tuplas y a la variable *i* se le asigna el valor del tercer campo. Las operaciones de buscar la coincidencia y eliminarla son atómicas, por lo que si dos procesos ejecutan la misma operación *in* al mismo tiempo, sólo uno de ellos tendrá éxito a menos que haya dos o más tuplas que coincidan. Inclusive, el espacio de tuplas puede contener varias copias de la misma tupla.

El algoritmo para buscar coincidencias que utiliza *in* es simple. Los campos de la primitiva *in*, que se conocen como la **plantilla**, se comparan (en concepto) con los campos correspondientes de todas las tuplas en el espacio de tuplas. Se produce una coincidencia si se cumplen las siguientes tres condiciones:

1. La plantilla y la tupla tienen el mismo número de campos.
2. Los tipos de los campos correspondientes son iguales.
3. Cada constante o variable en la plantilla coincide con el campo de su tupla.

Los parámetros formales, que se indican mediante un signo de interrogación seguido del nombre o tipo de una variable, no participan en la búsqueda de coincidencias (excepto para la comprobación de los tipos), aunque los parámetros que contienen el nombre de una variable se asignan después de una coincidencia exitosa.

Si no hay una tupla que coincida, se suspende el proceso que hizo la llamada hasta que otro proceso inserte la tupla necesaria, y en ese momento el proceso que hizo la llamada se revive de manera automática y recibe la nueva tupla. El hecho de que los procesos se bloqueen o desbloqueen de manera automática significa que, si un proceso está por enviar una tupla y hay otro proceso a punto de recibirla, no importa cuál se ejecuta primero. La única diferencia es que si se realiza la primitiva *in* antes que la primitiva *out*, habrá un ligero retraso hasta que la tupla esté disponible para removerla.

Podemos dar varios usos al hecho de que los procesos se bloqueen cuando la tupla que necesitan no está presente. Por ejemplo, se puede utilizar para implementar semáforos. Para crear o realizar una operación `up` en el semáforo *S*, un proceso puede ejecutar lo siguiente:

```
out("semaphore S");
```

Para realizar una operación `down`, ejecuta:

```
in("semaphore S");
```

El estado del semáforo *S* se determina con base en el número de tuplas ("semaphore S") en el espacio de tuplas. Si no existe ninguna, cualquier intento por obtener una tupla se bloqueará hasta que algún otro proceso suministre una.

Además de *out* e *in*, Linda también tiene una primitiva *read*, que es igual a *in* sólo que no quita la tupla del espacio de tuplas. También hay una primitiva *eval*, la cual hace que sus parámetros se evalúen en paralelo y que la tupla resultante se coloque en el espacio de tuplas. Podemos utilizar este mecanismo para realizar un cálculo arbitrario. Esta es la forma en que se crean los procesos paralelos en Linda.

Publicar/suscribir

Nuestro siguiente ejemplo de un modelo basado en coordinación está inspirado en Linda y se conoce como **publicar/suscribir** (Oki y colaboradores, 1993). Consiste en varios procesos conectados por una red de transmisión. Cada proceso puede ser un productor de información, un consumidor de información o ambos.

Cuando un productor de información tiene una nueva pieza de información (por ejemplo, un nuevo precio de las acciones), transmite esa información en la red como una tupla. A esta acción se le conoce como **publicar**. Cada tupla contiene una línea de tema jerárquica, que a su vez contiene varios campos separados por puntos. Los procesos interesados en cierta información se pueden **suscribir** a ciertos temas, para lo cual incluyen el uso de comodines en la línea de tema. Para realizar una suscripción, hay que indicar los temas que se deben buscar a un proceso de demonio de tupla en la misma máquina, que monitorea las tuplas publicadas.

En la figura 8-41 se ilustra la manera de implementar el modelo de publicar/suscribir. Cuando un proceso tiene una tupla que desea publicar, la transmite a la LAN local. El demonio de tupla en cada máquina copia en su RAM todas las tuplas que se han transmitido. Después inspecciona la línea de tema para ver qué procesos están interesados en esa tupla, y reenvía una copia a cada proceso interesado. Las tuplas también se pueden transmitir a través de una red de área amplia o de Internet, para lo cual una máquina en cada LAN actúa como enrutador de información, recolectando todas las tuplas publicadas y reenviándolas posteriormente a otras LANs para que las vuelvan a transmitir. Este reenvío también se puede realizar de manera inteligente, en donde sólo se reenvía una tupla a una LAN remota si ésta tiene por lo menos un suscriptor que quiera esa tupla. Para hacer esto, los enrutadores de información deben intercambiar datos sobre los suscriptores.

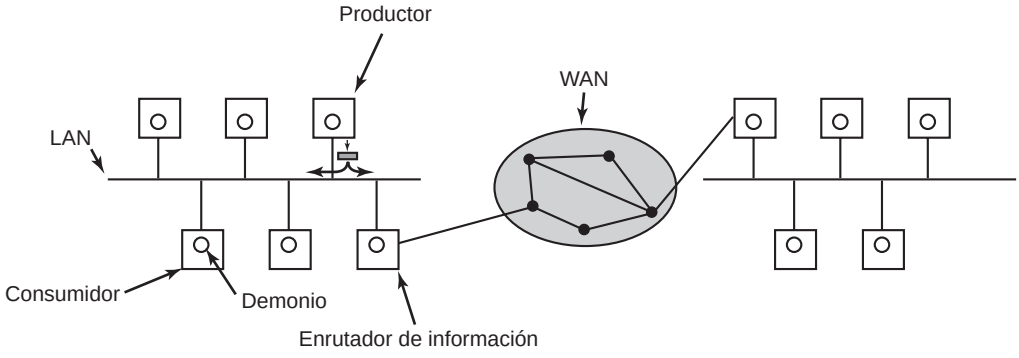


Figura 8-41. La arquitectura del modelo publicar/suscribir.

Se pueden implementar varios tipos de semántica, incluyendo una entrega confiable y una entrega garantizada, aun cuando exista la posibilidad de fallas. En el último caso es necesario almacenar tuplas anteriores, en caso de que se necesiten posteriormente. Una manera de almacenarlas es conectar un sistema de base de datos al sistema y hacer que se suscriba a todas las tuplas. Para ello,

hay que envolver el sistema de base de datos en un adaptador, para permitir que una base de datos existente funcione con el modelo de publicar/suscribir. A medida que lleguen las tuplas, el adaptador las capturará y las colocará en la base de datos.

El modelo de publicar/suscribir desacopla por completo a los productores de los consumidores, de igual forma que Linda. Sin embargo, algunas veces es útil saber quién más está ahí. Para adquirir esta información hay que publicar una tupla que realice esta pregunta básica: "¿Quién más está interesado en *x*?". Las respuestas llegan en forma de tuplas que dicen: "Yo estoy interesado en *x*".

Jini

Durante 50 años la computación ha estado centrada en la CPU, en donde una computadora es un dispositivo independiente que consiste en una CPU, memoria primaria y casi siempre algún dispositivo de almacenamiento masivo, como un disco. El modelo **Jini** (una variación de la palabra "genio" en inglés) de Sun Microsystems es un intento por cambiar ese modelo a uno que se podría describir como centrado en la red (Waldo, 1999).

El mundo de Jini consiste en un gran número de dispositivos Jini auto-contenidos, cada uno de los cuales ofrece uno o más servicios a los demás. Un dispositivo Jini se puede conectar en una red y empezar a ofrecer y utilizar servicios al instante, sin necesidad de un procedimiento de instalación complejo. Hay que tener en cuenta que los dispositivos se conectan a una *red*, y no a una *computadora* como se hace comúnmente. Un dispositivo Jini podría ser una computadora tradicional, pero también podría ser una impresora, una computadora de bolsillo, un teléfono celular, una televisión, un estéreo u otro dispositivo con una CPU, algo de memoria y una conexión de red (posiblemente inalámbrica). Un sistema Jini es una federación libre de dispositivos Jini que pueden operar según sea su voluntad, sin una administración central.

Cuando un dispositivo Jini desea unirse a la federación Jini, transmite un paquete en la LAN local o en la celda inalámbrica local, preguntando si hay un **servicio de búsqueda** presente. El protocolo que se utiliza para encontrar un servicio de búsqueda es el **protocolo de descubrimiento**, y es uno de los pocos protocolos fijos en Jini (como alternativa, el nuevo dispositivo Jini puede esperar hasta que llegue uno de los anuncios periódicos del servicio de búsqueda, pero en este libro no analizaremos ese mecanismo).

Cuando el servicio de búsqueda detecta que un nuevo dispositivo se quiere registrar, responde con una pieza de código que puede realizar el registro. Como Jini es un sistema desarrollado exclusivamente en Java, el código que se envía está en JVM (el lenguaje de la máquina virtual de Java), que todos los dispositivos Jini deben ser capaces de ejecutar, por lo general mediante un intérprete. Ahora el nuevo dispositivo ejecuta el código, que se contacta con el servicio de búsqueda y lo registra durante cierto periodo fijo. El dispositivo se puede volver a registrar si lo desea, justo antes de que expire tiempo. Este mecanismo implica que un dispositivo Jini puede salir del sistema con sólo apagarse, y se olvidará pronto su anterior existencia, sin necesidad de una administración central. Al concepto de registrarse por un intervalo fijo se le conoce como adquirir una **concesión**.

Observe que como el código para registrar el dispositivo se descarga en el mismo, se puede cambiar a medida que evolucione el sistema, sin afectar al hardware o software del dispositivo. De

hecho, el dispositivo ni siquiera conoce el protocolo de registro. La parte que el dispositivo conoce sobre el proceso de registro consiste en proveer ciertos atributos y código de proxy que otros dispositivos utilizarán después para acceder a este dispositivo.

Un dispositivo o usuario que busque un servicio específico podrá preguntar al servicio de búsqueda si conoce uno. La petición puede involucrar algunos de los atributos que utilizan los dispositivos al registrarse, Si la petición tiene éxito, el proxy que suministró el dispositivo al momento de registrarse se devuelve al dispositivo que hizo la petición, y se ejecuta para contactarse con el dispositivo. Así, un dispositivo o usuario puede hablar con otro dispositivo sin necesidad de saber en dónde se encuentra ni qué protocolo utiliza.

Los clientes y servicios de Jini (dispositivos de hardware o software) se comunican y sincronizan mediante el uso de **JavaSpaces**, que se modelan en el espacio de tuplas de Linda, pero con algunas diferencias importantes. Cada JavaSpace consiste en cierto número de entradas fuertemente tipificadas. Las entradas son como tuplas de Linda, excepto que son fuertemente tipificadas, mientras que las tuplas de Linda no tienen tipo. Cada entrada consiste en cierto número de campos, cada uno de los cuales tiene un tipo básico de Java. Por ejemplo, una entrada de tipo empleado podría consistir en una cadena (para el nombre de la persona), un entero (para su departamento), un segundo entero (para la extensión telefónica) y un valor Booleano (para saber si trabaja tiempo completo).

En JavaSpace sólo se definen cuatro métodos (aunque dos de ellos tienen una forma variante):

1. **Write**: coloca una nueva entrada en el JavaSpace.

2. **Read**: copia una entrada que coincide con una plantilla y la saca del JavaSpace.

3. **Take**: copia y elimina una entrada que coincide con una plantilla.

4. **Notify**: notifica al proceso que hizo la llamada cuando se escribe una entrada que coincida.

El método *write* proporciona la entrada y especifica su tiempo de concesión; es decir, cuándo se debe descartar. Por el contrario, las tuplas de Linda permanecen hasta que se quitan. Un JavaSpace puede contener la misma entrada varias veces, por lo que no es un conjunto matemático (al igual que en Linda).

Los métodos *read* y *take* proporcionan una plantilla para la entrada que se está buscando. Cada campo en la plantilla puede contener un valor específico que debe coincidir, o puede contener un comodín “no importa” que coincida con todos los valores del tipo apropiado. Si se encuentra una coincidencia se devuelve, y en el caso de *take*, también se elimina del JavaSpace. Cada uno de estos métodos de JavaSpace tienen dos variantes, que difieren en el caso en que ninguna entrada coincida. Una variante devuelve una indicación de falla de inmediato; la otra espera hasta que haya expirado cierto tiempo límite (que se proporciona como parámetro).

El método *notify* registra el interés en cierta plantilla. Si después se introduce una entrada que coincida, se invoca al método *notify* del proceso que hizo la llamada.

A diferencia del espacio de tuplas de Linda, JavaSpace soporta las transacciones atómicas. Al utilizar estas transacciones se pueden agrupar varios métodos. Puede ser que todos se ejecuten, o que ninguno se ejecute. Durante la transacción, las modificaciones en el JavaSpace no son visibles fuera de la transacción. Los demás procesos que hacen la llamada podrán ver los cambios sólo hasta que termine la transacción.

JavaSpace se puede utilizar para sincronizar procesos que se comunican entre sí. Por ejemplo, en una situación de productor-consumidor, el productor coloca elementos en un JavaSpace a medida que los produce. El consumidor los quita con *take* y se bloquea si no hay elementos disponibles. JavaSpace garantiza que cada uno de los métodos se ejecute en forma atómica, por lo que no hay peligro de que un proceso trate de leer una entrada que sólo se encuentre a la mitad.

8.4.7 Grids (Mallas)

Ningún análisis de sistemas distribuidos estaría completo sin por lo menos mencionar un desarrollo reciente, que se puede hacer importante en el futuro: los grids. Un **grid** es una gran colección de máquinas dispersas geográficamente y por lo general heterogénea, que se conecta mediante una red privada o Internet, y que ofrece un conjunto de servicios para sus usuarios. Algunas veces se le compara con una supercomputadora virtual, pero es más que eso. Es una colección de computadoras independientes, por lo general en varios dominios administrativos, las cuales ejecutan un nivel común de middleware para permitir que los programas y usuarios accedan a todos los recursos de una manera conveniente y consistente.

La motivación original para construir un grid fue la compartición de los ciclos de la CPU. La idea es que cuando una organización no necesita todo su poder de cómputo (por ejemplo, en la noche), otra organización (que tal vez se encuentre a varias zonas horarias de distancia) podría recolectar esos ciclos y después regresarle el favor 12 horas más tarde. Ahora, los investigadores de los grids también están interesados en compartir otros recursos, específicamente el hardware especializado y las bases de datos.

Por lo general, los grids funcionan así: cada máquina participante ejecuta un conjunto de programas que administran la máquina y la integran en el grid. Este software comúnmente se encarga de la autenticación y el inicio de sesión de los usuarios remotos, de anunciar y descubrir recursos, programar y ubicar trabajos, etcétera. Cuando un usuario tiene que hacer un trabajo, el software del grid determina en dónde hay capacidad inactiva que tenga los recursos de hardware, software y de datos para realizar el trabajo, y después envía el trabajo ahí, prepara su ejecución y después devuelve los resultados al usuario.

Un middleware popular en el mundo de los grids es **Globus Toolkit**, que está disponible para varias plataformas y soporta muchos estándares (emergentes) de grids (Foster, 2005). Globus provee un marco de trabajo para que los usuarios compartan computadoras, archivos y otros recursos de una manera flexible y segura, sin sacrificar la autonomía local. Se utiliza como base para construir muchas aplicaciones distribuidas.

8.5 INVESTIGACIÓN SOBRE LOS SISTEMAS DE MÚLTIPLES PROCESADORES

En este capítulo hemos analizado cuatro tipos de sistemas de múltiples procesadores: multiprocesadores, multicomputadoras, máquinas virtuales y sistemas distribuidos. También analizaremos con brevedad la investigación realizada en estas áreas.

La mayor parte de la investigación en los multiprocesadores se relaciona con el hardware, en especial la forma de construir la memoria compartida y mantenerla coherente (por ejemplo, Higham y colaboradores, 2007). Sin embargo, también se han realizado otros trabajos sobre los multiprocesadores, en especial los multiprocesadores de chip, incluyendo los modelos de programación y las cuestiones del sistema operativo para los multiprocesadores de chip (Fedorova y colaboradores, 2005; Tan y colaboradores, 2007), los mecanismos de comunicación (Brisolara y colaboradores, 2007), la administración de la energía mediante software (Park y colaboradores, 2007), la seguridad (Yang y Peng, 2006) y, desde luego, los futuros retos (Wolf, 2004). Además, la planificación es un tema siempre popular (Chen y colaboradores, 2007; Lin y Rajaraman, 2007; Rajagolapan y colaboradores, 2007; Tam y colaboradores, 2007; Yahav y colaboradores, 2007).

Es mucho más fácil construir multicomputadoras que multiprocesadores. Todo lo que se necesita es una colección de PCs o estaciones de trabajo y una red de alta velocidad. Por esta razón, son un popular tema de investigación en las universidades. Gran parte del trabajo se relaciona con la memoria compartida distribuida en una forma u otra. Algunas veces se basa en páginas, pero otras se basa por completo en software (Byung-Hyun y colaboradores, 2004; Chapman y Heiser, 2005; Huang y colaboradores, 2001; Kontothanassis y colaboradores, 2005; Nikolopoulos y colaboradores, 2001; Zhang y colaboradores, 2006). También se están analizando los modelos de programación (Dean y Ghemawat, 2004). El uso de energía en los grandes centros de datos es una de las cuestiones (Bash y Forman, 2007; Ganesh y colaboradores, 2007; Villa, 2006), al igual que escalar a decenas de miles de CPUs (Friedrich y Rolia, 2007).

Las máquinas virtuales son un tema muy activo, con muchos artículos sobre distintos aspectos del tema, incluyendo la administración de energía (Moore y colaboradores, 2005; Stoess y colaboradores, 2007), la administración de memoria (Lu y Shen, 2007) y la administración de la confianza (Garfinkel y colaboradores, 2003; Lie y colaboradores, 2003). También hay que tener en cuenta la seguridad (Jaeger y colaboradores, 2007). La optimización del rendimiento es también un tema de gran interés, en especial el rendimiento de la CPU (King y colaboradores, 2003), el rendimiento de la red (Menon y colaboradores, 2006) y el rendimiento de la E/S (Cherkasova y Gardner, 2005; Liu y colaboradores, 2006). Las máquinas virtuales hacen posible la migración, por lo que hay interés en ese tema (Bradford y colaboradores, 2007; Huang y colaboradores, 2007). Las máquinas virtuales también se han utilizado para depurar sistemas operativos (King y colaboradores, 2005).

Con el crecimiento de la computación distribuida, se ha realizado mucha investigación sobre los sistemas de archivos y almacenamiento distribuidos, incluyendo cuestiones como la capacidad de manutención a largo plazo frente a fallas de hardware y software, los errores humanos y las rupturas en el entorno (Baker y colaboradores, 2006; Kotla y colaboradores, 2007; Maniatis y colaboradores, 2005; Shah y colaboradores, 2007; Storer y colaboradores, 2007), el uso de servidores desconfiables (Adya y colaboradores, 2002; Popescu y colaboradores, 2003), la autenticación (Kaminsky y colaboradores, 2003), y la escalabilidad en los sistemas de archivos distribuidos (Ghemawat y co-

laboradores, 2003; Saito, 2002; Weil y colaboradores, 2006). Se ha investigado sobre la extensión de los sistemas de archivos distribuidos (Peek y colaboradores, 2007). También se han examinado ampliamente los sistemas de archivos distribuidos punto a punto (Dabek y colaboradores, 2001; Gummadi y colaboradores, 2003; Muthitacharoen y colaboradores, 2002; Rowstron y Druschel, 2001). Ahora que algunos de los nodos son móviles, la eficiencia de la energía también es una cuestión importante (Nightingale y Flinn, 2004).

8.6 RESUMEN

Los sistemas computacionales se pueden hacer más rápidos y confiables mediante el uso de varias CPUs. Hay cuatro organizaciones para los sistemas con múltiples CPUs: multiprocesadores, multicomputadoras, máquinas virtuales y sistemas distribuidos. Cada una de estas organizaciones tiene sus propiedades y cuestiones.

Un multiprocesador consiste en dos o más CPUs que comparten una RAM común. Las CPUs se pueden interconectar mediante un bus, un conmutador de barras cruzadas o una red de conmutación multietapa. Hay varias configuraciones posibles de sistemas operativos: otorgar a cada CPU su propio sistema operativo, que haya un sistema operativo maestro y el resto sean esclavos, o tener un multiprocesador simétrico, en el cual hay una copia del sistema operativo que cualquier CPU puede ejecutar. En el último caso se requieren bloqueos para proveer sincronización. Cuando no hay un bloqueo disponible, una CPU puede girar o realizar un cambio de contexto. Hay varios algoritmos de programación posibles: tiempo compartido, espacio compartido y planificación por pandilla.

Las multicomputadoras también tienen dos o más CPUs, pero cada una de estas CPUs tienen su propia memoria privada. No comparten ningún tipo de RAM común, por lo que toda la comunicación utiliza el paso de mensajes. En algunos casos, la tarjeta de interfaz de red tiene su propia CPU, en cuyo caso la comunicación entre la CPU principal y la CPU de la tarjeta de interfaz tiene que organizarse con cuidado para evitar condiciones de carrera. A menudo, la comunicación a nivel de usuario en las multicomputadoras utiliza llamadas a procedimientos remotos, pero también se puede utilizar la memoria compartida distribuida. Una de las cuestiones aquí es el balance de carga de los procesos, y los diversos algoritmos que se utilizan son: algoritmos iniciados por el emisor, algoritmos iniciados por el receptor y algoritmos de subasta.

Las máquinas virtuales permiten que una o más CPUs provean la ilusión de que hay más CPUs de las que realmente se tienen. De esta forma, es posible ejecutar varios sistemas operativos o varias versiones (incompatibles) del mismo sistema operativo al mismo tiempo, en la misma pieza de hardware. Al combinarse con diseños de multinúcleo, cada computadora se convierte entonces en una multicomputadora potencial a gran escala.

Los sistemas distribuidos son sistemas débilmente acoplados, en donde cada uno de sus nodos es una computadora completa con un conjunto completo de periféricos y su propio sistema operativo. A menudo, estos sistemas se esparcen sobre una gran área geográfica. Con frecuencia se coloca middleware encima del sistema operativo para ofrecer un nivel uniforme para que las aplicaciones interactúen con él. Los diversos tipos de middleware son: middleware basado en documentos, basado en archivos, basado en objetos y basado en coordinación. Algunos ejemplos son: World Wide Web, CORBA, Linda y Jini.

PROBLEMAS

1. ¿Se pueden considerar sistemas distribuidos el sistema de grupos de noticias USENET o el proyecto SETI@home (SETI@home utiliza varios millones de computadoras personales inactivas que analizan los datos radiotelescópicos para buscar inteligencia extraterrestre)? Si es verdad, ¿cómo se relacionan con las categorías descritas en la figura 8-1?

2. ¿Qué ocurre si dos CPUs en un multiprocesador intentan utilizar la misma palabra de memoria en el mismo instante?

3. Si una CPU emite una petición de memoria en cada instrucción y la computadora se ejecuta a 200 MIPS, ¿aproximadamente cuántas CPUs se requieren para saturar un bus de 400 MHz? Suponga que una referencia a memoria requiere un ciclo de bus. Ahora repita este problema para un sistema en el que se utiliza caché, y que las memorias caché tienen una tasa de aciertos de 90%. Por último, ¿qué tasa de aciertos de caché se necesitaría para permitir que 32 CPUs compartan el bus sin sobrecargarlo?

4. Suponga que se rompe el cable entre el conmutador 2A y el conmutador 3B en la red omega de la figura 8-5. ¿Quién está desconectado de quién?

5. ¿Cómo se realiza el manejo de señales en el modelo de la figura 8-7?

6. Vuelva a escribir el código entrar_region de la figura 2-22 utilizando sólo lectura para reducir el thrashing inducido por la instrucción `TSL`.

7. Las CPUs multinúcleo están empezando a aparecer en las máquinas de escritorio convencionales y en las computadoras laptop. No están muy lejanos los equipos de escritorio con decenas o cientos de núcleos. Una manera posible de aprovechar este poder es paralelizar las aplicaciones de escritorio estándar, como el procesador de palabras o el navegador Web. Otra manera posible de aprovechar el poder es paralelizar los servicios ofrecidos por el sistema operativo (como el procesamiento TCP) y los servicios de biblioteca de uso común (como las funciones de biblioteca de http seguro). ¿Qué método se ve más prometedor? ¿Por qué?

8. ¿Son realmente necesarias las regiones críticas en las secciones de código en un sistema operativo SMP para evitar condiciones de competencia, o también se pueden utilizar los mutexes en las estructuras de datos?

9. Cuando se utiliza la instrucción `TSL` para la sincronización de multiprocesadores, el bloque de caché que contiene el mutex se transportará de un lado a otro entre la CPU que contiene el bloqueo y la CPU que lo solicita, si ambas están en contacto con el bloque. Para reducir el tráfico de bus, la CPU que solicita el bloqueo debe ejecutar una instrucción `TSL` cada 50 ciclos de bus, pero la CPU que contiene el bloqueo siempre está en contacto con el bloque de caché entre las instrucciones `TSL`. Si un bloque de caché está compuesto de 16 palabras de 32 bits, cada una de las cuales requiere un ciclo de bus para transferirse, y el bus opera a 400 MHz, ¿qué fracción se ocupa del ancho de banda del bus al mover el bloque de caché de un lado a otro?

10. En el texto se sugirió utilizar un algoritmo de backoff exponencial binario entre los usos de `TSL` para sondear un bloqueo. También se sugirió tener un máximo retraso entre los sondeos. ¿Funcionaría correctamente el algoritmo si no hubiera un retraso máximo?

11. Suponga que no está disponible la instrucción `TSL` para sincronizar un multiprocesador. En su lugar se provee una instrucción `SWP` para intercambiar de manera atómica el contenido de un registro con

una palabra en memoria. ¿Se podría utilizar esta instrucción para mantener la sincronización de los multiprocesadores? De ser así, ¿cómo se podría utilizar? En caso contrario, ¿por qué no funcionaría?

12. En este problema usted debe calcular cuánta carga en el bus impone un bloqueo de espera activa. Imagine que cada instrucción ejecutada por una CPU requiere 5 nseg. Al completarse una instrucción se llevan a cabo los ciclos necesarios (por ejemplo) para una instrucción `TSL`. Cada ciclo de bus requiere 10 nseg adicionales antes y después del tiempo de ejecución de la instrucción. Si un proceso intenta entrar a una región crítica mediante el uso de un ciclo de `TSL`, qué fracción del ancho de banda consume? Suponga que está funcionando la caché normal, por lo que al obtener una instrucción dentro del ciclo no se consumen ciclos de bus.

13. Se dijo que la figura 8-12 describía un entorno de tiempo compartido. ¿Por qué sólo se muestra un proceso (*A*) en la parte (b)?

14. La programación por afinidad reduce los fallos en la caché. ¿Reduce también los fallos de TLB? ¿Qué hay sobre los fallos de página?

15. Para cada una de las topologías de la figura 8-16, ¿cuál es el diámetro de la red de interconexión? Cuente todos los saltos (host-enrutador y enrutador-enrutador) de igual forma para este problema.

16. Considere la topología de doble anillo de la figura 8-16(d), pero expandida a un tamaño de $k \times k$. ¿Cuál es el diámetro de la red? *Sugerencia:* considere los valores impares de k y los valores pares de k de manera distinta.

17. El ancho de banda de bisección de una red de interconexión se utiliza a menudo como una medida de su capacidad. Para calcularlo, se elimina un número mínimo de vínculos que dividen la red en dos unidades de igual tamaño. Después se suma la capacidad de los vínculos eliminados. Si hay muchas formas de realizar la división, la que obtenga el mínimo ancho de banda es el ancho de banda de bisección. Para una red de interconexión que consiste en un cubo de $8 \times 8 \times 8$, ¿cuál es el ancho de banda de bisección si cada vínculo es de 1 Gbps?

18. Considere una multicomputadora en la que la interfaz de red está en modo de usuario, por lo que sólo se necesitan tres copias de la RAM de origen a la RAM de destino. Suponga que para desplazar una palabra de 32 bits hacia/desde la tarjeta de interfaz de red se requieren 20 nseg, y que la red en sí opera a 1 Gbps. ¿Cuál sería el retraso por enviar un paquete de 64 bytes del origen al destino si pudiéramos ignorar el tiempo de copia? ¿Cuál sería el retraso con el tiempo de copia? Ahora considere el caso en el que se necesitan dos copias adicionales, una que va al kernel del lado emisor y otra que proviene del kernel en el lado receptor. ¿Cuál es el retraso en este caso?

19. Repita el problema anterior para el caso de tres copias y el caso de cinco copias, pero esta vez calcule el ancho de banda en vez del retraso.

20. ¿Qué diferencias deben tener las implementaciones de `send` y `receive` entre un sistema de multiprocesador con memoria compartida y una multicomputadora, y cómo afecta esto al rendimiento?

21. Al transferir datos de la RAM a una interfaz de red se puede marcar una página, pero suponga que las llamadas al sistema para marcar y desmarcar páginas requieren 1 μseg. El proceso de copia es de 5 bytes/nseg si se utiliza DMA, pero de 20 nseg por byte si se utiliza E/S programada. ¿Qué tan grande tiene que ser un paquete para que valga la pena marcar la página y usar DMA?

22. Cuando se recoge un procedimiento de una máquina y se coloca en otra para que sea llamado mediante RPC pueden ocurrir ciertos problemas. En el texto recalcamos cuatro de ellos: apuntadores, tamaños de arreglos desconocidos, tipos de parámetros desconocidos y variables globales. Una cuestión que no analizamos es lo que ocurre si el procedimiento (remoto) ejecuta una llamada al sistema. ¿Qué problemas podría ocasionar esto y qué podríamos hacer para resolverlos?

23. Cuando ocurre un fallo de página en un sistema de DSM, hay que localizar la página que se necesita. Liste dos formas posibles de buscar la página.

24. Considere la asignación de procesadores de la figura 8-24. Suponga que el proceso *H* se mueve del nodo 2 al nodo 3. ¿Cuál es el peso total del tráfico externo ahora?

25. Algunas multicomputadoras permiten migrar los procesos en ejecución de un nodo a otro. ¿Es suficiente con detener un proceso, congelar su imagen en memoria y sólo enviarlo a un nodo distinto? Nombre dos problemas no triviales que se tienen que resolver para que esto funcione.

26. Considere un hipervisor de tipo 1 que puede soportar hasta *n* máquinas virtuales al mismo tiempo. Las PCs pueden tener un máximo de cuatro particiones primarias de disco. ¿Puede *n* ser mayor que 4? De ser así, ¿en dónde se pueden almacenar los datos?

27. Una manera de manejar los sistemas operativos invitados que modifican sus tablas de páginas mediante el uso de instrucciones ordinarias (no privilegiadas) es marcar las tablas de páginas como de sólo lectura, y tomar una interrupción cuando se modifiquen. ¿De qué otra forma se podrían mantener las tablas de páginas ocultas? Explique la eficiencia de su método en comparación con las tablas de páginas de sólo lectura.

28. VMware realiza la traducción binaria un bloque básico a la vez, después ejecuta el bloque y empieza a traducir el siguiente. ¿Podría traducir el programa completo por adelantado y después ejecutarlo? De ser así, ¿cuáles son las ventajas y desventajas de cada técnica?

29. ¿Tiene sentido paravirtualizar un sistema operativo si está disponible el código fuente? ¿Qué pasa si no lo está?

30. Las PCs tienen pequeñas diferencias en el nivel más inferior, como la forma en que se administran los temporizadores, las interrupciones y algunos de los detalles del DMA. ¿Significan estas diferencias que los dispositivos virtuales en realidad no van a funcionar bien en la práctica? Explique su respuesta.

31. ¿Por qué hay un límite para la longitud de los cables en una red Ethernet?

32. Para ejecutar varias máquinas virtuales en una PC se requieren grandes cantidades de memoria. ¿Por qué? ¿Puede usted pensar en alguna manera de reducir el uso de la memoria? Explique.

33. En la figura 8-30, los niveles tercero y cuarto se etiquetan como Middleware y Aplicación en las cuatro máquinas. ¿En qué sentido son iguales entre una plataforma y otra, y en qué sentido son distintos?

34. En la figura 8-33 se listan seis tipos distintos de servicio. Para cada una de las siguientes aplicaciones, ¿qué tipo de servicio es el más apropiado?

(a) Video bajo demanda a través de Internet.
(b) Descargar una página Web.

35. Los nombres de DNS tienen una estructura jerárquica, como *cs.uni.edu* o *ventas.generalwidget.com*. Una manera de mantener la base de datos DNS sería como una base de datos centralizada, pero es-

to no se hace debido a que se obtendrían demasiadas peticiones por segundo. Proponga una manera en que se pudiera mantener la base de datos DNS en la práctica.

36. En el análisis sobre la forma en que un navegador procesa los URLs, se estableció que las conexiones se realizan en el puerto 80. ¿Por qué?

37. Podría ser más fácil migrar máquinas virtuales que procesos, pero la migración de todas formas puede ser difícil. ¿Qué problemas pueden surgir al migrar una máquina virtual?

38. ¿Pueden los URLs que se utilizan en Web exhibir transparencia en la ubicación? Explique su respuesta.

39. Cuando un navegador obtiene una página Web, primero realiza una conexión TCP para obtener el texto en la página (en el lenguaje HTML). Después cierra la conexión y examina la página. Si hay figuras o iconos, entonces realiza una conexión TCP por separado para obtener cada figura o icono. Sugiera dos diseños alternativos para mejorar el rendimiento en este proceso.

40. Cuando se utiliza la semántica de sesión, siempre es cierto que las modificaciones a un archivo están visibles de inmediato para el proceso que realiza el cambio, y nunca están visibles para los procesos en las otras máquinas. Sin embargo, no se sabe si deben o no estar visibles de inmediato para los otros procesos en la misma máquina. Proporcione un argumento para cada uno de los dos casos.

41. Cuando varios procesos necesitan acceso a los datos, ¿de qué manera es mejor el acceso basado en objetos que la memoria compartida?

42. Cuando se realiza la operación *in* de Linda para localizar una tupla, es muy ineficiente buscar en todo el espacio de tuplas en forma lineal. Diseñe una manera de organizar el espacio de tuplas para que se agilicen las búsquedas en todas las operaciones *in*.

43. El proceso de copiar búferes requiere tiempo. Escriba un programa en C para averiguar cuánto tiempo se requiere en un sistema al que usted tenga acceso. Use las funciones *clock* o *times* para determinar cuánto tiempo se requiere para copiar un arreglo extenso. Pruebe con tamaños distintos del arreglo para separar el tiempo de copia del tiempo de sobrecarga.

44. Escriba funciones en C que se puedan utilizar como resguardos de cliente y servidor para realizar una llamada RPC a la función *printf* estándar, y un programa principal para evaluar las funciones. El cliente y el servidor se deben comunicar mediante una estructura de datos que se pueda transmitir a través de una red. Puede imponer límites razonables en la longitud de la cadena de formato y el número, tipos y tamaños de las variables que aceptará su resguardo del cliente.

45. Escriba dos programas para simular el balanceo de carga en una multicomputadora. El primer programa debe establecer *m* procesos distribuidos entre *n* máquinas, de acuerdo con un archivo de inicialización. Cada proceso deberá tener un tiempo de ejecución elegido al azar, a partir de una distribución gaussiana cuya media y desviación estándar sean parámetros de la simulación. Al final de cada ejecución, el proceso crea cierto número de procesos nuevos, los cuales se seleccionan de una distribución de Poisson. Cuando un proceso termina, la CPU debe decidir entre dar procesos a otras CPUs o tratar de buscar nuevos procesos. El primer programa debe utilizar el algoritmo iniciado por el servidor para dar trabajo a otras CPUs si tiene más de *k* procesos en total en su máquina. El segundo programa debe utilizar el algoritmo iniciado por el receptor para obtener trabajo cuando sea necesario. Haga todas las demás suposiciones razonables que necesite, pero indíquelas con claridad.

46. Escriba un programa que implemente los algoritmos de balanceo de carga iniciado por el emisor e iniciado por el receptor que se analizan en la sección 8.2. Los algoritmos deben recibir como entrada una lista de trabajos recién creados, especificados como (proceso_creador, tiempo_inicial, tiempo_requerido_CPU), en donde el proceso_creador es el número de la CPU que creó el trabajo, el tiempo_inicial es el tiempo en el que se creó el trabajo y el tiempo_requerido_CPU es la cantidad de tiempo de la CPU que necesita el trabajo para completarse (especificado en segundos). Suponga que un nodo está sobrecargado cuando tiene un trabajo y se crea un segundo trabajo. Suponga que un nodo tiene poca carga cuando no tiene trabajos. Imprima el número de mensajes de sondeo enviados por ambos algoritmos bajo cargas de trabajo pesada y ligera. Imprima también los números mínimo y máximo de sondas enviadas por cualquier host, y recibidas por cualquier host. Para crear las cargas de trabajo, escriba dos generadores de carga de trabajo. El primero debe simular una carga de trabajo pesada, que genere (en promedio) *N* trabajos cada *AJL* segundos, en donde *AJL* es la longitud de trabajo promedio y N es el número de procesadores. Las longitudes de los trabajos pueden ser una mezcla de trabajos cortos y largos, pero la longitud de trabajo promedio debe ser *AJL*. Los trabajos deben crearse (colocarse) al azar entre todos los procesadores. El segundo generador debe simular una carga ligera, que genere al azar (*N/3*) trabajos cada *AJL* segundos. Juegue con otras configuraciones de parámetros para los generadores de carga de trabajo y vea cómo afecta esto al número de mensajes de sonda.

47. Una de las maneras más simples de implementar un sistema de publicar/suscribir es mediante un intermediario centralizado que reciba los artículos publicados y los distribuya a los suscriptores apropiados. Escriba una aplicación multihilo que emule un sistema de publicar/suscribir basado en intermediario. Los hilos publicador y suscriptor se pueden comunicar con el intermediario a través de la memoria (compartida). Cada mensaje debe empezar con una longitud de campo seguida de esa cantidad de caracteres. Los editores envían mensajes al intermediario, en los cuales la primera línea del mensaje contiene una línea de tema jerárquica separada por puntos, seguida de una o más líneas que componen el artículo publicado. Los suscriptores envían un mensaje al intermediario con una sola línea que contiene una línea de interés jerárquica, separada por puntos que expresan los artículos en los que están interesados. La línea de interés puede contener el símbolo comodín "*". El intermediario debe responder enviando todos los artículos (anteriores) que coincidan con el interés del suscriptor. Los artículos en el mensaje se separan mediante la línea "INICIO DE NUEVO ARTÍCULO". El suscriptor debe imprimir cada mensaje que reciba, junto con la identidad de su suscriptores (es decir, su línea de interés). El suscriptor debe seguir recibiendo los nuevos artículos que se publiquen y coincidan con sus intereses. Se pueden crear hilos publicador y suscriptor de manera dinámica desde la terminal, escribiendo la letra "P" o "S" (publicador o suscriptor) seguida de la línea de tema/interés jerárquica. Después los publicadores pedirán el artículo. Al escribir una sola línea que contenga "." se indicará el fin del artículo (este proyecto también se puede implementar mediante el uso de procesos que se comuniquen a través de TCP).